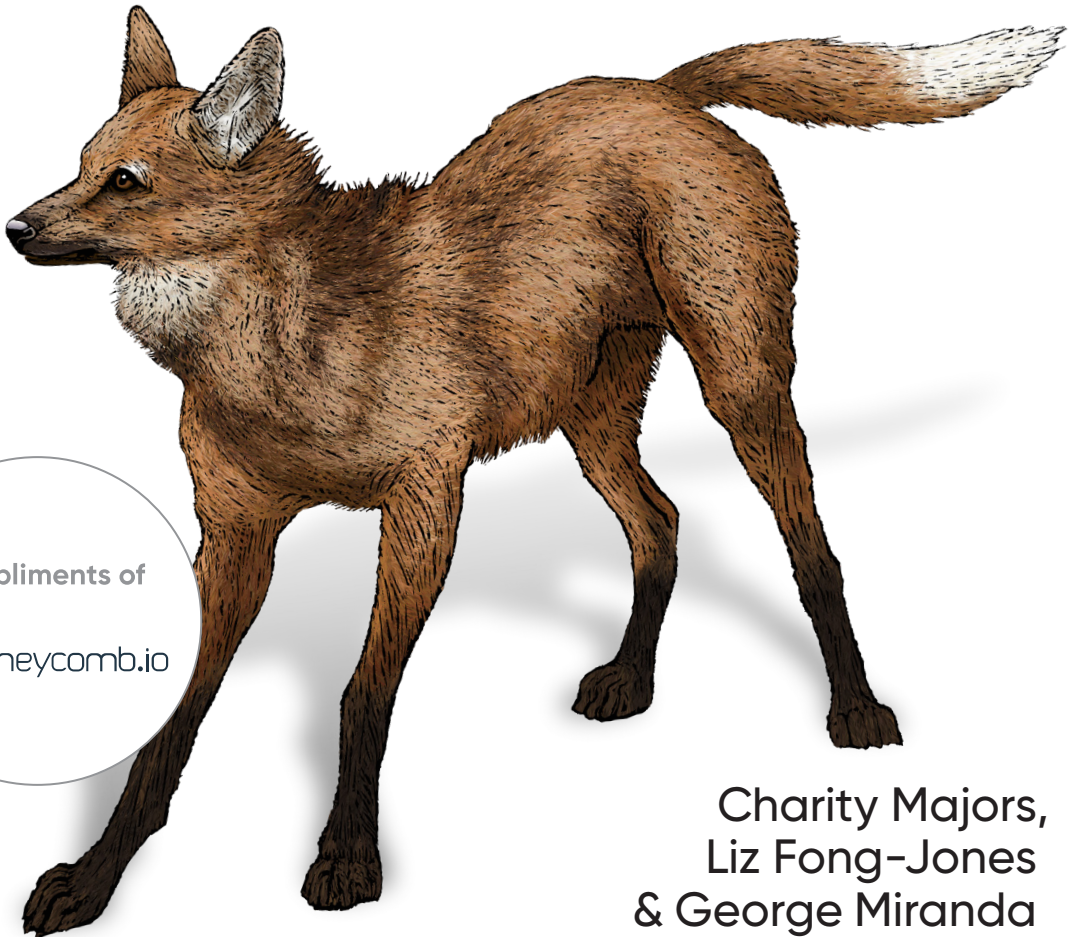


O'REILLY®

2nd Edition

Observability Engineering

Achieving Production Excellence



Compliments of



Charity Majors,
Liz Fong-Jones
& George Miranda
with Austin Parker

"This is as close to a guide to the software future we're rushing toward as you'll get in a book without 'AI' in the title. It's authentic, authoritative, and literally covers everything—past, present, and future. Recommended."

Niall Murphy, CEO, Site Reliability Engineering Limited

"As AI accelerates software creation, production becomes the only place where intent meets reality. This book shows how to build the feedback loops that make that reality legible."

Chad Fowler, general partner & CTO, BlueYard Capital

Observability Engineering

Production is not what happens after development ends; it's a stage of development, and the most important one. The only way to move swiftly with confidence is by building observability into every step, at every stage, and by validating your changes along the way. As AI continues to radically reshape how software gets built, that need has never been more urgent. In this second edition, which is twice as long as the original and mostly new or rewritten, authors Charity Majors, Liz Fong-Jones, George Miranda, and Austin Parker make the case for observability as a foundational practice of software development.

This is a book for builders and technical decision-makers. Ten new chapters for staff+ engineers, architects, and technical leaders cover the strategic and organizational side. Four more chapters tackle LLMs and AI agents, frontend observability, and open source tooling.

- Learn how to develop with observability, whether you're using AI or not
- Implement modern observability practices across your organization
- Make the business case and navigate build versus buy decisions
- Maximize the cost-effectiveness of your observability tooling
- Find answers fast when reliability is on the line

Charity Majors is the cofounder and CTO of honeycomb.io and writes about software, systems, and management at charity.wtf.

Liz Fong-Jones is a technical fellow at honeycomb.io, where she's an advocate for the SRE and observability communities.

George Miranda is VP of marketing at InsightFinder AI. He has a strong background in systems engineering, developer relations, and marketing.

Austin Parker is director of AI strategy at honeycomb.io and serves on the OpenTelemetry Governance Committee.

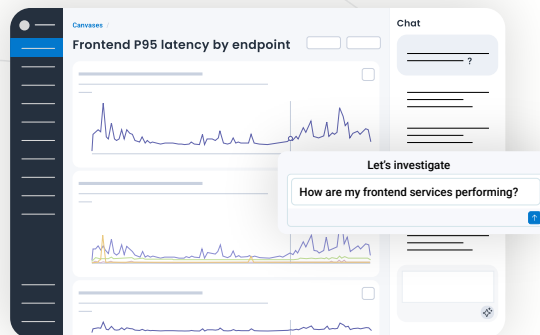
DEVOPS / SYSTEMS ENGINEERING

ISBN: 979-8-341-60841-2



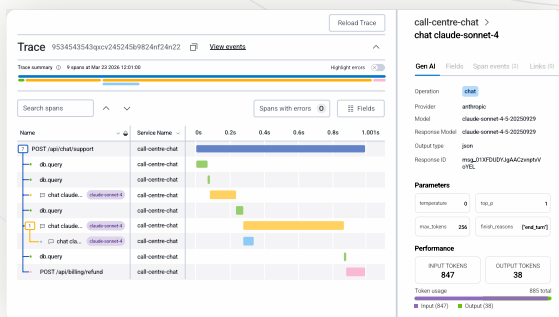
9

O'REILLY®



Honeycomb gives you all the **context**, **speed** and **scale** you need to understand and debug your code at the pace of AI.

START FOR FREE



Praise for *Observability Engineering*, Second Edition

Most enterprises still mistake observability for a monitoring dashboard. This book proves it is a production intelligence layer. By tying telemetry to business outcomes, it explains why the dashboards stay green while the mysteries become folklore, and helps those who sign the checks understand exactly what they're paying for.

—Rick Clark, *Global Head of Cloud Advisory, UST*

The best engineers understand what's actually happening in production, not just the code they wrote. This matters more than ever in an AI-driven world, and this book is a treasure trove of guidance on how.

—Darragh Curran, *CTO, Fin*

Observability gives our systems a voice, and as we hand the building over to agents, that signal from production becomes the truest context for making sense of our creations. This book teaches it completely and humanely, from first principles up.

—Annie Vella, *Distinguished Engineer, Westpac New Zealand*

This book! I feel seen. *Observability Engineering* is packed with battle-tested tools and methodologies for building and implementing an effective observability program. If it stopped there, it would be a great reference tome, but the last section—focusing on the socio-technical barriers to implementing a clear-eyed observability program in existing organizations—is pure gold. Any SRE or Principal Engineer suffering in a broken, siloed, or observability-starved organization will recognize the patterns Charity et al. describe.

When you are living it, it can be difficult to understand what is going on. This book names the problem and gives you practical advice on how to face it together!

—Todd Beckett, *Principal Software Engineer, Microsoft*

Reliability of complex sociotechnical systems is as much a cultural movement as it is a systemic one. This book captures an accurate picture of the path to building solid systems, with teams that own the accountability.

—Alex Ewerlöf, CTO, *Service Level Toolbox*

Observability Engineering is audacious: it tackles the entire observability space. No matter what part of the tech stack you focus on, or where in the org chart you sit, you'll find value here.

—Lorin Hochstein, *Staff Software Engineer—Reliability*

This book was already a great reference for what observability is and how to implement it. The second edition adds new chapters that dig deeper into the business case for observability, and the organizational changes that will allow you to be successful. Love it!

—Sarah Wells, *technical engineering leader and author of Enabling Microservice Success*

“The rise of LLMs has been polarizing: some (loudly!) proclaim the End of Software Engineering, while others dismiss them as slop generators. The truth lies somewhere in the middle. LLMs are an extraordinarily powerful tool, but one that must be wielded carefully. That care requires observability of our software systems. This timely second edition provides an expanded, updated foundation and also shows how LLMs can be leveraged to enhance our systems for observability. The future is weirder than ever, but with the guidance in this book, it needn't be opaque!”

—Bryan Cantrill, *cofounder and CTO, Oxide Computer Company*

SECOND EDITION

Observability Engineering

Achieving Production Excellence

Charity Majors, Liz Fong-Jones, and George Miranda
with Austin Parker

O'REILLY®

Observability Engineering

by Charity Majors, Liz Fong-Jones, and George Miranda, with Austin Parker

Copyright © 2026 Hound Technology, Inc. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Louise Corrigan

Development Editor: Rita Fernando

Production Editor: Beth Kelly

Copyeditor: Piper Content Partners

Proofreader: Sonia Saruba

Indexer: nSight, Inc.

Cover Designer: Susan Brown

Cover Illustrator: Karen Montgomery

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

May 2022: First Edition
June 2026: Second Edition

Revision History for the Second Edition

2026-06-16: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098179922> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Observability Engineering*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and Honeycomb. See our [statement of editorial independence](#).

979-8-341-60841-2

[LSI]

Table of Contents

Foreword.....	xxiii
----------------------	--------------

Preface.....	xxv
---------------------	------------

Part I. Introduction to Observability

1. What Is Observability?.....	3
The Origins of Observability	4
Applying Observability to Software Systems	4
Properties of Software Dependability	5
Observability Is a Property of Dependable Software	6
How Observable Is Your Software?	7
Two Competing Models for Telemetry: Three Pillars Versus Unified Data	9
The Three Pillars Model	9
The Unified Storage Model	12
Observability Is the Validation of Developer Intent	16
The Agentic Incursion Has Just Begun	16
Guardrails Are Having a Moment	17
Conclusion	18
2. How Code Crosses Over: Validating Developer Intent in Production.....	19
What Makes Software Good?	20
Good Software Serves Its Purpose	20
Good Software Delivers Efficiently Over Time	20
The Maintenance Horizon: Disposable Code Versus Durable Code	21
Durable Code Is a Model for Mutating and Maintaining Code in Place	21
Disposable Code Is a Model for Avoiding Maintenance Costs	22
The Maintenance Horizon Is Not Always Knowable Up Front	22
Production Quality Code Is a Function of Dependability	23
Identifying the Critical Path	23
The Closer You Get to Persisting Data, the More Cautious You Should Be	24

Development to Production: Tools for Crossing Over	25
Practice 0: Give Yourself the Gift of Rich Data and Precision Tooling	25
Practice 1: Build a Feedback Loop Between Developers and Production	26
Practice 2: Test Your Code Before You Deploy It	28
Practice 3: Instrument Your Code and Validate in Production	29
Practice 4: Decouple Deploys from Releases Using Feature Flags	30
Practice 5: Invest in Progressive Delivery, Canaries, Automated Rollbacks	32
Bonus Practices: Traffic Splitters, Capture/Replay, Strangler Figs	35
Observability Is the Feedback Loop of Feedback Loops	35
The Work of Development Is Not Done Until It's Working in Production	36
Conclusion	36
 3. The Origins of Observability in Software.....	 39
An Introduction from Charity	39
The Dominant Model of “Observability” Is Just Monitoring, Rebranded	40
We Lost the Fight to Define Observability	41
Production Has Become Too Complex for Us to Debug Via Intuition	42
The Facebook Experience That Showed Us What's Possible	43
The Control Theory Definition Made It All Click	44
The Modern Observability Landscape Is Confusing and Continues to Expand	45
Costs Are Driving the Need to Change, and AI Is Enabling That Change	46
Distilling the Lessons That Matter	47
Conclusion	48
Part II. Instrumentation Fundamentals	
 4. Getting Started with Instrumentation.....	 51
Instrumentation Basics	52
The Vocabulary of Telemetry: Logs, Metrics, and Traces	52
OpenTelemetry: The Universal Language for Observability	54
Automatic Instrumentation Versus Custom Instrumentation	55
Ownership of Instrumentation	56
Building a Custom Instrumentation Strategy	57
Cost Considerations, Volume Management, and Processing	59
Processing and Pipelines	59
Cost Considerations and Volume	59
Essential Concepts for Sampling	60
Conclusion	62
 5. Structured Events Are the Building Blocks of Observability.....	 63
What Is a Structured Event?	64

The Limitations of Metrics	65
What Is a Metric?	65
Metrics Are Typically Aggregated	66
The Inner Workings of Logs	68
Turning Traditional Logs Into Structured Logs	70
Is a Structured Log the Same as a Structured Event?	71
Tracking a Single Operation	71
The Inner Workings of Distributed Traces	72
A Brief Introduction to Distributed Tracing	73
The Components of Tracing	74
Turning Logs Into Distributed Traces	76
Traces Are Collections of Spans	79
Trace Context	79
Dynamically Generating the Right System Views	81
Conclusion	83
6. Making Structured Events Arbitrarily Wide.....	85
Service and Code Context	86
Service Metadata	86
Build Information	89
Feature Flags	91
Versions of Important Things	92
Request and Execution Flow	92
HTTP Information	93
Route Information	96
Timings	97
Async Request Summaries	98
Errors	99
User and Business Context	101
User and Customer Information	101
Rate Limits	103
Caching	105
Localization Information	105
Operational Information	105
Uptime	106
Metrics	107
A Convention to Filter Out Everything Else	110
Attributes Important to Your Specific Application	110
Conclusion	111
7. Instrumenting Your Code with OpenTelemetry.....	113
What It Means to Use OpenTelemetry	114

Effective Instrumentation With OpenTelemetry	115
Trace-First Telemetry	115
Traces for Common Architectural Patterns	118
Metrics, Spans, Logs, Events—Oh My?	125
Using AI Agents to Instrument Your Code	127
What Is an Agent, Anyway?	128
Instrumentation with Agents	128
Useful Strategies for Agents	130
Conclusion	131

Part III. Analysis Workflows

8. Getting Started with Observability Analysis.....	135
Debugging from Known Conditions	136
Debugging from First Principles	138
Using the Core Analysis Loop	139
Automating the Brute-Force Portion of the Core Analysis Loop	141
Automating Analysis with Generative AI	144
Agentic AI Personas	145
Using Agentic AI for Observability in Practice	146
Conclusion	147
9. Observability-Driven Development.....	149
Test-Driven Development	149
Observability in the Development Cycle	150
Determining Where to Debug	151
Debugging in the Time of Microservices	152
How Instrumentation Drives Modern Observability	153
Shifting Observability Left	155
Using Observability to Speed Up Software Delivery	156
Observability-Driven Development with AI	157
Conclusion	159
10. The Role of AI Agents for Observability.....	161
What Is an AI Agent for Observability?	162
The Pitfalls of Querying Without Context	164
Proven Use Cases for Observability Agents	165
Incident Response	166
Explaining Errors and Patterns in Telemetry Data	166
Improving Instrumentation Quality	166
Observability-Adjacent Use Cases for Agents	167

The Production Problem: The Mental Model That’s Disappearing	168
The Need for Context	171
Conclusion	171
11. Using Service Level Objectives for Reliability.....	173
Threshold-Based Alerting Creates Alert Fatigue	174
Threshold Alerting Is Only for Known-Unknowns	175
User Experience Is a North Star	178
Reliable Alerting with Service Level Objectives	179
Case Study: Changing Culture Toward SLO-Based Alerts	181
Accelerating SLO Adoption with Generative AI	184
From Targets to Implementation: Drafting a Service Level Indicator	186
Encoding Best Practices in Your Prompts	189
Conclusion	191

Part IV. Observability Technical Deep Dives

12. Acting On and Debugging SLO-Based Alerts.....	195
Alerting Before Your Error Budget Is Empty	195
Framing Time as a Sliding Window	197
Forecasting to Create a Predictive Burn Alert	198
Threshold-Crossing Alerts	199
Relative Burn Alerts	200
Predictive Burn Alerts	201
The Baseline Window	209
Acting on SLO Burn Alerts	210
Using Structured Event Data for SLOs Versus Time-Series Data	212
Conclusion	214
13. Efficient Data Storage with Retriever.....	215
The Functional Requirements for Observability	216
Time-Series Databases Are Inadequate for Unified Observability	217
Other Possible Datastores	219
Data Storage Strategies	221
Case Study: The Implementation of Retriever	224
Partitioning Data by Time	224
Storing Data by Column Within Segments	226
Performing Query Workloads	228
Queries on Parts of Fields and Aggregated Fields	230
Querying for Traces	230
Joins	231

Querying Data in Real Time	232
Making It Affordable with Tiering	232
Making It Fast with Parallelism	233
Dealing with High Cardinality	234
Scaling and Durability Strategies	234
Notes on Building Your Own Efficient Datastore	237
Conclusion	238
14. Efficient Data Storage with ClickHouse.....	239
ClickHouse Core Concepts	240
MergeTree Fundamentals	241
Query Execution	243
Collecting Data	248
The OpenTelemetry Collector and ClickHouse Exporter	248
Alternate Ingestion Approaches	249
Querying Data	249
Simple Queries	250
Complex Joins	252
Scaling Vertically	254
Profiling Your Queries	254
Practical Data Modeling and Optimization	255
Data Lifecycle Management	261
Scaling Horizontally	263
Replication	263
Sharding	267
Single-Region Observability Cluster	268
Multiregion Observability Cluster	269
SharedMergeTree	270
Visualizing Your Data	272
Using ClickHouse for Observability Workloads	272
Conclusion	273
15. Cheap and Accurate Enough Sampling.....	275
Sampling to Refine Your Data Collection	275
Using Different Approaches to Sampling	277
Constant-Probability Sampling	277
Sampling on Recent Traffic Volume	278
Sampling Based on Event Content (Keys)	278
Combining per Key and Historical Methods	279
Choosing Dynamic Sampling Options	279
When to Make a Sampling Decision for Traces	279
Translating Sampling Strategies into Code	280

The Base Case	281
Fixed-Rate Sampling	281
Recording the Sample Rate	281
Consistent Sampling	283
Target-Rate Sampling	284
Having More Than One Static Sample Rate	286
Sampling by Key and Target Rate	287
Sampling with Dynamic Rates on Arbitrarily Many Keys	288
Putting It All Together: Head and Tail per Key Target-Rate Sampling	290
Conclusion	291

16. Telemetry Management with Pipelines.....	293
An Introduction to Telemetry Pipelines	294
The Telemetry Pipeline Solution	294
Why This Matters Now	295
Core Functions of a Modern Telemetry Pipeline	295
Collect	296
Normalize and Secure	296
Enrich	297
Reduce	297
Route	298
Data Resilience	298
Pipeline Control and Observability	299
What You Should Remember About Core Functions	300
Pipeline Architecture in Practice	300
Core Components	300
Deployment Patterns	301
Scaling and Performance in Practice	302
Build-Versus-Buy Considerations	303
What You Should Remember About Pipeline Architecture	304
Adoption and Migration	304
A Phased Path to Adoption	304
What You Should Remember About Adoption and Migration	306
Use Cases: Collect and Reduce, Combined	307
Business Case	308
Cost Control	308
Vendor Neutrality	308
Telemetry as a Strategic Asset	309
The Role of Telemetry Pipelines	309
Conclusion	310

17. Ontologies as a Shared Language for Humans and AI.....	311
Ontologies and Their Role in Observability	312
Design the Ontology	313
Define the Core Entities (the Nouns)	314
Define the Invariants (the Rules)	314
Visualize the Semantic Grammar of the Domain	315
Glue the Schema Through Metadata	316
Schematize Intent with the ActionPlan	317
Validate the Contract: Continuous Integration	318
Layer Three Gates for Defense	318
Establish the Team Workflow	319
Create Signal Parity in Production: Continuous Deployment	319
Understand the Hierarchy of Signals	320
Create Shared Instrumentation: The Universal Payload	320
Implement the AI Sandwich Architecture	321
Close the Loop: Production Driving Tests	322
Putting Ontologies into Practice	323
Conclusion	324

Part V. Observability Use Cases

18. Observability for CI/CD Pipelines.....	327
Why Reliable and Fast CI/CD Matters	328
Build Observability Has the Best ROI of All Observability Applications	328
CI/CD Through the Lens of Observability	329
The Ontology of Continuous Integration and Deployment	329
Instrumentation Basics	330
Defining Service Level Indicators and Achieving Predictability	331
From Jobs, to Directed Acyclic Graphs and Traces	331
Making Improvements and Measuring Them	332
Understanding Performance: Treating Continuous	
Integration Like Production	333
Predictability: Real-World Trade-Offs	334
(The Lack of) Incrementality: The Bane	
of Continuous Integration's Existence	335
Keeping Your Build Performance Tight and Your Developers Happy	336
Case Study: The Importance of Quick Build Times at Honeycomb	336
History of Improving Build Times at Honeycomb	337
Applying These Lessons to Your Build System	339
Conclusion	341

19. Observability for Mobile and Frontend.....	343
Status Quo for Mobile and Frontend	344
Instrumentation Limitations	346
Overcoming Domain Challenges	348
Recognizing the Problem	348
Instrumentation Difficulties	350
Applying Local Storage and Real-Time Control to Mobile Observability	354
What to Observe and Why	358
Opportunities for Improvement	359
Making Observability User-Focused	359
Quantifying User Experience	361
Adapting Existing Approaches	362
Applying User-Focused Observability	363
Iterative Analysis Process	363
Example: Food Delivery App	363
Mobile and Frontend Applications Deserve Observability	366
Conclusion	367
20. Performance Engineering with Observability.....	369
The Case for Performance Engineering	369
Building a Performance Engineering Practice	372
Optimizing Cost Without Modifying Application Code	373
Infrastructure Purchasing Models	373
Fleet-Wide Optimization	375
Cost Optimizing Kubernetes	376
Cost Optimizing Serverless	380
Observing the Costs	381
How Application Observability Reduces Cost	382
Using CPU Profiling Tools	383
Using the Correct Observability Signals, Together	385
Conclusion	388
21. Observability for Large Language Models.....	391
Why Observability Matters for LLMs	392
Using Evaluations for LLM Reliability	393
Designing Telemetry for LLM Applications	395
Analyzing Telemetry for AI Applications	396
Feeding Observability Data Into AI Application Development	398
Using Evaluations and Observability Together	400
Conclusion	402

22. Fin's Case Study in Modern Engineering.....	403
Increasing Resolution Rate	404
Speeding Up Without Losing Efficiency	405
And Thus, "Time to First Token" Was Born	406
Speeding Fin Up	409
Finance Enters the Chat	411
Empathy	413
Conclusion	415

Part VI. Observability Governance

23. Organizational Learning Speed Is Now Your Biggest Constraint: An Open Letter to CTOs.....	419
An Open Letter to CTOs	420
The Sociotechnical Debts That Will Hold You Back	421
When Developers Live In a World of Tests, Not Reality	423
When Telemetry Gets Treated Like Infrastructure, Not Product	424
When Developer Tools Were Never Designed as Products	425
These Debts Will Sabotage Your Adoption of AI	426
Turning the Ship Around	428
Measuring Value	428
Build Good Feedback Loops	428
Change Actions That Drive This Strategy	429
A Letter from a CTO: How Fin Engineering Optimizes for Learning	432
Conclusion	434
24. Systems Thinking for Software Delivery.....	435
Sociotechnical Systems	435
You Must Optimize Both the Social and Technical at the Same Time	436
Changing Information Flows	437
How Feedback Loops Drive Change	438
Amplifying Feedback Loops Accelerate Change	439
Balancing Feedback Loops Create Stability and Equilibrium	439
Small Shifts Can Trigger Massive Changes	440
The Difference Between a Virtuous Cycle and	
a Death Spiral Is the Ability to Self-Correct	441
Feedback Loops in Software Delivery Systems	441
Observability in Amplifying Loops	442
Observability in Balancing Loops	444
Partial Feedback Creates Systemic Distortions	446
Leverage Points in Sociotechnical Systems	448

Leverage Points and the Limits of Observability	448
Push in the Right Direction	449
Conclusion	450
25. The Observability Landscape Through a Systems Lens.....	451
The Landscape Feels Noisy Because the Labels Are Noisy	451
The Loops Most Organizations Run Today (and What Is Missing)	452
Development Feedback Loops	452
Operational Feedback Loops	453
The Missing Loop	455
The Feedback Loop for Value Creation	456
Shipping Is Your Heartbeat	456
How to Build the Loop and Close the Gap	458
Why Closing the Gap Is Rare: The Economics of Cognitive Load	459
Two Observability Models, Two Feedback Loops	459
Three Pillars Model (Built for Operational Outcomes)	460
Unified Storage Model (Built for Developer Learning)	460
AI Changes the Game and Opens New Interaction Models	461
Both Feedback Loops Matter: Lead with the Right One	462
Align on the Outcomes You're Building Toward	463
Conclusion	464
26. The Business Case for Observability.....	465
Identifying Your Priorities	465
Complementary Roles of the Two Feedback Loops	466
Read the Fire Codes	466
The Business Case for Operational Loops	467
Mapping Observability Models to Operational Outcomes	467
The Operational Mandate	468
The Business Case for Developer Learning Loops	469
Estimating the Value of Developer Learning	470
The Developer Mandate Has Two Halves	471
Strategic Investment or Cost Center Optimization?	475
Observability as Organizational Investment	476
Conclusion	477
27. Diagnosing Your Observability Investment.....	479
Don't Pay Observability Prices for Monitoring Outcomes	480
The Firefighting Trap	480
An Investment Posture Mismatch	481
Activities Versus Learning	481
How to Know If Your Investment Is Working (or Not)	482

Advice for Observability Investments	483
Conclusion	484
28. The Organizational Shift.....	487
Recognizing Legacy Masquerading as Modern Observability	488
The Ownership Test	488
The Two-or-Three People Test	489
The Mystery Test	489
The Arbitrary Question Test	490
The Deployment Confidence Test	490
After the Tests	491
Understanding the Resistance	491
The Organizational Immune Response	492
Why Legacy Vendors Fight	492
Why Existing Teams Resist	492
Why Leadership Is Skeptical	493
Why You Cannot Fight It Alone	494
Building the Coalition	494
Find the People Whose Pain Is Immediate	494
Honor the Heroes Who Held It Together	495
What Makes a Good Ally	496
Building the Case Together	496
Securing the Mandate	496
Sponsorship Versus Authority	497
Finding the Right Executive	497
The Pitch That Works	497
The AI Forcing Function	498
The Roadmap	499
Start Small: One Domain, Full Depth	499
Pave the Path: Make the New Way Easier	499
Platform Engineering Principles	500
Demonstrate Wins: Rerun the Tests	500
Expand Deliberately	501
Building the Team	502
What the Observability Team Owns	502
Capabilities of the Observability Team	502
Bringing the Existing Team Along	503
Breaking the Vendor Identity Trap	504
What Not to Promise	505
Conclusion	505

29. Build Versus Buy (Versus Open Source).....	507
Which Do You Hate More: Your Money or Your Time?	508
Surfacing Our Biases	508
Observability Is Not Like Other Software	509
A Simple Framework for Evaluating “Build Versus Buy”	510
The Four Quadrants: Build, Buy, Should Not Exist, or Decide	512
Mapping Examples to Quadrants	515
The Generalized Pattern	517
What About Open Source?	517
A Framework That Includes Open Source	518
The Four Quadrants: Avoid, Create, Adopt, Acquire	519
There’s Always Vibe Coding	520
The Real Costs of Building Your Own	520
The Hidden Costs of “Free” Software	521
The Benefits of Building Your Own	522
The Risks of Building Your Own	523
The Real Costs of Buying Software	524
The Hidden Financial Costs of Commercial Software	524
The Hidden Nonfinancial Costs of Commercial Software	526
The Benefits of Buying Commercial Software	527
The Risks of Buying Commercial Software	527
Build Versus Buy Is Not a Binary Choice	528
This Is Where Your Observability Team Comes In	529
Get the Best of Both Worlds	529
Important Caveats	530
Conclusion	531
 30. The Art and Science of Vendor Partnerships.....	 533
Does Buying Software Even Count as Engineering?	534
The Difference Between Vendor Engineering and Buying Software	536
Training Grounds for Executive Skills	536
What to Know Before You Start	537
Find Local Guidance	537
Understand Your Role	538
Mapping Your Stakeholders	538
Engineering	538
Management	539
Finance	539
IT/Ops and Security	539
Procurement	540
Communicating Up the Ladder	540
Trust and Credibility with Internal Stakeholders	541

Credibility Versus Trust	541
Building Trust with Stakeholders	541
Breaking Trust with Stakeholders	542
Trust and Reciprocity with External Stakeholders	542
Building Vendor Relationships	542
The Vendor Trust Coefficient	543
The Reciprocity Principle	544
How to Influence Another Company's Roadmap	544
SLAs Encode the Vendor–Customer Relationship	546
Early-Stage Considerations	546
Define the Scope of Work	548
Identify Your Executive Sponsor	548
Define Success Criteria Before You Start	549
What to Do When Your Coworkers Perceive You as a Threat	549
The OpenTelemetry Decision	550
Design and Run a Meaningful Proof of Concept	550
What Does a Good Proof of Concept Look Like?	551
Structure Your Technical Evaluation	552
Validate Pricing at Scale (If Cost Is a Concern)	552
Build Champions and Advertise Your Wins	553
Maintain Momentum Through Procurement	553
Migration and Follow-Through	554
Handing Off to Another Team	554
Most Migrations Have Three Steps	554
You Aren't Done Until You Decommission	555
Plan for the Next Transition	555
Conclusion	556
31. Instrumentation for Observability Teams.....	557
What Observability Teams Need To Know	558
Semantic Conventions	559
Telemetry Schemas in Practice	559
Tooling, Migrations, and Paved Paths	562
Tools and Frameworks	562
Paved Paths and Special Hats	564
Instrumentation in Highly Secure Environments	565
Use Compliance Tiers to Your Advantage	566
A Minimal Set of Capabilities for Observability Teams	567
Valuable Capabilities in a Regulated Environment	568
This Is Hard Work, But Work Worth Doing	569
Conclusion	569

32. Where Do We Go From Here?..... 571
 Developers Must Return to Production 571
 The Changing Cost Model of Software 572
 The Most Important Parts of Our System Have Never Been Specified 573
 The Great Unbundling 574
 The Human Cost of Compressed Change 574
 The Claim: “AI Is Unprecedented” 575
 This Moment Has Many Precedents 576
 The Sysadmins Went Through This 576
 From Software Code to Software Systems 577
 Value Must Be Backed by Durability 578
 The Tools to Build Durable Systems Will Come 578
 Conclusion 579
Index..... 581

Foreword

Remember when we used to be able to predict the upcoming tech trends, the emerging architectural evolutions, and what the software world would be like in five years? I mean, we were never able to actually do so accurately, but it sure felt like we could. The first edition of this book came out in 2022, which somehow feels like it was only yesterday, yet it also feels like an entirely separate era of history.

Since the first edition came out, we've learned how to apply observability to mobile and the web, we've seen the rise of agentic AI, we've dug into vendor relationships, we've explored instrumenting regulated environments, and so much more. Every year has seen the momentum around observability increase. It's been so exciting to watch it happen in real time!

But that momentum isn't without a cost. The extraordinarily wide range of topics in observability has made the second edition of this book far longer (and also far better, in my opinion). It's almost an entirely new book! From behind the scenes, I saw all of the hard work that Charity, Liz, George, and Austin put into this book; their efforts, along with those of numerous other contributors, have shaped this into a book that could never have existed before now. I hope you appreciate the magic and sweat and tears. I hope you enjoy reading the results. It turns out that while writing about observability is already hard enough, knowing when to stop and cut the book release is an impossible challenge. Especially when AI threatens to make everything obsolete every two months. Well, not everything.

The hard-won knowledge about how complex sociotechnical systems work? That's never going away. The practitioners working to understand their systems? We're still here, and we're not leaving. Software complexity has dramatically increased. So has the growth of the tech industry. Remember when microservices and Kubernetes were "unnecessarily complex"? Now that's a three-pillar-era throwback. Throughout all this growth, we've had to figure out how to build the tools and practices necessary to keep our systems healthy. It's hard work! It felt like we had finally figured it out before AI arrived on the scene. Now, sometimes it feels like we're starting all over again.

Predicting what the world will look like in five years? I feel like I can barely predict what next week will bring, but I can't help but speculate. Candidly, here's what I see for the next five years in observability: the world is going to get even more intense, and we're going to have to keep up. How will we do that? I see a few key evolutions taking place to enable this: lakehouse data architectures, contextual tooling, and delivering value outside of engineering. These three things synergize with each other, and the combination is incredibly powerful.

Observability has traditionally been most limited by the constraints of the data engine backing the tooling, but lakehouse architectures eliminate most of those constraints, enabling maximum cardinality, flexibility, compliance, and more. Meaningfully using telemetry data, and accessing it where you need it, has been the hardest UX challenge of observability, but advances in contextual tooling are enabling new possibilities for bringing the right data to the engineers (or agents) who need it, and when they need it. Lastly, technological investments are increasingly cross-functional, and so the value that is produced from engineering practices also needs to evolve and become cross-functional. The days of buying tools for platform operators exclusively and hiding them away from the rest of the company are coming to an end.

Above all, my prediction for the future is that it's going to get weird. It's already weird. It's going to get weirder. It's going to change faster than we ever thought possible. We're going to have to rethink our prior assumptions; we're going to have to break our old mental models and build new ones. Our beliefs about software engineering will change, and our techniques are going to stop working. Just like "open the debugger and set a breakpoint" stopped working, those three pillars of observability and those shiny, sparkly dashboards are on their deathbeds as we speak.

I find a lot of comfort in that swirling vortex of unknown unknowns, personally. The reason for that is because it gives me that much more confidence in what will stay the same: people. Yes, even in a world of AI, people are still here. We're still going to work together. We're going to build new tools, new systems, and new methods. We're going to learn together. We're going to dance and laugh. We're going to knuckle down and push ourselves to the next level. We're going to reach ever upward, to the next generation.

And while we can't predict the future, we can prepare for it—starting by reading this book. So please, find a comfortable spot to relax and enjoy the tools, techniques, and insights this book offers. Together, let's learn how to understand and observe our systems of today and tomorrow.

— Hazel Weakly
Fellow, Nivenly Foundation

Preface

When we wrote the first edition of this book, the observability landscape was shifting, and practices were advancing faster than we could collectively write them down. In retrospect, we were chasing a moving target. It took three and a half *years* to write and edit the book that was released in 2022.

In the four years since, the landscape and practices for observability have solidified a great deal, which makes it easier to write about. However, life is never boring; ChatGPT and GitHub's Copilot were also released in 2022, kicking off an explosive period of innovation that has yet to subside.

As we write this preface, in March of 2026, the entire software industry is going through yet another unprecedented upheaval, a convulsive period of compressed change. New AI development workflows are being hammered out in real time, for durable code as well as disposable code. Harness engineering, agentic orchestration, Phoenix architectures...every day seems to come with new terms and new concepts. It feels overwhelming trying to keep up, and the only consolation is that everyone else feels just as overwhelmed.

We've seen AI models leap generations, entirely new categories of tooling appear, and workflows that felt cutting-edge become table stakes, all within one fiscal quarter. This is what exponential change feels like from the inside: not a smooth trajectory, but a series of jarring “wait—already?” moments. The hype trains are roaring, and it can be difficult to pick trustworthy voices out from the ambulance chasers seeking to cash in on our anxieties.

And yet, an emerging, ongoing punch line this year has had people dryly commenting each time we rediscover the principles we've known all along:

“Oh, so we should have *fast* feedback loops?”

“Tests are important, you say?”

“Are you insinuating that we should talk to our own users? To *each other*?”

This time around, it has only taken us nine months to write and edit the book, despite it being twice as long and almost entirely rewritten. Yet as we went into final edits, reading the book front to back, it was already painful to read some of the things we had written less than a year ago. A year ago, it was reasonable to be skeptical about whether or not generated code would ever be as good as the median developer could write, or how long it would take; now it is not reasonable. A year ago, it was reasonable to bifurcate the space into disposable versus durable code; now that division is blurring.

We have done our best to adjust for the shift in aperture, to acknowledge the ways that our spectrum of possible futures has widened and bent in recent times.

But this is not a book about AI; it is a book about software development. And the further we get, the more we learn, the clearer it becomes to us that *software (and people) are still the hard parts*. The struggle is less about the truly novel aspects of AI, such as wrangling nondeterminism, and more about the things that have always been hard: connecting cause and effect, constructing coherent mental models, and managing user expectations. Only now, the rate of change is revving up to 10 times, 100 times, 1,000 times as fast as ever before.

We take comfort in this. Observability itself has never felt like something flashy or new to us—it has always felt like an intrinsic property of the software discipline, derived from first principles, discoverable by anyone. We believe this still. And we fiercely believe these first principles are more relevant and desperately needed by practitioners than ever before.

This moment of compressed change will not last forever. Indeed, the one thing we know about exponential change is that *it must end*, because all resources have limits.

As uncomfortable as it is, this change is also an unprecedented window of opportunity. After 20 years of DevOps, the overwhelming majority of software engineers still don't include production in their development workflow; at many places, they aren't even allowed to. The number of companies that have the right guardrails and precision tools in place to validate AI-generated code in production is minuscule—in the single digits.

Why? Because it's *hard*. Building out feature flags, progressive deployment, and the right instrumentation has been time-consuming and labor-intensive, and the old ways worked well enough to get by, using human intuition as spackle and duct tape.

But AI breaks this stalemate. It enables key technological breakthroughs on the supply side. And on the demand side, it provides the competitive pressure to change. Engineers who have long agitated for better platform support and enablement should receive this opening as the gift that it is.

Observability is not just another developer tool; it is the keystone species of the ecosystem. It is the signal that connects “cause” with “effect” to make feedback loops, no matter how far apart “cause” and “effect” may reside. The quality of your observability has downstream consequences for every other thing you do.

The goal of this book is to arm you with a clear understanding of the principles at work. Most of the industry has historically chosen to iterate on what is familiar and comfortable instead of embracing a different, better, but admittedly more disruptive approach. AI seems likely to strain the old ways to the breaking point. But every organization needs to figure out where or if that applies to them, and chart a course from where they are to where they need to be.

That’s where you come in. We hope this book will equip you with the tools you need to diagnose, argue, debug, build, and instrument your way through this messy present and toward a better tomorrow.

Why We Wrote This Book

We wrote this book because we care about observability and the people who need it. We care about it *a lot*. We care about it so much that we pioneered the category, founded a company to work on the problem space, helped maintain OpenTelemetry and make it go mainstream, and have now spent close to 20 collective person-years writing three books on the topic.

While we all work for or have worked for **Honeycomb**, we are not trying to sell you on its tools. You can use the concepts in this book to build production software systems with observability using any tools, although we would be lying if we said all tools are equally effective.

We believe in enduring values like truth, reality, and shared understanding. We believe observability can help us navigate today’s rocky waters as the technology landscape shifts quickly. We believe this is more important than ever.

We hope this book can help bring structure and clarity to what has become a very muddled and marketing-driven public discourse about observability.

What’s Different in the Second Edition

First, we have a new coauthor. We are delighted to welcome Austin Parker, with deep subject matter expertise in AI and OpenTelemetry.

We’re also excited to incorporate a broader range of voices for this edition of the book, featuring a stellar lineup of guest writers. The opportunity to spotlight some of the people we look up to and enjoy working with has been one of the greatest pleasures of putting this book together.

In order of presentation in the book:

- Jeremy Morrell wrote one of the most comprehensive guides to using wide events and creating custom attributes that we’ve ever seen. He has contributed greatly to an update of [Chapter 5, “Structured Events Are the Building Blocks of Observability”](#), and has entirely contributed [Chapter 6, “Making Structured Events Arbitrarily Wide”](#).
- Boris Tane explored agentic AI use cases and examined the principles necessary to generate and retain context, which continues to be key for success as we shift toward more automation. We think you’ll enjoy the principles outlined in his contributed content, [Chapter 10, “The Role of AI Agents for Observability”](#).
- Mat Vine provided an in-depth case study on SLOs adoption for an updated look at its challenges and solutions. This replaces the Honeycomb-focused use case previously used in [Chapter 11, “Using Service Level Objectives for Reliability”](#).
- The engineering team at ClickHouse contributed an in-depth look at how their open source datastore is tuned to meet the needs of observability workloads in [Chapter 14, “Efficient Data Storage with ClickHouse”](#). This new chapter provides an alternative implementation to the Honeycomb storage engine, detailed in [Chapter 13, “Efficient Data Storage with Retriever”](#).
- Mike Kelly contributed an in-depth look at telemetry management by examining challenges and solutions through the lens of the open source Bindplane project in [Chapter 16, “Telemetry Management with Pipelines”](#).
- Frank Chen, guest contributor in the first edition, returned to the second edition with a look at ontologies and how to think about your entire instrumentation chain in [Chapter 17, “Ontologies as a Shared Language for Humans and AI”](#).
- Hugo Santos contributed greatly to an update of Frank Chen’s first-edition take on instrumenting tests and continuous delivery pipelines in a new [Chapter 18, “Observability for CI/CD Pipelines”](#).
- Hanson Ho and Matt Klein presented a look at the complex and challenging workflows necessary for implementing high-observability workflows in the mobile device domain in [Chapter 19, “Observability for Mobile and Frontend”](#).
- Phillip Carter contributed greatly when applying the principles in this book to the world of running generative AI applications in production in [Chapter 21, “Observability for Large Language Models”](#).
- Kesha Mykhailov contributed a first-person narrative of the observability journey at Fin (formerly Intercom) in [Chapter 22, “Fin’s Case Study in Modern Engineering”](#). This chapter demonstrates how many of the concepts detailed in the book often come together in real-world scenarios.

- Darragh Curran contributed an open letter to chief technology officers (CTOs) detailing how his organization used observability to tackle the problem of accelerating organizational learning speed in an AI era in [Chapter 23, “Organizational Learning Speed Is Now Your Biggest Constraint: An Open Letter to CTOs”](#).
- Rick Clark contributed a deep dive into driving change without formal authority in [Chapter 28, “The Organizational Shift”](#). This chapter covers how to recruit champions, drive consensus, and shepherd disruptive transformational change through complex sociotechnical systems.
- Hazel Weakly wrote a wonderful foreword for us on the shifting horizons of software development, and contributed greatly to [Chapter 31, “Instrumentation for Observability Teams”](#), especially for teams operating in highly secure environments.

Second, we have two parallel tracks of guidance for engineers who write code: one track for AI-assisted development and the other for classic development. You’ll find both tracks in each chapter on instrumentation and analysis. Instead of anchoring our guidance in any one particular technology or tool, we describe the principles necessary for success and how they affect outcomes.

Lastly, we’ve restructured the material into chapter groups based on functional roles. The first edition of this book was “about observability,” for “everyone.” This edition aims for a more targeted conceptual structure.

How to Read This Book

We wrote this book with software engineers and technical decision-makers in mind, but it should be relevant (and hopefully interesting) to anyone else who builds software or works with those who do.

Part I is for all readers. It contains foundational concepts that we will draw on for the rest of the book: what observability is, how to develop good software, and the user experience you’re aiming for. Readers of the first edition might skip [Chapter 3](#) (arguably, it was the point of the entire first edition). However, this part of the book has been entirely overhauled to better frame the “observability” conversation in our modern zeitgeist.

Parts II and III are written primarily for software engineers and observability practitioners. Every software engineer should understand how to instrument their code and use the telemetry it emits. The chapters in this section cover the technical basics that all modern software engineers should be familiar with.

Part IV is for engineers who want to go beyond the surface. You can think of these as the “show your work” chapters of the book. We dive deep into the internals and plumbing that enable the instrumentation and analysis demonstrated in **Parts II** and

III. This segment of the book is most relevant to engineering teams operating at scale or building their own observability tools.

Part V covers a number of different use cases for observability. Earlier sections of the book aim to be neutral on use case and setting, though they favor approaches best suited to working with backend systems. **Part V** shows how the same principles can be applied to other domains.

Part VI is for technical decision-makers. Its chapters are aligned with typical org chart concerns from the top down, starting with an open letter to CTOs and ending with instrumentation guidelines for observability engineering teams. Technical decision-makers and strategic thinkers come in many guises: not just executives, directors, and managers, but also observability engineering teams, platform engineering teams, site reliability engineers, cloud engineers, architects, and more. If you are trusted with decisions to empower and enable the software engineers in your organization, then this section is for you.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords

Constant width bold

Shows commands or other text that should be typed literally by the user

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://oreil.ly/7IcWz>.

If you have a technical question or a problem using the code examples, please send email to support@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Observability Engineering* by Charity Majors, Liz Fong-Jones, and George Miranda, with Austin Parker (O'Reilly). Copyright 2026 Hound Technology, Inc., 978-1-098-17992-2.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning



For more than 40 years, **O'Reilly Media** has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
141 Stony Circle, Suite 195
Santa Rosa, CA 95401
800-889-8969 (in the United States or Canada)
707-827-7019 (international or local)
707-829-0104 (fax)
support@oreilly.com
<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata and any additional information. You can access this page at *<https://oreil.ly/observability-engineering-2e>*.

For news and information about our books and courses, visit *<https://oreilly.com>*.

Find us on LinkedIn: *<https://linkedin.com/company/oreilly>*.

Watch us on YouTube: *<https://youtube.com/oreillymedia>*.

Acknowledgments for the First Edition

This book would not have been possible without the support of executive sponsors at Honeycomb: many thanks to Christine Yen, Deirdre Mahon, and Jo Ann Sanders. Nor would this book have been possible without the domestic sponsors who put up with many odd hours, lost weekends, sleepless weeknights, and cranky partners: many thanks to Rebekah Howard, Elly Fong-Jones, and Dino Miranda. Without all of them, we would probably still be trying to find the time to fully develop and tie together the many ideas expressed in this book.

We'd especially like to thank additional contributors who have made the content in this book much stronger by sharing their varied perspectives and expertise. **Chapter 16, “Telemetry Management with Pipelines”**, was made possible by Ian Wilkes (author of Honeycomb's Retriever engine, the basis of the case study) and Joan Smith (reviewing for technical accuracy of references to external literature). **Chapter 14, “Efficient Data Storage with ClickHouse”**, was authored by Frank Chen, and **Chapter 18, “Observability for CI/CD Pipelines”**, was authored by Suman Karumuri and Ryan Katkov—all of whom we thank for sharing their knowledge and the lessons they've learned from managing incredibly large-scale applications with observability at Slack. Many thanks to Rachel (pie) Perkins for contributions to several early chapters in this book. And thanks to the many bees at Honeycomb who, over the years, have helped us explore what's achievable with observability.

Finally, many thanks to our many external reviewers: Sarah Wells, Abby Bangser, Mads Hartmann, Jess Males, Robert Quinlivan, John Feminella, Cindy Sridharan, Ben Sigelman, and Daniel Spoonhower. We've revised our takes, incorporated broader viewpoints, and revisited concepts throughout the authoring process to ensure that we're reflecting an inclusive state of the art in the world of observability. Although we (the authors of this book) all work for Honeycomb, our goal has always been to write an objective and inclusive book detailing how observability works in practice, regardless of specific tool choices. We thank our reviewers for keeping us honest and helping us develop a stronger and all-encompassing narrative.

Acknowledgments for the Second Edition

This edition would not have been possible without the support of many external contributors, coauthors, editors, fellow travelers, and readers of the first edition. The words you're about to read aren't ours alone—they've been shaped by every hand that has touched this book. As you read it, we welcome you into the continuing conversation about observability, and hope that you will contribute to it in kind.

We would like to specifically thank our editors at O'Reilly Media—Rita Fernando, Virginia Wilson, John Devins, Megan Laddusaw, Louise Corrigan, and Jonathon Owen. Their tireless efforts, along with the entire O'Reilly team, have been invaluable to the success of this project. Additionally, we'd like to acknowledge and thank our external reviewers—Marty Lindsay, Alex Ewerlof, Phillip Carter, Jordan Simonovski, Rick Clark, Ranbir Chawla, RK Sharma, Dave Williams, and Lesley Cordero. Your contributions helped us refine, focus, and sharpen our thoughts.

Charity Majors

It is a truth universally acknowledged that when you ask the internet for opinions, you will get them—unfiltered, unrestrained, and in volumes suggesting you should not have asked. Nevertheless, I asked the internet for advice on the software buying process, and 87 people replied. I owe every one of you a debt of gratitude.

The back third of this book was headed in one direction before your stories; afterward, it veered in an entirely different one. If readers find it useful—and I hope they will—they have you to thank. (I, meanwhile, had to throw out my draft and start over from scratch in November, right when I was supposed to be wrapping up. I have come around to gratitude, but only after many months spent shaking a fist in your general direction.)

Sam Dwyer, Abraham Ingersoll, Benjamin Mann, Hazel Weakly, and the inestimable Marty Lindsay were especially generous with their time. Thank you. And Rita Fernando, our editor: I know we just thanked you like two sentences ago, but once is not enough. Have another.

Since I, like Austin, am Never Writing A Book Again™, I can't pass up this opportunity to thank my friend, co-founder, and rock-solid work partner of 10 years and counting, Christine Yen. You are the structure to my mess, the trellis for my aggressively overenthusiastic vinery, and my favorite person to embarrass with a compliment, no contest.

It's risky to start naming any of the brilliant, caring, dedicated people I get to work with at Honeycomb, because once I get started it's hard to stop. But I owe particular thanks to Ben Hartshorne and Ian Wilkes, who have been the heart and pickled spleen of Honeycomb Engineering since day one, and to Emily Nakashima, who does five impossible things each day before breakfast. Honeycomb is the product of many talented hands, but you are our—oh yes—three pillars.

Finally, and most importantly, to my family. I am grateful for your tolerance, your support, and your love—and perhaps even more for what you refuse to tolerate. Francis, Mark—thank you for crossing the ocean again and again to stay connected. Mom and Dad, Claire, Justice, Dan, Seri, and Rees: I'll see you at Thanksgiving. For real this time.

Liz Fong-Jones

A special thanks to all of my family and partners who tolerated my long work hours and stress, including Elly, Ellen, Alison, Lydia, and Amber. Eternal love to my fuzzy companion Flurry, who never got to see the book finished.

George Miranda

Looking back on the first edition's acknowledgements, I'm reminded of just how instrumental Deirdre Mahon was in making this book possible. Deirdre, I've said it before and I'll say it again now in print: thank you for the advice. For this edition, I'm looking at you, Jeff Gray. Logistically, you made this one possible. I can't begin to imagine where we'd be if you hadn't got involved. Thank you for your guidance, candor, and perspective that continues to resonate.

Austin Parker

Apparently, the best way to keep writing books is to loudly exclaim, "Well, never doing *that* again!" to anyone in earshot after completing one, as it indicates that you would indeed write another book. I'd like to acknowledge the thousands of contributors and maintainers of the OpenTelemetry project for their efforts and work over the years—you make observability a possibility for so many. I would also like to thank my spouse, Mandy, for her infinite patience and for being a constant source of inspiration and encouragement.

Introduction to Observability

This first segment lays the foundations that the rest of the book builds on.

You'll learn what observability actually means in the context of software development, how AI changes that picture, and how to gauge how observable your system really is. You'll also learn what investigative capabilities a high degree of observability unlocks, and how to test whether your system actually has them.

Chapter 1 traces where the term “observability” came from, how it was adapted for software systems, and what it actually means to have an observable system. It also looks at how to think about observability in the age of AI-assisted development and agentic applications.

Chapter 2 is about how code crosses over from development into production, and what it takes to do that well. It describes the full “test in prod” toolkit—instrumentation, feature flags, canaries, automated rollbacks—as a flywheel where each practice enables the next. And it makes the case that all of it depends on one thing first: observability that is rich and precise enough to actually trust.

Chapter 3 is a retrospective on the evolution of observability, written by coauthor Charity Majors from her own perspective. It focuses on the lessons that matter most now that “observability” has become ubiquitous, and closes with the functional requirements that anchor the concepts introduced throughout this part of the book.

In **Part II**, we'll introduce you to instrumentation principles and OpenTelemetry.

What Is Observability?

The topic of observability has generated a steady supply of attention and buzz over the past 10 years, ever since it was first introduced to the field of software development. However, as with most complex topics, a surging tide of practitioner adoption and interest in observability was swiftly countered by a wave of oversimplification and appropriation, fueled by hype and marketing budgets.

When you see this cycle play out repeatedly, it is easy to be cynical: maybe words just don't mean things when money gets involved. It's worth taking a step back to consider: why did we all get so excited in the first place? What was so revelatory, and why did it matter?

This chapter examines the mathematical origins of the term “observability,” and how the software development industry has interpreted it and adapted it to our world. We also look at the tectonic plates that are colliding at almost every company right now, between the old ways (which evolved out of server ops and still dominate the ecosystem) and the new ways—derived from first principles and far more expansive in capabilities, yet swimming upstream against the way things have always been done.

Along the way, we'll argue that observability isn't just a debugging technique or a category of tooling; it's a property of software dependability, as fundamental as reliability or availability.

Observability as a concept was born out of necessity, when traditional tools and techniques simply could not keep up with the long tail of novel problems coming out of our increasingly complex systems.

Our best guess is that AI will be the thing that forces teams to finally move away from the operator model of telemetry and toward a more unified model, the way business data did in the 2000s. Observability is the last line of defense for AI-generated code, and agentic behavior is inherently unpredictable and changes over time. The *only* way

to know what it's doing is with context-rich data and precision tooling as it runs in production.

Then again, we thought the same thing 10 years ago. We thought there was no way the three pillars model could survive the complexity of microservices, serverless, and polyglot persistence. But somehow, it did. We are engineers, not prognosticators.

We have a lot to pack into this chapter. Let's get started.

The Origins of Observability

The term “observability” was coined by engineer Rudolf E. Kálmán in 1960, in a paper that we now cite as the birth of modern **control theory**—the mathematics of systems you want to steer toward a desired state despite noise, disturbance, and incomplete information.¹

Kálmán's key insight was that observability and controllability are duals. A system you cannot observe is, in a precise mathematical sense, a system you cannot control, and vice versa.

For Kálmán, observability meant asking the question: given the outputs you can measure, can you infer the complete internal state of the system? If yes, the system is observable. If no, some internal state is permanently hidden, and no amount of clever engineering will recover it, because the information was never captured to begin with.

Software borrowed this from systems engineering. Like most things we borrow, it arrived with some of the original meaning intact and some parts lost in transit. (The controllability part has no equivalent in software.)

But when we apply the principles of observability to the squishier domain of software, it opens up radical new ways of interacting with and understanding the code you write.

Applying Observability to Software Systems

Software engineering has a long, rich history of borrowing from other domains: biology, architecture, and every type of engineering. We can do this in software because we are less tethered to physical constraints. The farther up the stack you go, the more you are unconstrained by practicalities like gravity, mass, or the speed of light. The only structure we get is the structure we give ourselves, using logic, language, and metaphor.

¹ Kálmán, Rudolf E. 1960. “On the General Theory of Control Systems”. *IFAC Proceedings Volumes* 1, no. 1 (August): 491–502.

Software may not be grounded in any of the constraints of its constituent parts—atoms, electrons, fiber optics, and so on—but software has its own constraints and considerations to worry about. And the principles behind Kálmán’s definition of observability have proven useful for reasoning about modern software systems.

Our systems and practices have changed continually over the years. Our understanding of observability has changed as well.

In the first edition, we described observability as a new breed of tooling, defined by the capabilities you could unlock by rethinking data storage and analysis. This definition ruffled many feathers—and understandably so, since it implied that more traditional tools afforded no observability whatsoever. We came to disfavor it for a different reason: it emphasized features and capabilities, but what people actually care about are outcomes.

We have come around to a more encompassing view since then, which is that observability is a property of software systems, like reliability or performance. We will argue that the observability of your software is a core attribute of its *dependability*.

“Dependability” is another borrowed concept (from systems engineering, this time) we now rely on. Software dependability is typically expressed on a spectrum ranging from low to high, as are each of its attributes.

Let’s start by examining the other properties of software dependability. Then, we’ll look at how to measure the property of observability.

Properties of Software Dependability

Software has eaten the world. Every time we call a friend, take the train, or get paid, we rely on a massive invisible web of software behaving more or less as expected, despite the evergreen fact that all hardware will break and all software has bugs.

Failures are a fact of life. The software industry may have made only modest gains at preventing bugs, but we’ve done a lot to make our systems more resilient to them. As technology plays an even greater role in our lives, our tolerance for unpredictability has decreased. This makes **dependability**—“the trustworthiness of a computer system”—a central concern of our era.

Software dependability cannot be quantified numerically. There’s no such thing as a *dependability score of 9* or a *grade B+ for dependability*. Some components, like availability, often get measured with nines—e.g., uptime that is 99.9%, 99.99%, or 99.999% available. You can measure seconds of availability, but that metric tells you nothing about dependability. Nines don’t matter if users aren’t happy.

Luckily, there's a well-established path for measuring dependability characteristics. In 1995, Jean-Claude Laprie proposed a framework for measuring software dependability by evaluating six complementary properties that together describe a system's overall trustworthiness:²

Availability

The ability of the system to deliver services when requested

Reliability

The ability of the system to deliver services as specified

Maintainability

The ability of the system to undergo repairs and evolution

Safety

The absence of catastrophic consequences

Confidentiality

The absence of unauthorized information disclosure

Integrity

The absence of improper system alterations

To assess dependability, you have to look at three things together: what the system is supposed to do (functions), what it actually does (behavior), and how it's built (structure). This applies across both hardware and software, including the environment they operate in—and, crucially, the people who interact with the system. Laprie defined software systems as inherently sociotechnical.

That last point is important and easy to overlook. You can't fully analyze dependability by staring at code. The humans in the system are part of the system.

Observability Is a Property of Dependable Software

In the mid-1990s, Laprie's framework was comprehensive enough to collectively describe software dependability, just like monitoring was enough to meet the reliability needs for systems of the era.

Monolithic systems were relatively predictable. The problems you'd encounter in production were mostly binary: the site was either up, or it was down. (Sometimes, if you were unlucky, it had a third state: inexplicably slow.) Monitoring was well-suited to this. It detected whether a service was up, down, or underperforming, and engineering teams responded with break-fix work to restore availability.

² Laprie, Jean-Claude. "Dependability — Its Attributes, Impairments and Means." In *Predictably Dependable Computing Systems*, edited by Brian Randell, Jean-Claude Laprie, Hermann Kopetz, and Bev Littlewood. ESPRIT Basic Research Series. Springer, 1995.

In the modern cloud era, many infrastructure failure modes are abstracted away and handled by the platform. Monoliths gave way to microservices. The industry-wide push to improve resiliency made everything more distributed, bringing many second-order effects. The most significant of these was a radical change in how we think about availability.

The cloud era ushered in a sprawling zoo of partial failure states. The previously rare third state—inexplicable slowness with no smoking gun—became the most common failure mode: a partially degraded load balancer; a hardcoded config pointing to `us-east-1` from containers in India; a dependency running on another team's infrastructure, behind an API, where you can't even see what it's doing. Sounds like a Monday.

Simple monitoring systems can tell you *that* your system is mostly up or down. More sophisticated monitoring can tell you *where* something is slow. But monitoring can't tell you *why*. You need all three to resolve issues in production.

Software dependability now has to include how quickly faults are detected, how quickly their sources can be identified, and how effectively teams can reason about why those faults are occurring, even when they are quite small. In other words, you need to be able to do this at any level: from a high-level view of the entire system, down to each network hop between services in a given session, and any subset or combination of requests in between.

This is why we believe it's time to add a seventh property to Laprie's list:

Observability

The ability to understand or debug any given system state

This isn't a minor amendment. We'll spend the rest of this book unpacking why it belongs there and how to achieve it.

How Observable Is Your Software?

Properties of dependability, like reliability, security, and maintainability, aren't measured numerically; neither is observability. But you can reason about it qualitatively, and that's a useful place to start.

Fault detection is relatively straightforward: do you know an issue is occurring before your customers report it? If you've managed a modern cloud application, you know how common it is for monitoring to miss critical issues. Your customer support team flags a spike in complaints, so you check the dashboard but everything looks fine—if that sounds familiar, you have a fault detection problem.

Triage—figuring out where an issue is occurring—gets harder as architectural complexity increases. In low-observability systems, this often manifests as alert storms: a failure in one underlying system triggers dozens or hundreds of alerts in others. When a critical dependency fails, do you know exactly where to look? Or do you have to check multiple sources and sleuth your way to the source of the cascade?

Understanding why an issue is occurring is usually the hardest part. Once you’ve found the source, you have to figure out what caused it. Identifying the causal trail can be very challenging, but it’s the meta process where teams struggle most: figuring out *how they figured it out*, and encoding the trail of breadcrumbs into the system instead of living exclusively in the heads of senior engineers and manifesting as intuitive leaps.

As Scotty from *Star Trek* once said: “I could tell you the speed that we were traveling by the feel of the deck plates.” Good for you, Scotty, but that doesn’t help the rest of the crew. Can we get the current warp speed displayed where others can see, please?

Can any engineer on your team, regardless of experience, quickly diagnose complex issues by interrogating the data your system emits? Or does a correct diagnosis require deep institutional knowledge and a sixth sense?

The answer to that question is how observable your software is. The changes that AI is bringing to the industry will make this exponentially more relevant.

Cardinality

In mathematics, cardinality refers to the number of elements in a set. In databases, it means the number of unique values in a column. A column called `Status` that contains “true” or “false” has a cardinality of 2, while a universally unique identifier (UUID) or any other unique set of values has the highest possible cardinality.

High-cardinality data is the most descriptive and identifying type of data, and it is vital for observability. Think of all the most useful attributes you would like to search or group by: build number, commit hash, container ID, span ID, user ID, app name, query hash, etc. All of which are high cardinality, and that is why they’re useful.

High-cardinality values can be downsampled (e.g., bucketing last names by prefix, or replacing all HTTP error codes with “5xx”), but this cannot be undone. If you aggregate the data before writing it out to disk, you can never put Humpty back together again.

Cardinality is a never-ending nuisance for teams that rely heavily on metrics. High-cardinality data is the most valuable data, but metrics aren’t built to handle it. A single metric can increase your bill by tens of thousands of dollars *per week* if enough cardinality slips into the data unnoticed.

In a software system with a high degree of observability, you should be able to:

- Understand any system state that your application may have gotten itself into, even completely novel ones, *without* shipping new code to handle it (which would imply you needed to know about it to find it).
- Compare one set of feature flags against any other set, then break down and group by endpoint, API key, query hash, or any other combination of interesting dimensions, with no limits on cardinality.
- See how users are interacting with the feature you just shipped. Trace by transaction request, browser session, mobile app session, or chat history.
- If any of your service level objectives (SLOs) are burning hot, you should be able to select the violating events and surface any patterns. (“The spike in slow requests is coming from read replicas in Australia, from a build ID we just deployed to 1% of canaries using flags x, y, and z; the client error string says DNS is timing out.”)
- Find outliers. Any kind. Easily.

To answer these questions, you need precision, flexibility, and (ideally) speed.

Software systems with lower degrees of observability are ones where these questions can only be answered with an enormous investment of time and manual labor, or by an expert who can practically intuit the answers.

Two Competing Models for Telemetry: Three Pillars Versus Unified Data

Any tool can be evaluated by how well it helps you achieve a high degree of observability. The principles are what matter—and the principles behind the two most common approaches are very different.

These two tectonic plates are on a collision course with each other right now, all over the industry, as the immovable rock of “how we’ve always done things” meets the unstoppable force of AI and competitive pressures. Will engineering teams upgrade their tool kit, or will the three pillars adapt and prove impossible to dislodge? We’ll find out.

The Three Pillars Model

The first plate is the old way. We call it the “three pillars”—metrics, logs, and traces—but that’s a polite fiction. There are far more than three types of signals: errors, exceptions, profiling data, product analytics, security events, etc. Each type of signal gets streamed off and stored in its own special silo, as shown in [Figure 1-1](#).

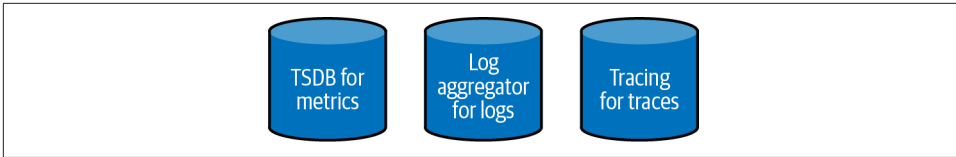


Figure 1-1. How the three pillars model stores telemetry.

Each type of signal is shipped off and stored in its own separate silo. Metrics go to a time-series database (TSDB), logs go to a log-aggregation service, and traces go to a tracing service. If the code generates exceptions, errors, profiling data, or any other signals, each would be stored in its own silo.

To illustrate the point, consider a simple web checkout request, like this:

```
GET /api/checkout trace_id=7f3a9c user_id=u_8821 200 OK 142ms
```

It would generate at least three distinct sets of telemetry data: metrics, logs, and a trace, as shown in Figures 1-2, 1-3, and 1-4, respectively.

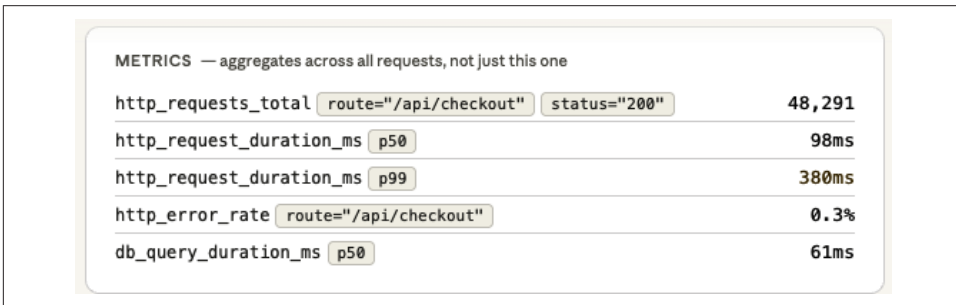


Figure 1-2. The metrics don’t know anything about the specific request. They store counters and percentiles aggregated across all requests. They’re useful for knowing something is wrong, but not for understanding “why” or “for whom.”

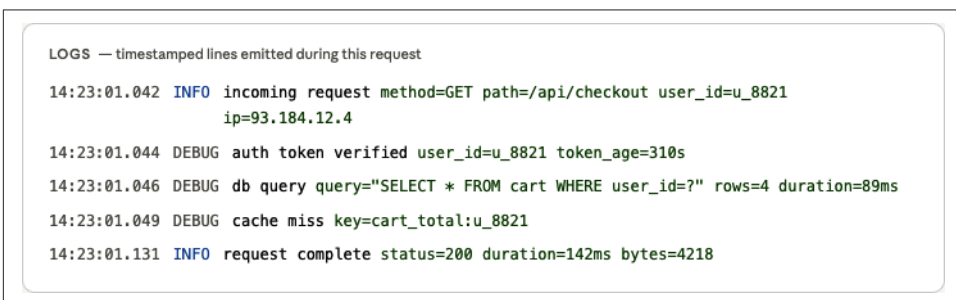


Figure 1-3. The logs tell you what happened in sequence, and you can see individual field values like `user_id` and `duration`. But to answer “what was slow?” you’d have to mentally reconstruct the timeline from the timestamps.

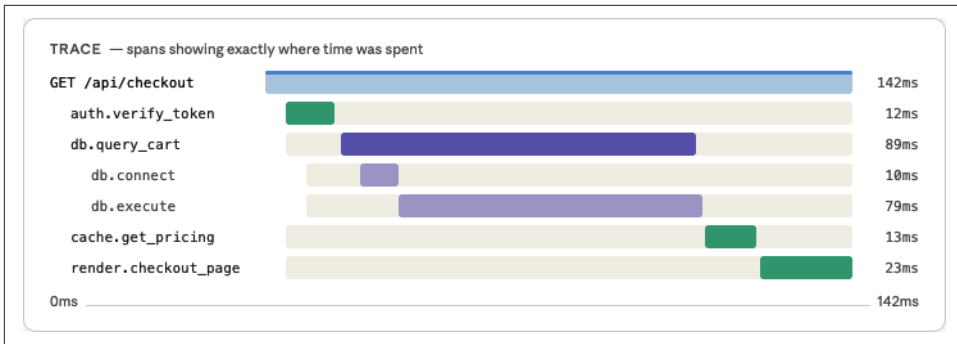


Figure 1-4. The trace answers that question instantly. You can see that `db.query_start` consumed 89 milliseconds of the 142 milliseconds, and most of that was in `db.execute`. You can see where most of the time went (into `db.execute`). Because that shares a `trace_id` with the logs, a unified tool can join them.

Most of the software engineers in this industry have never used anything but the three pillars approach and its silos. Why? Because infrastructure and operations have always owned the tooling, and the three pillars suit *their* needs.

If you work in infrastructure, you likely operate hundreds or thousands of components that you did not write and cannot change. Third-party code is going to emit what it wants to emit; you get little say. You just have to put it somewhere and make the best of it.

That's infrastructure, and it generates a roaring stream of logs and metrics at all times. The obvious solution is still the right solution: store the metrics in a metrics store, and store the logs in a logging store. And preferably cheap—these signals have *some* value, but not much.

That's the infrastructure problem. The development problem is different.

For the code that your team actually writes—the services that define your company, your competitive differentiators, your product—you have complete control over what gets emitted, how it's structured, and where it goes.

You are not stuck taking whatever the code emits and making the best of it. You can instrument deliberately. You can make good decisions about what context to preserve, which relationships to maintain, and how to organize your telemetry to preserve optionality for your future self.

And it unlocks tremendous value if you do so.



For the rest of this book, we'll use “traditional” or “legacy observability” to describe these older practices. We'll use “three pillars model” or “multiple pillars model” when we are explicitly referring to the practice of siloing off signal types, and “modern observability” or “unified model” for the newer approach.³

We'll treat observability as a spectrum: the traditional, three pillars style produces a low degree of observability, while the approach we advocate produces a high degree.

For the most part, teams haven't been doing that. They've been handed ops tools and told to make do—applying an infrastructure model designed for code you don't control to the code that matters most. Metrics and logs made sense for the router. They're a significant mismatch for the billing service.

This matters because if the true cost of software is measured by the cost of maintaining it, telemetry is probably the most powerful leverage point you have. Telemetry is your code talking back to you from production, telling you what's happening. The decisions you make about how to emit, store, and analyze that data are not infrastructure decisions but *product decisions*. Most teams have been making them by default, not by design.

The first tectonic plate, in short, is a set of tools built for operators and inherited by engineers. But it has accumulated enough ecosystem gravity that it remains the default—so deeply embedded in team structures, vendor relationships, and the ergonomics of muscle memory that most organizations never question it.

The Unified Storage Model

The second tectonic plate, the unified storage model, is the newer way of storing telemetry, derived from first principles, as shown in [Figure 1-5](#). An example is shown in [Figure 1-6](#).

All the telemetry that describes a request is stored once, preserving as much context and many relationships as possible, either in arbitrarily wide, structured log events, called “canonical logs,” or as spans and traces.

³ We have tried using “observability 1.0” for the three pillars and “observability 2.0” for wide, structured, traceable log events. This *also* ruffled many feathers, so we stopped. We are not trying to offend or cast aspersions on other tools or other vendors; indeed, we are trying to *not* offend. The three pillars model is the right model for third-party code and infrastructure use cases, just like the traceable, context-rich, high-cardinality model is right for code you develop in-house. We have never heard a vendor argue that this is untrue, only that it's rude to say so. Our goal is to be precise, not rude. But our ultimate loyalty must be to the reader.

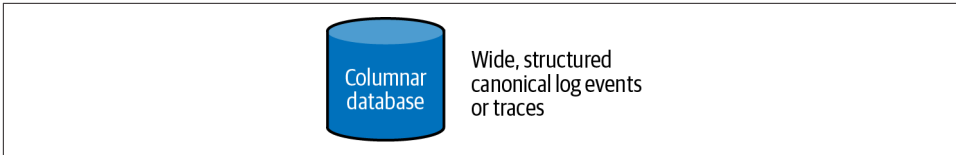


Figure 1-5. How the unified storage model stores data.

```
{
  // when
  "timestamp": "2024-11-14T14:23:01.042Z",
  "duration_ms": 142,

  // the request
  "http.method": "GET",
  "http.route": "/api/checkout",
  "http.status_code": 200,
  "http.client_ip": "93.184.12.4",
  "http.user_agent": "Mozilla/5.0 (iPhone; CPU iPhone OS 17_0)",
  "http.response_bytes": 4218,

  // the user
  "user.id": "u_8821",
  "user.plan": "pro",
  "user.country": "FI",
  "user.device_type": "mobile",

  // the app context
  "feature_flag.new_checkout": "true",
  "cart.item_count": 4,
  "cart.total_usd": 183.40,
  "cache.hit": "false",

  // where time went
  "db.query_duration_ms": 89,
  "db.rows_returned": 4,
  "auth.duration_ms": 12,
  "render.duration_ms": 23,

  // deployment context
  "service.name": "checkout-api",
  "deploy.build_id": "b_4471" // <— the exact build that served this
  "trace.trace_id": "7f3a9c",
  "trace.span_id": "1a2b3c",
}
```

Figure 1-6. An example of one wide, traceable structured log event with 24 dimensions. Note how rich context plus high-cardinality data gives you the ability to pick out any arbitrary subset of requests with extremely high precision.

These principles are worth naming explicitly, because they represent a genuinely different set of assumptions about what telemetry is for:

Data is made valuable by context, and that value increases combinatorially, not linearly.

Each additional piece of context you append doesn't give you one more thing to query; it makes the entire dataset exponentially more valuable⁴ by giving you new combinations with every field that already exists. The relationships between fields are the most powerful part of the data. Those relationships should be kept as intact as possible, not fragmented across silos. The technical term for this is "high dimensionality."

Details are the most identifying part of any dataset.

High cardinality—user IDs, trace IDs, build IDs, session IDs, any field that can take millions of distinct values—is a superpower, not a problem to be engineered away because metrics systems can't handle it. The things that make each request unique are precisely the things that let you find it later.

Pre-aggregation is a one-way trip.

If you aggregate your telemetry at write time, you can never un-aggregate it. The bucket is permanent. If you store raw events or traces, you can derive any aggregate on the fly—including ones you haven't thought to define yet. The cost of pre-aggregation is the permanent destruction of future investigative options.

You don't know what you don't know.

Preserve optionality for your future self by storing trace data or wide, structured log events (also called "canonical logs") in raw form, so you can aggregate at query time. Any schemas, indexes, or aggregates at write time create dead ends. A telemetry system that forces you to define your questions in advance is one where you will only ever find what you expected to look for.

Static dashboards have limited value.

What you need is flexibility and explorability: the ability to slice and dice, zoom from aggregate to individual request and back, and follow a trail of breadcrumbs wherever it leads. An investigation should feel like being in conversation with your data, but this requires preserving the relationships between datapoints. A static dashboard is a stock phrase; a dynamic interface is a conversation.

Fast feedback loops are everything.

Queries that take minutes break your train of thought. If you can't iterate rapidly—asking a question, getting an answer in seconds, asking the next—then you're not exploring; you're filing tickets.

⁴ If you have a wide, structured event or a trace that has 29 attributes, and you add a 30th attribute, it adds as much value as the entire preceding 29 attributes combined.

We'll go deeper on this history in [Chapter 3](#) (and the rest of this book). For now, it's worth noting: we didn't invent these principles and we weren't the only ones to have this idea. Rich, structured data backed by a columnar store that supports high cardinality and interactive exploration is how business intelligence (BI) software has worked for at least 20 years. Marketing teams run funnel analysis, cohort comparisons, and ad hoc exploration against billions of events as a matter of routine.

Imagine trying to run your sales pipeline off predefined buckets of metrics, and you'll get a sense of just how much software engineers have been fumbling around in the dark, using infrastructure tools to answer questions those tools were never designed or priced for.

Dimensionality

"Dimensionality" refers to the number of dimensions, features, or independent attributes a dataset possesses. It is a measure of the complexity and size of the data in mathematics and machine learning.

In the context of observability, dimensionality refers to the number of unique fields in your structured log events or traces. High-dimensionality datasets are critical for observability (and for AI) because it is a way of measuring the richness of your context.

If a structured log has four dimensions or attributes, you can combine them in 15 possible ways. The math escalates quickly:

4 fields

6 pairwise combos, 15 possible combinations

8 fields

28 pairwise combos, 255 possible combinations

30 fields

435 pairwise combos, 1.1 billion possible combinations

This is why the three pillars model is so costly. When you scatter signals across siloed tools by type, you don't just lose *some* of the value; *you lose most of it*.

To be fair, those BI tools used to cost a king's ransom, but that's no longer an excuse, and it's no longer the only option.

These two plates are not compatible. You can't store your data in one place and also in three places. They represent fundamentally different assumptions about what telemetry is for, how it should be stored, and what questions it should be able to answer.

Understanding which plate you are resting on (and what it would take to move) is one of the most important questions facing engineering organizations today, as they grapple with a surge of AI-generated code and get ready to support agentic workflows.

Observability Is the Validation of Developer Intent

For the past decade, the argument for observability has mostly centered on developer productivity and incident response. Both real, both valuable. And both, it turns out, are vastly understated as framings for what's actually at stake.

The entire software development lifecycle (SDLC) is being radically upended at this very moment. When agents can write, test, review, and ship code faster than people can, verifying that code becomes the bottleneck.

Telemetry is what's left. It's the last line of defense, the safety mechanism for the whole collapsed and compressed lifecycle.

The field of software engineering has a long and storied tradition of treating production systems like a foreign land, one they may travel to when called on to fix a gnarly bug, but not their preferred locale. If a bug makes it to production, as goes this line of thought, a developer must have failed in some way.

This has always been wrong. The line between bug and behavior is not always clear. What code will do in the chaos of production is not always knowable. By introducing nondeterminism as a feature, not an emergent property, AI shatters these fictions for good.

Only production is production. Test in prod, or live a lie.

The Agentic Incursion Has Just Begun

What's coming soon for most (and already here for many) are agents running in production. Not just writing code, but also testing it, deploying it, running it, observing it, and acting on what they find. (See [Chapter 10](#) for more on the topic of agents and observability.)

Agentic investigations are not deterministic. They change over time as they learn new things. Agents put the lie to the idea that preproduction testing can ever be sufficient. The *only* way to learn what an agent will do is by instrumenting it and observing what it does in the real world.

All the wisdom and connective tissue that senior engineers carry around in their heads—check latency by build ID after a risky deploy, watch cache behavior for days after a schema migration, look for N+1 query patterns when billing jobs slow down—needs to be encoded into the system, or it doesn't exist.

Agents running in production can only be as good as the data they have access to. Agents need context, relationships, and a diversity of sources. They need some ontological grounding via semantic conventions, and data with enough precision detail to track subtle changes over time. Most of all, they need us to think critically about the intuitive leaps we make and convert them into breadcrumbs of context, cardinality, and other clues.

High degrees of observability gives agents a fast, exploratory feedback loop with real, precise signals. Is this build ID showing elevated latency on this endpoint? Is this feature flag variant producing different error rates than the control? Is this user cohort experiencing something the rest aren't? The three pillars require intuitive leaps to move between pillars. Unified, high-cardinality, and context-rich telemetry do not require such leaps.

Guardrails Are Having a Moment

This is an exciting time to be in software. There's a necessary and important conversation happening right now about which guardrails you need to test in production in order to validate developer intent safely and dial it up incrementally.

Feature flags, progressive delivery, canary deployments, SLOs, characterization tests, automated rollbacks, etc.—these techniques are not new. They've been available for years (more on this in [Chapter 2](#)). Everybody agrees it would be great to have them, yet most organizations do not have them. Implementing these tools has been time-consuming and hard, putting them well out of reach for the median engineering organization.

AI changes this calculation in two directions at once. It is an enabler—it makes many of these things easier, including auto-instrumentation, anomaly detection, and the kind of systematic validation that used to require dedicated platform teams. And it is a forcing function—your competitors are using AI to build products and features at unprecedented speeds, and the pace of change is not going to slow down to wait for you. If you want to go fast, you need guardrails. If you want to cross over from handcrafted, artisanal code to the software factory, that's how you get there.

This is our opportunity to bring the engineering values we have long endorsed to the mainstream—ownership, agency, a blending of disciplines. But none of it is possible unless you can trace your intent into production and validate precisely what it is doing.

Conclusion

Although observability has been around for half a century, observability in software systems is a recent adaptation. For software systems, observability is a property of dependability: it measures your ability to understand or debug any given system state, represented on a continuum.

Modern systems are so complex, widely distributed, and resilient that mild degradations of service (and other unintended consequences of intentional change) are harder than ever to identify and diagnose. Many leaders seem to be pinning their hopes on AI SRE and agentic solutions to save the day. The extent to which this will work is not yet known, but our track record with “magic” is not outstanding.

The three pillars approach (logs, metrics, and traces) works well for infrastructure, but not for software development use cases. The pillars lack precision and depth. Engineers have always bridged that gap with intuition and acts of heroism, but agents can’t do that.

Storing all the signals together and widening the data is a technological and social step change. It solves the problems of correlation and cardinality that have plagued the three pillars and makes it possible to understand or debug any system state, regardless of complexity, at a much more efficient price point.

In the next chapter, we will talk about how to build confidence in your code as it crosses over to production.

How Code Crosses Over: Validating Developer Intent in Production

Developers develop preproduction, operators operate postproduction, and deploys mark the handoff point with a switch. This groove is etched so deeply that many companies don't even let developers *look* at their code in production. At other companies, it's just so awkward and cumbersome and difficult, and the payoff so minor, that they don't even try.

The dev/ops divide has persisted because it reflects both the need for change and the need for stability, in a way that maps neatly to org charts, budgets, and goals. But AI is collapsing the distance between writing code and understanding what it does in production—indeed, AI is forcing all the stages of the SDLC to blur and combine and be reinvented. An agent doesn't know if it's designing, testing, or coding. What matters is intent, context, and validation.

When code crosses over, it must be validated—not just in theory, with testing, but in reality. There are many techniques for validating code in production. These are not new. But they have never been widely adopted, because they are complicated and deploying them is hard. AI is challenging this from many sides: by making it technologically easier to achieve, by applying competitive pressure, and by upending the economics of code production.

This chapter covers the practices that make that possible. They form a flywheel: each one enables the next, and together they accelerate and amplify each other's capabilities. But all must be grounded in precise, context-rich observability.

We stand at the precipice of a new era in software development. In the past, our code was handcrafted; in the future, it will be generated. That much seems clear. How we get there seems far less clear. How does code cross over? There are tools, practices, and principles that can help, and they run through production.

We'll start with a question that sounds simple but isn't: what makes software good?

What Makes Software Good?

The quality of software is inseparable from what the software is meant to do. This might seem obvious, but it's easy to lose sight of.

Should you be writing code using AI as fancy autocomplete or should you be orchestrating fleets of agents? Is Rust or Go a better language? These questions are unanswerable in the abstract, but they are answerable if you begin with “for what purpose?” followed by “under which constraints?”

Good Software Serves Its Purpose

Software runs the world these days, so “purpose” has enormous range and variability.

The same goes for software practices—they are rarely good or bad; they simply serve a purpose. So, should we stop doing code reviews? Should we put developers on call? The answer is always, “It depends.” What are you trying to achieve? Who is your user, who are your builders, and how does your goal serve them?

It would be a waste of time to worry about writing fault-free code if you are a seed-stage startup in search of product market fit, but it would be the most important consideration of all if you're working on the Mars rover or a nuclear launchpad.

There are many attributes of purpose, but the time horizon you are building for is worth calling out separately, because this is where the cost of software is set, and it has implications for every other decision you make.

Good Software Delivers Efficiently Over Time

The cost of software has always been defined by the cost of maintaining it, not the cost of developing it. When people say, “It was never about the code,” this is often what they mean.

Software relied upon must be maintained. Libraries need upgrades. Bugs need fixing. Features must be added and scaling limits increased.

How much do those changes cost in time, effort, and user impact? The higher that cost, the more expensive and less efficient your software becomes—measured in missed opportunities and lost revenue, morale, brand trust, and customers.

In the past, “good code” meant elegant code. When human cognition is a gate for every change, readability is a good proxy metric for code that will be less expensive to extend and maintain over time.

To sum up: good software is software that fulfills its purpose, for as long as it’s needed, as efficiently as possible.

The Maintenance Horizon: Disposable Code Versus Durable Code

The difference between disposable and durable code is *not* about whether the code was generated with AI or written by hand. The difference is in how they are changed, which is derived from the maintenance model.

Writing software has historically been extremely difficult and time-consuming. Because of this, we have treated code in production like a sacred relic—unlike immutable infrastructure, which we happily destroy and regenerate all day long. That regeneration is possible because we understand it: what “good enough” looks like and what commitments it must deliver. If we understand it, we can regenerate it; regeneration is proof. This has liberated generations of developers from having to understand or think about how infrastructure is implemented.

We have not yet made the shift from durable code to durable *systems* with regenerable code, although the economics of code abundance would seem to make this an inevitability, at least for some workloads (as discussed in [Chapter 32](#)). What we have for now are the durable model for change management and the disposable model for change management.

Durable Code Is a Model for Mutating and Maintaining Code in Place

The durable code model prioritizes stability and continuity over all other considerations. It does this because that is actually the fastest, cheapest way to change a complex system where we don’t have any record of developer intent, user expectations, or dependency commitments. All of these things are bound up in the artifact of code.

We do not know which of our changes will be disruptive, or who or what it will disrupt. Mostly, we find these things out when we break them.

The most reliable way to change durable code is by applying a series of small, contained diffs to a baseline that is assumed to be good. Small batch changes shrink the blast radius when things go wrong.

“There is no test you can run that would let me trust a new snippet of code more than the code already running in production.” This is a truism where durable code is concerned. Currently running code is a known quantity—or at least, a *stable* quantity

composed of many unknown commitments. And there are plenty of situations where returning a predictable, consistent result is more important than returning the “right” or “best” result.

This is often a function of dependencies. No user wants their interface to change in subtle ways throughout the day, or for buttons to pop up in random places that are inconsistent with the mental model for the product. User expectations have a change budget that is smaller than you might think. Persistence problems can be similarly sticky. (It is no coincidence that both of these are about reconciling at the point where bits meet atoms.)

Durable code prioritizes stability over other considerations, but that comes at a cost. Maintenance is expensive. Every change is gated on understanding. If you don’t have to maintain code as durable code, you really probably shouldn’t.

Disposable Code Is a Model for Avoiding Maintenance Costs

If the cost of software is defined by what it costs to maintain, disposable code is cheap because you don’t even *try* to maintain it. You aren’t trying to anticipate and account for how it will scale into next year, or sweat over its readability for future generations. You just build what you need *today*.

If you want to add a function, you add a sentence to the spec and generate it all over again. Generate 10,000 variants and let them fight it out, if you feel like it. It’s so cheap and easy, it feels like magic.

This doesn’t mean that disposable code has no quality requirements. Sometimes it doesn’t, but sometimes it does. The “you only live once” (YOLO) mode is fine for prototypes and experiments. But upgrades and migrations are some of the most exciting use cases for disposable code, and they definitely need tests and safety checks. Anything that runs in production code must meet a higher bar for quality.

But you’re not going to maintain that migration script; you’ll just generate a new one next time. Therefore, the cost of maintenance remains low.

The Maintenance Horizon Is Not Always Knowable Up Front

How good a piece of code needs to be isn’t always clear in advance. The delineation tends to emerge over time. When you are iterating rapidly on design mocks or prototypes, you never know which one is going to click. The “one-off quick hack” that becomes load-bearing over time is a long-running joke.

When disposable code graduates to durable, you should put it through the same process as you do any other durable code, whether that means test coverage, peer review, or something else. Always add instrumentation and observability, and monitoring or service level objective (SLO) coverage if needed.

There are many purposes that software can serve, and just as many definitions of “good software.” However, for software services running in production, there are some common standards of reliability, understandability, and maintainability that are commonly thought of as “production quality.” Another term for this is dependability.

Production Quality Code Is a Function of Dependability

In **Chapter 1**, we discussed software dependability—a set of properties we typically associate with running highly available services in production.

When code graduates to production, it begins to accrue commitments. Its behaviors turn into expectations, intended or not, and user expectations should be honored. If we fail to meet expectations, we break user trust. This is why we have a higher bar for production code.

However, we cannot care about all software equally. How do we know where to spend our focus?

Identifying the Critical Path

Not all software has to be good, but the software in your critical path probably does. Start by writing down what you think is in (or out) of the critical path. Does everyone on your team agree? What about the adjacent teams?

Ask yourself: what happens if this code breaks? If the answer is “not much,” then the code can be a little sloppy. If the answer is “we could be in *The New York Times* tomorrow,” or “we lose \$20 million in high-frequency trading fees for every minute of delay when the markets open,” or “someone could die,” then this code needs to be good.

You may be tempted to make everything beautiful, but that’s a waste of time. Some code is like the local post office: solid and functional, nothing fancy. And some code is like a cathedral: inviting the awed stares of passing tourists and built to stand for centuries. (OK, maybe not quite, but the metaphor stands.)

Frequency, criticality, visibility, and blast radius are your compass for identifying the software in your critical path:

- If the code runs rarely or invisibly, or if breaking it will have little impact on the business: that code doesn’t matter. It’s just scaffolding.
- If the code runs rarely and invisibly, but handles core business functions like payroll or security: it needs to be solid, “post office” caliber code.
- If the code runs constantly, it carries the business, and users are acutely sensitive to its performance characteristics: this is your cathedral.



Important Caveat

Any code with a blast radius that could take down critical code is *also* critical code.¹ Tools like isolation and modularization are critical for reducing blast radius.

Once software sits in your value stream—once other people and functions depend on it—it stops being a toy and becomes your business. This can happen accidentally as well as on purpose, so it's worth auditing this list occasionally.

The critical path is not static; it changes often. The hard part is noticing when new dependencies have crept in, or identifying when a piece of throwaway code is now load-bearing. Mind the gaps.

The Closer You Get to Persisting Data, the More Cautious You Should Be

Here's one (unscientific) rule of thumb: the closer you get to writing bits out to disk, the more paranoid you should be about making changes.

The code in databases, operating systems, filesystems, and medical devices are likely to use the durable code model for the foreseeable future. It may never be possible to extract and define all the commitments, dependencies, and behaviors outside of the code itself, or they may be too tightly coupled to low-level hardware characteristics.

As anyone who's ever done a major database version upgrade on a busy production database knows, even very subtle changes can compound to unpredictable results at the application layer.

The further up the stack, the more forgiving mistakes can be. You can redeploy, roll back, or fix it in post. But persistence problems have a nasty little habit of being permanent. Mistakes in database migrations, schema changes, and storage layouts can be irreversible. You can't un-corrupt a database table. You can't easily undo a schema change that millions of records have been written against.

This isn't a reason to avoid AI assistance in these areas; it's just a reason to apply it differently—with more human understanding, more conservatively staged rollout, and more observability before, during, and after any changes to the persistence layer.

¹ Who among us has not been taken out by a rogue cron job? Let them cast the first stone.

Development to Production: Tools for Crossing Over

The dev/ops divide that locates development on the left and operations on the right, divided in the middle with a binary switch, is unhelpful for a number of reasons. Deploys in complex distributed systems are not binary; they are a process of building confidence over time.

Code crosses over boundaries all the time. The crossover from dev to prod likely happens every day. It should be boring.² And it can be boring—with the right tools.

The following practices form a sequence: your “test-in-prod” toolkit. Each one unlocks the next. Together, they help your code safely graduate into production—the phase where code meets reality, and it either serves its purpose and begins to earn your trust or it doesn’t.

Most people who have tried these tools have gotten mediocre results. Not because the tools don’t work (they do), but because they were running them on three pillars-style observability: pre-aggregated metrics, logs that don’t correlate with metrics, and traces that don’t correlate with anything. The tools are only as good as the signal underneath them. This is why the first practice in this chapter is a prerequisite for good results.

Practice 0: Give Yourself the Gift of Rich Data and Precision Tooling

In ecology, a keystone species is one whose impact on the ecosystem is disproportionate to its abundance. Remove it and the ecosystem doesn’t just degrade—it collapses into something fundamentally different. Modern observability is the keystone practice of the software delivery ecosystem. (We will talk about this in much greater depth in Chapters 24 and 25.)

Every other practice on this list is dramatically more valuable with high-quality observability. Feature flags are more useful when you can slice and dice by any arbitrary subset of flags before exploring the results. Canary deployments are only useful if you can compare the canary requests against the baseline. Automated rollbacks are only safe if the signal triggering them is precise and trustworthy.

This is why it’s practice 0, not practice 7. Observability is the foundation, not the cherry on top. If you build these practices on top of aggregated metrics and traditional logs, you too will get mediocre results and conclude that the tools don’t work.

² The more difficult crossovers are from one production system to another production system, under heavy load.

The tools work. But they rely on rich signals: high-cardinality, wide, structured events, and telemetry that preserves the relationships between fields rather than destroying them at write time. You also need to be able to select the signal with scalpel-like precision: aggregate at read time, select any arbitrary subset of fields, compare any subset against another subset, handle high-cardinality fields, etc. This means a columnar store, and the rest of the criteria we described in [Chapter 1](#).

Alternative options

We get it: most developers don't actually get a say in which tools are available to them. And the overwhelming majority of teams do not have access to modern observability (at least not yet). So, here are some hacky ways to approximate the effects.

If you're doing agentic development on a local dev environment, consider stashing traces in DuckDB, an open source columnar database, or in Apache Iceberg.

If you are locked into a multiyear contract with a three pillars vendor and you can't get out of it, there are ways to start improving your application telemetry immediately.

Start by standardizing on *either* logs *or* traces. Not both.

If traceability is not a top pain point for you, logs are the better choice. Follow the principles of canonical logs—wide, structured log events, organized around each unit of work or service hop. Make them as wide as you can, preserving as many bits of context in attributes as possible. Propagate important context along from service to service (you can do rough traces this way too).

If tracing is already a pain point, standardize on traces, and then load them up with context in much the same way.

Data is made valuable by context—it grows exponentially, combinatorially more valuable as you widen the dataset. Make it as valuable and as rich as your tools will allow you.

Practice 1: Build a Feedback Loop Between Developers and Production

Given the durable code model that exists today, the best practice we have for capturing and validating developer intent is as follows:

- Instrument your code as you write or generate it.
- Get your code into production immediately.
- Autodeploy from CI/CD if possible.

- Go and look at your changes in production, through the lens of the instrumentation you just wrote.
- Ask yourself, is it doing what you thought it would? Does anything else look... *weird* (technical term)?

It sounds faintly ridiculous, but this really is the gold standard for capturing and validating developer intent: “Look at it before you forget what you were trying to do.” Any other record of developer intent must be inferred from Slack threads, task trackers, documentation trails, etc. and is significantly less reliable.

We do not think this state of affairs will survive much longer; in order to collaborate with AI agents effectively, we must capture developer intent and encode it into the system in a way that is legible to agents.

It’s exciting to think about solving this problem. After all, having development and operations exist in two disjoint sets of heads, with no feedback loop connecting cause with effect (developer intent with production consequences), is exactly what has caused so much frustration, disconnection, and degradation in quality over the years.

Observability is what creates the feedback loop. The quality, speed, and precision of your observability sets the quality, accuracy, and usefulness of the feedback. If all you know is that your system was 99.99% before you deployed and is 99.99% after you deployed, you know nothing useful. Aggregates conceal a multitude of sins.

It has often been cumbersome and frustrating for developers to try and find their telemetry in traditional three pillars-style tools. Infrastructure tools typically rely heavily on dashboards, which must be constructed to surface changes in telemetry. AI is swiftly bringing down the cost of this tax. You should be able to get the telemetry you need, when you need it, served up on a platter in your development environment and/or Slack instance. MCP (Model Context Protocol) also lowers the barrier to entry by letting you explore and follow the trail of breadcrumbs without having to master the minutiae of an incredibly complicated set of buttons and knobs.

The most important thing to remember is this: the development process does not end when code is merged. “Development” is not *followed by* production. The work of development extends through the process of deploying and validating. You are not finished building until you know it works.

The interval immediately after code is born, when it first encounters the chaos of the real world, is one of the most important stages of development. It’s your first and best opportunity to spot any bugs or puzzling behaviors; after that, it will be up to your users. Most teams let this moment slip by, unmarked and unobserved. *Don’t.*

Practice 2: Test Your Code Before You Deploy It

We include this step just to be explicit. Yes, we are vocal advocates of testing in production—but as a complement to preproduction testing, not as a replacement.

A powerful test suite has never been more needed than it is now, in the era of generated code, despite its limitations. Many software engineers equate tests with correctness. We suggest the true purpose of tests is to prevent known classes of problems from recurring and give you enough confidence to begin validating in production. Those are narrower goals than most teams implicitly assume when they measure coverage.

The most valuable tests are the ones that protect things most likely to break in expensive ways:

Unit tests

For logic that is complex and easy to regress

Contract tests

Across team, org, or third-party boundaries

Integration tests

At the seams between services, where implicit contracts live

End-to-end tests

Less useful in practice than in theory—expensive to write and maintain, and flaky in practice (though SLOs, as the operational version of these, are typically more useful and robust)

A large suite of perfunctory tests is a maintenance burden and a time sink that adds little value. AI agents writing tests for code they just wrote will blithely apply the same assumptions to both. Do your tests validate developer intent, or merely implementation?

If your test suite passes, and your tests are exemplary, the confidence you have earned is that the mistakes you find in production will be *new mistakes*.

That's it. That's the bar. It's a surprisingly high one. To reach it, you will have to be disciplined about capturing novel errors, odd behaviors, and other surprises in production and wrapping them back into your test suite.

Production is where code grows up, where beautiful computing theory meets messy reality. In any contest between “how it should work” and “how it does work,” reality is likely to have the last word.

That is why we expect the test landscape to broaden over the next couple of years, to focus less obsessively on preproduction, stubbed-out tests, and to focus more on property-based testing, behavioral tests, and characterization tests—all of which

are less oriented around correctness and more around stability and continuity of expectations.

Practice 3: Instrument Your Code and Validate in Production

This continues to be a controversial statement, so let's get it out of the way: *everyone tests in production*. Even if you have 100% coverage of every known type of test, even if you test in staging—even if you spin up a perfect replica of your production infrastructure, capture 24 hours of production traffic, and replay it all on your replica—it will *still* be a test when you deploy to production, because every deploy is a unique and irreplacable intersection of events.

Acknowledging this fact is the first step toward doing it well.

Software engineers have historically lived in the world of code. But code is not the system. The system is the unique intersection of code, infrastructure, user behaviors, builders, operators, and data over time. The system matters most, not the code; the system is what users experience and what businesses rely on.

But code is uniquely powerful, because code can be instrumented. Instrumentation is how your code reports back to you from production. Good instrumentation reveals the system from the inside out, tracing the exact same route as your users do, revealing the world from their perspective. Yes, that is what we're saying: when done well, instrumentation is a bit like an endoscopy.

This is why we said in [Chapter 1](#) that *instrumentation is a product decision*, not an infrastructure decision. This is why instrumenting your own code is in a different category than infra logs and metrics. Logs and metrics are scanning from the outside, but instrumentation is running a little camera up and taking a look for yourself. Observability with high cardinality and rich context is having full control of the camera to zoom into nooks and crannies, zoom out to get the big picture, or move around to get a better look.

Look for any opportunity to orient yourself in the instrumentation. Stash the line number of the code that generated a trace span in a span attribute. Put the line number of the code that generated the query in a query comment...and the query comment as an attribute on your log or trace. Story query fingerprints, normalized queries, and query comments in attributes if you can.

The combination of OpenTelemetry (OTel) and large language models (LLMs) has transformed the experience of auto-instrumentation since we wrote the first edition. OTel has given us a wealth of reusable, well-documented patterns from open source software, which all LLMs are trained on. As a result, it is now genuinely easier to write code with instrumentation than without.

But instrumentation is only the beginning. What you do with it is validation. Your ability to validate your changes in production—or have AI agents do the validating for you—will be enabled (or constrained) by the context and cardinality of your data, and your ability to precisely select and compare the parts that matter to you.

There is a lot of talk right now about how to move the source of developer intent out of the code and into some kind of specification or architecture model, from which the code can be generated instead of the other way around. We love this. For now, instrumentation in production is the living spec, the record of what the system has been asked to do and how it responded, not in theory but in truth. Accept no substitute.

Practice 4: Decouple Deploys from Releases Using Feature Flags

Deploying code and releasing code are two different acts that serve different needs; feature flags are how you separate the two. If you fail to separate and treat them differently, you will serve at least one of them poorly—possibly both. The difference is this:

- A *deploy* is the process of building, testing, and rolling out changes to your production software.
- A *release* is the process of changing user experience in a meaningful way.

“Meaningful” is a subjective threshold. Most orgs allow bug fixes and small changes to ship continuously, but large changes are usually gated and tied to some other company goals or events. This may be to minimize change fatigue in users or to create marketing momentum via press releases or company events.

Feature flags for deploys

Deploys should be happening constantly; releases are more variable. Deploys are an engineering problem, while releases are governed by marketing and/or product. The tool that allows us to decouple these governance models and promote peace and harmony in the land is called “feature flags.” Flags can be very fancy and expensive, or they can be a table in a database with two simple columns, but they are a mandatory component of modern software systems for many good reasons. Let’s begin with engineering.

As we discussed, the safest way to change durable code is by issuing a steady stream of small, bounded changesets against a baseline that is known or assumed to be good. Big diffs are scary diffs, because we don’t actually understand the implicit contracts we have with our users. We also have no way of preserving and generating code from

developer intent, so the safest way to test it is to validate by hand before you forget what you were trying to do. Get each diff into production, stat.³



The best practice is to have CI/CD deploy *automatically* after stamping out an artifact—at least to a canary or internal production environment (something with real users). This is an ideal practice for multiple reasons: to isolate each set of changes, to make the interval as short as possible, and to make the interval predictable.

If you limit each deploy to one set of changes, you can establish expectations around ownership. Notify the developer when their changes are going out, and when the deploy is complete they can validate the results.

Batched-up changes are a recipe for breakage. If you're bundling up changes from 20 different people to go out at once, it's 20 times as likely that one of the changes will break and they will all have to get rolled back and debugged.

Developers should be able to treat merges and deploys like an atomic action. If they merge, their code is on the release train and will go out in a set amount of time by default. If they don't want it to go out, *don't merge*.

Feature flags for releases

Releases, on the other hand, are usually owned by product or marketing. Sometimes a new feature needs to be turned on at *precisely* 10 a.m., timed to line up with an important announcement or TV spot.

When you release something, you want it to have already been tried in production. You do *not* want to be kicking off an hour-long deploy process at 9 a.m. and nervously hoping it doesn't fail and roll back for some flaky reason. You want to flip a flag at precisely 10:00:00 and have it immediately roll out globally.

That is the basic separation of duties: deploys for engineering, and releases for product or marketing. But engineering has many, many other uses for feature flags.

Advanced uses

Using feature flags, you can land diffs and deploy them even when they are nowhere near ready for public consumption, which enables you to validate as you go instead of in one big bang. You can grant access to a set of internal users, or let customers opt themselves into experimental features. You can define sets of users by region, pricing

³ At Honeycomb, we have a sticker that says, “Undeployed code ages like fine milk.”

plan, language, or any other characteristic, and use those inputs to define what they can access. You can deploy to 1% of Brazilian users first, or a representative, randomized sample set from each time zone around the world, and progressively increase those percentages as your confidence increases.

Best of all, you can disable access to code, features, products, or landing pages *instantly* when something goes wrong, instead of needing to land a diff, build an artifact, and deploy a second time, or roll the dice on some infrequently exercised rollback path.

Feature flags are the foundational primitive for testing—*safely*—in production. Flags are not new. But managing flags and using them extensively can get a bit complicated and fiddly. They have been a heavy lift for some engineering teams due to the overhead of implementation and immaturity of tooling, but AI has brought the cost down significantly. Rolling out flags and maintaining them is a lot less labor-intensive these days.

The combination of modern observability and feature flags is worth more than the sum of their parts. Indeed, this is where the amplification effects of the flywheel really kick in. Your ability to target precise and controllable slices of production with feature flags, combined with the ability to pick out the same precise and controlled slices of telemetry and compare them against the baseline *or any other slice*—this is where things start to get fun.

But we're not finished yet. There is one more practice that should come standard in every production toolbox: progressive deployments.

Practice 5: Invest in Progressive Delivery, Canaries, Automated Rollbacks

The flywheel closes here. You have context-rich, combinatorially powerful data that you can slice and dice or visualize over time as a trace, with no dead ends. You auto-instrument your code via OTel patterns as you generate or develop the code, and receive notifications in your development environment when something is off. You test your code before you merge, then test again after it's been deployed, using the MCP to explore and look for patterns or outliers as well as validating your intent. You have separated deploys and releases using feature flags.

The only thing left to do is implement some form of progressive delivery, health checks, and automated rollbacks.

There are many variants of progressive delivery, but the three most common are:

Blue-green deployments

Where you spin up a copy of a service and flip traffic over at the router or load-balancer level, and then drain and decommission the older set

Canary deployments

Where you deploy to a single node or small slice of users, validate that it works, and then deploy to everyone

Rolling deployments

Where you replace instances one at a time or in small batches

Each variant has trade-offs in terms of speed, cost, complication, and ability to roll back. For the purposes of this section, we'll assume you use rolling deploys.

It is absolutely the case that feature flags and rolling deploys can get complicated. There's a reason companies have historically hesitated to invest in these guardrails until they are large enough to have a dedicated release engineering team (or teams, plural, or even a release department).

But there is an equally important reason why *every company at scale has this*. At some point, you have to give up the grip of relying on individual heroics to validate code in production, and this is the way you do that. Fortunately, for those of us who do not work at the scale of Google or Meta, AI has brought the cost and complexity down within reach. If you plan on setting AI agents loose in production, this is the flywheel you need to know about.



When we were at Facebook in the early 2010s, the process of rolling a change out to production took all week. Hundreds of diffs were in flight at any given time.

A developer would land a diff, which would get built, tested, then spun up in a container version of production, and tested again; this process would repeat through multiple sandboxes and staging environments. If the diff survived, it would go out to intern.facebook.com (the site employees used for work) and be left to bake for a week.

When the diff was ready to roll out to production, it would be shipped to 1% of a particular country (which shall remain nameless) and monitored for a while, and then the percentage got slowly dialed up. Over the next day or two, other regions would receive their own canaries and their percentages would get dialed up.

The new binary was subjected to a barrage of health and wellness checks at each step. If it failed any health checks, it would automatically revert and bounce back to the developer who submitted it.

We have heard from many people who report having disappointing experiences with canaries and rolling deploys. A representative story:

“We deployed to 1% of traffic, and then checked our dashboards. We made a special dashboard that showed the canary requests compared to the rest of prod as a baseline.

Everything looked fine. Latency was normal. Error rates were normal. Everything looked normal. So we manually approved and rolled it out to the rest.

Everything looked fine...and it was fine, until we hit 70% and every request started queueing up trying to hit the same row lock on the database. What good are canaries if they can't even catch most problems?"

Yeah, this story comes in many guises. There are the error rates too small to pick out of the noise, threshold effects, canary sample biases, aggregates that bucket or smooth over outliers, cascading failures, thundering herds and retry rates, contention on resources upstream or downstream, a structurally unrepresentative sample, or problems that don't percolate up to the metrics you're watching, such as data corruption.

You cannot predict everything that will happen in production. But there are two powerful techniques you can use to improve the robustness of your deploys over time.

The first is the same thing you do with preproduction tests: set the bar at "*make new mistakes*." Any time you experience a failure, wrap it back into your suite of automated health checks. Never fail the same way twice. It's a higher bar than it sounds like.

The other thing you can do is validate developer intent. Production is a very, very noisy place, and if you're relying on your ability to spot a problem visually on a dashboard or relying on SLOs to catch it, or any other *a priori* detection method, you are only going to spot the really big problems—the ones that pose a threat to the operational health of the system at large. You won't spot the subtle ones or the behavioral shifts.

Modern observability changes this. When you have precision tooling and rich datasets, with lots of context and high cardinality, *and you know what you're looking for*, you can find it. Even if it's only a handful of requests against a noisy background of tens of millions.

When you can compare arbitrary cohorts—users on the new build versus users on the old build, filtered to a specific endpoint, broken down by user tier—you can detect a regression in 1% of traffic. You can see that the new build is 40% slower for users on the enterprise plan hitting the billing endpoint. That signal is available at 1%. By the time it's moving aggregate metrics, it's already affecting a lot of people.

This can be tedious for humans to check by hand after every deploy, many times a day, every time a diff lands. But AI agents? They don't mind at all.

The last piece of the flywheel is following each change into production, validating it, and doing a rollback if any of your automated checks are violated. Automated rollback logic and escalations are the safety net. Of course, a rollback is only trustworthy when the signal that triggers it is trustworthy. Which brings us back to practice 0—because flywheels, naturally, are circular.

Bonus Practices: Traffic Splitters, Capture/Replay, Strangler Figs

Finally, here are a few special-purpose tools we'd like to put on your radar. They are exceptionally valuable tools to have around, as a complement to your characterization and property testing plans:

Capture/replay

This is the gold standard for “as close to prod as you can get without being prod” testing. These mostly get wheeled out for sticky database upgrades, but they have other uses.

You might take a snapshot of the database, then capture 24 hours of production query traffic, then restore the snapshot, warm it up, and replay the stored queries against the snapshot under a variety of conditions (different versions, hardware, concurrencies, as-fast-as-possible versus by-original-timestamps, etc.), and reset the snapshot between each run.

Traffic splitters

Using a traffic splitter is a common, if hacky, trick for diffing requests over the wire. If you are trying to rewrite an API server, you might set up a traffic splitter in front of the old API and new API, fork the request and send to both, and compare and diff the responses. Then, return the old API response to the user and drop the new API response on the floor. Write any differences out to a log.⁴

Strangler fig

This term originated and was popularized by [Martin Fowler](#). It is not a tool, but a technique for rewriting and migrating legacy codebases. This involves building transitional architecture around the legacy code, defining interfaces, extracting parts, linking to the old code and the new, decoupling data sources logically, sending writes to the new source, backfilling data, switching reads to the new source, and finally deprecating the old code.

Sounds complicated, doesn't it? It is. But all production code will eventually become legacy code. Building components with good, observable seams from the start is one of the best gifts you can give your future self.

Observability Is the Feedback Loop of Feedback Loops

More than ever, observability plays a pivotal role in creating good software. Observability is the sense-making apparatus of production systems. If fast feedback loops are the key to developer productivity, observability is the meta feedback loop—the feedback loop of feedback loops.

⁴ That works for reads. For writes, of course, you would need to put a replica of the database behind NewAPI.

At every step—whether you’re landing a diff, running a test suite, deploying to a canary, flipping a feature flag, reverting a changeset, etc.—your ability to swiftly, precisely identify the state of reality is what grounds you. There are many ways to construct and optimize these feedback loops, but the speed, precision, and accuracy of your observability will determine your ability to understand and act.

Observability alone cannot solve problems of collaboration, skill depth, team velocity, business alignment, or creativity. But it is a critical component of most of the answers to many of these problems, in ways that are often underappreciated.

The Work of Development Is Not Done Until It’s Working in Production

One of the most overused metaphors in software development is referring to a project as “changing the engines out of a plane in midair.” But it gets repeated for a reason. That really is what it feels like any time you do a complicated rewrite or migration or an upgrade.

The shift from handcrafted servers to immutable infrastructure taught us that mutability is the enemy of understanding. The durable code artifacts we have today have been accruing meaning, intent, expectations, and commitments for as long as they’ve been in production. That’s why rewriting or replacing them is so painful—we literally do not know what promises our system has made. The only way to detect them is slowly, carefully, leading with observability first at every step.

Practices 0 through 5 are the tools that every developer should have in their kit. Together, they give you the ability to validate your changes in production. The development stage does not end when you merge your code; it extends through the deploy process and crosses over into production for as long as it takes to gain confidence there.

You are not done building until you are confident it works. The only way to develop that confidence is to instrument your code and observe it in production.

Conclusion

These practices—feature flags, instrumentation, progressive deployments—aren’t new. Teams have been using them for years, with mediocre results; not because the practices were wrong, but because the underlying signal was weak and lacked precision. You can’t automate rollbacks on signals you don’t trust.

What’s changed is the cost. AI has made auto-instrumentation easier, feature flags simpler to manage, and progressive delivery more automated. The implementation overhead that kept these practices out of reach for most teams has dropped. And the cost of skipping them has never been higher: the rate of change is dialing up and the

human review is dialing back. Guardrails aren't optional in this world; they are how we make velocity safe.

The cost of software is defined by the cost of its maintenance. Any opportunity you have to *not* maintain and update code in production, you would be wise to take. As long as the expectations and commitments our software makes remain implicit, this will be the case.

The organizations that equip developers with these tools will also be prepared to let AI agents work autonomously in production, catch regressions early, and ship with confidence. The ones that don't will flounder. It's never been easier to write and ship code, but it's as hard as ever to understand it.

Production is reality. Observability is how you understand it. Everything else follows.

In the next chapter, we move from the sociotechnical strategy of production ownership to the technical emergence of observability itself, distilling the lessons of the past to define the context-rich data structures required for a dependable reality.

The Origins of Observability in Software

In [Chapter 1](#), we described observability as a property of system dependability and the sense-making capability of complex systems. In [Chapter 2](#), we talked about how code crosses over from development to production, and how to safely test in prod.

This chapter is written by Charity Majors, in her voice and perspective.

It shares her retrospective on how observability entered the software industry, and how it has changed over time. The popular understanding has diverged from the specific, technical meaning it once had. This chapter is about recovering that meaning, and reasserting the principles that matter most.

If you want to get straight to instrumentation and the practical bits, you have Charity’s blessing to skip ahead.

An Introduction from Charity

Hi! My name is Charity, and I am the cofounder and CTO of [Honeycomb](#). I come from the infrastructure side of the house; I have been an operations engineer, site reliability engineer (SRE), and database reliability engineer, working on systems like Linden Lab, Parse, and Facebook, as both engineer and manager.

For 10 years now, I’ve had a front row seat to the industry’s attempts to make sense of our complex, confusing, interdependent web of software, and I’ve contributed some attempts of my own. It’s a bit ironic, because as an engineer I always *hated* monitoring and dashboards, but I am no stranger to rage-driven development (it’s actually kind of my thing), and I feel really lucky to be here.

Christine Yen—my Honeycomb cofounder—and I are often credited with inventing or pioneering “observability” in software engineering. This is a distinction that has

always made me mildly uncomfortable, in part because it's not entirely true,¹ and in part because the popular understanding is so different now.

But while Christine and I may not have been the first to use “observability” in the context of software, we were the first to give it a technical definition, inspired by its control theory meaning, and build a product to that specification.

When we first started exploring observability, all we knew was that we were trying to do something different. We had tried all the leading monitoring and APM tools, but they all suffered the same limitations: strict caps on cardinality to control costs, queries that took minutes to run, and correlations that remained stubbornly manual and guesswork. As engineers with experience in databases (me) and analytics (Christine), we knew these limitations were not intrinsic to data, and we were confused why all the vendors kept copying each other, iterating on a data model that worked so poorly.

We also knew we were building for software engineers, not infrastructure or operations teams. Infrastructure is and was well served by the three pillars. Software engineering was not, and still is not.

As generative AI has turned the cost model of software upside down, the question of how software engineers can validate their code in production, swiftly and reliably, has become more important than ever. Building may have gotten easier, but understanding the code we built has not gotten easier; if anything, it has gotten harder. The solution, we are told, is agentic AI, but agents are only as good as their data. What AI needs most—context, relationships, ontological grounding—are what the old model sacrificed first.

What happened? Why would the entire tech industry standardize around a set of “pillars” handed down from infrastructure? What can be done?

The Dominant Model of “Observability” Is Just Monitoring, Rebranded

Thinking back 20 years ago, every tech company had an operations department, and the ops folks ran everything: monitoring and logging, DNS & BIND, operating systems and kernels, filesystems and file servers, LDAP (lightweight directory access protocol) and Kerberos, datacenters and hardware, networking switches and routers, and on and on it goes.

¹ The genealogy of observability in software, as far as I can tell, begins with [Bryan Cantrill](#), who put together a deck called *Software Observability* in 2003 to “fancy it up a little, to try and convince the Sun managers to keep funding DTrace.” From there it hopped over to then-Twitter, which had a team called “Observability Engineering” in the early 2010s. This is how the term got on my radar.

The first wave of cloud hit in 2006, when Google started hosting corporate email and when Amazon Web Services (AWS) launched S3 and EC2. Over the next 15-ish years, one by one, each of these operational tools and technologies we used to build was replaced by a managed API, SaaS (software as a service), or PaaS (platform as a service) that was better, faster, cheaper, and more reliable. As an industry, we became accustomed to the idea that we could now buy operational excellence in a tool: we didn't need to invest in these skill sets anymore.

And then, in the 2020s, one of those operational skill sets—*just* one of them—appears to have come furiously roaring back to life. Although we no longer talk about it as “monitoring” or “logging.” These days, we call it “observability”—and it accounts for 20-30% of all infrastructure spending at many companies, a dizzying rise over the past 15 years that shows no sign of slowing.

There's a broad assumption in our industry that observability is just fresh branding on top of monitoring. This is understandable, because all the monitoring tools *have* just rebranded themselves as observability tools, and most teams *are* managing observability as a cost center and an ops tool for infrastructure.

And therein lies the problem. Modern observability was never supposed to be an ops tool for infrastructure; it was supposed to be a tool for developers to understand their code in production. But that is not what it became.

We Lost the Fight to Define Observability

In 2016, when Christine and I started Honeycomb, we fought hard to give observability a technical definition. “Good observability means having the ability to understand whatever is happening in your system, at any time, even if you've never encountered this system state before,” we said. “To do that, you need tools that support high cardinality, high dimensionality, an explorable interface, and the ability to visualize events over time as a trace.”

We argued that observability was a tool for developers to follow their code into production and understand it there, in rich, exploratory detail; and monitoring, or logs and metrics, were for operations teams running infrastructure, and they were complementary. You needed both.

We lost that fight, of course, in a landslide. The idea of observability started to get traction in 2017, and got buzzy in 2018, and for a while it looked like it might stick. Vendors had been telling their customers for years that handling high-cardinality data was “impossible,” that it “couldn't be done”; when we showed that wasn't true, engineers started getting excited. “Observability” tracks started popping up at mainstream conferences. Monitorama renamed itself to “Monitorama (+ Observability).” O'Reilly asked us to write a book (this book!). But not everyone was on board. Customers said they wanted observability, so vendors did what vendors do: they

rebranded. Pretty much everyone in the monitoring, logging, or APM categories relaunched their existing products as “observability tools” between 2019 and 2021.

By the time Gartner added the “observability” category in 2022, the term had completed its long slide into meaninglessness. The reasons engineers got so excited about it in the early days—the reasons it was different—were long forgotten. Everyone believes they have observability these days, because observability can mean anything.

On the one hand, it’s hard not to get a little depressed by this. The technical definition we championed would have been really useful for all the people trying to understand their options in a saturated market where every technical term is over-marketed.

On the other hand, the technological currents that pushed us to develop a new paradigm for observability have only grown more intense. More and more teams are having to learn the same hard lessons Christine and I learned about the ways your relationship with production must be renegotiated after a step change increase in system complexity.

Production Has Become Too Complex for Us to Debug Via Intuition

Complexity has skyrocketed over the past 20 years. In 2010, the typical architecture was some variation on the LAMP stack: Linux, Apache, MySQL, and PHP. Monitoring and dashboards worked well in that world.

In 2026, there is no typical stack. Our systems are dynamic, ephemeral, and constantly in flux, a mesh of services, models, platforms, and APIs—and now (thanks to AI) we have nondeterminism as a feature, and there are agents in the mix. You can’t just hold a model of production in your head and reason about it anymore, or reliably intuit the answers.

A good analogy for monitoring versus observability comes from finance. If you’re a young person with a paycheck and a bank account, you don’t need any complicated financial arrangements. Set up direct deposit, set up savings, autopay your rent and utilities, and check your balance once in a while, and you’re golden. That’s monitoring.

Now let’s say you’re a financial services provider, or a bank. You have to track every single transaction going in or out of millions of accounts, pay and charge interest at different rates, track assets and asset classes and depreciation, and manage tax liability. You need to be able to drill down into any transaction between any set of accounts at any time, on a moment’s notice, and trace the historical path of any money that enters your system back until it exits the system. You need to set up proactive fraud detection and compliance alerts. Oh, and if you get any of it wrong, you could go to jail. That’s observability.

Does observability cost more than monitoring? Yes. Because it *does more*. Every question you ask about your system has to be mediated by your telemetry in some way, and that costs money. But once you do need observability, it would be far more expensive to try to hobble along without it.

The Facebook Experience That Showed Us What's Possible

Going back to the finance analogy: if you suddenly jump from a simple checking account to running a large, complex financial institution with millions of transactions, how do you even know where to start? That's not a common real-world scenario, but it's all too relatable in tech if you found yourself going from LAMP stack to a massively distributed systems wasteland practically overnight.

Shortly before founding Honeycomb, Christine and I found ourselves living through exactly that kind of painful shift. We'd been using tools built for simpler architectures to tackle incredibly large and complex problems. Both of us, working separately on **Parse**, had started shipping streams of data into a tool at Facebook called **Scuba**. And that experience forever changed how we thought about software.

In our experience:

- Novel problems that had been virtually impossible to find until you knew what to look for became possible to find in Scuba with no prior knowledge.
- Investigations that had taken time and energy became fast, simple, and cheap.
- Weirdest of all, the more junior engineers were starting to out-debug the experts who had built the system.

Entire categories of problems began to feel more like support tasks than engineering problems. This was mind-blowing. After leaving Facebook, we couldn't stop thinking about how much we missed Scuba. We were convinced the world needed more tools like it.

When engineers start companies, they typically fall into one of two groups: either they are subject matter experts (SMEs) who know exactly what they want to build, or they have an idea and need to find out if there's a market for it. Christine and I were neither. We didn't even fully understand *why* Scuba was so transformational—we just knew we missed it like crazy and couldn't imagine going back to a world without it.

We had the Scuba whitepaper, so we started experimenting, unpacking our experience, and sorting out what did and didn't matter. Christine and I were not SMEs, and most of the people we talked to had no idea what we were talking about (and even less interest). We were repeatedly told by investors and leading experts that what we were attempting was a “solved problem.”

Neither of us had any real background in this space. What we had was an experience and a lot of conviction. So, we started reading and reasoning from first principles to work out how to replicate the experience for others.

The Control Theory Definition Made It All Click

We knew what we were building was *different*—in workflow, storage, and signal types—than anything else we had seen in the software domain. We couldn't fit it into any existing category of tools.

Oddly, it seemed to have a lot more in common with *business analytics* than with anything used by software engineers or SREs. It wasn't monitoring, it wasn't logging, it wasn't tracing, it wasn't APM—so what were we building? How should we talk about it? What would make it click for people? That sent me off searching for new technical terms. That's when I read the **control theory definition of observability**:

Observability is a measure of how well internal states of a system can be inferred from knowledge of its external outputs.

Suddenly, it *clicked*. Fireworks.

The control theory definition nailed what was different about what we were building. With existing solutions, you had to know what you were looking for before you could find it. You had to have knowledge of the internal state before you could infer what was happening—a circular dependency. Observability broke that dependency.

If the goal is to understand any system state, at any given time, from the perspective of any user or intersection of users, just by observing the system outputs...what would that take?

Working backward, we derived six functional requirements (these may look familiar to you from **Chapter 1**):

1. High-dimensionality data: wide, wide structured events organized around a unit of work. All the context. Ideally, hundreds of attributes per event.
2. High-cardinality data, preserving as much detail as you want. Yes to unique IDs!
3. Store raw events and aggregate at read time, with flexible, inferred schemas and indexes. Pre-aggregation is a one-way street, and every key should be fast to query.
4. Zoom in to individual requests, and zoom out to the whole system, or any level of granularity in between. Visualize events over time as a trace.

5. Exploratory, open-ended user interfaces you can interact with, instead of static dashboards.
6. Queries must return within seconds, not minutes, so it is fast and interactive.

Traditional tools have always been good at known-unknowns. Once you know you have a problem, you can generally find what you're looking for. The tools work well for infra monitoring, and well enough for finding and fixing bugs and dealing with incidents.

What modern observability introduced was the ability to understand unknown-unknowns. To evaluate the quality of your product, and understand the experience each user is having. To follow your code into production and see how users interact with it, to be surprised by what they come up with. Modern observability is about so much more than just bugs and incidents. It's about understanding.

This new approach was useful to software engineers for developing and validating their code. That was the shift that took observability from a monitoring tool for infrastructure to the sense-making capability of software systems—a distinction I sometimes worry has gotten lost along the way.

The Modern Observability Landscape Is Confusing and Continues to Expand

Then again, maybe not. After all, Honeycomb is no longer the only company with a product that handles high-cardinality, high-dimensionality data with an explorable experience. In fact, every observability startup founded since 2019 (that we know of) has followed our recipe: wide, structured events, backed by a columnar data store. When Honeycomb was starting out, there were no publicly available columnar storage engines, so we had to write our own. Now there's DuckDB, ClickHouse, Snowflake, and more, and a new crop of observability vendors built on top of them.

Not that you'd know it just by looking at their websites: the three pillars, infrastructure monitoring type tools use all the same words and phrases as the structured events backed by columnar database-type tools. In 2024, I took another stab at differentiation via technical terms. Regrettably, I used the most hackneyed technical cliché of all: 1.0 versus 2.0:

Observability 1.0

This is when you send each type of signal to a separate tool or storage system. Metrics go to a time-series database, logs go to a log aggregator, traces go to a tracing tool, profiling data goes to a profiling tool, and so on. Any correlations between pillars must be predefined and maintained by hand. You pay to store it many, many times.

Observability 2.0

This is when you send your telemetry to one unified storage location. You send rich, structured data that preserves the relationships for you to interrogate later. You pay to store it once. In many tools, you can visualize the events over time as a trace, or slice and dice them like normal log events. No dead ends.

Simple, right? You can either have one source of truth or many sources, but you can't have both. It's clarifying. Recent to the time of writing in early 2026, Hazel Weakly proposed identifying data lakehouse architectures as “observability 3.0,” and there is, inevitably, already an “observability 4.0” on the loose.

This escalating arms race in version numbers is, of course, the inevitable downside of the numbering exercise, which is why I have mostly stopped using it. (It also made traditional observability vendors angry. Some feel the “1.0” label implies they are less advanced. They are not less advanced; they are simply built for a different use case. But I get it.)

Costs Are Driving the Need to Change, and AI Is Enabling That Change

My personal opinion is that the world is moving, slowly yet inexorably, toward the unified model. To be clear: this is my personal opinion, not the vendor-neutral authorial stance of the “we” voice elsewhere in this book.

Why? One big reason is cost. If you're using 10–20 observability tools—and according to Gartner, that's the norm—you're paying 10–20x in costs for every request that enters your system, *at minimum*. That cost multiplier is not sustainable.

Teams have been hemorrhaging their most valuable data—cardinality and context—in an attempt to cap cost growth. It's simply the wrong data model for the job.

When the distinction between observability and monitoring got lost, the losers were developers and finance. Developers, because production remains as hostile and opaque to their needs as ever. Finance, because when you're trying to meet developers' needs using infrastructure tools, you end up paying to store the same data over and over and over again, in slightly different formats. And then you end up paying over and over and over again to store data that bridges the gap between those formats, store different data types in different ways, and so on.

The other big reason is AI, which attacks the status quo on multiple fronts at once:

- AI is a technical enabler that can close the gaps that have made production feel uncomfortable to developers.
- Modern observability has been hamstrung by the ecosystem built around three pillars, but the AI ecosystem is springing afresh.

- AI is a competitive forcing function for execs to invest in reliability and guardrails.
- AI agents are gluttons for context, and AI is only as good as the data it's trained on.
- Nondeterminism is a feature. Developers who want to understand agents will have to instrument them and observe in production.
- Our ability to write code is no longer the bottleneck. Now, the bottleneck is our ability to understand what we have written and make decisions about what to do next.

So, what do we do with all of this?

Distilling the Lessons That Matter

From a computer science perspective, none of this is new. The business side of our organizations went through this exact transformation 20-odd years ago when columnar storage engines were first introduced. Suddenly, analysts went from having to predefine metrics and dashboards to being able to slice and dice and zoom in and out, exploring data, finding new trends on the fly. They were hooked.

Sales and marketing have always paid top dollar for great data tools because it means the difference between profits and losses. For engineering, that argument was never quite as clear and straightforward. And of course, engineering has long relied on tools that were selected, bought, rolled out, and maintained by infrastructure teams.

Today, most of the tools that handle software telemetry get bucketed under “observability.” Some of them enable engineers to understand the quality of their product from the perspective of any or each of their users, as we have described. Most do not. The yawning gap between the cost of observability and the value returned is a function of this mismatch.

The cornerstone of every high-performing engineering org is fast feedback loops, and you get those from observability—the sense-making function of production systems. That function has a compounding ripple effect. The more fast feedback loops that engineers have, the deeper and more meaningful the questions they ask. When they consistently find good answers quickly and easily, your systems and teams shift toward operating more sensibly and transparently. Your ability to ship swiftly, with confidence, picks up real speed. (We discuss this flywheel extensively in Chapters 24 through 25.)

That concept is not well understood by most engineering leaders right now—execs, managers, staff, and engineers alike. Probably because very few teams are actually operating anywhere near their theoretical peak potential.

But *some are*. This is not theoretical. And I’m actually uncharacteristically optimistic that more leaders are beginning to understand the role that observability plays in unlocking that potential. Leaders are waking up to the fact that their AI investments pay off only after they first invest in reliability and observability.

At an individual level, observability is a force-amplifying power. It makes you better at everything else you do and connects you with the impact of your work on other people. When you understand exactly how your code behaves, what it’s really doing, and how that translates into each user’s experience. That is what gives meaning to the work of software engineering.

Every company needs *somebody* who understands both observability and systems thinking and can make good choices—or else every migration, feature, deploy, and decision will be longer, harder, and more painful than necessary. Attention is the scarcest resource at every tech company right now, and the bottleneck has shifted from writing code to understanding the code we write. We don’t need everyone to understand observability deeply from first principles. But every company needs someone, or some team, who does.

If that person is you: good luck. My hope is that this book can help lead you to success.

Conclusion

Charity’s history of observability can be read in many ways. The takeaway we most hope you leave with is an understanding of the user experience that sparked the breakthrough—the moment that made “observability” ubiquitous.

The six functional requirements captured by that experience illustrate what separates a newer school of thought from more traditional debugging approaches. We think this is the most misunderstood lesson in observability—and that misunderstanding is largely responsible for the disillusionment that sets in when the costs don’t seem to match the value.

The rest of the book unpacks this in detail: technical implementations, use cases, and guidance for different roles and team types. If you haven’t already, the book’s preface can help you figure out which sections matter most for you.

Next, we start at the foundation: the data itself. Before you can analyze anything, you have to instrument your systems to capture it. That’s where **Part II** begins.

Instrumentation Fundamentals

Part I covered the concepts you need to understand observability. This part is where you start building it.

Instrumentation is the process of adding code to your systems so that they emit telemetry—and getting that right is the foundation that everything else rests on. This section is for most readers.

Chapter 4 introduces instrumentation and telemetry basics: what engineers should be thinking about when they instrument their code, and a first look at OpenTelemetry, the open source framework that's become the standard for doing so.

Chapter 5 challenges the three pillars model—the idea that logs, metrics, and traces are fundamentally separate things that need to be managed separately. This chapter breaks down why that framing is limiting, and shows how all three can be unified into a single substrate: the arbitrarily wide structured event. The key insight is that predefining the shape of your telemetry predefines the questions you can ask of it. Unifying your data frees you from that constraint—capture all the context now, and then let the investigation shape the query later. That idea is central to everything that follows. Jeremy Morrell contributed greatly to this chapter.

Chapter 6 is a guest chapter written entirely by Jeremy Morrell. It walks through common system conditions to show concretely what telemetry you should be capturing and what questions you can answer as a result. This is practical, hands-on guidance for how to start instrumenting your code well.

Chapter 7 goes deeper on OpenTelemetry—not as a how-to, but as a conceptual guide. It covers the project’s design, common instrumentation patterns, and how to work with it effectively when using AI-assisted coding.

In **Part III**, we’ll look at how to analyze the telemetry you’re now capturing and turn it into actual insight.

Getting Started with Instrumentation

Now that we're on the same page about overarching theories and the origins of observability as a practice, we'll get into concrete details about technical fundamentals and implementations. We will begin by examining the fundamentals of instrumentation.

There's an old business adage that roughly says, "you can't improve what you don't measure." That idea rings true for the systems you build with a high degree of observability. Because there are so many different functional aspects of modern application systems, in practice, we must collect an increasingly vast amount of data in order to fully understand them.

As you'll see in this section of the book, generating and collecting telemetry data is the foundational cornerstone of an observable system. Given a sufficient volume of telemetry data to work from, it's possible to understand any given state of your system—no matter how novel or bizarre. However, before we start talking about data analysis, we first need to dig into what a "sufficient volume of telemetry data" means. Let's dive into what it takes to properly instrument your systems.

This chapter provides an introduction to instrumentation, and it offers considerations when generating telemetry and regarding where that telemetry is sent. As a software engineer responsible for ensuring application reliability, these considerations must be essential to your workflows.

We also briefly introduce concepts that all software engineers must know when approaching observability: the OpenTelemetry project (an open source and vendor-neutral instrumentation framework), deployment strategies, and practical considerations like customization, costs, and sampling. This chapter focuses on introductory-level information, but we will build much further upon these concepts throughout the course of this book.

Instrumentation Basics

What is instrumentation in software engineering? It's the practice of adding code to your applications so that they emit data about what's happening as your software runs. You use that data to understand its behavior, measure performance, or diagnose runtime errors.

To use an analogy, modern observable systems are akin to flying home at night, looking out the window, and gazing down on bustling city lights. Thousands of lights will flicker on and off, traffic will flow across intricate pathways, and complex patterns of activity can be beautiful to observe but difficult to understand without the right vantage point.

Just as city planners need traffic sensors, population monitors, and utility meters to understand how their metropolis functions, software engineers need instrumentation to understand how their applications behave in production. One of the authors of this book came up with that comparison after a few days of intensive planning work for this book. Clearly, that work took its toll, but we think the analogy stands.

You can think of instrumentation as installing sensors and meters throughout your application's sprawling metropolis to report on everything from how long database queries take, to which features users are actually using, to what happens when errors occur. That telemetry data becomes the foundation for observability, or your ability to understand and explain any state your system can get into, no matter how novel or complex. The more comprehensive the instrumentation, the more granular your understanding becomes.

Without proper instrumentation, debugging production issues becomes an exercise in detective work with insufficient evidence. You might know that users are experiencing slowness, but you won't know which specific operations are slow, for which users, or under what conditions. If you don't start with proper instrumentation, you'll find yourself adding log statements after problems occur, waiting for issues to reproduce, and making educated guesses full of logical leaps in an attempt to arrive at elusive root causes with even more elusive contributing factors.

With good instrumentation, you can approach problems systematically, following the data to understand exactly what happened and why. This part of the book will teach you how to implement good instrumentation.

The Vocabulary of Telemetry: Logs, Metrics, and Traces

Before diving into instrumentation strategies, it's helpful to understand the types of telemetry data your applications can emit. Take this with a grain of salt for now (we will explain more about telemetry datatypes in the next chapter), but most

readers who are new to modern observable systems will think of these as what is also sometimes referred to as “the three pillars” of observability: logs, metrics, and traces.

While this book doesn’t promote the idea of “pillars,” we want to emphasize the importance of these *datatypes for telemetry* for observability. Each type serves a particular purpose, and it’s important to know which type of data to reach for when instrumenting an application.

Historically, these datatypes have served different purposes and answered different questions about your system’s behavior. For the purposes of this chapter, we’ll unpack the historical uses and connotations of these datatypes, and how to collect them (before breaking them down further).

Logs represent discrete events that happened in your application at specific points in time. The content of your logs might capture when a user logged in, when a payment was processed, or when an error occurred. Traditional logs were often unstructured text designed for human reading, but modern logging practices emphasize structured logs with consistent key-value pairs that machines can easily parse and analyze. A well-structured log entry might look like:

```
{
  "timestamp": "2024",
  "user_id": "630630",
  "action": "checkout_completed",
  [...]
}
```

Metrics represent numerical measurements of your system over a period of time.¹ They answer questions like “How many requests per second is my service handling?” or “What’s the 95th percentile response time for my API?” Metrics are typically captured as numeric values that aggregate or summarize data. Instead of storing information about every individual request, you might store a counter that increments with each request, or a histogram that tracks the distribution of response times. That aggregation makes metrics efficient to store and query, but it also means you lose the ability to drill down into any context behind the sum.

Distributed *traces* (also known more simply as “traces”) represent the journey of a single transaction as it flows through a system. When a user places an order on your site, that request might trigger calls to several dependent services to update inventory, calculate shipping, process payment, and email a receipt. A trace captures all of these operations as a connected sequence, showing you the timeline of what happened and where time was spent. Each step in this sequence is called a *span*, and spans can contain their own detailed information about what occurred during that operation.

¹ For more detail on the foundational datatypes of metrics, please see the [OpenTelemetry Metrics Specification](#).

As noted in [Chapter 1](#), these datatypes have been historically managed and stored separately, even though they complement each other. Each telemetry type has a use case for which it is best suited. Logs give you detailed context about individual events. Metrics give you aggregate views of system health and performance trends. Traces give you end-to-end visibility into complex operations that span multiple services.

In the next chapter, we'll look at ways to optimize for and unify these use cases to achieve a high degree of observability. For now, let's focus on modern instrumentation approaches that make it easy to emit rich telemetry that serves multiple purposes.

OpenTelemetry: The Universal Language for Observability

Historically, observability vendors required the use of their own proprietary agents and libraries to collect telemetry from your applications. If you wanted to try a different vendor or send data to multiple backends, you'd need to rethink your entire instrumentation strategy and, often, face a scenario where you'd need to rip and replace existing agents and code. That created a frustrating lock-in scenario, where your initial choice of observability tool dictated how you could instrument your code.

OpenTelemetry (or OTel) emerged as the solution to that problem. It's an open source project that provides vendor-neutral APIs, libraries, and tools for collecting telemetry from your applications. Think of OTel as a universal translator that lets your applications speak a common language that any observability backend can understand.

The power of OTel comes via its separation of concerns. Your application code uses OpenTelemetry APIs to emit telemetry, but the actual destination for that data is configurable. You can send the same instrumentation data to multiple backends simultaneously, making it easy to evaluate different observability tools or maintain redundancy. You can even change your backend completely without touching your application code by just updating the destination configuration. As a result, library authors can also safely call the API knowing it will default to performing no operations and thus have no overhead unless configured.

Out of the box, OTel supports automatic instrumentation for popular frameworks and libraries. That means you can get basic telemetry flowing with minimal code changes. For a typical web application, you might only need to add a few lines of configuration in order to automatically capture HTTP requests, database queries, and calls to external services. Automatic instrumentation gives you immediate visibility into common actions to understand your application's behavior without extensive manual instrumentation.

But the real strength of OTel becomes apparent when you start adding custom instrumentation specific to your business logic. While automatic instrumentation captures the technical plumbing of your application, custom instrumentation captures the

domain-specific operations that matter to your business. For example, custom instrumentation might be used to capture user registration flows, order processing pipelines, or how content recommendation algorithms perform. The simple reality of business applications is that they have many unique aspects that automatic instrumentation simply cannot and will not understand.

Automatic Instrumentation Versus Custom Instrumentation

Inevitably, most engineers will ask whether they should use automatic or custom instrumentation. The answer is to use both. The distinction between automatic and custom instrumentation represents different approaches to capturing telemetry, each with its own strengths and appropriate use cases.

Automatic instrumentation works by intercepting common operations in your application framework and libraries. When you make an HTTP request, query a database, or call an external service, automatic instrumentation can capture details about these operations and how they relate to each other without any explicit code on your part. Modern automatic instrumentation is remarkably sophisticated, capable of understanding complex frameworks and extracting meaningful telemetry with minimal configuration.

The primary advantage of automatic instrumentation is in its breadth of coverage with minimal effort. You can instrument dozens of common operations across your entire application by adding just a few lines of configuration. That minimal effort will give you immediate visibility into the basic technical health of your application with data like response times, error rates, dependencies, and resource usage.

However, automatic instrumentation can only capture what it understands about technical operations across the lowest common denominator for all applications. It can't understand your business logic, domain-specific workflows, or application-specific error conditions. For example, automatic instrumentation might tell you that a database query took 200 milliseconds, but it won't tell you that this query was part of a user registration flow that failed because the email address was already in use.

Custom instrumentation fills that gap by capturing telemetry that is specific to your application's business logic. That's what enables you to understand the state of things like user actions, business processes, and domain-specific operations. For example, custom instrumentation might capture when users complete specific workflows, when background jobs process certain types of data, or when business rules trigger specific decisions.

The most effective observability strategies combine both automatic and custom instrumentation. Automatic instrumentation provides comprehensive coverage of

the technical plumbing and the skeleton of how everything fits together. Custom instrumentation attaches insight into business operations and user experience, adding muscles to that skeleton.

Teams may initially be shocked when they turn on automatic instrumentation in production—the level of detail can be overwhelming and often counterproductive. Customizing your instrumentation isn’t just about adding *more*; it’s also about knowing what to take away. Developing an intuition for what’s valuable in different environments can pay dividends. For example, recording every database call and Redis query may be useful locally, but in production those small spans can add up to a real performance impact.

As your system matures, you’ll likely find that custom instrumentation becomes increasingly valuable for understanding the aspects of your application that matter most to your business. In [Chapter 7](#), we’ll cover this entire topic in more detail, along with specific guidance on instrumenting for both common and less-common application architectures.

Ownership of Instrumentation

One of the most important decisions your team will make in the course of an observability initiative is who owns the instrumentation strategy and how it evolves over time. In traditional monitoring, instrumentation was often the responsibility of a dedicated operations team that added monitoring after applications were deployed. That reactive approach often led to gaps in coverage and instrumentation that didn’t align well with the actual structure and business logic of applications.

Modern observability requires a shift in ownership. The engineers who write the application code are best positioned to understand what instrumentation would be most valuable. They know which operations are critical to the business, which code paths are most complex or error-prone, and what context would be especially helpful during debugging. When developers own instrumentation, it becomes part of the design process rather than an afterthought.

However, that shift in ownership doesn’t mean every software engineer needs to become an expert in observability tooling. Instead, successful organizations establish patterns and conventions that make it easy for developers to add good instrumentation without deep expertise. Those conventions might include techniques like providing standard libraries to more easily propagate common instrumentation patterns, code review guidelines that ensure new features include appropriate telemetry, or documentation that explains when and how to add custom instrumentation (for a more in-depth look, see [Part VI](#) on observability governance).

Your instrumentation strategy should evolve alongside your application. As your system grows more complex, as you learn from production incidents, and as your

understanding of user behavior deepens, so too will your instrumentation needs evolve. In other words, instrumentation isn't a one-time setup task. As a software engineer, you should build the evolution of instrumentation into your development process to ensure that your observability capabilities keep pace with your system's complexity.

In time, you will establish instrumentation as a primary deliverable in your development process, just as you have with tests (see [Chapter 9](#), on the parallels between test-driven development and observability-driven development). When planning new features, your team should explicitly discuss what telemetry would help you best understand how those features behave in production. During code reviews, your team should evaluate whether the proposed changes include a sufficient level of instrumentation. When conducting post-incident reviews, your team should identify gaps in telemetry that made the investigation more difficult than necessary.

If it hasn't already, instrumentation will become an integral part of your ongoing software development workflows.

Building a Custom Instrumentation Strategy

Developing an effective custom instrumentation strategy requires thinking systematically about what aspects of your application behavior would be most valuable to understand. Rather than instrumenting everything, focus on the areas where visibility will have the most impact on your ability to operate and improve your system.

Start by identifying the critical user journeys in your application (see [Chapter 11](#)). Critical user journeys are the sequences of operations that users must complete successfully in order for your business to function. Of course, the specific details will vary across industries and use cases. But, as an example, let's suppose you have an ecommerce application. Business-critical user journeys for you might include things like user registration, product search, cart management, and checkout. Or, let's say you have a content-management platform. Then, critical user journeys might include content creation, publishing, and consumption workflows. Instrumenting these journeys gives you visibility into the health of your core business operations.

Consider the error conditions that are specific to your domain. While automatic instrumentation might capture HTTP 500 errors, it won't understand business-specific failures like "insufficient inventory," "invalid coupon code," or "user not in the correct experiment group for this operation." Custom instrumentation can capture those domain-specific error conditions along with the context needed to identify, understand, and resolve them. Be sure to distinguish temporary versus permanent failures, as well as failures that are or are not actually actionable by an IT team.

It can take practice when you're first thinking about what to capture. To get started, try thinking about the questions you frequently need to answer during incident response or when analyzing system performance:

- Which users are being affected?
- What are the key stages of my pipelines?
- What feature flags are in use?

Remember that instrumentation is not just about capturing what happens. Instrumentation exists to capture any additional context that also helps you understand *why* it happened. Include relevant business context like user segments, feature flags that were active, or the source of a request (for more in-depth examples, see [Chapter 6](#)). That type of contextual information often makes the difference between quickly resolving an issue and spending hours trying to reproduce and understand it.

As your custom instrumentation evolves, you'll want to establish patterns and conventions that make it easy for all developers to add consistent custom instrumentation. That might include standard attributes that should be included in all business-operation traces, naming conventions, or templates for instrumenting common patterns like background job processing or API endpoints (for more tips, see [Chapter 31](#)).

Remember that your custom instrumentation needs will evolve as you become more immersed in these concepts and as your understanding of your system deepens. After major incidents, you'll begin to find opportunities for additional instrumentation that could have helped you detect or resolve similar issues more quickly. When you find yourself repeatedly digging through logs or manually correlating data to understand system behavior, you'll likely identify new custom instrumentation that could simplify that analysis.

The goal is to build instrumentation that not only helps you understand what's happening in your system, but also empowers you to ask and answer better questions that you haven't even thought of yet. Recall that systems with a high degree of observability give you the ability to understand and explain any state your system can get into, no matter how novel or complex. Great instrumentation captures a high level of context and provides the foundation for great observability.

That said, as you build custom instrumentation, it's tempting to try to capture every possible piece of information. While some observability tools are built to work with that strategy, for others that choice may not be practical. Time-series databases have explicit design constraints that lead to poor outcomes when the cardinality of a metric becomes too high. Full-text search databases require indexing and precomputed schemas. Too much instrumentation can create noise that makes it harder to

find the signal you actually need, and it can increase costs and operational overhead without providing proportional value.

Cost Considerations, Volume Management, and Processing

Inevitably, you'll end up needing to manage the volumes of telemetry data that properly instrumented applications produce. The fact of the matter is, most traffic is boring and commonplace, and so is the data for that traffic. But some traffic is interesting, or essential to understanding what's happening. Determining how to manage data along these lines isn't a simple calculus, and there are many levers that you have access to for performing this management and balancing trade-offs. We won't cover all of those options in-depth here, but we will introduce the basic concepts and considerations. In later chapters, these topics are discussed further.

Processing and Pipelines

It's common to process telemetry data before storing it, regardless of the type of instrumentation you use. For example, you may need to drop or transform data to reduce its volume or perform extraction or conversion of data into other formats. You might also need to do things like perform data normalization or filter out personally identifiable information before sending your telemetry to a backend for storage (see [Chapter 16](#)).

When you're writing instrumentation into your code, you should be agnostic to backend capabilities for the sake of portability, and try to defer processing decisions until later in the telemetry-collection process. After all, you don't know exactly what telemetry data you might need in your investigations. If any data captured becomes problematic, it's much easier to remove specific details that turn out to be unneeded rather than to be left wanting for attributes that were never collected to begin with.

Additionally, you should consider the scaling implications of your instrumentation strategy, design, and implementation. For example, not all users of an instrumentation library will require global replication or remote procedure calls with submillisecond response times. Some will, but not all. Good instrumentation design takes that variability into account. In cases like these, you can use techniques like telemetry layering (see [Chapter 7](#)).

Cost Considerations and Volume

You can capture all the telemetry you want, but what does it cost? Just about everything in your instrumentation strategy can be quantified in terms of cost.

Some costs are clear, such as the amount of storage, memory, or computation required to both store and query the data. These costs are typically incurred as a bill from an external vendor if you bought an observability solution. Or, they could be incurred as internal costs measured in both infrastructure and labor required to create and maintain custom-built observability solutions (see [Chapter 29](#)).

Many costs are more hidden. For example, a poor instrumentation strategy can lead to confusing data or disjointed datasets. If data is scattered across 10 or 15 different tools, you're likely paying hidden costs like cognitive overload and friction when incident responders are unable to quickly triage and correctly diagnose issues after alerts are triggered. Those costs often manifest visibly as delayed incident-resolution times, but the factors behind that increase are often invisible to executive stakeholders. In addition, cost-management practices have an overhead on developer productivity and incident resolution—trying to understand why spans are sampled out, or creating convoluted workarounds to reduce metric cardinality.

We'll discuss these costs further as we examine choices between buying and building observability solutions. For now, keep in mind that there's no such thing as a free lunch. Teams that are deciding to build versus buy may account for obvious costs like labor and infrastructure, but are often surprised by less readily apparent costs. For example, self-hosted telemetry datastores often still incur costs for things like network egress, which can quickly escalate while transferring telemetry data between availability zones, regions, clusters, datacenters, and so forth.

Most often, the overhead required to collect and transmit a trickle of telemetry data over the network for backend systems is quite small and manageable. However, when working at scale, distributed or cloud native workloads can turn that into a flood of data, especially if instrumentation is misconfigured. Various solutions exist for managing telemetry data at scale, such as utilizing telemetry pipelines to buffer high-resolution data locally, while shipping aggregated data across the network (see previous callouts for these concepts).

However, there is one specific element of cost control and volume management that deserves special attention in this introductory chapter: telemetry sampling.

Essential Concepts for Sampling

Rather than recording 100% of all generated telemetry, sampling is the practice of selectively deciding what data to keep (the collected sample) and discarding the rest. In general, we believe it is a best practice to sample all telemetry data, regardless of the scale at which you operate: sampling lets you capture truly valuable data while discarding irrelevant noise.

Conceptually, there are only two ways of sampling data: you can sample either before or after it gets emitted.

Before

Sampling before data gets emitted happens in various ways. For example, there's one very common method for pre-emitted telemetry sampling that most developers don't often appreciate: metrics. Metrics are a form of sampling. They are numerical values that are calculated by choosing measurements and dimensions, and then aggregating all measurements that occurred over a specified time interval. The individual events that occurred during that interval are considered noise and get discarded. Only the pre-aggregated summary value remains.

A more-often realized form is something like probability-based head sampling, which is a simple technique that boils down to "only sample a percentage of my traces or logs." Head sampling works very well for services operating at a truly high volume, but the simple probability-based approach presents significant drawbacks for services not operating at enormous scale.

Let's say your service has uniform traffic patterns and you only care about aggregate statistics. If that's the case, then this approach favors a reduction in overall telemetry data volume at the cost of potentially missing outliers or long-tail issues. In other words, in a scenario where we are only keeping a 1% sample of telemetry data, it's entirely likely that an error that only occurs 0.1% of the time would be completely missed. Again, for services operating at enormous scale, 0.1% of traffic is still a massive amount of traffic, but for anything less, you risk losing valuable error data by head sampling.

Sampling data before it gets emitted (at the source) most often favors resource efficiency over data fidelity and completeness. It is also oblivious to concerns upstream or downstream of its decision-making. For example, there is no way to know if a downstream service call results in an error at the time that a head-sampling decision must be made. If what you're after is a high degree of observability, the head-sampling trade-off is almost entirely skewed toward saving on cost at the expense of debuggability.

After

The second form of sampling happens after telemetry data is emitted. Unlike head sampling, this form of sampling should be used to optimize for telemetry data fidelity and completeness. A wide variety of techniques can be used for this purpose.

For example, with distributed traces, you can use tail-based sampling. Tail-based sampling is a technique where all spans for a trace are collected and buffered before a sampling decision is made. That way, you can apply processing logic and rules to examine the entirety of the transactional record to determine whether or not this specific record is worth keeping.

In other words, with tail-based sampling, you could examine each trace or log line entirely before making a decision to keep or discard. Does this trace look just like the thousands of other 200 OK status requests you’ve recorded over the last hour? Or did it capture something more significant that indicates it is worth keeping?

Techniques for effective telemetry sampling are sophisticated and nuanced because they ultimately must take into account specific rules or considerations for your business. For this introductory content, we won’t unpack how to sample effectively any further. But we do think it’s imperative to point out that not all the telemetry you generate is necessarily worth keeping. Remember, all telemetry has some level of cost. Practiced effectively, sampling is one of the most effective levers you can use to manage observability costs (for in-depth techniques, see [Chapter 15](#)).

Conclusion

You can’t improve what you can’t measure. Instrumentation is the foundational cornerstone of your observability experience. Every software engineer must learn to integrate instrumentation into every facet of the development lifecycle. You will rely on automatic instrumentation to quickly reveal the basic vital signs of operational health in your application. You’ll rely on custom instrumentation to understand the health of your business functions.

You’ll need to think about instrumentation holistically and form a cohesive strategy for its ownership and evolution. More than likely, you’ll add that instrumentation to your code via OpenTelemetry. As a software engineer, you may not be involved in all of the intricacies for managing observability costs, but you should have at least some fundamental understanding of the techniques used to do so and their implications. This chapter introduces concepts that all engineers should know, and we’ll continue to unpack those further in the remainder of this book.

Next, we’ll explore another telemetry-related concept that is fundamental to learning about systems with a high degree of observability. It’s only possible to move from a three pillars model to a unified model of observability by removing the erroneous distinctions between the telemetry types you know as logs, metrics, and traces.

Structured Events Are the Building Blocks of Observability

In **Part I**, we introduced differences between the three pillars and unified models of observability. In this chapter, we show how that unification happens by unraveling the myth that the standard observability telemetry signals—logs, metrics, and traces—must inherently be managed as different datatypes.

For years, we software engineers have been taught to squint at our systems through different keyholes: metrics for a pulse check, logs for the narrative, and traces for extended views of complex actions. Each of these views has its own tooling, with its own conventions and dogma, that treats each as fundamentally different kinds of data, collected and stored in separate silos. But the problem that separation creates isn't just philosophical. *The shape of the data you collect constrains the questions you can ask later.* When you pre-decide the shape, you pre-decide the investigation.

That siloed split that impedes your debugging abilities, however, is merely a historical artifact. Unfortunately, many of us software engineers haven't fully realized that because squinting through different keyholes is the only method we've known since we first started coding. Instead of collecting three different representations of the same system state, we can primarily capture one context-rich representation (the wide, structured event) and use that to generate the appropriate view we need (metric, log, or trace), on demand, from that single record.

It's best to show, rather than just tell. So, over the next two chapters, we will dive deep into deconstructing the structure, advantages, and limitations of the various observability telemetry datatypes. We will then show how to reconstruct them using a single common substrate: the arbitrarily wide, structured event—the fundamental building block for unified observability. In this chapter, we'll focus on examining the

common data format (a “structured event”). Then, in the next chapter, we’ll show how to make structured events more useful by making them “arbitrarily wide.”

Jeremy Morrell, guest contributor in the next chapter, is also a primary contributor for this chapter.

What Is a Structured Event?

To understand a structured event, you must first understand its components. Let’s start by defining an “event.” This is a commonly used term in software development, let alone observability. It’s an overloaded term with different meanings attached. So, we will be context-specific.

An “event” is any piece of data emitted from your application. When you log onto a terminal, the buffer of characters emitted to `stdout` can be considered an event (though it is free-form text encoding only one value: a message). For practical purposes, an *event* is a record of everything that occurred during a unit of work within your system.

A “structured event” is an event that can be parsed into a set of key-value pairs that may contain many values. For our purposes, you can visualize an event as a JavaScript Object Notation (JSON) object with no nested fields:

```
{
  "timestamp": "6:00:00",
  "user_id": "123",
  "user_action": "signup",
  "referral": false
}
```

For the purposes of illustrating how that data is stored and how that affects later questions that you can ask, let’s also imagine how that event might be written as a database record, where each field is mapped to a column in a database table. Here, we’ll include a second and similar event that happens mere seconds later:

timestamp	user_id	user_action	referral
6:00:00	123	signup	false
6:00:03	456	checkout	true

Looking at events in this way is sufficiently generic, and it enables us to usefully model each of the three pillar datatypes (metrics, logs, and traces) to better understand their limitations, inner workings, and how to reconsider them.

The Limitations of Metrics

To understand the limitations of metrics, first we must be clear about what metrics are. Let's start by disambiguating the term "metrics." Unfortunately, it is sometimes used as a generic synonym for "telemetry" (e.g., "All of our metrics are up and to the right, boss!"). In this context, we understand "metrics" to mean: the scalar values collected to represent system state, typically stored in a time-series database.

What Is a Metric?

In 1988, by way of the Simple Network Management Protocol (SNMPv1, as defined in [RFC 1157](#)), the foundational substrate of monitoring was born: the metric. A *metric* is a single number, with tags optionally appended for grouping and searching those numbers. Metrics are, by their very nature, disposable and cheap. They have a predictable storage footprint. They're easy to aggregate along regular time-series buckets. And, thus, the metric became the base unit for a generation or two of telemetry—the data we gather from remote endpoints for automatic transmission to monitoring systems.

Metrics data systems are designed to efficiently collect and aggregate some value on a set periodic interval, such as every 60 seconds. These values are used to track some aspect of our system over time:

- How many HTTP requests were served?
- How many HTTP requests resulting in a response with a 500-status code were served?
- What was the CPU utilization?
- What was the slowest response time?

It's important to note that, back in the 1980s when the first metric systems were developed, computer systems had orders of magnitude less memory, storage, and bandwidth available. Because of that lineage, above all else, metrics optimize for reducing data volume and resource usage.

A metric requires a few pieces of information:

Timestamp

Since we only collect values every 60 seconds, this can be rounded to the start of the nearest interval (i.e., 6:16:23 => 6:16:00).

Name

A descriptive name of the value being tracked, such as `http.request.count` or `cpu.utilization`.

Measurement

The value to be tracked over time, frequently (but not always) captured as a single number.

Type

Given a set of metric records, the metric system needs to know how to combine them (more about this in the next section).

Set of attributes

Also called tags, optional sets of key-value pairs that add additional context (e.g., `sensor_id`, `server_id`, `region`, etc.).

Together, all of this information creates a single metric record. As a structured event, it might look like this:

```
{
  timestamp: "6:00:00",
  type: "count",
  name: "http.request.count",
  value: 212,
  server_id: "web.1",
  region: "north_america"
}
```

It's worth noting that in OpenTelemetry, metrics are modeled as “metric events” over the wire in almost exactly this fashion (see [Chapter 7](#)).

Metrics Are Typically Aggregated

Metric records are distinct from the other event types because they are typically combined. All metric records that share the same name, set of attributes, and fall within the same time interval are aggregated into a singular record.

For example, if you are tracking how many HTTP requests your service handles, you can use a “counter” metric (see [Figure 5-1](#)). You'll get a different value from each instance of your service, and then they are added together to get the total requests served per one-minute interval.

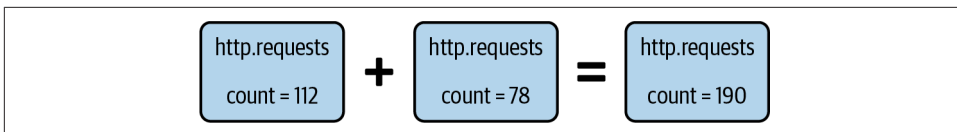


Figure 5-1. An example of a counter metric that shows how various measurements of `http.requests` can be aggregated over one time period by adding them together.

As a different example of aggregation, if you were reading a temperature sensor every 10 seconds, then you would have 6 measurements per minute. That can be aggregated as a “gauge” metric (see [Figure 5-2](#)) by dropping all but the last value. A gauge metric is a time-series value that can go up or down, representing values that fluctuate.

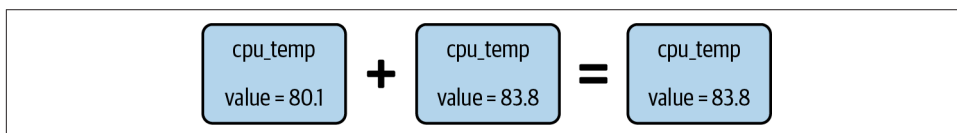


Figure 5-2. Unlike a counter metric, which only increases, a gauge can arbitrarily increase or decrease and captures the instantaneous value at a specific point in time.

Aggregations present some inherent limitations. As an example of a measurement that would not work, let’s assume you received the following as average `http.response.times`, per minute, per instance of your service:

Timestamp	Server	Response time (ms)
6:00:00	web.1	120
6:00:00	web.2	230
6:00:00	web.3	200

What questions can you answer with data that is shaped this way? For example, if you asked what the average response time is of this service, as a whole across all instances, can you get a correct answer?

You might be tempted to average all of the values together and say that the average response time of the service was 183 ms. However, that simplistic approach is problematic because each aggregated measurement may have contained a different number of datapoints. Unfortunately, there is no mathematically sensible way to combine these measurements once you have lost data by aggregating them into an average.

It’s possible to average averages. But only if the groups are the same size; otherwise, you must use a weighted average to get the correct overall average (which requires knowing the lost datapoints). If the groups had different numbers of datapoints (say, `web.1` had 100 requests, `web.2` had 57, and `web.3` had 98), then an unweighted average of averages would be grossly inaccurate, as it gives equal weight to each group’s average, regardless of its size.

If you anticipate this problem, instead, you can expose that information by collecting two slightly different values, such as the number of requests and the total time spent serving each, like so:

Server	Requests	Total response time (ms)
web.1	100	12,000
web.2	57	13,110
web.3	98	19,600

With these unaggregated values, you can calculate the average response time per instance, and also combine them all for an accurate average response time of 175 milliseconds. The “histogram” metric type is made to enable that type of data capture.

You can get the answer you need with the correct metric in place. However, this requires some guesswork. You might have just collected average response times across instances because that seems good enough. You may not even realize you needed to know what the average response time was for your service across all instances until a particular investigation demands an answer to that question.

If you hadn’t thought to set up a histogram metric before you needed that answer, there is no way to go back now and get it. The best you can do is set it up for future use and hope that someday you need to ask that same question again.

Aggregation is the key property that distinguishes metrics and what makes them extremely economical to use. However, it is also aggregation that hinders detailed analysis without already knowing what needs to be aggregated. Metrics were originally developed as a workaround for extremely resource-constrained systems, meaning they skew heavily toward cost reduction at the expense of debuggability. This *can* be the right trade-off, and has historically been the correct approach in eras where data storage and egress were more expensive.

The Inner Workings of Logs

When metrics can’t provide enough context, as developers, we tend to reach for logs. Logs are intuitive for debugging. They’re often one of the first things developers learn. We start with “Hello, world!” and then logs stay with us forever after.

Log files are essentially large blobs of unstructured text, designed to be readable by humans but difficult for machines to process. These files are documents generated by applications and various underlying infrastructure systems that contain a record of all notable events—as defined by a configuration file somewhere—that have occurred.

For decades, logs have been an essential part of system debugging applications in any environment. Logs typically contain tons of useful information: a description of the event that happened, an associated timestamp, a severity-level type associated with that event, and a variety of other relevant metadata (user ID, IP address, etc.).

Traditional logs are *unstructured* because they were designed to be readable by humans. Unfortunately, for purposes of human readability, logs often separate the vivid details of one event across multiple lines of text, like so:

```
6:01:00 accepted connection on port 80 from 10.0.0.3:63349
6:01:03 basic authentication accepted for user: foo
6:01:15 processing request for /super/slow/path
6:01:18 request succeeded, sent response code 200
6:01:19 closed connection to 10.0.0.3:63349
```

You might be used to seeing logs like this in local development, where you usually have a consistent stream of logs for a single test request at a time. Often, we learn the expected order and pattern of the logs and notice when something unusual is happening. While this works well in local development, that approach tends to fall apart when debugging issues in production at scale. Treating the logs as structured data can help us understand why.

We can map these to structured events by taking the timestamp and message separately:

```
{
  "timestamp": "2024",
  "user_id": "630630",
  "action": "checkout_completed",
  [...]
}
```

And we can think of this set of logs on disk as the following table:

timestamp	message
6:01:00	accepted connection on port 80 from 10.0.0.3:63349
6:01:03	basic authentication accepted for user: foo
6:01:15	processing request for /super/slow/path
6:01:18	request succeeded, sent response code 200
6:01:19	closed connection to 10.0.0.3:63349

It would be complicated to create a query to retrieve just this particular set of data. For example, if you want to know every unique user between two time periods, you'd be forced to use a tool like regex (regular expression) to parse each message field:

```
SELECT DISTINCT
  regexp_extract(message, 'authentication accepted for user: (\w+)', 1) AS user
WHERE timestamp BETWEEN '6:00:00' AND '7:00:00'
AND message LIKE 'basic authentication accepted for user: %';
```

While possible, this is also a difficult query for a database to process efficiently. Adding a full text-search index to the `message` field could help eliminate rows that don't match the filters, but you still have the overhead of a relatively expensive regular expression engine per matched row.

Further, modern systems no longer run at an easily comprehensible human scale. In legacy monolithic systems, human operators had a very small number of services to manage. Logs were written to the local disk of the machines where applications ran. In the modern era, logs are often streamed to a centralized aggregator, where they're dumped into very large storage backends.

Searching through millions of lines of unstructured logs can be accomplished by using some type of log file parser. *Parsers* split log data into chunks of information and attempt to group them in meaningful ways. However, with unstructured data, parsing gets complicated because different formatting rules (or no rules at all) exist for different types of log files. Even within the same log types, it's not uncommon to log only specific fields per instance, resulting in further complications in data sequence and consistency. Logging tools are full of different approaches to solving this problem, with varying degrees of success, performance, and usability.

The solution is to instead create *structured* log data designed for machine parsability.

Turning Traditional Logs Into Structured Logs

As structured data, the preceding example might instead look like a set of key-value pairs. Represented as JSON, it would look something like this:

```
{
  "timestamp": "6:01:00",
  "msg": "accepted connection",
  "port": 80,
  "authority": "10.0.0.3:63349"
}
{
  "timestamp": "6:01:03",
  "msg": "basic authentication accepted",
  "user": "alice"
}
{
  "timestamp": "6:01:15",
  "msg": "processing request",
  "path": "/super/slow/path"
}
{
  "timestamp": "6:01:18",
  "msg": "accepted connection",
  "port": 80,
  "authority": "10.0.0.3:63349"
}
```

```
{
  "timestamp": "6:01:19",
  "msg": "closed connection",
  "authority": "10.0.0.3:63349"
}
```

That structure is helpful. If your backend treats these logs as structured data, you end up with a table that looks like this:

timestamp	message	port	authority	user	path	status
6:01:00	accepted connection	80	10.0.0.3:63349			
6:01:03	basic authentication accepted			alice		
6:01:15	processing request				/super/ slow/path	
6:01:18	sent response code					200
6:01:19	closed connection		10.0.0.3:63349			

Now, expressing a query to “count unique users between two time periods” is much less complicated:

```
SELECT DISTINCT user
WHERE timestamp BETWEEN '6:00:00' AND '7:00:00'
AND message = "basic authentication accepted";
```

If your datastore treated the messages as strings, with a special index for keys and values, that would be an improvement over needing to build regular expressions for everything. That can even be an interim step if you already have a mature log-processing system within your organization, but it still leaves much room for further performance optimizations.

While clunky, this shows how you can structure log data and the types of questions you can answer given its shape.

Is a Structured Log the Same as a Structured Event?

At this point, you might find yourself asking: how are structured logs any different from structured events? The simple answer is that they aren't. A nuanced answer comes down to whether your datastore treats them as structured data or as strings.

Tracking a Single Operation

In our examples, we have been looking at logs from a single operation, but any real production system is processing many operations at once. When trying to understand what happened in our system, it helps to see a linear progression of logs, like we might see when developing locally. Many developers stumble onto the same idea: give

each request a unique ID and add it to each log line. That could look something like this:

timestamp	request_id	message	port	authority	user	path	status
6:01:00	7kX9m	accepted connection	80	10.0.0.3:63349			
6:01:03	7kX9m	basic authentication accepted			alice		
6:01:15	7kX9m	processing request				/super/slow/path	
6:01:18	7kX9m	sent response code					200
6:01:19	7kX9m	closed connection		10.0.0.3:63349			

Now we can find all of the events from a specific request with a simple filter:

```
SELECT *  
WHERE request_id = '7kX9m'
```

Distributed tracing adds a few more concepts. A simple mental model of distributed tracing to start with is “a set of events that all share the same ID.” We’ll build on this model next.

The Inner Workings of Distributed Traces

Distributed tracing, or just *tracing*, is simply a way to represent an interrelated series of events. Importantly, tracing also includes a hierarchy and sequence of events within its data model. Unfortunately, for newcomers, tracing can seem like deep magic. OpenTelemetry can seem mysterious and complex (see [Chapter 7](#)).

Many vendors treat instrumentation code like fancy, unknowable magic that gets sprinkled in, and it just “does stuff” to your app: “Install our SDK or agent. Don’t worry about it too much.”

It doesn’t help that in mature, battle-tested tracing libraries, the code itself tends to become cryptic over time, contorting itself to avoid edge cases, work across many environments, and optimize code paths to avoid impacting the performance of its host applications.

So, it’s no wonder that most developers approach tracing libraries as unknowable opaque boxes. We add them to our applications, cross our fingers, and hope they give us useful information when the pager app on our phone goes off at 2 a.m.

But tracing is likely a lot simpler than you'd expect. Let's peel back the layers to demystify tracing into a mental model that looks a lot like “fancy logs” combined with “context propagation” (in other words, “passing some IDs around”).

A Brief Introduction to Distributed Tracing

Since many developers are newer to traces, we'll start with a quick crash course. Let's begin with the name.

Tracing is a fundamental software-debugging technique wherein various bits of information are logged throughout a program's execution for the purpose of diagnosing problems. Since the very first day that two computers were linked together to exchange information, software engineers have been discovering the gremlins lurking within our programs and protocols. Tracing a process from end to end is incredibly effective at flushing them out.

Elusive issues buried in the seams persist despite our best efforts, and in an age when distributed systems are the norm, the tried and true debugging techniques we use have been adapted to meet more complex needs.

Distributed tracing extends that familiar debugging concept to track the progression of a single request—called a *trace*—as it's handled by various services that make up an application. Tracing in this sense is “distributed” because in order to fulfill its functions, a singular request must often traverse process, machine, and network boundaries.

The popularity of microservice architectures has led to a sharp increase in debugging techniques that pinpoint where failures occur along that route and what might be contributing to poor performance. Anytime a request traverses boundaries—such as from on premises to cloud infrastructure, or from infrastructure you control to software as a service (SaaS) that you don't, and back again—distributed tracing can be incredibly useful for diagnosing problems, optimizing code, and improving reliability.

That led to a sharp rise in popularity for distributed tracing. It also meant that several approaches and competing standards emerged to accomplish that task. Despite their design differences, one thing that all distributed tracing implementations share is that they are designed to help you understand system interdependencies.

System interdependencies can obscure problems and make them particularly difficult to debug unless the relationships between them are clearly understood. For example, if a downstream database service experiences performance bottlenecks, that latency can cumulatively stack up. By the time latency is detected three or four layers upstream, identifying which component of the system is the root of the problem can be incredibly difficult because now that same latency is being seen in dozens of other services.

Showing those interdependencies is done, essentially, by interconnecting a series of logs. That approach works well across several services and systems. As it turns out, that approach also works for singular processes running on the same machine; it connects seemingly discrete objects to *the same logical unit of work you wish to understand*.

To show how this works, we'll demonstrate by building a rudimentary tracing library from scratch later in this chapter. But first, let's cover the necessary components needed to form a distributed trace.

The Components of Tracing

To understand the mechanics of how tracing works, we'll use an example to show the various components needed to assemble a trace.

To quickly understand where issues may be happening between interdependencies, it's useful to show waterfall-style visualizations of a trace, as shown in [Figure 5-3](#). Each stage of a request is displayed as an individual chunk in relation to the start time and duration of a request being debugged.



Waterfall visualizations show how an initial value is affected by a series of intermediate values, leading to a final cumulative value.

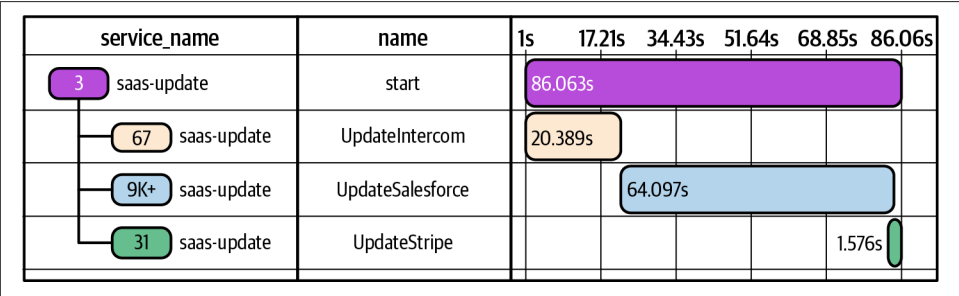


Figure 5-3. This waterfall-style trace visualization displays four trace spans during one request.

Each chunk of this waterfall is called a *trace span*, or *span* for short. Within any given trace, a span is either the *root span*—the top-level span in that trace—or, it is a span that is nested within the root span. Spans nested within the root span may also have nested spans of their own. That relationship is sometimes referred to as *parent-child*. For example, in [Figure 5-4](#), if service A calls service B, which calls service C, then for that trace, span A is the parent of span B, which is in turn the parent of span C.

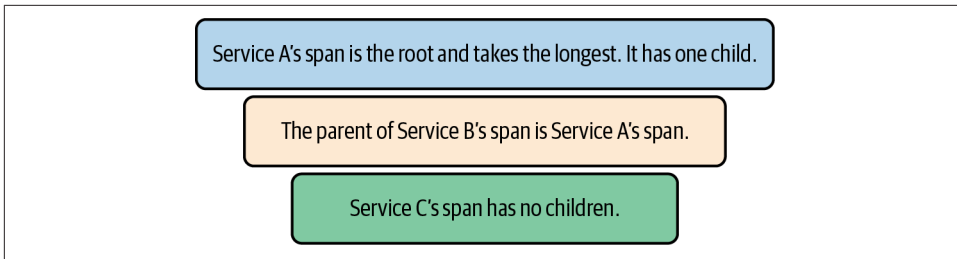


Figure 5-4. A trace that has two parent spans. Span A is the root span and is also the parent of span B. Span B is a child of span A and also the parent of span C. Span C is a child of span B and has no child spans of its own.

Note that a service might be called and appear multiple times within a trace as separate spans, such as in the case of circular dependencies or intense calculations broken into parallel functions within the same service hop (i.e., single network call). In practice, requests often traverse messy and unpredictable paths through a distributed system.

To construct the type of view needed to show any path taken, no matter how complex, we need five pieces of data for each component:

Trace ID

We need a unique identifier for the trace we're about to create so that we can map it back to a particular request. This ID is created by the root span and propagated throughout each step taken to fulfill the request (by passing that same ID to each subsequent span).

Span ID

We also need a unique identifier for each individual span created. Spans contain information captured while a unit of work occurred during a single trace. The unique ID enables us to refer to any particular span within the chain whenever we need it.

Parent ID

This field is used to properly define nesting relationships throughout the life of the trace. A parent span will propagate its ID to any of its children. A parent ID is absent in the root span (that's how we know it's the root).

Timestamp

Each span must indicate when its work began.

Duration

Each span must also record how long that work took to finish.

That's it. Those are all of the fields absolutely required in order to assemble the structure of a trace. With just those fields, you can create an interconnected series of structured logs that show you what happened during any end-to-end transaction.

In distributed traces, you'll typically find a few other helpful fields to identify what these spans are or how they relate to your system. For example:

Service name

For investigative purposes, you'll want to indicate the name of the service (or location) where this work occurred.

Span name

To understand the relevance of each step, it's helpful to give each span a descriptive name to identify or differentiate the work it captures. For example, names could be `intense_computation1` and `intense_computation2` if they represent different work streams within the same service or network hop.

Span attributes

To capture information about the operation beyond just the duration, each span has a collection of key-value pairs. This is where you can capture that a given HTTP request returned an `http.response.status_code` of 200. Any additional information that would go in a log line should be added here.

Pragmatically speaking, just with that data, we should be able to construct the type of waterfall visualization that lets us reason about system interdependencies.

Now, let's assess how structured logs can become a trace.

Turning Logs Into Distributed Traces

To demonstrate how logs can be converted to traces, we'll create a distributed trace from scratch. Let's start from the beginning: a simple log line.

Consistency is useful in logs for searchability. We'll start by ensuring each log has a consistently formatted timestamp or includes a searchable field like `name`:

```
function log(name, attributes) {
  console.log(
    JSON.format({
      timestamp: new Date().getTime(),
      name,
      ...attributes,
    })
  );
}
```

That code outputs a line that looks like the following:

```
{
  "timestamp": 1725845013447,
  "name": "user-authenticated",
  "userId": "1234",
  "remainingRateLimit": 100
}
```

Because we need to include a duration, we'll include a helper function to track the execution time for any given subtask:

```
function logTiming(name, lambda) {
  let startTime = new Date().getTime();

  // run some subtask
  lambda();

  let durationMs = new Date().getTime() - startTime;
  log(name, { durationMs });
}
```

The invocation and output could look like so:

```
logTiming("query-user-info", () => {
  db.fetchUserInfo();
});

// ...

{
  "timestamp": 1725845013447,
  "name": "query-user-info",
  "durationMs": 12
}
```

Congrats! With a name, timestamp, and duration, we're almost halfway to creating trace spans. Let's add the remaining required fields: trace ID, span ID, and a parent span ID.

In practice, spans are only useful when they contain a handful of other fields to describe system state (similar to a structured log). We'll cover those in-depth in the next chapter. For now, you can safely ignore these. Just know that all other fields can be added as additional keys and values in an attributes map.

In code, it could look something like this:

```
let span = new Span("api-request");
let response = await api("get-user-limits");
span.setAttributes({ responseCode: response.code });
span.End();
console.log(span);
```

```
// ...

class Span {
  constructor(name, context = {}, attributes = new Map()) {
    this.startTime = new Date().getTime();
    this.traceID = context.traceID ?? crypto.randomBytes(16).toString("hex");
    this.parentSpanID = context.spanID ?? undefined;
    this.spanID = crypto.randomBytes(8).toString("hex");
    this.name = name;
    this.attributes = attributes;
  }

  setAttributes(keyValues) {
    for (let [key, value] of Object.entries(keyValues)) {
      this.attributes.set(key, value);
    }
  }

  end() {
    this.durationMs = new Date().getTime() - this.startTime;
  }
}
```

The output would be:

```
Span {
  startTime: 1722436476271,
  traceID: 'cfd3fd1ad40f008fea72e06901ff722b',
  parentSpanID: undefined,
  spanID: '6b65f0c5db08556d',
  name: 'api-request',
  attributes: Map(1) {
    "responseCode": 200
  },
  durationMs: 3903
}
```

That could be formatted as an equivalent structured event or log line:

```
{
  "startTime": 1722436476271,
  "traceID": "cfd3fd1ad40f008fea72e06901ff722b",
  "spanID": "6b65f0c5db08556d",
  "name": "api-request",
  "responseCode": 200,
  "durationMs": 3903
}
```



In this example, we generated a root span (we know this because it has an undefined `parentSpanID`). Therefore, we did not carry over that field into our JSON.

As you can see, spans are essentially the same as structured logs when represented in this view. They just contain some fancier data. This is why we have nicknamed traces as “fancy logs.”

Now let’s create the interconnections in our example by generating several child spans.

Traces Are Collections of Spans

So far, this example is very simplistic. Traces are best to see interdependent events. Let’s use our rudimentary library to create a collection of spans that could represent different units of work in one end-to-end transaction.

As covered earlier, interrelated relationships are mapped using span ID, trace ID, and parent span ID. These three different IDs make up a trace context.

The first two are simple. The parent span ID is what allows a system to create a directed acyclic graph (DAG) out of each trace once it has received each individual span. When rendered as a tree, this results in a waterfall view, as shown in [Figure 5-5](#) (but it’s important to remember that waterfall is only one possible visualization of the trace data).

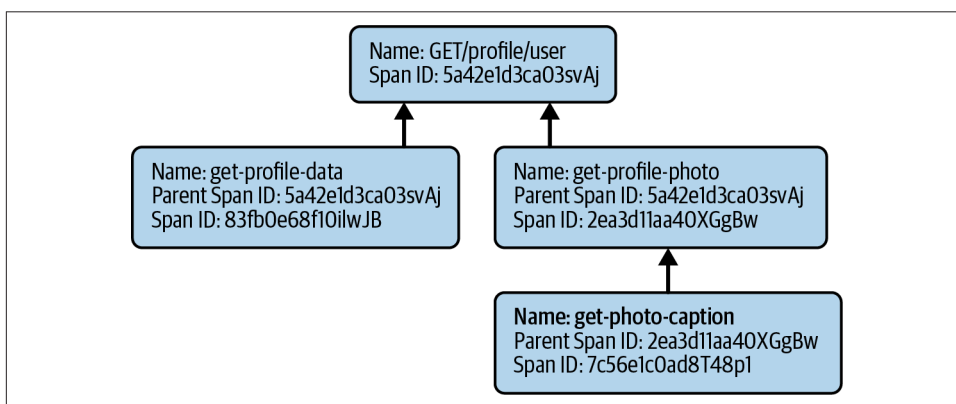


Figure 5-5. The trace parent span (GET /profile/user) has no parent ID. Its two child spans (get-profile-photo and get-profile-data) have the parent ID of the root span. The child of the get-profile-photo span (get-photo-caption) has the parent ID of its parent, which is not the root span.

Trace Context

Trace context is what interconnects all of our “fancy logs.” To do so, our context only really needs to track two values: the trace ID and the ID of the current span. When we create a new span, we can inherit the spanID if one exists, generate a new spanID, and set the new context:

```
let context = {
  traceID: "cfd3fd1ad40f008fea72e06901ff722b",
  spanID: "6b65f0c5db08556d",
};
```

We need some way of passing this context around our application. We could do this manually:

```
let { span, newContext } = new Span("api-req", oldContext);
```

Indeed, this is how it is done in the OpenTelemetry Go SDK. However, in many other languages this is done implicitly and handled automatically by the library. In Ruby or Python, you can use a thread local variable, but in Node.js you would use the [AsyncLocalStorage API](#).

At this point, it helps to wrap our span creation in a helper function:

```
import { AsyncLocalStorage } from "node:async_hooks";

let asyncLocalStorage = new AsyncLocalStorage();
let EMPTY_CONTEXT = {};
asyncLocalStorage.enterWith(EMPTY_CONTEXT);

async function startSpan(name, lambda) {
  let ctx = asyncLocalStorage.getStore();
  let span = new Span(name, ctx, new Map());
  await asyncLocalStorage.run(span.getContext(), lambda, span);
  span.end();
  console.log(span);
}

startSpan("parent", async (parentSpan) => {
  parentSpan.setAttributes({ outerSpan: true });
  startSpan("child", async (childSpan) => {
    childSpan.setAttributes({ outerSpan: false });
  });
});
```

And with that, we have the core of our tracing library done. It would output traces that look like so:

```
Span {
  startTime: 1725850276756,
  traceID: 'b8d002c2f6ae1291e0bd29c9791c9756',
  parentSpanID: '50f527cbf40230c3',
  name: 'child',
  attributes: Map(1) { 'outerSpan' => false },
  spanID: '8037a93b6ed25c3a',
  durationMs: 11.087375000000002
}
```



```
Span {
  startTime: 1725850276744,
  traceID: 'b8d002c2f6ae1291e0bd29c9791c9756',
  parentSpanID: undefined,
  name: 'parent',
  attributes: Map(1) { 'outerSpan' => true },
  spanID: '50f527cbf40230c3',
  durationMs: 26.076625
}
```

As before, these can also be represented as JSON. This example shows how distributed traces are essentially the same as structured logs, except with additional fields that show how the various child spans (or related logs) are connected.

Dynamically Generating the Right System Views

So far, we’ve mostly shown how each fundamental telemetry datatype can be broken down into structured JSON—a.k.a. our “structured event.” But how does that get recomposed into the right view needed for your investigation? Let’s use a concrete example that shows how to dynamically generate a metric from the structured events you’ve captured.

Let’s say our system has a shopping cart service that is instrumented to emit wide, structured events. In this example, our shopping cart service is used by several other services in our stack. Therefore, events for that service are typically written as trace spans. How can we use those events to generate a P99 latency metric time series?

A typical span using this service might look like:

```
{
  "timestamp": "2025-11-08T01:23:45.678Z",
  "service": "shopping-cart",
  "event_name": "transaction_completed",
  "duration_ms": 312.7,
  "status": "ok",

  "http.method": "POST",
  "http.route": "/checkout",
  "http.status_code": 200,

  "user_id": "823ab778",
  "session_id": "s9f2ch74",
  "order_id": "o_1188296",
  "cart_items": 4,
  "payment_provider": "stripe",
  "region": "us-east-1",
  "device_type": "mobile",
  "build_id": "cart-6c1a12",
```

```

    "trace.trace_id": "9b0f2c1g8d2k",
    "trace.span_id": "4a2189fg9cz8",
    "trace.parent_id": "...
  ...
}

```

An event captured in this format contains `duration_ms`, which is the latency we'll measure in our desired metric. Presuming we have captured a number of similar structured events, you can derive a P99 metric time series (i.e., a summary of the hundreds of datapoints that occurred within one particular minute of interest). If our data looks like this example, we could generate P99 latency for all shopping cart transactions into one-minute buckets, with a query like this:

```

WITH cart_tx AS (
  SELECT
    date_trunc('minute', timestamp) AS minute,
    duration_ms
  FROM structured_events
  WHERE service = 'shopping-cart'
    AND event_name = 'transaction_completed'
    AND status = 'ok'
    AND duration_ms IS NOT NULL
)
SELECT
  minute,
  PERCENTILE_DISC(0.99) WITHIN GROUP (ORDER BY duration_ms) AS p99_ms
FROM cart_tx
GROUP BY minute
ORDER BY minute;

```

Note that this event also contains other high-context fields like `user`, `region`, `build`, `device`, etc. (which we'll explore further in [Chapter 6](#)). These attributes are preserved within the event so that, later, we can slice the P99 by any other desired dimension without reinstrumenting.

Let's say we want to see P99 by region (or any other dimension). We don't need to have already captured a metric that includes those dimensions. Instead, we simply add it to `SELECT` and `GROUP BY` without any schema changes needed:

```

SELECT
  date_trunc('minute', timestamp) AS minute,
  region,
  PERCENTILE_DISC(0.99) WITHIN GROUP (ORDER BY duration_ms) AS p99_ms
FROM structured_events
WHERE service='shopping-cart'
  AND event_name='transaction_completed'
  AND status='ok'
GROUP BY minute, region
ORDER BY minute, region;

```

If you needed to refresh P99s continuously, you could do things like keep a streaming quantile sketch per time bucket (and optional dimension keys). At this point, more sophisticated views simply become a data-processing problem. Building on the streaming example, you could hold short grace periods to catch late events, before finalizing the aggregation. You could adjust the quantiles for further small-grained accuracy, using t-digest or HDR histograms. Or you could do anything else you need, depending on situational requirements.

Why this approach works is that it lets you decide, during the course of any given investigation, what data shape you need to answer your questions. You didn't store an aggregated metric and discard any of the additional context; rather, you derived a metric from a complete set of facts. Instead of pre-deciding the data shape and boxing yourself into pre-deciding the investigation, you capture an entire set of facts and post-decide the necessary shape.

From a single, wide, structured event, the P99 metric is just one of many possible data shapes you could choose to answer today's question without limiting tomorrow's. For example, you could:

- Recompute P99 with different windows (5 minutes, 1 hour), filters (only `device_type='mobile'`), or conditions (`http.status_code >= 500`), with no telemetry configuration deployment or re-ingestion required.
- Derive other views from the same events: a log-like narrative (`SELECT time stamp, user_id, duration_ms, http.route ...`), or a trace by stitching on `trace.*_id`.
- Avoid the classic metric pitfalls of pre-aggregation, which discards the very fields you'll need later to explain why P99 got worse.

This example span and accompanying queries are simplistic, but already you can see the benefits that this approach unlocks. Throughout further chapters of this book, we'll build on these examples.

Conclusion

To understand the basic building block for unified observability, we unraveled the structure, commonalities, and limitations of the three common telemetry types—logs, metrics, and traces.

Metrics are quick, cheap, and easy because they aggregate by design. That's also why they're lossy and narrow. Once you've averaged, bucketed, or rolled up the results, you can't later reconstruct any data that you didn't keep. That makes certain questions unanswerable after they've been written into the metric format.

Logs, originally optimized for human readability, contain a narrative. At scale, however, that story can quickly become a congressional library of unindexed prose: full of extraneous data, expensive to sift, and brittle to parse. Structured logs solve that problem, but still often only show a fragmented snapshot for any given unit of work. Traces can provide the missing connective tissue, because they're simply interrelated logs with context propagated to show how one unit of work fans out across various services.

If your experience in software development has revolved around the legacy three pillars observability model, consider this chapter a gentle but firm nudge to reconsider the limitations it imposes by segmenting views. The shape of the data you collect constrains the questions you can ask later. Unifying datatypes by decomposing metrics, logs, and traces into arbitrarily wide, structured events removes those constraints.

The arbitrarily wide, structured event captures all relevant context for a unit of work (inputs, discovered context, conditions, outcomes, etc.) so that, after the fact, you can slice along any dimension, aggregate them into summary time series, go back to read the narrative in any given log, or stitch them into an end-to-end transaction. It's our view that divisions that exist as historical artifacts shouldn't dictate how we instrument or investigate. The best representation of system state for any future investigation is one that allows data to be reshaped on demand. Unified observability removes historical limitations so that your immediate questions, not your data model's past, are what sets the boundaries of what you can learn.

In this chapter, we covered the “wide, structured event” part of the building block. But why is it “arbitrarily” wide? In the next chapter, we look at concrete examples of all the context-rich data that you can and should capture inside your structured events.

Making Structured Events Arbitrarily Wide

*This chapter is contributed by Jeremy Morrell,
Principal Engineer at Cloudflare*

A Note from Charity, Liz, George, and Austin

In the last chapter, we broke down how metrics, logs, and traces can all be decomposed into one common datatype: the arbitrarily wide, structured event—which can be used to generate any view necessary during observability-based investigations. However, we mostly focused on showing examples that illustrate the “structured event” part of that datatype. In this chapter, we show you how to make those structured events “arbitrarily wide.”

In observability, your structured events should be as wide as possible in order to capture comprehensive context for debugging. Having rich context about any given system event enables you to answer unexpected questions and perform complex analysis without prejoining data. Some of the most useful data when investigating novel or complex failures will typically involve comparing outliers across fields with high cardinality.

Wide, structured events collapse the boundaries between logs, metrics, and traces into a single data structure, offering a unified view for analyzing system behavior. That means proactively designing your events so wide that they include fields to help pinpoint issues such as user ID, feature flags, and request details in a single data structure, eliminating the need for expensive and complex joins.

An “arbitrarily wide” structured event means that there should be no practical limit to how much context can be stuffed into any single event. In a unified observability system, you should have as much data available in your events to aid your investigation as you think you might someday need. In practice, that means that any given arbitrarily wide, structured event contains data that has both high dimensionality and high cardinality.

Later, we'll examine practical constraints as we delve into analysis ([Chapter 8](#)) and datastore workloads ([Chapters 12 and 13](#)). For now, let's focus on what type of instrumentation you should include to make the widest, most useful events possible.

The rest of this chapter is written from Jeremy Morrell's perspective.

In order to answer any arbitrary question, you'll need to capture information about a wide variety of conditions present in your systems. In the previous chapter, we introduced simple examples structured around web server requests. Since our goal was to show how telemetry datatypes are interrelated, we glossed over what all the other useful attributes to capture would be.

You'll capture data about system state when the request occurred by writing it to various attributes. When considering what to capture, it's relatively simple to come up with a dozen or so attributes. But there are likely far more of these than you would expect. In a well-instrumented codebase, there will typically be hundreds of attributes.

In this chapter, I'll walk you through that wide variety of system conditions and concretely illustrate the types of values you should capture in each case. Note that while this section contains a long list of attributes, it is definitely not exhaustive.

Service and Code Context

As you consider which attributes to capture, service and code context are good places to start. These attributes are what turn questions like “Is something broken?” into answers like “We know exactly which thing is broken, where, who's responsible for it, and what changed.”

Service Metadata

Consider adding metadata about which team owns the system, or which Slack channel the owning team hangs out in—though note that this can be tedious to update if your workplace experiences frequent re-orgs. Sophisticated organizations can link the data in these attributes to a service catalog so they always stay up-to-date. [Table 6-1](#) lists attributes that are useful for connecting services to their owners.

Table 6-1. Service attributes

Attribute	Examples	Description
service.name	api shoppingcart	Name of this service
service.environment	production staging development	Environment where this service is running
service.team	web-services dev-ex	Team that owns this service—useful for knowing who to page in during incidents
service.slack_channel	web-services dev-ex	Where to reach out if you discover an issue

With these attributes in place, you can answer questions like: how many services does each team run? You can generate an answer using the following query:

```
SELECT
  COUNT_DISTINCT(service.name)
WHERE
  service.environment = "production"
GROUP BY service.team
```

Next, have you ever looked at the load on a system and then wondered “Is that appropriate for the machine this is running on?” And then you have to look through other tools or config files to get that information. Well, you can avoid that by throwing that context on the wide event so that it’s available when you need it.

Table 6-2 lists examples of infrastructure metadata that is useful to capture.

Table 6-2. Infrastructure attributes

Attribute	Examples	Description
instance.id	656993bd-40e1-4c76-baff-0e50e158c6eb	ID that maps to this one instance of the service
instance.memory_mb	12336	Amount of RAM available to this service
instance.cpu_count	4 8 196	Number of cores available to this service
instance.type	m6i.xlarge	Vendor’s name for this type of instance

With these attributes, you can use the following query to answer questions like “What are the services with the most memory that we run?” and “What instance types do they use?”:

```

SELECT
    service.name,
    instance.memory_mb,
    instance.type
ORDER BY instance.memory_mb DESC
GROUP BY service.name, instance.type
LIMIT 10

```

Going a step further, you should add all relevant information related to how you’re orchestrating your systems. The OpenTelemetry [Kubernetes semantic conventions](#) for instrumentation do a good job of laying these out, but I’ve included the examples in [Table 6-3](#) for your inspiration.

Table 6-3. Orchestration attributes

Attribute	Examples	Description
container.id	a3bf90e006b2	ID used to identify Docker containers
container.name	nginx-proxy wordpress-app	Container name used by container runtime
k8s.cluster.name	api-cluster	Name of the Kubernetes cluster your service runs in
k8s.pod.name	nginx-2723453542-065rx	Name of the Kubernetes pod your service runs in
cloud.availability_zone	us-east-1c	Availability zone where you’re running your service
cloud.region	us-east-1	Region where you’re running your service

The preceding examples presume an orchestrator like Kubernetes. But even if you’re using a platform as a service (PaaS), you can still pull out a lot of useful information. This is shown in [Table 6-4](#), using Heroku as an example.

Table 6-4. PaaS orchestration attributes

Attribute	Examples	Description
heroku.dyno	web.1 worker.3	Environment variable (env var) DYNO that is set on your app at runtime
heroku.dyno_type	web worker	First part of the DYNO env var before the dot (easier to query when separated)
heroku.dyno_index	1 3	Second part of the DYNO env var after the dot (easier to query when separated)
heroku.dyno_size	performance-m	Selected dyno size
heroku.space	my-private-space	Name of the private space that you are deployed into
heroku.region	virginia oregon	Region the app is located in

You can use the following query to answer questions like “How many Heroku dynos are we running?” and “What dyno types are they; for which services?”:

```
SELECT
  COUNT_DISTINCT(heroku.dyno_index)
GROUP BY service.name, heroku.dyno_type, instance.type
```

These are just a start, of course. But hopefully, using this as inspiration, you can see what other details might be useful for you and your teams to capture.

Build Information

Inevitably some of the first questions asked in any incident are “Did something just get deployed,” or “What changed?” Instead of jumping to your deployment tool or looking through GitHub repositories, you can add that build data to your telemetry.

Threading this data from your build system through to your production system so that it’s available at runtime can be a nontrivial amount of glue code, but having this information easily available during incidents is invaluable.

Table 6-5 shows a few examples of useful build information.

Table 6-5. Build information attributes

Attribute	Examples	Description
service.version	v123 9731945429d3d083eb78666c565c61bcef39a48f	Version string or a hash of the built image
service.build.id	acd8bb57-fb9f-4b2d-a750-4315e99dac64	ID given by your build system, with this context enabling you to audit the build if something goes wrong
service.build.git_hash	6f6466b0e693470729b669f3745358df29f97e8d	Git hash of the deployed commit so you can know exactly which code was running
service.build.pull_request_url	https://github.com/your-company/api-service/pull/121	URL of the pull request that was merged that triggered the deploy
service.build.diff_url	https://github.com/your-company/api-service/compare/c9d9380..05e5736	URL that compares the previously deployed commit against the newly deployed commit
service.build.deployment.at	2024-10-14T19:47:38Z	Timestamp when the deployment process started
service.build.deployment.user	keanu.reeves@your-company.com	Authenticated user who kicked off the build

Attribute	Examples	Description
service.build. deployment. trigger	merge-to-main slack-bot api-request config-change	Triggering event that deployed this version of the application—enabling quick distinction between an issue caused by a configuration change, an automatic deploy, or a manual deployment
service.build. deployment.age _minutes	1 10230	Age of the deployment—useful for the frequent incident question: “Did something just get deployed?” (age presumably updated each time telemetry is emitted)

With even just these few attributes, you can query answers to a question like “What systems have recently been deployed?” For example:

```
SELECT
  service.name,
  MIN(service.build.deployment.age_minutes) as age
WHERE
  service.build.deployment.age_minutes < 20
GROUP BY service.name
ORDER BY age ASC
LIMIT 10
```

Or, you could query to find out “What’s up with the spike of 500s when we did the last deploy?”:

```
SELECT
  COUNT(*)
WHERE
  service.name = "api-service" AND
  main = true
GROUP BY http.status_code, service.version
```

If the query was used to visualize performance, your graph might look something like [Figure 6-1](#).

The visualization shows that HTTP 200 OK requests served by v1 dropped quickly as v2 started to serve traffic. While that was happening, HTTP 500 Internal Server Error requests briefly spiked. Without being able to break down by version, it’s unclear if the 500 errors are due to side effects of bringing the v2 server up, or issues shutting the v1 server down. In this case, all the 500 errors are from v1, so we should make sure our system is properly handling all ongoing requests and can shut down cleanly.

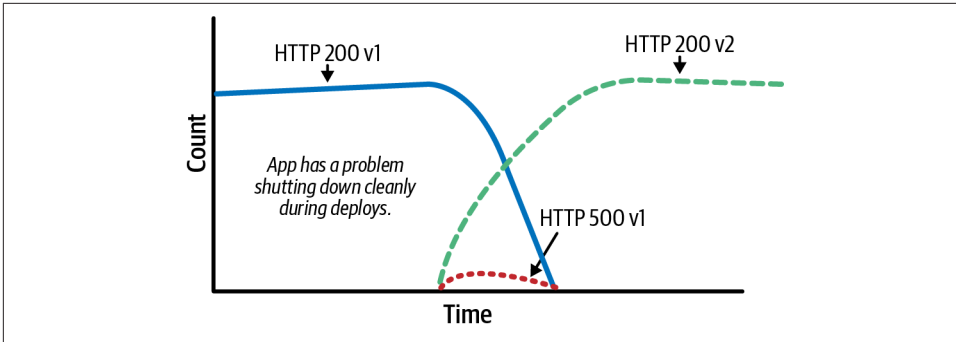


Figure 6-1. Requests grouped by HTTP status code and version. There is a spike of 500s correlating to v1 shutting down.

Feature Flags

Fine-grained feature flags are a developer superpower. They enable you to test code changes in production with only a fraction of your users or traffic. Coupled with the broad visibility of wide events (as we'll continue to see), the attributes in [Table 6-6](#) can make even tricky migrations vastly more manageable and enable you to ship code with confidence.

Table 6-6. Feature flag attributes

Attribute	Examples	Description
feature_flag.auth_v2	true false	Value of a particular feature flag for this request
feature_flag.double_write_to_new_db	true false	Value of a particular feature flag for this request

Adding these feature flag attributes would let you query an answer to “What errors are the users in the new authentication flows hitting?” and “How does it compare to the control group?”:

```
SELECT
  COUNT
WHERE
  main = true AND
  service.name = "api-service" AND
GROUP BY feature_flag.auth_v2, exception.slug
```

Adding the flag information per request enables you to compare how the new code is working as you opt more of your traffic into the new code path.

Versions of Important Things

Collecting the versions of runtimes, frameworks, and any major libraries you are using can provide really helpful context. Some examples are contained in [Table 6-7](#).

Table 6-7. Versioning attributes

Attribute	Examples	Description
go.version	go1.26.4	Version of the language runtime you are using
rails.version	7.2.1.1	Picking out any core libraries like web frameworks and tracking their version
postgres.version	16.4	Adding the versions of any datastores you're using

Now, let's say that a security issue with Ruby on Rails just got announced. You can now quickly find out what versions of the framework your services are using:

```
SELECT
  COUNT_DISTINCT(service.name)
WHERE
  service.environment = "production"
GROUP BY rails.version
```

Or, let's say that, suddenly, memory usage seems higher than it used to be. Didn't we upgrade Go versions recently? Does that correlate? Run a query to find out:

```
SELECT
  HEATMAP(metrics.memory_mb)
WHERE
  main = true AND
  service.name = "api-service"
GROUP BY go.version
```

There are many useful questions you can answer when system conditions change. But you can't answer them if you didn't capture the version of important system components. What other versioning information could be critical to understanding your system operations? Add that to your wide events. (To further see how this is enabled in practice, see [Chapter 8](#).)

Request and Execution Flow

Telemetry about request and execution flow is what turns a single trace or log line into a coherent narrative of how work moved through the system: what was called, in what order, how long each step took, and where it failed. It's the fastest way to go from "the service is slow" to "this route is slow because this downstream call is slow." Focusing on these attributes is how you make finding answers like that repeatable at scale.

HTTP Information

Although you should get most attributes related to HTTP information from your tracing library instrumentation, there are usually more that you can add. **Table 6-8** shows some common places to start.

Table 6-8. HTTP information attributes

Attribute	Examples	Description
<code>server.address</code>	<code>example.com</code> <code>localhost</code>	Host name of the HTTP server that received the request
<code>url.path</code>	<code>/checkout</code> <code>/account/123/features</code>	URI path after the domain
<code>url.scheme</code>	<code>http</code> , <code>https</code>	URI scheme being used
<code>url.query</code>	<code>q=test</code> , <code>ref=####</code>	URI query component
<code>http.request.id</code>	<code>79104EXAMPLEB723</code>	Platform requestID if one exists—e.g., <code>x-request-id</code> , <code>x-amz-request-id</code> ^a
<code>http.request.method</code>	<code>GET</code> <code>PUT</code> <code>POST</code> <code>OPTIONS</code>	HTTP request method
<code>http.request.body_size</code>	<code>3495</code>	Size of the request payload body, in bytes
<code>http.request.header.content-type</code>	<code>application/json</code>	Value of a specific request header— <code>content-type</code> in this case, though there are many more to choose if any are important for your service
<code>http.response.status_code</code>	<code>200</code> <code>404</code> <code>500</code>	HTTP response status code
<code>http.response.body_size</code>	<code>1284</code> <code>2202009</code>	Size of the response payload body, in bytes
<code>http.response.header.content-type</code>	<code>text/html</code>	Value of a specific response header— <code>content-type</code> in this case, though there are many more to choose if any are important for your service

^a The `x-` header prefix format is currently well entrenched in many orgs, although **the convention has been deprecated**.

With these attributes in place, you can now understand more about the performance of your HTTP requests. For example, you can answer a question like “Are there differences in the output size of my HTTP responses?”:

```
SELECT
  HEATMAP(http.response.body_size),
WHERE
  main = true AND
  service.name = "api-service"
```

The visualization in [Figure 6-2](#) shows a graphical representation of data received by the example query.

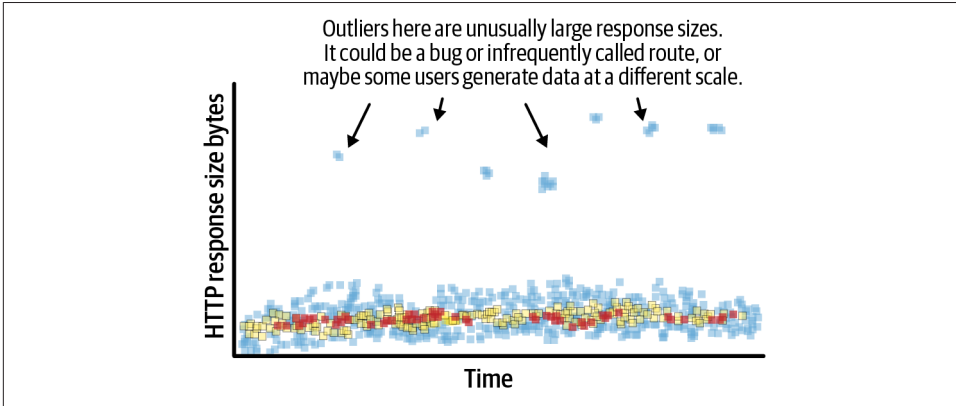


Figure 6-2. Heatmap of response sizes: most are within a fixed band, but there are sharp outliers that warrant more investigation.

Here, you can see that the body size of responses from some HTTP requests are abnormally large. Those outliers might provide useful clues when investigating performance anomalies. They could be a bug or they could be valid data—either from infrequently called routes or user requests returning a large dataset.

HTTP User-Agent headers contain a wealth of information. Don't rely on regex queries to try and make sense of them down the road. With wide events, there's no reason not to parse them into structured data from the beginning. [Table 6-9](#) provides useful user-agent metadata to capture.

Table 6-9. User-agent attributes

Attribute	Examples	Description
user_agent.original	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0 Safari/537.3	Value of the HTTP User-Agent header
user_agent.device	computer tablet phone	Device type derived from the User-Agent header

Attribute	Examples	Description
user_agent.OS	Windows MacOS	Operating system derived from the User-Agent header
user_agent.browser	Chrome Safari Firefox	Browser derived from the User-Agent header
user_agent.browser_version	129 18.0	Browser version derived from the User-Agent header

Now you can query answers to a question like “What browsers are my users using?”:

```
SELECT
  COUNT(*)
  GROUP BY user_agent.browser, user_agent.browser_version
```

If you have any custom user agents or headers used as a convention within your org, parse that out too, as shown in [Table 6-10](#).

Table 6-10. Custom user agent or custom header attributes

Attribute	Examples	Description
user_agent.service	api-gateway auth-service	If you have a distributed architecture, have each service send a custom User-Agent header with its name.
user_agent.service_version	v123 6f6466b0e693470729b669f3745358df29f97e8d	If you have a distributed architecture, have each service send a custom User-Agent header with its version.
user_agent.app	iOS android	If a request is coming from a mobile app, make sure it includes which app.
user_agent.app_version	v123 6f6466b0e693470729b669f3745358df29f97e8d	If a request is coming from a mobile app, make sure it includes which version is in use.

Again, you’ll get many of these types of attributes from standard instrumentation libraries automatically. But you also might not, depending on what you use. For example, if your organization uses nonstandard headers, you may need to add some of these manually.

Always review which attributes you get with automatic instrumentation. If you aren’t capturing telemetry for some of these useful scenarios, don’t hesitate to add it. Don’t settle for only what your instrumentation library gives you by default!

Route Information

We’re not done with HTTP attributes yet! One of the most important bits is the API endpoint that the request matched. Tracing libraries will *usually* give this to you automatically but not always. Consider extracting the route parameters and query parameters as additional attributes, as shown in [Table 6-11](#).

Table 6-11. Route information attributes

Attribute	Examples	Description
http.route	/team/ {team_id}/ user/{user_id}	Route pattern that the URL path is matched against
http.route.param.team_id	14739 team-name-slug	Extracted segment of the URL path as it’s parsed for each parameter
http.route.query.sort_dir	asc	Query parameters that are relevant to the response of your service (e.g., ?sort_dir=asc&...)

Adding route information to your attributes lets you better dig for underlying patterns within groups of HTTP requests. Let’s say you’re looking at P99 latency. You could use the following query to group a visualization of those requests to also show their routes:

```
SELECT
  P99(duration_ms)
WHERE
  main = true AND
  service.name = "api-service"
GROUP BY http.route
```

In [Figure 6-3](#), we see a visualization of P99 latency.

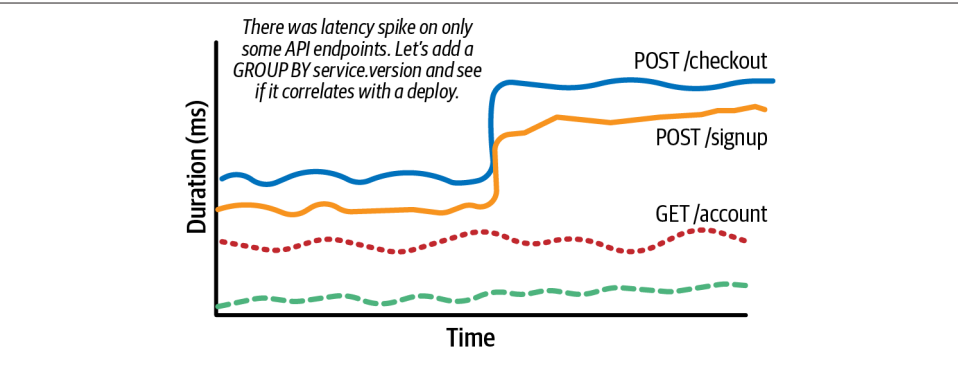


Figure 6-3. Chart of P99’s broken down by route. There is a spike on only some of them. Break down this view by application version to check if this was caused by a deployment.

Without route information, you'd only see a spike occurring in requests, but you'd have no further information on what those slow requests have in common. By grouping requests according to HTTP route, in this example, you can see that only requests to the `POST /checkout` and `POST /signup` routes are slower.

Timings

In the work that gets done to respond to a request, I find it super useful to break it up into a handful of important chunks and track how long each segment took on the wide event.

You might be asking yourself, “Wait, isn’t that what child spans are for?”

Wrapping absolutely everything in its own span is the most common failure mode I see when engineers first get access to tracing tools. You have to design the structure of your data for the way you want to query it.

Child spans are helpful for waterfall visualization for a single request, but can be difficult to query and visualize across *all* of your requests. Putting important timing information on a single span makes it easier to query and also helps with tools like Honeycomb’s BubbleUp (see [Chapter 8](#)), which can then immediately tell you that a particular group of requests was slow because authentication took 10 seconds for some reason.

An example of timing information to capture is shown in [Table 6-12](#).

Table 6-12. Timing attributes

Attribute	Examples	Description
auth.duration_ms	52.2 0.2	How long we spent performing authentication during this request
payload_parse. duration_ms	22.1 0.1	Identifying the core workloads of the service and adding timings for them

Let’s say in your system you’ve just shipped a new subsystem that prioritizes payload parsing for enterprise users, and you want to see the impact of that change on tail latencies across all of the regions where you have systems deployed. If all of those attributes are present on the wide event, then this is straightforward:

```
SELECT
  P99(payload_parse.duration_ms)
WHERE
  main = true AND
  service.name = "api-service"
GROUP BY
  user.type,
  cloud.region
```

However, this may seem like we are duplicating data available in the child spans. Surely we can accomplish the same thing by wrapping the payload parsing method in its own span?

This is a valid approach, but now if we want to query that data alongside any other data that we've captured in our wide event, querying has become much more complicated, forcing the use of JOIN:

```
SELECT
  P99(parse_span.duration_ms)
FROM spans AS main_span
JOIN spans AS parse_span ON main_span.trace_id = parse_span.trace_id
WHERE
  main_span.main = true AND
  main_span.service.name = "api-service" AND
  parse_span.name = "payload-parse"
GROUP BY main_span.user.type, main_span.cloud.region
```

Mature observability tooling is capable of running these types of queries at the cost of additional processing; however, we should keep in mind how we will be using this data. We want to prioritize quick, iterative exploration, often while responding to active incidents, and we want any engineer on our team to be able to easily navigate our observability tooling.

Viewed through this lens, identifying a few important timings and adding them to the wide event is well worth the slight data duplication.

Async Request Summaries

When using a tracing system, async requests should get their own spans, but it can still be useful to roll up some stats to identify outliers and quickly find interesting traces. Some examples of summarizing these into attributes are shown in [Table 6-13](#).

Table 6-13. Async request attributes

Attribute	Examples	Description
stats.http_requests_count	1 140	Number of HTTP requests triggered during the processing of this request
stats.http_requests_duration_ms	849	Cumulative time spent on these HTTP requests
stats.postgres_query_count	7 742	Number of Postgres queries triggered during the processing of this request
stats.postgres_query_duration_ms	1254	Cumulative time spent in these Postgres queries
stats.redis_query_count	3 240	Number of Redis queries triggered during the processing of this request
stats.redis_query_duration_ms	43	Cumulative time spent on these Redis queries

Attribute	Examples	Description
stats.twilio_calls_count	1 4	Number of calls to this vendor's API triggered during the processing of this request
stats.twilio_calls_duration_ms	2153	Cumulative time spent on these vendor calls

Now, you can write the following query to see exactly how much traffic is going to which sources:

```
SELECT
  HEATMAP(stats.postgres_query_count)
WHERE
  main = true AND
  service.name = "api-service"
```

Surely my service makes a reasonable number of calls to the database...right?

In [Figure 6-4](#), we see the results of this example query.

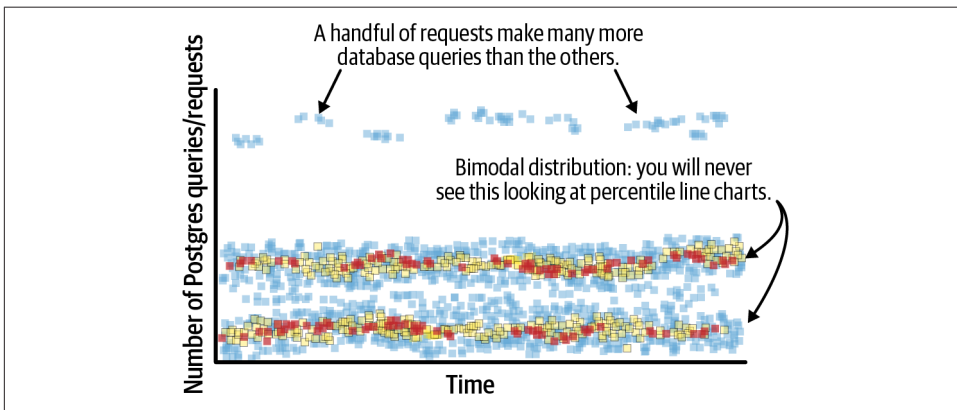


Figure 6-4. Heatmap of database queries per request: there is a bimodal distribution but also some outliers that make a lot of requests.

The visualization, shown as a heatmap, displays a bimodal distribution, where a majority of requests follow one of two patterns for database communications. However, you'll also notice that a handful of requests consume far more traffic than is nominal.

Errors

If you encounter an error and need to fail the operation, add attributes to the span that contain the error information: type, stack trace, etc.

One approach that I have found incredibly valuable is tagging each location, where we throw an error with a unique slug describing the error. If this string is unique within your codebase, it's easily found with a quick search. This enables you to jump

straight from a spike in errors on a dashboard to the exact line of code throwing the error. It also provides a convenient low-cardinality field to GROUP BY.

You're unlikely to be able to wrap all possible errors, but any time a failed request doesn't have an `exception.slug`, that is a good sign that you have places in your code where your error handling could be improved. It's now really easy to find examples of requests that failed in ways you didn't anticipate.

You could add error handling into your instrumentation code like so:

```
if isNotRecoverable(err) {
  // note the use of a plain string, not a variable
  // not dynamically generated
  // consider enforcing this with custom lint rules
  setErrorAttributes(err, "err-stripe-call-failed-exhausted-retries");
  throw err;
}
```

Table 6-14 shows various types of error information to capture in attributes.

Table 6-14. Error information attributes

Attribute	Examples	Description
error	true false	Special field for whether the request failed or not
exception.message	Can't convert 'int' object to str undefined is not a function	Exception message encoded in the exception
exception.type	IOError java.net.ConnectEx ception	Programmatic type of the exception
exception.stacktrace	ReferenceError: user is not defined at myFunction (/path/to/ file.js:12:2) ...	Capturing the stack trace if it's available to help pinpoint where the error is being thrown
exception.expected	true false	Enables filtering out of exceptions we can't prevent but don't need to worry about (e.g., a bot trying to hit a URL that doesn't exist)
exception.slug	auth-error invalid-route github-api-unavailable	Creating a unique, greppable slug value to identify the code location of an error if it's predictable at development time

With error information in your wide events, you can query for answers to questions like “Which of our enterprise users hit the most errors last week, and which errors did they hit?” For example:

```
SELECT
  COUNT_DISTINCT(user.id)
WHERE
  main = true AND
  service.name = "api-service" AND
  user.type = "enterprise"
GROUP BY exception.slug
```

Or, you can use the following query when you want to see traces where you likely need to improve your error handling:

```
SELECT
  trace.trace_id
WHERE
  main = true AND
  service.name = "api-service" AND
  error = true AND
  exception.slug = NULL
GROUP BY trace.trace_id
```

While tracing libraries provide solid defaults for many of these attributes, don’t stop there. Review what your instrumentation captures automatically, identify the gaps, and fill them in. The difference between “something is slow” and “the /checkout route is slow because authentication is taking 10 seconds for requests from mobile clients on version 3.2.8” is often just a handful of well-chosen attributes that you added yourself.

User and Business Context

Telemetry about user and business context is what lets you connect technical behavior to real-world impact: who was affected, what they were trying to do, and whether the issue maps to a specific tenant, segment, plan, workflow, etc. It turns debugging into prioritization because you can distinguish a “weird edge case” from a “revenue-critical path breaking for high-value customers.”

User and Customer Information

Once you get the basics down, this is *the most important* piece of metadata that you can add. No automagic SDK will be able to automatically understand the particular details of your user model.

First, let's cover types of user information to add, as shown in [Table 6-15](#).

Table 6-15. User and customer attributes

Attribute	Examples	Description
<code>user.id</code>	2147483647 user@example.com	This is the primary ID for a user. If this is an email and you're using a vendor, consider your org's policy on putting personally identifiable information (PII) in external services.
<code>user.type</code>	free premium enterprise vip	Businesses usually have a way of categorizing their users into groups or users that might have outsized importance to business fundamentals. Individual accounts are sometimes responsible for 10%+ of a business income or for an inordinate amount of system load. Make sure you can separate their traffic from others!
<code>user.auth_method</code>	token basic-auth jwt sso-github	This is the method the user used to authenticate into your system.
<code>user.team.id</code>	5387 web-services	Your system may have a way of grouping users together such as "team" or "organization." Capture attributes for all of these. Sometimes teams will hit errors or edge cases that affect no other users.
<code>user.org.id</code>	278 enterprise-name	If this user is part of an organization with an enterprise contract, track that! Identifying high-spend customers affected by an incident can enable your account managers to proactively reach out to them.
<code>user.creation_timestamp</code>	1753833600 1722307200	This is a timestamp of when this account was created. Using this timestamp enables you to distinguish between issues experienced by someone new to your app, or only once they've been around long enough to have saved a lot of data.
<code>user.assumed</code>	true	If you have an internal way of assuming a user's identity for debugging, be sure to track this. This is also likely compulsory to meet security audit requirements.
<code>user.assumed_by</code>	engineer-3@your-company.com	Also track which actual user is assuming the user's identity.

Adding user information into your wide events is what lets you segment investigative queries to see who exactly is affected by any particular issue. Understanding exactly which users are experiencing a problem is generally one of the first questions that incident responders will be asked, yet that continues to be a frustratingly difficult problem for most engineering teams.

But now, with user information in your wide events, you could write a query to understand which users are seeing slow performance when using your services:

```
SELECT
  P99(duration_ms)
WHERE
  main = true AND
  service.name = "api-service"
GROUP BY user.type
```

It’s common for a single user or account to be responsible for a 10%+ of business revenue, and frequently their usage patterns look significantly different than the average user. They probably have more users, store more data, use enterprise-only features, and hit limits and edge cases that will never show up for the user paying \$10 a month.

Be sure you can separate their traffic from others, by adding user attributes to your wide events!

Rate Limits

Whatever your rate-limiting strategy, make sure the current rate-limit information gets added as attributes too. With many tools, it’s unlikely that you can quickly find examples of users that are being rate-limited by your service. You can get around that limitation by adding attributes like what is shown in [Table 6-16](#).

Table 6-16. Rate-limit attributes

Attribute	Examples	Description
ratelimit.limit	200000	This is the limit being enforced by the rate-limiting system. Young systems may only have a single rate limit for all users, but as they mature there will likely be business reasons to have users with different rate limits.
ratelimit.remaining	130000	This is the budget remaining for this user.
ratelimit.used	70000	This is the budget for the current window that has been used.
ratelimit.reset_at	2024-10-14T19:47:38Z	This is the time when the rate limit will be reset. Different rate-limiting algorithms will have different parameters. Try to capture anything that would be helpful if you find yourself talking with an angry customer asking why they are being rate-limited.

Let's say that a particularly important user has opened a support ticket about being rate-limited. Rather than guessing at what might be happening, if you had these attributes in place, you could query directly to find out. You could see what the user was doing with a query like this:

```
SELECT
  COUNT(*),
  HEATMAP(ratelimit.remaining)
WHERE
  main = true AND
  service.name = "api-service" AND
  user.id = 5838
GROUP BY http.route
```

Figure 6-5 is a visualization of this query.

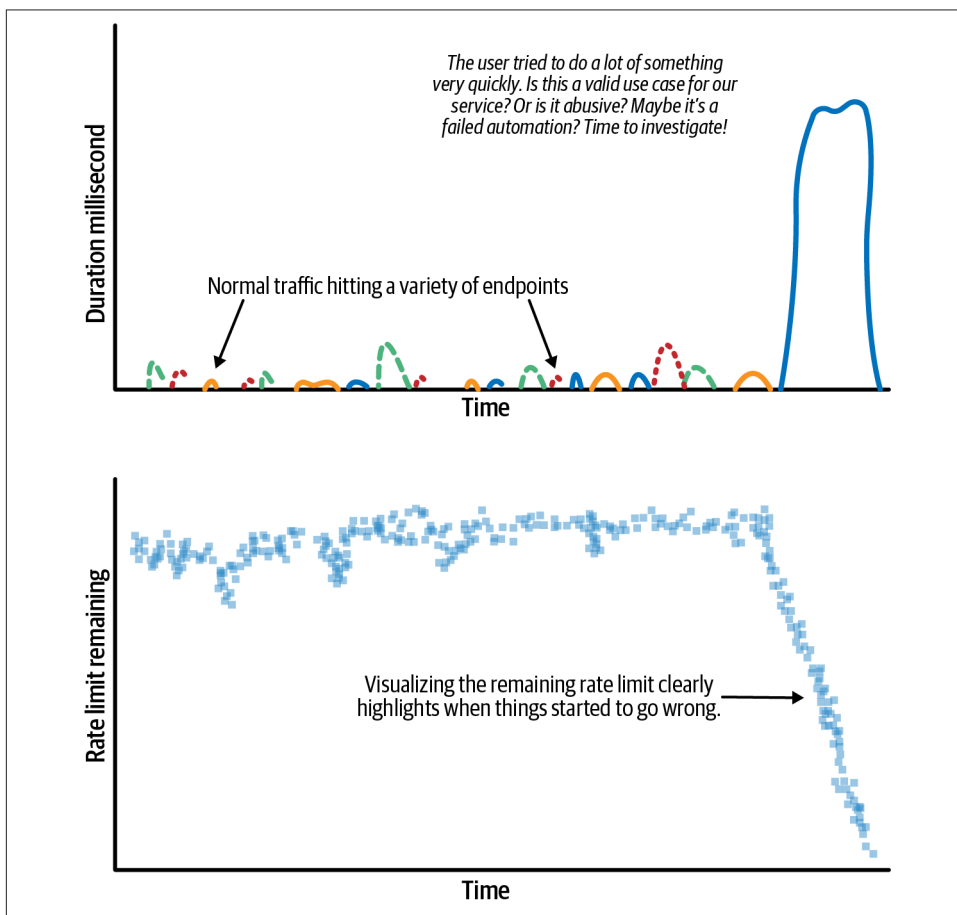


Figure 6-5. A graph of one user's activity: on the far right, there is a big spike hitting the same route. This graph provides a starting point for investigation.

In this case, we can now clearly see that the user’s behavior changed suddenly, hammering one endpoint until the quota was exhausted. This could be some automation the user is building with a bug, or maybe their authentication credentials have been leaked and now someone is spoofing their activity. Digging into metadata like the route being hit, the HTTP user agent, and the source IP should help quickly clarify what is happening here.

Caching

For every code path where we could shortcut with a cache response, add attributes to your wide events that record whether or not the caching operation was successful. These are relatively straightforward, as shown in [Table 6-17](#).

Table 6-17. Caching attributes

Attribute	Examples	Description
cache.session_info	true false	We might have session information in an in-memory cache or key-value store, or we might need to go back to our main database.
cache.feature_flags	true false	Feature flags are frequently cached to avoid needing to fetch them on every request.

Localization Information

What localization options has the user chosen? This can be a frequent source of bugs. Consider adding localization to your wide events, as shown in [Table 6-18](#).

Table 6-18. Localization attributes

Attribute	Examples	Description
localization.language_dir	rtl, ltr	The direction text is laid out in the user’s language.
localization.country	mexico, uk	This is the country with which the user associates. Remember that people travel! This is frequently different from their physical location.
localization.currency	USD, CAD	This is the currency the user prefers.

Operational Information

Telemetry for operational concerns is what makes your systems operable by people. This type of information is helpful in the context of incident response.

Uptime

Tracking how long the service has been running when it serves a request can help you visualize several classes of bugs:

- Issues that show up on a reboot
- Memory leaks that only start to show up when the service has been running for a long time
- Frequent crashes or restarts if you automatically restart the service on failure

It provides answers in many different scenarios, and it's easy to add, as shown in [Table 6-19](#).

Table 6-19. Uptime attributes

Attribute	Examples	Description
uptime_sec	1533	This shows the number of seconds since the service was started. It can be useful to visualize restarts.
uptime_sec_log_10	3.185	This shows the number of seconds since the service was started and passed through a <code>log10</code> function. This value grows sublinearly, which enables you to visualize long-running services and brand new ones on the same graph.

I recommend also either adding the `log10` of the uptime as an attribute or having some way of visualizing this (as shown in [Figure 6-6](#)). When graphed, this emphasizes the important first few minutes of a service without being squished into the bottom of the graph by instances with several days or more of uptime.

Let's say you have some support tickets for users who are frustrated that your service is flaky. Some requests are successful, but mostly they hang or return disconnect errors. Are your services even up? Visualize the uptime with this query:

```
SELECT
  HEATMAP(uptime_sec),
  HEATMAP(uptime_sec_log_10)
WHERE
  main = true AND
  service.name = "api-service"
```

Figure 6-6 shows a visualization.

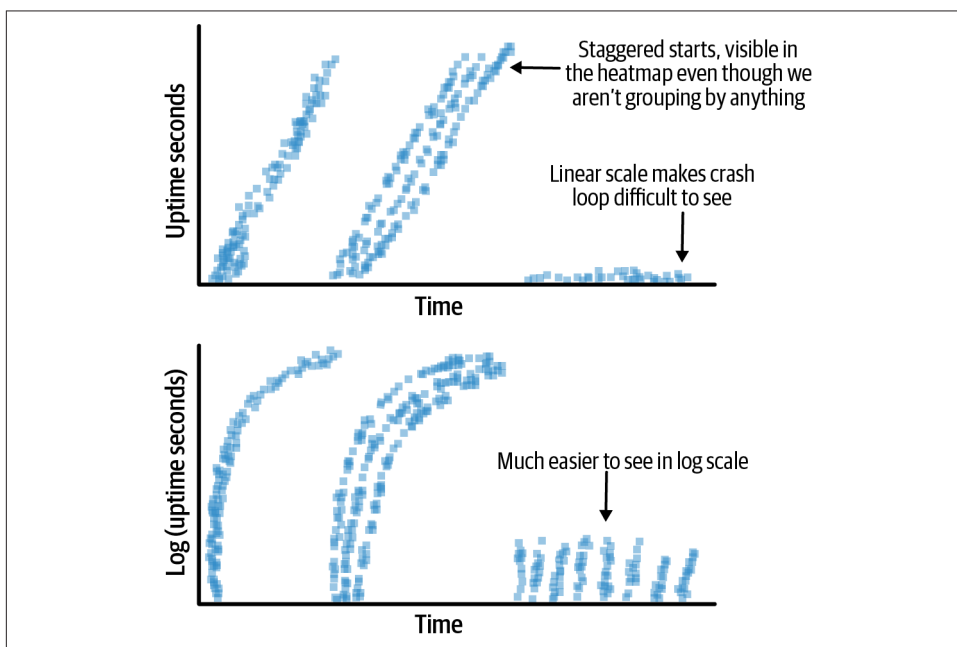


Figure 6-6. Heatmaps of uptime when a service enters a crash loop: the anomalies are much more visible when the data is presented in log scale.

Now, we have a big clue about what is going on! Each service comes up, runs for a few seconds where it serves some traffic, and then enters a crash loop. Our next step should be to add a `GROUP BY service.version` to see if this correlates with a new release. If it does, then metadata about what triggered the deploy, who deployed it, and the code changes that were deployed will likely give us the context we need and dramatically shorten the incident investigation.

Metrics

I've found it helpful to tag spans with context about what the system was experiencing while processing the request. I added code to fetch this information every ~10 seconds, cache it in memory, and add it to every wide event produced during that time.

Examples of useful metrics to add as attributes are shown in [Table 6-20](#).

Table 6-20. Metric attributes

Attribute	Examples	Description
<code>metrics.memory_mb</code>	153 2593	Amount of memory being used by the system at the time it services a request
<code>metrics.cpu_load</code>	0.57 5.89	CPU load of the system servicing this request, given here as a measurement of the number of active cores
<code>metrics.gc_count</code>	5390	Last observed number of garbage collections— cumulative (total since service started) or delta (e.g., number in the last minute)
<code>metrics.gc_pause_time_ms</code>	14 325	Time spent in garbage collections—could also be cumulative or delta (pick one and document which)
<code>metrics.go_routines_count</code>	3 3000	Number of Go routines running
<code>metrics.event_loop_latency_ms</code>	0 340	The cumulative time spent waiting on the next event loop tick, an important metric for Node.js apps

With these attributes, you can query an answer to questions like “Are these requests getting slow because we’re running out of memory space or available CPU?”:

```
SELECT
  HEATMAP(duration_ms),
  HEATMAP(metrics.memory_mb),
  HEATMAP(metrics.cpu_load)
WHERE
  main = true AND
  service.name = "api-service"
GROUP BY instance.id
```

Running this query produces a graph like [Figure 6-7](#).

Please note that capturing metrics in this way is *not mathematically sound*. Because you only get data when traffic is flowing, you can’t calculate a P90 for CPU load that would stand up to any rigorous scrutiny, but that’s usually fine in practice.

Visualizing results for the example query (as shown in [Figure 6-7](#)) are close enough to get some quick signal while you’re debugging without switching to a different tool, especially if you can avoid calculations and can visualize with a heatmap.

I wouldn’t recommend setting alerts on this data though. Regular metrics are great for that.

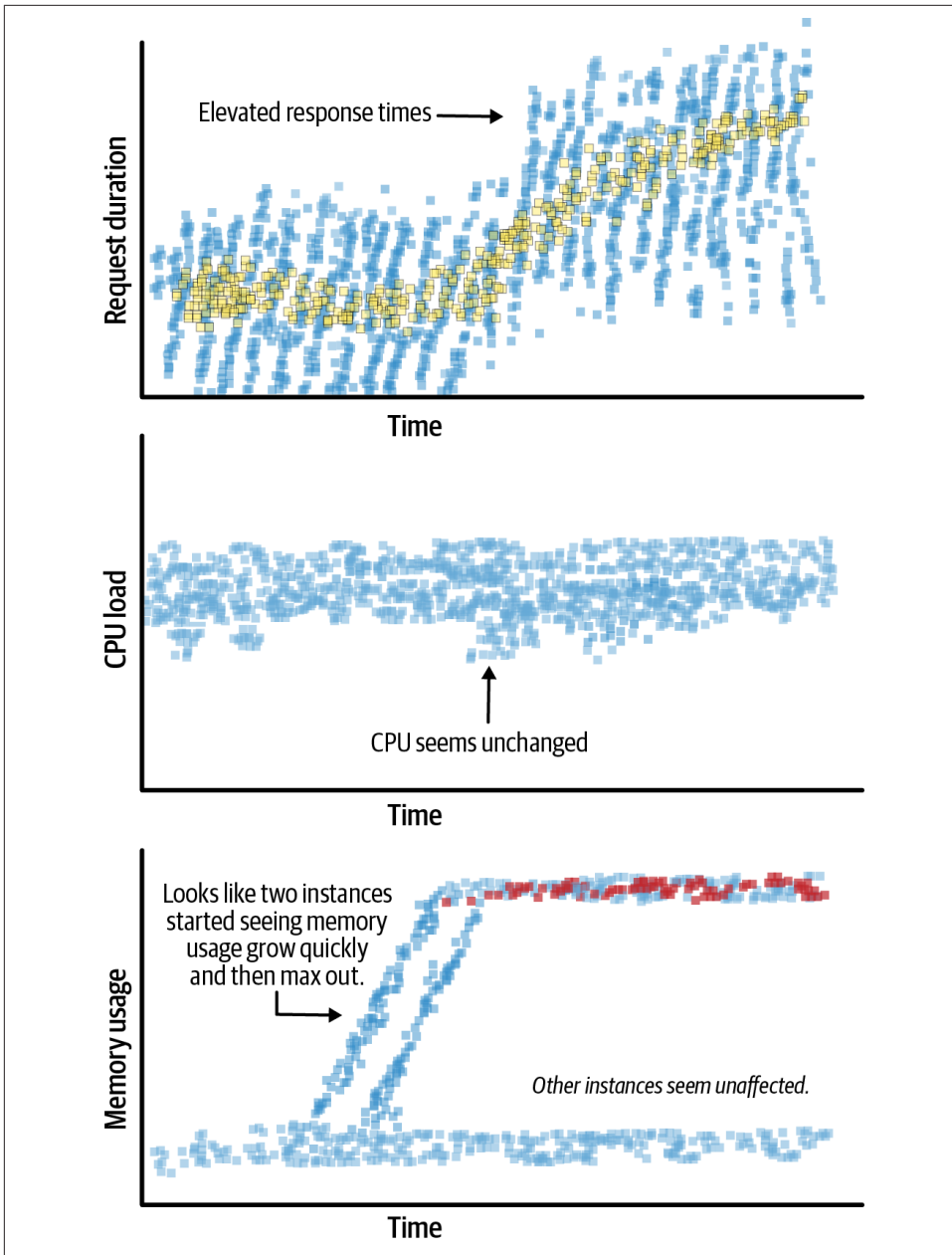


Figure 6-7. An example using the metrics data tagged on structured events to get context for what's happening with the system.

A Convention to Filter Out Everything Else

While we are carefully crafting certain events to be “wide,” these are almost certainly not the only events we’re emitting. You can still use other logs where it’s helpful!

I like to have a convention for identifying and searching for these “wide, structured events.” I’ve landed on calling them “main” events. An example of how to tag these with an attribute is shown in [Table 6-21](#).

Table 6-21. A filtering attribute

Attribute	Examples	Description
main	true	Present only for spans designated as a “wide event,” usually wrapping a request or response, or a background job

I find this helpful when approaching new systems. I can quickly get a view of what work a system does and an idea of how the traffic for that system is distributed by asking for all of the `main` spans and grouping by the span name:

```
SELECT
  COUNT(*)
WHERE
  main = true AND
  service.name = "other-teams-service"
GROUP BY name
```

It’s also helpful for eliminating noise. If I’d like to calculate the error rate of requests into the system, I might query based on the HTTP status code:

```
SELECT
  SUM(IF(http.response.status_code >= 500, 1, 0)) / COUNT(*) AS error_rate
WHERE
  main = true AND
  service.name = "api-service"
```

However `http.response.status_code` is also present on child spans for any outgoing subrequest. Without the `main = true` filter, we’ll get an incorrect error rate calculation here.

Attributes Important to Your Specific Application

Now we go off the map and get to the really valuable stuff. Your app likely does something unique or works in a particular domain. You might need to *really* care about which professional credentials a dentist using your app has, or which particular storage warehouse a package is in, or which chip is in the embedded tracking device installed in the cat that your app exists to track.

Some examples of possible attributes you could add are shown in [Table 6-22](#).

Table 6-22. Attributes for your specific applications

Attribute	Examples	Description
asset_upload.s3_bucket_path	s3://bucket-name/ path/to/asset.jpg	If you upload something, add context about where.
email_vendor.transaction_id	62449c60- b51e-4d5c-8464- 49217d91c441	If you interact with a vendor, track whatever transaction ID they give you in case you need to follow up with them.
vcs_integration.vendor	github gitlab bitbucket	If there are three or four types that something might fall into, be sure to add that context. (E.g., if 2% of requests start failing because Bitbucket is experiencing issues, this will help identify the source of the issue immediately.)
process_submission.queue_ length	153 1	Any time you interact with a queue, see if you can get the current length during submission.

Table 6-22 contains a few useful examples. But this is where your expertise matters most. No framework is going to be able to understand what parts of your domain are important to track and automate for you; you have to do that.

These suggestions provide a start, but I hope you're able to use the inspiration to help you get started on your own journey toward making your structured events incredibly wide.

Conclusion

Jeremy's walkthrough of practical instrumentation to add to your arbitrarily wide, structured events is thorough, thoughtful, and concrete. We'll reiterate that while this chapter contains a long list, it is definitely not exhaustive.

We'll note that—since we work in the observability space—one of the questions we authors most often get asked is: what exact custom instrumentation should I add to my events? And, most often, our answer has been, “it depends.” This chapter lists several considerations and concrete ways to get started down that path. Of course, much of the best value you'll get during your observability journey will be from instrumentation that is specific to your business and your applications. In further chapters of this book, we'll continue to see other examples of possibly useful attributes.

Next, we'll look at how exactly you should add custom instrumentation to your code by diving into what you should know about the OpenTelemetry project.

Instrumenting Your Code with OpenTelemetry

OpenTelemetry promises a unified standard for telemetry. But, for the uninitiated, it can sometimes feel a bit overwhelming in its complexity. There's a dizzying array of terminology, a broad application programming interface (API) and SDK surface area, and a learning curve sometimes described as more of a “learning cliff.” Should you use zero-code or library instrumentation? Which signals should you emit? Is switching to OTel even worth the effort?

This chapter is not an in-depth guide for everything you need to know about OTel. For that, we recommend reading *Learning OpenTelemetry* by Ted Young and Austin Parker (O'Reilly, 2024). Here, we'll focus on making OTel less intimidating and more practical to approach. Think of this as your set of ropes, harnesses, belay devices, helmets, and situational awareness training so that you won't fall off of that learning cliff.

In this chapter, we'll cover fundamental design philosophies to help you understand why and how OTel is built as the project you see today and how to gauge when it is (or isn't) a good fit. We'll also guide you through understanding *trace-first* design principles in complex architectures you're likely to encounter. From there, we'll dive into trade-offs behind its various signal types and how to use them effectively. Lastly, we'll also show you how AI-assisted coding helps with OTel instrumentation, especially when migrating from other existing frameworks.

What It Means to Use OpenTelemetry

In [Chapter 4](#), we introduced the city-planning analogy, where instrumentation acts as the sensors and meters that provide system insight. Now, imagine you're responsible for building and integrating those very sensors into your analytics pipeline.

You might find that some data is semantically linked (for example, traffic sensors that count the number of vehicles at an intersection and the status of the traffic lights themselves), yet it arrives in different formats, with different field names, and with different ideas about units and meaning. You'd need to spend time transforming signals into a normalized shape, documenting the changes, and then repeating the cycle whenever a sensor changes, a vendor sunsets a product, or a "compatible replacement" emits slightly different data.

If a sensor failed and the company that made it was out of business, or a new version emitted different data, you'd need to rebuild your pipeline in order to maintain your analytical integrity. Even if you manage to stitch it all together, you still end up with a firehose of partially duplicated information, and you'd be on the hook for making it coherent and usable.

OTel exists to reduce that burden. It provides an open standard for how telemetry is created, transmitted, and interpreted. It also provides semantic conventions (shared meaning for names and metadata) so you can swap components without rebuilding your entire pipeline. In practice, that means less vendor lock-in, fewer painful migrations, and cleaner cost controls because you're collecting what you mean to collect. In our analogy, it would guarantee that you don't have to worry about changing out sensors or updating your analysis tools; the same data will be emitted and will work in the same ways even with different components.

The project's ambitions extend beyond providing a common format. The goal is for telemetry to be native to the libraries, frameworks, runtimes, and infrastructure you already use; rather than being a bolt-on afterthought. When a rich braid of unified telemetry already exists, it enables a world where engineers can easily add business-specific context as needed, where telemetry and observability are transferable skills, and where observability is the default state of high-quality software.

In practice, that means your software system emits the right signals at the right level of abstraction, without bespoke configurations or transformations.

What does it mean for your team to use OTel? Like all good software questions, the answer is "it depends." This chapter is written for developers and site reliability engineers who are instrumenting applications and services (backend, frontend, full stack) and who want guidance on patterns, boundaries, and practical techniques.

Effective Instrumentation With OpenTelemetry

You can start using OTel without writing any new code. There's a bevy of zero-code instrumentation libraries, agents, and other solutions that will generate metrics, logs, and traces from the most popular libraries in every language. The hard part isn't generating telemetry; it's generating *useful* telemetry.

Effective instrumentation is a bit like sculpting: you're deciding what to keep and what to throw away. In this section, we'll focus on boundaries, span and attribute naming, and how to choose the right telemetry signals to support the questions you actually need to answer.

Trace-First Telemetry

The biggest difference between OTel and many older instrumentation approaches is context. "Context" is an overloaded term—it can refer to the state of what's happening inside a system, the data structure used to encode and propagate unique identifiers between functions and processes, or the specific values stored inside that structure.

We'll demystify "context" by examining how OTel designs instrumentation. OTel is described as "trace-first" because context is the glue that makes the rest of the model work. If you emit traditional metrics and logs without context you can still get value, but you miss the main benefit. In OTel, all telemetry signals can be associated back to a specific request, transaction, or unit of work as long as an active context exists.

Being trace-first is important because context propagation is how that identity moves through your system. There are two types:

Interprocess propagation

The identity crosses process boundaries, often via HTTP headers using W3C TraceContext that include trace ID, parent span ID, plus optional baggage (arbitrary key-value pairs) set earlier in a transaction.

Intrprocess propagation

The identity flows through methods, threads, coroutines, or components inside a process boundary.

Example 7-1 shows a simplified Java illustration of both types.

Example 7-1. Interprocess and intraprocess propagation examples.

```
// Injecting context into outgoing HTTP request
TextMapSetter<HttpRequest.Builder> setter = (carrier, key, value) ->
    carrier.header(key, value);

HttpRequest.Builder requestBuilder = HttpRequest.newBuilder().uri(uri);
W3CTraceContextPropagator.getInstance()
```

```

        .inject(Context.current(), requestBuilder, setter);

// Extracting context from incoming HTTP request
TextMapGetter<HttpServletRequest> getter = new TextMapGetter<>() {
    public String get(HttpServletRequest req, String key) {
        return req.getHeader(key);
    }
    public Iterable<String> keys(HttpServletRequest req) {
        return Collections.list(req.getHeaderNames());
    }
};
Context extractedCtx = W3CTraceContextPropagator.getInstance()
    .extract(Context.current(), request, getter);
// Making extracted context current for this thread
try (Scope scope = extractedCtx.makeCurrent()) {
    // All spans created here inherit the parent from extractedCtx
    Span span = tracer.spanBuilder("child-operation").startSpan();
    try {
        doWork();
    } finally {
        span.end();
    }
}

```

HTTP is a common carrier for interprocess propagation, but it's not the only one. Context can travel through a variety of methods, including message queue envelopes, environment variables, gRPC or other remote procedure call (RPC) framework headers, etc.—anywhere you can attach metadata reliably.

In languages like Java, or others with threads and strong concurrency primitives, intraprocess propagation is often as simple as establishing a scope (e.g., wrapping a method or block in a try, so you can easily capture failures) and then creating spans within it. Any metrics or logs emitted inside that scope inherit the active context as well.

Helpfully, low-level management of context like this is probably something you won't need to do, especially if you use automatic instrumentation.

At this point, a common thought often occurs: “If context is flowing, should I create a span for every function?” In most systems, that's the wrong instinct.

Another important part of OpenTelemetry's design is that telemetry should be *layered*; meaning that you should use the right signal for the right purpose. For instance, spans should model meaningful units of work: discrete operations that matter to performance and reliability.

Two questions help determine whether you should create a span:

1. *Is it “interesting”?* Does the work meaningfully impact performance (e.g., latency or failures) for the overall request?
2. *Is it “aggregable”?* If you group this span by name and/or attributes, will it produce useful trends and comparisons?

Table 7-1 demonstrates some practical examples of determining whether you should create a span.

Table 7-1. An example guide for determining whether to create a span

Operation	Interesting?	Aggregable?	Create a span?
HTTP request handler	Yes: variable latency, can fail, boundary crossing	Yes: group by route, method, status code	Yes
Database query	Yes: input/output bound, variable latency, failure prone	Yes: group by query type, table, operation	Yes
External API call	Yes: network latency, dependency failures	Yes: group by endpoint, service, status	Yes
Cache lookup	Yes: distinguishes fast versus slow paths	Yes: group by cache name, hit/miss	Yes
Message queue publish/consume	Yes: async boundary, processing delays	Yes: group by queue, message type	Yes
Private helper function	No: trivial CPU, predictable duration	No: too granular, no meaningful groupings	No
Loop iteration	Maybe: if iteration is slow	No: unbounded cardinality per request	No
Getter/setter	No: no meaningful duration or failure	No: nothing to group by	No
Input validation (pure CPU)	No: fast, predictable	Maybe: could group by validation type	No
Input validation (with input/output)	Yes: schema fetch can fail/be slow	Yes: group by schema, validation result	Yes
Business logic orchestration	No: just calls other instrumented code	No: duration is sum of child spans	No
Business logic transaction or saga	Yes: meaningful state change, can fail	Yes: group by transaction type, outcome	Yes

There’s an art to creating a good trace. In practice, it’s all too common to err on the side of too many or too few spans. Too many spans makes analysis expensive and unhelpful (a trace with millions of spans is far too detailed and rarely actionable). Too few spans often means collapsing minutes (or hours) of work into a single opaque handler that still leaves you guessing at optimizations or effective ways to monitor and alert.

You'll also notice a pattern that seems adjacent to Conway's law:¹ traces tend to mirror organizational boundaries (e.g., a group of developers or the microservices they manage). Resist that urge. Hold preference for spans that map to true **component and service domain boundaries**—even if that means crossing team lines—because that's what makes traces useful during incidents.

If your trace has a hundred two-millisecond spans, you probably have too many. Roll them up: combine them into a single span, or represent the details as an attribute. For fine-grained performance questions, profiling (see **Chapter 20**) is often a better tool for merging function executions into useful data.

The Perils of Automatic Instrumentation

Auto-instrumentation often errs on the side of producing too many unactionable spans, metrics, or logs in the pursuit of thoroughness across signal types. This isn't malicious; it's a reasonable default in a general-purpose library. You are responsible for making defaults sane within your environment.

Most of the time, you should prefer not duplicating telemetry unless you really need to. A common pattern is to keep traces sampled. It's expected that users will have different collection rules or sampling strategies across signals.

It's very useful to emit histograms for accurate and high-resolution counts of rate, error, and duration (RED) metrics from HTTP or gRPC endpoints in *addition* to trace data, especially for high-throughput services. Histograms can be stored cheaply, even if you have to rely on head sampling to reduce overall trace throughput.

If you're drowning in noise, turn it off: drop or sample out specific instrumentation, or replace it with custom spans and processors that summarize small work into attributes on a parent span.

Traces for Common Architectural Patterns

Most distributed tracing models (and OTel in particular) fit request/response architectures well: a client sends a request, the service does work, it calls dependencies, and the entire transaction can be visualized into the common “waterfall” trace representation.

The further your system architecture drifts from the request/response pattern, the more challenging it becomes to design your traces. Let's walk through a few patterns.

¹ Conway's law describes the link between the communication structure of organizations and the systems they design. For example, if a company has a fragmented communication style, its products and systems are also likely to reflect that fragmentation.

Tracing streaming architectures

Second to the request/response pattern, one of the most popular and widely used system architectures is stream processing, where data flows across queues, buses, and topics. It's used for everything from extract-transform-load (ETL) jobs such as processing of financial records or inventory management, to event-driven business workflows, to planet-scale aggregation for social media applications, and more.

Tracing stream-processing models is tricky for two reasons:

1. The smallest “unit of work” you can instrument isn't always the most useful unit of work to optimize.
2. The things you'd like to alert or measure in these pipelines often have more to do with the trace shape itself rather than the spans that comprise the trace.

Specifically, developers often want to measure pipeline metadata like stage dwell times (“How long are consumers waiting for new messages and what do those messages have in common?”), or the amount or ratio of successful-to-failed stages (common for developers implementing step functions or finite state machines atop of queues).

Some architectures, like event sourcing, present additional challenges. For example, when every logical action in a pipeline is a mutation or series of mutations against a golden event, what exactly is considered the root span, and when does that occur?

OTel can still help, but you'll usually need a blend of signals and a little design. Don't try to answer everything with spans alone. For example, let's say you have a workflow that looks like this:

```
orders-api (produces OrderPlaced)
→ topic: orders.events
  | payment-svc (consumes OrderPlaced, produces PaymentAuthorized/Declined)
  | inventory-svc (consumes PaymentAuthorized, produces ItemReserved/Backorder)
  | shipping-svc (consumes ItemReserved, produces ShipmentCreated)
  | reconcile-job (nightly batch over events store)
```

The first requirement is proper context propagation: serialize context into the message envelope when you publish, and then extract it when you consume. For Apache Kafka and similar queue or message bus systems, message envelopes make this straightforward.

The `orders-api` service emits a trace span like `messaging.publish` with attributes such as `system`, `destination topic`, `message ID`, and relevant business details (`transaction ID`, `customer ID`, etc.). That span is marked as a “producer” span (as opposed to “client”), and its context is serialized into the message envelope of the `OrderPlaced` event and then written to the topic.

As consumers read those messages, they extract the context from the envelope and create corresponding `messaging.process` spans. Those spans are linked to the parent via “span links,”² but not as direct children. When the consumers write new messages back, they would repeat the process; a producer span for the new message type, and so forth.

A useful approach is to store and hash a unique identifier for each transaction and propagate that through the entire pipeline. That propagation ties all of these traces together directly, rather than having to walk the tree in reverse. Span links can also help you relate these events together across multiple traces.

Conceptually, this shifts from creating one trace with a single root and many children into a timeline of traces that occur at different times. Because each logical transaction carries a unique correlation ID, you can easily pull all relevant spans into a result set, and then visualize them as needed.

That view can provide you with the relevant shape of your pipeline and the ability to understand the performance of each stage, but it has trade-offs. While providing excellent visibility into a single message’s journey, it can be tricky to generalize that approach for investigations at the fleet level.

In that instance, OpenTelemetry’s metric exemplars can help. An exemplar is an annotation on a particular metric point that associates it with a specific trace and span. While your spans and traces provide deep insight into how each specific stage or consumer is performing, metrics can provide top-level insight into the pipeline as a whole—and they’re also where you should focus your frontline alerting.

Your pipeline metrics look more like counters emitted by each stage showing success or failure (with attributes mirroring your span data, but at a lower cardinality—so you don’t necessarily want to put the transaction ID on each measurement, but you should add the consumer name). Emit a metric for successful and failed jobs, and then calculate and alert on the ratio through `failed_total`, `processed_total` per stage, type, region, etc. That lets you proactively alert on unusual Kafka behavior by creating histograms that track the time from publish to process, and use exemplars to attach your outlier datapoints to specific consumer traces.

If you’re working with step functions and are concerned about the number of stages per transaction, that’s another good use case for histograms. Record the number of stages at the terminal stage and, optionally, use baggage to pass down the number

² **Span links** allow developers to represent complex graphs of service dependency and relationship, even across domain or physical boundaries. For example, a consumer of a public API may trace their outbound request to the API, and the receiving server may create a new (internal) trace that preserves the link to the third party’s original trace. This is a feature that to this day has not been heavily utilized, but has interesting potential applications.

of consumers touched. Some of these measurements may also be available from automatic instrumentation or by translating existing infrastructure metrics into OTel format (e.g., Kafka broker health, queue length, etc.). Conveniently, much of this is automatic as long as you're recording your metrics inside active spans in your process. If you're properly passing your context around and managing it in-process, this part is already done for you.

Asynchronous work and jobs

For fan-out/fan-in or scatter-gather work patterns, the important decision is less about span topology and more about how the request is presented to the user (or caller). Let's suppose you have a video-processing pipeline that accepts uploaded content and then renders it into a consistent format for storage and presentation:

```
POST /render → orchestrator
├ publish task.transcode
├ publish task.thumbnail
└ publish task.caption
workers process → orchestrator joins when N succeeds or deadlines
```

The root span for these cases is the POST. You can choose to use parent-child or producer-consumer relationships, depending on how you're electing to monitor the application and the expected results. Either way, the root span should contain links to each child process, and have a final summarization span that rolls up the results.

Alerting often belongs on the entry point (POST) or on specific child tasks, depending on service architecture or ownership (Conway's law really does rule everything around us). In this case, metrics may be less useful than pure spans because there's no shared infrastructure that you can optimize other than network links.

Next, let's consider something similar but with a much longer time horizon, such as nightly jobs. Suppose you have a nightly workflow that compiles inventory reports from thousands of suppliers, all of whom have subtle differences in their data formatting. Extracting and transforming that data can take hours (and in some cases, days) before it's loaded into a business intelligence database. A logical representation of the work could look like the following:

```
scheduler → [job.run nightly_etl] (ROOT)
├ etl.extract
├ etl.transform (hours)
├ etl.load
└ report.write
```

In this scenario, you should avoid the temptation to turn the transform steps into direct child spans. Long-running jobs introduce the "million children trace" hazard. Instead, as recommended earlier, model each stage as its own trace and relate the stages with links and correlation identifiers. One challenge with that approach is that

your parent span may not necessarily complete until all children are complete. There are a few strategies to solve this problem, but they have trade-offs.

One option is to follow the disjointed ETL process elucidated earlier. You won't have a single root span for the job. Instead, you have a producer trace that's linked to your child traces. The downside is that it can be challenging to correlate all of your child traces together. Alternatively, you can have a root span that you're building up in-memory (or in a file), adding heartbeat events or status events as stages complete, which get closed and written out at finish.

The “Occam's razor” solution to this problem is to periodically flush the root span with updated timestamps as the child jobs progress. But that approach is not recommended, as the vast majority of tracing datastores don't handle updates to existing spans. In most cases, they are either rejected or cause undefined behavior.

In these situations, metrics or logs can also help by providing intermediate status reports that you can monitor without breaking the span and trace data model. As long as you're preserving trace context through your child-consumer processes and calls, you can associate metrics back to their underlying traces.

When deciding which trade-offs to favor, it can be useful to ask yourself and your team what your exact concerns are, and then use the answer to guide your thinking. Are you checking a box for requirements, or are you trying to understand and optimize the individual performance of these jobs?

If your primary goal is a safety net rather than fine-grained performance optimization, consider a “wide event” approach: a single summarization span (or log event) with hundreds or thousands of attributes that contain a significant amount of metadata about the job that gets written at completion, with rich metadata about what succeeded, what failed, and why. This is useful for tracking changes over time without dealing with the overhead of creating a well-formed discrete trace.

Serverless

Most of the guidance thus far applies to serverless, but the constraints change the trade-offs. There are also a few specific tips and considerations specifically when running in AWS Lambda.

First, you should probably favor using custom instrumentation for your serverless functions rather than relying on automatic instrumentation. While OTel instrumentation libraries for serverless exist, the resource constraints of serverless environments mean it's ideal to limit the spans you create in your functions. A single well-designed span per invocation, rich with attributes capturing the details of a few key events, often beats a sprawling hierarchy that burns cold-start time and memory.

Specifically, in AWS, an interesting (albeit somewhat work-intensive) strategy is to use stateless Lambda tracing. In short, emit lightweight events from the function and let a stateful OpenTelemetry Collector (with custom processors) build a map that assembles them into a span (see [Figure 7-1](#)). This approach reduces the need for statefulness in libraries and context management in the serverless function runtime, at the cost of more complex Collector-side logic you'll need to implement. If performance is a paramount concern, it might be worth that trade-off for you. Ideally, the type of extension layer necessary to accomplish this will be provided natively by more serverless providers in the future.

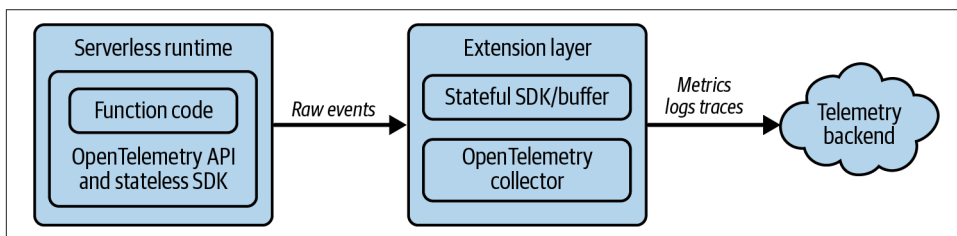


Figure 7-1. A logical diagram of using a stateless SDK in the serverless runtime layer to forward events to a stateful SDK and Collector in an extension layer. This technique allows for improved reliability and performance while minimizing cold start times and enables use of a consistent set of configuration management for Collectors.

From a practical perspective, you'll most likely need to use a mix of techniques for serverless instrumentation with OTel. Different serverless platforms will offer different hooks and service-specific extensions to create and emit telemetry data postexecution but presleep, and this data is often not emitted in native OTel formats. That data can be correlated post hoc through association across a universally unique identifier (UUID) or some sort of invocation ID. In this case, you'll need to: use telemetry pipelines or transforms to route the data to the same place as your other telemetry; use a query layer that allows for federated queries across multiple datastores; or batch, join, normalize, and forward these events together. As OTel continues to mature and gain wider adoption, we hope that these platforms will offer native OTel solutions for their users.

Extended Berkeley Packet Filters

So far, this chapter has assumed that it's possible for you to modify your code in order to perform instrumentation. The reality in some organizations is that doing so often requires a level of cooperation between observability engineers and software development teams that is either slow, infeasible, or flat out impossible. In these cases, extended Berkeley Packet Filter (eBPF) technology can help.

eBPF programs run in the Linux kernel itself,³ outside of user space, and enable observability at a different level of abstraction than previously covered techniques. eBPF is unique in that its programs are designed to be small, self-contained, low overhead, and very safe—they cannot corrupt memory or crash. Because they run outside of a process, they incur no additional runtime overhead and they can be enabled or disabled without requiring a restart of the process, or recompilation.

OTel supports eBPF instrumentation through the **OpenTelemetry eBPF Instrumentation (OBI) package**. OBI gathers metrics and traces from HTTP/HTTPS and gRPC services written in a variety of languages and allows for propagating trace context through services. It's best understood as an accelerator and a gap filler that can enhance existing instrumentation efforts, rather than something that replaces them entirely.

For instance, OBI allows you to capture kernel-level detail about network operations that would not be available to your applications. It allows you to propagate traces through legacy services where you would otherwise lose visibility. A full discussion of its capabilities, configuration, and limitations is beyond the scope of this book, but we'll touch on a few other key capabilities.

It carries some operational concerns, aside from the aforementioned limit on generated telemetry. Potential conflicts may exist with other eBPF programs and you must configure them carefully to avoid this. They may also need elevated permissions depending on the deployment. If you aren't running OBI as an administrative user (a.k.a. `root`), then you'll need to enable specific permissions for the process.

Currently, OBI and other eBPF-based instrumentation solutions offer real value, and future work in this area promises to unlock more. Perhaps we'll see channels for extracting state from requests, allowing for custom attributes to be added to span data or metrics. That could narrow the gap between “observing execution” and “understanding intent.” For now, you should think of OBI and related tools as a way to quickly generate tracing data for uninstrumented systems (e.g., to populate service maps), extend traces through uninstrumented services, or generate instrumentation as a part of debugging kernel-level behavior.

So far, we've covered options when instrumenting and strategies for designing traces for common architectural patterns that go beyond simple request/response. Quality traces are foundational when using OpenTelemetry because they connect everything else. If you don't have good traces, there's a lot of value you won't get from your metrics or logs either.

³ Microsoft is currently working on an eBPF layer for Windows, which can be viewed on the [GitHub repository](#). This will enable some pretty exciting use cases!

Next, we'll zoom out and talk about layered telemetry and how you can use traces, metrics, and logs together to achieve better observability outcomes.

Metrics, Spans, Logs, Events—Oh My?

Layered telemetry is the practice of using the right signal at the right level of detail for the outcome you need. OTel makes this easier because shared context can correlate signals back to the same request. We'll examine OpenTelemetry's design and semantics so you can choose the right mix of signals when observing your system.

As mentioned previously, OTel is trace-first, given its ability to correlate signals through a shared distributed context layer. In practical terms, trace-first correlation unlocks three benefits:

1. Developers can layer signals that provide multiple “views” of the same work without regard for any downstream sampling or ingestion decisions.
2. Operators can balance accuracy, visibility, and cost in observability pipelines for operational efficiency.
3. Telemetry consumers can drill down from a high-level symptom to the telemetry signal that explains what's happening in production, and how it relates to their goals.

OTel instrumentation should embrace a multisignal approach, but it's more nuanced than simply duplicating data. A useful three-question test for choosing signals is:

1. What needs causality and full-request context for consumers? (traces)
2. What needs inexpensive long-term storage and/or fast alerting? (metrics)
3. What is rare versus common, and what are the audit requirements? (logs or events)

Sometimes you'll layer two signals intentionally. For example, a common pattern for HTTP servers is to emit both a span and a histogram metric for each handled request. That gives consumers flexibility during investigations. If your HTTP service has a particularly high throughput, you could head-sample your traces. Histograms would provide last-ditch alerting for errors and duration outliers, and potentially reveal deeper correlation into unsampled spans. Business-critical or sensitive functions—such as billing events or service level agreement (SLA) conformance data—benefit from having isolated, event-only pipelines, even if your primary observability signals for those transactions are span-based.

The technique is called *layering* (instead of duplication) because you're providing multiple views depending on the needs of consumers. A suitable analogy would be using a microscope: the closer you zoom in, the more detail you see. The job of your

instrumentation is to provide the right level of detail at the right level of abstraction to reveal the necessary view of system performance.

In practice, that means starting with traces at your service boundaries. Think of these traces as the glue between all of your other signals. Next, you can layer on aggregated metrics at that same boundary, but with fewer dimensions and attributes. The key is in realizing those metrics don't need to be separate instrumentations in your code. You can use the OpenTelemetry Collector or other observability pipelines in order to generate metrics directly from the span data (see [Chapter 5](#)).



Sampling and Span Metrics

It's worth mentioning that generating accurate metrics from spans is difficult (or impossible) if you apply head sampling! There is work in the OpenTelemetry project to add support for probabilistic head sampling, allowing for more accurate calculation of metrics from head-sampled data, but it's currently experimental.

Once you know what to emit, you need to name it. Names should be descriptive, unambiguous, and easy to group by. Don't put identifiers or variable route components in names. Instead, identifiers should be replaced with a description, and the variable component moved to an attribute value. For example, `GET /users/profile/349827487124` becomes `GET /users/profile/{user.id}`.

In general, names for spans (or logs) should follow dot notation with snake case separators (`like_this`) for multiword phrases. Consult the OpenTelemetry [semantic conventions](#) for many examples. Similarly, metrics should have consistent names across services. In OTel, you should prefer metadata (such as the type information) on a metric to communicate what it is, rather than encoding it into a name. For example, `http.server.request.duration_ms` should be `http.server.request.duration`.



Schema-Driven Telemetry

An emerging practice is to define your telemetry conventions in a schema file, similar to how semantic conventions are structured, and use tools like [OpenTelemetry Weaver](#) to validate the emitted telemetry at compile and runtime. Weaver allows you to generate code as well, creating helpers for implementing the signals defined in the schema.

Most teams adopt OTel in brownfield systems. This means that migrating your instrumentation can present an opportunity to use schemas as an artifact and source of truth, which builds shared agreement on what different attributes mean.

Sample your existing telemetry data and normalize it into a new, OTel-flavored, harmonized format. Create a namespace for your organization, and use it to collect org-wide concepts: service owners, business units, cost centers. Bake them into a schema so teams stop reinventing field names. Check these schemas into source control, and use them to aid your migration and to help your AI-assisted coding efforts (for a more in-depth look, see [Chapter 16](#)).

Consistency and documentation is not optional. We've seen telemetry environments with dozens of conflicting attribute keys for the same logical value, even within a single signal type. That duplication and confusion is costly for everyone in the middle of an incident: it wastes valuable time and significantly lengthens investigation. That becomes worse when AI is introduced because models can't reliably infer which field is canonical, and they'll often confidently build incorrect conclusions using the wrong data.

Be thorough with your telemetry data. Include units as attributes on your signals, and avoid mixing them on the same key. Normalize groupable keys like name and route, or other entry points into your system. Practice defense in depth (DiD) for user data: hash potentially sensitive values, use SDK- and Collector-level processors to filter or use allowlist attribute keys and values, and leverage transform processors to filter out common personally identifiable information (PII) like identity numbers. Preference stable hashes for potentially sensitive data—like user identifiers—and ensure that you can reverse them through an internal API, preferably something that can be scripted.

Now that we've covered what to emit and how to shape it, there's still the practical part: actually coding all of that instrumentation. Historically, that's been time-consuming human labor. Today, AI can help perform much of the grunt work—if you use it deliberately.

Using AI Agents to Instrument Your Code

Before we start, we'll again acknowledge that we understand the perils of committing to writing about AI techniques in a textbook. As of this writing, several AI agents such as Claude Code, Cursor, Gemini CLI, and dozens of others are widely in use. By the time you read this book in print, it's entirely possible they've become obsolete in favor of some new technique.

However, we believe there's general guidance that will continue to hold true when coding OTel instrumentation into your applications, even in the era of AI agents. Let's examine where they shine and what they shouldn't handle.

What Is an Agent, Anyway?

Agents, as defined by Anthropic engineer Hannah Moran in 2025, are “**models using tools in a loop**”. Let’s unpack that statement so we can understand how AI agents can help instrument our code.

When we talk about AI agents, we’re broadly referring to software harnesses that (at a minimum) combine a generative LLM (e.g., OpenAI’s GPT-5, Claude Sonnet, Gemini, etc.) and some subset of tools. In this context, a “tool” is a procedure call that is exposed to the model through its API. Crucially, tools enable models to execute deterministic function calls in software.

Consider an LLM in isolation. It predicts the next token of a prompt, based solely on the data it’s been trained on. A model that’s tuned to function as a chatbot will do something similar, but each *turn* (or, back-and-forth request and response) is added to the conversation, and the model continues its prediction based on that entire history.

A chat model alone can answer questions, but it can’t reliably inspect an entire codebase, read the right docs at the right time, edit files safely, and validate the result. Instrumentation work requires all of these capabilities.

Reductively, an agent is just a harness that gives the model those abilities—similar to what we, as developers, need to do our jobs day-to-day. Agentic tools can be anything from shell access, to specialized web search tools (returning results in formats optimized for LLMs), to integrations with software as a service (SaaS) datastores or observability databases.

The end result is that agents can be effective partners for adding instrumentation, especially during migrations from existing libraries or frameworks to OTel, where the work is repetitive but still needs judgment.

Instrumentation with Agents

Historically, one of the biggest challenges in getting developers to add instrumentation to their software is that it’s viewed as toil. And, honestly, it’s often a tedious and formulaic process.

But the source of value isn’t typing span builder calls. The value is in everything we’ve discussed so far: the instrumentation strategy you develop. The value is in what you instrument, how you name it, and what questions it helps you answer.

Agents can help with both halves of that equation. In practice, two working modes matter: planning and execution.

In *planning mode*, you take your specifications and prompts, feed them to the agent, add appropriate context (e.g., references to user stories, tickets, related artifacts, software goals, OTel documentation, etc.), and ask the agent to work with you on creating an instrumentation plan. No code yet—just a spec you can review to identify span boundaries, names, and attributes.

Planning mode is especially valuable for new services and for migrations: it's easier to design once and have the agent implement as it goes, rather than retrofitting a sprawling codebase later. In this phase, you'll decide how to instrument, what to instrument, and which attributes and signals are critical. If you're using schema tools (like Weaver), a great artifact from planning is a schema spec and a short set of rules to add to your agent's memory.

In *execution mode*, use caution if you're working in a brownfield environment. You should give the agent patterns to follow and rules on how to adapt. AI models will imitate whatever patterns are most common in your repos, whether those patterns are good or bad. Brownfield systems are rarely consistent. So, often, this means your agent will need to implement small, specific, and targeted fixes that can be generalized into scripts or instructions. Keep changes small enough that code review actually works.

A useful pattern with agents is to have them automate the automation. Rather than asking the agent to edit a number of files manually, have it identify patterns that should change and write a script to make changes programmatically. For example, have an agent write a script that identifies all custom metrics (using some heuristics), and then tell it to analyze each result and determine what kind of change makes the most sense (e.g., converting the metric to a span attribute, deleting it entirely, or changing its type).

If you don't already have them (and most teams don't), consider "telemetry tests" that capture diagnostic output before and after a telemetry migration. Perhaps use your agent to first create those tests, and then use it to compare the semantics against each other. You can also do that with auto-instrumentation: if you're migrating from a proprietary APM (application performance monitoring) agent to OTel, capture the telemetry from both and ask your AI agent to diff them. This is exactly the kind of repetitive comparison AI is good at.

Finally: share rules. Treat agent instructions like shared tooling. Check them into version control, keep them simple, and make them consistent across the team. An emerging pattern is the idea of "agent skills," which are instructions for an agent that can be shared among teams, individuals, and organizations.



Treat Agent Rules Like Software

Prompting is a form of programming. If your team is relying on agent workflows, test them.

For example, create a small daily continuous integration (CI) job that uses your AI agent to generate or modify an appropriate artifact (e.g., library, toy app, etc.), instruments it to your standards, runs it so that it emits telemetry to a test backend with an MCP (Model Context Protocol) integration, and then independently evaluates the outputs.

Useful Strategies for Agents

AI models reward clear, concise, accurate instructions or documentation. They're bad at implicit nuance, sarcasm, or other forms of ambiguity. They're also great at sounding confident when wrong. Plan accordingly.

That said, some prompting folklore is outdated. You don't need to bribe models with rewards or threaten them with punishment.⁴ Extra emotional language often makes outputs worse, not better. What matters for successful instrumentation is managing context (or chat history) and giving crisp constraints.

Here are a few patterns we've seen work when asking agents to perform instrumentation, or coding tasks in general:

Manage agent context.

Agents will go offtrack as their context fills up, and they have a recency bias. Use features like subagents where an independent agent process performs repetitive work in isolated contexts, or keep an external task file in markdown that the agent periodically updates, especially for long-horizon tasks.

Work in small chunks.

You'll get better and more consistent results by focusing on a single file, or even a part of a file, at a time. Periodically commit your changes to source control, and don't be afraid of throwing things away and rolling back if the agent goes off the rails.

Don't be afraid of disposable code.

You can use agents to build tiny, fit-for-purpose instrumentation tools. Throw them away when you're done. You can even use an agent to build your own agent! This is especially useful for migrations as there's probably a lot of documentation you have access to. Consolidate and compile it, distill it down into system prompts and reference files, then build your agent around that core.

⁴ James Padolsey, "[Tipping AI for Better Responses?](#)", March 20, 2024.

Tailor the agent to a specific service, or programming language, or internal framework, etc. The tighter the specificity, the likelier you'll get good results with minimal oversight.

Let AI help write the documentation.

Use an agent to read through your existing libraries or code and have it create your initial state as a schema file in any format you prefer. Iterate to ensure it has accurate telemetry descriptions. You can even feed it any existing observability primitives (dashboards, alerts, etc.) so that it knows how the data is used. With that initial state, use it to generate planned diffs, or create mappings to existing OpenTelemetry concepts.

Bring observability into the software development loop.

Beyond migrations, having real-time observability data integrated into an AI agent can be powerful for developers. It reduces context switching for devs, and it allows your agent to make better choices based on real data rather than guesses.

As a real world example, one of the authors was implementing a feature that involved fetching data and needed to know a good default for page size. The agent was able to query telemetry data and determine the P50, P99, and P9999 size of result sets for the feature in question, and how long those database queries took in production. Rather than guessing or context switching to find out manually, the agent quickly calculated an optimal number of items to display per page, using real-world data.

None of this replaces engineering judgment. But used thoughtfully, agents can reduce the cost of doing instrumentation well, and make it easier to keep your telemetry consistent as your codebase grows.

Conclusion

OpenTelemetry can feel overwhelming at first, but the core idea is straightforward: model system behavior with shared context, emit signals with consistent semantics, and layer telemetry so operators can move from high-level symptoms to concrete causes.

In this chapter we covered trace-first design and context propagation, practical span boundaries, trace strategies for streaming and asynchronous architectures, and special considerations for serverless and eBPF-based instrumentation. We also covered how metrics, traces, logs, and events fit together—and how to choose the right signal for the job.

Finally, we discussed how AI agents can reduce the toil of writing and migrating instrumentation when you pair them with a real plan, consistent conventions, and reviewable execution. While the features and capabilities of AI will surely change

quickly, these guidelines illustrate how to balance human engineering judgment with AI's emerging capabilities.

At this point you should be well-equipped to generate actionable telemetry from your services. Next, we'll pivot from creation to analysis: how to make sense of all the data you're now collecting.

Analysis Workflows

Part II covered the practical skills for instrumenting your code and generating telemetry. Now we look at what to do with that data.

Analysis is the other half of the technical foundation. Instrumentation without analysis is just logging into the void. This part covers how to actually use telemetry—across development, deployment, and production—and we recommend it for most readers.

Chapter 8 introduces the core analysis loop: a repeatable workflow for sifting through telemetry data, surfacing patterns, and finding the source of issues quickly. The goal is to lead with data rather than intuition. This chapter also covers how to automate parts of that loop, including how to use AI assistants effectively.

Chapter 9 looks at how observability should shape developer workflows day to day—while coding, deploying, and monitoring what’s running. By showing how analysis makes deployments more reliable, it also works backward into how instrumentation should be woven into the development lifecycle. AI-assistant guidance is included here too.

Chapter 10 is a guest-contributed chapter by Boris Tane, founder and Chief Executive Officer (CEO) of Coreplane Labs, on using AI agents in observability workflows. The agentic space is moving fast, but there are durable patterns for working with these systems effectively. The considerations apply regardless of which AI tools you’re using.

Chapter 11 takes on the problems with traditional threshold-based alerting and shows a better path: service level objectives (SLOs). You'll learn how the telemetry from **Part II** can be used to define SLOs, why wide structured events are the right foundation for service level indicators (SLIs), and how to build them—with AI assistance where it helps.

In **Part IV**, we go deep on the internals—instrumentation, telemetry collection, storage, and alerting—for those who want to understand how it all works under the hood.

Getting Started with Observability Analysis

In **Part II** of this book, you learned about the fundamentals of instrumenting your code to emit telemetry. But that's only half of the equation. Collecting the right data is a fundamental requirement, but the true measure of observability is what you can learn about your systems from that data.

This chapter looks at how gathering the telemetry data you learned about radically changes how you're able to understand the behavior of your code. Working in high-observability systems, the analysis techniques you'll use differ drastically from those you typically use in more traditional, low-observability systems.

We'll start by examining the analysis techniques used in low-observability systems. Those approaches heavily rely on prior system familiarity and experience for investigators to successfully find answers. In contrast, with high-observability systems, investigators start with a structured process—the core analysis loop—that leads them to the correct source of issues. That difference results in both leveling the playing field when identifying issues and quickly accelerating the fixing of them.

The core analysis loop can be accomplished manually, but it's best when automated. In looking at how to automate analysis, we'll consider the roles that both humans and computers play in creating effective debugging workflows. We'll provide guidance when using AI assistants to automate telemetry data analysis. By demonstrating these analysis techniques, you'll understand why systems with a high degree of observability let you identify issues that are impossible to detect with low-observability tooling.

Debugging from Known Conditions

Prior to modern observability tooling, system and application debugging meant that you started from what you already knew. What does this particular threshold-based alert tell me I should be investigating? What does “normal” look like on this dashboard I’m seeing, and how does that compare to what is currently displayed?

Think back to when you first joined a new team and knew little about the system you were investigating. It’s likely you first needed to accumulate a fair amount of system knowledge and understanding of what “normal” meant before you could jump into an incident and lead the investigation.

When you’re new, it can seem downright magical when the most senior members of an engineering team approach their troubleshooting workflows. Somehow, they just know which questions are the right ones to ask. They instinctively know the right places to look. But that intuition isn’t magic; it’s more like accumulated scar tissue. With repeated exposure to previous system failures, they’ve memorized the various failure modes, the weird edge cases, and the dashboards or graphs that usually point at the right places to look.

Engineering team managers often understand that phenomenon quite well, because they’re usually the ones responsible for somehow bottling that “magic.” They urge their senior engineers to write detailed runbooks in an attempt to identify and solve every possible “root cause” they might encounter out in the wild. A scramble occurs to document the escalating collection of dashboards the team has accumulated and what each of them means.

Some documentation is essential: ownership, escalation, dependencies, and a few solid entry-point queries. But the time spent creating those runbooks and documenting dashboards is largely wasted in vain, because modern systems rarely fail in precisely the same way twice. And when they do, it’s increasingly common to configure an automated remediation that can correct that failure until someone can investigate it properly.

Anyone who has ever written or used a runbook can tell you a story about just how woefully inadequate they are. Perhaps they work to temporarily address technical debt: there’s one recurring issue, and the runbook tells other engineers how to mitigate the problem until the upcoming sprint when it can finally be resolved.

More often—especially with distributed systems—a long, thin tail of problems that *almost never happen* are responsible for cascading failures in production. Or, five seemingly impossible conditions will align just right to create a large-scale service failure in ways that might happen only once every few years.

On the Topic of Runbooks

The assertion that the time spent creating runbooks is largely wasted may seem a bit harsh at first. To be clear, some documentation you'll find in runbooks is essential. For example, for every service, you should capture basic information, including which team owns and maintains the service, how to reach the on-call engineer, escalation points, other services that this service depends on (and vice versa), and links to good queries or dashboards that serve as entry points to understanding its performance.

However, maintaining a living document that attempts to contain all possible system errors and resolutions is futile and dangerous. That type of documentation can quickly go stale, and *wrong documentation* is perhaps more dangerous than *no documentation*. In fast-changing systems, instrumentation itself is often the best possible documentation, because it combines intention (what are the dimensions that an engineer named and decided to collect?) with the real-time, up-to-date information of live status in production.

Yet engineers typically embrace these dynamics as “just the way that troubleshooting is done.” After all, that’s how debugging has worked for decades. But is that the best we can do?

Must we first intimately understand all parts of the system—whether through direct exposure and experience, documentation, or a runbook? Then we look at our dashboards and then we...intuit the answer? Or maybe we’re supposed to just make a guess at the root cause, and then start looking through our dashboards for evidence to confirm our guess?

Even after instrumenting your applications to emit observability data, you might still be debugging from known conditions. For example, you could take that stream of arbitrarily wide events and pipe it to `tail -f` and `grep` it for known strings, just as troubleshooting is done today with unstructured logs. Or you could take query results and stream them to a series of infinite dashboards, as troubleshooting is done today with metrics. You see a spike on one dashboard, and then you start flipping through dozens of other dashboards, visually pattern-matching for other similar shapes.

Collecting better telemetry data isn’t what enables you to debug unknown conditions. How you analyze that data is what makes all the difference.

At Honeycomb, it took us quite some time to figure this out, even after we’d built our own observability tooling. Our natural inclination as engineers was to jump straight to what we know about our systems. In Honeycomb’s early days, before we learned to stop debugging from known conditions, what better observability helped us do

was just jump to the right high-cardinality questions to ask about the hundreds of dimensions captured in our events.

If we'd just shipped a new frontend feature and were worried about performance, we'd ask, "How much did it change our CSS and JavaScript asset sizes?" Having just written that instrumentation, we would know to figure that out by calculating maximum `css_asset_size` and `js_asset_size`, and then breaking down performance by `build_id`. If we were worried about a new customer who was starting to use our services, we'd ask, "Are their queries fast?" Then we would just know to filter by `team_id` and calculate the P95 response time.

But that's still debugging from known conditions. What happens when you don't know what's wrong or where to start looking? What if you don't have the faintest idea of what could be happening? The key shift isn't just jumping further along in the process; it's *how* you investigate when you don't yet know the story.

When you don't yet know the story, then instead you *must* debug from first principles.

Debugging from First Principles

Debugging from first principles is the practice of treating your understanding as provisional until you can verify it with evidence. A **first principle** is a basic assumption about a system that was not deduced from another assumption. In philosophy, a first principle is defined as the first basis from which a thing is known. *Debugging from first principles* means using a methodology to understand a system scientifically.

Proper science requires that you do not assume anything. You must start by questioning what has been proven and what you are absolutely sure is true. Then, based on those principles: you form hypotheses, test them against your telemetry, and repeat until the explanation fits the observed behavior.

One of the most practical capabilities unlocked by modern observability is that the investigative process becomes both methodical and teachable. While intuitively jumping straight to the correct answer is wonderful, it becomes increasingly impractical as complexity skyrockets (and it *definitely* doesn't scale).

Instead of requiring everyone to become a system wizard for your team to achieve velocity, you can rely on a repeatable loop that guides investigators toward the conditions that actually explain the problem. This capability becomes even more essential in modern systems, where the sources of issues are often multicausal: what happens when you're wondering what went wrong, and the answer is "13 different things"?

That methodical and teachable loop is the foundation of analysis in high-observability systems.

Using the Core Analysis Loop

The core analysis loop is hypothesis-driven debugging: propose, test, refine. Form hypotheses, explore the data to confirm or deny them, and iterate based on what you learn.

That approach is not only more scientific than relying on intuition and pattern-matching, but it also democratizes the act of debugging. Using the core analysis loop is more rigorous than intuition-based troubleshooting, as it tends to reward curiosity and careful observation over pure system familiarity.

Success when debugging no longer skews mostly toward the most experienced engineers. Instead, anyone who is most curious or most diligent about observing their code in production can become the best investigator. That means even someone with very little knowledge of the system should be able to jump in and debug any given issue.

Let's look at how that occurs.

First, debugging from first principles typically begins when something signals that system conditions aren't normal; a customer report, a shift in performance, or an alert. (For effective alerting, see [Chapter 11](#).)

Regardless of the notification source, at this point you know that *something* is wrong (e.g., slow), but you don't know exactly what. Begin the process: query system telemetry to form an initial hypothesis, validate or invalidate that hypothesis with data, and iterate until you systematically arrive at an answer. [Figure 8-1](#) shows the four basic steps of the core analysis loop.

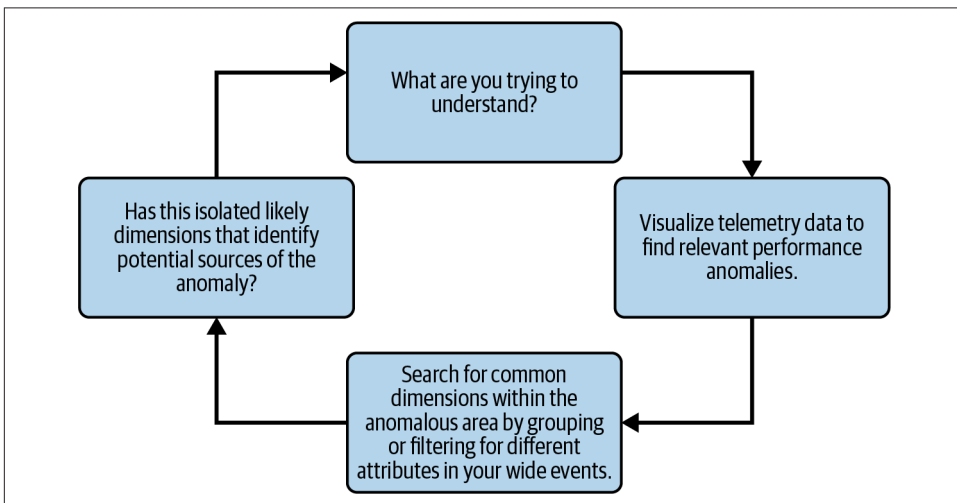


Figure 8-1. The core analysis loop.

The core analysis loop works like this:

1. Start with the overall view of what prompted your investigation: what did the customer, anomaly, or alert tell you?
2. Verify that what you know so far is true: is a notable change in performance happening somewhere in this system? Data visualizations can help you identify changes of behavior as a change in a curve somewhere in the graph.
3. Within that change of the curve, search for common dimensions that exist there but that aren't present in the baseline performance area (outside of the curve). Approaches to accomplish that might include:
 - a. Examining sample rows from the area that shows the change: are there any outliers in the columns that might give you a clue?
 - b. Slicing those rows across various dimensions looking for patterns: do any of those views highlight distinct behavior across one or more dimensions? Try an experimental group by on commonly useful fields, like `status_code`.
 - c. Filtering for particular dimensions or values within those rows to better expose potential outliers.
4. Do you now know enough about what might be occurring? If so, you're done! If not, filter your view to better isolate this area of performance as your next starting point. Then return to step 3.

This loop lets you move forward without needing a preexisting mental model of every subsystem. You can use it as a brute-force method to cycle through all available dimensions until you identify which ones explain or correlate with the outlier within the graph (but remember not to confuse correlation with causality). This method holds true, regardless of how familiar you already are (or aren't) with the system in question.

Of course, that brute-force method could take an inordinate amount of time and make such an approach impractical to leave in the hands of human operators alone. An observability tool should automate as much of that brute-force analysis for you as possible.

Automating the Brute-Force Portion of the Core Analysis Loop

The core analysis loop is a method to objectively find a signal that matters within a sea of otherwise-normal system noise. The brute-force work in that loop is churning through all of the noisy data to identify a signal that's different. Computers are great at that.

Rather than manually searching across rows and columns to coax out patterns, automation can retrieve the values of all dimensions, both inside the isolated area (the anomaly) and outside the area (the system baseline), diff them, and rank which fields and values are most strongly associated with the anomalous behavior. Automating that loop very quickly surfaces likely causal factors and gives an investigator the next best place to look.

For example, you might isolate a spike in request latency and, via an automated core analysis loop, get back a sorted list of dimensions ranked by how often they appear within that area. You might see the following:

- `request.endpoint` with value `batch` is in 100% of requests in the isolated area, but in only 20% of the baseline area.
- `handler_route` with value `/1/markers/` is in 100% of requests in the isolated area, but only 10% of the baseline area.
- `request.header.user_agent` is populated in 97% of requests in the isolated area, but 100% of the baseline area.

At a glance, that highlights what's different about the particular requests you care about—whether that be one deviation or dozens. As a concrete example, we'll look at the Honeycomb BubbleUp feature to show an automated core analysis loop.

With Honeycomb, you start by visualizing a heatmap to isolate a particular area of performance you care about. Honeycomb automates the core analysis loop with BubbleUp: you point and click at the spike or anomalous shape in the graph that concerns you, and draw a box around it. Then, BubbleUp computes the values of *all* dimensions both inside the box (the anomaly you care about and want to explain) and outside the box (the baseline), and it compares the two and sorts the resulting view by percentage of differences, as shown in [Figure 8-2](#).



Figure 8-2. A Honeycomb heatmap of event performance during a real incident. BubbleUp surfaces results for 63 interesting dimensions and ranks the results by largest percentage difference.

In this example, we’re looking at an application with high-dimensionality instrumentation that BubbleUp can compute and compare. The results of the computation are shown in histograms using two primary colors: blue for baseline dimensions and orange for dimensions in the selected anomaly area (blue appears as dark gray in the print image, and orange as lighter gray). In the top-left corner (the top results in the sort operation), we see a field named `global.availability_zone` with a value of `us-east-1a` showing up in only 17% of baseline events, but in 98% of anomalous events in the selected area (Figure 8-3).

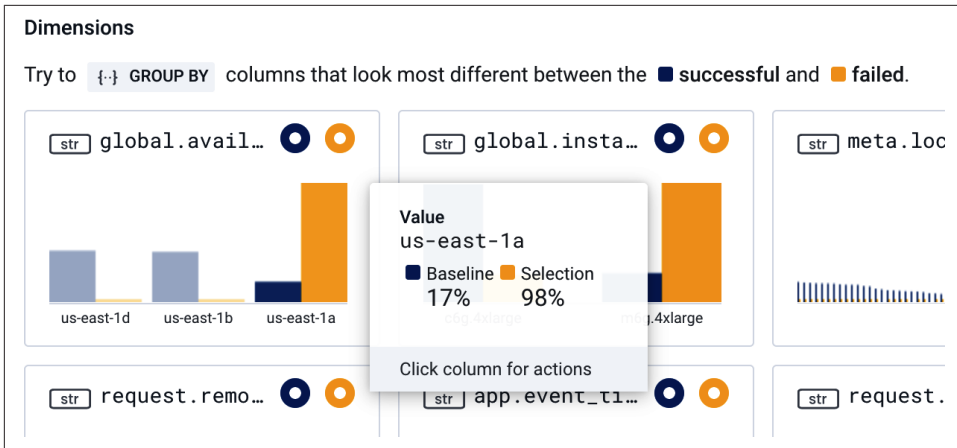


Figure 8-3. In this close-up of the results from [Figure 8-2](#), the orange bar (light gray in the print image) shows that `global.availability_zone` appears as `us-east-1a` in 98% of events in the selected area, and the blue bar (dark gray) shows that’s the case in only 17% of baseline events.

This information has been tremendously helpful. We now know the conditions that appear to be triggering slow performance: a particular type of instance in one particular AZ is much more prone to very slow performance than other infrastructure we care about. In that situation, the glaring difference pointed to what turned out to be an underlying network issue with our cloud provider’s entire AZ.

Not all issues are as immediately obvious as this underlying infrastructure issue. Often, you may need to look at other surfaced clues to triage code-related issues. The core analysis loop remains the same, and you may need to slice and dice across dimensions until one clear signal emerges, similar to the preceding example.

In this case, the example is drawn from an actual incident. We contacted our cloud provider and were also able to independently verify the unreported availability issue when our customers also reported similar issues in the same zone. If this had instead been a code-related issue, we might decide to reach out to those users, or figure out the path they followed through the user interface (UI) to see those errors, and fix the interface or the underlying system.

This is why arbitrarily wide, structured events matter: you need enough context to slice, dice, compare, and explain. Metrics typically cannot provide the necessary dimensionality. Logs can sometimes be correlated into event-like structures—presuming you have correctly appended all the request IDs, trace IDs, and other headers, and then done a great deal of postprocessing to reconstruct them into events. But that still requires the ability to aggregate those logs at read time and perform complex, custom querying against the aggregation.

In other words, even the manual version of the core analysis loop is unattainable without the basic building block of arbitrarily wide, structured events. And if it can't be done manually within your observability tooling to begin with, then it can't be automated.

The core analysis loop is a useful exercise to practice manually, to better understand exactly how the interesting causal dimensions get surfaced. However, automation is much more practical. Automation doesn't replace investigation. It removes the repetitive, time-consuming comparison work so humans can focus on interpretation and decision-making.

Because debugging from first principles does not require prior familiarity with the system, automation of the core analysis loop can be done simply and methodically, without the need to seed “intelligence” about the application being debugged.

That begs a question around how much intelligence should be applied to this number crunching. Is artificial intelligence and machine learning the inevitable solution to all our debugging problems?

Automating Analysis with Generative AI

In the first edition of this book, we wrote about the misleading promise of AIOps (artificial intelligence for operations)—a term coined by Gartner in 2017 to describe a functional intersection with observability that used machine learning to reduce alert noise and improve anomaly detection. At the time, we wrote that AI could only help if clearly discernible patterns existed and if it could be trained to use ever-changing baselines to model its predictions. Back then, that was a training pattern that had yet to emerge in the AI world.

We summed up the challenge of using AI as a tension between humans and machines, suggesting that we let computers do what they do best (churn through massive datasets to identify interesting patterns) and fuse that with what humans do best (add cognitive context to the interesting patterns that sift signals from noise). But now, in the era of generative AI, computers might arguably be able to do both. How does that impact today's considerations for automating analysis of observability data?

The pace of forward progress in the generative AI space is breathtaking, which presents a challenge for this book: can we write guidance that might not be obviated by the time you read this paragraph? With that in mind, instead we'll focus on process by discussing modes of analysis and how they intersect with the humans ultimately accountable for decisions and system reliability. We'll also look at how you can best prepare yourself for integrating agentic AI into your observability engineering practice.

The telos of observability is ultimately understanding and, through that understanding, to improve. Agentic AI systems rely solely on humans for the sense-making part of an investigatory loop—the “what” in “what are you trying to understand” mentioned earlier. Thus, you should begin your consideration of agentic tools by understanding where the AI sits in respect to the human: are the tools a copilot, a commander, or a caretaker? Let’s explore these three personas so that you can identify them.

Agentic AI Personas

The “copilot” sits alongside the human as they work to analyze an issue, investigate a performance regression, or generate further hypotheses. The copilot may have access to tools in order to independently execute tasks, or may use only what is provided by the human operator. Ultimately, the operator is using the agent as an extension of their knowledge and expertise. In return, the human must evaluate the output of the agent and steer it toward the correct place to look for answers and hypotheses. A practical example of this model is using ChatGPT or Claude Desktop to analyze stack traces, pictures of distributed trace data, and code in order to generate new queries or reason through function execution.

The “commander” is a step toward autonomy. In this mode, the human generates tasks and goals for an agent, which then goes to implement or run them with less oversight. A commander has access to tools, which enable the agent to write and run code, access remote data sources, surf the web, or almost anything else. Goals can be more fuzzily defined. For example, “Why is my service slow?” is a valid question. The agent will use its abilities to both interpret the question and determine an answer. Further, it can also offer solutions. An example of this type of agent is Cursor or Claude Code. Humans may choose to provide direct step-by-step oversight with these agents, or allow them to run in the background working on tasks, but they provide a level of separation between the work being performed and the operator. Commensurately, that separation also offers a weaker transference of understanding to the operator.

Lastly, the “caretaker” becomes not only autonomous, but also unmoored from direct human supervision. Rather than responding to queries or tasks, caretakers are passive observers with strictly defined goals. For example, these agents may be initialized with access to runbooks or other knowledge artifacts and placed in the path of alerts or deployments with a set of rules: “After a deploy, check each endpoint with a new feature flag, look up the ticket for that flag to understand why it exists, and see if the goal was attained” or “When an alert fires, go collect all relevant information and provide three hypotheses about what’s happening, and then post them in the appropriate incident channel.”

These agents are not science fiction: state-of-the-art generative AI models and machine learning technology puts them within reach, and interesting work is being done in this area.¹ Crucially, caretaker agents require a significant amount of knowledge transfer from humans to the agent as they are created or configured. The agent must be taught about the idiosyncrasies of your system.

Using Agentic AI for Observability in Practice

Considering these types of agents points us to a fundamental question: where does the context live? Is it in the mind of the human, or has it been properly transferred from human to agent? These are somewhat nuanced situations that will change as the technology evolves. However, let's consider what is true, regardless, in each of these cases.

Telemetry data quality is paramount. While a human may be able to cope with minor differences in attribute patterns or field naming across different services, those same subtle changes can have much larger implications to the quality of AI analysis. Does the primary actor have enough context to cope? One thing to note in these personas is that as the autonomy of AI increases, so does the amount of potential error—and the degree of error compounds.

If you're working with a copilot, you can just start a new session when the AI loses the plot. A commander may take erroneous actions, but you'll have some opportunities for more immediate oversight. A caretaker that collates incident data independently has many more opportunities for small interpretation errors to cascade into further service impacts, wasted time, loss of trust in tools, and even data loss or corruption.

Humans still matter—a lot. Irrespective of mode, goal, or tool, humans still own the outcomes. You will be responsible for the context the agent has, the goals that it seeks to achieve, and the outcomes of its behaviors.

As you experiment with agentic AI for observability, remember to embrace failure and uncertainty. A common critique of AI agents is that they're wrong. We would add some color to that critique. Often, it is less of a problem that the agent is wrong, and it's more that the agent is optimized for confidence.

A casual observer of this space may be under the impression that “hallucinations” are a major issue in AI agents. There's nuance to that take, and state-of-the-art models have better mitigated hallucinations. Far more likely is that the agent makes a rational

¹ Some interesting recent papers in this area include “[Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models](#)”, “[Building AI Agents for Autonomous Clouds](#)”, and “[X-lifecycle Learning for Cloud Incident Management using LLMs](#)”.

decision or inference based on the knowledge it has, and that response is perceived as irrational to its human observers based on their implicit knowledge.

That means it may take an agent more attempts to come to the same conclusion about an issue than it takes for a human; the agent must freshly reconstruct the system state each time. It might run multiple “incorrect” queries on its way to building a correct one. Here, your role as an operator is to ensure that the agent has the best information available so that it may steer itself in the right direction.

Let’s illustrate that concept in practice. An AI agent observed a telemetry field named `slow_request` that was always true when requests were over 300 milliseconds, and otherwise false. There was no other metadata available to this agent about what this field meant, and it could not determine any other immediately obvious cause.

The agent’s conclusion is that the requests were slow because of the `slow_request` attribute; it posits that perhaps this attribute indicates a load test, or that it is possibly used as a feature flag. In reality, this attribute simply indicates that the request took over 300 milliseconds. After adding a description explaining why this field exists, and why it would be true, the agent then omits it from its analysis.

The more autonomous that agentic AI becomes, the more critical it becomes for a need to provide better context about the environment. AI agents have specialized needs, but those needs are very human-like. AI needs accurate, succinct, and timely documentation about the systems it’s observing in order to reach better conclusions. Generative AI agents need high-quality telemetry data—not just normalized attributes and names, but semantic consistency, human-readable descriptions, and an ability to quickly search and enumerate the telemetry space. They need the freedom to investigate and iterate rapidly on hypotheses, but they also need guardrails and checks against potentially destructive actions.

In short, many of the things that you can do to improve system analysis with agentic AI also improves the ability of humans to perform the core analysis loop. (For a deeper dive into this topic, see [Chapter 10](#).)

Conclusion

Achieving a high degree of observability requires more than collecting the right telemetry. It requires analysis practices that work under any degree of uncertainty. The core analysis loop provides a repeatable, first-principles method for investigation that works well regardless of the complexity of your system. It works incredibly well for effective fault localization, but it can become very time-consuming for humans as data volume grows.

Automation makes it practical to apply the core analysis loop in even the most complex systems. Whether automated programmatically or by pairing with AI agents, the

workflow can be greatly accelerated. But that only further emphasizes the premium for collecting high-quality telemetry quality and capturing an abundance of system context. When you build your telemetry so that agents can reason about it safely and correctly, you also make it easier for humans to do the same.

Now that we've covered the basics for effectively analyzing observability data, we'll shift our focus to look at how that gets applied. Sometimes it's you or an agent as the one proactively looking for anomalies in the system. Other times, those anomalies in the system are the ones that come looking for you.

Observability-Driven Development

In **Part II**, we introduced instrumentation: what to emit and why. In the previous chapter, we focused on analysis: how telemetry becomes evidence that you can use to understand system behavior. This chapter closes the loop between those two actions.

Observability is often framed as something you use *after* a service is in production. That framing is incomplete. Observability should also be an early part of your software development practices. Telemetry should evolve alongside your code, and engineers should think about instrumentation any time a change could introduce new failure modes or shift performance in ways that matter to users.

That's the core idea behind observability-driven development; a practice similar to test-driven development (TDD), but covering different concerns. When observability drives code development, you'll ship new features together with the instrumentation needed to measure impacts and capture the context you'll need to verify their performance, compare them to previous versions, and debug them quickly when reality disagrees with your assumptions.

We'll start by reviewing TDD and its limits when applied to production environments. We'll look at how observability complements TDD by enabling rapid triage, systematic investigation in distributed systems, and shorter feedback loops between features and customer experiences. Lastly, we'll cover how this approach supports progressive delivery and how generative AI can assist—when guided by explicit standards.

Test-Driven Development

TDD is one of the most successful **shift-left** practices in modern software development. For the last two decades, it's been considered the gold standard for testing software prior to its release to production. In TDD, you define behavior through

tests. You run those tests constantly. You use failing tests to guide implementation. And this all works because the environment is controlled: data resets, dependencies are mocked, and variability is intentionally removed.

It's a powerful practice because it provides engineers with clarity around software operability. Application functions are defined by deterministic sets of constant, repeatable tests. If these repeatable tests pass, the application must be running as expected. Before changes to the application are produced, they start as a set of new tests that exist to verify that the change would work as expected. A developer can then begin to write new code in order to ensure the test passes.

TDD is particularly powerful because tests run the same way every time. Data typically doesn't persist between test runs; it gets dropped, erased, and re-created from scratch for each run. Responses from underlying or remote systems are stubbed or mocked. With TDD, developers are tasked with creating a specification that precisely defines the expected behaviors for an application in a controlled state. The role of tests is to identify any unexpected deviations from that controlled state so that engineers can then deal with them immediately. In doing so, TDD removes guesswork and provides consistency.

But that consistency and isolation are also what limits TDD's revelations about how your software behaves in production. Passing tests does not mean that users in production are experiencing good performance. Nor does it mean that errors or regressions will be easy to localize after releasing that code back into production.

Production is defined by variability. Any reasonably experienced engineer responsible for managing software running in production can tell you that production environments are anything but consistent. They're full of interesting deviations that your code might encounter out in the wild but that have been excised from tests because they're not repeatable, don't quite fit the specification, or don't go according to plan.

While the consistency and isolation of TDD makes your code tractable, it does not expose the long tail of strange conditions that only show up under real traffic, real data, and real dependency behavior.

TDD ensures correctness against a specification. Observability-driven development helps you verify behavior against reality.

Observability in the Development Cycle

Debugging a new feature is easiest immediately after that change ships. Catching bugs cleanly and resolving them swiftly is what prevents them from becoming a backlog of technical debt that weighs down development speed. Yet, software delivery and operations teams often hinder their ability to do so for a variety of reasons.

For example, consider that in some organizations, software engineers aren't responsible for operating their software in production. These engineers merge their code into main, cross their fingers in hopes that this change won't be one that breaks production, and essentially wait to get paged if a problem occurs.

Sometimes they get paged soon after deployment. The deployment is then rolled back, and the triggering changes are examined for bugs. But more likely, problems aren't detected for hours, days, weeks, or months after the code is merged. By that time, it becomes extremely difficult to pick out the origin of the bug, remember the context, or decipher the original intent behind why that code was written or why it shipped.

Right when the feature first ships, intent is still fresh in the original author's mind. The longer the detection is delayed—hours, days, weeks—the more expensive the investigation becomes (often by orders of magnitude) and the more likely teams are to rely on rollbacks and guesswork.

Observability-driven development addresses this by bundling instrumentation into the development workflow, so that it's present in production while the intent is still fresh. Done well, this bundling gives your team faster, more reliable answers when their systems behave unexpectedly after a deployment.

Determining Where to Debug

Newcomers to observability often mistake it as a replacement for debugging their code, similar to using highly verbose logging. While it's possible to debug your code using observability that way, that's not its primary purpose. Observability operates on the order of *systems*, not on the order of *functions*.

Emitting enough detail at the line level to reliably debug code could emit so much output that it would swamp most observability systems with an obscene amount of storage and processing needs. At scale, that approach is simply impractical and cost-prohibitive. And that still doesn't necessarily tell you where the problems are happening.

TDD is useful for debugging your code logic. *Observability is for figuring out where in your systems to find the code you need to debug.* High-observability tools narrow the search space. Which component originated the error? Where is latency being introduced? Where did a piece of this data get munged? Which hop is taking up the most processing time? Is that wait time evenly distributed across all users, or is it experienced by only a subset?

High-observability tools show you the component, dependency, user segment, build ID, infrastructure slice, or request path where the issue originates. Often, they'll also give you a good idea of what might be happening in or around an affected

component, indicate what the bug might be, or even provide hints as to where the bug is happening: in your code, the platform's code, or a higher-level architectural object.

Once you've isolated the likely location and conditions, observability's job is done. From there, if you want to dive deeper into the code itself, the tool you want is a good old-fashioned debugger (for example, the GNU Debugger, GDB) or a newer generation profiler (see [Chapter 20](#)).

Once you suspect how to reproduce the problem, you can spin up a local instance of the code, copy over the full context from the service, and continue your investigation. The following are examples of different paths you might take with debugging versus observability:

- You see a spike in latency. You start at the edge; group by endpoint; calculate average, 90th-, and 99th-percentile latencies; identify a cadre of slow requests; and trace one of them. It shows the timeouts begin at `service3`. You copy the context from the traced request into your local copy of the `service3` binary and attempt to reproduce it in the debugger or profiler.
- You see a spike in latency. You start at the edge; group by endpoint; calculate average, 90th-, and 99th-percentile latencies; and notice that exclusively write endpoints are suddenly slower. You group by database destination host, and note that the slow queries are distributed across some, but not all, of your database primaries. For those primaries, this is happening only to servers of a certain instance type or in a particular availability zone (AZ). You conclude that the problem is not a code problem, but one of infrastructure.

While they are related, the difference between an observability tool and a debugger is an order of scale; observability is the telescope that helps you aim the microscope. Those two scopes have some overlapping use cases, but they are primarily designed for different things.

Debugging in the Time of Microservices

When viewed through that lens, it becomes clear why the rise of microservices was tied so strongly to the rise of modern observability tools. Software systems used to have fewer components, which meant they were easier to reason about. An engineer could think their way through all possible problem areas by using only low-cardinality tagging. Then, to understand their code logic, they simply always used a debugger or IDE. But once monoliths started being decomposed into many distributed microservices, the debugger no longer worked as well because it couldn't hop the network.



As a historical aside, the phrase “*strace* for microservices” was an early attempt to describe this new style of understanding system internals before the word “observability” was adapted to fit the needs of introspecting production software systems.

When distributed systems communicate across networks, code logic and operational context become inseparable—irrevocably intertwined with the logic bugs we unintentionally shipped and inflicted on ourselves. In monolithic systems, it’s obvious to your debugger if a certain function slows immediately after code that modified that function is shipped. Now, such a change could manifest in several ways.

You might notice that a particular service is getting slower, or a set of dependent services is getting slower, or you might start seeing a spike in timeouts—or the only manifestation might be a user complaining. Who knows.

And regardless of how it manifests, it’s likely still incredibly unclear according to more traditional observability tools whether that slowness is being caused by any of the following:

- A bug in your code
- A particular user changing their usage pattern
- A database overflowing its capacity
- Network connection limits
- A misconfigured load balancer
- Issues with service registration or service discovery
- Some combination of the preceding factors

In a system with low observability, all you may see is that all of the performance graphs are either spiking or dipping at the same time.

How Instrumentation Drives Modern Observability

The promise of high-observability tooling depends on instrumentation quality. Those tools can pinpoint issue sources, causal attributes, and exactly which half a dozen or more things must all be true for the error to occur by examining behavior with rich context—build IDs, host groups, instance types, kernel versions, dependency targets, container versions, etc.

But that is only possible if that context is captured consistently by useful instrumentation. *Good instrumentation drives observability.* One way to think about how instrumentation is useful is to consider it in the context of pull requests. Pull requests

should never be submitted or accepted without first asking yourself, “How will I know if this change is working as intended?”

For example, let’s say you introduced a new caching mechanism into your application to improve performance. While helpful, that new functionality is also a potential source of failure. When introducing that change, you should also add instrumentation to your code that tracks specific cache operations, such as when data is put into the cache or retrieved from it. You should set attributes like `cache.put`, `cache.get`, `cache.key`, and `cache.hit` to capture information about the new cache operations in your functions.

Aim to develop instrumentation that creates reinforcement mechanisms and shorter feedback loops. In other words, tighten the loop between shipping code and feeling the consequences of errors. This is also known as: *putting the software engineers on call*.

One way to achieve this is to automatically page the person who just merged the code being shipped. For a brief period of time, maybe 30 to 60 minutes, if an alert is triggered in production, the alert gets routed to that person. When an engineer experiences their own code in production, their ability (and motivation) to instrument their code for faster isolation and resolution of issues naturally increases.

That feedback loop is not punishment; rather, it is *essential* to code ownership. You cannot develop the instincts and practices needed to ship quality code if you are insulated from the feedback of your errors. Every engineer should be expected to instrument their code such that they can answer these questions as soon as it’s deployed:

- Is your code doing what you expected it to do?
- How does it compare to the previous version?
- Are users actively using your code?
- Are any abnormal conditions emerging?

A more advanced approach is to enable engineers to test their code against a small subset of production traffic. With sufficient instrumentation, the best way to understand how a proposed change will work in production is to measure how it will work by deploying it to production. That can be done in several controlled ways. For example, that can happen by deploying new features behind a feature flag and exposing it to only a subset of users. Alternatively, a feature could also be deployed directly to production and have only select requests from particular users routed to the new feature. These types of approaches shorten feedback loops to mere seconds or minutes, rather than what are usually substantially longer periods of time waiting for the release.

If you are capturing sufficient instrumentation detail in the context of your requests, you can systematically start at the edge of any problem and work your way to the correct answer every single time, with no guessing, intuition, or prior knowledge needed. This is one revolutionary advance that observability has over monitoring systems, and it does a lot to move operations engineering back into the realm of science—not magic or intuition.

Shifting Observability Left

Teams that cannot reason accurately about production tend to treat it as fragile. They roll back quickly because they can't distinguish “real regression” from “expected variance,” or they can't isolate the cause. Over time, production becomes a place they fear to touch. Shifting left with observability changes that relationship.

Bundling instrumentation with new features gives engineers an opportunity to consider how they would see the small variance of changes that happen in production. Your software might behave according to specification in a small, sterile, and controlled testing environment, but what will it actually do when it reaches production? If engineers focus only on adherence to a spec, they inadvertently create the conditions that lead to fearing production deployments.

When engineers can't reliably reason about production, they begin to see it as a glass castle—a beautiful monument to their collective design over time, but one they're afraid to disturb because any unforeseen movement could shatter the entire structure. By contrast, engineers who bundle useful telemetry into new features learn how to understand behaviors in production. They become empowered to reach further and further into the development lifecycle to consider the nuances of detecting unforeseen anomalies that could be roaming around their castles unseen (for guidance on reasoning about and measuring software reliability, see [Chapter 11](#)).

Observability-driven development enables engineering teams to turn their glass castles into interactive playgrounds. Production environments aren't immutable but full of action, and engineers should be empowered to confidently walk into any game and score a win.

But this only works when observability is treated as a shared engineering practice; not a specialized activity reserved for SREs or operations roles. Software engineers must adopt observability and work it into their development practices in order to unwind the cycle of fear they've developed over making any changes to production.

Using Observability to Speed Up Software Delivery

A common perception in the software industry is that a trade-off usually occurs between speed and quality: you can release software quickly or you can release high-quality software, but not both. A key finding of *Accelerate: Building and Scaling High Performing Technology Organizations* was that this inverse relationship is a myth.¹ For elite performers, speed and quality go up in tandem and reinforce each other. When speed gets faster, failures become smaller, happen less often, and—when they do happen—are easier to recover from. Conversely, for teams that move slowly, failures tend to happen more often and take substantially longer to recover.

A practical “North Star metric” is lead time to production: the time from code written to code running in production. Observability-driven development supports improved lead time by reducing uncertainty during rollout and making investigation faster when something goes wrong.

When production is a glass castle, deployment rollbacks happen instinctively and comfortably at the slightest sign of any potential issues. Lacking visibility and confidence, these teams inadvertently take their foot off the gas and halt any further deployments or changes while they roll back the current change they don’t fully understand. That’s exactly the opposite reaction to what would be most helpful in that situation.

This all hinges on your team’s ability to release code to production on a relatively speedy cadence. If that’s not how your team currently operates, here are a few ways you can help speed up code delivery to production:

Reduce batching.

Ship a single coherent bundle of changes, one at a time, one merge by one engineer, more frequently. The single greatest cause of deployments that break “something” and take hours or days to detangle and roll back is the batching of many changes by many people over many days.

Invest in the deployment pipeline.

Spend real engineering effort on your deployment process and code. Assign experienced engineers to own it (not the intern). Make sure everyone can understand your deployment pipelines and that they feel empowered to continuously improve them. Don’t let them be the sole province of a single engineer or team. Instead, get everyone’s usage, buy-in, and contributions (see [Chapter 18](#)).

¹ Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: Building and Scaling High Performing Technology Organizations* (IT Revolution Press, 2018).

When software engineers practice observability-driven development, they deploy often and successfully. Patterns like using feature flags and progressive delivery that decouple deployments from releases enable engineers to observe and understand the performance of their new feature as it is slowly released to production (see [Chapter 2](#)).

Observability-Driven Development with AI

You should be adding instrumentation early in the code development lifecycle. You should use it to answer critical questions about how your code is performing. And you should release that code often. But exactly what instrumentation should you be adding?

In the past, frustratingly, the answer has been, “it depends.” Today, that’s still the answer. But a bright spot in guidance and determining exactly which instrumentation you should add can be addressed in part by leveraging AI-assisted coding.

An emerging pattern in the practice of software engineering is the “copilot” model of agentic AI (see [Chapter 8](#)), where a developer works alongside an AI agent. These agents are capable of building a semantic understanding of your codebase and system in order to write new code, discover bugs, and generally act as a useful—if slightly overeager and obsequious—assistant.

With the rapid pace of AI innovation, it’s challenging to provide evergreen guidance on how to best use these tools to practice observability-driven development. To illustrate, between when the first edition of this book was launched in 2022 and mid-2025, half a dozen AI-powered developer tools have not only launched but also achieved over \$100 million in annual recurring revenue (ARR).² There’s really no telling what disruptive changes will be next and no guessing which methods will soon be obsolete.

That said, a few patterns and practices should remain evergreen as we collectively navigate a course through the future of software engineering and development. To that end, we’ll share what we’ve learned so far about observability-driven development with AI.

First, AI makes it *extremely easy* to instrument your code. Prompting an agent to “add OpenTelemetry support” to a new or existing codebase will quite often simply *work*, creating useful spans, metrics, and logs. Where things get tricky is that working telemetry and *useful* telemetry are two very different concepts. Agents will happily

² GitHub Copilot (launched June 2022 → hit \$100 million ARR ≈ 12 months); ChatGPT (Nov 2022 → < \$100 million ARR threshold within ≈ 6–9 months; revenue already \$1.3 billion ARR by October 2023); Cursor (April 2023 → \$100 million ARR in ≈ 12 months); Replit AI Agent (Sep 2024 → \$100 million ARR in ≈ 8 months); Lovable (Nov 2024 → \$100 million ARR in ≈ 8 months).

invent attributes, duplicate metrics and logs with abandon, and create overly detailed traces that can overwhelm you with noise.

Crucially, you must provide detailed instructions, preferably in an agent configuration, with general guidance on the outcomes it should achieve. Group observability guidance under a single header to ensure that the AI doesn't have to manually collate guidance from multiple sections. Provide detailed but direct guidance.

Examples of good guidance include statements like:

- “OpenTelemetry spans are our primary observability signal. Ensure that span context is preserved through function calls.”
- “Display preference for applying attributes relating to an HTTP handler to its root span rather than children.”
- “Do not duplicate telemetry signals; layer them instead.”
- “Each HTTP route handler *must* contain the following attributes: `app.customer.id` and `app.customer.plan_type`. These are in the request context as ‘customerId’ and ‘plan.’”

As demonstrated, your objective is to provide the agent with as much context as it needs in order to make good decisions. If you know what the source for an attribute value is, provide that mapping to the agent. Provide documentation links or other guidance where appropriate, and refer to standards such as OTel—the AI most likely has context about the common semantic meanings to interpret the meanings of such things.

Second, AI makes code *cheap*, in aggregate. Perhaps you've read the concept that “all code is technical debt”. It's the idea that every feature, line of code, and design decision is a liability against your future productivity. Another way to think about this concept is that code itself has a *carrying cost* in terms of maintenance burden. That burden is expressed in different ways.

For example, poor interface design of an internal API can result in compounding performance issues due to unnecessary allocations. Repetitive patterns can increase the cognitive load of interpreting failures—a gap that can let bugs sneak into your code. And one-off features can result in dizzying combinations of failure scenarios that are difficult to test. AI assistance in writing code does not change any of these burdens. In fact, it makes them worse (see [Chapter 2](#)). In our internal use of AI

copilots for code generation, we have seen that AI-coauthored commits on a mature codebase were, on average, 3.4x longer and had a 5.5x increase in net impact.³

While it is possible that results like this will change as AI tooling improves, what won't change is that the importance of observability in generated code will remain paramount. Given that AI makes code cheap, your role as a developer using AI assistance is to reduce the carrying cost of the new code that's being added. The best way to do that is to give the AI good models and patterns to follow.

Rather than necessitating that the AI guess at what's important in terms of attributes, have it create centralized helper libraries of constants and methods that it can refer to and add to your code as it generates new methods. You can also leverage that same approach to have an AI migrate your legacy telemetry to new patterns. Spend some time working with an agent to perform these migrations and build up a comprehensive prompt that describes what information should be added or changed. For example, help the AI understand what parameters and variables should map to which attributes throughout the codebase, and then use that prompt on other legacy services.

Finally, consider blending test and observability-driven development patterns together with the assistance of AI. Tools like OpenTelemetry Weaver enable you to build schemas for your telemetry attributes and data, which can be fed into CI/CD pipelines as static checks for your codebase. You can also shift this left: give your AI agents access to your telemetry schemas, run local validation against the emitted telemetry from a service, and focus on tight loops with automated verification of changes. The more that the AI can check its own work against a known-good specification or pattern, the faster (and safer) it can make changes to a codebase (for more detail on using AI in observability workflows, see [Chapter 10](#)).

Conclusion

Observability can and should begin early in the SDLC. TDD helps ensure behavior against a specification. Observability-driven development helps ensure correct behavior in the variability and complexity of production. Adopting that practice enriches your team's analysis of their observability data.

High-performing teams treat observability as a core engineering practice. Instrumentation should evolve with the application so that analysis remains effective across the system's lifecycle. That evolution and constant practice of observing and understanding the variability and complexity of production is what stops production

³ Over one quarter, we measured coauthored commits between a human and AI developer. The AI-assisted commits were on average 607.4 lines of code versus 177.8 without, and the net change in codebase size was 444.1 versus 81.

from being a glass castle. And it's what equips engineering teams to ship faster, with confidence.

We've looked at analysis through the lens of actions initiated by engineers: often, we're the ones who go looking for issues. So far, our discussion of observability-driven development has assumed a human at every gate. Next, we've invited guest author Boris Tane to explore how AI agents are starting to take on some of the investigative groundwork, and what that means for how we design our systems.

The Role of AI Agents for Observability

*This chapter is contributed by Boris Tane,
Founder and CEO of Coreplane Labs*

A Note From Charity, Liz, George, and Austin

In this chapter, we're going to dive deeper into the role of AI and agents in the world of observability practices, and how they can help you implement observability-driven development.

We need to say this part out loud: this is one of the chapters that risks aging in dog years. The agentic AI space is moving fast. Models are improving, context windows are expanding, and the ways people build and operate agents are changing so fast that any advice we give could feel dated by the time you read this. So, rather than try to predict the exact shape of the tooling landscape, we are going to focus on something more durable: how to think about this problem space and which first principles should guide your decisions as the technology continues to evolve.

Today's agents are typically constrained by incomplete context, uneven tool use, and limits on what they can hold in working memory. Some of the advice in this chapter is a function of those limitations. But we hope that the deeper lesson you'll take from this content isn't about any one particular limitation or architectural pattern.

Observability has always been about the tension between context and precision. Whether held by humans or machines, the outcome will continue to depend on the quality, completeness, and accessibility of the context available to them.

To be clear, we authors are under no illusion that this space will do anything but keep up the dramatic rate of change. However, we believe the principles in this chapter should hold for the foreseeable future: instrument richly, preserve context, design strong feedback loops, and be mindful of where automation helps and where the best

judgment is possible. The exact tools and techniques may change. The need for these principles will not.

The rest of this chapter is written from Boris Tane's perspective.

The way engineers understand and operate software is changing in ways that would have seemed far-fetched just a few years ago. Observability has always been about giving humans better tools to make sense of their systems: better dashboards, better query languages, better alerting pipelines. The feedback loops that have shaped software engineering for decades—the cycle of writing, deploying, observing, and fixing—were built on the assumption of a human at every gate, synthesizing signals, and making judgments about what to do next.

That assumption is being quietly dismantled, and the implications for how we think about observability are significant. As AI agents begin to take a seat at the software development table—surfacing data but also reasoning over it, acting on it, and increasingly shaping the systems that produce it in the first place—we are witnessing an inflection point for the entire software development lifecycle. This shift promises a world where agents handle the mechanical investigative groundwork, but it also creates a new, urgent responsibility for human engineers: providing the high-fidelity context that these systems require to operate safely.

This chapter explores the practical utility of agents within observability, identifies the boundaries where they fall short, and argues that the gap between success and failure continues to come down to a single underlying problem: context.

What Is an AI Agent for Observability?

In [Chapter 7](#), an AI agent was defined as a software harness that combines a generative LLM and some subset of tools, or remote procedure calls exposed through an API. For observability, AI agents use an LLM as their reasoning engine, and the tools give it the ability to take actions such as querying databases, calling other APIs, writing and executing code, or triggering automated workflows.

Unlike a chatbot that simply responds to questions in a single turn, an agent can pursue a goal across multiple steps—deciding what to do next based on what it learns along the way and adjusting its approach when earlier assumptions turn out to be wrong. Applying that to an observability workflow means that an agent could follow a complex decision tree when an alert is triggered, such as shown in [Figure 10-1](#).

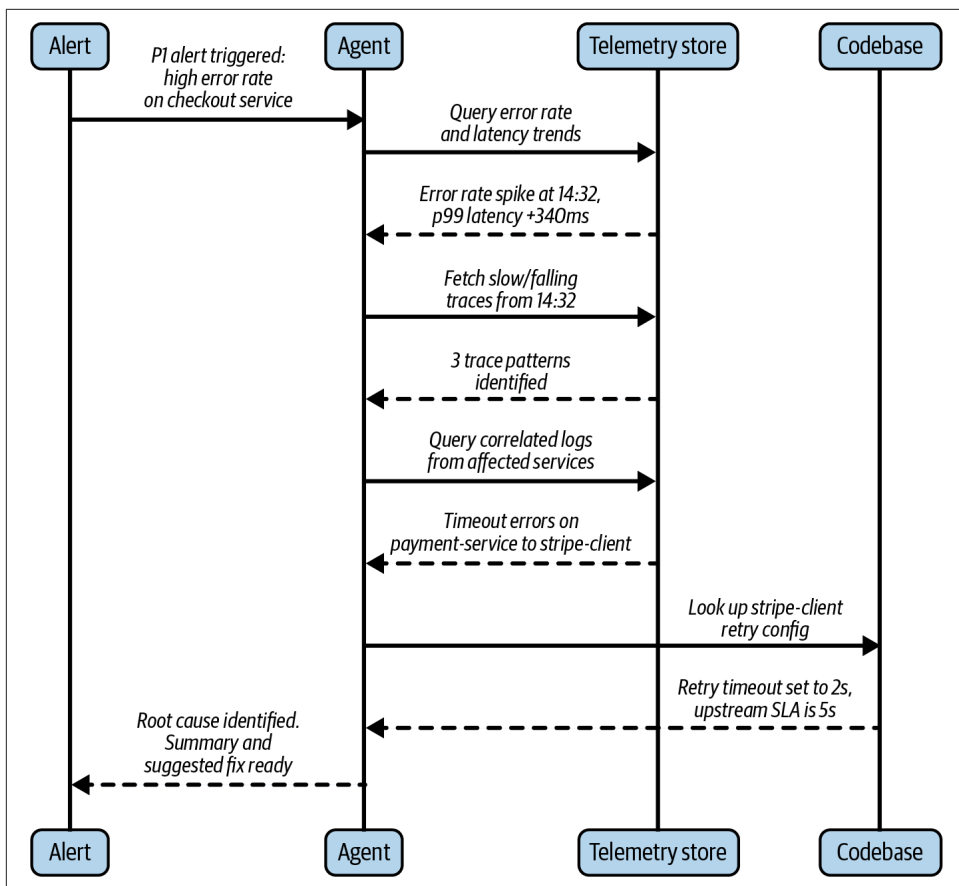


Figure 10-1. This diagram illustrates an observability agent’s role in an automated incident response workflow. Triggered by a P1 checkout error alert, the agent performs a multistep investigation: it analyzes telemetry trends, traces, and logs while inspecting code configurations.

In this example, the agent identifies a timeout mismatch in the `stripe-client` and provides a root-cause summary with a suggested fix, demonstrating how agents add value through complex, cross-system correlation rather than simple queries.

In **Figure 10-1**, the agent’s workflow starts with a P1 alert for a high error rate on the checkout service. It then shows the agent querying the telemetry store for error-rate and latency trends, pulling slow or failing traces, correlating logs from affected services, and checking the codebase for configuration details. In this example, the agent discovers that a retry timeout in `stripe-client` is set too low relative to the upstream SLA, and then returns a root-cause summary and suggested fix.

Observability agents are most useful when they can perform multistep investigations across telemetry and system context, not just answer a single query. The example paints a compelling picture. While this example is used to illustrate the principle, it's important to note that parts of this workflow already work today.

As of the publishing of this book, AI agents are already able to respond to alerts and perform tool calls across both telemetry data and codebase. Agents fall short when the data sources are too disparate, with poor labeling—leading the agent to go round in circles, trying to reason about unrelated data. However, the underlying technology is improving quickly.

While workflows like this can be prototyped, the path from “parts of this work” to “you can trust this in production at scale” is where things get complicated. Before you consider building anything serious on top of agent-powered observability tooling, it's important to understand why things get complicated.

The Pitfalls of Querying Without Context

One of the simplest ways to begin integrating an agent into your observability workflows is by asking it to query your telemetry data for analysis; in other words, you're delegating data retrieval.

Give an agent access to your telemetry data, and let it answer questions in natural language: “What's causing the spike in errors?” or “Which service has the highest latency over the past hour?” or “Show me all logs from the payments service in the last 30 minutes.” By granting an LLM access to your telemetry store, you can bypass the hurdle of complex query languages. For teams that spend hours wrestling with complex query languages, having a natural language interface is a genuine improvement.

While this is a natural place to start, the problem with this common agent integration approach is that querying in isolation produces poor results. The resulting failure modes are instructive because they reveal the fundamental limitation of agents operating without sufficient context:

The agent doesn't know what it doesn't know.

When an experienced engineer investigates an incident, they begin with a hypothesis that is shaped by weeks or months of accumulated context. For instance, they know the payments service tends to misbehave after deployments to a specific downstream dependency. They might know a particular upstream API has been flaky since the infrastructure team migrated it last quarter. They also know what “normal” looks like on a regular Tuesday afternoon, versus a Tuesday afternoon following a long weekend. An agent querying telemetry data cold has none of that. It will find what the query returns, which may be accurate but is often beside the point.

Naming is chaos in production.

Services get named by whoever created them, at whatever stage of a company's growth that happened to be, and the names accumulate rather than converge. You will have `user-service`, `users-svc`, `svc-user`, and `legacy-user-api` all doing related things in the same system, with varying degrees of documentation. An agent without a canonical map of that nomenclature will confidently draw conclusions from incomplete or misattributed data, and it will do so without flagging that it might be wrong.

The data surface is enormous.

A moderately complex production system might produce millions of spans per minute across dozens of services. Without knowing which signals are meaningful in a given context, an agent will either miss the relevant pattern in an ocean of noise or surface correlations that are technically present in the data but operationally irrelevant. Knowing what to look at is itself a form of expertise, and that expertise has to come from somewhere.

Correlation is not causation, and agents often conflate the two.

Agents (again, limiting to those using LLMs) are good at finding patterns in data, but they are not inherently good at distinguishing “these two things happened at the same time” from “this thing caused that thing.” In a distributed system where dozens of services are constantly interacting and covarying, spurious correlations are abundant. Without an explicit topology model and some sense of the causal structure of the system, an agent attributing a root cause is doing something closer to educated guessing than analysis.

You might get surprisingly good results by piping error logs directly to an LLM in a small, well-named, well-instrumented system where the services are few enough to reason about in a single prompt. But the systems that matter most to organizations—the ones where getting observability right is genuinely critical to the business—are the ones that are large, messy, and built over years by rotating teams with different conventions. Those are exactly the environments where naive agent approaches break down most visibly.

Proven Use Cases for Observability Agents

Agents are not just a future promise projected onto an incomplete present. There are areas where they are already adding real value in the observability space. Understanding what those areas have in common is as useful as understanding the individual use cases.

Let's look at three proven use cases where agents demonstrate a high degree of value.

Incident Response

When an alert fires, the first 15 minutes of an incident are often consumed by the same mechanical investigative steps: pulling the relevant dashboards, querying recent logs, checking what was deployed in the last few hours, and mapping the blast radius across dependent services.

This work is important but largely procedural, and it is well-suited to an agent. An agent can execute those steps faster than a human, run several queries in parallel, and surface a structured summary of the situation before the on-call engineer has even fully oriented themselves.

The best implementations here are careful about scope. They use agents to augment the on-call engineer rather than attempting to automate incident resolution end to end. The agent handles the investigative groundwork; the human interprets the summary, applies domain judgment, and makes the decision about what to do. Given where reliability and trust in autonomous agents currently sit, this division of responsibility is both practical and appropriate.

Explaining Errors and Patterns in Telemetry Data

One of the most underappreciated use cases for agents in observability is translation: taking raw telemetry signals and producing coherent, plain-language explanations of what they mean. A stack trace, a slow trace waterfall showing cascading latency, an anomalous shift in a metric's distribution—these things require genuine expertise to interpret, and that expertise is unevenly distributed across engineering teams.

An agent that can look at a trace and explain that “this shows a sequential chain of database calls where each one waits for the previous to complete, which is a classic N+1 query pattern introduced by an ORM (object-relational mapping) mis-configuration” is compressing hours of learning and debugging into something a junior engineer can act on immediately. This matters both for incident response, where time is the constraint, and for building organizational understanding over time by surfacing patterns that engineers might otherwise not have the bandwidth to investigate.

Improving Instrumentation Quality

Using agents to help with instrumentation is a use case that often doesn't get enough attention in discussions about AI and observability, but it can give you great long-term leverage (for a deep dive on improving instrumentation with agents, see [Chapter 7](#)).

Getting teams to instrument their code well is genuinely difficult. Developers understand, in the abstract, that they should add spans with meaningful attributes, emit

structured wide events with rich context, and propagate distributed tracing headers across service boundaries. In practice, instrumentation is consistently an afterthought. It gets added inconsistently, by whoever last touched a given file, and the quality degrades over time as the codebase grows and the people who set the original conventions move on.

Agents are well-positioned to address that gap. Given a function, a service boundary, or a pull request, an agent can identify what instrumentation is missing, what is structured poorly, and what does not conform to the team's conventions, and then implement improvements correctly and consistently. Best practices like wide events, proper attribute naming, and appropriate log levels—which take years to develop as institutional knowledge and spread unevenly across teams—can be encoded into an agent's context and applied at the point of code authorship rather than bolted on after the fact.

Across all three of these use cases, what will make or break your success is how much context your agent has over your system. With poor context or vague prompting, agents produce generic output that looks plausible but does not fit the system or the team's conventions. An agent instructed to “add observability to this service” without knowing your instrumentation library, your attribute naming conventions, your service's role in the broader architecture, or your logging standards will produce something that compiles but does not help. With precise, rich context, the same agent produces output that a senior engineer would approve and merge without significant changes.

Along these lines, there are a few additional places that agents can be incredibly useful. Let's look at those use cases as well.

Observability-Adjacent Use Cases for Agents

Beyond the core observability workflows, there is a whole category of operational work that teams have historically been poor at, not because the work is technically difficult, but because it requires sustained attention and is consistently crowded out by more urgent priorities. Agents are well-suited to a lot of this work.

Alert quality is a perennial disaster in most engineering organizations. Alerts get added during incidents or out of anxiety, tuned rarely if ever, and accumulate over time into a wall of noise that engineers learn to ignore. Teams end up with hundreds of active alerts, many of which fire regularly and are silently acknowledged without investigation because everyone knows they are not meaningful.

An agent can audit an alert configuration against actual incident history and traffic patterns, identify which alerts correlate with real user-facing problems, flag which ones fire constantly without ever leading to meaningful action, and propose specific threshold adjustments based on observed baselines. Importantly, this can be done

continuously rather than as a periodic cleanup exercise that never quite makes it to the top of the backlog.

Service level objectives suffer from a related problem. Defining good SLOs requires a real understanding of traffic patterns, historical reliability, the distribution of error types, and what users actually experience. Most teams either skip SLOs entirely, set them at whatever feels intuitively reasonable without grounding in data, or copy a template from a blog post and apply it without adapting it to their system.

An agent with access to historical performance data and a model of the services can propose SLO definitions that are grounded in observed behavior. It can identify which SLOs are being approached before they are breached, and can surface trends that indicate a service is drifting toward unreliability before it becomes an incident. (For detailed guidance on using agents for SLO creation, see [Chapter 11](#).)

Both of these represent work that teams universally agree is important but struggle to prioritize because the cost of not doing them is diffuse and deferred. Agents make this category of work tractable by removing the activation energy required to get started and by operating continuously rather than in occasional bursts of attention.

While the use cases we've covered demonstrate the high leverage that agents can provide in controlled scenarios, their effectiveness is fundamentally tethered to the environment they inhabit. But as we increasingly rely on more automation, we also risk losing the institutional knowledge that makes providing rich context possible.

The Production Problem: The Mental Model That's Disappearing

Production environments are messy, and that messiness is a fundamental property rather than a failure of discipline. Services accumulate over years of changing priorities, changing teams, and changing infrastructure constraints. Names drift as organizations restructure. The team that originally built a service has turned over twice, and the documentation, if it ever existed, was last updated 18 months ago. A critical business path runs through four different languages, three cloud providers, and a legacy service that nobody wants to touch but everyone depends on. There are operational quirks that only two people know about, and one of those people left six months ago.

When an experienced engineer investigates a production issue, they navigate this complexity using a mental model that has been assembled over months or years of working with the system. For example, they know the topology of services: which things call which other things, where the critical paths are, and which dependencies are known to be unreliable. They know the nomenclature: that `auth-proxy` and `api-gateway` are the same service, depending on which team you ask, and that `orders-v2`

is what actually processes checkouts despite the name suggesting otherwise. They know the history: that the telemetry from the inventory service has been partially broken since the Q3 migration and its error rates cannot be taken at face value. All of this knowledge is carried implicitly, and it shapes every query they write and every hypothesis they form.

For an agent to be effective in production, it also needs access to that mental model, represented explicitly as context that it can reason over, as shown in [Figure 10-2](#).

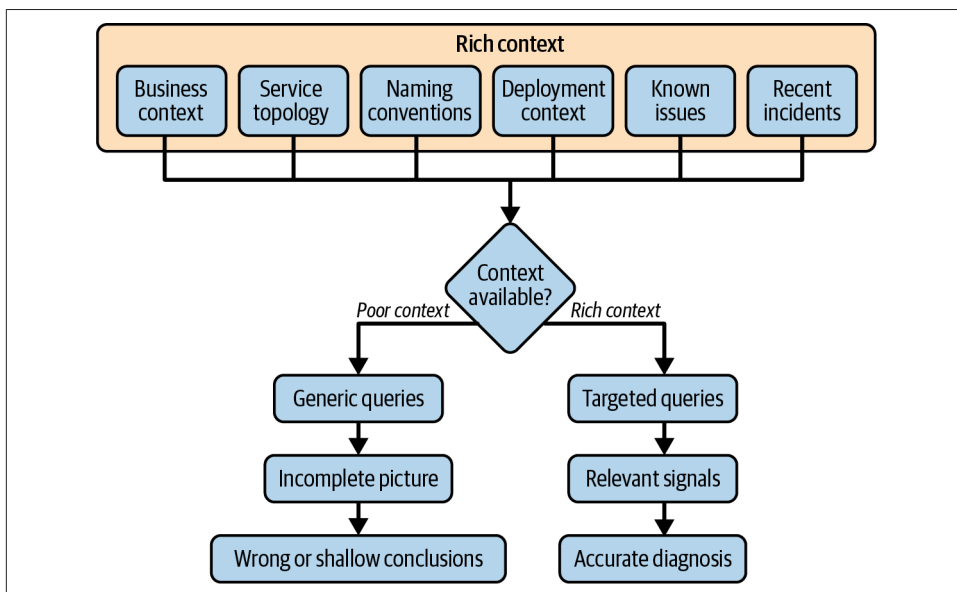


Figure 10-2. A decision flow centered around available operational context.

An agent may need rich context around concepts like service topology, naming conventions, critical paths, known quirks, or recent incidents. With poor context, the agent issues generic queries, gets an incomplete picture, and reaches incorrect or shallow conclusions. With rich context, the agent can run targeted queries, identify relevant signals, and arrive at an accurate diagnosis. The biggest differentiator in agent effectiveness is not the model alone, but the completeness and quality of the operational context it has available.

The context that an observability agent needs to cover may include several distinct layers of knowledge:

Service topology

What calls what, what depends on what, where the critical paths run, and which services are at the edges versus at the core of the system

Naming conventions

A canonical mapping of service names, their aliases, their owners, and what they actually do

Deployment context

What has changed recently, which services have active deployments, and where the risk surface is elevated

Known issues

Long-running bugs, degraded dependencies, services that are under maintenance or running in a degraded mode

Recent incidents

What went wrong in the last 30 days, which services were involved, how long they were degraded, what the blast radius was, and whether the root cause was ever actually confirmed

Business context

What a given error rate or latency change actually means for the business, which endpoints are revenue-critical, and what the acceptable thresholds are from a user impact perspective

The uncomfortable reality is that maintaining and providing this context so that an agent can be successful is getting harder over time, not easier. With the rise of coding agents that write and deploy code with limited human supervision, the engineers who would have built up that mental model through the ordinary process of authoring, debugging, and operating software are increasingly removed from the details. Code gets written and shipped at a pace that makes it difficult for any individual to maintain a coherent picture of the whole system, and the organizational knowledge that would have accumulated around a service through the ordinary cycle of incidents and postmortems forms more slowly. The mental model becomes fragmented and incomplete.

Agents operating in production need to compensate for this erosion of institutional knowledge. They need to be given—explicitly and in a maintained form—the context that engineers previously held in their heads. Building that context layer, keeping it current as the system evolves, and making it available to agents at the moment it is needed, is one of the most important and least-solved problems in this space. Teams that treat it as a first-class engineering concern will get significantly more out of their agent integrations than teams that treat it as something the agent should figure out on its own. (For more detail on increasing organizational learning speed, see [Part VI](#) of this book.)

The Need for Context

We are watching the software development lifecycle get reinvented in real time. Observability is at the center of that shift, because it's both a tool that agents use and a domain that agents are beginning to operate in. For decades, the cycle of writing, deploying, observing, and fixing assumed a human at every step. In the AI era, that assumption is being eroded simultaneously at multiple points in the cycle.

Coding agents write and modify code. Deployment pipelines automate the path to production with decreasing human checkpoints. Observability agents monitor running systems and surface findings. The role of the engineer is shifting from direct execution toward direction: setting context, reviewing outputs, and making judgment calls at the boundaries where automation is not yet reliable enough to be trusted alone.

The pace of change in AI is fast—faster than most organizations' capacity to adapt (see [Chapter 24](#)). Tools and patterns that were clearly experimental six months ago are now running in production today. What works is changing quickly, and the space is full of genuine uncertainty about what the right architectures and workflows look like.

What won't change is the underlying principle: just like their human counterparts, *agents are only as good as the context they're given*. That is true for incident response, for instrumentation quality, for alerting, for SLOs, and for everything else in the observability space. The teams that will get the most out of AI in their observability practice are the ones that invest seriously in building and maintaining that context layer—the machine-readable equivalent of the mental model that senior engineers carry and that has historically lived only in people's heads.

The tooling will keep improving. The models will get better. But the context problem is yours to solve.

Conclusion

Boris provides a salient look at how to navigate the fast-changing world of AI with regard to observability. Observability has been, and continues to be, highly dependent on context to reach the correct conclusions. Inevitably, how that context is gathered and who holds it throughout an investigation will change. But we believe the principles outlined in this chapter will continue to be true, even as the underlying technology rapidly evolves.

In the next chapter, we'll cover how to use SLOs for alerting and to trigger investigations. As you'll learn, SLOs are the best fit for alerting when working with high-observability systems because they focus on the “who” and “what,” and leave the observability tooling free to explain the “how” and “why.”

Using Service Level Objectives for Reliability

In the last few chapters, we've considered situations requiring analysis that are initiated by engineers curious about a specific aspect of their system, as well as how AI agents can proactively perform these types of analysis. But an equally important consideration is how we conduct reactive investigations and maintain system stability, and analyze the situations that come looking for us. How should you be setting up alerts in high-observability systems in order to defend the experience of your users?

In this chapter, we'll look at why threshold-based alerting so often burns out operators through noisy alarms (*alert fatigue*), why distributed systems make those problems worse, and how SLOs provide a more reliable, user-centered alternative. When you can debug production safely and effectively in high-observability systems, service level objectives (SLOs) provide several advantages over their traditional alerting counterparts. We'll examine those advantages and include a real-world case study of accelerating the adoption of SLOs and getting to market faster by starting with standardized, high-quality telemetry (guest contributed by Mat Vine).

One of the hardest parts of the SLO adoption journey is simply getting started by drawing a line in the sand. Conventionally, that has required a great deal of expertise, experimentation, and time. In this chapter, we'll also show you how to leverage generative AI to accelerate the process.

We'll start by recapping the state of monitoring and alerting in low-observability systems.

Threshold-Based Alerting Creates Alert Fatigue

In threshold-based monitoring, alerts are set up to measure what the team already views as its internal priorities, or what's easiest to measure. Metrics are used to track simplistic attributes derived from the system state in hopes that a specific threshold value correlates with service failure or may be a leading indicator of troubles ahead. For example, trigger an alert if CPU utilization is above 80% for five minutes, or if available memory drops below 10%, or if disk space is nearly full, or if more than x many threads are running—or any set of other simple measures.

These signals can be useful as data, but as alerting triggers they're often terrible. Deviations that spiked CPU utilization don't tell you whether your customers are having a bad experience. That might indicate that a backup job is running. It might be garbage collection. It might be a real outage. Or it may be the circuit breaker kicking in to prevent cascading failure, so the actual outage is longer than the spike we observe. The traditional monitor can't distinguish that granularity, so it triggers the alert. Over time, the team may intuitively learn that the alert isn't accurate enough and automatically dismiss it. When these signals are used as “potential-cause” indicators—indirect proxies that may or may not correlate to user impact—they tend to create a high percentage of false alarms.

Experienced engineering teams that own the operation of their software in production will often learn to tune out (or even suppress) these types of alerts because they routinely go off and do not correlate with actual production impact or urgency. Teams that do so regularly will adopt phrases like “Don't worry about that alert; we know the process runs out of memory from time to time. We might write automation to restart it if the memory gets too high.”

Becoming accustomed to alerts with many false positives is a known problem and a dangerous practice. In other industries, that problem is known as *normalization of deviance*: a term coined during the [investigation of the Challenger disaster](#). When individuals in an organization regularly shut off alarms or fail to take action, they eventually become so desensitized about the practice deviating from the expected response that it no longer feels wrong to them.

At best, failures that are “normal” and disregarded are simply treated as background noise. But that's not a harmless coping mechanism. When ignoring alerts becomes normal, the one alert that truly matters has to fight its way through a wall of learned distrust. The background noise sets up all the wrong incentives leading to oversights from cascading system failures.

In the software industry, the poor signal-to-noise ratio of threshold-based alerting often leads to alert fatigue—and to gradually paying less attention to all alerts, because so many of them are false alarms, not actionable, or simply not useful. Unfortunately,

with threshold-based alerting, incidents compound the noise as every alarm fires simultaneously.

Post-incident reviews often generate action items that create new, more important alerts that, presumably, would have triggered in time to prevent the problem. That leads to an even larger set of alerts generated during the next incident, unless stale or flappy alerts are cleaned up. That pattern of alert escalation can create an ever-increasing flood of alerts and an ever-increasing cognitive load on responding engineers to determine which alerts matter and which don't.

That type of dysfunction is so common in the software industry that many monitoring and incident-response tool vendors proudly offer various solutions labeled "AIOps," which group, suppress, or otherwise try to process that alert load for you. Additional layers mean additional complexity and potential sources of missed or spurious alerts.

How did that become the acceptable and "normal" state of affairs? If the future of running software in production is doomed to generate so much noise that it must be artificially processed, it's safe to say the situation has gone well beyond a normalization of deviance. Industry vendors have now *productized* that deviance and will happily sell you a machine learning solution for its management.

Metric and threshold-based alerting were once, long ago, the best tools we had to understand the state of complex systems. As an industry, we now are layering on further dysfunction and bandages rather than questioning whether those primitives are the correct ones to build on.

Distributed systems make that dysfunction harder to ignore. When teams scale the number of intercommunicating services they manage, and failure modes grow exponentially, what used to just be annoying becomes an unacceptable reliability risk. Modern systems require an alternative approach.

Threshold Alerting Is Only for Known-Unknowns

Threshold alerting works best when you can enumerate failure modes and write checks for each one. In smaller, self-contained systems (a.k.a. monoliths), operations teams can write specific monitoring checks for each precise state they expect to see in production and, generally speaking, that works. But let's be clear: threshold alerts work best when you already have a fairly crisp and comprehensive mental model of what "bad" looks like. In other words, they're a great fit for known-unknowns.

A "known-unknown" is something that you are aware exists, but you don't have specific information about. In other words, you know your system has specific risks, but not which ones will occur, or their exact severity. Known-unknowns are gaps in knowledge that we recognize, and that we (typically) have a plan for.

With threshold alerting, these are failure modes you know exist, you can name, and you can measure, even if you don't know whether they're happening right now. For example, known-unknowns are failure modes like:

- The disk is 95% full.
- The queue depth is over 70%.
- TLS (Transport Layer Security) certification expires in 10 days.
- The error rate is over 10% for five minutes.

You can write a check to measure system state, define a threshold, and perform an action if the threshold is crossed (for instance, sending an engineer a page), because this condition reliably predicts imminent customer pain, and we know what to do about it.

In distributed systems, failure modes multiply. Distributed systems don't eliminate known-unknowns; they just become a tiny fraction of all possible failures that could occur. As systems grow more interconnected, the number of plausible things that could go wrong stops being additive and it becomes combinatorial.

Teams shifting from smaller or self-contained to distributed systems often respond by trying to codify intuition: every remembered outage becomes a new alert, every scary graph becomes a new dashboard, and every “we should have known” incident review becomes another defined threshold.

For a while, that approach feels like rigor. In practice, it becomes an arms race you cannot win. Trying to anticipate every possible failure or system state leads to hundreds or thousands of checks that are brittle, expensive to maintain, and rooted in tacit knowledge from past outages. That approach isn't sustainable. The checks aren't maintainable, often the knowledge isn't transferable, and past historical behavior doesn't necessarily predict future failures. And you're still not focused on what customers are actually experiencing.

Distributed systems are where it becomes much more of the norm to encounter unknown-unknowns. Unknown-unknowns are the failures you didn't know to anticipate. They're things like emergent behaviors, weird interactions, cascading retries, partial brownouts, dependency timeouts that only matter under a particular traffic mix of very specific factors, a “harmless” deployment that tips a downstream cache into eviction churn, and on and on and on. These are risks or factors so unforeseen that they weren't anywhere even near the radar—often a completely novel and never previously contemplated issue.

You can't write threshold alerts for issues you can't yet imagine. Modern application architectures regularly distribute load across multiple disparate systems. They split up, shard, horizontally partition, and replicate data across geographically distributed

systems. While these architectural decisions optimize for performance, resiliency, and scalability, they can also make systemic failures near impossible to detect or predict. Here, unknown-unknowns aren't rare edge cases. They're the default tax that you pay for scale and complexity.

Worse, threshold alerts tend to fuse *symptom* with *cause* in a way that's comforting, but misleading. In a monolith, an abnormal number of threads could mean trouble—or it could mean garbage collection doing its job, or the app loading an updated library for the first time. In a distributed system, abnormal threads in one component might be unrelated and benign noise, or it might be the first breadcrumb of upstream latency that will hit customers in 10 minutes.

At 2 a.m., when production is inexplicably broken, those distinctions matter. The on-call engineer isn't just being asked to respond; they're being asked to *divine* whether the alert is real, whether it matters, and where to start looking. Thresholds are supposed to reduce uncertainty. Most often, they just move that uncertainty into your incident response channel.

Threshold alerts can also be triggered by automation. Modern systems already remediate many common failure modes without human help: autoscaling, failovers, circuit breakers, load balancing, and so forth. When your system heals itself, triggering a notification isn't necessary. Not only is that not an emergency but it's a terrible distraction. If your alerts wake people up for conditions that self-resolve, you're not building reliability—you're manufacturing burnout. Those issues still deserve investigation, but that's a low priority accomplished during normal business hours.

Here's the boundary: threshold alerting is a tool for known-unknowns. It can be a fit for conditions with strong, repeatable, direct correlation to user impact and with a clear remediation action. But they're not a fit for detecting unknown-unknowns. No static list of checks can cover the full space of emergent failures in complex systems.

In systems with high degrees of observability, we've seen how the core analysis loop lets you quickly debug anomalies. When you're able to readily debug issues in production, the way to detect unknown-unknowns isn't by elusively attempting to set alerts for every possible condition. Instead, you focus on keeping only helpful alerts and removing the rest.

Let's define the criteria for an alert to be considered helpful. Google's *Site Reliability Engineering*¹ (SRE) suggests that a good alert must reflect urgent user impact, must be actionable, must be novel, and must require investigation rather than rote action.²

1 Niall Richard Murphy, Betsy Beyer, Chris Jones, and Jennifer Petoff. *Site Reliability Engineering* (O'Reilly, 2016).

2 Rob Ewaschuk and Betsy Beyer. "Monitoring Distributed Systems". In *Site Reliability Engineering*. (O'Reilly, 2016, 63–64).

We'll take that a step further by defining the criteria for an alert to be considered “helpful” with a two-part subset. First, it must be a reliable indicator that the user experience (UX) of your service is in a degraded state. Second, the alert must be solvable. There must be a systematic (but not pure rote automatable) way to debug and take action in response to the alert—that does not require a responder to divine the correct course of action. If those two conditions are not true, any alert you have configured is no longer serving its intended purpose.

User Experience Is a North Star

To echo the words of the SRE book once more, potential-cause alerts have poor correlation to real problems, but symptom-of-user-pain alerts better allow you to understand user impacts and the state of the system as experienced by customers.

How, then, do you focus on setting up alerts to detect failures that impact user experience? This is where a departure from traditional monitoring approaches becomes necessary. The traditional metrics-based monitoring approach relies on using static thresholds to define optimal system conditions. Yet the performance of modern systems—even at the infrastructure level—often changes shape dynamically under different workloads. Static thresholds simply aren't up to the task of monitoring impact on user experience.

Setting up traditional alerting mechanisms to monitor user experience means that system engineers must choose arbitrary constants that predict when that experience is poor. For example, these alerts might be implemented to trigger when “10 users have experienced slow page-load times,” or “the 95th percentile of requests have persisted above a certain number of milliseconds.” In a threshold-based approach, system engineers are required to divine which exact static measures indicate that unacceptable problems are occurring.

Yet system performance varies significantly throughout the day as different users, in different time zones, interact with your services in different ways. During slow traffic times, when you may have hundreds of concurrent sessions, 10 users experiencing slow page-load times might be a significant metric. But that significance drops sharply at peak load times when you may have tens of thousands of concurrent sessions running.

Remember that in distributed systems, failure is inevitable. Small transient failures are always occurring without you necessarily noticing. Common examples include a failed request that later succeeds on a retry; a critical process that initially fails, but its completion is merely delayed until it gets routed to a newly provisioned host; or a service that becomes unresponsive until its requests are routed to a backup service. The additional latency introduced by these types of transient failures might blend

into normal operations during peak traffic, but P95 response times would be more sensitive to individual datapoints during periods of low traffic.

In a similar vein, these examples also illustrate the coarseness of time-based metrics. Let's say P95 response time is being gauged in five-minute intervals. Every five minutes, performance over the trailing five-minute interval reports a value that triggers an alert if it exceeds a static threshold. If that value exceeds the threshold, the entire five-minute interval is considered bad (and conversely, any five-minute interval that didn't exceed the threshold is considered good). Alerting on that type of metric results in high rates of both false positives and false negatives. It also has a level of granularity that is insufficient to diagnose exactly when and where a problem may have occurred.

Static thresholds are too rigid and coarse to reliably indicate degraded user experience in a dynamic environment. They lack context. Reliable alerts need a finer level of both granularity and reliability. This is where SLOs can help.

Reliable Alerting with Service Level Objectives

SLOs are internal goals for the measurement of service health. Popularized by the [Google SRE book](#), SLOs are a key part of setting external SLAs between service providers and their customers. These internal measures are typically more stringent than externally facing agreements or availability commitments. With that added stringency, SLOs can provide a safety net that helps teams identify and remediate issues before external user experience reaches unacceptable levels.



We recommend Alex Hidalgo's book, *Implementing Service Level Objectives* (O'Reilly, 2020), for a deeper understanding of the subject.

Much has been written about the use of SLOs for service reliability, and they are not unique to the world of observability. However, using an SLO-based approach to monitoring service health requires having observability built into your applications. SLOs can be, and sometimes are, implemented in systems without observability. But the ramifications of doing so can have severe unintended consequences.

SLOs quantify an agreed-upon target for service availability, based on critical end-user journeys rather than system metrics. That target is measured using service level indicators (SLIs), which categorize the system state as good or bad. Two kinds of SLIs exist: *time-based measures* (e.g., “99th percentile latency less than 300 milliseconds over each five-minute window”) and *event-based measures* (e.g., “proportion of events that took less than 300 milliseconds during a given rolling time window”).

Both attempt to express the impact on end users but differ on whether data about the incoming user traffic has been pre-aggregated by time bucket. We recommend setting SLIs that use event-based measures as opposed to time-based measures, because event-based measures provide a more reliable and more granular way to quantify the state of a service. We discuss the reasons for this in the next chapter.

Let's look at an example of how to define an event-based SLI. You might define a good customer experience as being a state when "a user should be able to successfully load your home page and see a result quickly." Expressing that with an SLI means qualifying events and then determining whether they meet your conditions. In this example, your SLI would do the following:

1. Look for any event with a request path of */home*.
2. Screen qualifying events for conditions in which the event duration is less than 100 milliseconds.
3. If the event duration is less than 100 milliseconds *and* was served successfully, consider it OK.
4. If the event duration is at least 100 milliseconds, consider that event an error even if it returned a success code.

Any event that is an error would spend some of the error budget allowed in your SLO. We'll closely examine patterns for proactively managing SLO error budgets and triggering alerts in the next chapter. For now, we'll summarize by saying that given enough errors, your systems could alert you to a potential breach of the SLO.

SLOs narrow the scope of your alerts to consider only symptoms that impact what the users of your service experience. If an underlying condition is impacting "a user loading the home page and seeing it quickly," an alert should be triggered, because someone needs to investigate why. However, there is no correlation as to *why* and *how* the service might be degraded. We simply know that something is wrong.

In contrast, traditional monitoring relies on a potential-cause-based approach: a previously known cause is detected (e.g., an abnormal number of threads), signaling that users might experience undesirable symptoms (slow page-load times). That approach fuses the "what" and "why" of a situation in an attempt to help pinpoint where investigation should begin. But, as you've seen, there's a lot more to your services than just the known states of up, down, or even slow. Emergent failure modes can impact your users long before coarse synthetic probes will tell you. Decoupling "what" from "why" is one of the most important distinctions in writing good monitoring with maximum signal and minimum noise.

The second criteria for an alert to be helpful is that it must be *actionable*. A system-level potential-cause alert that the CPU is high tells you nothing about whether users are impacted and whether you should take action. In contrast, SLO-based alerts are

symptom based: they tell you that something is wrong. They are actionable because now it is up to the responder to determine why users are seeing an impact and to mitigate. However, if you cannot sufficiently debug your systems, resolving the problem will be challenging. That's where a shift to observability becomes essential: debugging in production must be safe and natural.

In an SLO-based world, you need high-observability tooling because it allows you to debug from first principles. With rich telemetry, you can start wide and then filter to reduce the search space. That approach means you can respond to determine the source of any problem, regardless of how novel or emergent the failure may be. You can improvise and adapt solutions to find any problem in your system.

Instrumentation that provides rich and meaningful telemetry is the basis for that approach. Being able to quickly analyze that telemetry data to validate or falsify hypotheses is what empowers you to feel confidence in removing noisy, unhelpful alerts. Decoupling the “what” from the “why” in your alerting is possible when using SLO-based alerts in tandem with observability.

When most alerts are not actionable, that quickly leads to alert fatigue. To eliminate that problem, it's time to delete all of your unactionable alerts. If that seems infeasible, it likely means you'll also need a cultural shift to support that change.

To illustrate how culture change is achieved around the adoption of SLOs for monitoring, we'll look at a real example with a firsthand account from Mat Vine with ANZ Bank—a multinational bank and financial services company headquartered in Australia.

Case Study: Changing Culture Toward SLO-Based Alerts

*This section is contributed by Mat Vine,
Cloud, Quality, and Reliability Lead at ANZ Bank*

When I first joined an incumbent bank, I was challenged to build the SRE practice within the transformation arm of the enterprise. On my first day, I sat in on my new team's first “major incident” rehearsal. The very next day, my boss asked for a launch readiness estimate. I thought 12–18 months was a realistic timeframe. He countered with a goal of 12 weeks before our pilot launch. For many in operational roles, this immediate, high-pressure shift in expectations would be enough to send them into a panic. Ultimately, would you believe that our reliability team was able to green-light a launch within 12 weeks? Why was my estimate so off? Using high-quality telemetry, defining SLOs, and setting error budgets enabled us to ship with confidence.

Investing in telemetry first is what buys you the leverage to move faster on the cultural and organizational work that actually sticks. Fortunately, as a major Google Cloud partner, we had the privilege of leadership who'd already been educated about

SLOs and error budgets through Google’s Customer Reliability Engineering program and the SRE book, written by Googlers and published by O’Reilly. We had a modern cloud native architecture, built predominantly in-house, and we’d made early bets on OpenTelemetry—which gave us a level of telemetry standardization that we’d eventually heavily lean on.

On the surface, this was a luxurious greenfield launching pad. It should have made me far more optimistic than my estimate of 12–18 months suggested. The reality, though, was that we were in a phase of rapid development approaching our pilot launch. Infrastructure and software were being built and changed quickly. The product itself was still being designed, tested, and reimplemented. These were surprisingly poor conditions from which to start an SLO revolution. So, why didn’t we just fall back to tried-and-true threshold alerting? We viewed that as a one-way trap because arbitrary thresholds tend to calcify. The easiest objective to set is “whatever performance we have today,” but that creates a feedback loop where you are alerting on statistical deviations rather than customer impact.

Instead, we realized that a small, core set of user journeys touched the majority of our critical assets and dependencies. By focusing only on those, we could achieve high system coverage without drowning in noise or tempting team members to proliferate more thresholds. This enables us to treat the infrastructure of alerting as a commodity, while spending our limited time on higher-value work: building consensus on how the system *should* perform, rather than just alerting on how it was performing.

In mature organizations with established SLO adoption, you would start by introducing SLOs at the design phase for each new component or system. But in a new, rapidly developing system, without any immediate operational pressure or customer satisfaction to balance, SLOs are too abstract. My estimate of 12–18 months was based on a then-current lead time of one year, during which no SLOs had been set. I thought there to be two fundamental conditions for SLOs to deliver real value:

Real-world demand on your systems.

This means not theoretical traffic, not load tests, but actual users depending on the systems you’ve built.

Scale—real scale.

SLOs need to be anywhere across your organization where there exists delivery or operational pressure, not isolated to a single team or practice area. SLOs need to be understood as a part of your team’s operating model.

So, how could we replace these?

In our scenario, the approaching deadline forced clarity. We stopped debating what *might* matter and started identifying what *would* matter on launch day. This meant narrowing down to less than a handful of user journeys across multiple problem

domains that would be genuinely important to the day-one customer experience. There would be time later to flesh out more comprehensive SLOs.

Our early bets in OTel had taken the form of an in-house package which extended the OTel SDK with our own domain-specific semantic conventions. Because most of our services were already emitting telemetry with predictable schemas, we were able to avoid spending weeks writing regular expressions and mapping data fields across heterogeneous logs. That amount of time, which would have been enormous if we'd had a less standardized environment, was reinvested into more valuable activities: mapping user journeys, rationalizing which metrics made sense as SLIs, and building consensus across development, operations, and product stakeholders. Although we didn't have real-world demand, we knew that we could get faithful estimates with load testing, and adjust in real time after our product launched.

SLOs, like any high-observability implementation, are an investment. If the only time your teams interact with observability data is during incident response, you're either underinvesting in making that data useful, or you're paying too much for it. The same applies to SLOs; they require organizational conditions to thrive. Teams need to be challenged to make decisions that balance risk and velocity. If that type of pressure doesn't exist—as it rarely does during the early development phase—SLOs are unlikely to gain the traction needed to make an appreciable dent in the way you build or operate your systems.

I'm skeptical of applying the standard technology rollout for SLOs where you pick one high-performing team, get them to use the tool, and then hope that the trial usage tells you enough about how to scale it organization-wide. The problem is that high-performing teams almost never need SLOs. Those teams prioritize and govern themselves naturally. They balance risk and velocity through their own internal processes—be it internal tech debt policies, toil indicators, or strong stakeholder relationships. That performance tends to come with trust already built. For these teams, SLOs aren't a needle mover.

If you're starting from scratch, instead identify the most problematic or important areas in your organization. Target the teams that are supporting the systems most critical to your service, those experiencing the highest operational stress, or those with the least productive or trusting relationships with product and business stakeholders. These are the teams where SLOs will shift behavior and decision making.

When you target these teams, make sure to focus energy directly on what matters: establishing clear objectives, rationalizing the indicators you've selected, and building genuine consensus on those choices across all stakeholders. Wrangling telemetry data into the correct shape might eventually be necessary, but it's almost never the most important thing to do first. If a lot of your time is being spent there, you likely don't have the foundation to launch from.

My experience is that high-quality telemetry requires investment as a standalone capability. Good telemetry—standardized, semantically rich, and with high cardinality—provides immediate benefits for teams. It can help them learn about their systems and alleviate operational stress when it arises. Likely, the investment in a mostly standardized telemetry solution will pay off for most organizations even if the change stops there. Once in place, however, you will have a backbone that enables your engineering teams to focus on the important cultural and procedural scaffolding necessary to make SLOs scale across the organization effectively.

The reality is that my original estimate of 12–18 months was both right and wrong. Making SLOs a genuine part of our SDLC, bringing them into product prioritization conversations, and eventually mapping hundreds of user journeys and business processes into defined objectives *did* take 18 months. Fortunately, establishing a strong enough base for us to have confidence in our product’s beta launch was achievable in 12 weeks. We implemented relatively comprehensive SLO-based monitoring systems within a few months because our early investment in OTel and the standardization introduced enabled us to focus on architecture, business alignment, and our operational practices.

This difference matters when scaling across an organization. Everyone involved in the software delivery lifecycle falls in love with the idea of error budgets at first sight. But the work to set objectives, agree on them, and build consensus on what a working system is and how often it should work is both difficult and necessary. Every time you get pulled into crafting regex or stitching together session IDs across heterogeneous data formats, you lose time on the harder, more valuable work. By embracing unified observability from the start, and building SLOs based on that observability data, we lowered the friction required to succeed with SLOs.

Accelerating SLO Adoption with Generative AI

Mat Vine’s account of SLO adoption at a regulated enterprise highlights many of the challenges and benefits of SLO adoption. Seeking organizational alignment can be particularly challenging for many teams. Often, that’s what seems to make drafting SLIs and SLOs deceptively hard when you’re getting started. It’s less that teams face difficulty in writing good queries, and more that SLOs force you to get organizational clarity.

You have to decide which user journeys matter most, what “success” means for those journeys, and how much imperfection the business is willing to tolerate. That decision tree requires translating product intent into measurable criteria, aligning stakeholders who may have competing priorities, and then choosing thresholds that strike a balance between not constantly generating noise nor quietly missing a degraded experience. Early SLO efforts often get stuck in ambiguity: everyone agrees

reliability matters, but nobody wants to be the person who picks the number or owns making the trade-off.

Generative AI can help accelerate this early stage. LLMs can help by solving the blank-sheet-of-paper problem: they can turn vague intent into a concrete draft fast enough to then allow the real work to begin—review, debate, calibration, and iteration. There is no substitute for actual human understanding, ownership, and judgment.

In our practice as SREs, we have found LLMs to be useful for scenarios beyond simply writing application code. After all, infrastructure can be written as code, and so can observability primitives such as saved queries, dashboards, and SLOs. Those artifacts encode assumptions about what matters and how to measure it. With that context, LLMs are particularly good at taking ambiguous human language about requirements and producing draft recommendations that you can refine.

Conventionally, it would take hiring an experienced SRE (as an employee or consultant) to propose reasonable SLO targets and explain trade-offs. In many cases, an LLM can provide a workable first pass when you give it the right context about the service and the business expectations. It doesn't replace expertise, but it can reduce the time it takes to create good working drafts.

Even without post-training, custom prompting, or retrieval augmented generation (RAG), using open-weights models can still yield surprisingly good results. For example, when asked what reliability target to set for a public-cloud checkout workflow processing \$1 million per month, a model might initially recommend an aggressive target. When challenged on cost and diminishing returns, it may revise the target and provide a rationale that frames the trade-off in business terms:

```
Prompt = What reliability percentage goal should I target for an ecommerce
checkout workflow running on a public cloud, where the service
processes $1 million in transactions per month? Answer succinctly
with just a number and a rationale.
```

```
gpt-oss 20b = 99.999 % availability - the "five-nines" target keeps annual
downtime under 5 minutes, minimizing revenue loss and meeting high-expectation
e-commerce uptime standards.
```

```
Prompt = Are you sure? That seems really costly to achieve. Trying to prevent the
last 45 minutes of lost revenue seems not worth it.
```

```
gpt-oss 20b = **Target: 99.95 % availability** - this limits annual downtime to
~42 minutes, which for a $1 M / month (≈$12 M / yr) revenue stream translates to
<$1 k lost revenue per year. 99.95% is attainable with a simple multi-AZ,
load-balanced architecture and avoids the high cost of five-nines while still
keeping customer experience strong.
```

Another model may choose a different number, but still provide a plausible explanation. For example, Claude has a similar response:

Prompt = What reliability percentage goal should I target for an ecommerce checkout workflow running on a public cloud, where the service processes \$1 million in transactions per month? Answer succinctly with just a number and a rationale.

Claude Opus 4.6 = **99.99%** (four nines)

A checkout workflow is directly revenue-bearing, so every minute of downtime has a quantifiable cost: at \$1M/month, that's roughly \$23 per minute of lost transactions. Three nines (99.9%) gives you ~43 minutes of downtime per month, costing ~\$1,000 in lost revenue plus customer trust erosion and cart abandonment that compounds beyond the outage window. Five nines (99.999%) is largely unachievable on public cloud without multi-region active-active, which is expensive to build and operate correctly for stateful checkout flows.

Four nines (~4.3 minutes/month) hits the pragmatic sweet spot: its achievable on a single cloud region with redundant components, the ~\$100/month in theoretical lost-revenue exposure is tolerable, and the engineering investment (health checks, retries, circuit breakers, graceful degradation) is proportionate to the business value. It also aligns with what major cloud providers commit to for their own managed services, so you're not trying to out-SLO your dependencies.

As experienced SREs, we'd probably say the numerical values and explanations (after some challenging) are roughly in line with what we might suggest to a product owner asking us questions about return on investment (ROI) for targeting higher or lower reliability for a given transaction volume. You can continue to dig and ask "why not 99.9%?" or "why not 99.999%?" and get sensible-enough answers to chew on. Even when you disagree with the recommendation, you now have something concrete to evaluate—and that accelerates alignment.

From Targets to Implementation: Drafting a Service Level Indicator

LLMs are also useful for moving what to measure toward concrete expressions. They can then help you craft SLIs corresponding to the SLOs leveraging your observability tooling when given sufficient context about the telemetry data. Event-based SLIs often require carefully defined eligibility criteria and success criteria. Models can draft those expressions quickly. But details matter, and the mistakes tend to be subtle, like missing an important filter, choosing the wrong error signal, or failing to align with your system's query or grammar conventions.

The following example demonstrates how the conversation from earlier can be continued, assuming the user fills in the blanks for the grammar and functions available in their query language:

Prompt = Now, you will be generating an expression in a domain specific language to define a service level indicator (SLI). The SLI measures the performance and reliability of a service per request based on

certain criteria, to be used for computing a higher-level SLO.

Here is the title describing the goal of the overall SLO:

```
<title>
Checkout succeeds quickly enough
</title>
```

And here is a more detailed description of the overall SLO:

```
<description>
Checkout should succeed within four seconds, 99.99% of the time.
</description>
```

Use \$PROVIDER's field syntax, looking it up if necessary from the documentation. Assume standard OpenTelemetry semantic conventions.

separate <reasoning> and <expression> in your results.

Using this prompt with Claude Opus 4.6, for instance, and filling in the blanks with pointers to the provider-specific query language details from your chosen observability tooling, you can produce a reasonable draft of how you can measure the SLI/SLO, shown as follows:

Claude Opus 4.6:

```
<reasoning>
The SLO description is "Checkout should succeed within 4 seconds, 99.99% of the time." The 99.99% target and the rolling time period are configured on the SLO object itself; the SLI expression only needs to classify each event as true (good), false (bad), or null (not applicable).
```

There are three components to express:

1. **Qualifier** (scopes to checkout events only): Events where ``http.route`` matches the checkout endpoint. Non-matching events return null and are excluded from the SLO calculation. Adapt the route string to match your actual instrumentation.
2. **Success criterion**: The HTTP response status code is below 400 (i.e., no server-side or client-error failure). Using the stable OTel semantic convention field ``http.response.status_code``. If you're on older conventions, substitute ``http.status_code``.
3. **Latency criterion**: The span duration is at or below 4000ms. ``duration_ms`` is Honeycomb's standard span duration field.

The ``IF()`` wrapper ensures non-checkout events return null. The ``AND()`` inside requires both success and latency criteria to be met for the event to count as "good."

Configure the SLO with ``target_percentage: 99.9`` and an appropriate ``time_period`` (e.g., 30 days). Adjust ``"/api/checkout"`` to match your actual route, and if your checkout is a multi-span workflow rather than a single HTTP handler, scope the qualifier to the root span of the checkout trace instead (e.g., using ``CONTAINS($name, "checkout")`` combined with ``NOT(EXISTS($trace.parent_id))``).

```

</reasoning>

<expression>
IF(
  EQUALS($http.route, "/api/checkout"),
  AND(
    LT($http.response.status_code, 400),
    LTE($duration_ms, 4000)
  )
)
</expression>

```

At this point, an open-weights model like gpt-oss-20b being run inside a tool like Ollama will begin to produce inferior results as it has not necessarily been trained on your tooling, and it lacks the ability to directly search documentation and ingest it into context. Instead, you'll need to paste in the documentation or the underlying language grammar. Surprisingly, both models omit in their initial response the criterion for `NOT(EXISTS($trace.parent_id))` or some other filter to only server spans (not internal spans) that we'd expect for successful SLO representation in our tool, given its grammar. Opus does suggest this as an optional refinement, however.

The generative AI space moves fast, as we've noted several times before. It's perilous to commit firm recommendations about AI in a textbook. However, at the time of this publishing, we've found that high-capability hosted models in thinking mode have a higher success rate for structured authoring tasks like this than local models, though local models can still be useful for iteration and exploration. We're certain performance in these areas will only continue to improve, as they already have between the Claude 4.0 and 4.6 models.

Once upon a time, you would need to use the Google SLO workshop materials or read the entire SLO book by Hidalgo (mentioned earlier) to learn how to do this. Now, you can get 90% good enough by using an LLM with appropriate prompting, and iterate from there. It can be toilsome to help hundreds of teams come up with SLOs and keep them up to date. We can better scale ourselves by writing best practices and embedding them into prompts rather than becoming a bottleneck of expertise.

At the end of the day, it's better to have more people using imperfect SLOs than to have SLOs remain a tool of the elite SRE-enabled teams. And if your observability tooling allows you to edit SLOs with retroactive effect, the consequences of a mistake or oversight are relatively minor. It is much easier to iterate and adjust your SLOs once you have practical feedback on what's working versus what is not.

The right operating model for getting started is to treat the LLM as a draft generator, not an authority. Validate outputs against real telemetry. Run them on production-like data. Use your expertise to review them the way you would review a pull request.

Encoding Best Practices in Your Prompts

Similar to guidelines that we've provided previously when working with AI (see [“Instrumentation with Agents” on page 128](#)), ad hoc prompting can be a risky practice. The recommended approach is to capture your standards in version control and reuse them.

Let's look at an example by refining the SLI in our previous conversation so that it better reflects the critical user journeys we are interested in and excludes automated health checks:

```
Prompt = Please generate a revised service level indicator specification in
          $PROVIDER format that excludes user agents starting with `kube-probe`,
          and also filters specifically for POST requests to the `/checkout` HTTP
          route.
```

We'll continue showing responses from Claude, but this would work with gpt-oss or various other models:

```
Claude Opus 4.6:
<reasoning>
Three changes from the previous expression:
1. HTTP method qualifier: The OTel stable semantic convention for request
   method is `http.request.method`. We add `EQUALS($http.request.method, "POST")`
   to the qualifier.
2. Route narrowed: Changed from `/api/checkout` to `/checkout` per your
   specification.
3. Exclude health probes: Kubernetes liveness/readiness probes should not
   count towards (or against) the SLO. Using `NOT(STARTS_WITH(
   $user_agent.original, "kube-probe"))` ensures these synthetic requests
   return null and are excluded from the budget entirely. The field
   `user_agent.original` is the stable OTel semantic convention for the raw
   User-Agent header.
```

```
All three qualifier conditions are wrapped in an `AND()`` inside the first
argument to `IF()`, so any event that fails any of them returns null
(unqualified). Only events matching all three are evaluated against the
success criteria (status < 400 and duration ≤ 4000ms).
```

```
</reasoning>

<expression>
IF(
  AND(
    EQUALS($http.request.method, "POST"),
    EQUALS($http.route, "/checkout"),
    NOT(STARTS_WITH($user_agent.original, "kube-probe"))
  ),
  AND(
    LT($http.response.status_code, 400),
    LTE($duration_ms, 4000)
  )
)
```

```
)  
</expression>
```

That result nails it by accurately describing the SLI specification for our example use case. It has correct syntax and covers all of the conditions requested (excluding kube-probe health checks, and forcing HTTP method to be POST and route to be / checkout).

Going further, a strong prompt establishes what fields exist in an SLO definition, what good titles and descriptions look like, how event-based SLIs should evaluate eligibility versus success, and how to produce an improved (but functionally equivalent) version of an SLO definition in JSON. It also asks for concise feedback and a clear explanation of changes.

For example, your prompt may look like this:

```
Prompt = You are an AI that takes a user's service level objective (SLO),  
reviewing it based on best practices, and provides suggestions for improvement.
```

```
The SLO has exactly six things: name, description, time period in days, rate, SLI  
alias, and SLI expression that gets computed on each event.
```

```
When grading an SLO, you think step by step through each component and provide  
concise feedback on how it could align better with best practices.
```

```
An SLO aims to measure the reliability of a product, feature, or service and  
provide a realistic target to manage the system according to.
```

```
Good titles clearly describe the critical user journey and its success criteria  
using affirmative language.
```

```
Good descriptions also use affirmative language to elaborate on what success  
means, and also elaborate with specific eligibility criteria for events and the  
evaluation criteria to compute for them. The evaluation criteria often has a  
numeric threshold, or checks a boolean value like success or error.
```

```
SLIs typically have an eligibility criterion for which events are in scope,  
and a success criterion representing whether each in-scope event delivered a  
satisfactory user experience. If the eligibility criterion is not met, the  
derived column should evaluate to NULL. Otherwise, it should evaluate to TRUE  
on a good user experience and FALSE on a failed user experience.
```

```
Good SLIs typically have as simple of an expression as possible.
```

```
Finally, when you finish reviewing an SLO, you provide a JSON object of a  
modified SLO definition that aligns better with best practices, and then an  
explanation of why it's better.
```

```
When you suggest an improved SLI, make sure it's always functionally equivalent.
```

Once you find the best prompts that work for your organizational context and your tooling, you should check them into version control, keep them simple, and make them consistent across the team. Instead of requiring every team to “borrow” SRE expertise to get started, you embed that expertise into a repeatable workflow that can be shared among teams, individuals, and organizations.

Sharing that expertise can be a key accelerator. It’s better to help more teams adopt imperfect but improving SLOs than to keep SLOs confined to the teams already fluent in them. The key to improving SLOs is ensuring they are reviewed and refined, which is simple when SLO definitions can be edited retroactively.

In summary, you can supercharge your SRE practices and roll out SLIs and SLOs faster with the combination of good observability tooling and a small amount of LLM assistance. Over time, your teams may take deeper ownership and refine definitions by hand; either using the model as a reviewer, a drafter, or simply as a rubber duck to understand the trade-offs.

Conclusion

In this chapter, we introduced SLOs as a more effective alerting strategy than traditional threshold monitoring. Alert fatigue is so prevalent in our industry, largely because traditional monitoring relies on potential-cause signals that are easy to measure and trigger often, yet are often very weakly correlated with real user impact.

Alert fatigue can be solved by focusing on creating only helpful alerts that meet two criteria. First, they must trigger as reliable indicators that the user experience of your service is in a degraded state. Second, they must be actionable. Any alert that does not meet that criteria is no longer serving its purpose and should be deleted.

SLOs help because they decouple the “what” from the “why.” They alert on symptoms that map directly to customer experience rather than on presumed causes. When SLOs are measured with event-based SLIs, alerts tend to be more reliable and less disruptive, with fewer false positives and false negatives—especially when distinguishing systemic problems from sporadic failures.

However, that’s only half the solution. SLO alerts can tell you *that* user pain exists, not why it’s happening. To make SLO-based alerts operationally effective, production systems must be debuggable, which is where having a high degree of observability becomes essential.

Adopting SLO-based alerting is a sociotechnical challenge. Removing unactionable alerts requires shared standards, confidence in the new signals, and a deliberate shift away from long-held traditions. Mat Vine’s firsthand account illustrates getting buy-in and operationalizing these practices.

Generative AI can help accelerate SLO adoption. Drafting SLIs and SLOs is often where teams stall—not because the mechanics are hard, but because the work requires clarity and alignment. Used appropriately, LLMs can reduce time to first draft, giving teams something concrete to review, validate against real telemetry, and refine over time.

In the next part of this book, we dive deep into technical implementation details to make these concepts more concrete. In the next chapter, we'll begin by looking at the inner workings of SLO burn budgets and examine how they're used to create effective alerts.

Observability Technical Deep Dives

The earlier parts of this book covered fundamentals for anyone working in observability. This part goes deeper.

We're peeling back the covers here—internals, implementation details, trade-offs. This section is for advanced practitioners, teams thinking about building their own observability tooling, and teams operating at serious scale. If that's not where you are yet, feel free to skip ahead and come back.

Chapter 12 digs into how SLO calculations actually work under the hood. Building on **Chapter 11**, it shows why wide events produce more accurate, actionable, and debuggable alerts than metrics-based approaches.

Chapter 13 examines what a datastore needs to do to support real investigative observability workloads: the functional requirements, the performance trade-offs, and how to optimize for the way engineers actually query telemetry. It includes a case study of the Honeycomb custom-built Retriever engine as a concrete example.

Chapter 14 offers a counterpoint. Written by the engineering team at ClickHouse, this guest-contributed chapter shows how the open source ClickHouse database can be tuned to meet observability workload requirements. A different set of trade-offs is also presented, and worth understanding.

Chapter 15 covers sampling—how to handle large telemetry volumes without drowning in data or losing the signal you need. It includes code samples showing how common sampling techniques work in practice.

Chapter 16 is a guest-contributed chapter by Mike Kelly, CEO and founder of Bindplane, on telemetry pipelines—what they are, what organizational problems they solve, and how to use them. It is written through the lens of Bindplane, an open source pipeline manager built on the OTel standard.

Chapter 17 looks at managing telemetry at scale across services, environments, and teams. When your data is diverse and distributed, ontologies can help prevent semantic failures and make telemetry more accessible, especially when you're working with large language models. This guest chapter is contributed by Frank Chen, Cofounder and CTO at Balto Energy.

In **Part V**, we zoom back out and look at how these techniques apply across different domains beyond the backend systems we've focused on so far.

Acting On and Debugging SLO-Based Alerts

In the preceding chapter, we introduced SLOs and an SLO-based approach to monitoring for more effective alerting. This chapter closely examines how observability data is used to make those alerts both actionable and debuggable. SLOs that use threshold-based monitoring data—or metrics—create alerts that are not actionable because they don’t provide guidance on fixing the underlying issue. However, using wide, structured event data for SLOs makes them both more precise and more debuggable.

Regardless of the degree of observability in your systems, using SLOs to drive alerting can be a productive way to make alerting less noisy and more actionable. SLIs can be defined to measure customer experience of a service in ways that directly align with business objectives. Error budgets set clear expectations between business stakeholders and engineering teams. Error budget *burn alerts* enable teams to ensure a high degree of customer satisfaction, align with business goals, and initiate an appropriate response to production issues—without the type of cacophony common in symptom-based alerting, where an excessive alert storm is the norm.

In this chapter, we dive deep into SLOs to examine error budgets and the mechanisms available to trigger SLO-based alerts. We’ll break down what an SLO error budget is and how it works, the forecasting calculations available for predicting SLO error budget exhaustion, and why it’s *necessary* to use wide, structured events rather than time-based metrics to make reliable calculations.

Alerting Before Your Error Budget Is Empty

An *error budget* represents the maximum amount of system unavailability that your business is willing to tolerate. If your SLO is to ensure that 99.9% of requests are successful, a time-based calculation would state that your system could be unavailable

for no more than 8 hours, 45 minutes, and 57 seconds in one standard year (or 43 minutes and 50 seconds per month). As shown in the previous chapter, an event-based calculation considers each individual event against qualification criteria and keeps a running tally of “good” events versus “bad” (or errored) events.

Because availability targets are represented as percentages, the error budget corresponding to an SLO is based on the number of requests that came in during that time period. For any given period of time, only so many errors can be tolerated. A system is out of compliance with its SLO when its entire error budget has been spent. Subtract the number of failed (*burned*) requests from your total calculated error budget, and that is known colloquially as the *amount of error budget remaining*.

To proactively manage SLO compliance, you need to become aware of and resolve application and system issues long before your entire error budget is burned. Time is of the essence. To take corrective action that averts the burn, you need to know whether you are on a trajectory to consume that entire budget well before it happens. The higher your SLO target, the less time you have to react. **Figure 12-1** shows an example graph indicating error budget burn.

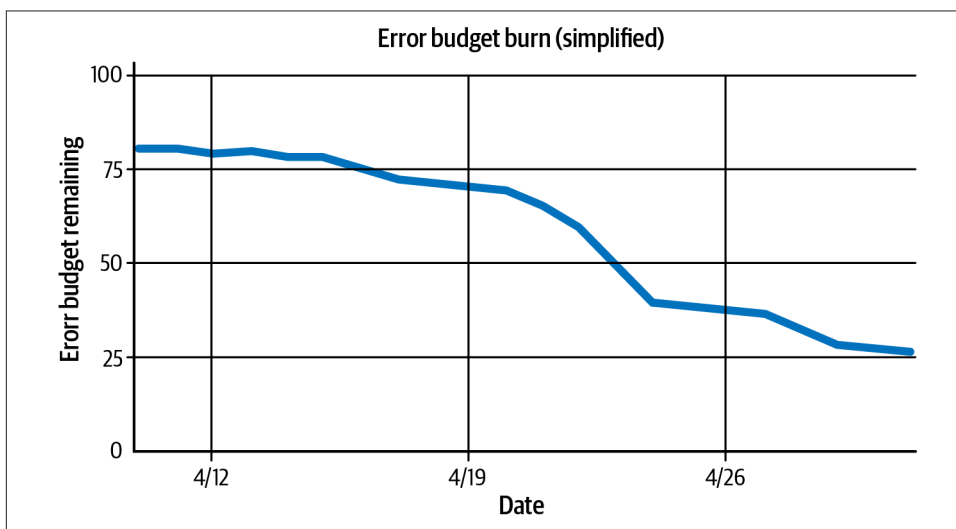


Figure 12-1. A simple graph showing error budget burn over approximately three weeks.

For SLO targets up to about 99.95%—indicating that up to 21 minutes and 54 seconds of full downtime are tolerable per month—a reasonable enough time period exists for your team to be alerted to potential issues and still have enough time to act proactively before the entire error budget is burned.

Error budget burn alerts are designed to provide early warning about future SLO violations that would occur if the current burn rate continues. Therefore, effective burn alerts must forecast the amount of error budget that your system will have

burned at some future point in time (anywhere from several minutes to days from now). Various methods are commonly used to make that calculation and decide when to trigger alerts on error budget burn rate.

Before proceeding, it's worth noting that these types of preemptive calculations work best to prevent violations for SLOs with targets up to 99.95% (as in the previous example). For SLOs with targets exceeding 99.95%, these calculations work less preventively but can still be used to report on and warn about system degradation.



For more information on mitigations for this situation, see [Chapter 5 of *The Site Reliability Workbook*](#), edited by Betsy Beyer et al. (O'Reilly, 2018). We recommend reading the whole chapter to those who have a deeper interest in alternative perspectives on this topic.

The rest of this chapter closely examines and contrasts various approaches to making effective calculations to trigger error budget burn alerts. Let's examine what it takes to get an error budget burn alert working. First, you must start by setting a frame for considering the all-too-relative dimension of time.

Framing Time as a Sliding Window

The first choice is whether to analyze your SLO across a fixed window or a sliding window. An example of a *fixed window* is one that follows the calendar—starting on the 1st day of the month and ending on the 30th. A counterexample is a *sliding window* that looks at any trailing 30-day period (see [Figure 12-2](#)).

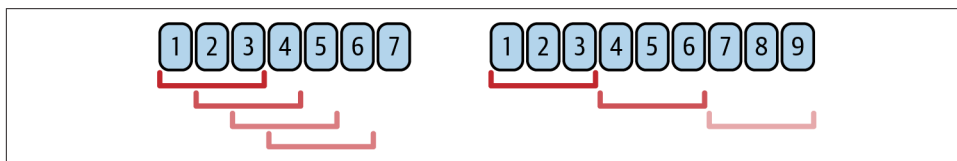


Figure 12-2. A rolling three-day window (left) and a three-day resetting window (right).

For most SLOs, a 30-day window is the most pragmatic period to use. Shorter windows, like 7 or 14 days, won't align with customer memories of your reliability or with product-planning cycles. A window of 90 days tends to be too long; you could burn 90% of your budget in a single day and still technically fulfill your SLO even if your customers don't agree. Long periods also mean that incidents won't roll off quickly enough.

You might choose a fixed window to start with, but in practice, fixed window availability targets don't match the expectations of your customers. You might issue a customer a refund for a particular bad outage that happens on the 31st of the

month, but that does not wave a magic wand that suddenly makes them tolerant of a subsequent outage on the 2nd day of the next month—even if that second outage is legally within a different window.

As covered in **Chapter 11**, SLOs should be used as a way to measure customer experience and satisfaction, not legal constraints. The better SLO is one that accounts for human emotions, memories, and recency bias. Human emotions don't magically reset at the end of a calendar month.

If you use a fixed window to set error budgets, those budgets reset with dramatic results. Any error burns away a sliver of the budget, with a cumulative effect gradually counting down toward zero. That gradual degradation continually chips away at the amount of time your team has to proactively respond to issues. Then, suddenly, on the first day of a new month, everything is reset, and you start the cycle again.

In contrast, using a sliding window to track the amount of error budget you've burned over a trailing period offers a smoother experience that more closely resembles human behavior. At every interval, the error budget can be burned a little or restored a little. A constant and low level of burn never completely exhausts the error budget, and even a medium-sized drop will gradually be paid off after a sufficiently stable period of time.

The correct first choice to make when calculating burn trajectories is to frame time as a sliding window, rather than a static fixed window. Otherwise, there isn't enough data after a window reset to make meaningful decisions.

Forecasting to Create a Predictive Burn Alert

With a timeframe selected, you can now set up a trigger to alert you about error budget conditions that you care about. The easiest alert to set is a *zero-level alert*—one that triggers when your entire error budget is exhausted.

What Happens When You Exhaust Your Error Budget

When you transition from having a positive error budget to a negative one, a necessary practice is to shift away from prioritizing work on new features and toward work on service stability. It's beyond the scope of this chapter to cover exactly what happens after your error budget is exhausted. But your team's goal should be to prevent the error budget from being entirely spent.

Error-budget over expenditures translate into stringent actions such as long periods of feature freezes in production. The SLO model creates incentives to minimize actions that jeopardize service stability once the budget is exhausted. For an in-depth analysis of how engineering practices should change in these situations and how

to set up appropriate policies, we recommend reading *Implementing Service Level Objectives* by Alex Hidalgo, as mentioned in [Chapter 11](#).

A trickier alert to configure is one that is preemptive. If you can foresee the emptying of your error budget, you have an opportunity to take action and introduce fixes that prevent it from happening. By fixing the most egregious sources of error sooner, you can forestall decisions to drop any new feature work in favor of reliability improvements. Planning and forecasting are better methods than heroism when it comes to sustainably ensuring team morale and stability.

Therefore, a solution is to track your error budget burn rate and watch for drastic changes that threaten to consume the entire budget. In this section, we'll discuss three models that are widely deployed in software operations to trigger burn alerts above the zero-level mark.

Threshold-Crossing Alerts

The first is to pick a nonzero threshold on which to alert. For example, you could alert when your remaining error budget dips below 30%, as shown in [Figure 12-3](#).

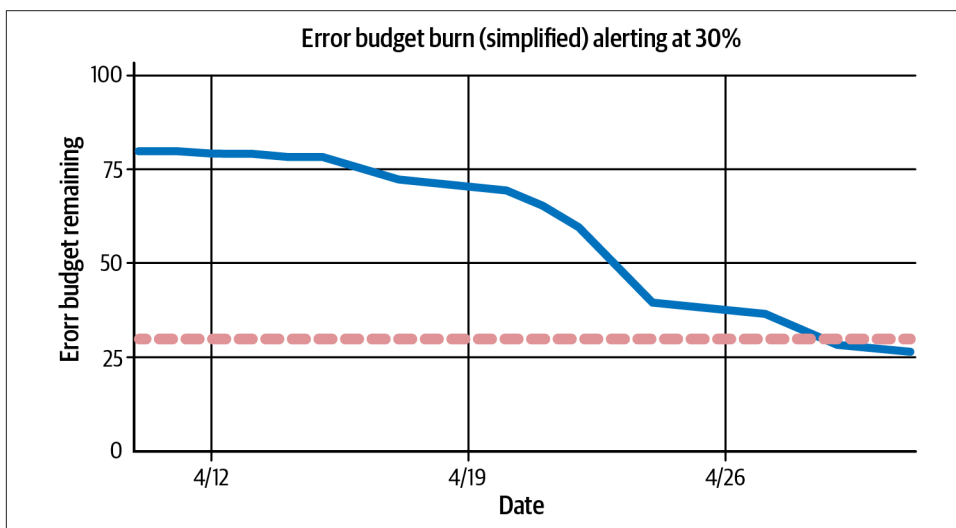


Figure 12-3. In this model, an alert triggers when the remaining error budget (solid line) dips below the selected threshold (dashed line).

A challenge with this model is that it effectively just moves the goalpost by setting a different empty threshold. This type of “early warning” system can be somewhat effective, but it is crude. In practice, after crossing the threshold, your team will act as if the entire error budget has been spent. This model optimizes to ensure a slight

bit of headroom so that your team meets its objectives. But that comes at the cost of forfeiting additional time that you could have spent delivering new features. Instead, your team sits in a feature freeze while waiting for the remaining error budget to climb back up above the arbitrary threshold. If fixed thresholds are insufficiently flexible, what mechanisms might work for assessing risk?

Relative Burn Alerts

A mechanism outlined in [Chapter 5 of *The Site Reliability Workbook*](#) is to calculate the ratio of observed and allowed errors according to the error budget during a certain baseline (or lookback) window. For instance, you might want to look at the past 15-minute window and determine the number of failing requests during that window. And then alert if the number of failing requests exceeds the number of failing requests proportionally allowed by the error budget for that window, as shown in [Figure 12-4](#).

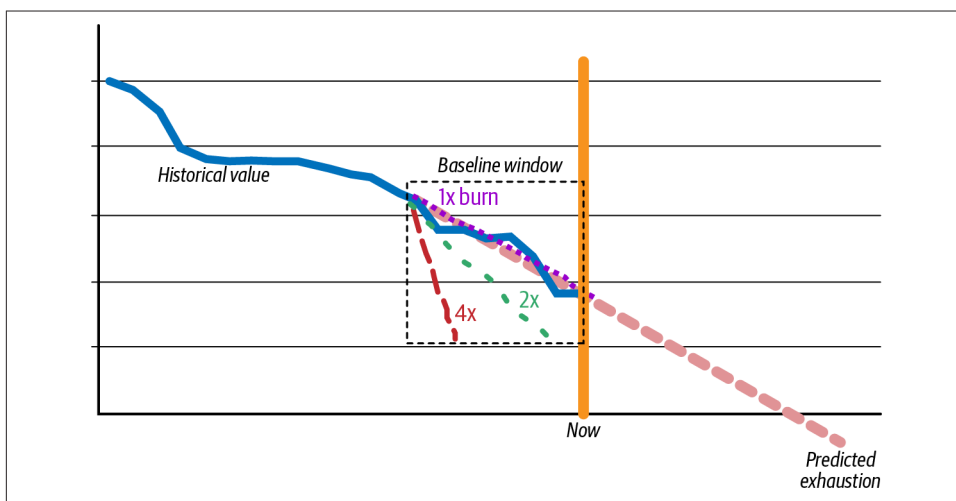


Figure 12-4. Different slopes of burn during a baseline window that you may want to alert on, (e.g., 1x burn is normal, 2x for a sustained period might trigger a non-paging alert, and 4x or more in the baseline window would cause a page).

Let's work some examples. If your service receives 10 million requests per month, and aims to have a target SLO of 99.9%, then 10,000 requests can fail each month. There are 2,880 15-minute windows per month, so approximately 3.4 requests can be safely failed at a steady state per 15 minutes, assuming all windows receive equal traffic levels.

To use a relative burn alert of 4x in this case, you would want to send an alert if more than 13 requests failed in any trailing 15-minute window, *regardless of how many requests succeeded in that window*. A 20x relative burn alert would be triggered if more than 68 requests failed in any trailing 15-minute window.

The key factor that differentiates this from static threshold alerting is that it's still aware of the total expected request rate over the entire SLO time range, rather than simply alerting when the error rate for any given lookback window exceeds 0.1%, 0.3%, or 2% respectively, for 1x, 3x, and 20x ratios. At a time of low traffic, 1 request failing out of 20 total received shouldn't cause your alert to fire spuriously.

Predictive Burn Alerts

A third model for triggering alerts above the zero-level mark is to create *predictive* burn alerts (see [Figure 12-5](#)). These forecast whether current conditions will result in burning your entire error budget.

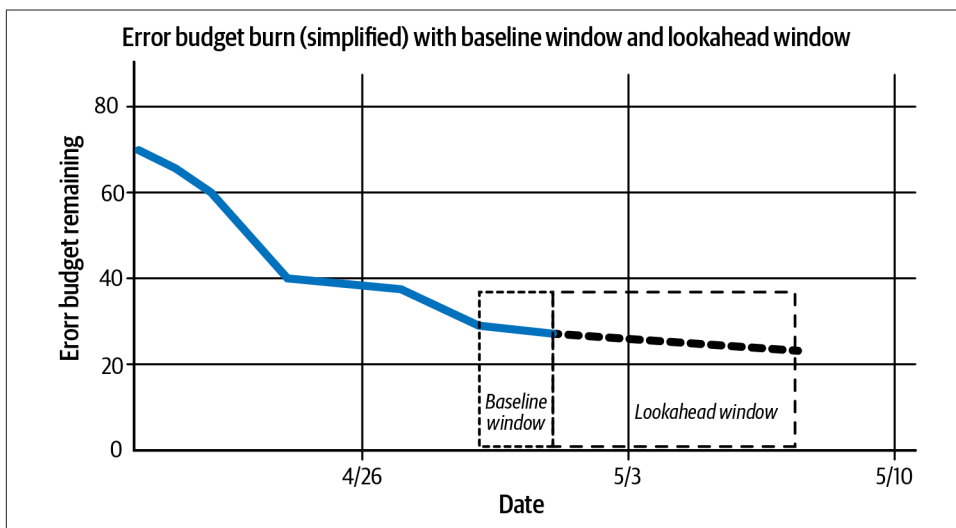


Figure 12-5. For predictive burn alerts, you need a baseline window of recent data to use in your model and a lookahead window that determines how far into the future your forecast extends.

When using predictive burn alerts, you need to consider the *lookahead window* and the *baseline* (or *lookback*) *window*: how far into the future are you modeling your forecast, and how much recent data should you be using to make that prediction? Let's start by considering the lookahead window, which is simpler to consider.

The Lookahead Window

In the predictive model, we can see that not every error that affects your error budget requires waking someone up. For example, let's say you have introduced a performance regression such that your service, with a 99.9% SLO target, is now on track to instead deliver 99.88% reliability one month from now. That is not an emergency situation requiring immediate attention. You could wait for the next business day for someone to investigate and correct the trajectory back above 99.9% reliability overall for the full month.

Conversely, if your service experiences failures of a significant fraction of its requests, such that you are on track to reach 98% within one hour, that should require paging the on-call engineer. If left uncorrected, such an outage could hemorrhage your error budget for the entire month, quarter, or year within a matter of hours.

Both of these examples illustrate that it's important to know whether your error budget will become exhausted at some point in the future, based on current trends. In the first example, that happens in days, and in the second, that happens in minutes or hours. But what exactly does “based on current trends” mean in this context?

The scope for current trends depends on how far into the future we want to forecast. On a macroscopic scale, most production traffic patterns exhibit both cyclical behavior and smoothed changes in their utilization curves. Patterns can occur in cycles of either minutes or hours. For example, a periodic cron job that runs a fixed number of transactions through your application without jitter can influence traffic in predictable ways that repeat every few minutes. Cyclical patterns can also occur by day, week, or year. For example, the retail industry experiences seasonal shopping patterns with notable spikes around major holidays.

In practice, that cyclical behavior means that some baseline windows are more appropriate to use than others. The past 30 minutes or hour are somewhat representative of the next few hours. But small, minute-by-minute variations can be smoothed out when zooming out to consider daily views. Attempting to use the microscale of past performance for the last 30 minutes or hour to extrapolate what could happen in the macroscale of performance for the next few days runs the risk of becoming flappy and bouncing in and out of alert conditions.

Similarly, the inverse situation is also dangerous. When a new error condition occurs, it would be impractical to wait for a full day of past performance data to become available before predicting what might happen in the next several minutes. Your error budget for the entire year could be blown by the time you make a prediction. Therefore, your baseline window should be about the same order of magnitude as your lookahead window.

In practice, we've found that a given baseline window can linearly predict forward by a factor of four at most without needing to add compensation for seasonality (e.g.,

peak/off-peak hours of a day; weekday versus weekend; end/beginning of the month). Therefore, you can get an accurate enough prediction of whether you'll exhaust your error budget in four hours by extrapolating the past one hour of observed performance four times over. That extrapolation mechanism is discussed in more detail in [“Context-aware burn alerts” on page 206](#).

Extrapolating the future from the current burn rate

Calculating what may happen in the lookahead window is a straightforward computation, at least for your first guess at burn rate. As you saw in the preceding section, the approach is to extrapolate future results based on the current burn rate: how long will it take before we exhaust our error budget?

Now that you know the approximate order of magnitude to use as a baseline, you can extrapolate results to predict the future. [Figure 12-6](#) illustrates how a trajectory can be calculated from your selected baseline to determine when your entire error budget would be completely exhausted.

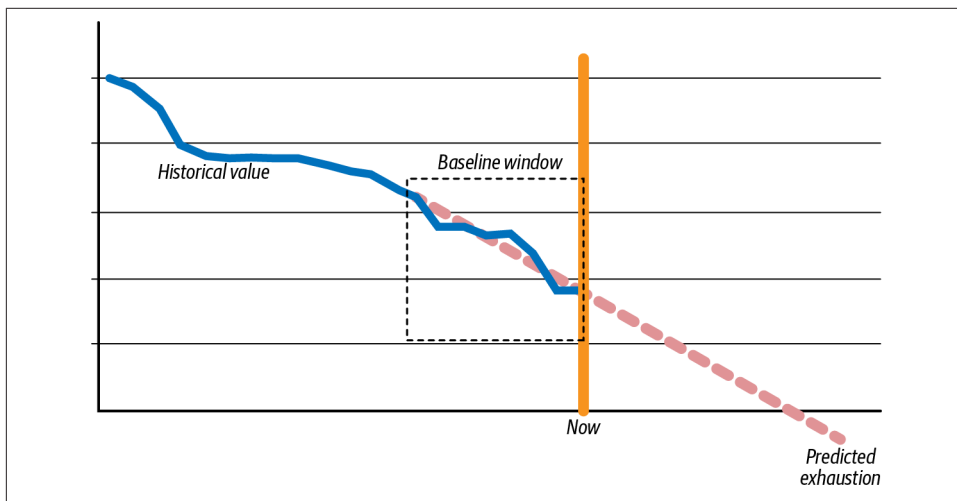


Figure 12-6. This graph shows the level of error budget (y-axis) remaining over time (x-axis). By extrapolating a baseline window, you can predict the moment when the error budget will empty.

This same technique of linear extrapolation often surfaces in areas such as capacity planning or project management. For example, if you use a weighted system to estimate task length in your ticketing system, you will have likely used this same approach of sprint burndowns to extrapolate when a feature might be delivered during your future sprint planning. With SLOs and error budget burn alerts, a similar logic is being applied to help prioritize production issues that require immediate attention.

Calculating a first guess is relatively straightforward. However, you must weigh additional nuances when forecasting predictive burn alerts that determine the quality and accuracy of those future predictions.

In practice, we can use two approaches to calculate the trajectory of predictive burn alerts. *Short-term burn alerts* extrapolate trajectories using only baseline data from the most recent time period and nothing else. *Context-aware burn alerts* take historical performance into account and use the total number of successful and failed events for the SLO's entire trailing window to make calculations.

The decision to use one method or the other typically hinges on two factors. The first is a trade-off between computational cost and sensitivity or specificity. Context-aware burn alerts are computationally more expensive than short-term burn alerts. However, the second factor is a philosophical stance on whether the total amount of error budget remaining should influence how responsive you are to service degradation. If resolving a significant error when only 10% of your burn budget remains carries more urgency than resolving a significant error when 90% of your burn budget remains, you may favor context-aware burn alerts.



In the next two sections, *unit* refers to the granular building block on which SLO burn alert calculations are made. These units can be composed of time-series data like metrics. Coarse measures like metrics have an aggregated granularity that marks a unit of time (like a minute or a second) as being either good or bad. These units can also be composed of event data, with each individual event corresponding to a user transaction that can be marked as either good or bad. In [“Using Structured Event Data for SLOs Versus Time-Series Data” on page 212](#), we’ll examine the ramifications of which type of data is used. For now, the examples will be agnostic to datatypes by using arbitrarily specific *units* that correspond to one datapoint per minute.

Let’s look at examples of how decisions are made when using each of those approaches.

Short-term burn alerts

When using *short-term*, or *ahistorical*, burn alerts, you record the actual number of failed SLI units along with the total number of SLI-eligible units observed over the baseline window. You then use that data to extrapolate forward, and compute the number of minutes, hours, or weeks it would take to exhaust the error budget. For this calculation, you assume that no errors occurred prior to the events observed in the baseline window, as well as the number of measurement units that the SLO typically sees.

Let's work on an example. Say you have a service with an SLO target indicating that 99% of units will succeed over a moving 30-day window. In a typical month, the service sees 43,800 units. In the past 24 hours, the service has seen 1,440 units, 50 of which failed. In the past six hours, the service has seen 360 units, 5 of which failed. To achieve the 99% target, only 1% of units are allowed to fail per month. Based on typical traffic volume (43,800 units), only 438 units are allowed to fail (your error budget). You want to know if, at this rate, you will burn your error budget in the next 24 hours.

A simple short-term burn alert calculates that in the past six hours, you burned only five units. Therefore, in the next 24 hours, you will burn another 20 units, totaling 25 units. You can reasonably project that you will not exhaust your error budget in 24 hours because 25 units is far less than your budget of 438 units.

A simple short-term burn alert calculation would also consider that in the past one day, you've burned 50 units. Therefore, in the next eight days, you will burn another 400 units, totaling 450 units. You can reasonably project that you will exhaust your error budget in eight days, because 450 units is more than your error budget of 438 units.

This simple math is used to illustrate projection mechanics. In practice, the situation is almost certainly more nuanced because your total amount of eligible traffic would typically fluctuate throughout the day, week, or month. In other words, you can't reliably estimate that one error over the past 6 hours forecasts four errors over the next 24 hours if that error happens overnight, over a weekend, or whenever your service is receiving one-tenth of its median traffic levels. If this error happened while you saw a tenth of the traffic, you might instead expect to see something closer to 30 errors over the next 24 hours, since that curve would smooth with rising traffic levels.

So far, in this example, we've been using linear extrapolation for simplicity. But proportional extrapolation is far more useful in production. Consider this next change to the preceding example.

Let's say you have a service with an SLO target indicating that 99% of units will succeed over a moving 30-day window. In a typical month, the service sees 43,800 units. In the past 24 hours, the service has seen 1,440 units, 50 of which failed. *In the past six hours, the service has seen 50 units, 25 of which failed.* To achieve the 99% target, only 1% of units are allowed to fail per month. Based on typical traffic volume (43,800 units), only 438 units are allowed to fail (your error budget). You want to know if, at this rate, you will burn your error budget in the next 24 hours.

Extrapolating linearly, you would calculate that 25 failures in the past 6 hours mean 100 failures in the next 24 hours, totaling 105 failures, which is far less than 438.

Using *proportional extrapolation*, you would calculate that in any given 24-hour period, you would expect to see 43,800 units per 30 days, or 1,440 units a day. In the last six hours, 25 of 50 units (50%) failed. If that proportional failure rate of 50% continues for the next 24 hours, you will burn 50% of 1,440 units, or 720 units—far more than your budget of 438 units. With this proportional calculation, you can reasonably project that your error budget will be exhausted in about half a day. An alert should trigger to notify an on-call engineer to investigate immediately.

Context-aware burn alerts

When using *context-aware*, or *historical*, burn alerts, you keep a rolling total of the number of good and bad events that have happened over the entire window of the SLO, rather than just the baseline window. This section unpacks the various calculations you need to make for effective burn alerts. But you should tread carefully when practicing these techniques. The computational expense of recalculating these values at each evaluation interval can quickly become financially expensive as well. At Honeycomb, we found this out the hard way when small SLO datasets suddenly started to rack up over \$5,000 of AWS Lambda costs per day when ordinary user-initiated querying against them cost only tens of dollars per day. Caching became a requisite strategy for our system to scale.

To see how the considerations are different for context-aware burn alerts, let's work on an example. Say you have a service with an SLO target indicating 99% of units will succeed over a moving 30-day window. In a typical month, the service sees 43,800 units. In the past 26 days, you have already failed 285 units out of 37,960. In the past 24 hours, the service has seen 1,460 units, 130 of which failed. To achieve the 99% target, only 1% of units are allowed to fail per month. Based on typical traffic volume (43,800 units), only 438 units are allowed to fail (your error budget). You want to know if, at this rate, you will burn your error budget in the next four days.

In this example, you want to project forward on a scale of days. Using the maximum practical extrapolation factor of four, as noted previously, you set a baseline window that examines the last one day's worth of data to extrapolate forward four days from now.

You must also consider the impact that your chosen scale has on your sliding window. If your SLO is a sliding 30-day window, your adjusted lookback window would be 26 days: 26 lookback days + 4 extrapolated days = your 30-day sliding window, as shown in [Figure 12-7](#).

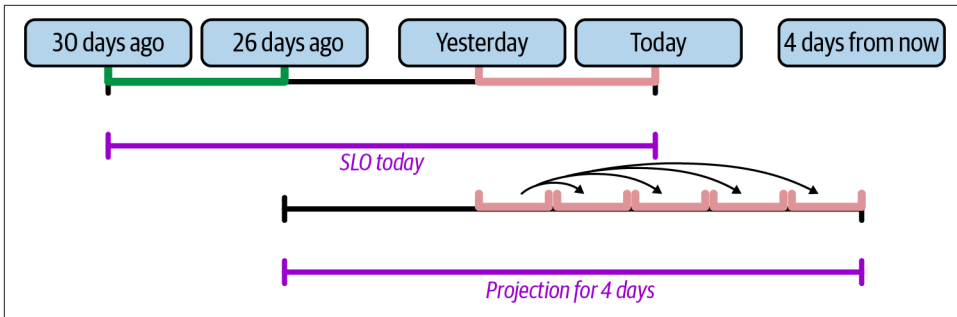


Figure 12-7. An adjusted SLO 30-day sliding window. When projecting forward four days, the lookback period to consider must be shortened to 26 days before today. The projection is made by replicating results from the baseline window for the next four days and adding those to the adjusted sliding window.

With those adjusted timeframes defined, you can now calculate how the future looks four days from now. For that calculation, you would do the following:

1. Examine every entry in the map of SLO events that has occurred in the past 26 days.
2. Store both the total number of events in the past 26 days and the total number of errors.
3. Reexamine map entries that occurred within that last one day to determine the baseline window failure rate.
4. Extrapolate the next four days of performance by presuming they will behave similarly to the one-day baseline window.
5. Calculate the adjusted SLO 30-day sliding window as it would appear four days from now.
6. Trigger an alert if the error budget would be exhausted by then.

Using a Go-based code example, you could accomplish this task if you had a given time-series map containing the number of failures and successes in each window:

```
func isBurnViolation(
    now time.Time, ba *BurnAlertConfig, slo *SLO, tm *timeseriesMap,
) bool {
    // For each entry in the map, if it's in our adjusted window, add its totals.
    // If it's in our projection window, store the rate.
    // Then project forward and do the SLO calculation.
    // Then fire off alerts as appropriate.

    // Compute the window we will use to do the projection, with the projection
    // offset as the earliest bound.
    pOffset := time.Duration(ba.ExhaustionMinutes/lookbackRatio) * time.Minute
```

```

pWindow := now.Add(-pOffset)

// Set the end of the total window to the beginning of the SLO time period,
// plus ExhaustionMinutes.
tWindow := now.AddDate(0, 0, -slo.TimePeriodDays).Add(
    time.Duration(ba.ExhaustionMinutes) * time.Minute)

var runningTotal, runningFails int64
var projectedTotal, projectedFails int64
for i := len(tm.Timestamps) - 1; i >= 0; i-- {
    t := tm.Timestamps[i]

    // We can stop scanning backward if we've run off the end.
    if t.Before(tWindow) {
        break
    }

    runningTotal += tm.Total[t]
    runningFails += tm.Fails[t]

    // If we're within the projection window, use this value to project forward,
    // counting it an extra lookbackRatio times.
    if t.After(pWindow) {
        projectedTotal += lookbackRatio * tm.Total[t]
        projectedFails += lookbackRatio * tm.Fails[t]
    }
}

projectedTotal = projectedTotal + runningTotal
projectedFails = projectedFails + runningFails

allowedFails := projectedTotal * int64(slo.BudgetPPM) / int64(1e6)
return projectedFails != 0 && projectedFails >= allowedFails
}

```

To calculate an answer to the example problem, you note that in the past 26 days, there have already been 37,960 units, and 285 failed. The baseline window of one day will be used to project forward four days. At the start of the baseline window, you are at day 25 of the dataset (one day ago).

Visualizing results as a graph, as shown in [Figure 12-8](#), you see that when the one-day baseline window started, you still had 65% of your error budget remaining. Right now, you have only 35% of your error budget remaining. In one day, you've burned 30% of your error budget. If you continue to burn your error budget at that rate, it will be exhausted in far less than four days. You should trigger an alarm.

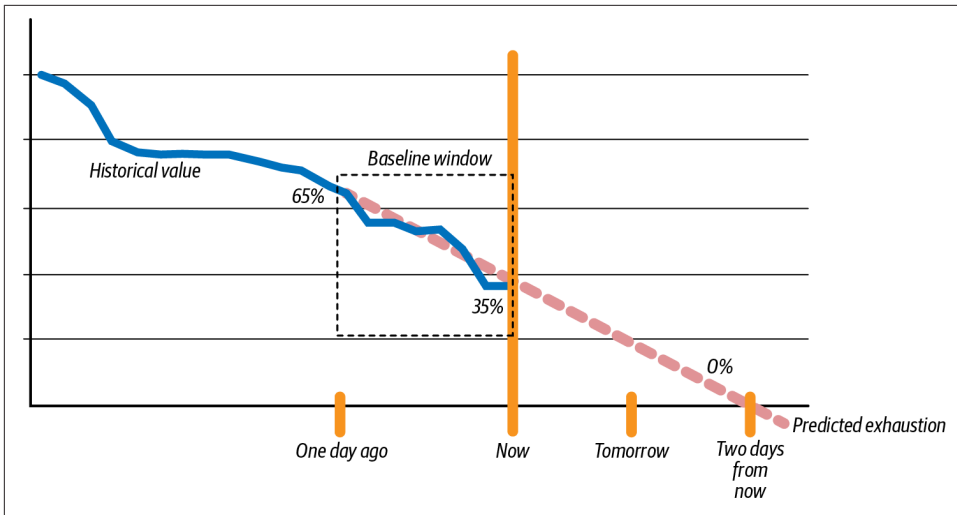


Figure 12-8. One day ago, 65% of the error budget remained. Now, only 35% of the error budget remains. At the current rate, the error budget will be exhausted in less than two days.

The Baseline Window

Having considered the lookahead window and the way it's used, let's now shift our attention back to the baseline window.

There's an interesting question of how *much* of a baseline to use when extrapolating forward. If the performance of a production service suddenly turns downhill, you want to be aware of that fairly quickly. At the same time, you also don't want to set such a small window that it creates noisy alerts. If you used a baseline of 15 minutes to extrapolate three days of performance, a small blip in errors could trigger a false alarm. Conversely, if you used a baseline of three days to extrapolate forward an hour, you'd be unlikely to notice a critical failure until it's too late.

We previously described a practical choice as a factor of four for the timeline being considered. You may find that other, more sophisticated algorithms can be used, but our experience has indicated that the factor of four is generally reliable. The baseline multiplier of four also gives us a handy heuristic: a 24-hour alarm is based on the last 6 hours of data; a 4-hour alarm is based on the last 1 hour of data, and so forth. It's a good place to start when first approaching SLO burn alerts.

An odd implication when setting baseline windows proportionally is also worth mentioning. Setting multiple burn alerts on an SLO by using different lookahead window sizes is common. Therefore, it becomes possible for alarms to trigger in seemingly illogical order. For example, a system issue might trigger a burn alert that says you'll exhaust your budget within two hours, but it won't trigger a burn alert that says you'll exhaust it within a day. That's because the extrapolation for those two alerts comes from different baselines. In other words, if you keep burning at the rate from the last 30 minutes, you'll exhaust in two hours—but if you keep burning at the rate from the last day, you'll be fine for the next four days.

For these reasons, you should measure both, and you should act whenever either projection triggers an alert. Otherwise, you risk potentially waiting hours to surface a problem on the burn-down timescale of one day—when a burn alert with hour-level granularity means you could find out sooner. Launching corrective action at that point could also avert triggering the one-day projection alert. Conversely, while an hourly window allows teams to respond to sudden changes in burn rate, it's far too noisy to facilitate task prioritization on a longer timescale, like sprint planning.

That last point is worth dwelling on for a moment. To understand why both time-scales should be considered, let's look at the actions that should be taken when an SLO burn alert is triggered.

Acting on SLO Burn Alerts

In general, when a burn alert is triggered, teams should initiate an investigative response. For a more in-depth look at various ways to respond, refer to Hidalgo's *Implementing Service Level Objectives*. For this section, we'll examine a few broader questions you should ask when considering response to a burn alert.

First, you should diagnose the type of burn alert you received. Is a new and unexpected type of burn happening? Or is it a more gradual and expected burn? Let's first look at various patterns for consuming error budgets.

Figures 12-9, 12-10, and 12-11 display *error budget remaining over time*, for the duration of the SLO window. Gradual error budget burns, which occur at a slow but steady pace, are a characteristic of most modern production systems. You expect a certain threshold of performance, and a small number of exceptions fall out of that. In some situations, disturbances during a particular period may cause exceptions to occur in bursts.

When a burn alert is triggered, you should assess whether it's part of a burst condition or is an incident that could burn a significant portion of your error budget all at once. Comparing the current situation to historical rates can add helpful context for triaging its importance.

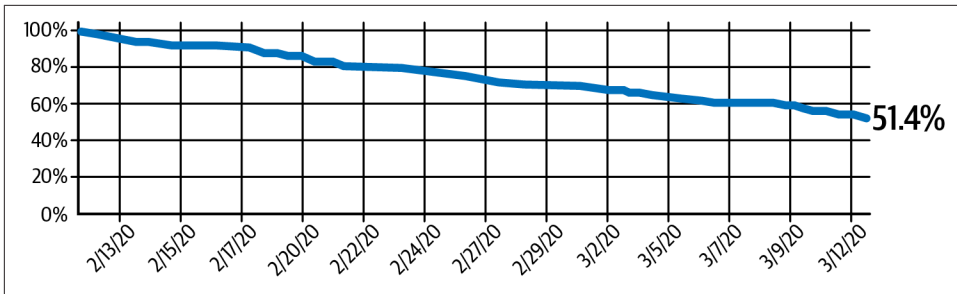


Figure 12-9. A gradual error budget burn rate, with 51.4% remaining.

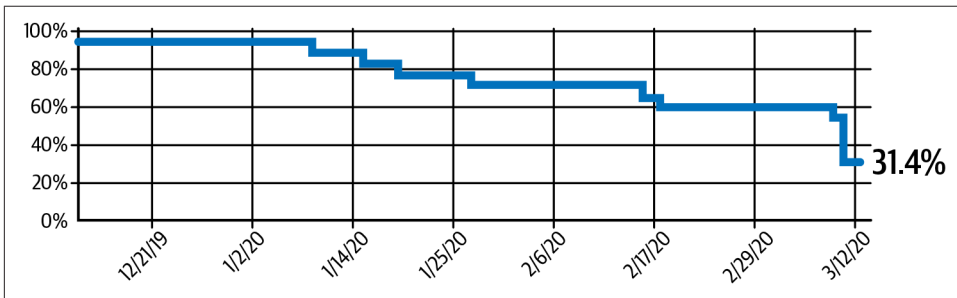


Figure 12-10. An error budget of 31.4% remaining, which is burned in bursts.

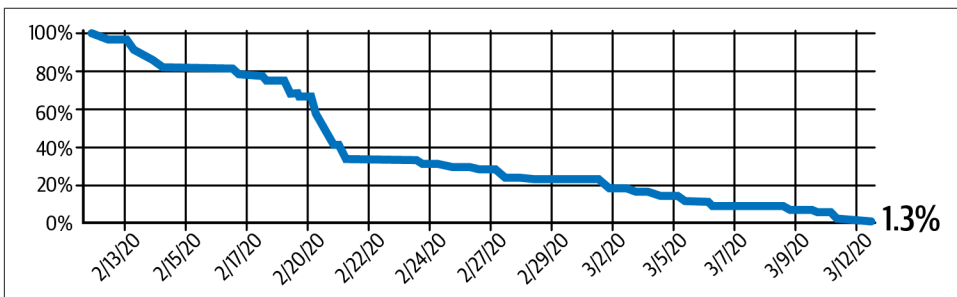


Figure 12-11. A system that usually burns slowly but had an incident that burned a significant portion of the error budget all at once, with only 1.3% remaining.

Instead of showing the instantaneous 31.4% remaining and how we got there over the trailing 30 days as we did in [Figure 12-10](#), [Figure 12-12](#) zooms out to examine the 30-day cumulative state of the SLO for each day in the past 90 days. Around the beginning of February, this SLO started to recover above its target threshold. This likely occurs, in part, because a large dip in performance aged out of the 90-day window.

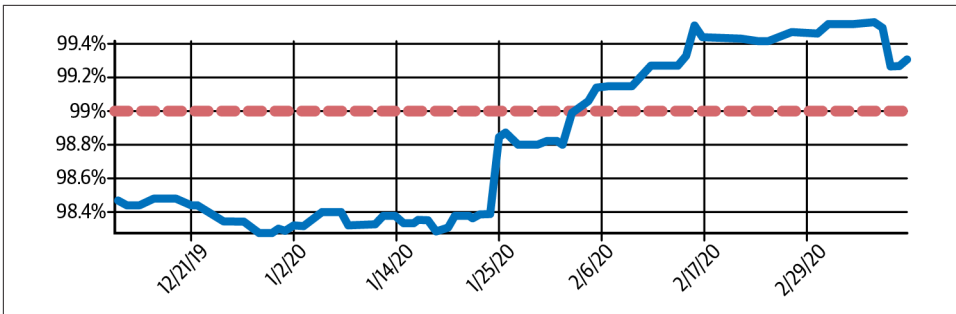


Figure 12-12. A 90-day sliding window for an SLO performs below a 99% target before recovering toward the start of February.

Understanding the general trend of the SLO can also answer questions about how urgent the incident feels—and can give a hint for solving it. Burning budget all at once suggests a different sort of failure than burning budget slowly over time.

Using Structured Event Data for SLOs Versus Time-Series Data

Using time-series data for SLOs introduces a few complications. In the examples in earlier sections, we used a generic unit to consider success or failure. In an event-based world, that unit is an *individual user request*: did this request succeed or fail? In a time-series world, that unit is a *particular slice of time*: did the entire system succeed or fail during this time? When using time-series data, an aggregate view of system performance gets measured.

The problem with time-series data becomes particularly pronounced in the case of *stringent SLOs*—those with 99.99% availability targets and higher. With these SLOs, error budgets can be exhausted within minutes or seconds. In that scenario, every second especially matters, and several mitigation steps (for example, automated remediations) must be enacted to achieve that target. The difference between finding out about critical error budget burn in one second versus one minute won't matter much if a responding human needs three minutes to acknowledge the alert, unlock their laptop, log in, and manually run the first remediation they conjure (or even more time if the issue is complex and requires further debugging).

Another mitigation can come by way of using event data for SLOs rather than time-series data. Consider the way time-series evaluation functions in the context of stringent SLOs. For example, let's say you have an SLI specifying that requests must have a P95 less than 300 milliseconds. The error budget calculation then needs to classify short time periods as either good or bad. When evaluating on a per-minute basis, the calculation must wait until the end of the elapsed minute to declare it either

good or bad. If the minute is declared bad, you've already lost that minute's worth of response time. This is a nonstarter for initiating a fast-alerting workflow when error budgets can be depleted within seconds or minutes.

The granularity of “good minute” or “bad minute” is simply not enough to measure a four-nines SLO. In the preceding example, if only 94% of requests were less than 300 milliseconds, the entire minute is declared bad. That one evaluation is sufficient enough to burn 25% of your 4.38-minute monthly error budget in one shot. And if your monitoring data is collected in only five-minute windows (as is the fee-free default in Amazon CloudWatch), using such five-minute windows for good/bad evaluation can fail only one single evaluation window before you are in breach of your SLO target.

In systems that use time-series data for SLO calculation, a mitigation approach might look something like a system set up so that when three synthetic probes fail, automatic remediations are deployed and humans are then expected to double-check the results afterward. Another approach can be using metrics to measure the instantaneous percentage of requests that are failing in a shorter window of time, given a sufficiently high volume of traffic.

Rather than making aggregate decisions in the good-minute/bad-minute scenario, using event data to calculate SLOs gives you request-level granularity to evaluate system health. Using the same previous SLI scenario, any single request with a duration lower than 300 milliseconds is considered good, and any with a duration greater than 300 milliseconds is considered bad. In a scenario with 94% of requests that were less than 300 milliseconds, only 6% of those requests are subtracted from your SLO error budget, rather than the 100% that would be subtracted using an aggregate time-series measure like P95.

In modern distributed systems, 100% total failure blackouts are less common than partial failure brownouts, so event-based calculations are much more useful. Consider that a 1% brownout of a 99.99% reliability system is similar to measuring a 100% outage of a 99% reliability system. Measuring partial outages buys you more time to respond, as if it were a full outage of a system with a much less stringent SLO. In both cases, you have more than seven hours to remediate before the monthly error budget is exhausted.

Event data that traces actual user experience with your services is a more accurate representation of system state than coarsely aggregated time-series data. While you can always optimize and precompute SLO data from your event data, the inverse transformation is not possible starting from pre-aggregated time-series data, in case you change your SLI criteria. Backfilling changes to your SLIs and SLOs from the raw data enables you to confidently iterate on them rather than having to wait months to see if your change to SLIs and SLOs had the desired effect. When deciding which data to use for actionable alerting, using observability data enables teams to focus on

conditions that are much closer to the truth of what the business cares about: the overall customer experience that is occurring at this moment.

It's important to align your reliability goals with what's feasible for your team. For very stringent SLOs—such as a five-nines system—you must deploy every mitigation strategy available, such as automatic remediation, based on extremely granular and accurate measurements that bypass traditional human-involved fixes requiring minutes or hours. However, even for teams with less stringent SLOs, using granular event-based measures can create more response time buffers than are typically available with time-series-based measures. That additional buffer ensures that reliability goals are much more feasible, regardless of your team's SLO targets.

Conclusion

We've examined the role that error budgets play and the mechanisms available to trigger alerts when using SLOs. Several forecasting methods are available that can be used to predict when your error budget will be burned. Each method has its own considerations and trade-offs, and the hope is that this chapter shows you which method to use to best meet the needs of your specific organization.

SLOs are a modern form of monitoring that solve many of the problems associated with noisy monitoring. SLOs are not specific to observability. What is specific to observability is the additional power that event data adds to the SLO model. When calculating error budget burn rates, events provide a more accurate assessment of the actual state of production services. Additionally, merely knowing that an SLO is in danger of a breach does not necessarily provide the insight you need to determine which users are impacted, which dependent services are affected, or which combinations of user behavior are triggering errors in your service. Coupling event data to SLOs helps you see where and when failures happened after a burn budget alert is triggered.

Using SLOs with wide, structured events is an important component of both the SRE approach and the observability-driven-development approach. As seen in previous chapters, analyzing events that fail can give rich and detailed information about what is going wrong and why. It can help differentiate systemic problems and occasional sporadic failures.

Shifting to using wide, structured events instead of metrics presents a number of challenges. For example, you wouldn't use a time-series database if you're working in a unified observability model and not solely relying on metrics. In the next chapter, we'll examine the functional requirements for building a datastore that meets the performance needs of observability workloads, and then look at two example implementations.

Efficient Data Storage with Retriever

In this chapter, we'll look at the challenges that must be addressed to effectively store and retrieve your observability data when you need it most. Speed is a common concern with data storage and retrieval, but other functional constraints impose key challenges that must be addressed at the data layer. At scale, the challenges inherent to observability become especially pronounced.

We will lay out the functional requirements necessary to enable observability workflows. Then we will examine real-life trade-offs and possible solutions in columnar data storage engines by practical example, using the implementation of Honeycomb's proprietary Retriever datastore. In the next chapter, you'll also learn about an alternative application of these concepts using ClickHouse, the widely deployed open source columnar storage engine.

Across both chapters, you will learn about the various considerations required at the storage and retrieval layers to ensure speed, scalability, and durability for your observability data. You will learn about how columnar datastores are architected, why they are particularly well-suited for observability data, how querying workloads must be handled, and considerations for making data storage durable and performant.

When we started working on modern observability systems, many believed storing high-cardinality event data at scale was infeasible without pre-aggregating it into metrics. Today, multiple proven event-based approaches exist. The solutions presented in these chapters are not the only possible solutions to the various trade-offs you may encounter. However, they're presented as real-world examples of achieving the necessary results when building an observability solution.

The Functional Requirements for Observability

When you're experiencing an outage in production, every second counts. Queries against your observability data must return results as quickly as possible. If you submit a query and can brew a fresh cup of coffee while you wait for results to return, you're fighting a losing battle with a tool unfit for production firefighting (see [“Debugging from First Principles” on page 138](#)). Getting results within seconds is what makes observability a useful investigative practice that lets you iterate quickly until you find meaningful results to advance your investigation.

As covered in [Chapter 5](#), events are the building blocks of observability, and traces are a collection of interrelated events (or trace spans). Finding meaningful patterns within those events requires an ability to analyze high-cardinality and high-dimensionality data. Any field within any event (or within any trace span) must be queryable. Those events cannot be pre-aggregated because, for any given investigation, you won't know in advance which fields may be relevant. All telemetry data must be available to query in full, unaggregated resolution, regardless of its complexity, or you risk hitting investigative dead ends.

Further, because you don't know which dimensions in an event may be relevant, you cannot privilege the data-retrieval performance of any particular dimension over others; they must all be equally fast. Therefore, all possible data that's needed must be indexed (which is typically prohibitively expensive), or data retrieval must always be fast without indexes in place.

Typically, in observability workflows, users are looking to retrieve data in specific time ranges, for specific services. That means the only exceptions to privileged data are the dimensions of time and partitioning by service. It is imperative that queries return all data recorded within specific time intervals, so you must ensure that it's indexed appropriately. A time-series database (TSDB) would seem to be the obvious choice here, but as you'll see later in this chapter, using one for unified observability presents its own set of incompatible constraints.

Because your observability data is used to debug production issues, it's imperative to know whether the specific actions you've taken have resolved the problem. Stale data can cause engineers to waste time on red herrings or make false conclusions about the current state of their systems. Therefore, an effective observability system should include not just historical data but also fresh data that reflects the current state in close-to-real time. No more than seconds should elapse between when the system receives data and when it makes that data available to query.

Lastly, that datastore must also be durable and reliable. You cannot lose the observability data needed during your critical investigations. Nor can you afford to delay your critical investigations because any given component within your data store failed. Any mechanisms you employ to retrieve data must be fault-tolerant and designed to return fast query results despite the failure of any underlying workers. The durability of your datastore must also be able to withstand failures that occur within your own infrastructure. Otherwise, your observability solution may also be inoperable while you're attempting to debug why the production services it tracks are inoperable.

Given that these functional requirements are necessary to enable real-time debugging workflows, traditional data-storage solutions are often inadequate for high-observability systems. At a small enough scale, data performance within these parameters can be more easily achieved. In this chapter, we'll examine how these problems manifest when you go beyond single-node storage solutions.

Time-Series Databases Are Inadequate for Unified Observability

At its core, observability data in a unified model consists of structured events representing information about a program's execution. As seen in [Chapter 5](#), these structured events are essentially collections of key-value pairs. In the case of tracing, the structured events can be related to one another in order to visualize parent-child relationships between trace spans. Some of those fields may be “known” (or predictable), given the auto-instrumentation use case seen in [Chapter 4](#). However, the most valuable data will be custom to your particular application. That custom data is often generated ad hoc, meaning that the schema used to store it is often dynamic or flexible.

As discussed in [Chapter 5](#), time-series data (metrics) aggregates system performance into simple measures. Metrics aggregate all underlying events over a particular time window into one simple and decomposable number with an associated set of tags. This allows for a reduction in the volume of data sent to a telemetry backend datastore and improves query performance, but limits the number of answers that can later be derived from that data. While some metric data structures (like those used to generate histograms) may be slightly more sophisticated, they still essentially bucket similar value ranges together and record counts of events sharing similar values (see [Figure 13-1](#)).

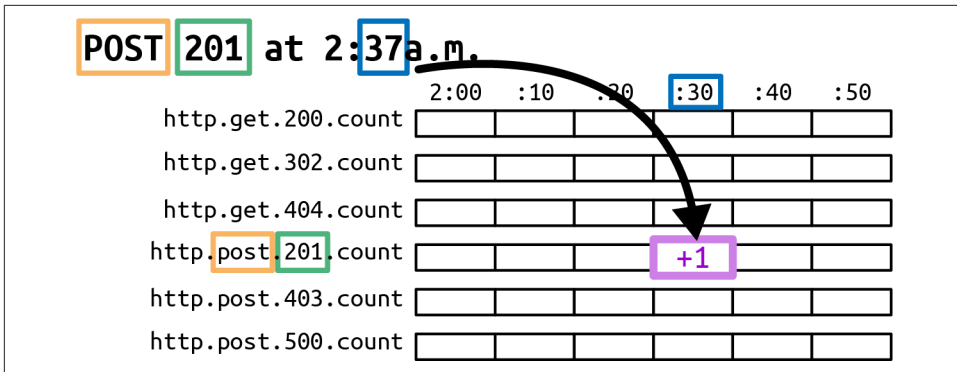


Figure 13-1. A prototypical TSDB showing a limited amount of cardinality and dimensionality, with tags for HTTP method and status code, bucketed by timestamp.

Traditionally, a *time-series database* (TSDB) is used to store aggregated metrics. Time-series data storage mechanisms aim to amortize the cost of additional traffic by ensuring that new combinations of aggregations and tags are rare; a high overhead is associated with creating a record (or database row) for each unique time series, but appending a numeric measurement to a time series that already exists has an extremely low cost.

On the query side, the predominant resource cost in a TSDB is finding which time series matches a particular expression; scanning the set of results is inexpensive, as millions of events can be reduced into a small number of time-windowed counts. The complexity increases in the number of time series that are filtered and the number of buckets that are examined, but not the number of events in each bucket.

In an ideal world, you could simply switch to recording structured events utilizing the same TSDB. However, the functional observability requirement to surface meaningful patterns within high-cardinality and high-dimensionality data makes using a TSDB prohibitive. Although you could convert each dimension to a tag name, and each value to the tag's value, this will create a new time series for each unique combination of tag values (see [Figure 13-2](#)).

The cost of creating new time series will amortize to zero across measurements only if the same tags are reused and incremented often. But each structured event is often unique and will cause the overhead of row creation to be linear with the number of events received. Each such row would receive only a single event and then have no value for all subsequent time periods. This problem of cardinality explosion thus makes TSDBs unsuitable for storing structured events. We need a different solution.

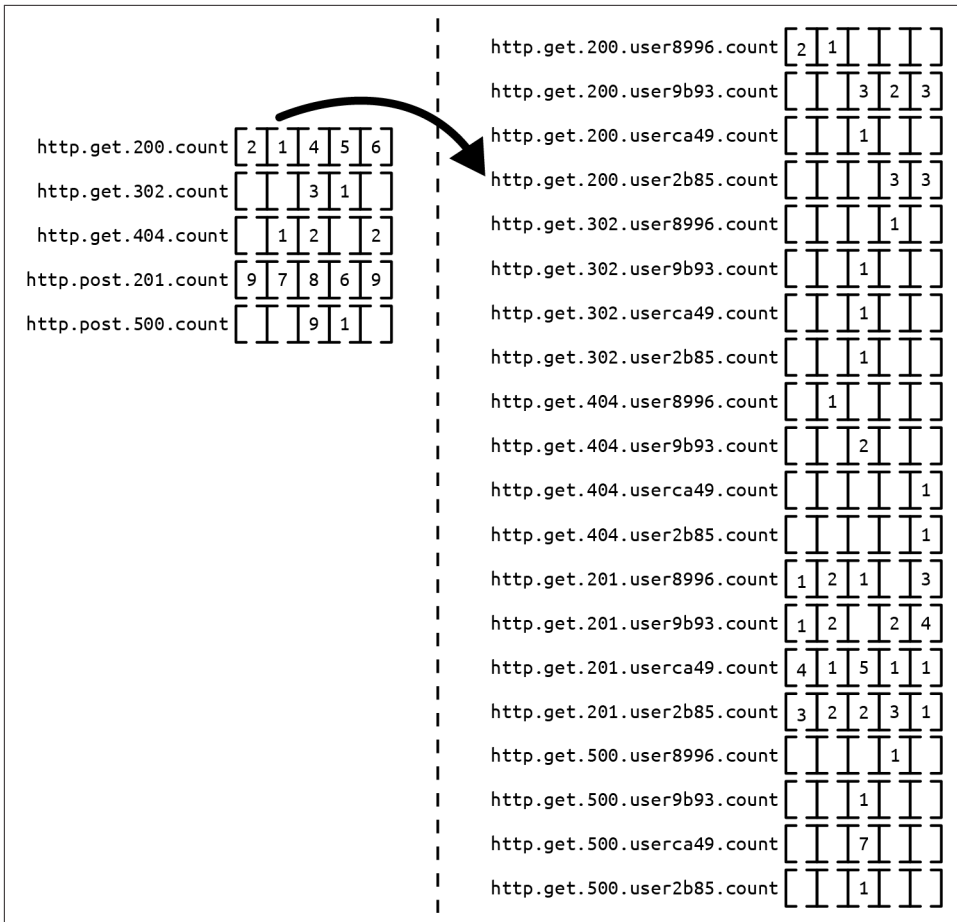


Figure 13-2. The explosion of that same TSDB when a high-cardinality index, *user id*, is added.

Other Possible Datastores

At first pass, storing structured sets of key-value pairs may appear similar to other workloads that can be addressed by general-purpose NoSQL databases, like MongoDB or CockroachDB. However, while the ingress of event telemetry data (or event ingestion) is well-suited to those databases, the egress patterns for that data (or event querying) are dramatically different from traditional workloads because of the functional requirements for observability.

That iterative analysis approach (described in [Chapter 8](#)) requires the ability to understand more than one dimension at a time, regardless of the number of possible

values each of those dimensions contains. In other words, you must be able to query on data with arbitrary dimensionality and cardinality.

A NoSQL database is likely to be slow for these kinds of arbitrary queries, unless the query utilizes only specific fields that are already indexed. But pre-indexing doesn't allow you to arbitrarily slice and dice; you're limited to slicing and dicing along only one dimension at a time, and it only works for the dimensions you remembered to index or pre-aggregate up front. And if each value is unique, it will take up as much space in the index as in the original table, resulting in a collection of indexes larger than the original data if we index every column. So trying to tune a NoSQL database to optimize its queries won't work. What if instead of trying to make a subset of queries fast, we tried to make *all* queries possible to complete within a reasonable amount of time?

At Facebook, the Scuba system effectively solved the problem of real-time observability querying through consumption of huge amounts of RAM in order to allow full table accesses.¹ However, it made the performance trade-off of using system memory for storage to speed up querying. That implementation decision made Scuba economically infeasible for mass adoption beyond the walls of Facebook. As of the time of publication for the second edition, 1 TB (terabyte) of RAM costs approximately \$5,000, whereas 1 TB of SSD costs less than \$50. If attempting to use Scuba as a datastore, an organization would be limited to querying only the amount of data available in RAM. For most large organizations with a reasonably sized infrastructure, that constraint would limit the time window for queryable telemetry data to minutes rather than hours or days.

Perhaps we can look to other datastores that are used to store data that is shaped like structured, wide events—namely, telemetry backends for tracing solutions. When we wrote the first edition of this book, a few open source implementations of large-scale storage engines existed, mostly **interoperable with the Jaeger tracing frontend**: Apache Cassandra, Elasticsearch, OpenSearch, ScyllaDB, and InfluxDB. These enabled ingestion and durable storage of tracing data using a schema, but they were not necessarily purpose-built for tracing specifically.

Grafana Tempo is an example of a newer implementation that is purpose-built for tracing. However, as of its v2 architecture, it's not necessarily built for the functional querying requirements of unified observability data, as we will see throughout this chapter. Observe Inc. has built a proprietary solution on top of Snowflake, which is great for data warehousing but less optimal for interactive investigations. Overall, and more importantly, none of the solutions mentioned so far are built to manage a variety of telemetry signal datatypes.

1 Lior Abraham et al., “Scuba: Diving into Data at Facebook,” *Proceedings of the VLDB Endowment* 6, no. 11 (2013): 1057–1067.

As of the writing for this second edition, **ClickHouse** has become the de facto standard open source component used by many organizations creating their own observability stack, and of a new generation of vendors marrying the open source OTel standard and an open source backend. You will learn about its datastore design directly from the ClickHouse team in the next chapter.

For now, let's examine the technical requirements to both store and query trace data so that a solution can meet the functional requirements of observability. While anything is technically possible, the chapter will look at how you can store your event data and derive insights through aggregating events at scale, beyond what is possible from just examining individual traces.

Data Storage Strategies

Conceptually, wide events and the trace spans they represent can be thought of as a table with rows and columns. Two strategies can be used to store a table: row-based storage and column-based storage. For the observability domain, *rows* pertain to individual telemetry events, and *columns* pertain to the fields or attributes of those events.

In *row-based storage*, each row of data is kept together (as in **Figure 13-3**), with the assumption that the entire row will be fetched at once. In *column-based storage*, columns of data are kept together, disaggregated from their rows. Each approach has trade-offs. To illustrate these trade-offs, we will use the time-tested **Dremel**, **ColumnIO**, and **Bigtable** as examples of the broader computational and storage models, rather than focus on specific open source telemetry stores used in practice today. In fact, the **Dapper tracing system** was originally built on the Bigtable datastore.

idx	Col1	Col2	Col3	Col4	Col5	Col6	Col7	Col8	Col9	ColA	ColB	ColC	ColD	...
1														
2														
3														
4														
5														
6														
7														
8														
9														
10														

Figure 13-3. A row store, indicating sharding of contiguous rows kept together as a single unit (a tablet) for lookup and retrieval.

Bigtable uses a row-based approach, meaning that the retrieval of individual traces and spans is fast because the data is serialized and the primary key is indexed (e.g., by time). To obtain one row (or scan a few contiguous rows), a Bigtable server needs to retrieve only one set of files with their metadata (a *tablet*). To function efficiently for tracing, this approach requires Bigtable to maintain a list of rows sorted by a primary row key such as trace ID or time. Row keys that arrive out of strict order require the server to insert them at the appropriate position, causing additional sorted insertion work at write time that is not required for the nonsequential read workload.

As a mutable datastore, Bigtable supports update and deletion semantics and dynamic repartitioning. In other words, data in Bigtable has flexibility in data processing at the cost of complexity and performance. Bigtable temporarily manages updates to data as an in-memory mutation log plus an ongoing stack of overlaid files containing key-value pairs that override values set lower in the stack. Periodically, when enough of these updates exist, the overlay files must be “compacted” back into a base immutable layer with a process that rewrites records according to their precedence order. That compaction process is expensive in terms of disk input/output (I/O) operations.

For observability workloads—which are effectively write once, read many—the compaction process presents a performance quandary. Because newly ingested observability data must be available to query within seconds, with Bigtable the compaction process would either need to run constantly, or you would need to read through each of the stacked immutable key-value pairs whenever a query is submitted. It’s impractical to perform analysis of arbitrary fields without performing a read of all columns for the row’s tablet to reproduce the relevant fields and values. It’s similarly impractical to have compaction occur after each write.

Bigtable optionally allows you to configure *locality groups* per set of columns in a column family that can be stored separately from other column families. But examining data from even a single column within a locality group still reads the entire locality group—while discarding most of the data, slowing the query, and wasting CPU and I/O resources. You can add indexes on selected columns or break out locality groups by predicted access patterns, but this work-around contradicts the observability requirement of arbitrary access. Sparse data within a locality group’s columns is inexpensive within Bigtable because columns exist only within the locality group. But locality groups as a whole should not be empty because of their overhead.

Taken to the extreme limit of adding an index on each column or storing each column in a separate locality group, the cost adds up.² Notably, the Google Dapper backend paper mentions that attempts to generate indexes in Bigtable on just three fields (service, host, and timestamp) for the Dapper Depots produced data that was

² “[Schema Design Best Practices](#)”, Google Cloud Bigtable, accessed April 10, 2026.

76% the size of the trace data itself.³ Therefore, if you could entirely eliminate the need for indexing, you'd have a more practical approach to storing distributed traces.

Independent of strategies like Bigtable, when using a column-based approach, it's possible to quickly examine only the desired subset of data. Incoming data is partitioned into columns (as in **Figure 13-4**), with a synthetic `id` column providing a primary key for each row. Each column file with a mapping from primary key to the value for that column is stored separately. Thus, you can independently query and access the data from each column.

However, the purely column-based approach does not guarantee that any given row's data is stored in any particular order. To access the data for one row, you may need to scan an arbitrary quantity of data, up to the entire table. The Dremel and ColumnIO storage models attempt to solve this problem by breaking tables down with manual coarse sharding (e.g., having separate tables for `tablename.20250820`, `tablename.20250821`, etc.) and leaving it to you, the user, to identify and join together tables at query time. This becomes immensely painful to manage at large data volume; the shards either become too large to efficiently query, or are broken into such numerous small shards (`tablename.20250820-000123`) that a human being has trouble constructing the query with the right shards.

idx	Col1	Col2	Col3	Col4	Col5	Col6	Col7	Col8	Col9	ColA	ColB	ColC	ColD	...
1														
2														
3														
4														
5														
6														
7														
8														
9														
10														

Figure 13-4. Column store showing data broken down by column rather than by row, with each set of columns accessible independently, but with a full row scan necessitating pulling each column file.

Neither the purely row-based or purely column-based approach is entirely adequate to meet the functional needs for observability. To address the trade-offs of both row

³ Benjamin H. Sigelman et al., “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” Google Technical Report, April 2010.

and columnar storage, a hybrid approach—utilizing the best of both worlds—can meet the types of tracing workloads needed for observability. A hybrid approach would enable you to efficiently perform partial scans of both rows and columns. Remember that, for observability workloads, it's more important for query results to return fast than it is for them to be exhaustively complete.

To illustrate how you might practically achieve such a balance, we will examine the architecture of the Honeycomb storage engine that represents one way to achieve those functional requirements.

Case Study: The Implementation of Retriever

In this section, we explain the implementation of Honeycomb's columnar datastore (a.k.a. Retriever) to show how you can meet the functional requirements for observability with a similar design. We hope to illustrate concrete implementation and operational trade-offs that you would not otherwise see in an abstract discussion of a theoretical model, so that you can compare and contrast our implementation to others available to you. In the next chapter, you will also learn what trade-offs ClickHouse made in its design.

Partitioning Data by Time

Earlier, when we discussed the challenges of row-based versus column-based storage, we suggested a hybrid approach of reducing the search space through partitioning it by timestamp. However, time in a distributed system is never perfectly consistent, and the timestamp of a trace span could be the start rather than end time of a span, meaning data could arrive seconds or minutes behind the current time. It would not make sense to reach back into already saved bytes on disk to insert records in the middle, as that would incur costs to rewrite the data. So, how exactly should you perform that partitioning instead to make it efficient and viable in the face of out-of-order data arrival?

One optimization you can make is assuming that events are likely to arrive in close proximity to the timestamps they were actually generated at, and that you can correct out-of-order arrival at read time. By doing this, you can continue to treat files storing incoming data as append-only, which significantly simplifies the overhead at write time.

With Retriever, newly arriving trace spans for a particular tenant are inserted at the end of the currently active set of storage files (*segment*) for that tenant. To be able to query the right segments at read time, Retriever tracks the oldest and newest event timestamp for the current segment (creating a window that can potentially overlap with other segments' windows). Segments eventually do need to end and be finalized, so you should pick appropriate thresholds. For instance, when one hour has elapsed,

when more than 250,000 records have been written, or when it has written more than 1 GB (gigabyte), Retriever finalizes the current segment as read-only and records the final oldest and newest timestamps of that segment in the metadata.

If you have adopted this windowed segment pattern, at read time you can utilize the metadata with timestamp windows of each segment to fetch only the segments containing the relevant timestamps and tenant datasets for the query you wish to perform (as shown in [Figure 13-5](#)). Typically, an observability workload is looking for data in a specific time range when queries are performed (e.g., “now to two hours ago,” or “2025-08-20 00:01 UTC to 2025-09-20 23:59 UTC”). Any other time range is extraneous to the query and, with this implementation, you don’t need to examine data for irrelevant segments that lack an overlap with the query time.

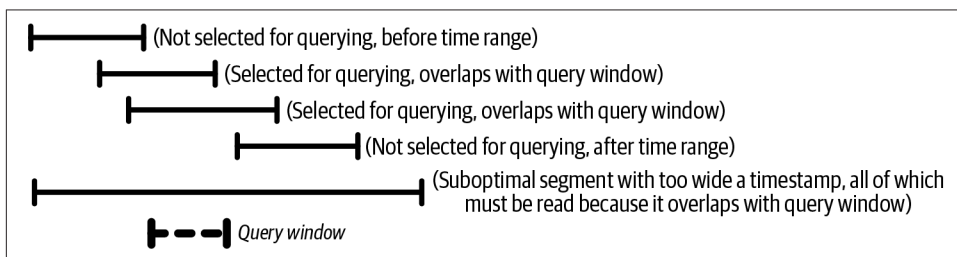


Figure 13-5. Segments selected for querying are those that overlap at least in part with the query window; segments that start and end before or after the query window are excluded from analysis.

The advantage of this segment partitioning by time is twofold:

1. Individual events do not need to be sorted and put into a strict order by event timestamp, as long as the start and end timestamps of each segment are stored as metadata and events arrive with a consistent lag.
2. The contents of each segment are append-only artifacts that can be frozen as is once finished rather than needing to be built as mutable overlays/layers and compacted periodically.

The segment-partitioning approach has one potential weakness: when backfilled data is intermingled with current data (for instance, if a batch job finishes and reports timestamps from hours or days ago), each segment written will have metadata indicating it spans a broad time window covering not just minutes, but potentially hours’ or days’ worth of time.

In this case, segments will need to be scanned for any query in that wide window, rather than being scanned for data in only the two narrower windows of time—the current time, and the time around when the backfilled events happened. Although the Retriever workload has not necessitated this to date, you could layer on more

sophisticated segment partitioning mechanisms (or post facto rewriting and compaction mechanisms) if it became a significant problem.

Storing Data by Column Within Segments

As with the pure Dremel-like column-based approach outlined earlier, the next logical way to break down data after time has been taken out of the equation is by decomposing events into their constituent fields, and storing the contents of each field together with other values from the same field present in other events. This leads to a layout on disk of one append-only file per field per segment, plus a special timestamp index field (since every event must have a timestamp). As events arrive, you can append an entry to each corresponding file in the currently active segment for the columns they reference.

Once you have performed the filtering by segment timestamp described in the preceding section, you can restrict the amount of data that you access upon query to only the relevant columns specified in the query by accessing the appropriate columns' own files. To be able to reconstruct the source rows, each row is assigned a timestamp and relative sequence within the segment. Each column file exists on disk as an array holding—a sequence number followed by the column's value—repeated for each row in the segment.

For instance, this source collection of rows (which we've supposed is ordered by time of arrival) might be transformed as follows:

```
Row 0: { "timestamp": "2025-08-20 01:31:20.123456",  
        "trace.trace_id": "efb8d934-c099-426e-b160-1145a87147e2", "field1": null, ... }  
Row 1: { "timestamp": "2025-08-20 01:30:32.123456",  
        "trace.trace_id": "562eb967-9171-424f-9d89-046f019b4324", "field1": "foo", ... }  
Row 2: { "timestamp": "2025-08-20 01:31:21.456789",  
        "trace.trace_id": "178cdf06-dbc5-4c0d-b6b7-5383218e1f6d", "field1": "foo", ... }  
Row 3: { "timestamp": "2025-08-20 01:31:21.901234",  
        "trace.trace_id": "178cdf06-dbc5-4c0d-b6b7-5383218e1f6d", "field1": "bar", ... }
```

Timestamp index file

idx value

```
0 "2025-08-20 01:31:20.123456"  
1 "2025-08-20 01:30:32.123456" (note: timestamps do not need to be chronological)  
2 "2025-08-20 01:31:21.456789"  
3 "2025-08-20 01:31:21.901234"  
[...]
```

Raw segment column file for column `field1`

(note there is neither a row index nor value for missing/null values of a column)

idx value

```
1 "foo"  
2 "foo"  
3 "bar"  
[...]
```

Given the tendency of columns to either be null or to contain repeated values that may have been seen before, you could use dictionary-based compression, sparse encoding, or run-length encoding to reduce the amount of space that each value takes when serializing the entire column.⁴ Compacting the data using knowledge of the finished column reduces its size on disk. This benefits your backend in terms of the amount of data that must be tiered across your backend storage, as well as the amount of time that it takes to retrieve and scan that data:

```
compacted timestamp index file:
["2025-08-20 01:31:20.123456",
 "2025-08-20 01:30:32.123456",
 "2025-08-20 01:31:21.456789",
 "2025-08-20 01:31:21.901234",
 ...]

compacted segment column file, with dictionary, for column `example`
dictionary: {
  1: "foo",
  2: "bar",
  ...
}

presence:
[Bitmask indicating which rows have non-null values]

data from non-null row indexes:
[1, 1, 2, ...]
```

Depending on its distribution, numerical and timestamp data is also particularly susceptible to numerical compression techniques, for instance delta encoding, shifting integers to eliminate trailing zeros, and separation of the float exponent from the mantissa. Trace and span IDs in hexadecimal string encoding can be coerced into integers, representing the ultimate high-cardinality data, as no span or trace ID should collide. For booleans, roaring bitmaps may present an efficient storage format. And some numerical data may be sufficiently repetitive or enum-like that a dictionary is more efficient than any of the other formats.

Once a column is compressed with its data encoded to be more compression-friendly, it (or the segment as a whole, consisting of a collection of column files) is then compressed with a standard algorithm such as LZ4 in order to further reduce the space it takes when archived. Given that Retriever has pushed much of the compute costs to read time, decompression speed is a greater imperative than compression ratio.

⁴ Terry Welch, “A Technique for High-Performance Data Compression,” *Computer* 17, no. 6 (1984): 8–19.

In a column store, the cost of creating and tracking a column gets amortized against all of the values written to it; its analogue in the metrics world is the row or set of tags whose cost is amortized across all values. So, it is performant to create new columns whenever you might occasionally have non-null values, but it isn't performant to create one-time-use columns that have only a single row with a non-null value.

In practice, this means that you don't need to worry about manually adding new attributes or key names to your spans and events when writing your code. However, you don't want to programmatically create a column named `timestamp_2025061712345` that gets used once to write `true` and is never written to again; instead, use `timestamp` as the column value and `2025061712345` as the key. You can always define ad hoc aggregations or virtual columns based on existing data, rather than duplicate the data on disk forever.

Performing Query Workloads

Performing a query by using a column store designed similarly to Retriever consists of six steps:

1. Identify all segments that potentially overlap with the time range of the query, using the start and end times of the query and start and end times of each eligible segment.
2. Independently, in each matching segment: for the columns used for the filters of the query (e.g., `WHERE`) or used for the output (e.g., used as a `SELECT` or `GROUP`), scan the relevant column files only for fields referenced in the query. To perform a scan, track the current offset you are working on. Evaluate the timestamp of the row at the current offset first to validate whether the row falls within the time range of the query. If not, advance to the next offset and try again.
3. For rows that fall within the timeframe, scan the value at that offset for each column used as an input or output, emitting output values into the reconstructed row where the input filters match. Increment all offsets for all open column files to the next row once the row is processed. Individual rows can be processed and then discarded, minimizing the amount of data held in RAM to just a single value rather than the entire file for the column's values in the segment.

Note that an implicit `GROUP` is performed on `timestamp` where there's the need for subwindows within the original time range (e.g., reporting 24 five-minute windows for the past two hours of data).

4. Aggregate within the segment. For each `GROUP`, merge selected values from step 3. For instance, `COUNT` simply collects the number of matching rows for each `GROUP` value, while `SUM(example)` might add all identified values for a column example for each `GROUP` value. After each segment is analyzed, its file handles can be unloaded.

5. Aggregate across segments by sending results to a single worker for each GROUP. Merge the aggregated values from each GROUP for each segment, and aggregate them into a single value (or set of values per time granularity bucket) for the time range.
6. Sort the groups and pick the top K groups to include in the graph. A default value of K should be used to avoid transmitting thousands of matching groups as a complete result set, unless the user requests it.

We can work through here in pseudocode, calculating SUM, COUNT, AVG, MAX, etc. based on keeping a cumulative sum or highest value seen to date—for instance, on the value of the field x grouped by fields a and b , where y is greater than zero:

```
groups := make(map[Key]Aggregation)
for _, s := range segments {
    for _, row := range fieldSubset(s, []string{"a", "b", "x", "y"})
        if row["y"] <= 0 {
            continue
        }
        key := Key{A: row["a"], B: row["b"]}
        aggr := groups[key]
        aggr.Count++
        aggr.Sum += row["x"]
        if aggr.Max < row["x"] {
            aggr.Max = row["x"]
        }
        groups[key] = aggr
    }
}
for k := range groups {
    groups[k].Avg = groups[k].Sum / groups[k].Count
}
```

For production use, this code would need to be generalized to support encoding arbitrarily many groups, arbitrary filters, and computing aggregations across multiple values, but it gives an idea as to how to compute aggregations in an efficiently parallelizable way. Algorithms such as quantile estimation with [t-digest](#) and [HyperLogLog](#) are required for calculating more complex aggregations such as P99 (for the 99th percentile of a set of numerical values) and COUNT DISTINCT (for the number of unique values in a column). You can further explore these sophisticated algorithms by referring to the academic literature.

In this fashion, you are able to performantly solve the problems of high cardinality and high dimensionality. No dimension (aside from timestamp, and the partitioning of the data by service) is privileged over any other dimension. It is possible to filter by an arbitrarily complex combination of one or more fields, because the filter is processed ad hoc at read time across all relevant data. Only the relevant columns are

read for the filter and processing steps, and only the relevant values corresponding to a matching row ID are plucked out of each column's data stream for emitting values.

There is no need for pre-aggregation or artificial limits on the complexity of data. Any field on any trace span can be queried. A finite cost is associated with ingesting and unpacking each record into constituent columns at write time, and the computational cost of reading remains reasonable.

Queries on Parts of Fields and Aggregated Fields

Querying ad hoc values not defined in the schema is also possible because all of the computation is done at read time rather than write time. Thus, the query engine could map multiple data fields together to construct calculated fields. Hypothetically, suppose you were renaming `session_id` to `session.id`; you could construct a temporary field `session_migration` that took the first of the session ID fields present:

```
let session_migration = COALESCE($session.id, $session_id)
```

The rest of the query engine could just use `session_migration` as any other field, performing a `COUNT DISTINCT` on it, or a filter with `WHERE` or similar, as we discussed in [Chapter 11](#) on event-based SLOs and SLIs.

For migrating from unstructured logs to structured data, while it would be best to clean the data up at write time to generate many separate columns and allowing for each column to be read independently, you can use regexes to extract parts of the fields. For instance, you can use `INT(REG_VALUE($message, `status_code=([0-9]+)`))` to extract a numeric status code from a larger log line. This can either be constructed ad hoc per query, or persisted as a synthetic column definition in the database.

Similar to how traditional search works across less sharded datasets, lightweight full-text search corpuses can be built from each segment's string fields, stored alongside each segment, and used as a mechanism to identify relevant row indexes without having to scan each field and value. This allows for queries asking about n-grams and keywords present in any field in a more traditional sense, while still offering the parallelism and durability even if the columnar property is not fully leveraged.

Querying for Traces

Retrieving traces is a specific, degenerate kind of query against this columnar store. To look for root spans, you query for spans `WHERE trace.parent_id is null`. And looking for all spans with a given `trace_id` to assemble a trace waterfall is a query for:

```
SELECT timestamp, duration, name, trace.parent_id, trace.span_id
WHERE trace.trace_id = "guid".
```

With the column storage design, it's possible to get insights on traces and to decompose traces into individual spans that can be queried across multiple traces. This design makes it feasible for Retriever to query for the duration of a span relative to other similar spans in other traces.

In other solutions that are more limited in cardinality, `trace.trace_id` might be special-cased and used as an index/lookup, with all spans associated with a given trace stored together. While this approach provides higher performance for visualizing individual traces and creates a special-case path to avoid the cardinality explosion of every `trace_id` being unique, it suffers in terms of inflexibility to decompose traces into their constituent spans for analysis.

Joins

To solve the problem of doing cross-span lookups and trace-level aggregations (e.g., expressing “parent has an attribute with a value” or “root does not have this attribute” or “any child has an error message”), it does become necessary to ensure that data associated with one trace is routed to the same dataset and same partitions, by consistently assigning a partition based on the trace ID. Instead of requiring multiple passes, Retriever reads data off of disk once, segment by segment, and builds up an in-memory map of spans that match filters for the “left” and “right” sides of the join. When it finds a match on the join condition (for instance, `left.span_id = right.parent_id`), we emit a “virtual row” to the rest of the query processing engine that contains information from both of the joined spans.

To manage memory, Retriever makes an effort when reading rows off of disk to send them to the joiner in temporal order. It keeps a running time window of viable joins that moves forward as the joiner consumes rows. As the window moves, it removes spans from memory if they are no longer in the window. The size of the window is dynamically adjusted based on available memory on the host.

Unlike ClickHouse, full arbitrary joins beyond those on a single trace span require a multipass querying approach which, as of this writing, is not implemented because it would require shuffling and merging data from multiple partitions. The Retriever design is optimized for each partition being able to perform query operations fully independently, except for the final merge step for the results from each partition. ClickHouse has different origins and thus a different design for handling joins, as you'll see in the next chapter.

Querying Data in Real Time

As established in the earlier functional requirements for observability, data must be accessible in real time so as to give operators accurate data about the system's state and whether their interventions have worked. Therefore, querying in-flight data must be possible within the design of the observability system. You cannot wait for segments to be finalized, flushed, and compressed before making them eligible for querying.

In the implementation of Retriever, we ensure that open column files are always queryable and that the query process can force a flush of partial files for reading, even if they have not yet been finalized and compressed. In a different implementation, another possible solution could be to allow querying data structures in RAM for segment files that have not yet been finalized and compressed. Yet another possible solution (with significantly higher overhead) could be to forcibly flush data every few seconds, regardless of the amount of data that has arrived—though this could be the most problematic at scale.

The first solution of on-demand partial flushes allows for separation of concerns between the ingestion and query processes. The query processor can operate entirely via files on disk, and the ingestion process can focus only on creating the files on disk without having to also maintain shared state. We found this to be an elegant approach appropriate to the needs of our Retriever.

Making It Affordable with Tiering

Depending on the volume of data managed, using different tiers of data storage can result in pretty significant cost savings. Not all data is queried equally often. Typically, observability workflows are biased toward querying newer data, especially in the case of incident investigation. As you've seen, data must be available for querying within seconds of ingestion.

The most recent minutes of data may need to live on the local SSD of a query node while it is being serialized. But older data can and should be offloaded to a more economical and elastic datastore. In the implementation of Retriever, closed segment directories older than a certain age are compacted (e.g., by rewriting with a bitmask of present/absent entries or being compressed) and uploaded to a longer-term durable network file store. Retriever uses Amazon S3, but you could use a different solution like Google Cloud Storage. Later, when that older data is needed again for querying, it can be fetched back from S3 into the memory of the process doing computation, and then unpacked in memory if it was a collection of sparse files. The column scan can then be performed on each matching column and segment.

Making It Fast with Parallelism

Speed is another functional requirement for observability workflows: query results must return within seconds to drive iterative investigation. And this should hold true regardless of whether queries are retrieving 30 minutes of data or an entire week of data. Users must often compare the current state to historical baselines to have an accurate idea of whether behavior is anomalous.

When performing queries against data in a cloud file storage system in a **MapReduce-style pattern** with serverless computation, we have found that performance can be faster than executing that same query against serial data in local SSDs with a single query engine worker. What matters to end users is the speed at which query results return, not where the data is stored.

Fortunately, using the approaches seen in earlier sections, you can independently compute the result of a query for each segment. There's no need to process segments serially, as long as a reduction step takes the output of the query for each segment and merges the results.

If you have already tiered data onto a durable network file store—S3 in our case—there is no contention upon a single query worker holding the bytes on disk. Your cloud provider has already taken on the burden of storing files in a distributed manner that is unlikely to have single points of congestion.

Therefore, a combination of a MapReduce-style approach and serverless technology enables us to get distributed, parallelized, and fast query results for Retriever without having to write our own job management system. Our cloud provider manages a pool of serverless workers, and we simply need to provide a list of inputs to map. Each Lambda (or serverless) function is a mapper that processes one or more segment directories independently, and feeds its results through a collection of intermediate merges to a central reduce worker to compute the final results.

But serverless functions and cloud object storage aren't 100% reliable. In practice, tail latency can be orders of magnitude higher than the median invocation time. That last 5–10% of results may take tens of seconds to return, or may *never* complete. In the Retriever implementation, we use RPC impatience to return results in a timely manner. Once 90% of the segment-processing requests have completed, the remaining 10% are re-requested in parallel, without canceling the still-pending requests. These parallel attempts race each other; whichever responds first is used to populate the query result. Even if it's 10% more expensive to always retry the slowest 10% of subqueries, a different read attempt against the cloud provider's backend will likely perform faster than a request stuck on S3 or network I/O that may never finish before it times out.

For Retriever, we've figured out this method to mitigate blocked distributed performance so that we always return query results within seconds. For your own implementation, you will need to experiment with alternative methods in order to meet the need of observability workloads.

Dealing with High Cardinality

What happens if someone attempts to group results by a high-cardinality field (for instance, a raw log message)? How can you still return accurate values without running out of memory? The simplest solution is to fan out the reduce step by assigning reduce workers to handle only a proportional subset of the possible groups. For instance, you could follow the pattern **Chord does** by creating a hash of the group and looking up the hash correspondence in a ring covering the keyspace.

But in the worst case of hundreds of thousands of distinct groups, you may need to limit the number of returned groups by the total amount of memory available across all your reducers, estimate which groups are most likely to survive the ORDER BY/LIMIT criterion supplied by the user, or simply abort a query that returns too many groups. After all, a graph that is 2,000 pixels tall will not be very useful with a hundred thousand unique lines smudged into a blur across it!

Scaling and Durability Strategies

For small volumes of data, querying a single small-throughput data stream across arbitrarily long time windows is more easily achievable. But larger datasets require scaling out horizontally, and they must be kept durable against data loss or temporary node unavailability. Fundamental limits exist on how fast any one worker, no matter how powerful, can process incoming trace spans and wide events. Additionally, your system must be able to tolerate individual workers being restarted (e.g., either for hardware failure or maintenance reasons).

In the Retriever implementation, we use streaming data patterns for scalability and durability. We've opted not to reinvent the wheel and instead leverage existing solutions where they make sense. Apache Kafka has a streaming approach that enables us to keep an ordered and durable data buffer that is resilient to producers, consumers, or intermediary brokers restarting. We've drawn how we stripe incoming events across Kafka partitions and indexes in **Figure 13-6**.



See *Kafka: The Definitive Guide* by Gwen Shapira et al. (O'Reilly, 2021) for more information on Kafka.

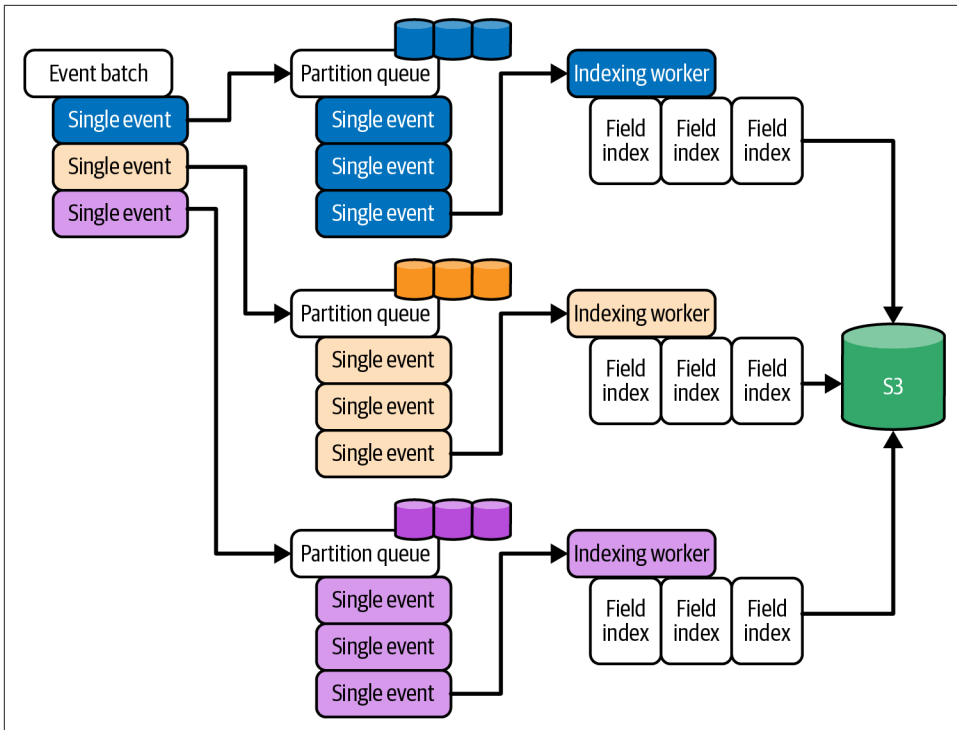


Figure 13-6. The life of an incoming wide event as it is written to Kafka by an ingest worker, then read from Kafka by indexing workers. Each partition is replicated across multiple Kafka nodes.

For example, a row (wide event or trace span, in our use case) stored at index 1234567 in a given topic and partition will always precede the row stored at index 1234568 in Kafka. That means that two consumers that must read data starting at index 1234567 will always receive the same records in the same order. And two producers will never conflict over the same index—each producer’s committed rows will be written in a fixed order.

When receiving incoming telemetry, you should use a lightweight, stateless receiver process to validate the incoming rows and produce each row to one selected Kafka topic and partition. Stateful indexing workers then can consume the data in order from each Kafka partition. By separating the concerns of receipt and serialization, you’ll be able to restart receiver workers or storage workers at will without dropping or corrupting data. The Kafka cluster needs to retain data only as long as the maximum duration that would be needed for replay in a disaster-recovery scenario—hours to days, but not weeks.

To ensure scalability, you should create as many Kafka partitions as necessary for your write workload. Each partition produces its own set of segments for each set of data. When querying, it's necessary to query all matching segments (based on time and dataset) regardless of which partition produced them, because a given incoming event could have gone through any eligible partition (although a given trace's spans will all be consistently routed to the same partition based on the trace ID). Maintaining a list of which partitions are eligible destinations for each tenant dataset will enable you to query only the relevant workers; however, you will need to combine the results from workers performing the query on different partitions by performing a final result-merging step on a leader node for each query. As different workloads use varying amounts of capacity, we found it necessary to write an automatic rebalancer that would shift clients between partitions, and partitions between brokers to even out the load and ensure good performance as each client scales up.

To ensure redundancy, more than one ingestion worker can consume from any given Kafka partition. Since Kafka ensures consistent ordering and the ingestion process is deterministic, parallel ingestion workers consuming a single partition must produce identical output in the form of serialized segment files and directories. Therefore, you can select one ingestion worker from each set of consumers to upload finalized segments to S3 (and spot-check that the output is identical on its peer).

If a given ingestion worker process needs to be restarted, it can checkpoint its current Kafka partition index and resume serializing segment files from that point once it's restarted. If an ingestion node needs to be entirely replaced, it can be started with a data snapshot and Kafka offset taken from an earlier healthy node and replayed forward from that point.

In Retriever, we've separated the concerns of ingesting and serializing data from those of querying data. If you split the observability data workloads into separate processes that share only the filesystem, they no longer need to maintain shared data structures in RAM and can have greater fault tolerance. That means whenever you encounter a problem with the serialization process, it can only delay ingestion, rather than prevent querying of older data. This has the added benefit of also meaning that a spike in queries against the query engine will not hold up ingestion and serialization of data.

In this fashion, you can create a horizontally scalable version of the column storage system that allows for fast and durable querying across arbitrary cardinality and dimensionality. As of November 2025, the production US-hosted deployment of Retriever operates as a multitenant cluster consisting of approximately 1,500 Graviton4 vCPU of receiver workers, 240 Graviton2 vCPU of Kafka brokers, and 2,000 Graviton3 vCPU of Retriever ingest + query engine workers, plus bursts up to 200,000 Graviton2 vCPU using Lambda serverless workers. This cluster serves queries against approximately 1.5 PB (petabytes) of columnar data in over 1 billion

compressed segment archives representing two months of our customers' historical data. It ingests 3–5 million trace spans per second (representing up to 20x that produced by our clients once sampling is considered) with a maximum lag of milliseconds until they are queryable. It does this while delivering dozens of read queries per second with a median latency of 50 milliseconds and a P99 of five seconds.⁵

Notes on Building Your Own Efficient Datastore

Observability workloads require a unique set of performance characteristics to ingest telemetry data and make it queryable in useful ways throughout the course of investigating any particular issue. A challenge for any organization seeking to build its own observability solution will be to set up an appropriate data abstraction that enables the type of iterative investigative work for open-ended exploration, to identify any possible problem. You must address the functional requirements necessary for observability in order to use your telemetry data to solve hard problems.

It's not necessary to start with half a dozen different “macroservices” like we have if you're only intending to self-host a small volume of traffic, or if durability and reliability are not critical. The complexity of Honeycomb does not scale down well to tiny traffic volumes. For that kind of workload, the single-binary design of ClickHouse offers a better initial experience. But ensuring you have a path to scale up ensures your investment will stand the test of time.

If your problems are too hard to solve, you probably have the **wrong data abstraction**. Building on the correct data abstraction allows for making the best trade-offs for solving a given data domain. For the problem of observability and tracing, a distributed hybrid columnar store segmented by time is one approach that meets all of the critical requirements of speed, cost, and reliability. While previous generation datastores struggle to meet the demands of arbitrary cardinality and dimensionality and of tracing, the Retriever datastore at Honeycomb exemplifies one approach that can solve the problem.

We hope the lessons we have learned serve you well in understanding the underlying architecture of a modern observability backend, or in solving your own observability problems should you need an in-house solution. We also urge you not to suffer through the headaches of maintaining an Elasticsearch or Cassandra cluster that is not as suitable for this purpose as a dedicated column store such as Retriever or ClickHouse.

⁵ This may not seem impressive, until you realize routine queries scan hundreds of millions of records!

Conclusion

Numerous challenges exist when it comes to effectively storing and performantly retrieving your observability data in ways that support real-time debugging workflows. The functional requirements for observability necessitate queries that return results as quickly as possible, which is no small feat when you have billions of rows of ultrawide events, each containing thousands of dimensions—all of which must be searchable, many of which may be high-cardinality data. And none of those fields are indexed or have privileged data retrieval over others. You must be able to get results within seconds, and traditional storage systems are simply not up to performing this task. In addition to being performant, your datastore must also be fault-tolerant and production-worthy.

We hope that the case study of the Honeycomb Retriever implementation helps you better understand the various trade-offs that must be made at the data layer and possible solutions for managing them. Retriever is not the only way to implement solutions to these challenges. When we first started building modern observability systems, skeptics claimed it was impossible to store high-cardinality event data and query it interactively at scale. A decade later, multiple production-proven approaches have emerged.

Other columnar datastores commonly used to power observability workloads include ClickHouse, Google Cloud BigQuery, and Apache Druid. Each makes different architectural trade-offs around query performance, operational complexity, and cost. In the next chapter, you'll see how ClickHouse has approached these same challenges with its design decisions. The ClickHouse team will walk through how they've tuned their open source columnar engine specifically for observability use cases, demonstrating how the core principles we've discussed can be implemented successfully in multiple ways.

Enabling high-cardinality and exploratory observability is a solved problem. Your choice of implementation depends on your specific operational constraints, not on whether it's possible.

Efficient Data Storage with ClickHouse

This chapter is contributed by ClickHouse team members Rory Crispin, Vlad Seliverstov, Xander Garbett, and Dale McDiarmid

A Note from Charity, Liz, George, and Austin

In the previous chapter, you learned about the design and implementation trade-offs used by the Honeycomb Retriever datastore to manage observability workloads. In this chapter, you'll see how a popular open source datastore makes contrasting trade-offs to solve the same problems. Multiple implementations of columnar database technology can enable the exploratory data workflows we've advocated throughout this book; we've invited ClickHouse to share their battle-tested approach. You may find it especially valuable to read this chapter if you are already using ClickHouse as a storage backend for your observability data, or considering building on ClickHouse.

Modern observability workloads require flexible schemas to capture evolving telemetry, ingestion rates that can sustain several million rows per second, and query latencies low enough to support interactive exploration over trillions of rows. You don't have to be tied to any one specific vendor or solution, but knowing that those solutions exist and something about their architecture can help you make better technical decisions to support the business and operational needs.

The rest of this chapter is written from the ClickHouse team's perspective.

ClickHouse is an open source, column-oriented database management system designed for real-time analytical querying at massive scale. It is built to address the demands of modern observability workloads, through its columnar storage engine, vectorized execution, and support for schema-rich event data. These design decisions enable it to serve as a unified backend for telemetry, without relying on pre-aggregation or rigid pipelines.

The ability of ClickHouse to ingest and query large volumes of event data at high speed enables ad hoc analysis for fast root-cause identification and operational decision making. Observability data is exposed through standard SQL, enabling teams to use familiar syntax and tooling. ClickHouse embraces open standards, supporting direct American National Standards Institute (ANSI) SQL access for interactive queries, dashboards, and integration with downstream systems such as alerting or AI pipelines. This ensures interoperability and avoids query or schema lock-in. Open source projects, community integrations, and custom tooling such as HyperDX, provide additional options for visualization and workflow integration. For instance, Dash0, SigNoz, and SolarWinds use ClickHouse as their datastore under the hood.

By consolidating multiple telemetry signal types into high-cardinality wide events, ClickHouse not only supports structured, high-dimensional data but also enables correlations across diverse non-observability sources at query time. This capability enables you to use ClickHouse as a backend for modern observability workloads while also connecting it to data sources used for data warehousing and other emerging workloads.

This chapter demonstrates how ClickHouse enables an observability model built on wide events, supporting fast exploration, real-time alerting, and root-cause analysis. We introduce the core concepts behind ClickHouse performance, walk through practical ingestion with OTel, outline key schema and query optimizations, and cover strategies for scaling from a single node to multiregion clusters. We close with an overview of how visualization layers connect to ClickHouse for analysis.

ClickHouse Core Concepts

Building effective observability systems with ClickHouse begins with an understanding of how its architecture aligns with the characteristics of telemetry data. Observability workloads are demanding. Modern systems generate massive volumes of real-time, high-cardinality data and time series; these need to be ingested efficiently and made available for analytical queries. In this section, you will explore MergeTree, the ClickHouse storage engine. You'll also see how ClickHouse executes queries internally, how primary keys influence data organization and read efficiency, and how various caching layers contribute to consistent, low-latency performance.



MergeTree is deployed within ClickHouse as a single binary with a homogeneous architecture—every node can handle both reads and writes. This uniform design is key to grasping how MergeTree tables work, as data ingestion, background merges, and query processing all follow the same principles across nodes. Scaling out from a single server with sharding and replication builds on this common foundation. Dedicated read or write paths are an optimization on the same core architecture.

MergeTree Fundamentals

The **MergeTree family** of table engines forms the foundation of nearly all persistent storage in ClickHouse, with variants designed for aggregation, sharding, and replication provided to extend this core architecture. While it is possible to use other engines within ClickHouse, only MergeTree is optimized for observability.

The MergeTree engine organizes a table as multiple **parts**, each containing a subset of rows stored in a columnar format and sorted by the table's designated ordering key. A part is produced for each insert, typically consisting of several thousand rows. Within each part, data is further divided into granules—contiguous blocks of rows that serve as the smallest unit of index and that facilitates data skipping. Once written, these parts are immutable, with background processes continuously merging smaller parts into new, larger ones up to a configurable maximum size—currently 150 gigabytes (GB) by default at the time of writing.



Observability data is mostly write-once: you append events and rarely mutate them. That fits the ClickHouse immutable-parts design and fast insert path. When changes and redactions are needed, ClickHouse does support lightweight updates and deletes, but they're inherently slower than append-only writes and are best kept off the hot path.

This approach is reminiscent of a **log-structured merge-tree (LSM tree)**: immutable writes, background consolidation, and eventual compaction. However, MergeTree differs in two critical ways: it is columnar rather than row-oriented, and it's explicitly ordered by a **user-defined key** (“primary key”). These differences optimize for analytical workloads rather than transactions.

This “one part per insert” architecture is what enables ClickHouse to sustain extremely high ingest rates without coordination overhead. **Background merges** gradually consolidate these parts into larger, sorted blocks, which not only compress efficiently but also preserve data locality for fast access. The result is a storage engine uniquely well-suited for wide, high-cardinality event data: it ingests at streaming scale, compresses to minimize storage footprint while maintaining ordering, and accelerates analytical queries that must scan across billions of rows.

To better understand how inserts work in MergeTree, consider **Example 14-1**. It demonstrates how a raw insert is transformed into a durable, queryable data part by creating an events table and inserting a few sample records.

Example 14-1. Example SQL statements for creating a table and inserting an event in ClickHouse.

```
CREATE TABLE events
(
    `Timestamp` DateTime,
    `ServiceName` String,
    `SeverityText` String,
    `Message` String,
    `RequestId` String
)
ENGINE = MergeTree
PRIMARY KEY ( ServiceName, Timestamp)

INSERT INTO events (Timestamp, ServiceName, SeverityText, Message)
VALUES
    ('2025-09-05 10:15:00', 'nginx-d', 'info', 'User login successful');
```

After running this code, ClickHouse carries out a sequence of operations to transform the inserted rows into a durable, queryable data part:

Sorting

Rows are first ordered by the table's primary key. These should be the columns that define the natural ordering of the data. Sorting makes subsequent filtering and range scans efficient.

Index creation

As it sorts, ClickHouse builds a **sparse primary index**. This isn't a full index on every row; instead, it records the first key value at the start of each granule—a sorted block of rows, typically configured to span approximately 8,000 (see **Figure 14-1**). This makes the index small enough to keep in memory while still giving ClickHouse the ability to seek and skip large portions of irrelevant data.

Columnar storage and compression

Next, the values for each column are written separately in a columnar format, using the row ordering. Before being written, each column is compressed. Observability data often achieves compression ratios of 20:1 or more.

Data part finalization

The compressed column files, sparse index, and metadata are bundled together into the immutable part mentioned earlier. Parts are later merged in the background to form new, larger ones, but each individual part is immediately available for queries, enabling data that arrives late to be stored and queried effectively.

Data organization

ClickHouse can then store related parts in a group or folder called a **partition**. Parts are never merged across partitions, so it allows certain operations to act on the folder as a whole. They should be coarse-grained and based on a low cardinality column, as setting a fine-grained partition prevents ClickHouse from optimizing the parts.

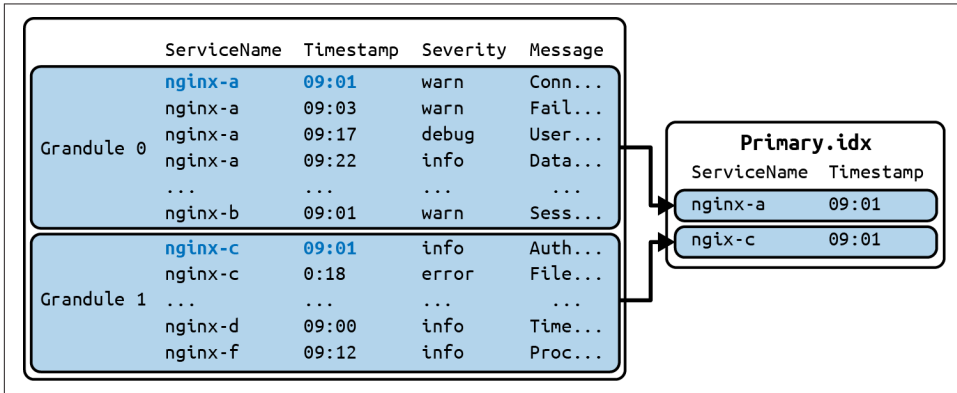


Figure 14-1. Rows are stored in granules, sorted according to the columns defined in the primary key.

Storing the sparse primary index of the start of each granule enables ClickHouse to optimize the speed of seeking to a specific position.

Query Execution

While creating tables and inserting data are essential steps, understanding how queries execute is fundamental to ClickHouse. After all, storing the data quickly does not matter unless you can query it quickly as well! Query execution defines how data is retrieved and processed, and mastering it enables users to optimize performance, debug complex workloads, and design efficient interfaces.

With the data from the previous example now loaded into our table, let's look at how to query and access it. The most common observability query is "Show me data for X service in the last N minutes." Let's see how ClickHouse handles such a query.

The query shown in [Example 14-2](#) retrieves 1,000 log events for the kube-system service between two timestamps, returning the Message field in chronological order.

Example 14-2. Example query for log event filtering.

```
SELECT Message
FROM events
WHERE Timestamp BETWEEN '2025-07-22 10:13' AND '2025-07-22 10:15'
      AND ServiceName = 'kube-system'
ORDER BY Timestamp ASC
LIMIT 1000
```

Here is an outline of the sequence of operations that occur when the query runs:

1. Partition pruning

First, ClickHouse tries to exclude entire partitions from the scanned data. For instance, if the example table was partitioned by `toStartOfDay(Timestamp)`, the database would immediately discard all partitions that don't overlap with 2025-07-22 10:13 to 2025-07-22 10:15.

2. Part pruning

ClickHouse then examines each data part's metadata. This metadata contains the minimum and maximum values for the primary key columns (`ServiceName`, `Timestamp`) within that part. If a part's `Timestamp` range does not overlap with 2025-07-22 10:13 to 2025-07-22 10:15, it is immediately discarded.

3. Index scanning and granule pruning

ClickHouse consults the sparse primary key indexes for the remaining parts. Because `ServiceName` sorts the data within the part and then the `Timestamp`, ClickHouse can perform a binary search on the index values to quickly identify the small range of candidate granules whose data might match.

4. Columnar scanning

After identifying the candidate granules, ClickHouse reads the `ServiceName` and `Timestamp` column data for those specific granules. A filtering pass is performed on this data to determine the exact rows that match the `WHERE` clause.

With the matching row offsets, the required granules from the `Message` column are fetched, decompressed, and read at the offsets—this will be the final output of our query. Note that, similar to Retriever as discussed in [Chapter 13](#), columns not considered in the query were never fetched, and the `Message` column was not read until the final set of rows was identified (also known as **lazy materialization**). This characteristic is an essential feature for efficient queries on wide events, where a majority of columns are not queried.

In the following sections, we explore the differences between structured and unstructured logs and how each impacts query performance within ClickHouse. We examine the advantages and trade-offs of both approaches, showing how users can balance structure and flexibility to achieve optimal results. We then discuss the importance of

primary key design for query efficiency in the context of observability, and offer an overview of the key caching mechanisms available in ClickHouse.

Structured logging

As discussed in [Chapter 5](#), transforming plaintext logs into structured events enables faster queries against the data, more efficient storage, and much more powerful aggregate query investigations.

A natural first step is to store structured events in an SQL database by creating columns for each significant attribute. For example:

```
ALTER TABLE events ADD COLUMN HttpMethod String;  
ALTER TABLE events ADD COLUMN HttpStatusCode UInt16;  
ALTER TABLE events ADD COLUMN HttpPath String;
```

With data stored like so, costly LIKE operations against a plaintext log string are instead accelerated by reading directly from the column, as with the query:

```
SELECT * FROM events  
WHERE Timestamp BETWEEN now() - INTERVAL 1 HOUR AND now()  
AND HttpStatusCode = 404;
```

As you saw earlier, storing data as an integer is more efficient than using an unstructured string. It uses fewer bytes on disk, making data faster to fetch, scan, and cache, while also enabling quicker comparisons and the option to add indexes for further acceleration.

However, this schema does not scale cleanly. Modern applications evolve rapidly, and new attributes—`customer.id`, `feature_flag.key`, `pod.name`, and countless others—are added frequently. Adding a new top-level column for every emerging field quickly becomes unsustainable, especially when many attributes are short-lived or only relevant to specific subsets of events. For fast-moving engineering teams, a rigid column-per-attribute schema creates friction rather than flexibility.

Balancing structure and flexibility with map

To address the scalability challenges of evolving attributes in observability data, ClickHouse offers powerful semi-structured data capabilities. You can maintain a few critical, top-level columns while storing all other attributes in a flexible **Map datatype**. This gives you the best of both worlds: performance for common attributes as concrete columns, and flexibility for arbitrary attributes in the Map type.

Let's enhance the table with a map for log attributes:

```
ALTER TABLE events ADD COLUMN LogAttributes Map(String, String) CODEC(ZSTD(1));
```

Now, your event-ingestion pipeline can parse the incoming event and populate the structured columns and the attributes map, regardless of whether the field names have been seen before.

This approach keeps your primary schema clean while preserving all contextual information.



The Map type has performance trade-offs. Accessing a key in a Map requires reading all keys and values, making it more input/output (I/O) intensive than querying a dedicated column. Map keys also cannot be part of the primary key. For these reasons, maps should be used sparingly, and replaced with more performant structure for your query load.

This hybrid model of structured columns and semi-structured maps is a cornerstone of building a robust and adaptable observability platform. As we've discussed, the next step is to optimize queries against this map data further, using skip indexes and materialized columns for attributes that become “hot” query targets over time.

The **JSON type** introduced to ClickHouse in 2025 aims to simplify observability use cases by accepting wide JSON (JavaScript Object Notation) events and automatically creating concrete columns for frequently occurring fields, while storing rare data in a separate blob structure with type hints. Before JSON, it was the observability platform maintainer's responsibility to actively manage this split between materialization of columns, where necessary. The JSON type also makes it ergonomic to manage deeply nested JSON within ClickHouse and overcomes the performance challenges of the Map type.

With an understanding of how ClickHouse stores and retrieves data, you can now examine how to design schemas that leverage these capabilities effectively. The early decisions around primary key selection, datatypes, and retention policies establish the foundation your observability platform will build upon.

Primary key design

The choice of primary key is fundamental to ClickHouse performance. The primary key determines the data's physical sorting order and the primary key index's structure, which enables fast query pruning. For observability workloads, this decision directly impacts how efficiently you can filter and aggregate telemetry data.

Consider common query patterns in your observability system:

- Do you typically filter by service first, then time? For example:
 - PRIMARY KEY (ServiceName, Timestamp)
- Are you more likely to search all services within a specific time window? For example:
 - PRIMARY KEY (Timestamp, ServiceName)

The primary key should align with your most frequent and performance-critical queries. As these examples illustrate, no single schema perfectly suits every use case. ClickHouse addresses this challenge by providing materialized views and projections that enable you to transform data and apply alternative primary keys during ingestion. We'll explore these capabilities further in [“Practical Data Modeling and Optimization” on page 255](#).

Caches

During an investigation, you often execute many searches over the same dataset while iteratively adjusting filters. You might start with a broad scan for errors and then narrow down to a handful of services or filter by specific dimensions. Without caching, each refined query would redundantly process data that ClickHouse has already scanned, wasting computational resources and slowing down your investigative workflow.

ClickHouse uses a multilayer caching system to dramatically accelerate query performance. Each cache type operates at a different level of the query execution pipeline, from storing complete query results to caching individual data blocks. In this section, we'll highlight four primary caching mechanisms, but there are [others](#) you can explore.

The [query cache](#) stores a full query result in the cache, using the [abstract syntax tree](#) of the query in order to match entries. It is advised that the query cache is enabled for specific, frequently run queries at the query level via a setting, to avoid thrashing the cache entries—this can occur if the capability is enabled for less frequently run queries.

The [query condition cache](#) (otherwise known as the predicate cache) stores information about granules scanned while queries are running. For example, if a query has a threshold `> 10`, ClickHouse will store a single bit for that condition on each granule scanned in memory, storing a 0 if the granule has no matching entries. This allows subsequent queries that use threshold `> 10` to skip scanning that granule, improving query performance.

The **filesystem cache** is primarily used when reads from a disk or object store are typically slow or expensive. It stores data on a faster disk to avoid making a call to the slower storage. It caches the raw data from a table, but also caches other data, such as table and granule metadata, or even full raw files for other formats like Parquet.

The **userspace page cache** is an in-memory cache. Like the filesystem cache, it can store data from either the tables or the metadata of the tables. However, unlike the filesystem cache, it stores all the information in memory. This has advantages over a pure memory-backed filesystem like tmpfs, as ClickHouse can manage memory proactively, rather than delegating memory management to the kernel.

Collecting Data

Having explored how ClickHouse stores, organizes, and queries observability data, we can now look at optimizing how that data arrives in the first place. In this section, we'll examine how to collect and insert telemetry efficiently using the OpenTelemetry Collector and the ClickHouse exporter. You'll learn how these components fit together to form a fast, reliable ingestion pipeline; how to tune insert performance and durability; and what alternative ingestion options exist for different architectural needs.

The OpenTelemetry Collector and ClickHouse Exporter

Although ClickHouse supports **many ways of ingesting data**, we will focus on getting up and running fast with existing telemetry pipelines. This means using OpenTelemetry Collector to ingest data. The **ClickHouse exporter** has general-purpose schematic defaults and well-documented settings for adjusting the exporter's performance.

By default, the ClickHouse exporter provides a solid baseline for inserting large amounts of data, without needing to tune the exporter, providing a ClickHouse schema that adheres to the **OTel Logs model** for small to mid-sized workloads.

You can maximize ClickHouse insert performance by doing fewer but larger insertion operations. Adding the **batch processor** is a good way to achieve this, assuming you are using a centralized OTel gateway.

If you're not using a central gateway, setting **async_insert=1** lets ClickHouse batch records on the server side using ClickHouse's **asynchronous inserts capability**. This allows multiple exporters—for instance, when OTel is run as a DaemonSet or side-car—to send small payloads without handling batching themselves, removing the need for an external agent or gateway and simplifying the architecture.

You can also control durability by choosing **wait_for_async_insert=1** for confirmed inserts with retries and backpressure, or **0** for fire-and-forget inserts that lower latency at the cost of durability.

ClickHouse's out-of-the-box idempotency also helps handle network failures. By default, the hash of each block's contents is retained in ClickHouse (for a configurable window), and exact matches of inserts are deduplicated and returned as successful. This enables clients to safely retry failed inserts.

Alternate Ingestion Approaches

Although the previous section focused on data ingestion via OpenTelemetry Collector, this is not the only observability pipeline option.

Your data pipeline will almost always have unique requirements. In the case of a highly durable pipeline, you could use a durable store such as Kafka or S3 to push messages, and then ingest the data via native ClickHouse engines such as the Kafka table engine or **S3Queue**. This provides a larger queue in the event of backpressure or planned downtime than a collector's local disk. In our experience, we don't need to use a durable pipeline for performance reasons, as our ClickHouse cluster comfortably keeps up with our load of tens of millions of inserts per second, but your requirements around resilience may differ.

Additionally, ClickHouse supports a wide variety of **input formats** and **language clients**, so it's simple to connect any bespoke sources written in your preferred language to your central ClickHouse store.

You can even connect to external databases like PostgreSQL via the table engine. Or you may have an external system that can produce JSON payloads via webhooks (for example, PagerDuty webhooks). ClickHouse's **HTTP interface** provides a way to capture that data and integrate it.

This reinforces the broader point: by treating observability data as just another data problem, you can seamlessly correlate observability signals with other observability and business datasets in ClickHouse for deeper context and richer understanding of issues.

Querying Data

With data now flowing into ClickHouse, the next step is understanding how to extract insight from it. In this section, we'll explore how to query observability data effectively—from simple searches and aggregations that reveal patterns in logs and metrics, to more advanced joins that connect disparate event streams in real time. These examples illustrate how ClickHouse transforms raw telemetry into actionable insights, whether you're investigating individual errors or correlating complex behaviors across systems.

Simple Queries

Let's start by looking at some very simple queries that represent a common first step in observability analysis. [Example 14-3](#) retrieves all rows from the last hour where the service name is equal to `nginx` and the log message begins with the keyword `exited`.

Example 14-3. A simple query.

```
SELECT *
FROM events
WHERE Timestamp > now() - INTERVAL 1 hour
      AND ServiceName = 'nginx'
      AND Message LIKE 'exited%'
```



You could also use HyperDX, which is an open source observability frontend built by ClickHouse, to compose this query using Lucene-like syntax (i.e., `ServiceName:nginx` and `Message:exited*`). See [“Visualizing Your Data” on page 272](#) for further details.

A slightly more advanced query can help us move beyond simple filtering to understanding performance patterns in our events. Supposing we wanted to identify which `nginx` endpoints are slowest under real traffic conditions. With a few aggregations, we can summarize latency across all requests in the last hour, highlight endpoints with meaningful load, and order them by their worst response times, shown in [Example 14-4](#).

Example 14-4. A more complex query.

```
SELECT
  Path,
  count() AS Requests,
  round(quantile(0.95)(RequestTime), 3) AS P95_s,
  round(quantile(0.99)(RequestTime), 3) AS P99_s
FROM events
WHERE ServiceName = 'nginx'
      AND Timestamp > now() - INTERVAL 1 HOUR
GROUP BY Path
HAVING Requests >= 100
ORDER BY P95_s DESC
LIMIT 20;
```

The results of this query (see [Table 14-1](#)) tell us three things at once:

Volume

We can see how many requests each endpoint served in the period.

Tail latency

The 95th and 99th percentile response times for each endpoint, aggregated at query time, are far more useful than simple averages when spotting performance issues.

Prioritization

By filtering to endpoints with at least 100 requests and ordering by worst P95, we focus attention on endpoints that matter to users.

Table 14-1. Results from the query in [Example 14-4](#)

Path	Requests	P95 (s)	P99 (s)
/api/v1/orders	40,723	1.034	1.78
/api/v1/users	23,161	0.699	1.08
/	56,093	0.555	0.895
/static/img	20,088	0.491	0.82

From this result set, we can already see a few insights:

- `/api/v1/orders` is the slowest endpoint by far, with a P95 above one second and a P99 close to two seconds, making it a prime candidate for investigation.
- `/api/v1/users` also shows elevated tail latency compared to lighter endpoints, though it is still substantially faster than `/api/v1/orders`.
- High-volume but lightweight endpoints such as `/` and `/static/img` are serving tens of thousands of requests with subsecond tail latencies—no immediate concern there.

This ability to surface the “pain points” at a glance is the key advantage. Instead of raw logs (too noisy) or averages (too smoothed), we see immediately where the user experience may suffer. This is better than a pre-aggregated metric; it is computed on demand, using precisely the dimensions and aggregations we needed. And because these numbers are derived from wide events, we can seamlessly zoom from these aggregated metrics into individual events.

Although this example was simple (showing only a `GROUP BY path`), you could imagine wanting to query both by path and by version—or by path and version and remote IP address—which no metric system could feasibly have a time series for without exceeding cardinality limits!

Complex Joins

For a more complex example of querying with ClickHouse, let’s consider how to connect arbitrary events with the **ASOF JOIN**.

For ClickHouse, observability isn’t just about looking at a single stream of events. Real systems produce many types of signals: application logs, infrastructure events, Kubernetes events, and business metrics. These streams often evolve independently. The challenge we see is not just collecting them, but connecting them when you need to answer a question.

Traditional tracing tries to solve this by enforcing relationships up front: fixed schemas, trace IDs, and correlation at the application layer. That works for predictable flows, but it breaks down when the relationships are ad hoc or when you need to investigate workflows that weren’t anticipated.

ClickHouse takes a different approach. Instead of defining links at ingest time, you can defer them until query time. Using JOIN operators, particularly the ASOF JOIN, you can connect events on the fly, even if they were never part of a single trace. ClickHouse JOINS can be used with standard columns, fields inside the Map type, or event columns derived from query time expressions (such as extracting a substring from a field).

Suppose we want to measure how long it takes a container to recover after a crash. We have two tables, shown in Tables 14-2 and 14-3.

Table 14-2. Table of events (as defined earlier)

Timestamp	ContainerId	Message
2025-08-13 10:00:00	nginx	exited abnormally with signal 9
2025-08-13 10:00:00	apiserver	exited abnormally with signal 9
...		

Table 14-3. kube_events table containing Kubernetes events

Timestamp	ContainerId	EventType
2025-08-13 10:16:01	nginx	ContainerReady
2025-08-13 11:16:03	apiserver	ContainerReady
...		

At a glance, it’s clear from Tables 14-2 and 14-3 that nginx went down at 10:00:00 and recovered at 10:16:01. But with real systems, you want to compute this systematically across all containers. That’s where the ASOF JOIN comes in. Unlike a regular JOIN, which looks for exact matches between tables, an ASOF JOIN can join across tables that contain an ordered sequence of events, connecting neighboring rows. This works

well for observability use cases, where events arrive at different cadences and you need to connect them—for example, matching a container failure to the first recovery event that follows.

Example 14-5. ASOF JOIN example.

```
SELECT
    failures.ContainerId as ContainerId,
    failures.Timestamp AS FailureTime,
    recoveries.Timestamp AS RecoveryTime,
    dateDiff(second, failures.Timestamp, recoveries.Timestamp) AS RecoverySeconds
FROM
    events AS failures
ASOF LEFT JOIN
    kube_events AS recoveries
    ON failures.ContainerId = recoveries.ContainerId
    AND failures.Timestamp < recoveries.Timestamp
WHERE
    failures.Message LIKE 'exited abnormally%'
    AND recoveries.EventType = 'ContainerReady'
ORDER BY
    failures.ContainerId,
    failures.Timestamp;
```

This produces rows like those shown in [Table 14-4](#).

Table 14-4. Resulting rows from the ASOF JOIN in [Example 14-5](#)

ContainerId	FailureTime	RecoveryTime	RecoverySeconds
nginx	2025-08-13 10:00:00	2025-08-13 10:16:01	961
apiserver	2025-08-13 10:15:00	2025-08-13 10:16:03	63

In plain language, the nginx container took about 16 minutes to recover, and the apiserver recovered in just over a minute.

With this query, we’ve essentially built a recovery-time metric *on demand*, directly from raw events.

Now, supposing we were concerned that a particular release version caused a regression in the mean time to repair (MTTR) for nginx, we might update the query to include the column `image_tag` from the table `kube_events`, adding `recoveries.image_tag` to our `SELECT`, and filtering with `AND container_id = 'nginx'`. Now we can calculate the recovery time for each image type.

By using JOINS across metric sources, we moved from raw event streams to actionable metrics with just a few lines of SQL. More importantly, we did this without predefining relationships or changing our ingestion pipeline, enabling us to refine and extend questions.

Scaling Vertically

As your observability platform matures and data volumes grow, the initial schema and infrastructure decisions that worked well at a smaller scale may need refinement. This section explores the iterative process of evolving your ClickHouse deployment to handle increasing load, changing query patterns, and new operational requirements.

At scale, performance bottlenecks in observability systems often come down to how data is stored—and more importantly, how it’s queried. Semi-structured fields like `Map(String, String)` or JSON are flexible, but querying them efficiently across trillions of rows requires users to perform some optimizations.

The key insight is that scaling isn’t just about adding more hardware—it’s about evolving your approach to data modeling, query optimization, and infrastructure management based on real-world usage patterns and performance data.

In this section, we begin by examining how to profile queries to identify bottlenecks. Then, we’ll explore data modeling and optimization techniques that improve performance within and across tables. Finally, we discuss lifecycle management strategies that help balance speed, cost, and retention as data volumes expand. Together, these topics provide a practical framework for scaling ClickHouse efficiently without adding unnecessary complexity.

Profiling Your Queries

As your observability system grows, optimization should be driven by systematic measurement and hypothesis-driven experimentation, not random guesswork. Profiling means examining how queries behave: what resources they use, how efficiently they access data, and where time is spent. These insights guide schema design and tuning far more effectively than intuition.

ClickHouse makes profiling straightforward with its internal system tables. Users interested in exploring these capabilities can find more details on [optimizing queries in the ClickHouse documentation](#).

Practical Data Modeling and Optimization

This section explores the various approaches that ClickHouse supports to better model your data at query time. Each approach has its own trade-offs around compute or storage at ingest time in exchange for faster queries.

These approaches are effectively split into two styles: optimization within a table and optimization across tables.

Within a table

The first set of strategies focuses on optimizing data retrieval and query performance within a single MergeTree table. These techniques refine the table's internal data structure to maximize efficiency against a fixed schema.

Skip indexes. **Skip indexes** are data structures that enrich a granule (a collection of consecutive rows) with statistical metadata. The goal of a skip index is to inform ClickHouse during query planning whether it should skip (hence the name!) scanning that granule or not. So, an ideal skip index should have clustered results within a granule. Thus, skip indexes will be most effective when applied to a column that correlates with the table's write order, as dictated by the primary key.

A positive result should only occur for a small number of granules out of the total set, thus ensuring the majority of granules are skipped. In other words, if your data is likely to have positive results across most granules, it's not worth adding a skip index. The extra skip index structures will bloat your table storage and can even degrade performance, as ClickHouse will read the skip index before determining that it will not be able to skip granules.

To illustrate this, the SQL shown in **Example 14-6** is adapted from the default OTel logs schema.

Example 14-6. Example skip indexes.

```
INDEX idx_request_id RequestId TYPE bloom_filter(0.001) GRANULARITY 1, ❶
INDEX idx_res_attr_key mapKeys(LogAttributes) TYPE bloom_filter(0.01)
  GRANULARITY 1, ❷
INDEX idx_res_attr_value mapValues(LogAttributes) TYPE bloom_filter(0.01)
  GRANULARITY 1, ❸
...
INDEX idx_message Message TYPE tokenbf_v1(32768, 3, 0) GRANULARITY 8 ❹
```

- ❶ It defines an index called `idx_request_id` on the column `RequestId`. The `bloom_filter(0.001)` specifies a false positive rate of 0.1%, which is useful for needle-in-a-haystack queries such as `RequestId` lookups. `GRANULARITY 1` indicates the index covers one granule (by default, 8,192 rows).
- ❷ It creates a Bloom filter index on the keys of the `LogAttributes` map, enabling efficient search through attribute names without scanning the full map structure.
- ❸ This is similar to the previous index, but it targets the values of the `LogAttributes` map, improving performance for queries filtering by attribute values.
- ❹ A `tokenbf_v1` Bloom filter is defined on a tokenized version of the `Message` column. For example, a message like “*The quick brown Jabberwocky*” is split into tokens (The, quick, brown, Jabberwocky), each inserted into the Bloom filter. `32768` sets the filter size in bytes, and `3` specifies the number of hash functions used.

All four skip indexes in [Example 14-6](#) are a variant of the **Bloom filter**. The concepts behind Bloom filters have multiple excellent [explanations](#), so we will focus on the specifics within ClickHouse. The [ClickHouse documentation](#) provides approaches to selecting and optimizing these parameters.

The net effect of this index is a simple text-based search on text tokens. The best performance is when applied to string columns that contain a large number of unique values and are queried for relatively rare terms (the more granules skipped, the better).

MinMax indexes are another type of skip index that store the minimum and maximum values for a specific column of each granule. They can be particularly useful in observability datasets when searching for anomalies such as unusually long spans. For example, in a table of trace spans with a `Duration` column, the index stores the minimum and maximum duration for each granule. When you query for outliers (e.g., spans where `Duration` exceeds a threshold), ClickHouse can decide with a single comparison whether an entire granule can be skipped, avoiding thousands of unnecessary row scans.

[Figure 14-2](#) shows an example of how a MinMax index filters the granules for a query, and thus reduces the data to be read.

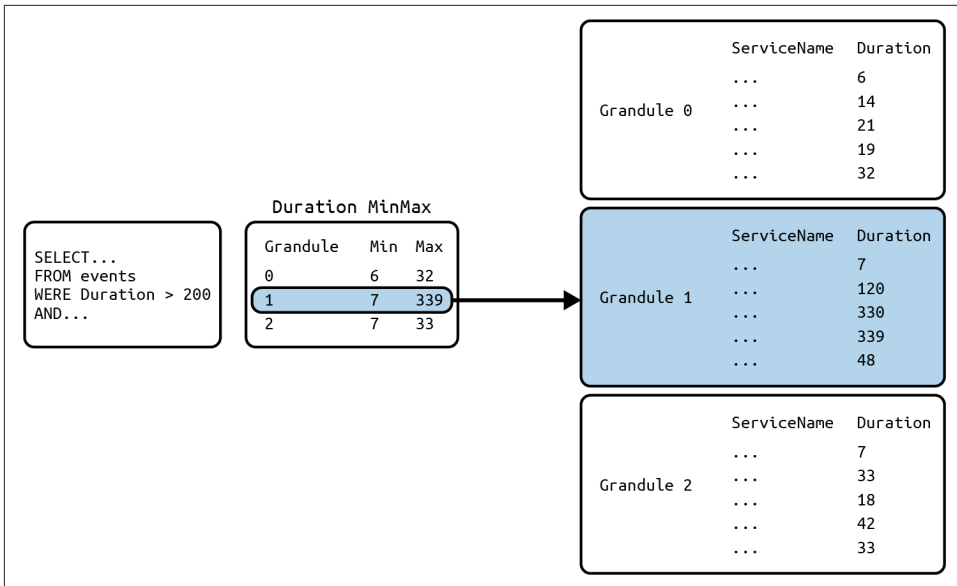


Figure 14-2. MinMax index filter example.



ClickHouse introduced **inverted indexes** on columns to accelerate full-text search. This feature extends the concept of skip indexes by pruning irrelevant granules for token-based queries using **inverted indexes**. The result is significantly faster log and text-search performance with minimal configuration overhead and deterministic results. This approach is similar to the use of Bleve-based full-text search indexes on Retriever.

Column materialization. A powerful feature for data modeling in ClickHouse is the ability to **materialize columns**. This means that a column, often derived from an existing one, is stored separately on disk and updated automatically. While some flexible datatypes, like the Map type, are excellent for initial ingestion of semi-structured data, filtering or querying against them can be slower at scale as they require runtime parsing of the entire map.

The practical solution is to materialize fields that users filter by regularly. Pulling high-value keys like `pod_name`, `region`, or `query_id` out of maps and into top-level columns allows the system to leverage ordering keys, compression, and skip indexes effectively.



One advantage of the JSON type is that it handles materialization automatically. During inserts, any new fields not already represented as columns are detected, and corresponding columns are created.

Because observability requires the ability to synthesize and aggregate data at read time—not just write time—we see convergent evolution, with calculated fields on a single row as a feature present in Retriever that is similar to column materialization in ClickHouse.

Projections. A **projection** is an embedded, query-optimized subset of columns for a MergeTree table that reorders (and optionally pre-aggregates) data with a different primary key index from the base table. Internally, ClickHouse maintains one or more hidden projection structures and automatically updates them during ingestion. This isn't a skip index, as the projection structure is a full copy of the columns it includes by default. When a query is run, ClickHouse transparently selects the projection (or the base table) that minimizes the data scanned, without requiring query rewriting or extra joins—thus optimizing query execution time.

Projections are especially useful when filtering or grouping on columns not included in the primary key. They improve performance by reducing I/O and computation at query time. Use projections when you need to support multiple query patterns with different dimensions (e.g., ordering by `Timestamp` or `ServiceName`) while keeping a single table for simpler queries and easier maintenance. Users query a single table while ClickHouse automatically routes their query to the optimal projection.

Projections add storage and ingestion overhead, because more data must be written, increasing I/O and memory usage. The additional cost of storage can be minimized if using object storage for long-term retention.

Across tables

The techniques mentioned in the previous section around optimizing a single table will take you a long way. Optimizing a single table can improve data layout or indexing, but its structure remains fixed. For example, the primary key is immutable, and avoiding projection costs may still leave limitations. As requirements grow, specialized tables often deliver better compression and query performance, and ClickHouse provides these benefits without complex pipelines or queries.

Materialized views (incremental). In ClickHouse, an **incremental materialized view** acts like an `AFTER INSERT` trigger on the source table. It captures newly inserted rows from the source table in its `SELECT` statement and writes transformed or aggregated results into a separate target table at insert time. This can be used to shift computa-

tion from query time to insert time by creating subsets of smaller data, either filtered or pre-aggregated.

Inserts to tables with a single materialized view are atomic, using the same deduplication logic as the rest of ClickHouse, minimizing issues with retrying inserts. The materialized view then persists only the transformed results, ensuring the original data isn't stored when it's not needed.

A table can have multiple materialized views, which can also be chained. This enables you to create simple streaming data pipelines—filtering, aggregating, or routing data to a specific table or even another materialized view. In the example shown in [Figure 14-3](#), a materialized view is responsible for sending a copy of the error logs from the wider dataset to a dedicated table. A second chained materialized view computes aggregates over the error logs, which can then be used for low-latency querying with minimal resources to power features such as alerting.

Queries against the materialized view are much faster because the data has already been precomputed, making it ideal for repeated access patterns or real-time analytics.

The advantage of using an incremental materialized view for this approach is twofold. First, it provides a consistent API for insertion; your source table defines the single data entry point, with its schema acting as the baseline of the metadata that your events must have.

Second is the ability to derive aggregations or apply filtering on event data during insert time. A concrete example is `SELECT`ing events that are considered business critical, such as errors in the [Figure 14-3](#) example, and routing them to another table for faster querying with a different retention configuration.

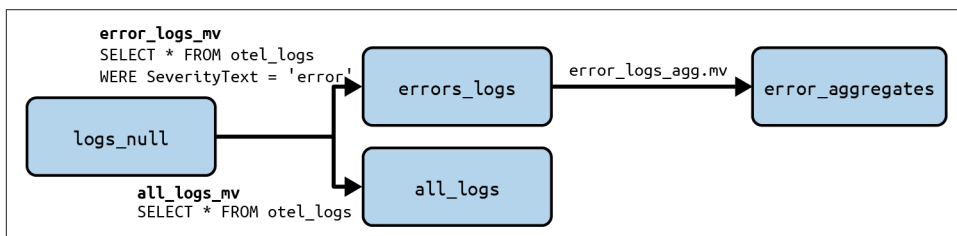


Figure 14-3. Illustration of materialized views filtering and aggregating data on insert. Rows land in `logs_null`, a view copies all events to `all_logs`, another filters errors into `error_logs`, and a chained view aggregates those errors into `error_aggregates` for fast queries.

Merge table. In many cases, the previously described techniques result in multiple tables, each containing different subsets of events. This can lead to situations where users need to run queries that span several tables simultaneously. To improve the ergonomics of querying across multiple tables, ClickHouse provides the [Merge table](#).

Although the name is deceptively close to the name of the MergeTree engine, this feature is conceptually different. It enables you to define a view over multiple underlying tables regardless of whether they differ in schema, partitioning, or sort order. To users, it still behaves like a single table, even though a Merge table may combine several schema variants accumulated over time, each tuned to the data shape when it was written.

A useful example of this is the system table `query_log`, where the old data is rotated to another table, `query_log_0`, `query_log_1`, etc., on ClickHouse version and schema updates. To SELECT across all of them, the following syntax can be used:

```
SELECT * from merge('system', '^query_log.*')
```

The regex matches all of the tables with `query_log`, and ClickHouse will transparently split the queries and will even automatically choose the minimum viable schema that returns results for all of the tables.

Using this approach enables you to evolve your ingestion strategy and schema over time without rewriting historical data or breaking queries, providing a stable API that engineers or tooling can use. For instance, this might cover a scenario in which a team changes from integers to enum datatypes for HTTP status codes, or promotes `user_id` from a map key to a full key.

The combination of materialized views and the Merge Table engine allows for splitting ingested data into dedicated tables, but querying across all of them with a unified view via the Merge table engine. In an example of convergent evolution, the Retriever datastore has also implemented a variant of this feature, as the COALESCE calculated field that can select from any extant fields.

As an example, [Figure 14-4](#) illustrates how logs inserted into the `logs_null` table can be filtered into two separate tables—one containing all nonerror logs and another storing only error logs. Users may then query across both tables using the Merge table function to create a unified view of all logs.

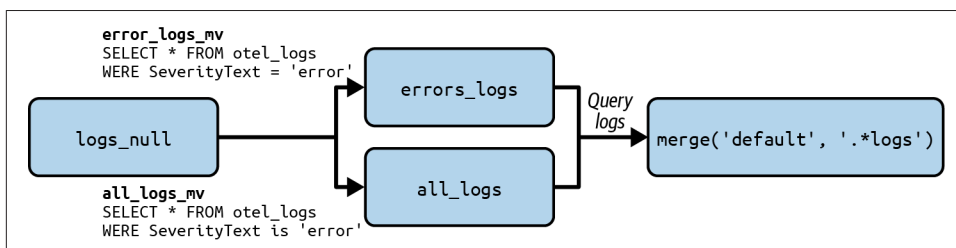


Figure 14-4. Example of how the Merge function can be used to return combined results from two separate tables.

Whether improving query efficiency within a single table through skip indexes and projections or structuring data across multiple tables with materialized views and Merge tables, each approach enables better alignment between schema design and real-world query patterns. By continuously evolving your data model as workloads grow and change, you can maintain fast, predictable performance without increasing operational complexity—laying the groundwork for the next stage of scaling your observability platform.

Data Lifecycle Management

Observability data follows predictable access patterns: recent data is frequently queried for real-time debugging and alerting, while historical data is accessed less often for trend analysis and forensics. This natural lifecycle makes tiered storage an ideal cost-optimization strategy.

ClickHouse's **time to live (TTL) policies** enable automatic data movement between **storage tiers** based on age. We recommend using solid state drive (SSD) storage—whether instance attached or persistent volume—for your hot tier, where data is initially written, before moving it to an object storage tier (S3 or Google Cloud Storage, for instance) based on time or disk-fill percentages.

The time period here would typically be based on your access patterns and the period over which most queries occur. In contrast with Retriever, which attempts to eagerly offload data to S3 to expand the set of workers that can potentially read the data in parallel, ClickHouse (in this deployment pattern) aims to keep data locally for as long as it's likely to be queried on that host. Data can then transition through progressively cheaper storage classes. This idea is captured by **Figure 14-5**, where data in NVMe (non-volatile memory express) is queried more frequently than that in spinning disk or even object storage.

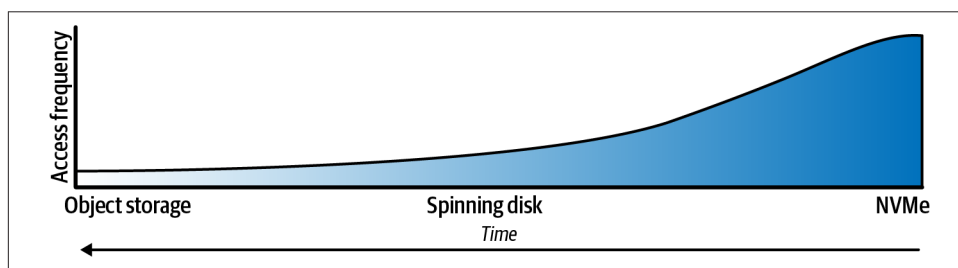


Figure 14-5. Data in NVMe is accessed more frequently than that in spinning disk or object storage.

The query interface remains unchanged, with ClickHouse fetching data from the appropriate tier as needed. ClickHouse supports most object stores (such as S3, Google Cloud Storage, MinIO, and Azure Blob Storage), each with options for reduced-cost archival storage.

The following SQL statement modifies the events table specifying several TTLs—data older than 90 days is moved to the disk `s3_ia_disk`, with events older than a year deleted:

```
ALTER TABLE events
MODIFY TTL
    Timestamp + INTERVAL 90 DAY TO DISK 's3_ia_disk',
    Timestamp + INTERVAL 365 DAY DELETE
```

TTL policies piggyback on top of background merges to adjust the data location. At high volume, this can be expensive, requiring ClickHouse to scan lots of data in order to determine which rows to drop or move. By adding a `SETTING` of `ttl_only_drop_parts=1`, ClickHouse can efficiently drop aged-out data by only dropping the full part, rather than trying to remove row by row. This still relies on your data being clustered by the TTL condition. If your primary key begins with a time column, this is already covered, with alternative techniques only needed if this isn't the case.

An alternative approach is using a `PARTITION BY` on a low granularity timeframe matching your TTL, such as `toStartOfWeek(Timestamp)`. ClickHouse will group parts by that time range, even if the timestamp is not part of the overall sorting key, enabling TTLs to run quickly without putting significant load on ClickHouse.

TTL policies have even more powerful features for longer-term historical queries. TTLs can be selectively applied with `WHERE` clauses, letting you choose to keep high-signal data for longer (for example, aging out non-error events). They can be used on a column-by-column basis, or `GROUP BY` clauses can aggregate or downsample the data by only storing a summary.

In summary, scaling within a single ClickHouse instance is about precision, not just power. By profiling queries, refining data models, and applying targeted optimizations, you can sustain low-latency performance and cost efficiency even as your observability workloads grow by orders of magnitude.

Scaling Horizontally

Scaling horizontally builds on the principles covered earlier by extending the ClickHouse architecture across multiple nodes and regions. In this section, we examine how replication improves durability and availability, how sharding enables horizontal scalability, and how SharedMergeTree brings cloud native elasticity through separation of storage and compute. Together, these capabilities enable ClickHouse to scale seamlessly from a single powerful server to globally distributed observability deployments.

Modern servers are powerful, and ClickHouse is designed to fully exploit their capabilities and resources. A single well-provisioned host (CPU, RAM, NVMe SSDs) can handle terabytes of data with high ingest and query rates.

Observability workloads, however, can outgrow a single host. When data size, request rates, or processing demands exceed one server—or when high availability and failure isolation are required—scaling out becomes necessary: shard for capacity and replicate for high availability.



Where possible, ClickHouse users should scale vertically before considering sharding to scale horizontally. Unlike other systems constrained by Java Virtual Machine or architectural limits, ClickHouse can fully utilize modern hardware. Vertical scaling additionally minimizes the use of the slowest part of any database: the network.

Horizontally scaling should also not be confused with high availability or cross-region redundancy. This can be achieved with ClickHouse through replication.

Replication

To improve availability and durability, ClickHouse supports **data replication**. In a sense, replication is orthogonal to sharding: each shard can be replicated to multiple nodes for fault tolerance. MergeTree tables can use the **ReplicatedMergeTree engine**, which relies on Keeper—a Raft-based coordination service replacement for Apache ZooKeeper to maintain a configurable number of replicas for each shard. On startup, a ClickHouse binary can start as either a server or a keeper.

Replication in ClickHouse is based on table states—the combination of table parts and metadata (e.g., column names and types). As an active-active replication model, any node can advance the state of a table using any of the following three operations:

Inserts

Add new parts.

Merges

Create new parts while removing old ones.

Mutations/DDL (data definition language) statements

Add or remove parts and/or modify metadata (e.g., add column).

Each operation is performed locally on a node and recorded in Keeper as a state transition in a global replication log.

A typical three-node Keeper ensemble maintains the replication log. All replicas start from the same log position. As nodes execute local inserts, merges, mutations, or DDL statements, the log is asynchronously replayed to other replicas. This eventually makes replicated tables consistent—queries may briefly return stale data until replicas converge. For most observability data, eventual consistency is good enough, but when stronger guarantees are required, ClickHouse supports quorum writes through the `insert_quorum` session-level setting. When enabled, an INSERT succeeds only if the data is written to the configured number of replicas. If quorum is not reached, the write fails and the inserted block is removed from all replicas.

Let's look at an example of a replicated table in a cluster of three ClickHouse nodes. The following steps illustrate how inserted data is replicated across multiple nodes within a ClickHouse cluster:

1. Initially, the replicated table is empty (see [Figure 14-6](#)). Then, Node 1 receives two insert statements, creating two parts (see [Figure 14-7](#)): Part A and Part B. It records the metadata in the replication log, which is stored in the Keeper.

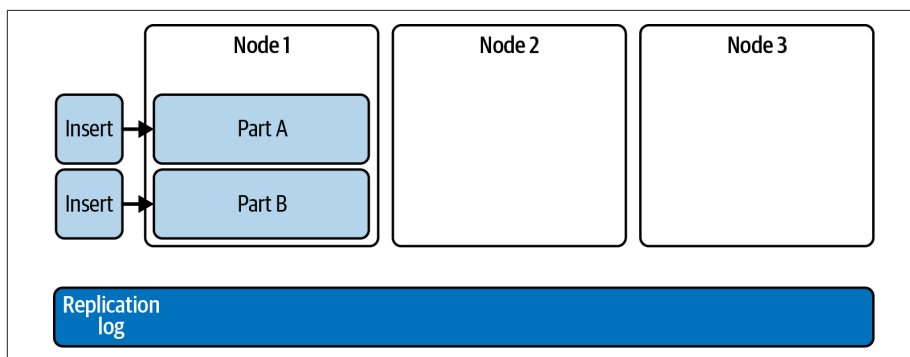


Figure 14-6. Two parts, A and B, are created in Node 1.

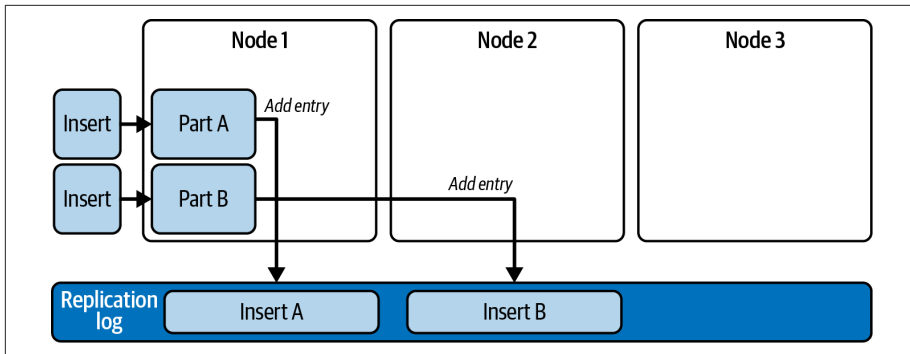


Figure 14-7. An entry is added to the replication log for both Parts A and B.

2. Next, Node 2 replays the first log entry by fetching it, and downloads Part A from Node 1 (see Figure 14-8).

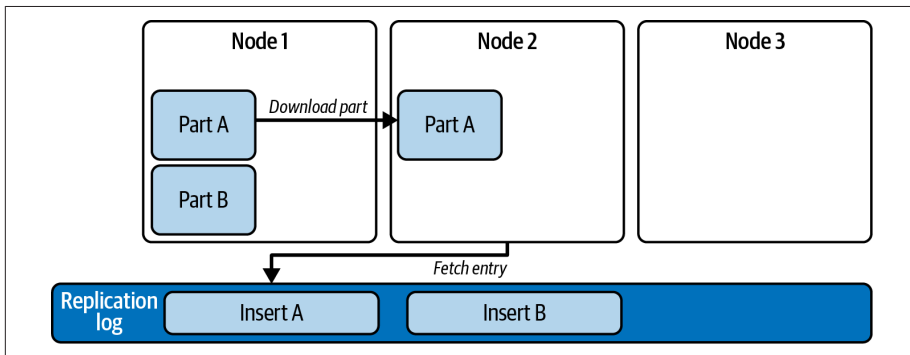


Figure 14-8. Node 2 downloads Part A from Node 1.

3. Node 3 replays both log entries and downloads both Part A and Part B from Node 1 (see Figure 14-9).

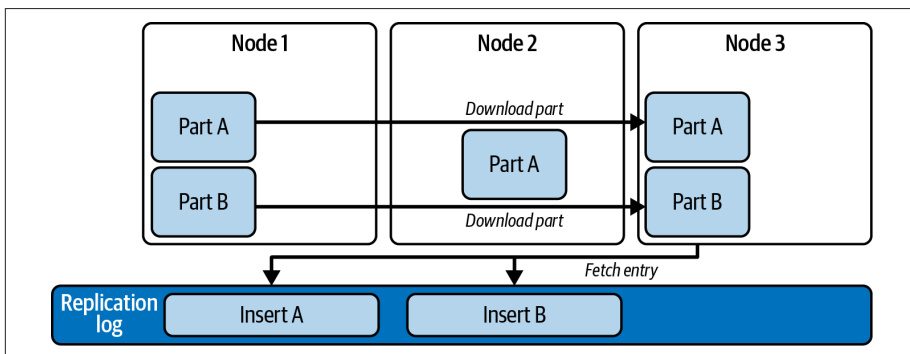


Figure 14-9. Node 3 downloads Part A and B from Node 1.

- Then, Node 3 merges Part A and Part B to Part C, deletes the input parts, and records a merge entry to the replication log (see [Figure 14-10](#)).

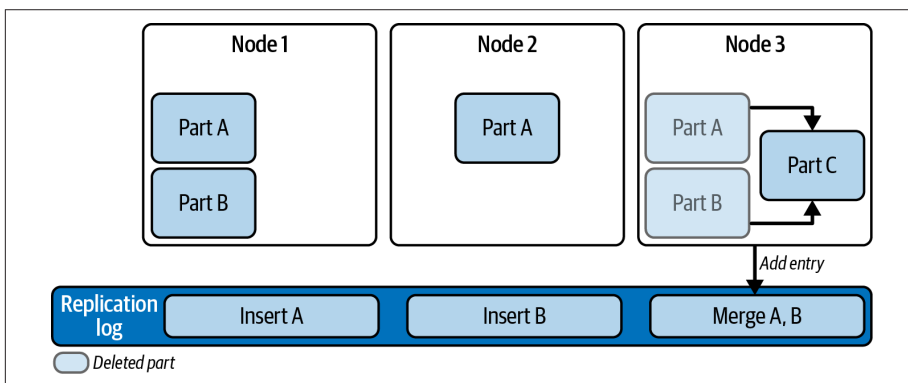


Figure 14-10. Node 3 merges Part A and Part B to Part C.

- Node 2 catches up and downloads Part B from Node 1 (see [Figure 14-11](#)).

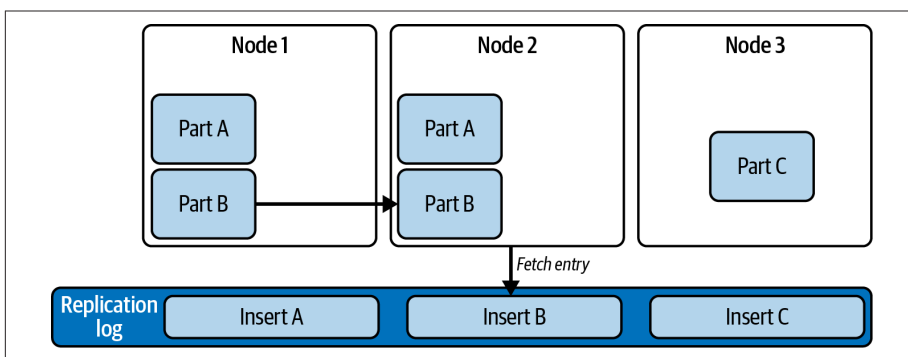


Figure 14-11. Node 2 downloads Part B from Node 1.

- When Node 1 and Node 2 get to the Merge record in the replication log, they will download Part C from Node 3 or perform the merge themselves, depending on the cluster settings. Afterwards, Nodes 1 and 2 will delete Parts A and B (see [Figure 14-12](#)).

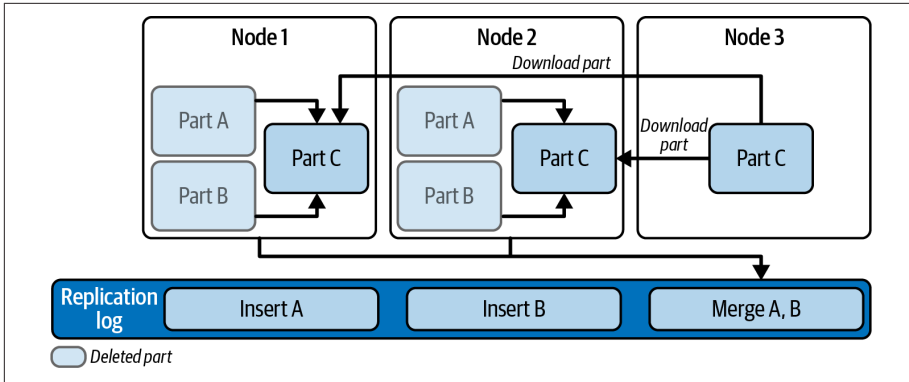


Figure 14-12. Node 1 and Node 2 download Part C from Node 3.



ClickHouse also supports **backups**, including support for object storage. This can be used to restore a system from a cold start—where no data exists in the cluster.

Replication protects against hardware failures and enables automatic failover. It can also boost query performance. With the **parallel replicas** feature, ClickHouse automatically splits the query and fans the chunks out to all replicas to process in parallel.

Sharding

When the capacity of one host is exceeded, you should add a host and split a table's data into two (or more) **shards** to horizontally scale. ClickHouse uses a shared-nothing approach: shard hosts don't coordinate, and each shard's table is independent. **Figure 14-13** shows a simple example of a ClickHouse cluster with two shards.

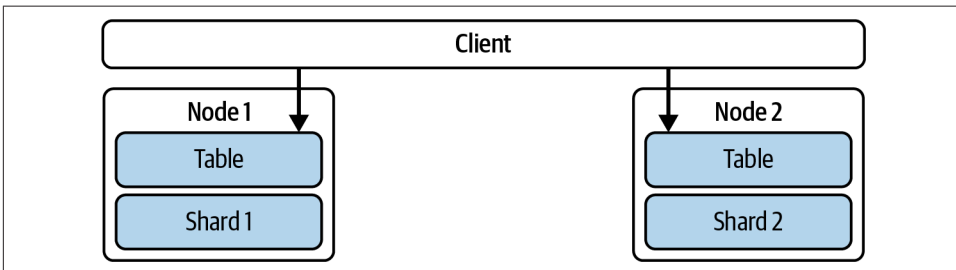


Figure 14-13. ClickHouse cluster with two nodes and two shards.

In this schema, writes are fully independent, and the application that's inserting the data decides which shard to use, as shown in [Figure 14-14](#). It's a very flexible approach that supports any sharding schema.

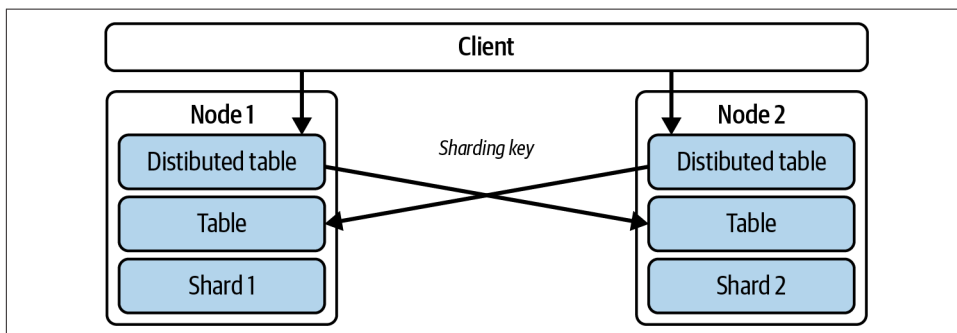


Figure 14-14. ClickHouse cluster with two nodes and two shards, with a Distributed table and a sharding key to distribute reads and writes.

For convenience, you can create a **Distributed table**. It doesn't store data itself, but acts as a “view” or a “proxy” over the local and remote tables. The inserted data can be automatically distributed among the cluster's shards based on a specified “sharding key” derived from data (e.g., `ServiceName` or `Region`) that determines which shard a particular row will be stored on. The sharding key does not have to be related to the primary key. Alternatively, ClickHouse chooses a shard randomly on every insert for even data distribution across the cluster nodes.

In this example, there is a Distributed table configured across both shards so clients can connect to any node and insert or select data. When inserting the data, ClickHouse evaluates the sharding key for each row, determines where this row should be stored, and routes the insert to this specific shard. When selecting the data, the Distributed table forwards the query to all servers hosting shards of the target table, where each ClickHouse server computes its local aggregation result in parallel. Then, the ClickHouse server hosting the initially targeted Distributed table collects all local results, merges them into the final global result, and returns to the query sender.

Single-Region Observability Cluster

A simple ClickHouse cluster for observability workloads typically consists of two ClickHouse nodes using `ReplicatedMergeTree` tables and a Keeper ensemble with three replicas. For reliability, it's best to host Keeper on separate nodes. Writes can be load-balanced across the ClickHouse nodes to improve throughput, as shown in [Figure 14-15](#).

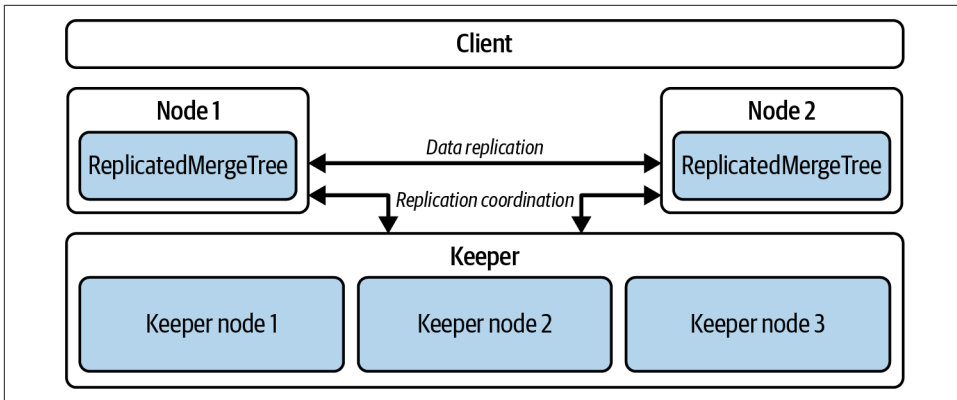


Figure 14-15. ClickHouse cluster using *ReplicatedMergeTree* for multinode replication.

On modern hardware, a single ClickHouse server with 32 CPU cores and 128 GB RAM can ingest around 20 billion events per day—peaking at roughly 300,000–400,000 events per second. With proper load balancing between replicas, that can reach 500,000 events per second. Adding more nodes will scale ingest bandwidth almost linearly. Thanks to strong compression within ClickHouse, observability data often shrinks by 10–20 times. For perspective, a single day’s raw volume of 20 terabytes (TB) can compress down to just 1–2 TB on disk. Consider two servers, each with 7.5 TB of locally attached NVMe disk in a high-availability architecture (i.e., each server storing a complete copy of the data): these could store at least three days of data before object storage was required for longer-term retention.

Query performance in ClickHouse depends heavily on query patterns. Within this architecture, applying best practices such as efficient schema design, appropriate partitioning, and effective indexing makes it possible to achieve subsecond response times on typical observability workloads, even at a large scale.

Multiregion Observability Cluster

In a multiregion deployment each region handles its own ingestion. Writes are directed to local ClickHouse nodes, which minimizes latency, improves reliability, and reduces costs. A Distributed table sits on top, as shown in [Figure 14-16](#), using a Region column as the sharding key—primarily for routing queries to the right region—while all inserts remain local.

Intraregion scaling is achieved by adding shards and replicas within a region. This increases both ingestion bandwidth and query parallelism, while replicas improve fault tolerance.

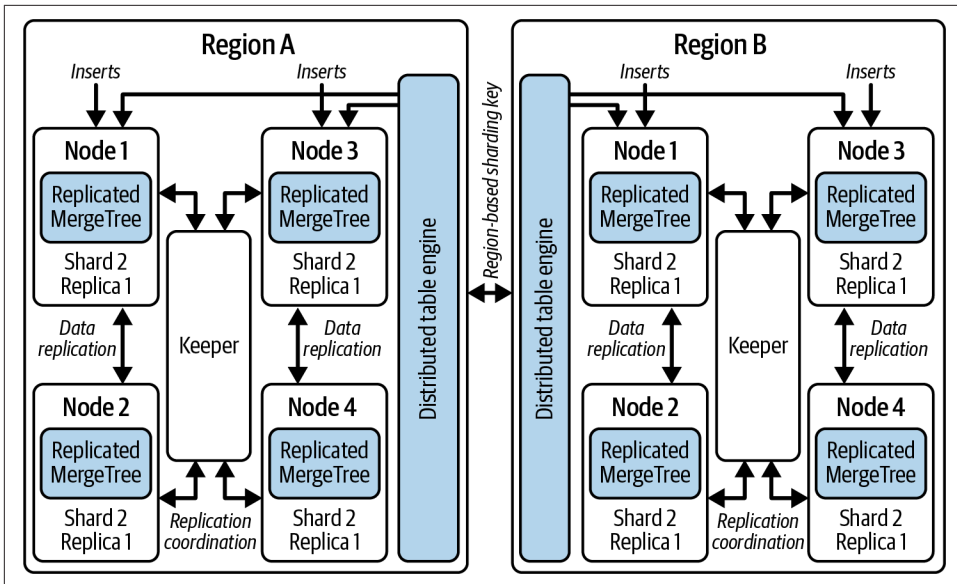


Figure 14-16. Multiregion observability cluster using *Distributed table engines* and *ReplicatedMergeTree*.

Cross-region scaling is straightforward configuration and provisioning: adding a new region is as simple as extending the *Distributed table* definition with another **shard** keyed by the *Region* column. Queries automatically become aware of the new region, enabling federated analytics across all regions or scoped queries to a single region for efficiency.

Geo-distributed fault isolation comes naturally with this design. If one region becomes unavailable, queries can continue to run against the remaining regions. Data remains isolated within its home region for ingestion, but the global *Distributed table* provides a unified view, ensuring both resilience and availability across the deployment.

SharedMergeTree



The ClickHouse *SharedMergeTree* and the Honeycomb *Retriever* both provide a way to separate storage from compute for high-volume analytics. They share a similar architectural idea of letting stateless query nodes read from a distributed object store rather than relying solely on local disks. However, neither *Retriever* nor *SharedMergeTree* are required to get a basic observability backend operating.

The **SharedMergeTree table engine family** is a proprietary cloud native replacement for ReplicatedMergeTree, optimized for shared object storage (e.g., AWS S3, Google Cloud Storage, Azure Blob Storage, MinIO). It powers ClickHouse Cloud and offers several advantages over ReplicatedMergeTree:

There is no need for data sharding.

All data is backed by object storage using a single shard. Data is not replicated between the nodes; instead, they become aware of the state of the single shard via Keeper.

There is a deeper separation of storage and compute.

Compute can be scaled horizontally, as well as vertically, with multiple nodes reading from the same single shard in parallel. In this case, replicas do not hold a copy of the data and rather just provide additional “replica compute” over the data.

Query execution can be shared across nodes.

And each node completes a subset of the required work. A higher rate of insert throughput can be sustained across ingest workers.

Quicker scale-up and scale-down operations.

This is because data is not replicated. It also enables scale to zero, where a replica is awakened just in time.

Lightweight, strong consistency for SELECT queries.

Storage costs become lower due to the use of object storage rather than more expensive NVMe drives.

Unlike ReplicatedMergeTree, replicas in SharedMergeTree don’t communicate directly. Instead, they coordinate via shared storage and Keeper. Keeper ensures nodes remain synchronized by replicating the global state modified by inserts, updates, and merges. Through Keeper, each node is aware of the latest data and maintains an up-to-date view of the single copy. Because nodes themselves don’t maintain state, scaling up or down is faster, with merges and mutations having a lower overhead. SharedMergeTree supports hundreds of replicas per table, enabling elastic scaling without shards. ClickHouse Cloud uses distributed query execution to leverage additional compute resources for a single query.

We believe that SharedMergeTree is a well-suited choice for scaling out most observability workloads, as long as you are able to use a proprietary service. It offers higher insert throughput and lower storage costs—effectively enabling retention to become a nonissue.

Visualizing Your Data

With all of our telemetry in ClickHouse, the most critical question is: how do we make these capabilities more accessible to SREs and developers?

As ClickHouse supports standard SQL, users can build their own observability tooling on top to suit their needs. This can be useful for teams operating at extreme scale or with highly specialized workflows, though most users simply need a familiar interface that works out of the box, such as that offered by HyperDX and illustrated in Figure 14-17.

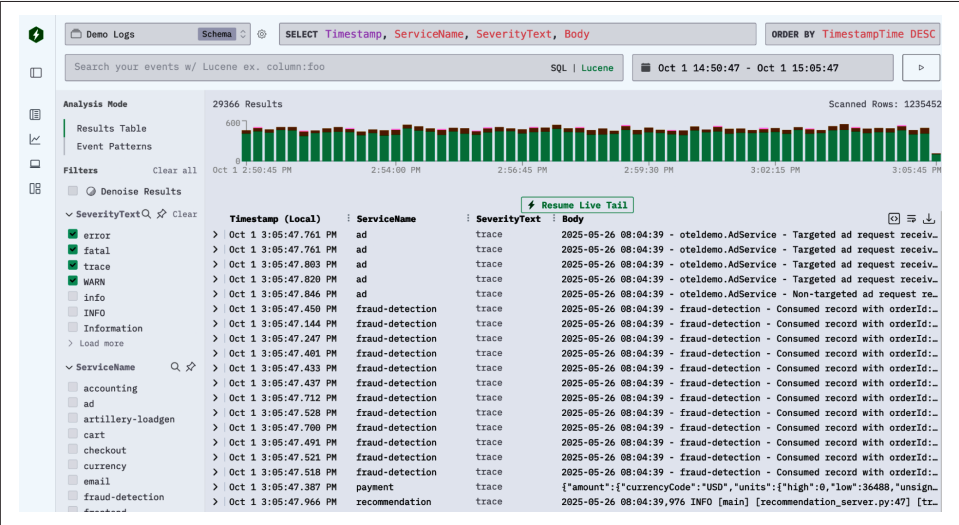


Figure 14-17. HyperDX open source interface for searching logs and traces.

For most teams, **HyperDX**—an MIT-licensed, open source visualization layer built natively for ClickHouse—provides an intuitive way to explore observability data. It offers advanced filtering, timeline views, and real-time dashboards, along with a Lucene-style query syntax that makes complex searches simpler than SQL. This didn't exist when we were building our own UI, but had it been available, we would have built on it without hesitation.

Using ClickHouse for Observability Workloads

Modern observability stacks need a datastore that can absorb huge volumes, adapt to changing telemetry, and return answers quickly. ClickHouse meets these needs with its columnar architecture, MergeTree engine, and flexible schemas. This supports efficient ingestion, smooth evolution, advanced indexing, and scalable deployments.

Paired with OTel and tools like HyperDX, it gives teams a fast, cost-efficient, and open foundation for building their own observability platforms.

Today, many teams use ClickHouse to power custom observability systems, from commercial vendors to large self-managed deployments. Its performance, scalability, and open design provide a strong alternative to proprietary software as a service (SaaS) offerings.

Conclusion

We're delighted to share this content from the ClickHouse team showing an alternative datastore implementation to contrast against the previous chapter's detailed coverage of the Honeycomb Retriever engine. ClickHouse is a robust and powerful datastore for teams looking to build their own observability solutions. Datastores for observability workloads should be designed to enable fast analysis of very large datasets, and this chapter gives you the basis to start your own customized build.

In the next chapter, we'll examine a different aspect of managing telemetry data volumes by considering sampling. At scale, a small trickle of observability data can turn into a torrential flood. Effective sampling strategies can help you balance resources and even reduce the load on the observability backend you decide to use.

Cheap and Accurate Enough Sampling

In the preceding two chapters, we examined datastore configurations and implementations that can efficiently store and retrieve large quantities of observability data. In this chapter, we'll look at techniques for reducing the amount of observability data you may need to store.

At a large enough scale, the resources necessary to retain and process every single event can become prohibitive and impractical. Sampling events can mitigate the trade-offs between resource consumption and data fidelity. This chapter examines why sampling is useful (even at a smaller scale), the various strategies typically used to sample data, and trade-offs between those strategies.

In this chapter, we use code-based examples to illustrate how these strategies are implemented, and progressively introduce concepts that build upon previous examples. We'll start with simpler sampling schemes applied to single events as a conceptual introduction to using a statistical representation of data when sampling. Then, we build toward more complex sampling strategies as they are applied to a series of related events (trace spans), and propagate the information needed to reconstruct your data after sampling.

Sampling to Refine Your Data Collection

Past a certain point, the cost to collect, process, and save every log entry, every event, and every trace that your systems generate dramatically outweighs the benefits. At a large enough scale, it is simply not feasible to run an observability infrastructure that is the same size as your production infrastructure. When the typical trickle of observability events instead quickly becomes a flood of data, the challenge pivots to managing a trade-off between scaling back the amount of data that's kept and

retaining the crucial information your engineering team needs to troubleshoot and understand your system's production behaviors.

The reality of most applications is that many of their emitted events are virtually identical and successful. The core function of debugging is searching for emergent patterns or examining failed events during an outage. Through that lens, it is then wasteful to transmit 100% of all events to your observability data backend. Instead, certain events can be selected as representative examples of what occurred, and those samples can be transmitted along with metadata that your observability backend needs to reconstruct what actually occurred among the events that weren't sampled.

To debug effectively, what's needed is a representative sample of successful, or "good," events, against which to compare the "bad" events. Using representative events to reconstruct your observability data enables you to reduce the overhead of transmitting every single event, while also faithfully recovering the original shape of that data. Sampling events can help you accomplish your observability goals at a fraction of the resource cost. It's a way to refine the observability process at scale.

Historically in the software industry, when facing resource constraints in reporting high-volume system state, the standard approach to surfacing the signal from the noise has been to generate aggregated metrics containing a limited number of tags. As covered in [Chapter 5](#), aggregated views of system state that cannot be decomposed are far too coarse to troubleshoot the needs of most modern systems. Pre-aggregating data before it arrives in your debugging tool means that you can't dig further past the granularity of the aggregated values.

With a high degree of observability, you can sample events by using the strategies outlined in this chapter and still provide granular visibility into system state. Sampling gives you the ability to decide which events are useful to transmit and which ones are not. Unlike pre-aggregated metrics that collapse all events into one coarse representation of system state over a given period of time, sampling allows you to make informed decisions about which events can help you surface unusual behavior, while still optimizing for resource constraints. The difference between optimization via sampled events or aggregated metrics is that full context is preserved on each dimension included in the representative event when sampling.

At scale, the need to refine your dataset to optimize for resource costs becomes critical. But even at a smaller scale—where the need to shave resources is less pressing—refining the data you decide to keep can still provide valuable cost savings and cognitive load reductions. First, let's start by looking at the strategies that can be used to decide which data is worth sampling. Then, we'll look at when and how that decision can be made when handling trace events.

Using Different Approaches to Sampling

Sampling is a common approach when solving for resource constraints by collecting only a fraction of available telemetry. Unfortunately, the term *sampling* is used broadly as a one-size-fits-all label for the many types of approaches that might be implemented. Many concepts and implementations of sampling exist—some less effective for debugging than others. We need to disambiguate the term by naming each sampling technique that is appropriate for observability and distinguishing how they are different, and similar, to one another.

Constant-Probability Sampling

Because of its understandable and easy-to-implement approach, *constant-probability sampling* is what most people think of when they think of sampling: a constant percentage of data is kept rather than discarded (e.g., keep 1 out of every 10 requests).

When performing an analysis of sampled data, you will need to transform the data to reconstruct the original distribution of requests. Suppose that your service is instrumented with both events and metrics, and receives 100,000 requests. It is misleading for your telemetry systems to report receiving only 100 events if each received event represents approximately 1,000 similar events. Other telemetry systems such as metrics will increment a counter for each of the 100,000 requests that your service has received. Only a fraction of those requests will have been sampled, so your system will need to adjust the aggregation of events to return data that is approximately correct. For a fixed sampling rate system, you can multiply each event by the sampling rate in effect to get the estimated count of total requests and sum of their latency. Scalar distribution properties such as the P99 and median don't need to be adjusted for a constant probability sampling, as they are not distorted by the sampling process.

The basic idea of constant sampling is that, if you have enough volume, any error that comes up will happen again. If that error is happening enough to matter, you'll see it. However, if you have a moderate volume of data, constant sampling does not maintain the statistical likelihood that you still see what you need to see. Constant sampling is not effective in the following circumstances:

- You care a lot about error cases and not very much about success cases.
- Some customers send orders of magnitude with more traffic than others, and you want all customers to have a good experience.
- You want to ensure that a huge increase in traffic on your servers won't overwhelm your analytics backend.

For a highly observable system, a more sophisticated approach ensures that enough telemetry data is captured and retained so that you can see into the true state of any given service at any given time.

Sampling on Recent Traffic Volume

Instead of using a fixed probability, you can dynamically adjust the rate at which your system samples events. Sampling probability can be adjusted upward if less total traffic was received recently, or decreased if a traffic surge is likely to overwhelm the backend of your observability tool. However, this approach adds a new layer of complexity: without a constant sampling rate, you can no longer multiply out each event by a constant factor when reconstructing the distribution of your data.

Instead, your telemetry system will need to use a weighted algorithm that accounts for the sampling probability in effect at the time the event was collected. If one event represents 1,000 similar events, it should not be directly averaged with another event that represents 100 similar events. Thus, for calculating a count, your system must add together the number of *represented* events, not just the number of collected events. For aggregating distribution properties such as the median or P99, your system must expand each event into many when calculating the total number of events and where the percentile values lie.

For example, suppose you have pairs of values and sample rates: {1,5}, {3,2}, {7,9}. If you took the median of the values without taking the sample rate into account, you would naively get the value of 3 as a result—because it's at the center of [1,3,7]. However, you must take the sample rate into account. Using the sample rate to reconstruct the entire set of values can be illustrated by expanding out the entire set of values in long form: [1,1,1,1,1,3,3,7,7,7,7,7,7,7,7,7]. In that view, it becomes clear that the median of the values is 7 after accounting for sampling.

Sampling Based on Event Content (Keys)

A dynamic sampling approach involves tuning the sample rate based on event payload. At a high level, that means choosing one or more fields in the set of events and designating a sample rate when a certain combination of values is seen. For example, you could partition events based on HTTP response codes and then assign sample rates to each response code. This enables you to specify sampling conditions such as:

- Events with *errors* are more important than those with *successes*.
- Events for *newly placed orders* are more important than those checking on *order status*.
- Events affecting *paying customers* are more important to keep than those for customers using the *free tier*.

With this approach, you can make the keys as simple (e.g., HTTP method) or complicated (e.g., concatenating the HTTP method, request size, and user-agent) as needed to select samples that can provide the most useful view into service traffic.

Sampling at a constant percentage rate based on event content alone works well when the key space is small (e.g., there are a finite number of HTTP methods: GET, POST, HEAD, etc.) *and* when the relative rates of given keys stay consistent (e.g., when you can assume errors are less frequent than successes). It's worth noting that sometimes that assumption can be wrong, or it can be reversed in bad cases. For those situations, you should validate your assumptions and have a plan for dealing with event traffic spikes if the assumed conditions reverse.

Combining per Key and Historical Methods

When the content of traffic is harder to predict, you can instead continue to identify a key (or combination of keys) for each incoming event, and then *also* dynamically adjust the sample rate for each key based on the volume of traffic recently seen for that key. For example, base the sample rate on the number of times a given key combination (such as `[customer_id, dataset_id, error_code]`) is seen in the last 30 seconds.

If a specific combination is seen many times in that period, it's less interesting than combinations that are seen less often. A configuration like that allows proportionally fewer of the events to be propagated verbatim until that rate of traffic changes and it adjusts the sample rate again.

Choosing Dynamic Sampling Options

To decide which sampling strategy to use, it helps to look at the traffic flowing through a service, as well as the variety of queries hitting that service. Are you dealing with a front-page app, and 90% of the requests hitting it are nearly indistinguishable from one another? The needs of that situation will differ substantially from dealing with a proxy fronting a database, where many query patterns are repeated.

A backend behind a read-through cache, where each request is mostly unique (with the cache already having stripped all the boring ones away), will have different needs, in turn, than a frontend or proxy. Each of these situations benefits from a slightly different sampling strategy that optimizes for their needs.

When to Make a Sampling Decision for Traces

So far, each of the preceding strategies has considered *what* criteria to use when selecting samples. For events involving an individual service (or one singular unit of work), that decision solely depends on the preceding criteria. For trace events (or

interconnected events spanning multiple units of work), *when* a sampling decision gets made is also important.

Trace spans are typically collected across multiple services—with each service or collection point potentially employing its own unique sample strategy and rate. In that distributed scenario, the probability is relatively low that every span necessary to form a complete trace will also be the event that each service chooses to sample. To ensure that every span in a useful trace is captured, special care must be taken depending on *when* the decision about whether to sample is made.

As covered earlier, one strategy is to use a property of the event itself—the return status, latency, endpoint, or a high-cardinality field like customer ID—to decide if it is worth sampling. Some properties within the event, such as endpoint or customer ID, are static and known at the start of the event. In *head-based sampling* (or *up-front sampling*), a sampling decision is made when the trace event is initiated. That decision is then propagated further downstream (e.g., by inserting a “guarantee sample retention” header bit) to ensure that every span necessary to complete the trace is sampled.

Some fields, like return status or latency, are known only in retrospect after event execution has completed. If a sampling decision relies on dynamic fields, by the time those are determined, each underlying service will have already independently chosen whether to sample other span events. At best, you may end up keeping the downstream spans deemed interesting outliers, but none of the other context. Properly making a decision on values known only at the end of a request requires *tail-based sampling*.

To collect full traces in the tail-based approach, all spans must first be collected in a buffer and then, retrospectively, a sampling decision can be made. That buffered sampling technique is computationally expensive and not feasible in practice entirely from within the instrumented code. Buffered sampling techniques typically require external collector-side logic.

Additional nuances exist for determining the *what* and *when* of sampling decisions. But at this point, those are best explained using code-based examples.

Translating Sampling Strategies into Code

So far, we’ve covered sampling strategies on a conceptual level. Let’s look at how these strategies are implemented in code. The following pedagogical example uses Go to illustrate implementation details, but the examples would be straightforward to port into any language that supports hashes/dicts/maps, pseudorandom number generation, and concurrency/timers.

The Base Case

Let's suppose you would like to instrument a high-volume handler that calls a downstream service, performs some internal work, and then returns a result and unconditionally records an event to your instrumentation sink:

```
func handler(resp http.ResponseWriter, req *http.Request) {
    start := time.Now()
    i, err := callAnotherService()
    resp.Write(i)
    RecordEvent(req, start, err)
}
```

At scale, this instrumentation approach is unnecessarily verbose and would result in sky-high resource consumption. Let's look at alternate ways of sampling the events this handler would send.

Fixed-Rate Sampling

A naive approach might be probabilistic sampling using a fixed rate, by randomly choosing to send 1 in 1,000 events:

```
var sampleRate = flag.Int("sampleRate", 1000, "Static sample rate")

func handler(resp http.ResponseWriter, req *http.Request) {
    start := time.Now()
    i, err := callAnotherService()
    resp.Write(i)

    r := rand.Float64()
    if r < 1.0 / *sampleRate {
        RecordEvent(req, start, err)
    }
}
```

Every 1,000th event would be kept, regardless of its relevance, as representative of the other 999 events discarded. To reconstruct your data on the backend, you would need to remember that each event stood for `sampleRate` events and to multiply out all counter values accordingly on the receiving end at the instrumentation collector. Otherwise, your tooling would misreport the total number of events actually encountered during that time period.

Recording the Sample Rate

In the preceding clunky example, you would need to manually remember and set the sample rate at the receiving end. What if you need to change the sample rate value at some point in the future? The instrumentation collector wouldn't know exactly when the value changed. A better practice is to explicitly pass the current `sampleRate` when sending a sampled event (see [Figure 15-1](#))—indicating that the

event statistically represents `sampleRate` similar events. Note that sample rates can vary not only between services but also within a single service.

Status code	Time	Sample rate
ok	1:00	100
ok	1:00	100
err	1:00	1
ok	1:01	80
err	1:01	1
ok	1:01	80
ok	1:01	80

Figure 15-1. Different events may be sampled at different rates.

Recording the sample rate within an event can look like this:

```
var sampleRate = flag.Int("sampleRate", 1000, "Service's sample rate")

func handler(resp http.ResponseWriter, req *http.Request) {
    start := time.Now()
    i, err := callAnotherService()
    resp.Write(i)

    r := rand.Float64()
    if r < 1.0 / *sampleRate {
        RecordEvent(req, *sampleRate, start, err)
    }
}
```

With this approach, you can keep track of the sampling rate in effect when each sampled event was recorded. That gives you the data necessary to accurately calculate values when reconstructing your data, even if the sampling rate dynamically changes. For example, if you were trying to calculate the total number of events meeting a filter such as `err != nil`, you would multiply the count of seen events with `err != nil` by each one's `sampleRate` (see [Figure 15-2](#)). And, if you were trying to calculate the sum of `durationMs`, you would need to weigh each sampled event's `durationMs` and multiply it by `sampleRate` before adding up the weighted figures.

This example is simplistic and contrived. Already, you may be seeing flaws in this approach when it comes to handling trace events. In the next section, we will look at additional considerations for making dynamic sampling rates and tracing work well together.

Status code	Time	Count (reweighted)
ok	1:00	200
err	1:00	1
ok	1:01	240
err	1:01	1

Figure 15-2. Total events can be calculated using weighted numbers.

Consistent Sampling

So far in our code, we’ve looked at *how* a sampling decision is made. But we have yet to consider *when* a sampling decision gets made in the case of sampling trace events. The strategy of using head-based, tail-based, or buffered sampling matters when considering how sampling interacts with tracing. We’ll cover how those decisions get implemented toward the end of the chapter. For now, let’s examine how to propagate context to downstream handlers in order to (later) make that decision.

To properly manage trace events, use a centrally generated *sampling/tracing ID* propagated to all downstream handlers instead of independently generating a sampling decision inside each one. Doing so lets you make consistent sampling decisions for different manifestations of the same end user’s request (see [Figure 15-3](#)). In other words, this ensures that you capture a full end-to-end trace for any given sampled request. It would be unfortunate to discover that you have sampled an error far downstream for which the upstream context is missing because it was dropped based on how your sampling strategy was implemented.

HASH (TraceID)	Sampled?
8464143	
9976727	
2046000	YES
8697994	
~~~~~	
1983000	YES
3427217	
6152331	
4919000	YES
6122453	

Figure 15-3. Sampled events containing a *TraceID*.

Consistent sampling ensures that when the sample rate is held constant, traces are either kept or sampled away in their entirety. And if children are sampled at a higher sample rate—for instance, noisy Redis calls being sampled 1 for 1,000, while their parents are kept 1 for 10—it will never be the case that a broken trace is created from a Redis child being kept while its parent is discarded.

Let's modify the previous code sample to read a value for sampling probability from the TraceID/Sampling-ID (note: W3C TraceContext more rigorously specifies this, but we will use Sampling-ID and Upstream-Sample-Rate headers for this chapter's examples), instead of generating a random value at each step:

```
var sampleRate = flag.Int("sampleRate", 1000, "Service's sample rate")

func handler(resp http.ResponseWriter, req *http.Request) {
    // Use an upstream-generated random sampling ID if it exists.
    // otherwise we're a root span. generate & pass down a random ID.
    var r float64
    if r, err := floatFromHexBytes(req.Header.Get("Sampling-ID")); err != nil {
        r = rand.Float64()
    }

    start := time.Now()
    // Propagate the Sampling-ID when creating a child span
    i, err := callAnotherService(r)
    resp.Write(i)

    if r < 1.0 / *sampleRate {
        RecordEvent(req, *sampleRate, start, err)
    }
}
```

Now, by changing the sampleRate feature flag to cause a different proportion of traces to be sampled, you have support for adjusting the sample rate without recompiling, including at runtime. However, if you adopt the technique we'll discuss next—target-rate sampling—you won't need to manually adjust the rate.

## Target-Rate Sampling

You don't need to manually flag-adjust the sampling rates for each of your services as traffic swells and sags. Instead, you can automate this process by tracking the incoming request rate that you're receiving (see [Figure 15-4](#)).

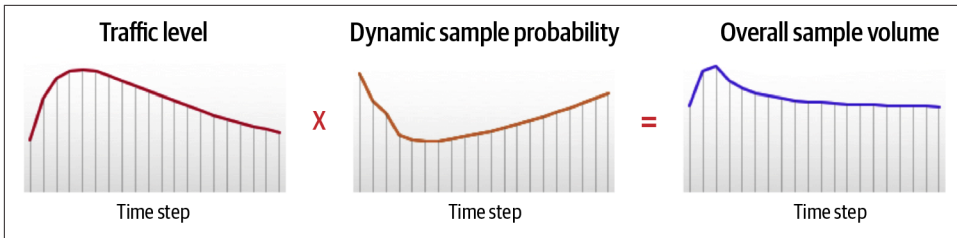


Figure 15-4. You can automate the calculation of overall sample volume.

Let's see how that is done in our code example:

```
var targetEventsPerSec = flag.Int("targetEventsPerSec", 5,
    "The target number of requests per second to sample from this service.")

// Note: sampleRate can be a float! It doesn't have to be an integer.
var sampleRate float64 = 1.0
// Track requests from previous minute to decide sampling rate for the next
// minute.
var requestsInPastMinute *int

func main() {
    // Initialize counters.
    rc := 0
    requestsInPastMinute = &rc

    go func() {
        for {
            time.Sleep(time.Minute)
            newSampleRate = *requestsInPastMinute / (60 * *targetEventsPerSec)
            if newSampleRate < 1 {
                sampleRate = 1.0
            } else {
                sampleRate = newSampleRate
            }
            newRequestCounter := 0
            // Real production code would do something less prone to race
            // conditions e.g. atomics
            requestsInPastMinute = &newRequestCounter
        }
    }()
    http.Handle("/", handler)
    [...]
}

func handler(resp http.ResponseWriter, req *http.Request) {
    var r float64
    if r, err := floatFromHexBytes(req.Header.Get("Sampling-ID")); err != nil {
        r = rand.Float64()
    }
}
```

```

    start := time.Now()
    *requestsInPastMinute++
    i, err := callAnotherService(r)
    resp.Write(i)

    if r < 1.0 / sampleRate {
        RecordEvent(req, sampleRate, start, err)
    }
}

```

This example provides predictable experience in terms of resource cost. However, the technique still lacks flexibility for sampling at variable rates depending on the volume of each key.

## Having More Than One Static Sample Rate

If the sampling rate is high, whether due to being dynamically or statically set high, you need to consider that you could miss long-tail events—for instance, errors or high-latency events—because the chance that a 99.9th percentile outlier event will be chosen for random sampling is slim. Likewise, you may want to have at least some data for each of your distinct sources, rather than have the high-volume sources drown out the low-volume ones.

A remedy for that scenario is to set more than one sample rate. Let's start by varying the sample rates by key. Here, the example code will sample any baseline (non-outlier) events at a rate of 1 in 1,000 and chooses to tail-sample any errors or slow queries at a rate of 1 in 1 and 1 in 5, respectively:

```

var sampleRate = flag.Int("sampleRate", 1000, "Service's sample rate")
var outlierSampleRate = flag.Int("outlierSampleRate", 5, "Outlier sample rate")

func handler(resp http.ResponseWriter, req *http.Request) {
    start := time.Now()
    i, err := callAnotherService(r)
    resp.Write(i)

    r := rand.Float64()
    if err != nil || time.Since(start) > 500*time.Millisecond {
        if r < 1.0 / *outlierSampleRate {
            RecordEvent(req, *outlierSampleRate, start, err)
        }
    } else {
        if r < 1.0 / *sampleRate {
            RecordEvent(req, *sampleRate, start, err)
        }
    }
}

```

Although this is a good example of using multiple static sample rates, the approach is still susceptible to spikes of telemetry traffic. If the application experiences a spike

in the rate of errors, every single error gets sampled. Next, we will address that shortcoming with target-rate sampling.

## Sampling by Key and Target Rate

Putting two previous techniques together, let's extend what we've already done to target specific rates of telemetry. If a request is anomalous (for example, has latency above 500 milliseconds or is an error), it can be designated for tail sampling at its own guaranteed rate, while rate-limiting the other requests to fit within a budget of sampled requests per second:

```
var targetEventsPerSec = flag.Int("targetEventsPerSec", 4,
    "The target number of ordinary requests/sec to sample from this service.")
var outlierEventsPerSec = flag.Int("outlierEventsPerSec", 1,
    "The target number of outlier requests/sec to sample from this service.")

var sampleRate float64 = 1.0
var requestsInPastMinute *int

var outlierSampleRate float64 = 1.0
var outliersInPastMinute *int

func main() {
    // Initialize counters.
    rc := 0
    requestsInPastMinute = &rc
    oc := 0
    outliersInPastMinute = &oc

    go func() {
        for {
            time.Sleep(time.Minute)
            newSampleRate = *requestsInPastMinute / (60 * *targetEventsPerSec)
            if newSampleRate < 1 {
                sampleRate = 1.0
            } else {
                sampleRate = newSampleRate
            }
            newRequestCounter := 0
            requestsInPastMinute = &newRequestCounter

            newOutlierRate = outliersInPastMinute / (60 * *outlierEventsPerSec)
            if newOutlierRate < 1 {
                outlierSampleRate = 1.0
            } else {
                outlierSampleRate = newOutlierRate
            }
            newOutlierCounter := 0
            outliersInPastMinute = &newOutlierCounter
        }
    }()
}
```

```

    http.Handle("/", handler)
    [...]
}

func handler(resp http.ResponseWriter, req *http.Request) {
    var r float64
    if r, err := floatFromHexBytes(req.Header.Get("Sampling-ID")); err != nil {
        r = rand.Float64()
    }
    start := time.Now()
    i, err := callAnotherService(r)
    resp.Write(i)
    if err != nil || time.Since(start) > 500*time.Millisecond {
        *outliersInPastMinute++
        if r < 1.0 / outlierSampleRate {
            RecordEvent(req, outlierSampleRate, start, err)
        }
    } else {
        *requestsInPastMinute++
        if r < 1.0 / sampleRate {
            RecordEvent(req, sampleRate, start, err)
        }
    }
}
}

```

That extremely verbose example uses chunks of duplicate code, but is presented in that manner for clarity. If this example were to support a third category of request, it would make more sense to refactor the code to allow setting sampling rates across an arbitrary number of keys.

## Sampling with Dynamic Rates on Arbitrarily Many Keys

In practice, you likely will not be able to predict a finite set of request quotas that you may want to set. In the preceding example, our code had many duplicate blocks, and we were designating target rates manually for each case (error/latency versus normal).

A more realistic approach is to refactor the code to use a map for each key's target rate and the number of seen events. The code would then look up each key to make sampling decisions. Doing so modifies our example code like so:

```

var counts map[SampleKey]int
var sampleRates map[SampleKey]float64
var targetRates map[SampleKey]int

func neverSample(k SampleKey) bool {
    // Left to your imagination. Could be a situation where we know the request
    // is a keep-alive or health check we never want to record, etc.
    return false
}

```



```

// Boilerplate main() and goroutine init to overwrite maps and roll them over
// every interval goes here.

type SampleKey struct {
    ErrMsg      string
    BackendShard int
    LatencyBucket int
}

// This might compute for each k: newRate[k] = counts[k] / (interval *
// targetRates[k]), for instance.

func checkSampleRate(resp http.ResponseWriter, start time.Time, err error,
    sampleRates map[any]float64, counts map[any]int) float64 {
    msg := ""
    if err != nil {
        msg = err.Error()
    }
    roundedLatency := 100 * (time.Since(start) / (100*time.Millisecond))
    k := SampleKey {
        ErrMsg:      msg,
        BackendShard: resp.Header().Get("Backend-Shard"),
        LatencyBucket: roundedLatency,
    }
    if neverSample(k) {
        return -1.0
    }

    counts[k]++
    if r, ok := sampleRates[k]; ok {
        return r
    } else {
        return 1.0
    }
}

func handler(resp http.ResponseWriter, req *http.Request) {
    var r float64
    if r, err := floatFromHexBytes(req.Header.Get("Sampling-ID")); err != nil {
        r = rand.Float64()
    }

    start := time.Now()
    i, err := callAnotherService(r)
    resp.Write(i)

    sampleRate := checkSampleRate(resp, start, err, sampleRates, counts)
    if sampleRate > 0 && r < 1.0 / sampleRate {
        RecordEvent(req, sampleRate, start, err)
    }
}

```

At this point, our code example is becoming quite large, and it still lacks more sophisticated techniques. Our example has been used to illustrate how sampling concepts are implemented.

Luckily, existing code libraries can handle this type of complex sampling logic. For Go, the *[dynsampler-go](#)* library maintains a map over any number of sampling keys, allocating a fair share of sampling to each key as long as it is novel. That library also contains more advanced techniques of computing sample rates, either based on target rates or without explicit target rates at all.

For this chapter, we're close to having put together a complete introductory tour of applying sampling concepts. Before concluding, let's make one last improvement by combining the tail-based sampling you've done so far with head-based sampling that can request tracing be sampled by all downstream services.

## Putting It All Together: Head and Tail per Key Target-Rate Sampling

Earlier in this chapter, we noted that head-based sampling requires setting a header to propagate a sampling decision downstream. For the code example we've been iterating, that means the parent span must pass both the head-sampling decision and its corresponding rate to all child spans. Doing so forces sampling to occur for all child spans, even if the dynamic sampling rate at that level would not have chosen to sample the request:

```
var headCounts, tailCounts map[interface{}]int
var headSampleRates, tailSampleRates map[interface{}]float64

// Boilerplate main() and goroutine init to overwrite maps and roll them over
// every interval goes here. checkSampleRate() etc. from above as well

func handler(resp http.ResponseWriter, req *http.Request) {
    var r, upstreamSampleRate, headSampleRate float64
    if r, err := floatFromHexBytes(req.Header.Get("Sampling-ID")); err != nil {
        r = rand.Float64()
    }

    // Check if we have a non-negative upstream sample rate; if so, use it.
    if upstreamSampleRate, err := floatFromHexBytes(
        req.Header.Get("Upstream-Sample-Rate")
    ); err == nil && upstreamSampleRate > 1.0 {
        headSampleRate = upstreamSampleRate
    } else {
        headSampleRate := checkHeadSampleRate(req, headSampleRates, headCounts)
        if headSampleRate > 0 && r < 1.0 / headSampleRate {
            // We'll sample this when recording event below; propagate the
            // decision downstream though.
        } else {
            // Clear out headSampleRate as this event didn't qualify for
            // sampling. This is a sentinel value.
        }
    }
}
```

```

        headSampleRate = -1.0
    }
}

start := time.Now()
i, err := callAnotherService(r, headSampleRate)
resp.Write(i)

if headSampleRate > 0 {
    RecordEvent(req, headSampleRate, start, err)
} else {
    // Same as for head sampling, except here we make a tail sampling
    // decision we can't propagate downstream.
    tailSampleRate := checkTailSampleRate(
        resp, start, err, tailSampleRates, tailCounts,
    )
    if tailSampleRate > 0 && r < 1.0 / tailSampleRate {
        RecordEvent(req, tailSampleRate, start, err)
    }
}
}

```

At this point, our code example is rather complicated. However, even at this level, it illustrates a powerful example of the flexibility that sampling can provide to capture all the necessary context needed to debug your code. In high-throughput modern distributed systems, it may be necessary to get even more granular and employ more sophisticated sampling techniques.

For example, you may want to change the `sampleRate` of head-based samples for increased probability whenever a downstream tail-based heuristic captures an error in the response. In that example, collector-side buffered sampling is a mechanism that would allow deferring a sampling decision until after an entire trace has been buffered—bringing together the advantages of head-based sampling to properties only known at the tail.

## Conclusion

Sampling is a useful technique for refining your observability data. While sampling is necessary when running at scale, it can be useful in a variety of circumstances even at smaller scales. Sampling is most useful for optimizing resource consumption while maintaining data fidelity.

The code-based examples illustrate how various sampling strategies are implemented. It's becoming increasingly common for open source instrumentation libraries—such as OTel—to implement that type of sampling logic for you. As those libraries become the standard for generating application telemetry data, it should become less likely that you would need to reimplement these sampling strategies in your own code.

However, even if you rely on third-party libraries to manage that strategy for you, it is essential that you understand the mechanics behind how sampling is implemented so you can understand which method is right for your particular situation. Understanding how the strategies (static versus dynamic, head versus tail, or a combination thereof) work in practice enables you to use them wisely to achieve data fidelity while also optimizing for resource constraints.

Similar to deciding what and how to instrument your code, deciding *what*, *when*, and *how* to sample is best defined by your unique organizational needs. The fields in your events that influence how interesting they are to sample largely depend on how useful they are to understanding the state of your environment and their impact on achieving your business goals. Sampling reduces the overall volume of telemetry data your applications transmit to an observability backend.

In the next chapter, we'll examine an approach to routing large volumes of telemetry data: telemetry management with pipelines.

---

# Telemetry Management with Pipelines

*This chapter is contributed by Mike Kelly,  
Founder and CEO at Bindplane*

## A Note from Charity, Liz, George, and Austin

In the previous chapter, we looked at strategies to reduce the deluge of telemetry volume at the source. In this chapter, we'll look at a different way to manage large volumes of telemetry data: with pipelines.

Telemetry pipelines can help manage a variety of complexity scenarios, beyond just managing data volume. In simpler systems, telemetry from applications can be sent directly to an observability backend. In more complex systems, you may need to route telemetry data to multiple backends in order to isolate workloads, meet security and compliance needs, satisfy different retention requirements, or for a variety of other reasons.

When you consider data volume on top of that, managing the many-to-many relationship between telemetry producers and consumers can become extraordinarily complex. Telemetry pipelines help you abstract away that complexity.

In this chapter, Mike Kelly details the inner workings of telemetry pipelines, use cases, and adoption patterns through the lens of Bindplane: a telemetry pipeline built on the OTel standard. Mike is CEO of Bindplane, the company behind the project. We're delighted to be able to share his perspectives with you in the context of this book.

The rest of this chapter is told from Mike Kelly's perspective.

# An Introduction to Telemetry Pipelines

If you’ve worked in observability, you’re familiar with the challenge: modern systems generate an overwhelming amount of telemetry. Every signal is critical (or is it?). Every new cluster, new service, new application, and even the expansion of legacy systems compounds the challenge. Observability, security, compliance, and AI are hungry for data, and even if you know what can be trimmed, doing that at enterprise scale quickly overwhelms even the best-prepared teams.

This is the reality that many organizations find themselves in as their observability practice matures. When we started Bindplane, this challenge didn’t have a name, let alone a clear solution. OTel was in its infancy, and the term “telemetry pipeline” didn’t exist. Observability teams beginning to hit this limit were searching for quick fixes, or began designing their own systems with mixed results. It was a finger-in-the-dam approach of temporary patches and improvised pipelines meant to hold back the flood.

In our early days, we built what we called “monitoring integration as a service.” In hindsight, the description simply reflected how early the space was. At the time, however, we lacked a vocabulary to discuss the challenge and solution set. More importantly, it was missing the point: customers and platforms reported that to “get the data in” was simply too difficult. How do you collect every signal from every device, and do this in a platform designed for simplicity and scale? That’s the problem we initially focused on solving, but in doing so, it magnified a parallel challenge we hadn’t yet recognized. You can instrument and collect every trace, log, event, and metric, and now your team is happy, and your observability platform has everything it needs. But the relief is short-lived, and quickly turns into difficult questions from the CFO, along with a growing infrastructure team that’s scrambling to manage the new scale issues created by all that data.

It’s a classic scaling problem. You make it scale, and then reveal a whole range of new challenges. In observability, the explosion of data volume and the emergence of new use cases for this data have made old approaches unsustainable. The cost and complexity of managing telemetry at scale demand a new solution. If you’re feeling that pressure, you’re not alone. And this is exactly where telemetry pipelines come in.

## The Telemetry Pipeline Solution

The solution starts with recognizing the problem. It’s not just that there’s too much data; it’s that the data flow itself was never treated as its own concern. Instead of every application determining where telemetry should go, a telemetry pipeline should become the backbone by collecting, normalizing, processing, and routing signals everywhere they’re needed.

Take, for example, a power grid. Wiring every appliance directly to a power plant would be fragile and inefficient. This is solved by utilizing substations, routing, and redundancy. All that's required for the consumer is a plug and an outlet, and the remaining delivery is handled. Telemetry pipelines work the same way.

All modern telemetry pipelines share several core functions:

- *Collect* signals from any source emitting telemetry.
- *Normalize and secure* data into a standard model while enforcing compliance.
- *Enrich* telemetry with context to make it actionable.
- *Reduce* volume with filtering, aggregation, and sampling.
- *Route* the right telemetry to the right tool: observability, security, analytics, and/or long-term storage.

These functions ensure that telemetry pipelines manage end-to-end delivery for logs, metrics, traces, or any other signal.

## Why This Matters Now

Pipeline adoption is growing exponentially because of two basic principles: *cost and standardization*. Sending everything everywhere is no longer sustainable, and OTel has become the vendor-neutral standard. With its Collector and libraries, teams can build flexible pipelines supported by every major observability and security vendor.

Cost and standardization have made telemetry pipelines a foundational piece of modern infrastructure. With a pipeline in place, you can:

- Add a new observability tool without touching application code.
- Enforce compliance rules company-wide, from a central location.
- Reuse the same telemetry through multiple domains (e.g., observability, security, AI and machine learning, or BI).

Taken together, a well-architected pipeline transforms telemetry from a scaling issue into an asset.

## Core Functions of a Modern Telemetry Pipeline

In the previous section, I outlined the core functions of a telemetry pipeline. Now, let's go a layer deeper. Rather than a patchwork of components, you can think of a telemetry pipeline as a sequence of stages: *collect* → *normalize* → *enrich* → *reduce* → *route*. All of these stages are supported by two critical requirements: *data resilience* and *pipeline control and observability*, as shown in [Figure 16-1](#).

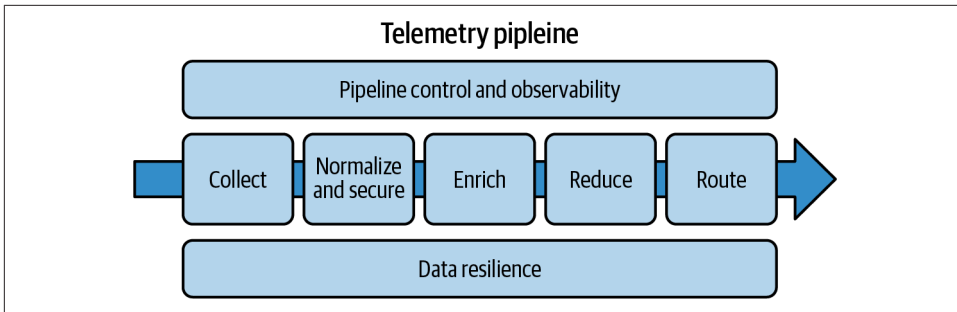


Figure 16-1. Core functions of a modern telemetry pipeline.

In this section, I'll explain each stage, why it matters, and how OTel brings it to life.

## Collect

Telemetry collection is the first and most important step for most observability practitioners. Applications, infrastructure, and devices all emit logs, metrics, traces, and events. A telemetry pipeline must collect every signal that matters, from legacy systems and cloud providers to modern microservices and greenfield deployments. Without a reliable collection, decisions are made on incomplete telemetry. Telemetry collection ensures the full context required to troubleshoot, secure, and optimize your systems.

OTel has become the standard for collection. It's open source and vendor neutral with industry-wide support, and able to gather logs, metrics, and traces through a single collector. It includes broad support for both new and legacy technologies, with over 100 receivers in the OTel project, including generic options like OpenTelemetry Protocol (OTLP), Prometheus, StatsD, Java Management Extensions (JMX), and host metrics. The depth of support and flexibility makes OTel an ideal building block for your telemetry pipeline.

Moving away from proprietary agents and SDKs is critical. As long as collection is tied directly to a single destination, you will never achieve the reusability and standardization that a pipeline enables.

## Normalize and Secure

Telemetry normalization and security is the next step. Raw telemetry is messy, and this stage makes it structured, consistent, and compliant. This is accomplished with:

### *Parsing and coercion*

Convert unstructured logs into structured events, fix timestamps, and enforce schemas.



### *Normalization*

Transform mixed formats (for example, JSON logs or Zipkin traces) into a common model such as OTLP. This prepares signals for downstream processing.

### *Security and governance*

Redact personally identifiable information (PII) and apply data access, retention, and residency controls to manage compliance requirements—e.g., General Data Protection Regulation (GDPR), Federal Risk and Authorization Management Program (FedRAMP).

Normalizing early ensures that all downstream data is clean, usable, and compliant, reducing both rework and risk later in the pipeline.

## **Enrich**

*Telemetry enrichment* adds the context that makes telemetry meaningful. Raw signals are enhanced with metadata, business context, and external lookups. Enrichment includes:

### *Infrastructure context*

Attach details like Kubernetes pod, node, region, or availability zone.

### *Business context*

Add business-specific details such as customer ID, environment (prod, staging), or service tier.

### *External lookups*

Resolve IPs to locations or map DNS to hostnames.

Early enrichment makes telemetry immediately actionable and shortens time to detection. (For more examples on telemetry enrichment, see [Chapter 6](#).)

## **Reduce**

Telemetry reduction, or reducing overall data volume, is one of the biggest reasons teams adopt pipelines. It keeps costs under control and helps maintain a high signal-to-noise ratio. Modern systems generate more data than most teams can reasonably handle, and low-value telemetry can easily bury the data that matters most.

The overall volume of telemetry can be reduced with:

### *Filtering*

Drop low-value logs, redundant metrics, or trivial traces.

### *Deduplication*

Consolidate repeats without losing visibility.

### Aggregation

Roll up metrics or events to higher-level views.

### Sampling

Apply head, tail, or adaptive sampling to noisy sources.

Centralized reduction preserves visibility while removing waste and noise. This makes it possible to adapt your reduction strategy in real time as needs change.

## Route

Routing telemetry to multiple destinations is where the value of a pipeline becomes obvious. Once signals are standardized, enriched, and reduced, they can be used for multiple purposes. There are two main use cases for routing:

### Fan-out

Send logs to a security information and event management (SIEM), metrics to APM, and events to analytics, and store high-fidelity copies in cloud storage for compliance, as shown in [Figure 16-2](#).

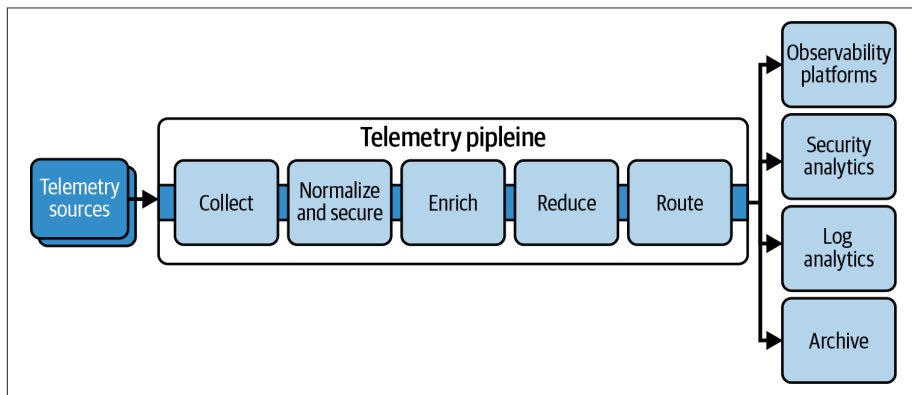


Figure 16-2. Telemetry routing to multiple destinations—observability platforms, security analytics, log analytics, and archive.

### Flexibility

Add or test new tools without touching application code or reinstrumenting your systems.

Routing turns a single clean data stream into fuel for observability, security, compliance, and AI, without duplication.

## Data Resilience

*Data resilience* ensures telemetry keeps flowing when systems fail. Backends go down, networks spike, and data volumes burst. A resilient pipeline absorbs those shocks and

ensures telemetry is delivered reliably even when parts of the system are degraded. That resiliency can be achieved with:

#### *Buffering and persistence*

Temporary, adjustable buffers at each stage of the pipeline with optional disk persistence to prevent data loss

#### *Retries and flow control*

Retry logic control with exponential backoff and backpressure to avoid the “thundering herd” effect or cascading failures impacting downstream consumers

#### *High availability and failover*

Redundant agents, gateways, and control planes to provide continuity when nodes or regions fail

For example, in Bindplane, resilience is critical at scale. Our collectors and gateways buffer telemetry locally, queue it for delivery, and support failover paths when a backend is unavailable. This prevents gaps in observability data during outages and helps teams maintain trust in their telemetry.

## **Pipeline Control and Observability**

Even the best pipeline fails without management and visibility. Pipelines are not “set it and forget it.” They need to be monitored, tuned, and governed. A control plane makes this possible, providing the capabilities that keep pipelines operationally viable at scale. All telemetry pipelines should include:

#### *Real-time visibility*

Dashboards and metrics show throughput, error rates, and latency across the pipeline.

#### *Policy enforcement*

Centralized controls apply compliance rules, retention settings, and data-handling policies consistently.

#### *Configuration management*

Pipelines must be adjustable at runtime to roll out changes quickly, update destinations, or alter reduction strategies.

#### *Incident response*

Telemetry control planes make it possible to rehydrate historical data, reroute flows, or isolate faulty components during incidents.

#### *Fleet management*

Standard protocols such as Open Agent Management Protocol, or **OpAMP**, define how collectors report health, fetch configurations, and receive updates. Tools like Bindplane build on this to add enterprise-grade features such as

versioned rollouts, remote upgrades, failover handling, and rehydration of historical data.

This layer of control is what makes pipelines valuable. It lets you manage fleets, preview changes, and keep environments consistent so you can truly simplify and standardize at high scale.

## What You Should Remember About Core Functions

A modern telemetry pipeline is built from a sequence of core functions. They work together to transform raw signals into a governed, reusable stream of data. Each stage builds on the last. Collecting, normalizing, enriching, reducing, routing, and ensuring resilience and control form the backbone of a modern telemetry pipeline.

When they operate in harmony, telemetry shifts from an unruly firehose into a strategic asset that powers observability, security, compliance, and innovation. This foundation sets the stage for the architecture and adoption strategies that I'll walk you through next.

## Pipeline Architecture in Practice

So far, you've learned about the core functions of a telemetry pipeline. Architecture is where they come together in practice. How you deploy the components determines scalability, resilience, and manageability. Get the architecture right, and you can start small and scale to thousands of collectors without losing control.

## Core Components

At the heart of any telemetry pipeline are three categories of components:

### *Agents*

These are lightweight OTel Collectors deployed on hosts, containers, or as sidecars. They capture telemetry as close to the source as possible with minimal overhead.

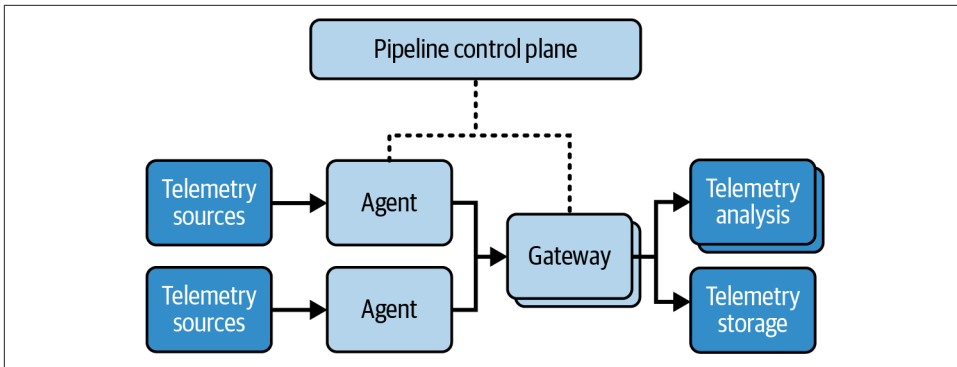
### *Gateways*

These are centralized, scalable nodes that aggregate data from agents, apply processing and compliance rules, reduce volume, and route signals to destinations. They are typically deployed in clusters for resilience and throughput.

### *Control plane*

This is the management layer that makes pipelines operationally viable at scale. In Bindplane, this is where you configure, visualize, and update collectors and gateways across fleets.

These categories are shown in [Figure 16-3](#).



*Figure 16-3. Core pipeline components—agents, gateways, and a centralized control plane providing orchestration and configuration.*

Another essential core component in Bindplane is the OTEL Collector. A telemetry collector is the foundation of most modern pipelines. The OTEL Collector has a pluggable architecture that includes:

#### *Receivers*

For ingest (HTTP, gRPC, Kafka, Syslog, etc.)

#### *Processors*

For transformations like normalization, enrichment, sampling, and batching

#### *Exporters*

For sending data to backends (Prometheus, Elasticsearch, Datadog)

#### *Extensions*

For cross-cutting functions like zPages, health checks, and authentication

A single binary can run in multiple roles, which is where the flexibility shines:

#### *Agent*

Runs close to workloads, performs minimal processing, and forwards telemetry to gateways. This keeps the edge footprint light and reduces impact on local systems.

#### *Gateway*

Runs as resilient, load-balanced clusters. Gateways handle the heavy lifting like reduction, compliance, routing, buffering, and queuing on dedicated nodes that won't interfere with workloads.

## Deployment Patterns

How the core components come together depends on the organization's size and needs. Some examples are shown in [Figure 16-4](#).

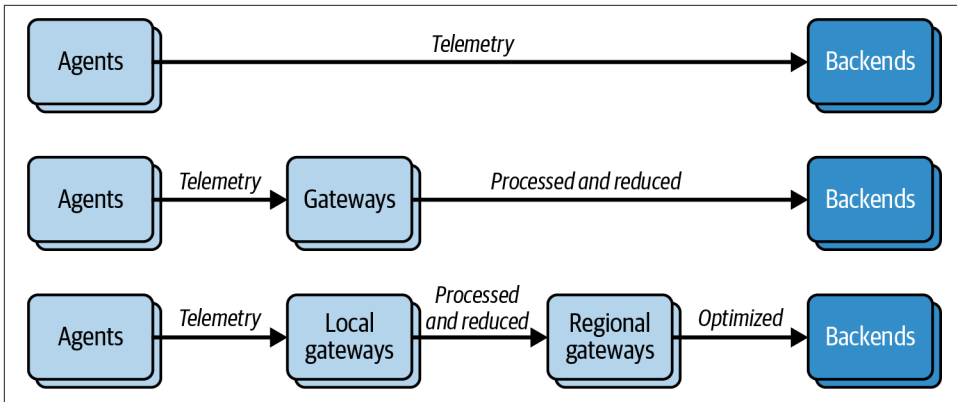


Figure 16-4. Common scaling patterns—agent-only, agent + gateway, and multi-tier global architectures for high availability and cost efficiency.

The flexibility of these components means they can be composed in various ways to suit different organizational use cases. The most common configurations include:

#### *Agent-only*

This is the simplest approach. Each agent sends telemetry directly to one or more backends. This works in small environments, but it doesn't scale and it creates tight coupling between sources and destinations.

#### *Agent + gateway*

This is the most common production pattern. Agents forward telemetry to gateways, which centralize processing, reduction, and routing. This decouples collection from delivery and establishes compliance and control in one place.

#### *Multi-tier topologies*

This pattern supports the needs of large, distributed organizations. Local gateways handle local collection, reduction, and compliance before forwarding optimized signals to regional gateways. This enables high availability, disaster recovery, and cost efficiency at scale.

## Scaling and Performance in Practice

At Bindplane, we work with customers processing petabytes of telemetry daily across hundreds of thousands of OTel collectors running as both agents and gateways. Performance and resilience at that scale require careful planning, but with the right patterns, you can avoid common pitfalls, such as:

*Agents*

Agents should stay lightweight. Apply minimal processing, set clear memory and CPU limits, and use small local buffers only to handle short bursts of data. Their primary role is reliable forwarding to the gateway layer without impacting local workloads.

*Gateways*

Gateways carry the heavy load. Scale them horizontally, add redundancy and load balancing to eliminate single points of failure, and use buffering and queuing to absorb bursts. Health checks should track queue depth, latency, and error rates to catch issues.

Applied consistently, these practices enable pipelines to handle massive throughput without data loss or degraded performance.

## Build-Versus-Buy Considerations

Organizations face a key decision: build and operate pipelines themselves, or leverage vendor solutions. There are three main approaches to consider when navigating that decision, and the approach you decide to take depends on the trade-offs that work best for your needs, as shown in [Table 16-1](#).

Table 16-1. Comparison of three main approaches for build versus buy

Approach	Description	Strengths	Challenges	Best for
Open source/DIY	Build and operate all pipeline components in-house using open source tooling.	Maximum flexibility Full customization of every component No vendor lock-in	High operational burden Requires deep in-house expertise Scaling, upgrades, and compliance become full-time work	Teams with strong engineering capacity and complex custom requirements
Vendor supported	Leverage a commercial solution that provides enterprise-grade pipeline features out of the box.	Reduced operational overhead Faster time to value Provides enterprise features out of the box	Flexibility may be constrained Potential vendor lock-in Ongoing licensing costs	Teams that prioritize speed and low operational load over customization
Hybrid	Adopt open standards (e.g., OTel, OpAMP) for pipelines while using a commercial control plane (e.g., Bindplane) for fleet management.	Reduced operational overhead Faster time to value Fleet management, governance, and cost controls included Vendor-neutral	More complex to architect Requires understanding of both open and commercial tooling	Organizations wanting open standards with enterprise reliability; increasingly common

Ultimately, the decision to build or buy comes down to several organizational factors. For more about deciding whether to build or buy, see [Chapter 29](#).

## What You Should Remember About Pipeline Architecture

Pipeline architecture is an evolution. The smartest path is to start with an agent + gateway pattern so that you have best practices in place from day one and can expand without rework. Next, add a control plane to manage and govern your fleet consistently. As your needs grow, move to a multi-tier regional architecture that gives you resilience, compliance, and cost control at scale.

Consider Bindplane as a reference implementation to show how this comes together: OTel gives you the building blocks, OpAMP brings standardized management, and Bindplane makes everything operational at enterprise scale.

## Adoption and Migration

Few companies start thinking about pipeline architecture, where it fits, and how best to use it from a clean greenfield slate. More commonly, pipelines are introduced in brownfield telemetry architectures. By the time you're considering adding a telemetry pipeline, you already have hundreds of streams running on proprietary agents. The challenge is adapting it to the ideal pipeline architecture.

In this section, we'll look at a phased strategy for introducing telemetry pipelines into preexisting application architectures.

It's rare that you'll need to reinstrument your entire environment to gain the benefits of a telemetry pipeline.

### A Phased Path to Adoption

At Bindplane, we guide customers through a staged approach to a telemetry pipeline:

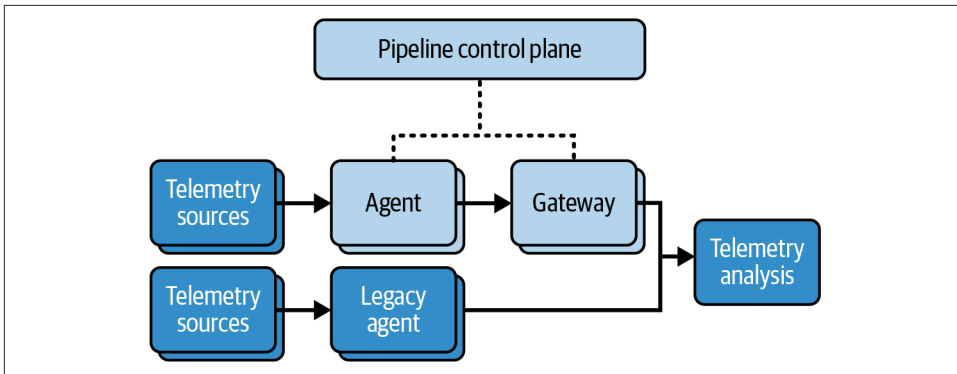
1. Start with a greenfield app.
2. Establish best practices and future-proofing with OTel standards.
3. Incrementally redirect legacy flows to the new telemetry pipeline.

A phased approach delivers early wins, builds familiarity, and reduces risk by avoiding disruptive cutovers. The process starts small and then incrementally incorporates more systems until the majority of your telemetry is standardized and centrally managed.

#### Phase 1: Greenfield

Start with a simple pipeline connected to a single application or service. This can be applied regardless of the source type. The goal is to validate the pipeline end to end and provide a low-risk way to build confidence in new tooling and practices. This is typically done with agents and gateways, as shown in [Figure 16-5](#).





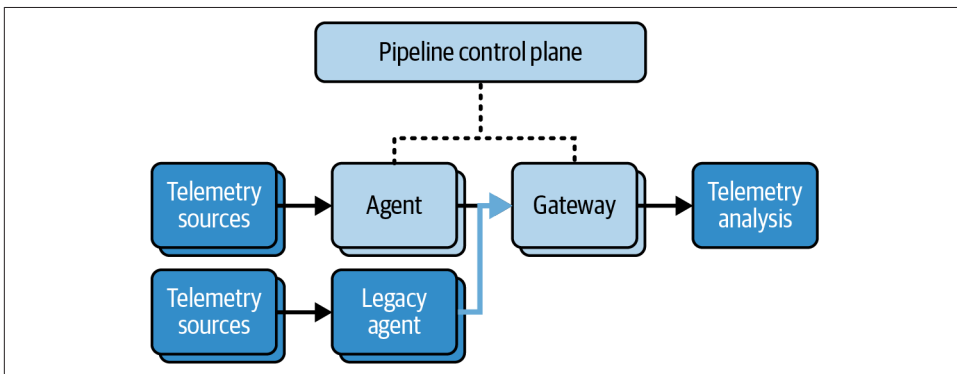
*Figure 16-5. Phase 1: Initial pipeline deployment with a greenfield application using agents and gateways under orchestration.*

Most organizations start by instrumenting a new microservice or existing internal application.

## Phase 2: Redirecting existing agents

While replacing all of your existing instrumentation with OTel may be ideal, it's not realistic. In cases where replacement isn't feasible or may be too time-consuming, an easier path is to redirect those telemetry streams to gateways within your telemetry pipeline.

You'll shift legacy telemetry flows into the pipeline to manage compliance and reduction from a centralized location, as shown in [Figure 16-6](#).



*Figure 16-6. Phase 2: Redirecting existing legacy agents to pipeline gateways for centralized compliance and reduction.*

OTel's flexibility in the types of signals it can ingest makes this an easy incremental step toward a complete telemetry plane.

### Phase 3: Replacing existing agents

In the final phase, most of your telemetry is flowing through the pipeline, and the team is confident in managing it. Security and compliance standards are enforced consistently, and this is the point to replace legacy agents and standardize collection. A single method for collection and management reduces complexity and overhead.

To that end, during this phase you'll replace legacy agents with OTEL Collectors, as shown in [Figure 16-7](#).

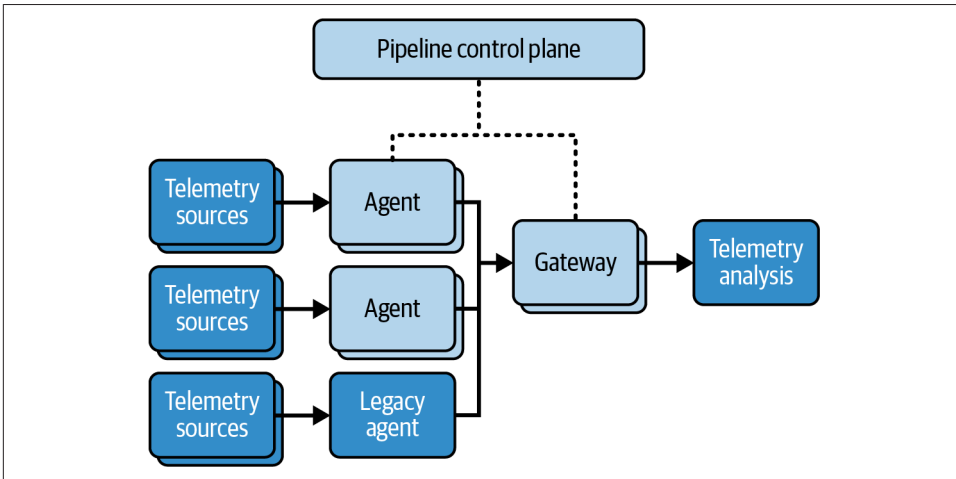


Figure 16-7. Phase 3: Replacing legacy agents with OTEL collectors for full standardization and unified management.

That said, not everything is worth the lift. A fragile legacy app that no one wants to touch may stay in phase 2 indefinitely, and that is often an acceptable trade-off when modernizing your telemetry pipeline.

## What You Should Remember About Adoption and Migration

Adopting new tools and migrating architecture components for your services can be a long and challenging process for some organizations, especially those with very large or broad brownfield application architectures. During your migration, you should keep a few best practices in mind:

*Don't boil the ocean.*

Focus your effort on the highest-value telemetry streams first.

*Communicate trade-offs.*

Teams need to understand why some systems may remain on legacy instrumentation longer.

*Run old and new side by side.*

Overlapping systems give you confidence and a rollback path during transitions.

*Invest in education and culture.*

Successful migrations require not only technology changes but also building trust and expertise across teams.

Treat adoption of a telemetry pipeline as a journey. When you do, you avoid the risks of a disruptive cutover and you'll see the benefits of a telemetry pipeline quickly. Over time, the phased approach enables you to standardize, simplify, and modernize without impacting operations along the way.

## Use Cases: Collect and Reduce, Combined

In this section, I will summarize two of the use cases that often prompt organizations to look toward telemetry pipelines as a solution: collection and reduction. I've seen these have the greatest impact on demonstrating value.

Large enterprises tend to face the same challenges:

- Too much data
- Too costly
- Too much operational overhead

In one case study (as of yet unpublished), 40% of SIEM ingest was never queried. Millions were wasted on signals that no one used. **In another**, platform teams had to manage fleets of 25,000+ collectors across devices, fighting drift and config errors daily.

The solution was a telemetry pipeline built on OTel: agent collectors at the edge, a gateway collector layer in the middle, and a control plane to manage it all. The aim is to collect once, manage centrally, and process data before it ever hits an expensive vendor endpoint.

Two core functions made these customers successful:

1. *Collect*: Bindplane pushes versioned configs fleet-wide, validates changes, and shows real-time status. Thousands of collectors stay in sync with no manual intervention.
2. *Reduce*: The pipeline cuts noise. Drop high-churn fields, collapse duplicates, sample noisy sources, and redact sensitive data. Route critical signals to your telemetry backend of choice, and archive the rest to low-cost storage like Google Cloud Storage or Amazon S3.

Costs can drop by 50% or more. **One customer** eliminated a \$2.6 million overage and achieved roughly \$2 million in savings. Their platform and observability teams gained confidence and reliability with centralized management and real-time pipeline visibility. Compliance improved as PII was removed and security rules were enforced centrally. Because routing is decoupled from collection, switching or adding new telemetry sources or destinations became a config change rather than a reinstrumentation project.

## Business Case

Throughout this chapter, I've focused primarily on the technical advantages of telemetry pipelines over existing solutions. While critical, making the case for pipelines also means articulating their impact to those who control the budget. In this section, we'll examine telemetry pipelines through a business-value lens.

### Cost Control

Telemetry pipelines deliver direct financial impact by addressing one of the biggest pain points in observability: ingest volume. Most vendors charge based on the amount of data sent, meaning you pay for everything, regardless of whether it's valuable or not. In fact, we've often worked with teams that discovered that 20–40% of their SIEM data was never actually queried. A telemetry pipeline solves this by filtering, deduplication, and sampling. Better yet, all of this happens before the data ever reaches expensive telemetry storage.

An even bigger win is cost predictability. Instead of explaining why observability spend spiked 30% last quarter, pipelines provide real-time visibility into data flows and the cost drivers behind them. This turns observability from an unpredictable cost to a manageable and forecastable monthly invoice. CFOs stop asking why costs are exploding, enabling teams to focus on improving insights rather than justifying budgets.

### Vendor Neutrality

Pipelines built on OTel remove the lock-in of proprietary agents and plumbing. Switching tools becomes a configuration change, not a multiyear migration project. This flexibility creates negotiating power with vendors and gives organizations the confidence to adopt new tools without the fear of disruption.

## Telemetry as a Strategic Asset

At the same time, pipelines elevate telemetry from a cost liability to a strategic asset. Think of telemetry as raw material:

- Observability teams use it for uptime and performance.
- Security teams rely on it for detection and response.
- Compliance teams need it to enforce security policies and regulations.

Without a pipeline, each team duplicates effort, reinstruments systems, and pays vendors to store overlapping telemetry. A pipeline unlocks a unified, clean, enriched stream of telemetry, fanned out to every team and tool that needs it.

Pipelines also reduce compliance and security risk. Sensitive fields can be redacted before they leave your environment, with GDPR, FedRAMP, or the Health Insurance Portability and Accountability Act of 1996 (HIPAA) controls enforced centrally instead of scattered across dozens of tools. This reduces breach exposure, simplifies audits, and ensures consistent governance without relying on downstream vendors.

The economics are straightforward:

- Save 50% or more on vendor ingest costs.
- Reinvest ~10% into pipeline infrastructure.
- Net out at ~40% savings, while gaining more control and vendor neutrality.

As telemetry grows with your business, telemetry pipelines ensure that observability costs don't outpace revenue. The result is sustainable scaling, improved visibility and security, predictable costs, and telemetry evolving from a headache into a competitive advantage.

## The Role of Telemetry Pipelines

Telemetry pipelines are now a core part of observability engineering. They solve the scaling trap that every growing system encounters: too much data, too many tools, and not enough control. By collecting, normalizing, enriching, reducing, and routing with resilience and governance, pipelines turn raw telemetry into something you can rely on.

Today, telemetry pipelines are no longer optional. They are the backbone that makes observability, security, and compliance sustainable at scale. When you invest in a telemetry pipeline, you can expect lower costs, predictable spend, vendor freedom, and stronger governance. Your telemetry does indeed become a strategic asset.

# Conclusion

Mike's walkthrough of telemetry management is both thorough and compelling. While telemetry pipelines may not always be adopted as part of an organization's observability strategy, almost all organizations face many of the pain points that this elegant solution addresses. We hope this chapter gives you a better understanding of what telemetry pipelines can do for your team and why you should use them.

In the next chapter, we'll round out our technical deep dives into various aspects of observability by examining organizational approaches to managing your telemetry data at scale. Once you're capturing a large volume of telemetry across various services, environments, and teams, using ontologies can help mitigate the semantic failures inherent to diverse data.

---

# Ontologies as a Shared Language for Humans and AI

*This chapter is contributed by Frank Chen, CTO at Balto Energy  
and Designer-in-Residence at University of California  
San Diego (UCSD) Design Lab*

## A Note from Charity, Liz, George, and Austin

A shift toward a high degree of observability relies on high-context telemetry and data-driven analysis. Often, teams quickly discover that “more visibility” also means managing a lot more data. Once you’re capturing rich events across services, environments, and teams, you run into a familiar problem from the broader data world: *semantic failures*.

The same concept gets measured in multiple places, labeled in multiple ways, or defined with slightly different rules. At first, it’s simply inconvenient. But left unchecked, suddenly dashboards disagree, queries don’t line up, and nobody is quite sure which version of the “truth” is actually the real truth. This is a well-known failure mode in analytics and data engineering, and the observability domain is not immune.

This chapter introduces ontologies as a practical mechanism for establishing shared understanding across development, operations, and business stakeholders. An ontology is a structured map of a domain: its core entities, the relationships between them, and the rules that constrain valid behavior. In observability work, this map becomes operational through semantic conventions—standardized names and schemas that services emit consistently, so that telemetry encodes both system state and domain meaning.

In this chapter, Frank Chen demonstrates how to define a minimal ontology and how to reuse those definitions from development through production. It also extends

that approach to nondeterministic AI systems, showing schemas and invariants that provide guardrails and enable repeatable evaluation by “closing the loop” from production telemetry back into validation logic.

The rest of this chapter is written from Frank Chen’s perspective.

It’s 2 a.m. on a Saturday. Your pager fires. You open your laptop and stare at two contradictory dashboards. The infrastructure dashboard is all green: “99.9% uptime,” “latency < 200 ms,” “HTTP 200 OK.” According to the platform metrics, the system is perfect.

But the business dashboard is screaming. Customers are reporting bills that contradict the laws of physics—showing curtailed solar exports instead of crediting them. The AI agent you shipped last week to optimize grid interactions has started hallucinating price windows that don’t exist.

Nothing is “broken” in code. The servers are up. The API is responding. The AI is answering prompts. The failure is invisible to your standard observability tools because it isn’t a failure of *infrastructure*; it’s a failure of *semantics*.

In complex systems—especially those combining rigid rules with nondeterministic AI—data is everywhere, but truth is scarce. “Availability” can mean one thing to a developer (the server is up), another to an SRE (the latency is low), and something totally different to the business (the bill is accurate). I’m going to demonstrate how to fix that silent failure using *ontologies*.

Before exploring how to design a minimal, practical ontology for a nondeterministic AI system, let’s first take a look at how ontologies fit into observability.

## Ontologies and Their Role in Observability

Ontologies are formal, explicit specifications of shared concepts and their relationships within a domain. They act as both a common vocabulary and a knowledge map for humans and machines to forge a shared understanding of data and its meaning. For clarity, I’m not talking about knowledge graphs used by academics. I’m talking about defining a map of your domain—the core objects, relationships, and rules—and using it as a layer in your organization and, in particular, your observability data. This ontology is how organizations can guide their sociotechnical systems.

The goal of ontologies is to avoid fragmented definitions. Terms like “reliability” create contradictory signals that sow divisions and chaos between your teams, as shown in [Figure 17-1](#).



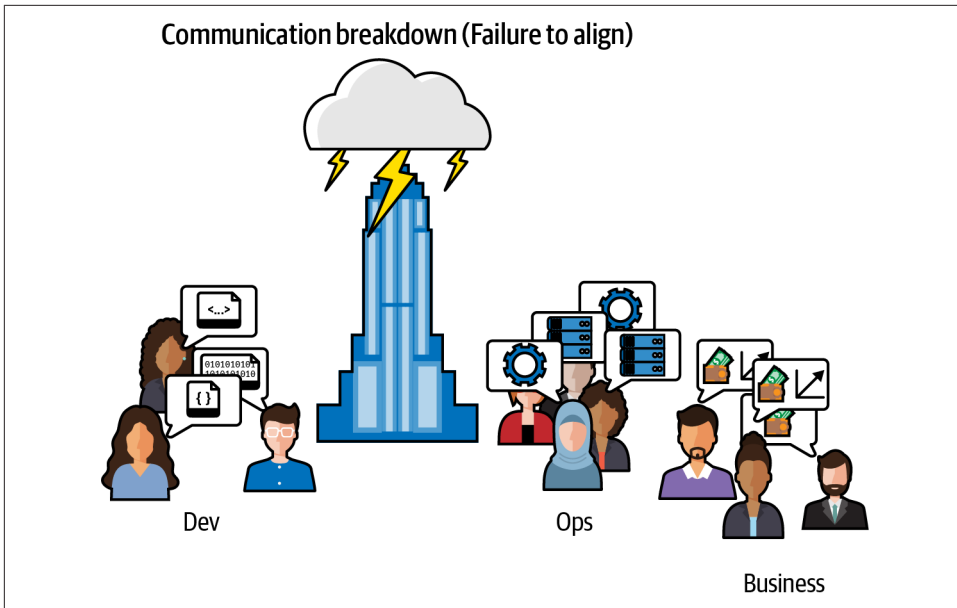


Figure 17-1. Chaos ensues when dev, ops, and business stakeholders lack a unified domain language.

Ontologies and semantic conventions are the missing layer in observability. For observability work, an ontology turns scattered metrics and logs into named, linked concepts that are codified in your team’s semantic conventions, runbooks, and documentation. By doing that, terms like “uptime,” “user,” “availability,” and “incident” will mean the same thing across multiple teams and tools. The practical payoffs are fewer alerts, faster diagnoses, and signals that match from development to production.

When using ontologies and semantic conventions, you gain the ability to catch errors that traditional monitoring misses, and you gain a powerful way to place guardrails around nondeterministic AI.

An ontology remains an academic exercise until it is codified. In the next section, we’ll look at how to design a minimal ontology.

## Design the Ontology

In observability, we implement our ontology through *semantic conventions* (standardized names, structures, and schemas that every service in your system agrees to use).

Think of these semantic conventions as a translation layer for your organization. For business stakeholders, they provide a clear vocabulary of “cost” and “usage” that maps directly to customer value. For engineers, they ground abstract technical metrics, like

latency or throughput, into actual business processes. And for the system itself, they serve as rigid schemas that turn ambiguous data into verifiable truth.



Semantic conventions through ontologies can be explored in frameworks for modeling additional entities, relations, and constraints such as [Palantir Foundry](#), [Apache Jena](#), [Apache Atlas](#), or [OpenMetadata](#).

To build a reliable system, we need to start by mapping our “nouns” (entities) and our “invariants” (rules). For our running scenario, consider an energy management system designed to help customers understand and optimize their utility costs. The system’s primary job is to ingest raw usage data and accurately price a bill based on complex, shifting utility rates. To add value, the system also uses an AI agent to generate billing proposals and optimization plans—such as suggesting when to shift appliance usage to save money.

## Define the Core Entities (the Nouns)

To ensure this system remains accurate and safe, we can reduce its complex world to a minimal set of fundamental objects and define their relationships in a map:

`IntervalSeries` (*the input*)

This consists of a timestamped series of raw usage data.

`Tariff` (*the logic*)

This is the deterministic ruleset (time-of-use windows, rates) effective for a specific period.

`BillSet` (*the output*)

The calculated result is produced by applying a `Tariff` to an `IntervalSeries`.

`ActionPlan` (*the AI proposal*)

This is a tentative object unique to AI systems that represents the system’s structured interpretation of the user’s intent. It bridges the gap between vague natural language (“save me money”) and specific system operations. The `ActionPlan` is untrusted until validated.

Once we have these objects defined, we can define the invariants.

## Define the Invariants (the Rules)

Invariants are the mathematical truths that must always hold. If an invariant is violated, the system is broken, even if no exceptions are thrown. The core rules are as follows:

### *Coverage (completeness)*

Every timestamp in the `IntervalSeries` must map to exactly one pricing window in the `Tariff`. No gaps, no overlaps in `pricing_coverage`.

### *Arithmetic (integrity)*

The `total_cost` in the `BillSet` must equal to the sum of its individual components (within a floating-point tolerance of  $\pm\$0.01$ ). The “whole” must strictly equal the sum of its parts.

### *Determinism (reproducibility)*

Given the same `series_id` and `tariff_id`, the system must produce the same `determinism_hash` (`BillSet`) result, every single time. A rerun of yesterday’s data must yield yesterday’s result.

In a traditional system, you might treat these rules as simple unit tests. But when you introduce nondeterministic AI, invariants become your primary line of defense. You can’t write a unit test for every possible hallucination that an LLM might generate. You can’t predict if the AI will suggest a tariff that doesn’t exist or a time-shift that defies the laws of physics.

Instead of testing the input (which is infinite), you use invariants to constrain the output. By treating these invariants as runtime assertions, you create a semantic firewall. When the AI generates an `ActionPlan`, we simulate the result and check it against the invariants. If the simulation creates a gap in coverage or an arithmetic error, we know—with mathematical certainty—that the AI’s plan is flawed, and we reject it. This enables you to deploy creative, nondeterministic agents into production while maintaining safety guarantees of a deterministic system.

## Visualize the Semantic Grammar of the Domain

Once you have defined the nouns and invariants, you can then map their relationships in a graph, such as the one shown in [Figure 17-2](#).

To understand this graph, follow the flow of data from left to right. The system begins with the `IntervalSeries`, which is the raw, physical truth of energy consumption measured by hardware. This data is inert until it meets the `Tariff`, which represents the logic and pricing rules effective for that specific time period. When you combine the physical data (`IntervalSeries`) with the logic (`Tariff`), you produce the `BillSet`—the financial truth.

The `ActionPlan` sits outside the deterministic flow as a potential modifier. Note how it doesn’t produce a bill directly. Instead, it proposes changes to the other entities—perhaps shifting consumption in an `IntervalSeries` or switching to a different `Tariff`. It is the “intent” waiting to be tested against the physics of the rest of the graph.

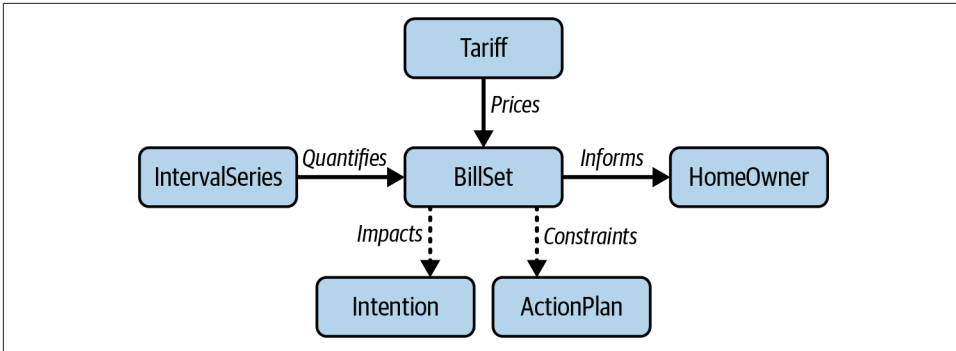


Figure 17-2. Ontology entity-relationship graph: this diagram visualizes the semantic grammar of the domain, defining the fundamental objects (nouns) and rules (invariants).



### Resources for AI Engineers

To dive deeper into distributed energy resources (DERs) and domain modeling, the Agile Electrification group at the UCSD Design Lab maintains open source playbooks. These resources include specific patterns for parsing natural language intent into structured ActionPlans and standard ontology definitions for energy. You can find them on the project's [GitHub repo](#).

Once you have the core entities, you can define the invariants.

## Glue the Schema Through Metadata

To implement this, you must establish *semantic conventions* that ensure every data point has clear provenance. For example, when you look at a `BillSet`, you shouldn't just see a dollar amount; you should see exactly which input and which logic version produced it. Provenance enables you to understand how, what, and why this thing exists in your database. Metadata is the glue that lets you see what created a `BillSet`.

The following is an example of the simplified schema for our *deterministic core*, focusing on how we link inputs to outputs:

```
// IntervalSeries (the Input)
{
  "series_id": "uuid-home-1234",
  ...
  "lineage": {"source": "smart_meter_api", "fetched": "yyyy-mm-dd"}
}
// Tariff(the Logic)
{
  "tariff_id": "uuid-tariff-55",
```

```

...
  "rules": {"type": "time_of_use", "windows": [{ ... }]}
  "ruleset_version": "2025.12.1"
}
// BillSet (the Output)
{
  "billset_id": "uuid-bill-100",
  "total_cost": 150.76,
  ...
  "lineage": {
    "input_series_id": "uuid-home-1234",
    "applied_tariff_id": "uuid-tariff-55",
    "calculation_engine_version": "2025.5.1"
  }
}

```

When building your schema, ensure that you:

- Map every output back to its specific input `series_id`.
- Version your logic (e.g., `ruleset_version`) so results are reproducible.
- Embed lineage directly into the metadata to automate troubleshooting.

By explicitly linking the `input_series_id` and `applied_tariff_id` within the Bill Set metadata, you transform a simple cost figure into a traceable audit log. This structure allows your observability tools to automatically correlate spikes in cost with specific changes in logic or input data.

By establishing these foundational schemas, we prepare the system to handle “parsed intent”—transforming vague AI suggestions into the structured snapshots required for the deterministic validation that we will explore in the next section.

## Schematize Intent with the ActionPlan

If your system uses AI to process or generate actions, you must also define a schema for intent. While the objects discussed previously capture what *actually* happened, the introduction of AI adds a new requirement: capturing what *might* happen and why. We cannot trust an LLM to “fix the bill” directly. Instead, we force the model to fill out an ActionPlan schema.

The ActionPlan schema acts as a constraint. It forces the model to convert “vibes” or conversational nuance into discrete, measurable fields that the deterministic system can validate. It is a *parsed intent*: a structured snapshot of what the AI thinks the user wants, as shown in this schema definition:

```

// ActionPlan (the Parsed Intent)
{
  "plan_id": "uuid-plan-99",
  "status": "PROPOSED",

```

```

    "predicted": {"bill_delta_usd": -94.50},
    "actions": [
      {"device": "ev_charging", "time_shift": {"start": "22:00", "end": "06:00"}},
    ],
    ...
    "lineage": {...}
  }

```

Notice that with provenance, you can reason about how the input (`IntervalSeries`) quantifies the `BillSet`. With the `ActionPlan`, you can reason about why the AI suggested a change before you ever allow it to execute.

With the nouns, invariants, and now the `ActionPlan` defined, you have established a bare minimum ontology. This core contract provides the foundation you need to begin enhancing your system with automated validation, CI/CD pipelines, and shared instrumentation.

## Validate the Contract: Continuous Integration

Evaluation of AI-driven actions and intent should be boring and repeatable. If you are building a system that relies on AI agents, you cannot rely on “vibes” or manual testing. You need a rigorous evaluation pipeline that treats your ontology as a contract.

This is one place where observability “shifts left.” In a traditional workflow, we often think of a vulnerability as something that happens after deployment. But with a strong ontology, we can observe the system’s behavior before it ever touches production data. By using immutable fixtures (e.g., a standard input `IntervalSeries` and `Tariff` for the “golden path” and “edge cases”), we can assert that our invariants hold true under every code change.

## Layer Three Gates for Defense

Instead of scattered lists of tests, we organize our CI gates into three distinct layers of defense:

### *Deterministic layer*

The first layer checks the rigid physics of the system. We verify determinism: given the same `IntervalSeries` and `Tariff`, the system must produce the exact same `BillSet` hash. If the hash changes, the code has drifted. We verify arithmetic integrity, ensuring that the `total_cost` strictly equals the sum of its components. Finally, we verify coverage, ensuring that 100% of the input data maps to valid logic windows without gaps or overlaps. These are binary pass/fail checks. If any of these fail, the build is broken.

### *Simulation layer*

Once the math is solid, we check the logic. For an AI agent proposing an Action Plan, we run a simulation. We take the agent’s proposed changes, feed them into our deterministic engine, and verify the prediction. Did the agent claim this plan would save \$20? The simulation must confirm that delta within a tight tolerance. If the AI hallucinates savings that the physics engine cannot reproduce, the gate closes.

### *Semantic layer*

The final layer evaluates the quality of the communication. This is where we use *golden dialogs* (predefined ideal interactions) to score the AI’s explanations. For example, we check for a confidence floor in which the system rejects any proposal where the model’s internal confidence score drops below a safe threshold (e.g., 0.6).



Evaluation pipelines and instrumenting LLMs are beyond the scope of this chapter, but see [Chapter 21](#) for a more thorough explanation of evals with AI, especially when it comes to framing up golden dialogs and scoring. Golden dialogs are a predefined ideal set of interactions across the given model.

## **Establish the Team Workflow**

When a gate fails, the ontology tells you where to look. If a structural gate fails, the engineer fixes the schema. If a coverage gate fails, they inspect the logic mapping. If a semantic gate fails, they tune the prompt or the retrieval context. By treating these failures as observable events—emitting the same JSON structure in CI as we do in production—we ensure that “fixing the build” is indistinguishable from “fixing the system.”

## **Create Signal Parity in Production: Continuous Deployment**

The ultimate payoff of an ontology with semantic conventions is *signal parity* within a continuous deployment (CD) pipeline that goes to production. In most organizations, the code that tests the system in CI is completely different from the queries that monitor the system in production. This disconnect means that a bug caught by a “unit test” feels completely different from the same bug triggering an alert.

We close this gap by ensuring that our operational dashboards hang directly off the ontology’s objects. We emit the exact same structured payload in production that we generated in our CI pipeline. This means that the logical assertion preventing a pull request from merging is the exact same query that triggers an incident in production.

## Understand the Hierarchy of Signals

Instead of generic “error rates,” our semantic conventions allow us to monitor specific categories of failure that map to business impact.

First, we monitor structural integrity. We track pricing coverage to ensure no intervals were missed, and we validate tariff effective dates to guarantee the ruleset matches the time period. A drop in coverage is not just a “data error”; it is a violation of the system’s contract.

Second, we monitor logic stability. We look for bill anomalies by comparing current outputs against historical baselines or “twin” simulations. We also track ranking drift—if the system suddenly recommends a different tariff for the same usage pattern, the logic has become unstable.

Finally, we monitor AI reliability. Because our AI workflow emits standardized `ActionPlans`, we can track the proposal acceptance rate and the evaluation pass rate in real time. If the model’s fuzzy-match score against golden dialogs drops, or if users start rejecting proposals at a higher rate, we know the model has drifted, even if the infrastructure is healthy.

## Create Shared Instrumentation: The Universal Payload

To make signal parity real, we define a set of shared instrumentation fields used in every workflow, whether it’s a deterministic batch job or a dynamic AI interaction. Every log line emitted by the system must carry the context required to reconstruct the decision.

This payload begins with identity and versioning. We log the `home_id` and `series_id` to bind the data to the user, but we also strictly identify the `tariff_id` and `rule_set_version`. This enables us to answer the question “Did the bill change because the usage changed, or because we deployed a new rule?”

Next, we include the invariant results. The payload explicitly states the `pricing_coverage` score and the `determinism_hash`. If a calculation fails, the log doesn’t just say error; it identifies exactly which invariant was violated.

For AI-driven workflows, we might enrich additional context for models and prompts in our observability workflows. We want to capture the `model_name` (e.g., GPT-5) and the `prompt_version`. By standardizing these fields, a “low acceptance rate” dashboard in production can immediately drill down to see if the issue is isolated to a specific prompt version or model variant.

By codifying these shared instrumentation fields, we move beyond theoretical mapping and into active system defense. This universal payload ensures that every component—whether deterministic code or a generative model—speaks the same

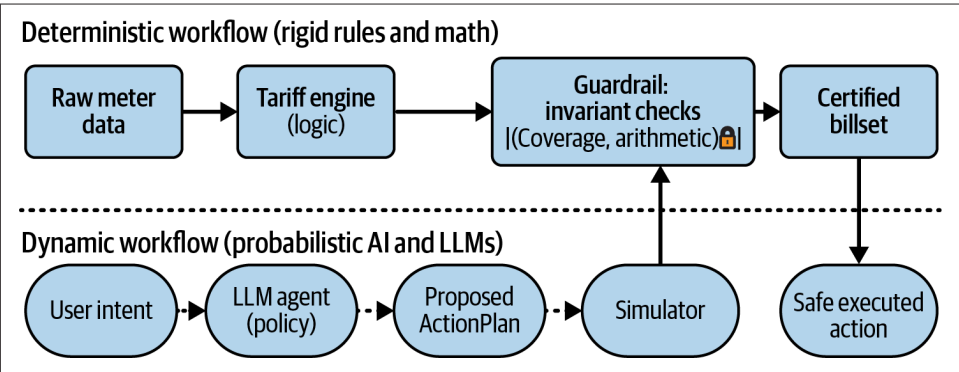


language. With this semantic foundation in place, we can now examine the “AI sandwich architecture” where these definitions serve as the critical interface between rigid rules and fluid AI intent.

## Implement the AI Sandwich Architecture

To operationalize the minimal ontology we have defined (comprising our nouns like the `BillSet` and our invariants like arithmetic integrity), we use the *AI sandwich architecture*. This architectural pattern *sandwiches* a nondeterministic AI layer between two layers of deterministic, rule-based logic. This ensures that the dynamic, AI-driven process remains grounded in the bedrock of truth provided by the deterministic workflow.

To guarantee reliability, we must strictly separate the systems’ physics from its imagination. By decoupling the “physics” (our immutable business rules) from the “imagination” (the probabilistic nature of LLMs), we prevent nondeterministic AI hallucinations from corrupting the core integrity of our financial data. [Figure 17-3](#) illustrates the decoupling of these two worlds.



*Figure 17-3. This dual-workflow architecture shows how a guardrail can create reliability. The AI workflow (bottom) must pass through its proposed actions against a deterministic rules-based workflow (top) for validation against invariant checks before any action is executed.*

We divide the application into two distinct loops, each with its specific responsibilities and observability needs:

### *Deterministic workflow*

The deterministic workflow is the boring, safe part of the system. It takes an input (`IntervalSeries`) and applies a set of rigid rules (`Tariff`) to produce an output (`BillSet`). It is mathematically pure; if you run it 10 times, you get the exact same hash 10 times. We use this workflow for everything that requires legal

precision: pricing a bill, comparing historical costs, or generating the “ground truth” for a simulation. Here, observability focuses on invariant enforcement. We monitor coverage, arithmetic integrity, and hash stability. If this layer fails, the code is broken.

### *Dynamic AI workflow*

The dynamic AI workflow is the creative part of the system. The AI analyzes the user’s context and synthesizes an `ActionPlan`. We treat this object as “intent capture”—a snapshot of what the AI thinks the user wants, frozen in a format we can simulate. Because LLMs are nondeterministic, this workflow is prone to hallucination. The AI might propose charging an electric vehicle (EV) during a time window that doesn’t exist, or it might optimize for a tariff the user isn’t on.

If we treat rules as guardrails, we never execute the AI’s proposal directly. Instead, we feed the `ActionPlan` into the deterministic workflow to validate it. We treat the AI’s output as a hypothesis and the rules-based engine as the test. We can ask:

- Does the proposed plan result in a valid `BillSet`?
- Does the simulation confirm the predicted savings?
- Does the plan violate any hard constraints (like 100% coverage)?

If the deterministic layer says “no,” then the AI’s proposal is rejected before the user ever sees it. In our observability data, we record this as a “proposal rejection.” This metric—the rate at which the guardrails reject the proposed plan—becomes our highest-fidelity signal for AI reliability.

Instead of executing unchecked proposals, the rules-based engine acts as a continuous validation layer, rejecting any output that violates our core invariants before it ever reaches the user, and sending a signal back to restart the loop. In our observability data, this proposal-rejection rate becomes our highest-fidelity signal for monitoring AI reliability in production.

## Close the Loop: Production Driving Tests

In the AI sandwich architecture, production is not just where code runs; it’s where test cases are born. We “close the loop” by taking the telemetry and edge cases captured in production and feeding them directly back into our development and testing cycles. When the AI creates a proposal that the rule engine rejects, that is a learning opportunity. We don’t just log the error; we capture the specific inputs (the prompt, the `IntervalSeries`, and the `Tariff`) that confused the model. We scrub this data of personally identifiable information and promote it to a golden dialog in our CI environment. This creates a continuous feedback loop where real-world failures automatically become the next generation of automated tests.

By treating production traces as future test fixtures, we create a data flywheel. If the model fails to understand a specific edge case—like a leap-year tariff change—we capture that artifact, add it to the eval suite, and block future deployments until the model passes that specific scenario. We track the “eval pass rate” as an SLI. A sudden drop in this rate is treated exactly like a latency spike: a reliability incident that requires immediate remediation.

## Putting Ontologies into Practice

To visualize how this architecture functions in a live environment, let’s walk through two specific scenarios using our example ontology.

### Scenario A: The deterministic comparison

Consider a user who wants to know if they should switch pricing plans. The system begins by binding the user’s historical `IntervalSeries` and fetching three candidate `Tariffs`. It then calls the deterministic engine to generate three distinct `BillSets`. Before returning any data, the system performs an invariant check, asserting that every single interval in the history maps successfully to a specific price window. Once validated, the system ranks the `BillSets` by `total_cost`.

From an observability perspective, the logs emit a payload containing the `determinism_hash` and the `ruleset_version`. This creates a perfect audit trail; if the user later calls support to ask why the numbers changed, an engineer can retrieve and replay the exact tariff version used in that specific calculation.

### Scenario B: The AI proposal

Now, consider a user asking an AI agent to “save me money.” The model generates an `ActionPlan` suggesting a shift of EV charging to 10 p.m. Instead of executing this immediately, the system enters a simulation phase. It creates a synthetic `IntervalSeries` (a counterfactual copy of the data modified by the proposed actions) and runs this counterfactual through the deterministic workflow to calculate the new cost.

The system then performs a verification step, comparing the deterministic engine’s calculated savings against the AI’s predicted savings. If the AI predicts a \$20 saving, but the engine calculates only \$0.50, the system flags the proposal as a hallucination and discards it. The observability layer records this event with a specific structured value like: `proposal_status=rejected` and `rejection_reason=simulation_mismatch`. This ensures that bad advice is caught by the system, not the customer.

# Conclusion

We started this chapter at 2 a.m. with a pager barking and dashboards lying. The root cause was disjoint language. Dashboards disagree, alerts duplicate, and the on-call engineers burn out because teams don't share the same nouns and verbs. Ontologies and semantic conventions are not just documentation; they help shape your observability strategy. These objects and invariants help transform unknown-unknowns into clear, actionable signals.

This dimensional thinking matters for software teams working with nondeterministic AI as part of their workflows. You should model input and output as untrusted, understand its respective components, validate workflows when possible with rules-based invariants, and use these invariants consistently.

Ontologies with semantic conventions won't make outages or observability challenges disappear, but they transform that 2 a.m. pager bark from a confusing hunt for disjointed data into a directed search. By following the advice in this chapter, you ensure that the next time you're woken up, you have the shared vocabulary needed to identify exactly which invariant was violated and fix the problem before the alert even hits your phone.

---

# Observability Use Cases

**Part IV** was about diving deep into implementation details. In Part V, we zoom out.

Here, we look at observability across different domains: how the techniques you’ve learned apply when the context changes. This section is for engineers who want to take those practices into their specific area of work.

When “observability” was first borrowed from control theory and applied to software, it was mostly about backend reliability. That’s no longer the whole picture. The concept now extends to data quality, security monitoring, compliance, capacity planning, and more—far more than we can cover in a single book. So, what we’ve focused on are the use cases where we’ve seen these practices actually work.

**Chapter 18** looks at applying observability techniques to CI/CD pipelines—how to debug and improve software delivery performance. Hugo Santos, CEO of Namespace Labs, contributed greatly to this chapter.

**Chapter 19** is a guest-contributed chapter by Hanson Ho, with an insert by Matt Klein, on frontend mobile observability. Mobile is a cardinality explosion: countless device types, operating system (OS) versions, network conditions, session lengths, and user behaviors. This chapter tackles what it takes to actually understand what’s happening out there.

**Chapter 20** is about performance engineering—how observability practices can help you gather baselines, run experiments, and quantify the impact of architectural changes. If you need to make your system faster, this is how you do it rigorously.

**Chapter 21** tackles the boss-level nondeterministic enigma that is ensuring the reliability of applications based on LLMs in production. This chapter examines both techniques for using observability practices to ensure LLM reliability and shows how to use production data to better refine your LLMs.

**Chapter 22** is a guest-contributed chapter by Kesha Mykhailov, Staff Product Engineer at Fin. It follows the journey of building and operating Fin, its AI agent—a story that pulls together many of the techniques from earlier in the book and shows what they can do together in the real world.

In **Part VI**, we shift perspective entirely, from practitioners to leaders, and look at how to build the organizational alignment that lets you actually move at AI speed.

---

# Observability for CI/CD Pipelines

*This chapter was written by Hugo Santos, CEO at Namespace Labs, and Liz Fong-Jones, Technical Fellow at Honeycomb*

Previously in this book, we've described the value of observability in accelerating feedback loops by enabling teams to understand what changes they need to make to software. But a barrier that often holds teams back from completing that feedback loop is how long it takes to actually get feedback on code that's under development or to push code into production. If it feels like you're operating the Mars rover manually with a minutes-long delay between putting in a command and seeing results, it doesn't matter how quickly you can analyze and get results. Observability and controllability are duals, and you will need to make sure both feedback loops are in balance.

To specifically define the terms that we will be using throughout this chapter, we define CI to be the process of creating a build artifact from your source code and inputs, and CD to be the process of rolling that build artifact out to production. Both are required elements for closing the feedback loop between writing code and running it in production, but organizations may have differing degrees of complexity in each.

In this chapter, you'll learn to look at the fundamentals of CI/CD through a new perspective, and understand how investments in observability can help you improve the performance of your CI/CD processes. We'll conclude with a case study showing how Honeycomb used observability for data collection to identify the correct places to innovate and improve build performance. Later on, in [Chapter 23](#), you will learn further about the feedback loop of feedback loops, and how tightening each feedback loop ensures your team can ship software quickly and predictably.

## Why Reliable and Fast CI/CD Matters

Let's say a developer in your team has just finished writing a piece of functionality and wants to get it landed and shipped to production. The first step is to get a pull request submitted. You know the next thing they're doing is waiting for tests to pass. And before tests pass, the test and production binary builds need to be completed. Because the [xkcd comic](#) about "my code's compiling!" still rings true in 2026, they'll use the moment to get a coffee while they wait. Now, with the appropriate beverage in hand, they see a test fail—only to pass on a rerun? It's probably a flake, safe to ignore (that's what the team folklore says, anyway).

The faster you make your build, and the more deterministic you make it, the faster you'll then be able to put your changes into production to get real feedback through your observability tooling. Then you can have the virtuous cycle come into effect by making any changes needed, pushing those out, and developing a stable rhythm.

CI/CD pipelines are, at their core, *production* systems for developer feedback. Developer time is expensive, so treating CI/CD as an afterthought or demanding anything less than production rigor results in wasted cycles. Yet they're often treated as unmanageable opaque boxes: if builds succeed, the system is "fine." But if your CI/CD platform is opaque to you as an internal customer, you'll never know why a build is slow, what caused a flake, or where to intervene, let alone why it costs so much.

## Build Observability Has the Best ROI of All Observability Applications

Your platform engineering team can magnify its own impact by identifying and eliminating bottlenecks to fast delivery of software into production. While human response time matters for things like code review and any manual processes involved for quality assurance (QA) testing, once QA is automated, the bulk of the delay between a developer writing code and that code serving real production users is time spent compiling, testing, and progressing through deployment stages.

Your company might operate services that process millions of requests per second, each of which may have only fractions of a penny of value. Without implementing sampling and other techniques to reduce telemetry volume, it can be difficult to see ROI from adding full instrumentation. But build systems are nothing like customer-facing high-RPS (requests per second) systems—every request by definition has a developer waiting on it to unblock them, so the request failing or being slow potentially wastes tens of dollars of compute and hundreds of dollars of developer time.

As of this writing, the cost of developing code to create a pull request is coming down, but is still far from zero. LLMs and agents cost tokens, and thus money, to create pull requests. And developers cost even more money in the form of their salaries.



Therefore, the number of requests to kick off a build will be relatively bounded and small, perhaps on the order of a dozen pull requests per day per developer at most, even with AI assistance. Scaling past that point would sacrifice human understanding of the code being written, with code reviews and quality checks becoming perfunctory.

This kind of traffic volume is an ideal use case for getting started with observability and seeing it in action for yourself, rather than waiting to implement it in your live production services. It easily fits in the volume that is possible to self-host and manage for open source observability tooling, as well as in the free tier of commercial observability solutions. And the returns for developer productivity can be enormous.

Observability brings light to this blind spot. It enables teams and organizations to reason about performance, reliability, and efficiency using real data, and not just gut feeling or institutional knowledge (like “it’s fine to ignore that test”).

## CI/CD Through the Lens of Observability

To answer questions, you first need data. And we can start simple, by looking at what the verbs and nouns are and how we might measure them. This serves both as a refresher on the basic elements of CI/CD systems, as well as a guide to applying observability to those basic components.

### The Ontology of Continuous Integration and Deployment

In [Chapter 17](#), you learned about the importance of ontologies, and defining clear verbs, nouns, and adjectives to describe your system and assigning consistent names to things. Now, let’s apply that to a CI/CD system and see how it overlaps with the ontologies of observability.

Each system has its own building blocks for orchestration, and units of work. GitHub Actions has workflows, jobs, and steps. Buildkite has pipelines, builds, jobs, and steps. And the open source systems of Argo and Concourse have units of applications, projects, and states. Workflows connect back to pull requests, and provide the critical signal that developers are waiting for. What might be topically “are my tests passing?” is in fact “do workflows X, Y, and Z pass?” Each workflow then subdivides into multiple jobs that can execute concurrently, as long as each of their prerequisites is satisfied.

In the world of CI, each Job may have multiple steps that execute serially within the job. In turn, each step might actually be a command to execute part of a test suite comprising multiple tests in a testing framework. For CD systems, jobs might include steps to fetch credentials, push config files, mutate production, or otherwise actuate the deployment of a change into production in order to make the desired state match the current production state.

Fortunately, these concepts also map well to spans and traces in the production observability world. Each workflow is a single trace. Each job is a set of spans subordinate to the root span for the workflow. And each step within the job is a span within its contained job. And you likely have the detailed logs that emerge from executing each individual step as well.

If you are a developer, the choices you have control over tend to be at the execution level, not at the orchestration level. Typically, this means the jobs that run your test code and build your binaries. And because they run only your code, that's the simplest for you to change first and improve your own experience.

As a platform team, you may have additional points of leverage—for example designing the orchestration and how and where it runs, and on what units of work. For instance, you might be able to design batching systems to run multiple pull requests together and only break them apart if one request fails. Or, you might be able to conditionally run tests depending upon whether relevant code was modified. However, we'd encourage you to start simple and only add complexity if it results in substantial productivity savings.

## Instrumentation Basics

Once you've established a clear ontology for your CI/CD system, the natural next step is to begin collecting the data in your observability tooling. Start by exporting individual job run data from your CI/CD solution: when they start, when they finish, and whether they've succeeded. Fortunately, this has straightforward analogies to trace spans in the broader observability world. Each job has a name, has a start and end time (or start and duration), belongs to a collection of jobs in a workflow, and may have other attributes worth tracking.

One of the most important pieces of telemetry is which repository and code branch is being passed through CI. Branches are important because they play a key role in how a developer is bringing changes to the codebase. Whether tests are running for `main` or for a pull request branch, for example, may play a role in how important that particular set of workflow invocations is and what your performance standards may be. If someone writes a single inappropriately slow or flaky test, that will slow down their invocations, but not those of the rest of the team. However, if that change merges and now everyone's build is slow, including builds for the main branch, now you have a serious problem.

Here you can rely on the integrations available to your CI system, or you can make the integration yourself. There's usually a well-defined “first step” and “last step” you control where you can emit events from (e.g., “this job started” and “this job ended”).

If you are on a platform like GitHub Actions, exporting telemetry data can be as simple as using an OTEL action that exports Job and Step duration to your observability

stack. Empowered with the information in these basic metrics, your teams can start deriving expected behavior, and should be able to answer questions like “How long should this take?” and “Does this job fail often?”

## Defining Service Level Indicators and Achieving Predictability

Data on typical success rate and overall latency serve as your first implicit SLIs and SLOs. This understanding transforms the CI/CD system from a mystery into a measurable process, enabling you to track its behavior and act on changes. Rather than accept what the opaque box gives you, you can understand it and control it. At minimum, setting these expectations enables you to detect and prevent regressions even if you’re not yet ready to invest in an overhaul of your entire pipeline.

Being empowered with data transforms the conversation: with your customers (the developer teams), the feedback is no longer based on “feel,” but on real data (e.g., “job duration has been degrading over time”). And you can show your leadership team where the real problems are, and motivate the work necessary to fix them.

In a CI system, we tend to define performance as the “end-to-end user time” that it takes from start until a change has been validated. And it best tracks the experience of our customers. Developers are waiting for software to be validated so that they know they’re done with their change and that code can be submitted into their repository. Just like in a production system, you want to think through and define what’s important for your customers (and who your customers are).

Even if they don’t directly impact the typical latency of a CI/CD run, you may also want to track metrics that impact cost or predictability. For instance, you may want to ensure that only a small percentage of tests and associated runs are flakes (as measured by percentage of successful retries), or ensure that your CI presubmit hooks are working correctly by tracking test failure rates on your main branch. For cost, resource usage can help you ensure you are pacing acceptably against your CI provider’s commitments to avoid surprises with wasted contractual spend or unexpected charges.

## From Jobs, to Directed Acyclic Graphs and Traces

Collecting data at the job level is the simplest entry point, as we control code that is run as part of the job. But we know from before that jobs only tell part of the story. Pull requests wait on the entire workflow to finish, rather than the individual steps or jobs that comprise them. While steps within jobs run sequentially, the workflow in the CI platform winds up growing into a directed acyclic graph (DAG) of multiple jobs, where each job can only depend on those before it.

These DAGs tend to emerge naturally. It’s expensive to always build a container image used for testing every time you want to spawn a different set of tests, so you

add a prebuild job that builds that container image once, and makes it available to subsequent jobs. You now have a dependency: if the prebuild job slows down, everything else depending upon it also has to wait. If the prebuild job fails, the subsequent jobs can't run. And if the subsequent jobs don't run, you don't have data on them.

By mapping your CI/CD ontology to the primitives of distributed tracing, you move away from team folklore about the slowness of the build and toward a measurable process where every workflow has a verifiable result. However, measuring a job in isolation only tells part of the story. Developers don't wait for individual steps; they wait for the entire validation sequence to finish. In modern delivery, these sequences naturally evolve into a DAG of interdependent tasks. To understand why a build is truly slow, we must move beyond the list of jobs and visualize this graph as a trace to identify the critical path—the only place where optimization will actually yield a faster feedback loop.

## Making Improvements and Measuring Them

As your product infrastructure evolves, these workflows tend to become more complex. You don't just have one container image prebuild; you have five. You have jobs with multiple dependencies and jobs that are dependent upon them. You are now executing a series of jobs with well-defined dependencies. And success is defined as all of the terminal jobs succeeding. The split into different jobs affords you the opportunity to rely on pipelining and parallelism of expensive steps. Why is this complexity worth it? And how can you measure the results?

Pipelining is an important property to increase the performance of your CI/CD platform. But it comes with trade-offs. In a world of serialization, each step would run after the other, and it would be simple to understand how far the workflow is. But builds can take tens of minutes, and end-to-end suites can take hours. If you serialize them, it may take a long time until you can start deploying.

So you lean on pipelining, and end up running multiple expensive jobs in parallel. When you do, observing the performance and behavior of a single job in isolation no longer tells the full story. For example, if your systems are composed of a React front-end, a Go backend binary, and a Rust backend binary, the team working on the Go backend binary may flag that their builds got slower. Perhaps they started depending on a larger Go library and are spending more in the compilation phase. But when you zoom out, if the Rust build dominates the run time, the small differences to the Go build end up not playing a role on the overall performance of the workflow. The overall duration remains unchanged.

So yes, you have a slow Go build, but your real problem lies elsewhere. You just found your critical path. If you had spent time optimizing the Go build, you still wouldn't have changed the end-to-end time.

## Understanding Performance: Treating Continuous Integration Like Production

A CI job is fundamentally a production workload running on a deadline. While capturing the duration of individual steps helps us see that a build is slowing down, reducing the critical path requires the same forensic rigor we apply to performance in production. Like any production system, your pipeline may be constrained by CPU, memory, input/output operations per second (IOPS), or even just input/output (I/O) (e.g., calling other systems). Your production jobs likely run the same codebase that your test jobs do, but with the added pressure of doing a lot of work in a small period of time.

To move beyond guessing why a job might be slow, we must be able to correlate our traces with host-level metrics and logs to identify where our toolchains are hitting their limits. Only experimentation and collecting data can tell you the truth.

The following are examples of how experimentation can reveal bottlenecks:

- A team assumed that increasing their runner resources (i.e., number of cores) will fix a slow JavaScript build. Their instrumentation data shows the build time remains flat. The truth is that the job uses webpack, but without the required configuration (e.g., `ThreadLoaders`) to benefit from parallelism. The extra cores sat idle while the build remained single-threaded.
- A team assumed that their Go build is as fast as it can be on their current hardware. After doing a trial run with more cores available to their Go jobs, they see that the build time was cut in half. This revealed that the native support in Go for parallelism makes it highly elastic and it can be effective to buy speed with more CPU.
- A team was unable to push releases to production quickly despite having very efficient incremental builds. They were pushing container images to all regions from a single source job and were network-bandwidth constrained rather than host constrained.

By effectively leveraging your observability into your build system, you can optimize what truly matters rather than guessing what to improve upon next or admitting defeat.

## Predictability: Real-World Trade-Offs

Many of the build- and test-related systems we rely on are built around the idea of producing predictable outputs from a well-defined set of inputs. When that relationship can be trusted, it can be cached.

For example, pulling the same container image where the digest is known will always yield the same bytes. That property is not an implementation detail; it is a correctness contract. And because of that contract, pulling the same digest once and reusing it across multiple runs doesn't compromise correctness. It is a simple and effective way to improve incrementality.

This pattern shows up everywhere in CI systems. If the inputs are stable, and the outputs are deterministic, you can safely remember the result. And remembering the result is often the difference between a fast system and, well, a slow one.

But reproducibility doesn't exist in a vacuum. External constraints and shared resources can limit it. There is a constant tension between doing more work in parallel and fully utilizing resources for cost efficiency. Packing more tends to lead to higher contention, and higher contention tends to lead to more flaky tests or jobs that intermittently fail. At that point, the question is no longer purely technical. Instead, it becomes “does your engineering culture tolerate flakes?”

When your tests are reproducible, resource constraint-based flakes are effectively a tail-latency tax—retries don't disappear; they add time. A useful way to reason about these trade-offs is to compare P90 versus P50 latency. A singular view of individual jobs or workflows may not surface this information, but treating each workflow run as a single trace and applying modern observability techniques can help you identify trends in P50 and P90 and make the right choices for your developers.

Managing resource constraints is also uneven across toolchains. Some ecosystems make this easier than others. For example, `go test` makes it relatively straightforward to scale across additional cores. End-to-end browser testing, on the other hand, is much harder to parallelize effectively and often ends up constrained by memory, I/O, or external services rather than raw CPU.

One way teams offset this is by relying on ephemeral, dedicated resources. Instead of packing unrelated jobs together and hoping for the best, they provision short-lived environments with known capacity. This can look more expensive at first glance, but often ends up cheaper once you account for retries, investigation time, and developer wait time.

Because builds and tests tend to have a high fan-out and batch-oriented shape, you should also expect to see a countable number of failures increase as fan-out increases. This can happen as your engineering team grows or as coding agents start producing changes at a higher rate.

The key question then becomes: are these real failures, or are they infrastructure failures? The more predictable your build and test runs are, the easier it is to answer that question. For example, a subtle synchronization or timing issue hiding in a test might never show up until your underlying hardware platform changes and everything suddenly runs faster.

Because CPU is an elastic, time-shared resource, the more robust your tests are to timing variance, the better you can utilize your infrastructure. That robustness directly enables higher packing ratios: more work on fewer CPUs—without paying the flake penalty.

To reason about these limits, make sure you instrument OS-level metrics, or work with a provider that does. Your jobs are subject to many constraints beyond CPU and memory: open file descriptors, kernel shared memory, socket buffer sizes, and more.

When running integration or end-to-end tests, an effective way to understand limits across all of your services is to rely on containers and collect cgroup-level metrics. That gives you visibility, not just into your own process but into the aggregate resource usage of your dependencies too.

At this level, CI observability starts to look a lot like production observability—and that is not an accident.

## **(The Lack of) Incrementality: The Bane of Continuous Integration's Existence**

If you interview a bunch of developers, you may hear a lot of pushback on CI performance. You may even hear things like: “I run our whole test suite in 5 minutes on my MacBook and CI takes 20 minutes; can we just run our CI in our Apple machines?” You can, but that’s a story for another day.

CI should be fast. Why is it not?

Many workflows start simple, and for the sake of reproducibility, they do the same work from scratch all the time. That’s actually good: you want reproducibility. You want to know that your builds and tests run consistently and are not dependent on state left over by other jobs. But starting over from scratch every time is expensive.

Thankfully, many toolchains have a solution to this problem: they maintain intermediate files. Using language and toolchain-specific semantics guarantees a certain level of correctness in that only intermediate files produced from the same inputs end up being reused (if, that is, you are persisting the cache between workflow runs).

Go, for example, has module and build caches. If you build the same Go program with a small change to one of its source files, your build will be fast because the remainder of the output files are going to be reused. Leaning on the incrementality



of your toolchain (and its correctness guarantee) enables you to have the best of both worlds: manageable isolation, with great performance.

## Keeping Your Build Performance Tight and Your Developers Happy

When you're able to measure build performance, you might start seeing a relationship between tail latency (i.e., your P90 job duration or higher) and flakiness. For certain categories of tests, finding full reproducibility is hard. And it ends up being costly to ensure consistent behavior from all of your tests.

The more complex the test, the more components it exercises, and the higher the chance that you'll see nondeterminism. Many workflow solutions have a magical solution: retries. You have a job that fails 5% of the time (you measured), so you enable retries (either on the entire job or sometimes on individual failing tests within the job), and you barely see failures. But your tail latency just shoots up. Your P50 barely moved, but sometimes you end up waiting twice as long. Only by integrating observability into the wider range of runs can you understand how often things fail and whether the delay to retry a test or step is worth it.

And that's the hard trade-off. Different teams will be at different levels of maturity and will decide differently whether this is a problem or not. For some, confidence that it "mostly" passes is fine. Perhaps you have other ways to test the behavior of the changes being made that bring you extra confidence.

An interesting perspective here is that your tests are your first line of representation of production. If something fails in your tests, it may be a factor of the test harness, but it may also be a real problem that you'll find only later in production.

## Case Study: The Importance of Quick Build Times at Honeycomb

At Honeycomb, we believe in ensuring an awesome developer experience for our own engineering teams; it's the only way that we can compete with companies that have way more engineers than us. We've historically tried to keep our build times under 15 minutes, taking percussive action when build times went over that. In fact, we have an internal SLO on the data, which alerts us if more than 5% of builds take longer than 15 minutes!

But it turns out that 15 minutes is still not a great experience; devs will task switch rather than wait for builds to complete, losing valuable context. Ideally, build times under six to seven minutes keep developers around to watch their builds, ready to rapidly correct CI failures. The mental state that's tracking implementation details—kept in a developer's head—never gets paged out, as they don't feel the need to turn to Slack or pull the next ticket while waiting.



Could observability potentially help us here, too?

## History of Improving Build Times at Honeycomb

Observability-driven optimization of build times and reliability has enabled us at Honeycomb to keep our developers happy and productive. Observability-driven development is not just for code; it's for any sufficiently complex system! And we would argue that our build system is one of our mission-critical complex systems just as much as our storage engine is. If our build system goes out, we cannot deploy new code, and pull requests from our developers begin aging like milk. Undeployed code is perishable.

Between June 2018 and May 2019, the median amount of time it took to build Honeycomb doubled from 7 minutes to nearly 14 minutes. This prompted us to switch CI orchestration providers from one that had limited support for parallelism to one that offered much better native support. Without having adequate observability and longer-term history tracking, we would not have been able to identify the issue before it reached a boiling point of developer frustration. Tracking SLOs on build time enabled us to avoid having regression over time—the “boil the frog” effect, where developers feel frustrated but lack evidence beyond anecdota.

From 2020 to 2025, we went from 10–20 pull requests landing on the main branch per week to 100+ consistently every week. The total number of lines of code in our codebase approximately doubled over those five years, increasing the amount of work incurred both for compilation and for unit testing. Amdahl's law became a limiting factor even as we attempted to add parallelism.

There again, being able to trace execution to identify slow steps and success-after-failure job spans enabled us to determine our flake rate and what portions of the build to optimize next. We needed to find the right places to intervene rather than making off-target optimizations that would not impact the total critical path latency.

Eventually, we hit the point of no longer being able to brute-force apply more parallel workers or shorten build steps. We needed to actually rethink our entire caching strategy. How did we determine that we were unable to proceed any further? Amdahl's law states that even if a parallelizable step is shrunk to take no time through infinite parallelism, the sum of all the remaining serial steps will dominate the critical path and minimum time needed to run the workload.

Because we had traces from our CircleCI `buildevents` instrumentation, we could study them to understand where all the time was going. The tests actually were some of the faster parts of the build, and it was the binary build and artifact creation and upload steps that were the slowest. Why?

## Build steps and data transfer

To our surprise, the main issue holding back our performance was transferring files between different build steps. Nearly 3.5 gigabytes (GB) of data were being passed between intermediate steps and the final jobs that uploaded the built artifacts. We discovered that the data was being *gzipped* rather than compressed with *zstd*, so meaningful size reductions purely through compression were not possible.

Instead of trying to compress the data, I (Liz) started thinking about whether it was possible to entirely avoid moving all that data around. The primary artifacts we need to ship to our Kubernetes cluster are Go binaries and bundled-and-minified JavaScript built with Webpack. Those get compiled into images pushed to Amazon Elastic Container Registry (ECR), as well as archival unstripped copies of the binaries uploaded to S3. But with the move of all our services to containers and Kubernetes, there was no longer a need to upload stripped binaries to tarballs stored in S3, nor to make those binaries part of the ECR images. What if we could just build everything in one single step and avoid shipping state around?

## Docker bake

The first thing that we considered was an Omnibus build step to do the Go build, then the JavaScript build, and then bringing it all together into build artifacts. While this minimized data transfer, it was horrifically inefficient to run the steps in serial, waiting for each step to finish before starting the next step. My (Liz's) research then took me toward Docker Bake, a relatively new method (introduced in 2019 and popularized around 2022) of chaining together multistep Docker image builds.

## Parallel builds

By allowing steps to progress independently—and blocking only when they needed output from a parallel build step that had not yet finished—we were able to effectively interleave multiple parallel builds on a single virtual machine. You might expect that the CircleCI built-in support for Docker layer caching (DLC) to be helpful here. However, our experience was that passing state via object storage in the CircleCI workspace was slower than we would've liked. Compressing and uncompressing indiscriminate amounts of cached data via DLC would take longer than either creating the state from scratch or passing in base layers generated from periodically built and explicitly cached tarballs.

## Removing bottlenecks

Migrating the steps from separate CircleCI jobs run on the Docker executor into a single Docker Bake on a CircleCI machine executor allowed us to establish the floor of our build time. We then carefully adjusted parallelism to make the tests (which still run on standard CircleCI parallel runners using the CircleCI parallel test sharding

helper) take approximately the same amount of time. We spent a lot of time staring at graphs generated using `buildevents` instrumentation data created to understand the latency distribution of each step and ensure that no single job would become a bottleneck. For instance, we discovered that steps like initializing our test database for Go integration tests—previously having little effect on total build time—now needed to be optimized themselves.

## Caching network fetches

We also were able to dive into individual traces to identify if there were specific steps contributing to tail latency, and squished those down through more aggressive caching. For instance, we found that the necessary steps to update the system dependencies for Puppeteer usually ran fast but could have enormous variance. So it was better to pay a flat tax of a few seconds to cache the `apt` lists and `.deb` packages from the day's first run instead of redownloading every time.

## Results

One of the really cool things that happened during our investigation was stitching together `buildevents` traces at the CircleCI job and step level together with the actual Docker BuildKit command tracing data. It turns out that BuildKit has integrated OTel support since its inception, so by setting the `TRACEPARENT` environment variable to match the `buildevents` trace and span IDs, we were able to directly observe each individual command execution, ECR upload step, and blocking while waiting for other steps to complete.

The payoff of having build times consistently below seven minutes was well worth the effort. These benefits have compounded as the volume of AI-written code and pull requests have increased. Being able to validate code as fast as a language model can spit it out is a competitive advantage.

## Applying These Lessons to Your Build System

Now that you've learned about how we achieved engineering productivity wins from investing in our telemetry and observability in our CI/CD pipelines, here's how to achieve similar results from your build systems.

The first step, as outlined earlier in the chapter, is to simply know how bad things are at the top level. You need to know how long each build step is taking, even if you can't yet decompose the build step into each constituent piece. After all, it may not make sense to add fine-grained instrumentation to each individual command of a job that takes one second, when another job that blocks build progression takes two hours to complete!

In order to do this, export data from your build system using whatever mechanism it natively supports. While at Honeycomb we were using a legacy Honeycomb-specific tool called `buildevents`; OTel is now the best way to instrument new build workflows. For instance, GitHub Actions can integrate with OTel in one of two ways: either via an **OTel Collector receiver** that pulls data using a GitHub API key, or via a **GitHub Actions add-on** that takes an OpenTelemetry Protocol (OTLP) destination to emit telemetry about the currently running job. Systems such as BuildKite natively export OTel traces.

For tracing individual build steps, you can use `otel-cli` as a wrapper. The basic syntax is to run `otel-cli exec --name "human-readable span name" echo "do stuff"`. Then, `otel-cli` handles the serializing of the tracing state and any attributes you may set to environment variables (and optionally a file on disk). For instance, you can nest execution by running a shell script inside of `otel-cli`, and have commands in the shell script themselves traced with `otel-cli`. If build steps traverse the boundary of a single machine, you may need to manually serialize the state and pass it along rather than rely upon the `TRACEPARENT` environment variable or the local filesystem.

But what about if your build system is partly or entirely written in Docker or produces container images as an output? Given that a majority of cloud native development deploys onto Kubernetes, and thus requires creation of containers on container registries, this is not an idle concern. You'll have a big step named `docker build` or `docker bake` in your build telemetry, without the ability to break it down more granularly.

Instead of trying to bring `otel-cli` into our `docker build` steps and wrapping each command, Docker supports natively exporting OTel **within the docker client itself**, or the **underlying build and buildx servers** that the docker client calls. For the docker client, you can set the `DOCKER_CLI_OTEL_EXPORTER_OTLP_ENDPOINT` variable inside of your CI environment before Docker commands are invoked. If you do this, you shouldn't also use `otel-cli`, as it will result in duplicate instrumentation.

Tailor your instrumentation to your current build system; use the simplest abstractions first, and only go to manual instrumentation for trace spans if your CI system does not have built-in automatic instrumentation at the granularity you need. And just as you would add in custom attributes in a production system, you should also add in custom attributes tracking the sizes of outputs, the amount of cached data, whether the module cache was a hit, the number of files using deprecated or legacy patterns, and so forth, to get even more value out of your CI data.

# Conclusion

Build systems and observability are both foundations of the development feedback loop. By treating build systems as a production system and using the same observability tooling to understand it, you can eliminate bottlenecks to software delivery. CI/CD systems can be both expensive and brittle, but observability helps you gain an understanding of them and mitigate risks over time.

In the next chapter, you'll learn about how to apply observability to the opposite end of the delivery cycle: the last mile of mobile, web, and frontend systems.



---

# Observability for Mobile and Frontend

*This chapter is contributed by Hanson Ho,  
Android Architect at Embrace*

## A Note from Charity, Liz, George, and Austin

We continue our look into observability use cases by exploring the world of mobile and frontend applications. Backend applications and infrastructure are generally controlled: you have a known OS, predictable resources, uniform networking, stable time synchronization, etc. With frontend or mobile systems, you generally inherit the environment: thousands of device models, OS, and browser versions; unreliable cellular networks; resource throttling; captive portals; ad blockers; and concurrent processes.

When you control deployments, configs, and sampling, and can correlate across consistent infrastructure, it's safe to say that observability happens within bounds: the problem surface is mostly within your backyard. When dealing with frontend or mobile applications, observability happens beyond the fence and in actively hostile territory. You can't control the runtimes, networks, or versions, and you have to be extremely careful with taking any action that impacts device performance, available bandwidth, user privacy, or cost.

As such, many of the observability practices discussed earlier in this book must be adapted to fit in the frontend and mobile worlds. This chapter, written by Hanson Ho, details considerations for observability in frontend and mobile. Matt Klein also contributes a section on considerations for telemetry management for mobile systems. We begin with Hanson's perspective.

# Status Quo for Mobile and Frontend

For developers who work on mobile and web apps, the term “observability” is still relatively new—if they know it at all. Those familiar with the term may mostly think about it as an evolution of monitoring and ops practices for distributed systems. For frontend or mobile engineers building websites and apps, it may seem synonymous with the production telemetry they are used to—like stability issues or user actions like taps and page navigation. The latter is commonly referred to as real user monitoring (RUM), error tracking, or crash reporting when used by development teams to determine what the user is doing in the app when problems occur.

On mobile and web, this data is often aggregated at the client and then sent to a backend system to compute familiar metrics like daily crash rate and time to first paint. These metrics summarize the state of the app in production and provide insight into common problems users are experiencing. Unfortunately, the ability to slice and dice this data is not great beyond some easily obtained dimensions like locale and app/OS versions, making it difficult to find particular cohorts or ask interesting questions when looking for deeper insights.

Production telemetry for mobile and web apps is primarily used in two ways: to surface and quantify problems at runtime, and to provide additional datapoints to help developers reproduce the problems internally. This is done by examining specific problem reports along with metadata describing the execution context and RUM events on the same device around the same time, in order to narrow down the code path that could have led to the problem. Investigations tend to require access or knowledge of the underlying code, as the data itself is generally not enough.

In classic monitoring approaches, the nature of the instrumentation (where problems are identified on the device and reported) means that you generally have to know the kinds of problems you are interested in finding beforehand. That means that this kind of telemetry is not expected to be used to answer unanticipated questions. The mobile and web app debugging workflow relies heavily on first reproducing the issue, and then debugging the root cause locally. Luckily, unlike distributed systems, even the trickiest of problems (if identified) can be reproduced on a single device. We are, after all, dealing with a single app running on a single device.

Here’s an example of how this data is used:

1. A few seemingly related crashes, identified by similar stack traces, happen across a variety of app screens. The top of the stack trace suggests a `NullPointerException` (or NPE) during view-tree traversal.
2. The crashes are a minority of cases—most of those screens load correctly without error.



3. RUM data can be used to find correlations with these crash events: looking through the RUM data of a few sessions that ended in this crash, there's always a click on a specific button whose label contains non-ASCII characters.
4. With that knowledge, you examine the source code for that button and find an assumption about the nature of the string used for the label. If it contains non-ASCII characters, it triggers a code path resulting in an NPE.
5. By combining the recorded crash telemetry and the RUM event correlation via a session, you can pinpoint the precise conditions needed to reproduce the issue and identify the root cause.

While production telemetry can be used in this way to surface some app stability issues, more subtle problems that are harder to determine like client-side performance degradation aren't typically reported, unless it's automatically collected by first-party tooling. There are a number of reasons why this is the case, but none of them are that mobile and web developers don't care about performance; they just approach the problem differently.

For the most part, performance changes on mobile and web are monitored through the use of synthetic benchmark tests that run locally or in CI (e.g., macrobenchmarks on Android), or through a synthetic testing suite. These tests are preferred to production data by devs because they overcome the biggest shortcoming of production telemetry: the inability to control the execution environment. Synthetic performance tests enable devs to lock down key performance determinants like hardware and device state, which means runs are more repeatable and the iterations are more comparable. With that setup, you can detect small but statistically significant changes more easily, without requiring the large sample sizes and magnitudes of change needed if production data were used. The downside of this is that your coverage is only as good as the workflows that are benchmarked and tested. This makes this a reasonable but ultimately imperfect solution, especially if your goal is to understand how the user is experiencing your app.

Another kind of telemetry collected in production is what is known as “product analytics.” Unlike telemetry used to identify and debug issues, this data is used by product managers and other parts of the business to determine which features of the app the users are actually using. Interestingly, the line between this type of telemetry and RUM and stability events can be quite blurred, as they tend to model the same app and user behaviors.

The difference between the two is often not what is being proxied, but how the recorded telemetry is actually used, as well as the types of tooling and vendors involved in collecting this data. For instance, the same page view might be logged by a product analytics vendor's SDK so that product managers can create DAU (daily active users)

graphs for their feature, and also by a RUM vendor's SDK so that developers can see the events leading up to a crash.

But product analytics are usually not connected to the production telemetry used by developers because they go into a totally different data set. This is why a common and frustrating problem you might encounter on a mobile and frontend team is the irate product manager, wondering why a key metric of theirs dropped since the last version was shipped, with a team that can't explain why because all of their own dashboards are green.

## Instrumentation Limitations

Mobile and web apps are used in contexts that are under constant change, so even the most stable and performant ones are not free of problems that impact user experience (UX). Many of these, like crashes, can be and are routinely detected and resolved using production telemetry, aided by RUM data and contextual metadata to narrow down the reproduction workflows. For important events and workflow performance that are reported as telemetry and properly contextualized, engineers that know the code should be able to quantify and solve any problems they reveal.

But one limitation of the production telemetry that's collected on mobile and web apps is that they can be really noisy due to the variety of factors that can impact runtime. To mitigate that, you need to model all the relevant factors that affect runtime and record it as metadata along with your telemetry. This process can be difficult given the wide array of relevant contextual information, but unless that is done, problems that only impact certain users may not be easily discoverable. Without knowledge of the precise dimensions that are material to a particular workflow, you can't distinguish between your user cohorts or tease out the signal from the noise.

In practice, only the static, easy-to-capture metadata on mobile and web are captured, which often leaves the dataset lacking enough context to do all but the most basic of filtering. If you want to group by app version or language, for example, that should not be a problem. But if you want to sort users by something difficult to get and model like network connection strength, or something ephemeral like UX, you're generally out of luck. Often, it's key dimensions like this that are difficult to obtain that provide the most useful insight in your analysis of app performance.

This leads to the obfuscation of major problems that greatly (but narrowly) impact a small number of users. For example, an issue that impacts only 1% of your users may not move the P50 very much—maybe not even P99, depending on the magnitude of the change. The reason why this is happening may also not be clear, even if you have all the right exemplars.

Problems like this on mobile and web can't merely be deemed outliers because those who are affected, no matter how small a percentage of your user base they represent,

are totally and completely affected. That 99% of other users aren't impacted matters not at all to them. If your DAU is 100 million, and 1% of those users are experiencing performance that is 2x worse, that's a million people who are affected. This means that even if you have the right instrumentation to surface a particular problem, you may not be able to see it easily.

And therein lies another problem: in practice, many important events and workflow performances aren't even captured as telemetry. Typically, only obvious errors like crashes or common workflows, like app startups that can be automatically captured, are recorded as such. But we all know the myriad ways a mobile or web app can perform poorly without crashing that leads to user frustration, like when some important workflow is too slow for the user (by some unknowable definition). And if that slowness isn't from a workflow that can be automatically instrumented like app startup or network requests, the chances of user frustration resulting from it being reflected in your collected telemetry would be pretty low. In other words, there are whole classes of problems that are effectively invisible to most app developers due to a lack of telemetry.

While the instrumentation friction described isn't unique to mobile and web apps—as bespoke code needs bespoke instrumentation—the problem is a lot more glaring, as most important workflows important to an app are also unique to it. This means they couldn't be easily and automatically instrumented like that of the relatively deterministic execution of backend services. The increased level of friction, along with other factors, make manual instrumentation in mobile and web apps relatively rare. So, compared to distributed services, code from these apps are far less instrumented.

It follows that when the situation calls for it, manual instrumentation is still not done as often as it should be because it's just not something that mobile and web developers are used to doing. It's a vicious cycle. And in the rare case that they are done, complex workflows spanning multiple screens and components—where execution happens on multiple threads sequentially, unbeknownst to the instrumentation—consist of instrumentation that is inherently difficult to maintain. All this in tandem results in a substantial universe of important events and workflows in a mobile or web app that aren't reflected in production telemetry.

What's more, even with the right telemetry and metadata, assessing the impact of a crash or performance regression on users can be nontrivial because the raw numbers aren't enough. Different users are impacted differently in a technical sense (e.g., varying degrees of change). So, if you're looking at metrics with little segmentation—as most mobile and web developers are used to—the impact may not even be perceivable. What does it mean to users when your P75 app startup time increased by 20%? What if P50 increased 80%, but only for folks using Android 12 and below?

Also, different users react to the same level of performance regression differently. Further, performance degradation matters more in some workflows than in others.

Even if you can quantify the regression down to how users are affected individually, how that impacts their overall UX or desire to use your app in the future can vary as well. For some, a 20% slowdown is barely noticeable. For others, it can push them over the edge in terms of losing patience with your app. For some, even major slowdowns don't affect usage because their workplace has mandated the use of your app. For others, even a bit of perceived decrease in performance can trigger a switch to a competitor's app. So, while it's great to know the actual magnitude of change, it's only part of the picture when assessing user impact.

Raw performance telemetry, even combined with good instinct, can only go so far in terms of telling you how badly users are affected by app problems. To more objectively assess that, there needs to be, at minimum, a correlated change in some other metrics for which you are more certain of the user impact, like DAU or revenue. By bringing in data that the business cares about, it takes you into the realm of product analytics, which are generally not related to telemetry used for observability. Why is that? The reason may be more ridiculous than you think.

Suffice it to say, while the current usage of production telemetry brings some value to mobile and web app developers, there are a lot more questions that they cannot answer—important questions that can help identify and quantify app problems impacting users and businesses negatively. With the lessons learned in driving improved observability in distributed systems, we can make similar changes to how we collect and use telemetry on these notoriously unstable platforms so that the investment in the infrastructure can be further leveraged—solving problems that even the operators of the most well-instrumented distributed systems can only dream of.

But first, there's that mountain of challenges that need to be overcome.

## Overcoming Domain Challenges

Bringing the kind of observability used by backend engineers to mobile and web presents some unique challenges that are inherent to the domain. Some of them are technical—though the nontechnical, sociological ones are perhaps just as impactful in creating barriers that stymie the use of production data by engineers in understanding the unknown-unknowns. Much of these have to do with the status quo, both in observability (or *ol1y*) and in the mobile and web development communities, which combine to create problems that inhibit adoption of observability on mobile and web—problems that are not typically seen in the backend.

## Recognizing the Problem

The first domain challenge to overcome is the prevailing complacency present in the mobile and web development communities. For the most part, mobile and web devs

are satisfied with the options they have for tracking down and solving performance and reliability problems in production. This is because they often believe that the current state of the art tooling would solve all their problems, if only their company would pay for it.

But the ideal they strive for isn't observability as the backend understands it, and it has a much lower ceiling than what is possible. What they typically look for in production tooling are better diagnostic capabilities and identification for known problems, and they lean on CI tooling to detect performance regressions introduced by code changes based on benchmarks.

In particular, it's the prevailing thought in the community that performance regressions are best kept in check using synthetic performance tests. These tests run in controlled environments as part of CI, so material change to performance can be identified before or soon after a commit has been merged. The goal of never unknowingly shipping a performance regression is laudable, but the implicit limitation of it is that you're only finding regressions in your code when specific workflows happen under specific device and runtime conditions, which may not necessarily mirror real-world runtime conditions faced by users.

This strategy is generally pretty good at uncovering pervasive regressions in key workflows. Identifying which workflows matter most should be straightforward, as should prioritizing tests around them. It's not totally free of complications, however: the workflows you consider most important or most used may not be the most vulnerable to performance regressions, nor necessarily the most critical to get right—some workflows have an outsized impact on user satisfaction, if perhaps only psychologically. This points to a broader challenge of needing to know what problems you're looking for in instrumentation, which is anathema to addressing problems that are unknown-unknowns.

The next part of the equation is harder to resolve: how do you determine the specific device and runtime conditions that you should always test against? The problem with production is that there isn't any one set of conditions that will get you a high degree of coverage when you're talking about an app that the general public can access. The variance across multiple dimensions—whether it be device quality, browser and OS versions, concurrent app execution, network availability, etc.—means that whatever set conditions you have for your tests, you know the majority of your users wouldn't be under those precisely.

In fact, that much is recognized in these tests: they are repeatable by design, with as few variations in the execution environment as possible, which is the opposite of what you get in chaotic, heterogeneous production environments. These conditions under which the tests are run are designed to maximize the ability to surface code regressions, which comes at the expense of being realistic. But that's really a feature, not a bug.

In other words, the ideal setup for determining mobile and web performance regression hinges on having predefined workflows running under unrealistic conditions. While it's great for doing what it's designed to do, it's not a great way to understand what's happening in production, especially when the most problematic issues tend to be the ones you don't anticipate.

Mobile and web devs recognize this limitation, so they use production data to fill in that gap. But the gap they perceive is not in identifying nonobvious problems, but in quantifying the severity of obvious problems, and getting the right diagnostic information to solve or reproduce them. This is especially important when a new release gains adoption, as devs want to know if they've introduced any new issues that their tests didn't find—and if they did, they want to know how to reproduce and solve them.

The last part is crucial: it's understood that knowing the codebase is a requirement to use production data to solve problems in mobile and web. But it's not expected that one can simply look at the data and know exactly where the problem lies. This is why on larger teams, the key function of mobile on-calls is to be a master triager, not necessarily the person fixing the issues. This assumption is so ingrained in the community that devs don't expect they can narrow down the root cause of most complex issues based on production data alone, which means they don't look for it in their tooling.

This means that, explicitly or implicitly, the majority of mobile and web developers feel good about the role that production data plays in their day-to-day work. Some may wish that more app state and behavior are captured so that there's more runtime information to work with when issues surface, but they don't expect any more out of production data than what they're getting: diagnostic info and the surfacing of obvious issues. Worse yet, some believe that they are already getting all they can get out of production telemetry! This complicity has stunted the growth of observability on these platforms. It's the curse of being “good enough.”

## Instrumentation Difficulties

There are many ways in which poor performance and reliability can manifest themselves in mobile apps—and do so without crashing. But while crash reporting is well-supported in the mobile ecosystems, generic performance instrumentation is not. This leaves a whole swath of problems that are not quantified by existing telemetry coming from mobile apps, which means they are effectively invisible to app developers.

Why is generic performance monitoring—such a seemingly important feature—supported so poorly in both mobile and web ecosystems? Properly instrumenting an app and ensuring that the telemetry reliably gets back to your datacenter is surprisingly complex. And when you have it, the data—collected from heterogeneous sources

under varying conditions—tends to be quite noisy, so it may not be able to detect nonobvious issues unless you have a large sample. It's yet again a vicious cycle: it's hard to do and not always useful, so people don't do it; if people don't do it, there isn't market demand for first-party or vendor solutions; this makes it harder for the community to adopt en masse; and this then reduces demand.

What makes instrumentation and telemetry collection on mobile and web so complex? Let's first talk about app architecture.

## Application architecture

For distributed systems, the backbone of observability telemetry are distributed traces augmented by related wide events, aggregated and properly contextualized. Noteworthy incidents don't generally arise from detecting a small number of outlier events; they arise with the whole system's performance degrading (or with leading indicators suggesting that degradation can be expected soon). So, to know when your system's behavior gets to a point of noteworthiness, you model its performance—all of it. Particularly, latency and execution duration should be monitored, because slower generally means worse, so detecting slowdowns (along with failures) becomes a key determinant of system health.

In fact, you can go a long way into high-level summarization of a distributed system's behavior and performance by using a distributed trace to capture execution duration, code paths, and errors. You may need more data and context to know what's going on under the hood, but the answer of whether it's doing what it's expected to do in a timely manner can be answered through the performance data. This is because backend services have one job: to process requests and return the results. And a distributed trace gives you the bones of that.

For mobile and web apps, that simply isn't the case. There isn't an automatically instrumentable workflow that summarizes an app's behavior—one whose trace can determine whether the app is in a healthy state. Unlike backend services, usage of these apps follows nonlinear paths, and success is not purely determined by execution duration, but by whether the user gets what they were looking for out of using the app. In other words, little can be known if you only had traces around process execution. The fact that a user lingered on a screen for 10 minutes is not an indication of whether the app is behaving well or poorly. So if you created a span to represent it, its duration will not necessarily tell you much about how the app is performing.

Instead, the best way to model the usage of a mobile or web app, especially when the users encounter unexpected behavior, is through recording telemetry for important events (like crashes) or for workflows (like payment). But therein lies the problem: if an event or workflow isn't explicitly captured by instrumentation, it effectively doesn't exist for observability purposes, because there's no way to passively or implicitly model them. In practice, this means that events and workflows that aren't easy to



instrument or couldn't be done automatically without direct intervention from app developers are usually not recorded as telemetry. You may get data about crashes and app startup, but not about unanticipated out-of-memory errors, or your bespoke payment workflow failing due to a client-side validation error.

The main reason that manual instrumentation of workflows on mobile and web is so difficult is because you have to identify not only the workflow, but the precise area in the code that represents the beginning and end of that workflow—as well as everything in between that can add additional context. All of these could reside in different modules written and maintained by different teams. And threading the needle through all of them is only the first step—ensuring that code changes unbeknownst to the instrumentation writer don't break the instrumentation is the second, even more difficult step.

In other words, the “trace the execution” idiom that allows new telemetry to be associated with the “current” trace on the executing thread doesn't apply. A workflow on mobile or web that can be thought of as a single trace can, and often is, executed serially in disconnected chunks across multiple threads and screens, all connected loosely by bespoke components within the app that don't share any execution context. Basically, the logical grouping cannot simply be divined by looking at the code, and there isn't an implicitly shared context that ties them today.

With a service inside of a distributed system, modeling its execution as a span and then adding wide events to give more details is relatively straightforward: start a span when a request is received, end it when it finishes, and ensure all events are associated with the current span—which is often just whatever span was last started on the running thread. The trace matches up nicely with the execution of the service. On mobile and web, you have to know where you want to start the workflow and where you want to end it. Unless it's built into the app architecture, this otherwise requires knowledge of the code and how all the components work together.

Given that a workflow can cross multiple components, it means that knowledge of the instrumentation has to be embedded pretty tightly into the business logic of the code. This can be tricky because different parts of the code enabling that workflow may not contain a shared context. So the simple act of knowing which span to stop may be challenging if you can't do something like “stop whatever span is active on the current thread.” This makes the complexity of instrumentation increase with the amount of instrumentation that is present, which is a tricky proposition when you want good coverage.

### **Life in a chaotic, remote sandbox**

Overcoming instrumentation friction caused by app architecture may be nontrivial, but it can be done. What is more difficult, if not impossible, to overcome is the extraction of relevant information on mobile and web platforms, and delivering the



telemetry to the server reliably. Mobile and web apps are essentially processes run in sandboxed environments at the whim of the user and OS. App developers have little control or visibility into the underlying system resources, or when execution starts and stops. This limits the degree to which the app can provision enough memory and CPU cycles to run the app properly, and also restricts the types of information that the instrumentation has access to in order to explain performance.

Is a particular instance of a span an outlier caused by poor hardware, code regression, or the fact that a YouTube video is playing concurrently? The available runtime context can suggest plausible reasons (e.g., low-end device model), which can then be corroborated with more data points (e.g., all workflows on this model are varying degrees of slow). But the APIs afforded to apps to extract that context in production are restricted to only what that app process has access to due to privacy reasons—mobile devices and browsers, after all, can be seen as multitenant devices. As such, they aren't nearly exhaustive, particularly when it comes to what's running on the device concurrently with the execution of the app. So you are left with many potential causes of performance degradation that are not represented in the data.

And even when your instrumentation is able to capture and contextualize the right telemetry, that telemetry must first be sent over the internet to your servers before it can be consumed. That it has to be done by processes living at the mercy of the user and OS—over spotty network connections with limited bandwidth and high latency (if a network connection exists at all)—means that recording the telemetry locally is just the first step in the long journey to observability.

This point is especially pertinent because an app's ability to record and send telemetry is most unreliable when it's running low on resources or operating under suboptimal conditions, which is precisely when telemetry is the most useful. Making sure telemetry is reliably delivered when everything is working fine is easy. But the challenges are tenfold when your process isn't being scheduled and the memory pressure is causing so much garbage collection that it makes your app unresponsive, increasing the likelihood that a user will manually terminate the app without your instrumentation being able to save and send the telemetry it has recorded in memory.

The insidious implication is that unless you can quantify data loss, you wouldn't even know what data you're missing, because lost telemetry will be skewed toward situations when the app is having issues. So while the data you collect may suggest that your app and users are doing fine, that may not be the case: telemetry from problematic situations is lost at a much higher rate than from stable ones. This could lull you into a **false sense of security**—a delusion that everything is fine simply because errors and slowdowns aren't being reported.

# Applying Local Storage and Real-Time Control to Mobile Observability

*This section is contributed by Matt Klein, CTO at bitdrift*

In some ways, mobile observability is the “final frontier” of observability practice. While the importance of server-side observability is well-established at this point, very few mobile teams have any observability practice other than crash monitoring. This is unfortunate for two reasons.

First, in mature mobile applications, typically over 99.9% of user sessions are crash-free. This means that the ability to understand subpar behavior (functionality bugs, hangs, performance issues, rage taps, rage quits, and so on) is actually much higher leverage when aiming to improve UX, yet most teams don’t have access to any tools that can help.

Second, as an industry, we love to talk about our server-side success rate and all of the tools that we have at our disposal to understand why we aren’t achieving the requisite number of nines. Yet, as I like to say, a 99.99% success rate at the server-side edge doesn’t mean much when malformed JSON is being sent via a 200 OK response, causing the app’s golden flow to simply stop working. I’ve stopped counting the number of times this has happened in my career; it’s certainly not theoretical.

So if mobile observability is arguably the most important observability—as it allows us to understand what our customers are actually experiencing when using our products—why isn’t it more established? The reason is that it’s *extremely* hard to do well, and efforts to simply take the architectures and methodologies that have worked well in the server world and apply them to mobile have failed time and time again.

So, what makes mobile observability difficult and different from the server world? A few things:

## *Long release cycles*

Due to the epically long release cycles inherent in mobile, the simple act of adding a log or analytic event can take weeks for initial rollout, and months if not years to reach the entire population of app users. This means that all things being equal, mobile engineers would like to add as much logging and analytics as possible ahead of time in case the data is needed. However this is unfortunately at odds with the fact of the next challenge.

## *Users caring about data usage*

Users that are on metered data plans care deeply about how much data an app is using, and will notice if an app is using “too much.” Furthermore, even if users have unlimited data, slow networks and sporadic connectivity mean that observability data is sharing available bandwidth with critical application

traffic—i.e., sending too many logs can materially impact the performance of the main application. Note that nearly every large mobile app has measured that perceived application performance (time to first interaction, delay when moving screens and clicking buttons, etc.) impacts conversion and core business metrics, so this is not an academic concern.

#### *Sporadic connectivity*

Unlike on server, sporadic or poor connectivity on mobile must be considered a normal event and not an outlier. This means that observability systems must plan ahead for sophisticated batching, local storage, and so on, or enough data will be lost to impact the reliability of the observation system.

#### *Unplanned termination as the norm*

Because apps can be terminated at any time and may not be able to run in the background, mobile observability systems must plan for this case—similar to how sporadic connectivity might be dealt with.

#### *Huge cardinality and cost*

Depending on the app, a relatively small server infrastructure may support hundreds of thousands or even millions of monthly active mobile users. Storing individual telemetry for each active user is very costly, so aggregation and summarization systems are typically needed for mobile observability.

#### *Naive implementation of local storage for observability data, impacting user interface (UI) performance*

While mobile apps can be multithreaded, in general a single “main” thread is used for drawing the UI and handling user input events. Care must be taken to not block the main thread unnecessarily or the app’s frame rate and general responsiveness may suffer. This is particularly important for local storage of observability data which needs to be durably stored due to several of the previously described issues. Thus, the observability implementation needs to carefully consider how logging data is processed and stored so as to not adversely impact app performance.

We have established that mobile observability is as important, if not more important, than server observability. We have also established that mobile observability is very challenging, and applying server observability methodologies directly is unlikely to yield a satisfactory result, both in terms of cost and data quality. So, what then? We need to step back, start from first principles, and design an observability system tailor-made for mobile—one that provides excellent data quality, at a satisfactory price point, and with high performance.

Fundamentally, an observability system made for mobile needs to reconcile two competing goals:

1. It takes a really long time to add telemetry (let's assume for now that all telemetry is wide events and that traces/metrics can be derived from those events), so developers should have a way to add as many logs as possible before shipping the app.
2. We can't ingest all telemetry at all times from hundreds of millions of mobile devices, for both cost and performance reasons.

It turns out there *is* a way to satisfy both of these requirements, though it requires a fundamental shift in how we think about our observability practice. The way to accomplish this is via coupling local storage of telemetry data with a real-time control plane that can instruct every device on what telemetry to send for ingestion. Let's unpack these concepts in more detail:

#### *Local storage*

Instead of sending every single log line over the network to a server-side log ingestion system, we can instead store all of the logs *locally* in a high-performance circular buffer of finite size. The circular buffer itself is persisted to disk. This solution solves multiple difficulties described previously:

- Developers can log as much as they want because, by default, logs don't leave the device, which mitigates concerns around high ingestion costs and app performance overhead for telemetry collection.
- Disk persistence means the telemetry survives unplanned app terminations.
- High performance means that the act of logging has minimal impact on app functionality, especially the main thread and UI performance.

#### *Real-time control plane*

Instead of devices aimlessly pushing all telemetry in real time to a remote ingestion service, we use an intelligent bidirectional streaming connection between the ingestion service and the device. The server provides instructions to the client about what data to send under what conditions. And the client sends data in response to those instructions.

These two changes (local storage and a real-time control plane) may sound small, but the implications are profound. Instead of sending all the data all the time, we now have the ability to send a large amount of data when we need it, and none when we don't. This "fidelity dial" means that operators can decide how to get the best value from their telemetry ingestion, and change it on the fly if our needs shift.

Let's look at a couple of examples of changes to workflows:

*For monitoring purposes*

Synthetic metrics can be derived from logs (both counters and histograms) and sent directly from devices in a highly efficient and compressed form. The metrics that are sent can be changed immediately and without any app deployments, meaning that if monitoring needs change, the ingested data can be changed.

*For debugging purposes*

The entire circular buffer can be ingested to produce a “session replay.” This can be done in response to highly specific conditions (i.e., a finite state machine sent from server to client tells the device what conditions to look for). For example, “Wait for the user to view the checkout screen. If the user leaves the checkout screen or 15 seconds have passed without a checkout, increment a counter and dump a full session with session replay.”

Using local storage, real-time control, and matching conditions based on finite state machines, it now becomes possible to debug highly specific scenarios depending on the need at hand. From looking at specific cohorts (all users in a geographic area on a particular app version on a particular device model) to even debugging a single specific user (very common in customer support scenarios), local storage and real-time control make it easy to accomplish whatever targeting is needed. Further, building the entire system on top of arbitrarily wide, structured events means that user journeys, session replays, and so on can all be derived on demand.

Mobile observability certainly has unique challenges. Its importance requires unique solutions, and our willingness to set aside long-established methodologies that have largely worked well on server—cost control has *not* worked well on server, and similar issues plague naive mobile solutions.

By replacing the “simple pipe”—which streams all data from device to server with a smart system based on local storage and real-time control—we can reduce costs and vastly increase productivity by allowing mobile developers to log to their hearts content in every app release, while giving them access to this bounty of extra signal when they actually need it.

Further, we can finally provide real observability to mobile engineering teams across the industry that, in many ways, are decades behind their server peers in their ability to dig into the unknown-unknowns, previously only having access to crash monitoring. Crashes are important, but they are a minute piece of the overall puzzle. The future is bright in the mobile observability world, and our end users will thank us for providing more delightful and less buggy apps.

# What to Observe and Why

Matt Klein's section highlights a subtle but important differentiating factor of observability in mobile and web: we are not observing one complex system, but many, many simple ones. Aggregates create summaries not of a series of connected nodes and services acting to serve the same set of incoming requests, but of millions and millions of unrelated processes running on different devices, operated by different users, each with different usage goals and expectations.

Mobile observability is akin to surveying a heterogeneous population; truths are summaries that describe cohorts of users with shared characteristics, ephemeral or concrete. Conclusions about the behavior of one application instance running on one device controlled by one user can only tell you but so much about a different instance running on a different device controlled by a different user. Insights have to be qualified, locked to the particular context from which they were derived.

The variability of P50 and P99 can be vast on mobile and web simply because the operating conditions are wildly different. It's no fault of the application if devices from 10 years ago are running alongside brand new ones. It follows that knowing what the P50 experience is can only tell you so much about what P99 may be. That fact alone should make folks who only benchmark with a single device worry about how representative their data is of what their users experience.

To put it another way, knowing what the P50 duration is for a workflow in a distributed system tells you what you can expect to happen when more requests come in, in terms of performance, assuming a normal amount of variability. Knowing what the P50 duration is for a workflow in a mobile app doesn't tell you very much about what you can expect at all, unless the execution context is very similar to that of the instance that provided the P50 value. It's perfectly reasonable if a cold app startup P50 is 1 second while the last measurement is 10 seconds, if the device generating that last measurement was released nine years ago.

This is why the sets of tooling and use cases that help the discovery of problems and then give insight into how those problems can be solved can be very distinct. The former requires sifting through the data to identify small cohorts of similar users, to find issues that impact them that aren't obvious because they don't impact everyone, while the latter is about finding enough details about the experiences of the impacted cohort so that you can understand why they are impacted. The former requires breadth while the latter requires depth—perhaps to a degree that isn't feasible to capture in production.

What this means is while production telemetry can tell you quite a bit about the state of the app and device, various factors make it so that the depth of data you can capture at runtime pales in comparison to the kind of information you have access to when you can reproduce the issue locally.

For instance, the Android platform gives you the ability to take profiling data of your device while your application is running. This is a feature of the OS, and it provides information not only about your app's process, but also all the other processes that are running on the device, as well as low-level system information like CPU scheduling, memory, and hardware states. Android apps simply aren't granted the permission to access that kind of system data, and even if they were, the data volume would be too large for it to be feasibly captured and transmitted at runtime in a way that doesn't affect UX.

This is why, for debugging purposes, Android devs have traditionally preferred local tools that give all the diagnostic information you would want. And the fact that issues are always reproducible if they are well-understood means that if observability tooling can provide that understanding, devs would have the data they need to do their job without asking more from it.

As such, discovering nonobvious app issues and pinpointing the precise conditions under which issues are reproducible are the dual primary goals of mobile observability. Current tooling available to most mobile and web developers falls short of these goals, but production telemetry combined with context is extraordinarily well-suited to address them. Finding diagnostic information, while useful, is only a secondary goal.

## Opportunities for Improvement

While the challenges for creating observability in mobile and web applications are unique and nontrivial, the opportunities that it presents can be described similarly: unique and nontrivial. The reason for that comes down to having direct access to a source of data that has previously been untapped by observability practitioners: users. Not only can users provide telemetry through their actions that aren't normally accounted for, but that data also comes with rich context from which cause and effect can be inferred at the workflow instance level. That direct linkage is invaluable for finding needles in haystacks, when deducing from aggregates becomes difficult as conditions narrow and sample sizes dwindle.

The unique and nontrivial opportunities to leverage data from users is what makes all the toil to overcome the myriad challenges of mobile and frontend observability worthwhile, to see all the work bear fruit. The juice, as they say, is worth the squeeze.

## Making Observability User-Focused

Traditionally, instrumentation of distributed systems mostly produces telemetry that models the execution of services: how long they took to execute, whether the executions encountered any errors, and the context under which they run. Users of the application, meanwhile, are an abstraction—the makers of requests—and their role

can largely be summarized by the kinds of requests they make and the parameters on those requests. So it's not a large leap to say that they are pretty well represented in existing observability systems, assuming request parameters are properly captured to contextualize the associated telemetry.

Even when observability moved to the mobile and web spaces, we largely replicated the same behavior when it comes to instrumentation: what is happening in the app and how well is it performing? Even when the telemetry is the direct result of user actions (e.g., RUM events), what we capture is still very matter-of-fact events that we can observe: screen taps, page navigation, application startup duration, and so on. But stopping there means ignoring a key advantage of making mobile and web apps observable—our ability to directly observe in its entirety a single user and/or session—and how we can leverage that connection to increase what we ultimately want to understand: UX.

Trying to tie together telemetry for a given user or session in a distributed system is possible and potentially very useful. However, knowing all the requests from a single user or session can only tell you so much about how the user is experiencing the app, because so much of what happened between requests is not known to the backend. Why is there a two-minute gap between request A and request B? Is that reasonable? There is only so much you can glean about why requests are made when you only have data from your backend.

But what if you have access to all that is happening on the app itself? Knowing that an attempt at checkout was made but ultimately not followed through with, as well as what happened during the session after the initial attempt and what potential factors could have caused it—that's when you can start applying observability techniques to understand not only application behavior, but also user behavior. Once you have telemetry that models user actions related to other telemetry and context collected from the app, that's when your observability becomes truly user focused.

After all, isn't the whole point of all this to ensure that users have good experiences when they use your app? Experience is hard to understand directly when users are an amorphous abstraction of request makers. But when you can get data that ties users to app telemetry over a long period of time, you can paint a much clearer picture of how their usage might vary depending on the UX.

A wise person once said that the nines don't matter if your users aren't happy. If you have a mobile or web app and you don't try to quantify user happiness using data collected on the client side, are you even doing observability right?



## Quantifying User Experience

Before going further, it's important to be more concrete about what is meant by “quantifying user happiness.” Too often, vendors make up easy-to-understand “scores” that sum up the quality or stability of an app because evaluating said app along multiple complex dimensions is, well, too complex for most stakeholders. Similarly, “happiness” sounds a bit like sentiment analysis, which is, at best, an imprecise notion to quantify.

To proxy the quality of UX via telemetry in such a way that it's stable and useful for observability, neither generic algorithmic aggregates of metrics nor fuzzy sentiment analysis would stand up to scrutiny. We need something that will consistently be sensitive to change in UX, all else being equal. In other words, it has to be relatively unbiased events from the app, or metrics that derive from it.

Stepping back a bit, the reason users use an app is to accomplish a goal: look at pictures, buy an album, order some food. UX describes the journey a user has from the moment they pick up their phone to the moment they accomplish what they set out to do. A good UX—like if the app launches quickly, the UI is responsive, and there are minimal delays—tends to allow a user to quickly do what they set out to do with little frustration. Conversely, a bad UX—like if the app launches slowly, is unresponsive to user actions, or has confusing UI—often causes users to give up before they can accomplish their goal.

Mobile and web apps, due to their proximity to the user, can gather telemetry that describes the user experience and intention and whether their goals are achieved. While instrumentation can only record what's happening in the app, there are events that can be used as proxies for user intention and outcome achievement, which means we can correlate them with other telemetry and context in order to understand the factors that lead to a higher likelihood of a user intention being met with a positive outcome. The ability of telemetry to do this—and to describe and reflect user intention and outcome, along with the runtime context—is what makes observability “user focused.”

To extend this further, one can assume that if a user wants to do X, they are happier if they were able to do X. While there are many reasons why a user who expresses an intention doesn't ultimately achieve a successful outcome, one can assume that bad UX—including poor performance and bugs—is one of them. Therefore, this source of users not achieving a positive outcome can be seen as something the app can directly control. Something that completely removes all friction from the process (i.e., as soon as an intention is expressed, a positive outcome is achieved) will make failure nearly zero. Conversely, something about UX that is so bad that it prevents users from achieving a positive outcome, will make failure the only outcome.

Therefore, user happiness can be proxied by the degree to which users achieve a positive outcome—which in turn can be proxied by the likelihood that a user who expresses an intention ultimately achieves one. So, something as simple as “checkout completion rate”—which could be calculated by taking the number of complete checkouts in an ecommerce app and dividing it by the number of checkout attempts—can be seen as a metric that captures user happiness (i.e., did the user do the thing they wanted to do?).

## Adapting Existing Approaches

Now, you might be thinking: isn’t this just product analytics (i.e., the kind of data that product managers use to determine which features are used)? In a way, that is true. The actions modeled by user analytics products are similar to the user intention and outcomes described here. The difference is that other than static context that can be used for basic segmentation, product analytics data isn’t tied to telemetry that describes the UX and app performance.

This is where user-focused observability provides utility: it ties together previously siloed data used by both product managers and devs so that they can mutually explain each other, allowing user behavior changes to be correlated with app performance data and quantifying the real impact of regressions in terms that the business understands. By linking a particular instance of a successful or failed use journey with all the context and telemetry that is collected in the past and concurrently, you have a means to explain why the journey was or wasn’t successful.

Too often, product managers with access to product analytics will find changes to conversion funnel data but, based solely on the data, cannot explain why. This is because product analytics merely reports what happened and to whom, nothing more. Mobile engineers are then tasked with finding out the reason, often knowing little more than the timeframe in which the change was perceived. But because mobile apps are relatively simple—at least compared to distributed systems—the root causes of many issues can be isolated simply by looking at the changes to the app that were made around that time.

While this workflow is generally good enough, there are many cases where it isn’t. If the issue is nonobvious, multifactorial, affects only a subset of users, or is not even caused by a change in the code, the amount of time that could be lost to investigations could be significant—if the root causes can be determined at all. This is where user-focused observability shines, and it’s not trying to isolate the root cause using only the data, just that there is a user-impacting problem to investigate.

Being user focused flips the paradigm of observability from something that first relies on detecting anomalies with the app and then determining user impact, to one where you first detect user impact and then determine the underlying app anomalies causing said impact. This switch eliminates the question of whether an app issue is

impacting users given that the reason why it was found in the first place was due to users being impacted. There will be no more debating whether a 20% increase in P75 startup time merits an investigation. You will have data correlated with the increase—potentially even more if you were running an experiment.

## Applying User-Focused Observability

User-focused observability flips the traditional paradigm: instead of detecting technical anomalies and guessing their impact, you first detect user impact and then isolate the underlying technical anomalies. This switch ensures that every investigation is centered on a material problem affecting UX or business key performance indicators (KPIs).

### Iterative Analysis Process

The goal is understanding how user behavior is impacted by app behavior through the direct linking of user intentions and outcomes with app telemetry and metadata at the workflow level, which can then be verified repeatedly at a statistically significant level in aggregate. The crux of this process relies on the ability to filter the data by a variety of dimensions in order to tease out the relevant ones so that the differences between cohorts is statistically significant.

The process plays out when a difference in user behavior is detected—when we iterate to isolate pertinent dimensions that are correlated with the change, and identify other correlations that could be investigated further to provide even more clues as to the areas in the app where the root cause(s) may lie.

Given the noise inherent in the production data that is collected on mobile and frontend apps whose usage may be cyclical, there will be false positives and dead ends. But the idea is that any repeatable pattern of correlation suggests some root cause, and it's the observability practitioner's job to determine whether it's caused by something under their control, which is often the case.

Let's take a look at a fictional example to show how user-focused observability can help track down app issues in production that are otherwise difficult to identify.

### Example: Food Delivery App

FakeBurger is a food delivery app with tens of millions of daily active users globally. An important metric monitored by product managers is “checkout rate,” defined as how often a checkout workflow that starts is completed successfully. That is, when a user shows sufficient intention to order food from a restaurant, they go through the workflow to pay for it successfully. While this metric can fluctuate during the day due to a number of factors, in the aggregate, there's a range within which that metric is expected to be.

The checkout rate is important to the business because drops may indicate an increase in the friction experienced by users doing the main thing the FakeBurger app is used for: ordering food. It can be viewed as a proxy of how happy users are with the app, at least in the narrow sense that they were ultimately able to do what they set out to. A change to this rate may not necessarily indicate that something about the app has changed, but changes to the app can definitely affect this positively or negatively. A decrease of the checkout rate could mean lost sales in the short term and customer churn in the long term, so it's an important one to keep an eye on.

In terms of the telemetry to calculate the checkout rate, we can model this in a number of ways. The key data required are two events (e.g., `start_checkout` and `end_checkout`) that are linked by some unique ID wherein the second event knows it's associated with the first, and the first event can be recorded without requiring the second event to have happened. Depending on the backend, this could be a structured log, two structured logs, a span, or whatever can keep track of this information.

While the telemetry tells you what happened in the app, contextual metadata that distinguishes one checkout instance from another can tell you why it succeeded or didn't. For FakeBurger, they passively collect context like `sessionId`, `userId`, and app version for all the telemetry they record. For workflows they are especially interested in, such as checkout, they add unique and relevant metadata that help further contextualize the telemetry, like payment type, discount code, previous app screen, and so on.

It's important to note that much of the contextual metadata isn't directly represented in the data sent to the server in API calls. In other words, app-only concepts like the previous screen that the user was on are not represented in telemetry used for backend observability. That this metadata describes factors that can cause the app to behave differently means that they represent important information that can help explain user behavior and app performance—data that would otherwise be lost if telemetry weren't collected from the mobile app. FakeBurger has learned the hard way that looking solely at backend data is not enough to understand what the end users experience in their app.

In addition, other RUM telemetry collected by the app during the session—such as user navigation and screen taps—can also be considered contextual metadata with which to differentiate checkout instances. It could help answer important questions like “what did the user do during the checkout?” and to document app states that are otherwise lost.

As mentioned before, FakeBurger uses the checkout rate as a proxy to user happiness and satisfaction with the app. When it changes to a degree that is statistically significant—after controlling for the expected time and date-based fluctuations—they will know it's very likely that the status quo with the app has changed: whether it's due to

the UI being modified, a new app bug, a change in the user base, or even a change to the instrumentation implementation.

Changes like this in mobile apps tend to coincide with a new app version, feature rollout, or significant shift to the user base (e.g., entering a new market with users who behave differently in aggregate than existing users). This is why after a new app version is released and reaches a critical mass, FakeBurger mobile engineers look at how key metrics like checkout rate have changed in production. They not only look at these in aggregate for the whole population—they also split out data in other ways, like comparing data from the new version to the rest of the population, as well as comparing the magnitude of difference in some sample groups when previous versions were rolled out.

The reason they split the data in multiple ways is so each view can corroborate the others, as different comparisons carry different embedded biases. For instance, splitting users by new and old versions during a specific time period will show a difference. This is because early adopters of new FakeBurger versions tend to have better phones, making slowness-related abandonments less likely and checkout rates higher than those of late adopters, who abandon checkouts at a higher rate due to performance-related issues.

As is apparent in this example, simply determining whether a change to the checkout rate in production is caused by an app change is nontrivial. Doing that justice is outside the scope of this chapter and book, but let's assume they are able to determine whether a change is caused. When they do, they can ask questions like the following:

- Was there a cohort within the population for which the drop was outsized?
- Did any other metrics change?
- When you split out the sessions that contained a failed checkout and compared the data before and after the checkout rate drop, were there any differences in terms of the metadata or events that occurred prior to the failure?

The telemetry and metadata that FakeBurger records around the checkout workflow will be able to answer these questions for them. Through this data—after it's properly processed and made available via dashboards—mobile engineers and SREs alike are able to find app issues like the following, which would be difficult to surface were it not for user-focused observability:

- UI bugs not explicitly logged that add friction or prevent users from checking out (e.g., the “Next” button rendering below the fold)
- A new discount code format that disabled the “Submit” button the first time the user enters the checkout flow—resolved only by abandoning and re-entering it a second time (potentially occurring without a new app version, as it might not have been caused by a client change)

- Serious errors occurring infrequently enough not to move top-line app metrics in a statistically significant way, but which disproportionately impacted checkout rate by affecting heavy users
- Multiple small glitches introduced over time, each too infrequent to move app metrics on their own, but collectively degrading the UX enough to impact conversion rates

Through user-focused observability, FakeBurger is able to not only identify regressions but also further ask the data questions that they had not initially anticipated asking. The reason is that they center on what matters first—user experience—and let telemetry and metadata explain their changes. They will no longer only chase changes to telemetry (not knowing which actually matters to their users and their business), but instead focus on metrics that they know for sure move the needle. They let the data suggest avenues of investigation or causes that they wouldn't otherwise think of in the chaotic environment of end user-facing mobile and web apps.

## Mobile and Frontend Applications Deserve Observability

Observability on mobile and frontend applications needs to center users and their experiences in order to maximize the utility provided by the unique perspective of telemetry and metadata gathered that way. Instead of trying to isolate small problems within the app in a sea of noisy and inconclusive data, it's better to turn that around and identify changes to user behavior and, through the data, provide reasons as to why that might be the case.

The paradigm shift of looking first for user impact before app-level problems can be jarring for developers who are used to looking at production data solely for more information about app problems. Instead, user-focused observability goes a level higher and identifies problems in the UX that are perhaps correlated with app-level problems that are less easy to detect unless you know what to look for. It reduces the size of the haystack in which you have to find the needle and, in doing so, ensures every haystack you look through is causing material problems in UX and impacts KPIs.

This is only possible through user-focused observability, because we have the ability to model user intentions and outcomes alongside traditional app telemetry and metadata—linking them directly rather than just inferring correlations in the aggregate. Not only is that difficult to do on the backend, but the data used is missing a pertinent source entirely: what is happening in the app that the end-user is actually using.

So, while data coming from mobile and frontend clients may be noisy, incomplete, and difficult to consistently obtain in a timely manner, it represents a treasure trove of valuable data that you can't otherwise obtain. The juice may be hard to squeeze, but its sweetness hits like nothing else.

## Conclusion

Hanson provides a very thoughtful and detailed look of the many challenges that are unique to the world of observability when applied to mobile and frontend applications. Paired with Matt's insights on managing telemetry storage and transmission, we get a comprehensive look at the various challenges that happen when considering performance across a global network of incredibly diverse devices that have unpredictable and often unreliable network connections.

In the next chapter, we'll look at other observability use cases focused on performance by considering system profiling. You'll learn how to optimize infrastructure decisions and cost models using the observability data you've collected.





---

# Performance Engineering with Observability

Observability is often framed as a tool for reliability—keeping the lights on and fixing what breaks. However, a mature observability practice also serves as the foundation for performance engineering: the disciplined pursuit of making systems faster, more efficient, and more cost-effective. In this chapter, we explore how to move from reactive firefighting to proactive optimization by leveraging the overlap between these two fields.

## The Case for Performance Engineering

Imagine you had a mystery bug that was silently adding latency to every request in your services. That was the situation our team found ourselves in, and the catalyst for our performance engineering journey.

In December 2021, the Honeycomb engineering team was surprised to find a discrepancy between the request latency for our ingest endpoint as measured by the load balancer logs and the auto-instrumentation spans in our application traces. The effect was most pronounced for POST requests to our ingest service endpoint that ingested single wide events rather than larger batches. Somehow, time was going missing, and requests that would take tenths of a millisecond to complete on the server side were showing up in the load balancer as single-digit milliseconds.

Larger requests to our batch endpoint also showed the same discrepancy of single-digit millisecond variation between the data sources, but the difference was less noticeable as a proportion of the request time because batches vary in size and can take tens to hundreds of milliseconds. Outside the visibility of our instrumentation for traditional observability, something was slowing down *every* request.

With no specific ideas for what could be amiss, and to rule out the possibility of the tracing overhead itself being the problem, we used the Go programming language’s built-in support for performance profiling to search for clues. We expose the `net/http/pprof` endpoint on our pods, so it was a matter of running the `go tool pprof` utility as a one-off to obtain a flame graph (see [Figure 20-1](#)).



*Figure 20-1. A Go pprof flame graph showing an unusual amount of time spent in `regexp.Compile` within `mux.newRouteRegexp`, taking similar CPU time relative to the `ServeHTTP` endpoint code.*

Manually reaching for separate tooling outside of our own always-on tracing was an impediment to the investigation, but we were able to identify the problem based on where the CPU was spinning for excess cycles. The `regexp.Compile` path shouldn’t have been consuming 17% of all CPU in the process and bogging down requests. Under normal operation, we’d have expected regular expressions to be compiled only once upon startup and reused after that. Why were we reinitializing the HTTP router `mux` and reparsing its configuration over and over?

A dive into the code confirmed that the HTTP router was being discarded after each request rather than being reused as a singleton. The fix to the slow performance impacting our customers was just five lines of code (see [Example 20-1](#)). And we could not have caught the problem with tracing alone, as the tracing hook is called by the `mux` router *after* it has already been instantiated—once each request has already been matched to a route.

*Example 20-1. The entire five-line Git diff for the bug fix, with `initializeRouter()` pulled out into a method and called in the `Start()` function rather than being called for each `ServeHTTP()` request.*

```
@@ -115,6 +115,7 @@ type App struct {
    zstdDecoders chan *zstd.Decoder

    grpcServer *grpc.Server
+   router     http.Handler
}

func (a *App) Start(ctx context.Context) error {
@@ -149,6 +150,7 @@ func (a *App) Start(ctx context.Context) error {
    return err
}

+   a.initializeRouter()
    a.server, err = hd.ListenAndServe(&http.Server{
        Addr:      a.Port,
        Handler:    renameBeelineTraceHeaders(gz(a)),
@@ -448,5 +450,8 @@ func (a *App) ServeHTTP(w, r) {

    // Allow cross-origin API operation from browser js
    w.Header().Set("Access-Control-Allow-Origin", "*")
+   a.router.ServeHTTP(w, r)
+}

+func (a *App) initializeRouter() {
    router := mux.NewRouter()
```

This investigation prompted us to spread knowledge of performance engineering across our team. Our performance journey was similar to the shift that we espoused earlier in this book—from reactive to proactive debugging, and from single points of failure to team-wide expertise in observability. Performance engineering and cost control share many goals with observability, and ideally benefit from using shared tooling.

For a very long time, performance engineers have used disjoint and disparate tooling when compared with operations teams or software engineering teams. What we've both been trying to achieve has been observability, but the tooling has constrained us into inspecting different layers of the stack and at different granularity. In an ideal world, unified observability will offer useful functionality to all facets of software delivery and optimization, regardless of whether they are investigating individual assembly instructions or the end-to-end user journey. It is time for our disciplines to come together, and to focus on empowering *every* engineer to own their code and systems in production.



While we can't cover performance engineering in full detail, and must focus only on the overlaps with distributed-systems observability, Brendan Gregg's seminal works *BPF Performance Tools* (Addison-Wesley, 2019) and *Systems Performance* (Pearson, 2020) are strong resources for those who choose to further explore the discipline.

## Building a Performance Engineering Practice

Performance engineering addresses two critical business needs: lowering costs to aid the bottom line, and speeding up customer experiences. While it certainly can be a fun academic exercise to hyperoptimize a specific function, real impact comes only from moving the needle on tangible metrics. For any given experiment, you should first gather baseline data on performance and cost in advance of running the test. Are you working on the right part of your codebase to have the largest possible impact? And can you quantify any future improvement you might make—whether to latency, data size, dollars, or effective tons of carbon dioxide emission? Thinking more broadly about a performance engineering function, dedicated headcount only may make sense if given the correct mandate and with appropriate scope.

In many cases, where the cloud bill is small, investment in performance engineering may simply not feel worth it relative to the salaries of the engineers required to optimize versus build new features or maintain reliability. Similar to considerations on payoff of time spent automating versus **how much time the automation will save**, we would caution developers to avoid premature optimization where the salary and opportunity cost outweighs the cost or customer performance gain. However, failing to implement any guardrails can lead to waste, excessive expenditure, or customer-facing latency creeping up—and collectively, we are wasting a significant amount of resources and potentially worsening the climate crisis. Therefore, knowing how to employ performance engineering is still valuable, even if you decide that now is not yet the time to invest in doing it.

For us, the investment was worth it because of our relatively high cloud bill relative to our headcount—and the fact that we operate a software as a service (SaaS) startup, where gross margins and other engineering-influenced business metrics directly affect our viability as a company. Having intentionally built a culture of observability-driven development meant that our developers embraced performance engineering relatively quickly.

Like observability-driven development, performance engineering requires identifying the most impactful possible changes, adding instrumentation, making the change, and then inspecting the results in production. The team already had a sense of curiosity and ownership, but was stymied by the ad hoc nature of the siloed tooling that we had employed in our initial forays into performance engineering. We needed

to improve the usability of the tooling we made available to our development teams, and make our practice continuous rather than one-off. We also sought not to build a separate individual repository of knowledge or a dedicated performance engineering team.

## Optimizing Cost Without Modifying Application Code

Getting performance engineering onto the engineering roadmap for teams can be challenging. Developers prefer the experience of receiving performance and cost gains automatically with no additional effort on their part to change code. Fortunately, there are a few options for optimizing on the backend without requiring developers of each application to take manual action. In this section, you'll learn about three techniques for hands-off optimization.

### Infrastructure Purchasing Models

One of the most important factors impacting the overall cost of your infrastructure is the type of compute platform you use. Even before choosing specific instance types, you must first choose a product based on its developer ergonomics and economic model. We don't mean choosing between an m4.large versus an r5.xlarge (although we'll discuss that topic later in this chapter). There are six models for obtaining compute capacity:

- Capital expenditure (building your own datacenter (Oxide Computer Company))
- Reserved capacity (AWS Savings Plans, Azure savings plans, Google committed/sustained use discounts)
- Persistent, defined capacity (Amazon EC2, Azure virtual machines [VMs], Google Compute Engine)
- Interruptible, defined capacity (Amazon EC2 Spot, Azure Spot VMs, Google Preemptible VMs)
- Persistent, elastic capacity (AWS Fargate, Azure Container Apps, Google Cloud Run)
- Interruptible and flexible, for short requests (AWS Lambda, Azure Functions, Google Cloud Functions)

Each of these methods of procuring infrastructure has benefits and drawbacks in terms of money, customer experience, and the developer time required to care for them. With a function as a service (FaaS) approach like Lambda or Azure Functions, you achieve perfect utilization—only paying when your software is actively running. You need not worry about the software system-level dependencies or underlying hardware. As a trade-off, you pay a higher unit cost for each CPU-second (or CPU-millisecond, if your provider supports granular billing), because the cloud provider is

assuming the risk of capacity management and is unable to sell all of their capacity pool. And some requests may take longer to serve if the cloud provider needs to allocate and boot up new capacity for your workload.

Running entire virtual machines (VMs) instead of using serverless means you can keep your processes active, persist data to a cache, or maintain connections to dependencies, making requests return a lot faster. However, you're paying for that server time while you're not using it, and you need to make sure you can scale up within time if you get more requests than you expected. And if you're using self-hosted bare metal rather than VMs, your CPU-hour costs will be much lower, but you'll have to make the capital outlay for the hardware and assume responsibility for provisioning and availability.

Then there are the models lying somewhat between—for instance, purchasing reserved capacity, or using a blended serverless compute engine that abstracts away the underlying VMs but prices for the CPU consumed by each task rather than the duration of each request. When using VM-based infrastructure, you can also achieve significant performance and cost optimizations by using a container orchestration approach, such as Kubernetes, to pack multiple applications into a single VM. However, the conversation about containerization is partially orthogonal to the compute model: serverless functions can operate with raw binaries, supplying their own OS and container, and it's possible to run Kubernetes, Red Hat OpenShift, Amazon Elastic Container Service, or another container orchestrator on top of any type of underlying compute—ranging from bare metal to Fargate.

Which should you choose? Well, that's dependent on how your system scales, and also how your consumers use that application. Ultimately, you're going to want a mix of different capabilities to make it work optimally. For instance, you may want to run your database servers in the VM model, managed through Amazon Relational Database Service or Google Cloud SQL. Your user-facing frontends might run via serverless, and you might assign batch or streaming worker pools via Fargate or Cloud Run. The choice of platform is not a fixed, one-time decision. Over the longer term, optimizing the cost of compute requires using observability data to your load profile and deciding when to change even if there is some operational work required when migrating between compute models. And the options are not all or nothing; a hybrid approach may make sense.

If you have a very spiky workload, you may still want to try Lambda as a compute layer, but this could prove to be very costly (or result in persistent cold starts and refused connections) if you get a spike that's unexpectedly large. If, however, the time of executing Lambda directly correlates to your revenue, this may be fine. Observability for the serverless case is relatively straightforward, as the duration of trace spans straightforwardly converts to the total price—in fact, you can calculate

additive measures such as “request cost” and graph those dimensions directly in your tool of choice!

Observability plays a key role in gathering the data needed for deciding when it’s time to migrate onto an additional compute platform or retire an existing one. Concurrent execution counts—derived from either provider-exported metrics or from the number of concurrent trace spans—enable you to determine how much serverless concurrency to provision with your cloud provider. Utilization metrics enable you to determine when a sustained workload could be shifted toward a more server-ful or sustained-use model. Just be mindful of the human cost of managing and operating toward the physical end of wrangling real servers.

## Fleet-Wide Optimization

Once you have nailed down what compute model(s) you are using, the next step is making that compute cost as little as possible. There are three broad techniques to try: squeezing, optimizing, and migrating architectures. An architecture switch is a complex effort with correspondingly huge potential payoffs, but there are many smaller such experiments and projects we undertake on an ongoing basis at Honeycomb to optimize our costs and improve the performance our customers see.

Assuming you have set SLOs (see Chapters 10 and 12) and have a healthy culture of chaos engineering, the first step is to see how few machines you actually need to still achieve those SLOs. Reduce task counts, reduce CPU and memory allocations per task, and set autoscaling targets to higher target utilization—with the ultimate goal of shutting down VMs. In the past, standard practice was to run Intel architecture-based instances with hyperthreading or simultaneous multithreading (SMT) at 40% utilization, and Arm instances (or any instance with SMT disabled) at 70–80%. However, that recommendation may no longer hold for the kinds of cores you are using. Test until latency actually begins to degrade. Your observability data will come in handy, especially given that degradation may impact a small tail of complex but crucial requests rather than the overall system.

Always test against the most recent generation of hardware that your vendor offers. As this book has and will continue to go several years between revisions, any specific recommendations we might make would rapidly become out of date. After taking away instances and shrinking underutilized instances, checking the appropriate ratios of CPU to memory to storage may help you save money. Most cloud providers charge a premium for higher-RAM configurations, as well as charging for use of their managed durable block storage products (e.g., Amazon Elastic Block Store). For I/O-bound workloads, you may benefit from using NVMe drives directly attached to instances (e.g., AWS `-d` suffix and `i-` prefix instances, or Azure L-series instances) instead of durable block storage for data volumes.



Fleet-wide profiling (discussed later in this chapter) also comes in handy in terms of identifying code bottlenecks. For now, we'll note that those bottlenecks might not be in your own application code, but instead in OS libraries, the kernel, or common shared library code. If you can identify and solve a performance issue impacting multiple workloads, rather than just one workload, it may be well worth it. While saving 1% off of 1% of your workload is likely not worth spending more than a few minutes of engineering time, if that 1% generalizes across all of your workloads, it could be a meaningful amount of money. For more on this, see the paper by Gang Ren et al., "Google-Wide Profiling."¹

Another potential avenue to make large-scale performance and cost gains is via recompiling your binaries to run on a more efficient CPU architecture designed for the cloud. We've previously written on the Honeycomb blog about our switch to the [Arm64 architecture using AWS Graviton instances](#). Microsoft offers Cobalt Arm instances, and Google offers Axion Arm instances. Each generation of Arm processors appears to show [30% price-performance gains per generation](#), outstripping the rate of price-performance gains of the x86 architecture. All of these are worthwhile competitors to their equivalent-generation Intel and AMD competition, but do require you to potentially reconfigure or recompile your workloads.

Today, there are [many guides for carrying out an Arm64 migration](#), but the basics are to switch over container or OS base images to the corresponding architecture, then test that your container or binary build pipeline still works using the new architecture and new base images. You'll then need to test deploying the new architecture, sending it representative traffic, and then load testing to determine how it performs relative to your original configuration. Remember, because most Intel/AMD instances use SMT and Arm does not, you may find that it is safer to run your Arm64 instances more saturated since you will not be contending against your own workload. Always test to breakage/saturation to see the true performance and packing gains and not the superficial results!

## Cost Optimizing Kubernetes

If your workload uses Kubernetes, the tricks mentioned in the previous section can help you make the most efficient choices about your underlying architecture and sizing. Kubernetes also enables you to make more efficient use of machines by packing multiple workloads onto the same hosts, ensuring that a misshapen workload consuming a high amount of CPU but low RAM can be paired with one that consumes little CPU but maintains a lot of state in RAM, better utilizing the capacity without stranding any.

---

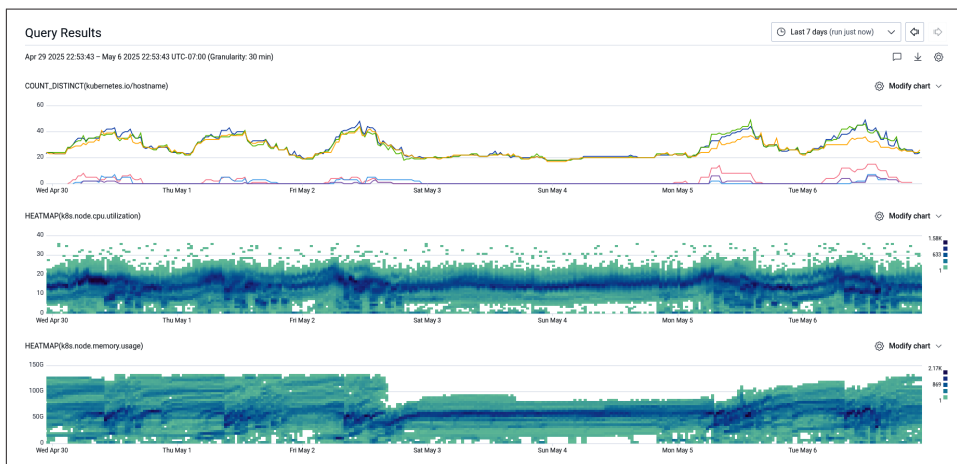
¹ Gang Ren, Eric Tune, Tipp Moseley, Yixin Shi, Silvius Rus, and Robert Hundt, "Google-Wide Profiling: A Continuous Profiling Infrastructure for Data Centers," *IEEE Micro* 30, no. 4 (2010): 65–79.



The first trick to deploying with Kubernetes—as it has the concept of fault tolerance and automatic task migration—is to embrace infrastructure flexibility. If your code doesn’t keep large amounts of local state and can tolerate periodic migration between instances with a lame duck period of approximately two minutes, you may find interruptible instances to be a good fit for your workload. With the Karpenter plug-in for Kubernetes (or ordinary autoscaling groups), you can benefit from instance flexibility on the spot or interruptible instance market. Just make sure you use the relevant Node Termination Handler plug-in to move workloads out of the way gracefully before the nodes are terminated!

Measuring the impact of your changes works with your current observability tooling. You don’t need any special tooling other than running the OpenTelemetry Collector on each node (as a DaemonSet) in order to understand and optimize the efficiency of your Kubernetes clusters. Installing the OpenTelemetry Collector with the kubelet stats receiver and k8sattributes processor is a power tool for understanding the health and behavior of your cluster.

The visualization layer is the most key when looking at Kubernetes cluster data. There can be a wide variance in the saturation and utilization of each pod and each node as a whole, so percentiles will often be misleading. Heatmaps, such as the ones shown in [Figure 20-2](#), are a preferable mechanism of looking at how your nodes are doing and whether you are “clipping” 100% saturation and at risk of requests being blocked on access to the CPU, slowing throughput and delaying responses to users.



*Figure 20-2. A set of charts showing the number of distinct hosts of each instance type in each availability zone, and aggregate heatmaps showing the amount of CPU and memory utilized for a time period spanning a week.*

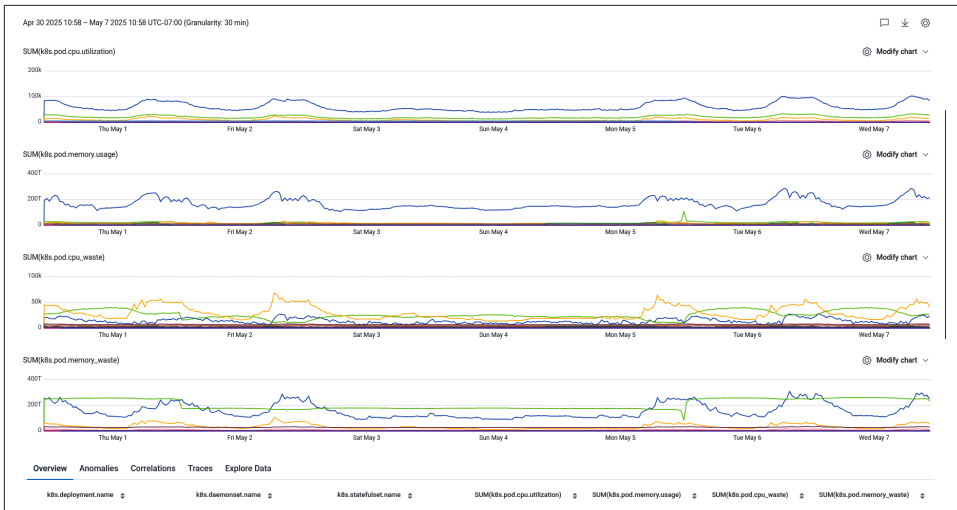
With knowledge of the load distribution on each Kubernetes node, it's possible to identify both hotspots where there is too much load, as well as undersaturation where nodes are not being adequately utilized. Statistical analysis on those outliers might reveal instance shapes that have particularly poor utilization, delays in scaling down, or other factors that lead to the fleet not running as expected. Any field that is attached to the metadata can help, such as node group, instance type, architecture, or availability zone.

In order to further debug the behavior of cluster scaling, Kubernetes helpfully emits logging data from its control plane, including from the Cluster Autoscaler (CAS) or Karpenter processes. Ingesting those logs into your observability tooling can **reveal the reasoning** behind the decisions to create or remove particular types of nodes at particular times.

One easy opportunity for optimization, even if not plotted on a resource chart, is to remove excess boot volume size. Disk space is most commonly used to store logs, but if logs are being exported to your favorite observability tool or to cold storage, you do not need to persist very many on disk. Also consider whether instance NVMe storage may be more appropriate than managed remote data volumes.

Moving beyond node optimizations, how can you identify the biggest opportunities for workload-level cost savings on Kubernetes? Plotting the sum of all CPU and memory used by each service name or DaemonSet name can enable you to find the largest consumers of CPU and memory to then examine the tracing and profiling data. See **Figure 20-3** for an example. After all, when engineering time is scarce, you're better off solving the problems with the largest impact first; saving 1% of 10% is better than saving 5% of 1%!

The Kubernetes metrics that will be your biggest friends are the pod CPU utilization, pod memory requests, and limit usage values. That usage is the genuine demand that cannot be eliminated through clever bin-packing and scaling tricks. So, at the end of the day, to progress any performance engineering efforts will require reducing that demand. However, you should be wary of potential pitfalls with observability tooling when examining usage by pod within a given deployment. The intended surge in pods during the rollout will inflate the usage for a wider time window, as will the appearance of pods that terminated just after the start of the time window, and pods that started just before the end of the time window. Thus, a service might appear to have up to double the number of pods that it normally has during an aggregation window, causing the sums of CPU consumed by all pods to appear artificially inflated. To avoid this kind of artifact, plotting each configuration template hash separately within the service avoids the apparent duplication of monitored resources.



*Figure 20-3. Four line plots showing total CPU and memory utilization and unused reservations of pods, broken down by deployment, DaemonSet, and StatefulSet names.*

In addition to seeing which services are consuming the most actual resources, you may find it helpful to view which services have *reserved* the most resources, and whether any of the resources are going unused. Some underutilization is an expected part of the slack present in any robust system, especially given the consequences of an OOM (out of memory) or cascading failure. However, excessive allocations of RAM or CPU may be starving other processes on the machine of resources they could benefit from and may cause your cluster to become larger than it needs to be.

Besides the resources we intend to run in a cluster, we need to understand how much overhead we are introducing for our observability and compute infrastructure itself, the Kubernetes compatibility layers, and any security needs. For instance, on our clusters, you'll find Lacework running, OpenTelemetry collectors, and internal Kubernetes services like node-local-dns, kube-proxy, and Container Storage Interface (CSI) drivers. These often run as DaemonSets, one per kubelet or host—and across hundreds of hosts, their overheads can add up.

When provisioning a Kubernetes cluster, don't forget about the reasons you adopted Kubernetes. If one or two workloads predominate in your cluster, you may want to consider sizing to allow for optimal bin-packing of combinations of the most common workloads' pods. It may not show as underutilization of an individual pod or service to have capacity lying fallow on a given machine, but if a given machine has 32 cores, and you have pods that require 20 cores each, then 11 to 12 out of the 32 cores will be potentially wasted once DaemonSets are accounted for. This is because you cannot fit two 20-core pods on the same machine, and DaemonSets should take up a maximum of 1 vCPU in total.

There may be other smaller jobs that can fill the cracks, but if your 20-vCPU job is the primary workload you run, inevitably, you'll be spinning up machines just to create capacity for that workload. It's better in that case to upsize to 64-core machines so three 20-vCPU jobs will fit on one machine, or to run a higher number of smaller tasks by reducing to 15 vCPU per pod, to allow better bin-packing. Also, be mindful of stranding CPU or memory on a machine, even if the CPU and memory ratios are appropriately tuned for the entire cluster. For example, if you have a workload dominated by high-CPU, low-memory jobs, ensure that there is still enough room after the high-CPU jobs are running to allow for the smaller amount of CPU requested by high-memory jobs.

## Cost Optimizing Serverless

For serverless, the underlying compute platform takes on the responsibility for managing utilization and reducing wasted resources. The primary cost driver is the duration that your functions execute for, but there are subtleties lurking in the weeds. For instance, startup time and cold starts result in billing, as does the amount of CPU/RAM that you assign to each function, and the cost of the API calls required to invoke your function and that the function itself makes.

Your cloud provider's metrics may be a good start for understanding the amount of time taken by each function. For instance, Amazon CloudWatch offers metrics for the number of concurrent executions, invocations, and execution duration. Similarly to earlier sections about not using the resources that you don't need, taking away excess memory and CPU will save you money—to a point. That point comes when increased execution time or instability such as OOMing makes further reductions no longer worthwhile. Lambda Power Tuner can be a powerful tool for minimizing the cost function that is a product of CPU and execution duration, by executing your function many times with different sizing parameters.

Cold starts—where your application is not yet running at the time that your provider needs an instance of your function to serve a request—don't only incur a latency penalty to the end user. Cloud providers have pushed the cost of initialization work done during a cold start onto the client, meaning that you are billed for the time it takes to become ready to serve. Such work might include setting up fresh connections to dependencies, loading datasets into memory, or more. To measure and lower this cost, consider implementing observability into the runtime cycle of your serverless applications. You'll need to implement OpenTelemetry instrumentation into your main function, create a span at the start of the main function, and end it just after setup has finished but before allowing the serverless runtime code to take over execution. Then, for each invocation, create a “running” span for the duration of execution, and persist a “sleeping” span started just before the function is frozen to be sent once the function is thawed.

Be aware that the costs of getting data to your serverless functions may be more significant than you expect. For instance, the cost of S3 read requests is approximately 25% of the total cost of running the serverless functions that power the Honeycomb Retriever storage engine (see [Chapter 13](#)). Reducing the costs of compute can only contribute so much to reducing the total cost of ownership as a result.

While provisioned concurrency may appear attractive, it essentially trades money for cold-start latency and gives you the worst of all worlds in terms of cost. You have to handle manual capacity planning, you don't receive a meaningful discount in exchange for committing to usage, and you're still unable to amortize setup costs across longer application lifecycles, as you would with a persistent application.

## Observing the Costs

Making your workload execute faster will almost always be the most effective way to lower your compute costs. Whether that's optimizing a serverless function's execution time or your API hosted in a container, no savings actually manifest until you consume less of whatever the billing dimension is. The question is, how do you safely reduce resources and ensure that you're not adversely affecting the user experience?

There are multiple dimensions to the cost of your compute layer. It's not simply "time," because the different machines and instance sizes also have an impact on price—otherwise, you could just throw more compute resources at your workload to make it "faster"! So, you'll want to identify what dimensions to optimize.

However, consider also whether giving a workload more memory or faster network might actually make the execution happen faster too, and avoid otherwise idling or wasting resources. Observability can help find these bottlenecks by showing which steps are the slowest. Conversely, it may be that a majority of your workload cost is not the compute but is instead some ancillary dimension such as networking or storage.

Ingesting your cloud bill into your observability tooling can help identify opportunities for improvement, focusing on the greatest sources of spend first, and drilling down into what common tags or attributes those share in common. Tagging also ensures accurate chargeback or other attribution of cost to use cases and potentially to individual tenants in an SaaS model. The rows in the cost and usage report translate well into wide events; the cost in dollars is one field, the service is another, the usage type is another, and so on and so forth. Each tag on a resource becomes a column, and each value of the tag becomes the value's field. Aggregations work the usual way via performing summing operations grouped by resource, tag, and so on.

Observability tooling also contributes to the cost of running a workload, but can be a valuable source of data for investigations. For thoughts on how to improve your ratio of value to cost, see Chapters 15, 26, 27, and 29 for build-versus-buy and cost levers that you can use.

In addition to focusing on the highest consumption tags and services, it can be valuable to look for quick wins that require no code change. For instance, if you have a consistently high utilization of raw compute VMs, your service provider may have savings plans or committed use discounts available. Having the data available to see what the minimum hourly spend is will enable you to rightsize a commit and ensure that it will be effective over the contract term.

Likewise, if you are spending a lot of money on load balancers, network address translation (NAT) gateways, or data transfer between availability zones, consider reconfiguring your topology. Keeping data transfers within the same availability zone—going directly from node to node—avoids unnecessary costs for shuffling bytes around. In some cases, the cost of the network can outstrip even the cost of the compute or the storage!

## How Application Observability Reduces Cost

Having insight into the performance of your workload enables you to gain both direct and indirect benefits. When you improve performance, you can frequently remove resources to get the same performance as before, or reduce waste that was caused by the additional usage. However, as you’ve learned in the previous section, it’s important not to waste time on the smaller or less vital components of your application. Only focus attention on optimizing the most resource-intensive or most user-visible parts of your application.

In order to identify where you’re spending the most time for your most important requests, a first step is to use distributed tracing (as we’ve discussed in the rest of the book), and wrap major functionality in spans. By visualizing the concurrency of spans and the sum of their duration, you can get a quick and dirty idea of what the critical path is and what you might want to optimize. Anything that makes a call to another service or performs a computationally expensive operation on data is worth wrapping in a span. The good news is that your existing observability should cover these cases well through automatic instrumentation of remote procedure calls (RPCs), plus manual instrumentation to fine-tune coverage for internal spans within your services. Always aim to account for where your application is spending its time to the nearest tenth of a second (hundred milliseconds). Long trace spans with no activity and no child calls suggest mystery-meat time that could be better accounted for by adding subspans with incremental detail.

Due to the overhead of instrumentation, there is a limitation on how much detail you can obtain per span. If you were to wrap every function in a span, you would slow down your application and do just as much instrumentation work as real work! Instead, you need to consider how to aggregate smaller units of work into larger units of work that measure in tens of milliseconds or more, rather than in microseconds or single-digit milliseconds. For that finer degree of detail, you'll need to use profiling at the individual function level. We'll discuss this approach in the next section.

## Using CPU Profiling Tools

How, exactly, did we get the insight into where our code was slow in the original example at the start of the chapter? `pprof` is the name of the specific tooling that the Go programming language offers, but the generic technique is called profiling. Profiling allows for determining how much time programs are spending in each function, line of code, and even assembly instruction. The key insight behind profiling is that we can interrupt a program at any point with OS signals, and retrieve the previously running program counter (the assembly instruction currently being executed) and the stack frames above it. A lightweight interrupt handler can record the stack info to a ring buffer, and we can periodically read the ring buffer to populate maps with counts of how often a given program counter was observed, and with what parents.

Given a binary with appropriate symbol tables, we can symbolize the program counter to lines of code and function names. Sampling the call stack every millisecond might be overkill and introduce too much overhead, but sampling every 10 milliseconds might give a picture of which assembly instructions and lines of code are most likely to crop up. Over time, a statistical picture of a program approximates reality—similar to sampling of distributed traces, a single sample might not be representative, but a thousand or a million recorded datapoints give a pretty accurate picture of the percentage of time spent in each function or assembly instruction.

We're big fans of distributed tracing, but it's not the best tool for every job. Profiling enables us to get granular detail, instead of having to create a span for each and every function call within a service, let alone each line of code.

### Setting up and interpreting profiling data

Profiling Go binaries as a one-off is remarkably straightforward, thanks to features of the Go runtime. There's a built-in HTTP handler that you can enable on your application—but remember to set the HTTP server listening on a different port that is not internet visible. Then, `curl` the endpoint while the behavior you're looking for is manifest, save to `$profile`, wait a minute, and then you'll have a file that you can feed to `go tool pprof $profile`. We recommend running with `-http :8080` for a friendlier web user interface; you'll also want to supply the original source code directories for line-level detail, and the original binary corresponding to the running



server in order for assembly-level detail. You can also use `-profilecpu` with `go test -bench` to write profiles for your canned benchmarks.

In other languages, you'll need to consult runtime documentation—for instance, Java uses Java Flight Recorder (JFR). In the worst case, you might need to use the `perf` tools bundled with your Linux kernel, or use an eBPF collector agent. Regardless of how you collect them, what should you do with the results of profiles?

Profiling tooling typically defaults to either a listing of source code with statistically estimated numbers of milliseconds attached, or a directed acyclic graph (DAG) rendering—with boxes representing functions and their total time, lines representing calls between functions, and line weights representing seconds spent on each path in the call tree. Both of these can be somewhat obtuse to work with, so instead we recommend a different approach.

If you took the advice to run `go tool pprof` with its web frontend, you can open up the visualization menu and choose a flame graph instead. Flame graphs are a more intuitive way to work with the data. Width represents the total amount of time spent in a function and its children, stacking one function beneath another represents a child function call, and individual icicles or flames represent collections of full stack traces sharing the same functions in order in the stack frames.

This enables visualization for glanceability of three things that are not obvious when looking at raw source dumps or the full DAG:

1. What the most expensive part of the code is
2. If it's the code or the libraries being called
3. How much is attributable to one function itself versus its children

The best targets to optimize are those functions that are called *often* with medium-sized running time (e.g., the library function that is called from 10 different places and constitutes a significant chunk of the running time of each callsite), or those that are often found close to the root of the flame with little time spent in their children. This has parallels to how you might reason about breaking down traces and optimizing the single largest blocking request preventing completion of a traced request—except instead of each bar representing a specific request or subsection of a request, each bar represents the aggregated sum of time spent making calls to a given function, even if many different shorter calls were made.

## Scaling out profile collection and analysis

While it's handy to catch something misbehaving in staging (or in a unit test or benchmark), problems rarely manifest in the real world while we're looking. And by the time we're alerted, they may have already disappeared. We need our systems to



give us the telemetry continuously for us to interrogate later, so that we don't need to catch them in the act. This is the whole reason why we have observability, right?

Fortunately, you can either run profiling continuously with agents that hook into the runtime, or continuously scrape the profiling HTTP endpoints on your applications. The data volume is manageable because it's highly redundant—while each minute-long profile segment might be five megabytes (MB) in size, two hours of data is not going to be 120x as expensive. The same function calls and even lines of assembly repeat often, and the data compresses nicely.

This is the value proposition behind continuous profiling tools such as Grafana Pyroscope, Polar Signals, and zymtrace. They handle the lift of both the agent and of the collection and visualization pipeline so that you can get answers when you need them. That way, you won't be trying to remember which debug endpoint to curl, or how to run the pprof debugging command at 2 a.m. We're believers in running less software at Honeycomb, and this is one wheel we were glad to not have to reinvent. With OpenTelemetry offering a common profiling agent, SDKs, and encoding of profiling data, the vendor-neutral future of the profiling observability signal has never looked brighter.

## Using the Correct Observability Signals, Together

Engineers tend to prematurely optimize: we will jump into optimizing SQL queries or look for loops that could be trimmed with early exits. However, without knowing that these are actually the biggest impact on your code, it's essentially educated guesswork. For seasoned engineers, those who have spent time in that ecosystem or that domain, this can actually be very effective. But how do we give all engineers this capability?

To some degree, profiling requires you to already know you are interested in a very specific portion of your system. Therefore, it's a tool that is best used in conjunction with a tool like tracing. Using tracing—both from an analytics perspective and from a waterfall-view perspective—enables you to validate hypotheses quickly.

### When to use each signal type

In our opinion, tracing gives you a bird's eye view and enables you to navigate the system as a whole. High-cardinality data analysis enabled by the unified observability model equips you to explore individual fields and their values to understand outliers and weird behavior. Analyzing hundreds of millions of traces can give you astonishing insight into what real customers are experiencing in the system. But sometimes you really need to grab the microscope to look at the rings (lines) of the tree (code) rather than look at the forest of microservices as a whole. Profiling is that tool.

First, you might begin an investigation by asking:

- Is this endpoint normally slow?
- Is this database query slow only when it's called for this API endpoint?
- When this endpoint is slow, is it always this database call causing it?

Armed with this information, you can establish before, and after, your change whether you're targeting the right area of the system to make the biggest impact. Having a scientific or engineering mindset involves employing the scientific method to hypothesize testable changes and then performing the experiments required to validate—not just throwing spaghetti at the wall. Sometimes, your change will be visible in the profiles from processes at large, while having only a small change at the trace level. Other times, it'll be the reverse: much faster traces, but with the percentage of time spent in each function remaining unchanged.

### **Blending performance engineering and observability**

What if you didn't have to make the trade-off between request-level but outer-function-level granularity for tracing, and inner-function-level but only process-wide granularity for profiling? When profiles are indexed at the level of individual seconds of data, or are tied to individual trace spans, magic happens. You can understand what was slow, for whom, why, and how, all at the same time instead of guessing.

It's premature optimization to write the very most efficient, inscrutable code everywhere. But having the visibility post facto to know what's actually being called in a tight loop and needs to be ratcheted down enables you to optimize just in the right places, with an explanation as to why you're doing it. We admit to feeling skeptical about the “day 2+” value of profiling. Wouldn't we just squish the “obvious” bugs and then stop finding value? Well, that's not true. You're always adding new services or changing them, and creating new bugs along the way.

The golden example of this discovery for us at Honeycomb was finding that we were serializing a very complex query—containing nested derived columns—from protobuf to JSON on behalf of a user (see [Figure 20-4](#)). We were also then invoking Lambda hundreds of thousands of times, doing the serialization each time for each invoked Lambda—on a single Retriever merge node that was already running very hot from coordinating other queries. While our Lambda functions are massively parallel at mapping across millions of files on S3, the nodes dispatching Lambda invocations have finite resources.



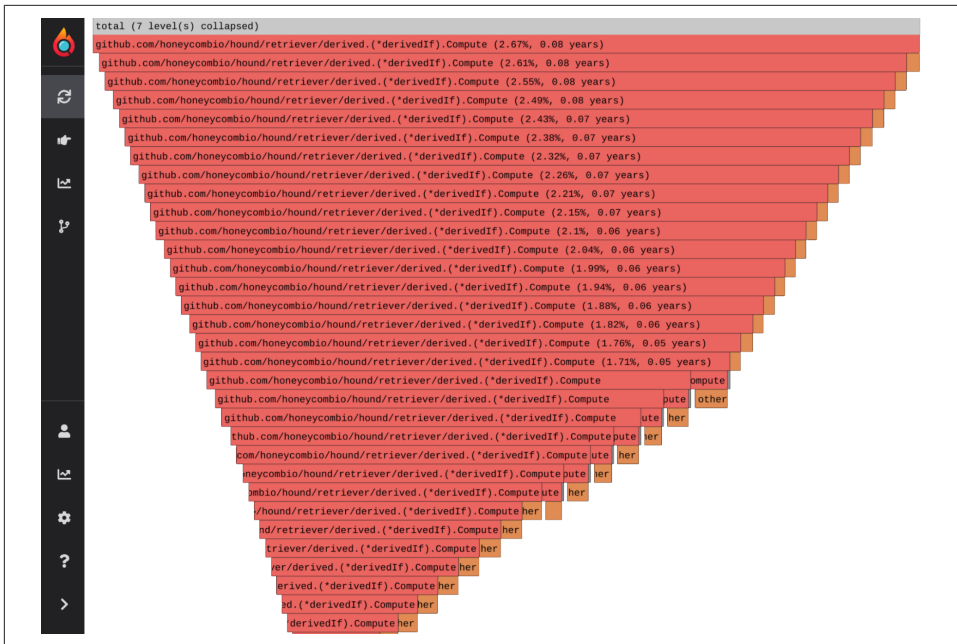


Figure 20-5. Time spent evaluating a deeply nested set of IF conditional blocks in the Honeycomb calculated-field language evaluator.

The customer was using an `if { } else { if { } else { if { } else { if { } else { if { ... } else { ... } } } } }` pattern, which we could simplify by offering a switch statement. Once we transformed away the complexity of the query and refactored the customer’s query to use the new syntax, execution was dramatically faster.

This behavior would never have turned up inside of our traces, as we simply cannot afford to trace each function that’s called when processing each record in the inner loop. But profiling on a high-cardinality, per-customer basis showed us what was wrong and how to fix it.

## Conclusion

Like the practice of observability, performance engineering is not a one-and-done activity. Instead, it is a scale with diminishing returns toward the very top end, and large gains at the beginning. And it must be continuously reinforced. Incremental 0.5% or 2% improvements lead to compounding benefits over time, even if they are not as flashy as a 30% performance optimization on a fresh, unprofiled codebase. The core loop is: measure to find potential improvement, do the improvement, measure the effects, and repeat.

By late 2024, when we were reviewing options and faced the choice of what continuous profiling tooling to employ, it no longer was just one singular engineer that wanted a say in the requirements. Instead half a dozen different teams at Honeycomb each saw benefits from being able to proactively investigate performance.

Our finance and engineering teams were also collaborating on cost tooling, leveraging data from our AWS bill and from our application and Kubernetes cluster telemetry to model and predict future costs (and to be able to bake predicted future optimization into our cost modeling).

Performance engineering, much like observability-driven development, thrives on shared knowledge and continuous practice. With a robust observability foundation, any engineer can own and optimize their code in production. But as the technological landscape evolves, so too do the systems we observe. In the next chapter, we will explore the cutting edge of this evolution, diving into how to use observability with LLMs to embrace observability in the modern age of AI-driven systems.



---

# Observability for Large Language Models

Artificial intelligence has been a revolution over the past two years. AI and machine learning have been powering both transformative consumer digital experiences (such as recommendation engines, search engines, and spam protection) as well as business workflows (such as translation and logistics) for decades. However, the introduction of LLMs has ushered in a new era of AI-powered applications and tools. Organizations have eagerly embraced the potential of LLMs for programming assistance, natural language interfaces to data, new categories of search products, support bots, and more.

While LLM applications often impress through a small up-front investment to prove a concept, the real work happens after the demo. LLMs are unlike most traditional software workloads, and the unpredictability inherent in their design and use leads to a fundamental challenge: users and engineers alike expect predictability in the outputs of a software system, but LLMs are fundamentally nondeterministic. How can we square this circle?

In this chapter, we'll dive into the realm of observability for this new generation of LLM-powered applications. You'll learn about how to continuously improve the performance and user experience (UX) of these applications through evaluations. Beyond the theory, we'll use case studies from two different LLM-powered Honeycomb products—Query Assistant and Canvas—to demonstrate practical applications of what we're discussing. We'll introduce each of these products as we go through the chapter.

Phillip Carter served as a principal contributor for this chapter.

# Why Observability Matters for LLMs

LLMs represent a step change in the capability and accessibility of machine learning models for both organizations and developers. Traditionally, AI has been used to solve various problems for users where there was no one “right answer”—like recommending a movie based on your taste, or determining if an email is legitimate—but usually at a great cost. LLMs have changed this calculus dramatically by lowering the barrier of entry to “anyone with a credit card” simply by issuing prompts to a model via an API.

Traditional observability and monitoring approaches are not suitable for understanding these types of applications, though. While helpful at establishing a high-level signal or monitoring cost, traditional approaches don’t give any insight into behavior that leads to an undesirable outcome. The primary reason? Opacity. You can’t connect a debugger to an LLM to understand why it gave a specific output for a given input; you can’t even guarantee that it will produce that output again, irrespective of how many times you give it the same input. While there are parameters—such as temperature and top_p—that control the distribution of outputs, they **do not yet guarantee repeatability**. Confounding our attempts at sense-making, the spectrum of natural language inputs is an impossibly broad space. Your users will invariably find the most interesting ways imaginable to ask for the same thing—this assumes, of course, that your users are *also* not trying their darndest to make your LLM say or do something funny.

In addition to all of this, LLMs are advancing at a breakneck pace. In 2025 alone, models leapfrogged their capabilities within a year thanks to improved tooling, harnesses, and advances in techniques like tool calling. Traditional methods of iterative design and development can’t keep up with this manic pace. Even in a future world where the rate of progress slows, it’s still beneficial to iterate rapidly when working with LLM applications. Adjusting model behavior is often a question of adjusting *prompts* (natural language instructions given to an LLM) rather than pushing deterministic code changes. Achieving greater accuracy, reliability, or task completion rates often involves both.

This model stands in stark contrast to the conventional wisdom for understanding the UX of web applications. Popular models of quantifying this—such as Apdex or other derived measures—primarily use API or endpoint speed as a proxy metric. In 2006, Greg Linden **presented that Amazon’s latency experiments** found substantial revenue drops for every 100 milliseconds of added page delay. Similarly, in the same year, **Google found that a half second of latency** resulted in a 20% drop in traffic for search!

This does not hold true for AI applications—while measurements such as “time to first token” can be valuable, fast (but incorrect) responses may not drive satisfaction.



In addition, in more complex agentic applications, such as research assistants or code reviewers, failure may not be a valuable signal in and of itself. An agent may fail to fetch a web page for many reasons, or pass incorrect parameters to a remote API. In these cases, it's less important that the discrete operation failed and more important to understand if the overall task completed. In addition to these, AI applications introduce a very discrete cost component. GPU time is valuable, and is priced accordingly. Understanding the trade-offs between token usage and user satisfaction is of paramount importance to not just the business, but also engineers.

Opacity of system internals, rapid technological advancement, and poor proxies for user experience in telemetry may not be *completely* unique to LLM applications, admittedly. It's ironic in some ways that these three points mirror the basic tenets of observability, as applied to the emergent behavior of a distributed system. We expect that our systems will fail, we expect that other services are untrustworthy, and we know that everything has a price. The answer in both cases is the same: we need instrumentation and telemetry that describe the system behavior. This is what makes modern systems interpretable, knowable, and improvable at scale. But what should we test? We need some sort of yardstick to measure our system performance against. For this, we turn to *evaluations*.

## Using Evaluations for LLM Reliability

**Evaluations** (evals for short) are a foundational component of building reliable LLM applications. While they bear some resemblance to unit or integration tests—in that they have an expected output for a given input—they introduce a level of indeterminate expectation that sets them apart. In short, an eval can have three results: pass, fail, and *maybe*.

The power of evaluations is that you can blend them together and build them into your observability program in order to achieve reliable, helpful, and useful LLM applications. For example, at Honeycomb, we use a set of evals as a key input to the development of our Canvas product. Canvas is an AI chat experience, where users can ask open-ended questions that the model answers by calling tools allowing it to query telemetry data. Let's walk through these evals in order to explain how they work.

We have two sets of evaluations: question evals and task evals. The first are a series of questions that involve extracting factual items from a set of *golden data*. Golden data is hand-crafted (or otherwise representative) input data for a task. In the case of Canvas, it's a variety of query results from Honeycomb. We know exactly what is in this data, and we evaluate the model's ability to extract specific facts, such as “at what time did the most requests occur” or “which category of errors was most prevalent.” These questions are more precisely known as the *task function* of an evaluation—it's what generates the output that would be presented to a user or passed back into

an agent loop. Finally, we have a *scoring function* that is used to grade the model’s response. For our purposes, we use a simple string-matching heuristic—we know the actual values present in the golden data, and we simply verify if the model presented this known answer.

These question evals are paired with another set that we call “task evals” (see [Figure 21-1](#)). These differ in that they do *not* have a simple scoring function; they are fuzzy, subjective tasks based on our internal production monitoring. Rather than asking for something with a single answer, we ask the models to perform investigations, determine the root cause of a performance regression, or other similarly judgement-based workflows. There is often no single “right” answer here, so what we score them on are the actual steps taken by the model. In order to do this, we must have telemetry for the execution of a workflow—and this forms a positive feedback loop, where we can use our telemetry to identify scenarios of user unhappiness or where the model performance is lacking, and then turn those into evals that we can iterate toward improving.

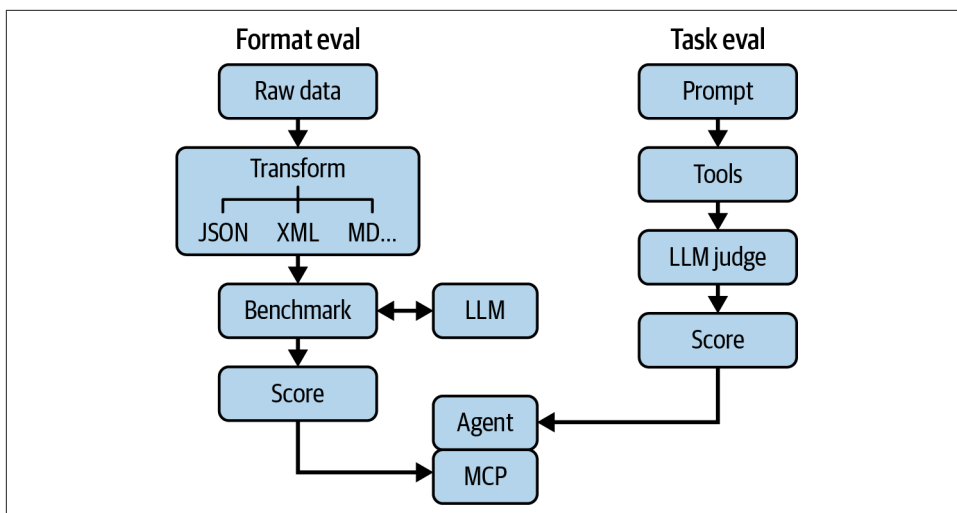


Figure 21-1. Independent evaluation loops.

It’s this loop that is at the heart of driving continuous improvement for LLM applications. Having a reliable set of evaluations offers you the opportunity to experiment. When a new model version is released, you can run your evaluations against the new model and measure the change. You can replay existing sessions against changes in prompts or tool harnesses and track the differences in behavior.

While a full discussion of eval design is outside the scope of this book,¹ it's important to remember that your evals are similar to unit tests. Ultimately, they only test a small portion of the overall function of an LLM application—the part where the application interacts with the language model. You can have great evals and a terrible UX; this is where observability fills the gap. Achieving observability for these applications starts with telemetry.

## Designing Telemetry for LLM Applications

Instrumentation and telemetry for LLM applications is conceptually similar to instrumentation for any other application in that you care about inputs and outputs, requests and responses, the duration of API calls, and so on. Helpfully, quite a bit of this instrumentation can be generated automatically via built-in OpenTelemetry instrumentation in LLM client libraries. Modeling for most LLM interactions can be found in the OTel [generative AI semantic conventions](#), and libraries that interact with LLM providers should follow these.

If you look at the conventions, you'll see that common values such as input and output token count, model name, sampling parameters, and system prompts are recorded on client spans. If you are using your own LLM client framework, you'll want to follow the conventions in order to create maximally portable telemetry data.

There are a few unique considerations and questions that you'll need to answer when designing telemetry for your application, though:

*How much visibility should you have for inputs and outputs?*

User prompts to an LLM are impossibly varied, and can often be incredibly large. This is discussed at length in the conventions, so we'll avoid rehashing all of it here, but it's worth considering. There are a few different ways to deal with this, but one to consider is that you're possibly recording the inputs and outputs of a model independently from your telemetry—rather than writing the inputs and outputs to an attribute value, add a link to the system of record as the attribute value. This has downsides (namely, you'll need to follow those links to actually see what was in a session), but it's better than not having any visibility at all. You can choose to elide only certain inputs or outputs this way (e.g., binary files like images or PDFs). You should also consider user privacy expectations and regulations when designing telemetry. Users can—and will—use poor discretion with LLM systems, and you should be considerate to yourself when thinking about what you want or need to save.

---

¹ For more details on evaluations and evaluation frameworks, please see the [“Evaluating Large Language Model \(LLM\) systems: Metrics, challenges, and best practices”](#) article for a good overview that includes more depth on specific metrics and measurements, along with some practical examples of building evals.

*How are you integrating your eval loop with your telemetry?*

Ideally, you should be able to “promote” certain traces of user interactions to evals easily. This goes in tandem with the prior consideration of inputs and outputs. Helpfully, many off-the-shelf evaluation frameworks also support OTel, enabling you to make an apples-to-apples comparison between a user session across two different models or prompt variations.

*How should you approach cost tracking?*

As LLM usage (and spend) continues to rise, organizations are demanding a greater ROI—or at least the ability to attribute value to their monthly AI bill. You may note that the semantic conventions don’t include a standard attribute key for cost data. You may elect to create your own, or you can derive this cost at query time based on the model and amount of tokens used. We recommend the latter option, as it allows you to easily create eyeball cost differences between different prompt versions or model updates.

*How are you integrating user feedback tracking into your telemetry?*

Traditional observability approaches can use proxies for UX, such as errors or API response time, as a primary signal of UX. This isn’t necessarily the case with LLM applications. Fast but incorrect responses are unlikely to spark joy in your users. Adding UX attributes such as feedback signals (thumbs up/down, user retry, etc.) to your telemetry is a powerful technique to better understand how your system is performing in production as well as to identify new evals.

LLM applications and features may exist as islands of functionality in your system, but most of the time they need to interact with other services in your backend for everything from authorization and authentication to databases. As of this writing, an emerging pattern for AI applications is the agent pattern: an LLM calling tool in a loop, with the ability to modify its environment. Agents require access to internal and external APIs for all manner of functions and functionality—everything from contextual data about the user that’s interacting with them, to historical records of prior conversations, to embedded or third-party knowledge bases and remote APIs.

While they’re often being developed as greenfield applications, agents interact and touch almost every aspect of an existing system. The key insight is that telemetry design for LLMs isn’t just about seeing what the LLM is doing or how users are interacting with it, but about the connections between the AI part of the application and the rest of your system.

## Analyzing Telemetry for AI Applications

As mentioned, natural language inputs to an AI application are impossibly varied, complex, and nondeterministic. While you may have telemetry data recording every aspect of an interaction with the AI, looking at that data along a single axis often

results in unsatisfying results. Looking at just errors may give you a priority list of bugs to fix, but this may not move the needle on the overall user experience of your product.

Broadly, it's useful to think of your analysis workflows in terms of two different outcomes:

1. Ensuring that the deterministic aspects of your AI application are functioning properly
2. Building a continuous improvement loop for the nondeterministic aspects of your application

The former outcome has been written about at length in this book, and much of the same advice applies. Multidimensional analysis of attributes enables you to find bugs, spot performance regressions, and improve overall performance. A common example of this, as of writing, is using observability to understand upstream LLM providers as a resource. Tracking rate limiting, retries, and response time from a remote provider is a key detail in both improving UX as well as managing your cost profile.

A specific example of this is using telemetry analysis to determine cache behavior. LLM providers heavily encourage callers to leverage token caching, and incentivize this through significant cost reductions. Telemetry can help you identify opportunities for caching, or to discover where you're incorrectly invalidating your cache. Providers will usually provide this information in their API response, so be sure to include it on your spans!

Additionally, due to a variety of factors, upstream model providers will often time out or fail to service otherwise-valid requests. Being able to track and monitor these through service level objectives (SLOs) is a valuable use of your time. Often, engineers will reach for threshold-based alerting to notify them about problems such as this, but these alerts are fundamentally unactionable and should be avoided (unless, of course, the reader happens to be part of an LLM inference provider, in which case you probably *should* have threshold alerting for things like this), as there's little you can do about it. SLOs are a better option here because they smooth out these intermittent spikes and help your team focus on meaningful degradation to UX.

The Honeycomb Model Context Protocol (MCP) server is an excellent example of this flywheel in action. When developing the MCP server, we only had a limited dataset to pull from in developing evals and gathering feedback about the real-world performance of our tools. Compounding this, the instrumentation and telemetry available to Honeycomb engineers is rather idiosyncratic compared to both the average Honeycomb customer and the average organization. Honeycomb, after all, is an observability tool—our engineering team is accustomed to having a tool available that supports high-cardinality data. In order to understand how customers were

using the MCP server, we needed to instrument the tooling in such a way that we could not only see what tools customers were using, but also where those tools were failing. We paid special attention to errors in this process in order to determine where to invest our engineering resources. An MCP server is an interesting case, in that you cannot see the caller's logic or rationale behind why it's using a particular tool or passing in specific parameters; you can only see what it's actually doing. We decided to set an SLO of 90% success rate for queries and waited to see what happened.

In doing so, we discovered several interesting facts. First, calling models tended to jump to conclusions about field names, even when prompted to first check available fields. Second, we noticed that models had significant issues with interpreting and modifying time ranges. This feedback informed our evaluations, giving us entire classes of errors and issues that we could build test cases for and start to iterate on prompting, tool descriptions, and more.

This basic flywheel is uniquely suited to the development of AI applications, irrespective of their type. Telemetry data becomes the golden standard for understanding how users are interacting and engaging with your application, which provides the fuel of continuous improvement for your development process.

## Feeding Observability Data Into AI Application Development

There are two primary ways that you'll want to integrate observability data from AI applications into your development cycle. The first, as mentioned, is as a source for evaluations. The second is as feedback for your software harness. What does this mean? Broadly, the harness is the deterministic code that you're building around the AI in your application. This harness is responsible for everything from providing tools to the AI (methods and functions it can call to perform work), to correcting errors in inputs or outputs, to how the AI communicates with end users. It's the plumbing, so to speak, between your application and its various services and the language model itself.

While it's important to do this for any application, it's more urgent for AI applications precisely because they're unlikely to behave the same way across environments or users. In a traditional application, you can build voluminous amounts of test cases that cover practically every interaction a user may have, and then run those tests across a homogenous array of environments. This is not the case for AI applications. There is no amount of testing that will possibly capture every interaction, and the pace of innovation in AI is so high at this point that there's little guarantee that the decisions you make today will have a shelf life of over six months. You *must* test in prod.

It can be appealing to try and address all challenges through improved prompt engineering. But while this is an important aspect of building AI applications, often a little code will go a long way. An example of this is the Honeycomb Query Assistant feature. When released, it produced an error of some kind about 25% of the time. This high rate of errors is quite common for LLM-powered features at the outset. When Honeycomb developers looked at some of the most common errors, they found that many were parsing or validation errors—the response that OpenAI gave back was a JSON object whose structure couldn’t pass validation checks. For example, consider the following JSON object:

```
{
  "calculations":[
    {"op":"COUNT", "column":"exception.message"}
  ],
  "filters":[
    {"column":"exception.message","op":"exists"},
    {"column":"parent_name","op":"exists"}
  ],
  "breakdowns":[
    "exception.message","parent_name"
  ],
  "orders":[
    {"op":"COUNT","order":"descending"}
  ]
}
```

In the Query Assistant feature, the object attempts to represent a query for “exception count by caller”—a common scenario that users use Honeycomb for. However, it’s subtly incorrect. The structure that represents a COUNT operator is wrong, because COUNT doesn’t take a column as input. A correct response, like this one, would not have included the column:

```
{
  "calculations":[
    {"op":"COUNT"}
  ],
  "filters":[
    {"column":"exception.message","op":"exists"},
    {"column":"parent_name","op":"exists"}
  ],
  "breakdowns":[
    "exception.message","parent_name"
  ],
  "orders":[
    {"op":"COUNT","order":"descending"}
  ]
}
```

In this example, the first object is mostly correct. In fact, programmatically removing the column field from the COUNT calculation object not only makes the query object fully valid, but the resulting response from this post-processing is an ideal query to generate given the input. And so, instead of failing validation and creating no result for the user, Honeycomb decided to intervene on this kind of error by correcting the structure programmatically so it passes the validation check.

As a result of analyzing outputs like this and issuing only these kinds of manual corrections, the error rate in Query Assistant dropped from 25% to about 14%. Later, improvements to a prompt brought the error rate to less than 1% (excluding timeouts or other uncontrollable errors).

Finally, an interesting sidebar to this story is that the broader engineering team realized these AI-generated queries were actually representative of an unmet need with our users. People actually *wanted* to be able to COUNT with a column, and we shipped a change allowing that query syntax. This, helpfully, meant we could reduce some of our error correction code and improve the performance of Query Assistant as well!

## Using Evaluations and Observability Together

If you're already collecting inputs, outputs, and errors through production traces of your AI application, then you're 90% of the way toward an effective eval flywheel. You have real data about how users are actually using your feature: the sort of things they ask for, the data sources and tools that the LLM uses, and the responses. You can export this data—or, in a purpose-built tool, generate new evals from it—and begin to update your evals based on what's actually happening.

The basic loop here is as follows:

1. Are the new inputs something we want to evaluate against? Generally, the answer here is yes. Then:
  - a. For these inputs, is there an ideal output? Was the existing output the ideal output?
  - b. If not, update the output to what it *should* be based on the inputs.
2. Repeat step 1 until you've fleshed out your golden dataset with real user behaviors.
3. Remove items from the golden dataset that are *not* representative of real user behaviors.

Rerun your evaluations and compare with historical trends. If this is a new application or workflow, you'll most likely see a dramatic drop in eval performance—this



is to be expected, as real-world use cases often intersect with things that you didn't expect.

A practical example of this is how we update our task evals for Canvas. As mentioned earlier, the task evals are a very general goal with a known answer. We care about how many steps it takes for a model to arrive at this answer, how many tokens it uses, and if the answer contains a few specific conclusions. We use **LLM-as-a-Judge** to score the response into a simple pass/fail based on some criteria, then select a sample of these to review by hand on a regular cadence.

This manual review is one of the most important parts of effective observability (and evals), because it surfaces insights that the LLM might miss. We noticed that a particular task had a persistent failure due to the model misinterpreting the meaning of a particular column in the result data. This column was named `is_slow` and was set to true when a request took over 500 milliseconds to complete. The model was unable to see that this column was a calculated field (a Honeycomb query-time construct that is able to create a new value based on other attribute values in an event), and it assumed that the column was actually a *feature flag*. Because it couldn't find any other reason, the model's logic was that the `is_slow` attribute meant there was a load test (or some other external event) happening that made the request slow, and incorporated it into its result. This misinterpretation caused the model to ignore the underlying factors contributing to latency—it effectively went off on a wild goose chase.

Our solution? We updated the column to include a description of what it actually was, and suddenly the model started ignoring it! It then was able to successfully investigate and identify the underlying cause. In practice, this is an incredibly common failure mode for AI agents and AI applications: the model infers or imputes meaning incorrectly, leading to incorrect assumptions and conclusions. Good observability and good evals enable you to catch these sorts of things in production and update your software harness to correct them, or update your prompts to better steer model behavior.

A final note here on public benchmarks and generic metrics: many organizations have published and promulgated various measurements for things like hallucinations, toxicity, tone, truthfulness, bias, etc. Frontier labs will evaluate their models against these published benchmarks and produce very pretty graphs demonstrating how they stack up. These benchmarks and measurements are not *bad*, per se, but they can be highly *misleading*. You may find that your AI application scores quite well on these benchmarks while utterly failing to deliver value to end users. Put in the extra effort to integrate telemetry between your AI system and the rest of your application, and create the learning flywheel of evals with observability, and you'll gain a better understanding of how well these features are working for your end users and how you can improve them toward those ends.

# Conclusion

At its core, observability for AI applications looks a lot like observability for any other application. The nondeterministic nature of an LLM presents unique challenges, but teams already practicing observability will find their existing tools and frameworks transfer readily.

The key insight from this chapter is the learning flywheel: production telemetry feeds evaluations, evaluations inform harness improvements, and those improvements show up in telemetry. This loop is how you move from a 25% error rate to less than 1%, how you catch a model misinterpreting a column name and fix it with a description, how you discover that models are guessing at fields and fix their behavior.

In the age of AI, it's tempting to reach for AI-native solutions to every problem. But the fundamentals haven't changed: instrument well, analyze with intent, monitor what matters to your users and your business, and create feedback loops that compound. The barrier to building AI applications has dropped dramatically. But the bar of success for operating them well is still having empathy for your customers, your business, and your team.

Now, we turn to a synthesis of the lessons from this chapter and the entire book provided by guest contributor, Kesha Mykhailov, who is Staff Product Engineer at Fin (formerly Intercom). The journey for Fin toward building and operating its AI agent, Fin, deeply illustrates the opportunities that observability can unlock for modern software engineering teams.

# Fin's Case Study in Modern Engineering

*This chapter is contributed by Kesha Mykhailov,  
Staff Product Engineer at Fin*

### A Note from Charity, Liz, George, and Austin

We end our look into observability use cases by seeing how many of the concepts we've discussed come together for engineering teams in the real world. The previous chapters of this book have covered individual concepts or contexts to illustrate how observability is practiced effectively in modern systems. But we think it's best to show how those concepts and practices are integrated to materially change real outcomes for real teams at leading software companies.

In this chapter, Kesha Mykhailov shares the remarkable journey of how the Fin team transformed their approach to observing and operating **Fin.ai**, an AI agent they designed to handle customer service interactions. While the initial customer experience was characterized by slowness and frustration, the efforts detailed in this chapter elevated Fin to become a leading AI agent in the space. For Fin, the core of this turnaround was finding ways to measure customer experience and connecting every change, experiment, and optimization back to that one measurement.

This chapter is adapted from the original delivery of this story as a conference presentation. We've tried to maintain as much of Kesha's original voice as practical in this adaptation.

When our company, formerly known as Intercom, embarked on building Fin, our main goal was nothing short of creating a truly revolutionary customer service agent.

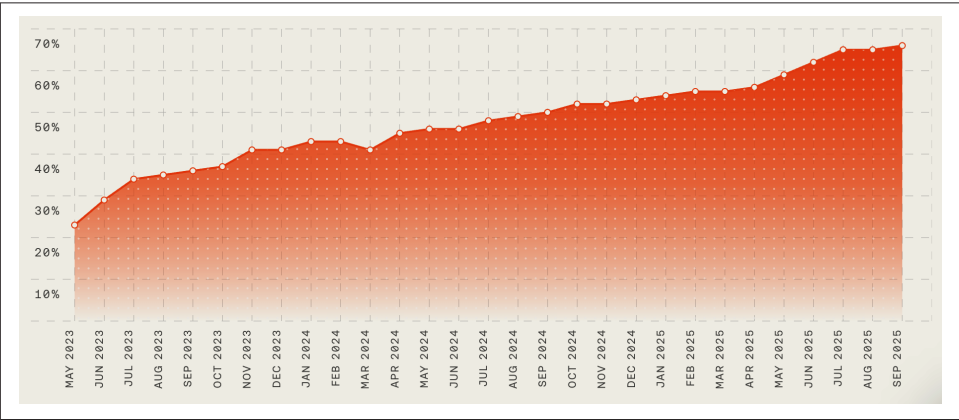
The main job of such an agent is to answer end-user questions on our customers’ behalf. In the age of AI, Fin’s customers are true pioneers: they are transforming their support operations to offload repetitive and mundane conversations to Fin while freeing up their support teams to work on more involved and complicated requests.

Once Fin answers a question while chatting with the end user, we consider that conversation resolved. Hence, the percentage of conversations that Fin is able to resolve when engaged is the most important efficiency metric—we call it the *resolution rate*.

## Increasing Resolution Rate

Naturally, our customers are incentivized to increase Fin’s resolution rate by providing high-quality content that Fin can use to provide accurate answers. On the other hand, we push ourselves to increase Fin’s efficiency because we only charge our customers for the conversations that are, in fact, resolved.

These “aligned incentives” are a secret sauce that makes Fin a top-tier customer service agent. Since Fin launched in March 2023, its resolution rate has steadily increased by about one percentage point per month, as shown in [Figure 22-1](#). And this is not some marketing data tweaked to make Fin appealing; this is an honest signal averaged across all of the customers using Fin.



*Figure 22-1. Fin’s average resolution rate shows a steady month-over-month increase in its resolution rate.*

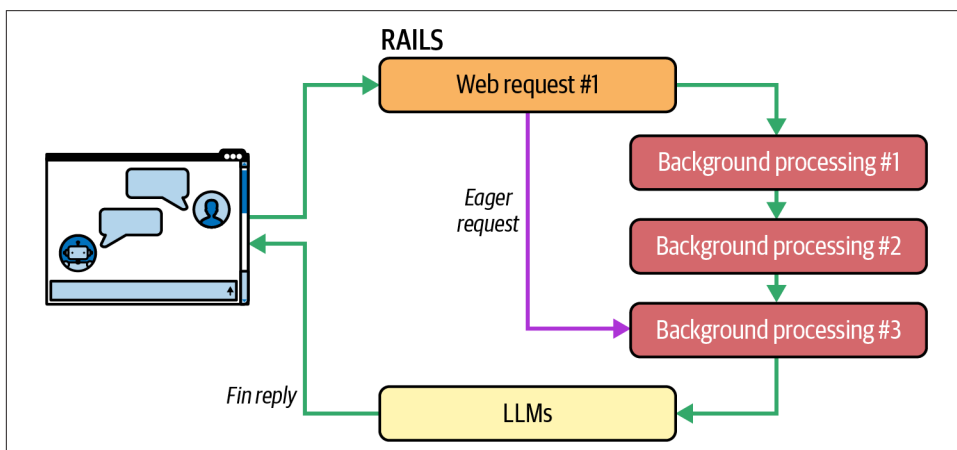
Resolution rate is not an easy metric to improve. We had to evolve Fin from a single LLM call into a complicated agentic system that follows customer guidance, invokes actions, and completes tasks.

At any given time, we have dozens of A/B tests running in production that are adding or refining those powerful capabilities. Every individual test delivers only a fraction of a percentage point of improvement to the resolution rate by itself. It is only by stacking them up together over time that this steady growth has been achieved. Such evolution comes at a cost: new functionality increases the time Fin takes to process a query. While we kept an eye on Fin's latency, we deliberately prioritized capabilities and efficiency over speed.

## Speeding Up Without Losing Efficiency

In early 2025, we started getting complaints from both existing customers and potential buyers that Fin was slow. Despite our strategy, we took customer complaints seriously and started an engineering effort to figure out how to speed Fin up without affecting its efficiency.

To understand what happened, we have to take a look at what Fin is from the engineering perspective: Fin is built on top of the existing customer platform on Intercom, and is deeply integrated with existing support workflows. As a result, Fin is a complex distributed system with multiple network transactions involved in processing a given interaction. In the example shown in [Figure 22-2](#), a simple user question like “how do I activate my account?” creates several background processing requests that utilize LLMs at various speeds in order to return a correct resolution.



*Figure 22-2. Fin is a complex distributed system with multiple transactions necessary to process any given interaction.*

On top of this, Fin is constantly evolving. Hundreds of engineers from 20 different engineering teams ship changes to the code directly involved in processing Fin's interactions. At Fin, shipping is our heartbeat—and even on a typical Friday, our heartbeat is measured at ~100 individual changes shipped per day. This rate of change is where the *real* complexity comes from.

As customer complaints surfaced, engineers got to work. Optimization pull requests were flying left and right, with many charts indicating significant improvements in Fin's individual components. But our customers and prospects wanted more.

It became clear that while we were focusing on the performance of the individual components, we lacked a signal representing the time that end users were waiting for Fin to reply. To build deep empathy with our customers and to experience that slowness in exactly the same way they did, we decided to surface an end-to-end signal: the time from when the user message is submitted to the time the first tokens of Fin's reply are rendered in Fin Messenger.

## And Thus, “Time to First Token” Was Born

Measured on the frontend, this “time to first token” accurately represented the customer's experience. Time to first token is a measure used to reflect customer experience: it measures the time between when a customer submits their question and when the first character in the reply gets rendered. In other words, this is the time elapsed between when the customer asks a question and perceives that an answer is being returned.

From then on, every change we made to the system had to positively affect “time to first token,” as shown in [Figure 22-3](#).

Our engineers could answer the question of *what* (e.g., “what is the median time to the first token?”), but they couldn't understand the *why* behind it, as they couldn't map that high-level signal to the code they were working on. Our new tracing data lacked required details for the engineers to dig for the root cause.

While we were trying to empathize with our customers, we lost touch with our own engineers. But failing is part of the process, and we did learn something from this experience: we understood exactly why our initial attempts to optimize Fin's speed were not successful.

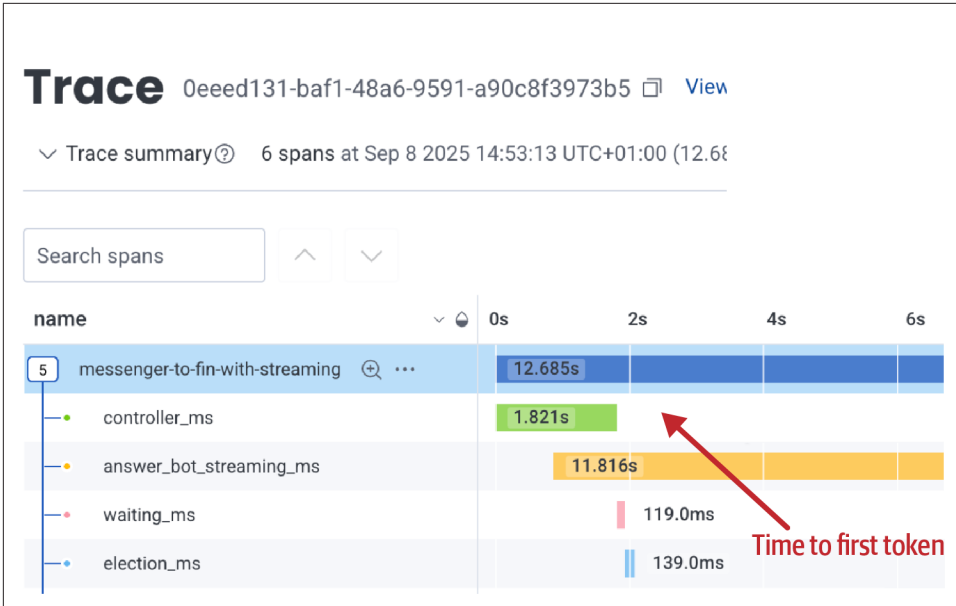


Figure 22-3. This distributed trace waterfall view shows time to first token.

If you think about a distributed trace of a complex system like Fin, you might naively think that transactions over time would look something like Figure 22-4.

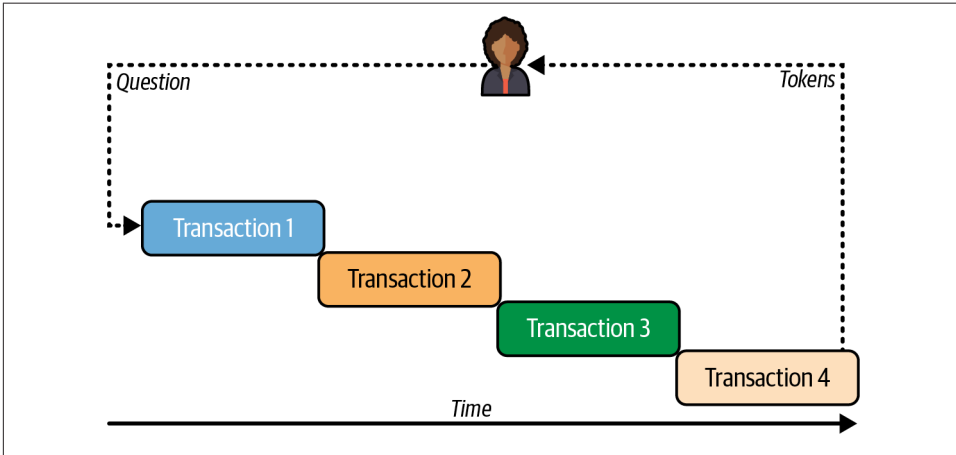
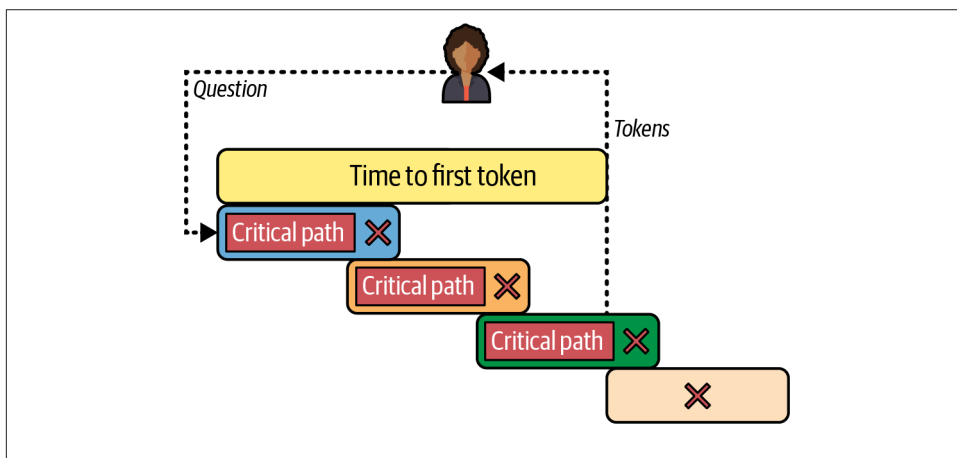


Figure 22-4. An oversimplified view of what is required to return the first token, wherein all transactions must finish serially, to completion, before the first token is returned. This does not reflect the reality of our system.

Here, every transaction’s “start” is aligned with the “end” of the previous transaction, with the answer tokens streamed back to the user at the end. In reality, it’s a lot more complicated, and the process looks more like [Figure 22-5](#).

The work being done by the agent to start responding to the user doesn’t necessarily align with the actual underlying transactions happening in each service.

As we improved the speed of individual components, we didn’t know which were on the critical code path. Were they directly blocking the progress of Fin’s interaction, or were they part of the processing done after Fin’s interaction moved to the next transaction?



*Figure 22-5. In reality, parallel transactions don’t have to fully complete in order for the first token to return. Some transactions may start in the middle of the previous one, based on control flows within the critical path, and some transactions may not be in the critical path at all.*

To understand a system like this, a distributed trace has to include low-level telemetry to identify where exactly control flow is passed over to a different transaction *and* be able to identify things that lie on the critical path. We had to start from scratch and build new distributed traces with low-level spans attached to the transaction spans, as shown in [Figure 22-6](#). With the new traces, not only were we able to empathize with the customers by answering the question of *what*; we also gave our engineers a way to answer *why*.

Armed with this new telemetry, we were able to quickly move forward and meaningfully improve Fin’s latency.



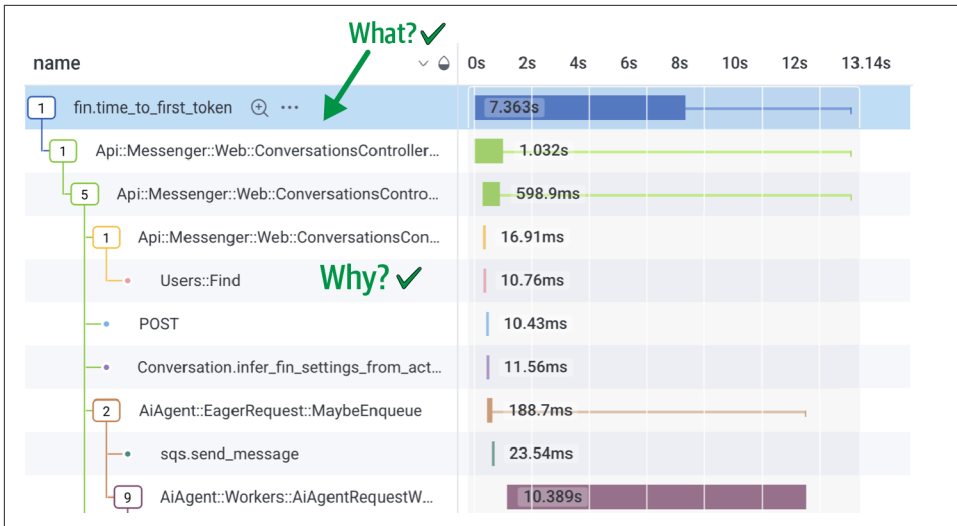


Figure 22-6. By reinstrumenting distributed trace spans to contain information about when control flow is passed to the next transaction, engineers were able to see both what was happening and understand why time to first token was impacted.

## Speeding Fin Up

Let’s take a look at one of the most impactful optimizations we’ve implemented: eager requests.

Unsurprisingly, the most time-consuming part of Fin’s interactions is the LLM-based agentic workflow making calls to multiple models. But even before interactions reach that stage, there’s a bunch of work that has to be done, like storing end-user messages in the database, progressing through customer-configured workflows, figuring out if Fin has to be engaged in this given conversation, and so on.

What if we skipped that part and eagerly invoked LLM-based processing? There’s a good chance “normal” request flows will alter the payload affecting the prompts. But if it doesn’t, by the time we get to the AI-based part of the interaction, we might already have an LLM-generated answer at hand, as shown in [Figure 22-7](#).

As it turns out, in the majority of cases, the payload stays intact and we *are* able to stream the response back to the end user quicker. This optimization alone helped us to shave two seconds off the median time to first token.

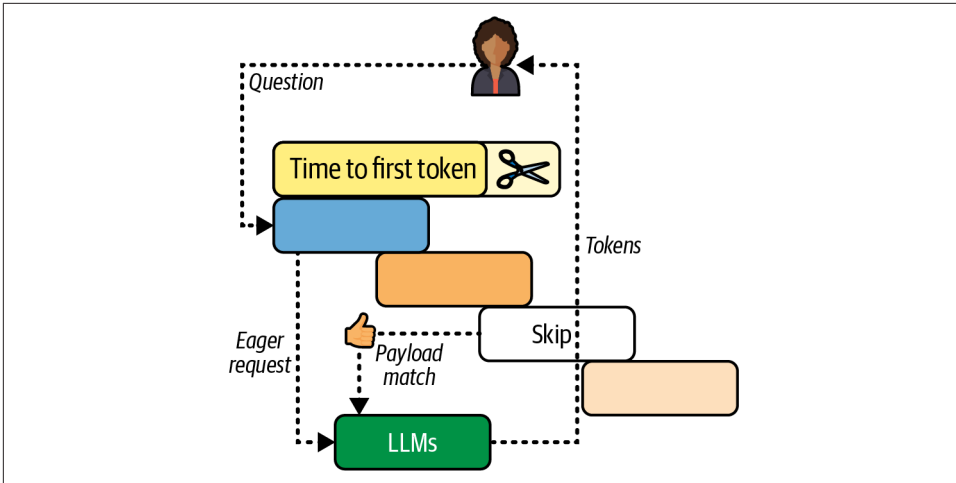


Figure 22-7. We created the concept of eager requests to prioritize time-consuming calls to LLMs. Because they take the longest time to complete, prioritizing calls to LLMs in the control flow allowed us to dramatically lower the time to first token.

But making Fin faster is only part of the job. The real challenge is to protect the speed gains over time, especially given the rate of changes shipped on a daily basis. We knew that things would degrade again and we needed a way to proactively manage slowdowns.

Given the unpredictable availability of the models (provider outages, daily seasonality, lack of capacity, etc.), a simple threshold-based alarm would be too noisy. We decided to adopt an event-based SLO that tracks the percentage of interactions faster than a newly established baseline, as shown in [Figure 22-8](#).

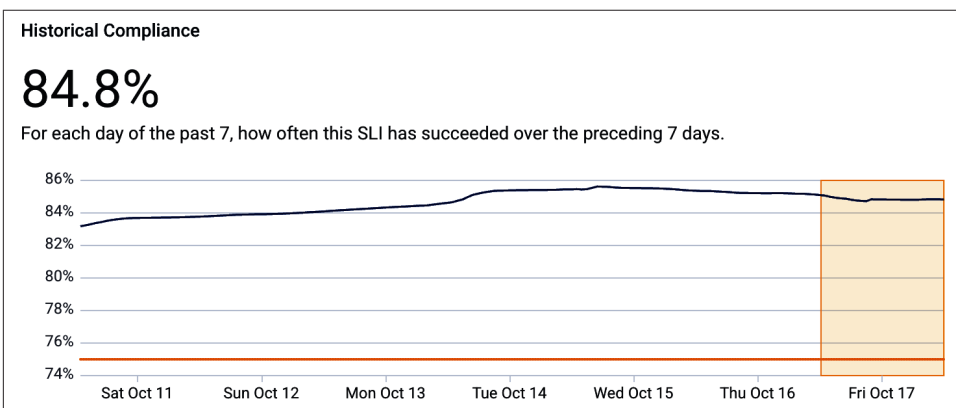


Figure 22-8. Daily compliance of the time to first token SLO over seven days.

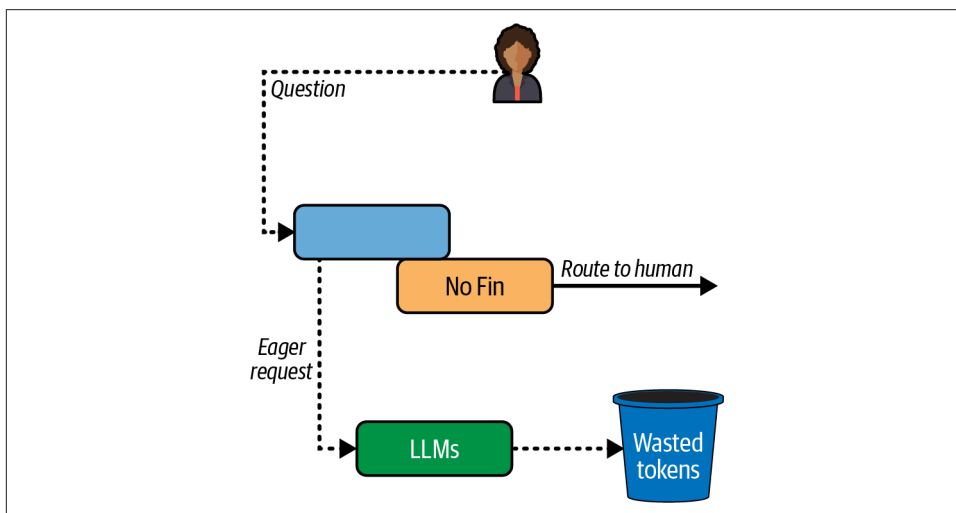
Focusing on our long-term trends of SLO compliance helped us stay focused on meaningful degradations and not be distracted by intermittent short-lived spikes.

## Finance Enters the Chat

At this point, the story could have come to a happy end. We made our customers happy by improving Fin’s latency, and our engineers had a signal they could trust to surface real impact. That customer-centric signal was connected to the low-level data they could use to immediately investigate any degradation detected by the breach of the SLO.

However, there was still one problem to be solved with our eager-request optimization. When a “normal” request confirms a payload match, we stream the reply prepared in advance by the eager-request flow. But what happens if Fin was never engaged in the processing of a given end-user message in the first place?

We allow our customers to configure sophisticated multistep workflows where Fin is one of many steps. But sometimes, customers want to route a given conversation straight to a human based on certain conditions (e.g., sensitive topic around payments). In this case, the normal flow halts and all the LLM requests emitted as part of eager-request flow are wasted, as shown in [Figure 22-9](#).

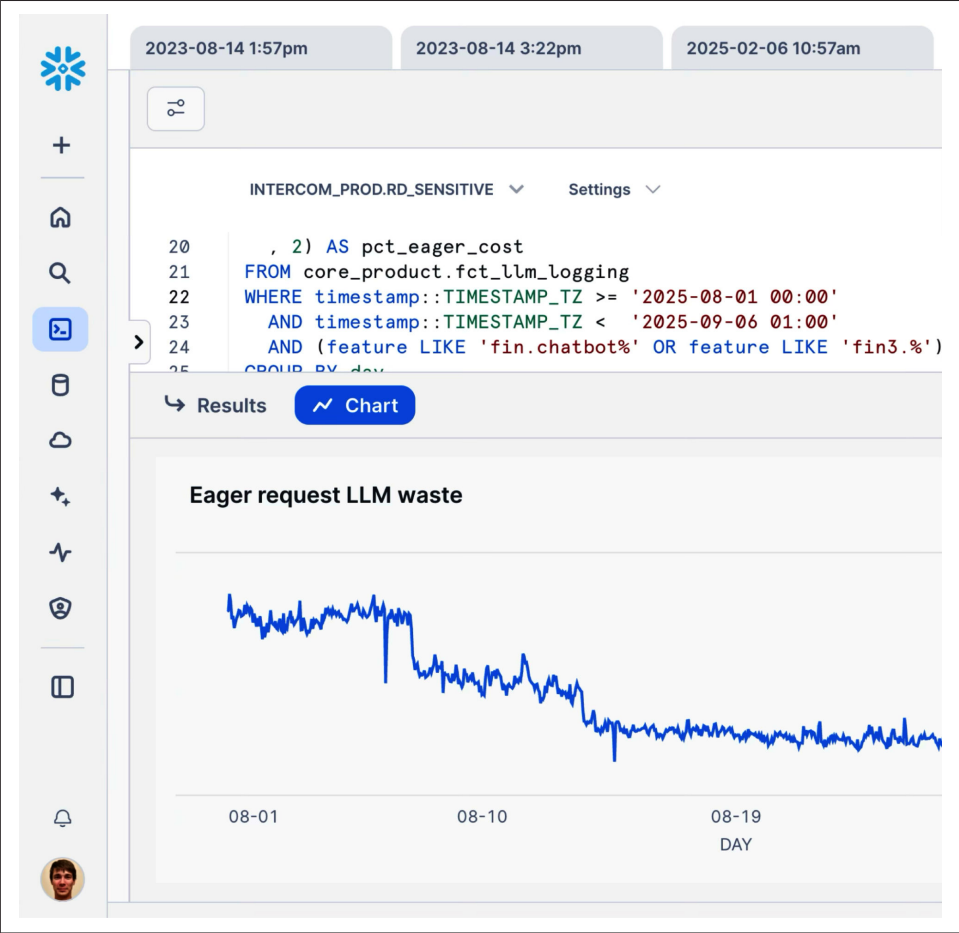


*Figure 22-9. Users can configure workflows that bypass the use of Fin. In those cases, eager requests may still be emitted as part of the control flow, but those tokens are unused. This was a malfunction of the eager-request implementation.*

As there was no Fin interaction in processing that end-user message, no latency metrics were emitted. As a result, this eager-request malfunction was flying under the

radar. Engineering learned of this only after the finance team did a quarterly review detecting a decrease in Fin’s gross margins. We were so focused on Fin’s latency that we lost touch with our own business by incurring this extra operational cost.

Luckily, thanks to the great tracing we already had, we were able to identify and fix the issue, quickly surfacing all cases where there was no need to trigger eager requests. To validate our fixes, we used the same query that the finance team was using to quantify the extra LLM cost, as shown in [Figure 22-10](#).



*Figure 22-10. We were able to query our trace data to identify situations where eager requests were inadvertently triggered. Thanks to our fine-grained instrumentation, we were able to quantify cost and use that to identify when that was happening.*

We were happy to observe the LLM waste percentage going down, confirming our fix. But we also wanted to stay on top of this signal.

The problem with the aforementioned query is that it takes minutes to run. More importantly, the data in the warehouse is updated on a daily basis and takes hours to refresh. And while this is okay from the finance team’s perspective (who operate on a quarterly basis), it isn’t good enough for the engineers.

Engineers need real-time feedback on their actions. That’s an important precondition to being able to safely ship hundreds of changes per day. So, we added Fin’s cost per interaction to the same distributed traces we already had, as shown in [Figure 22-11](#).

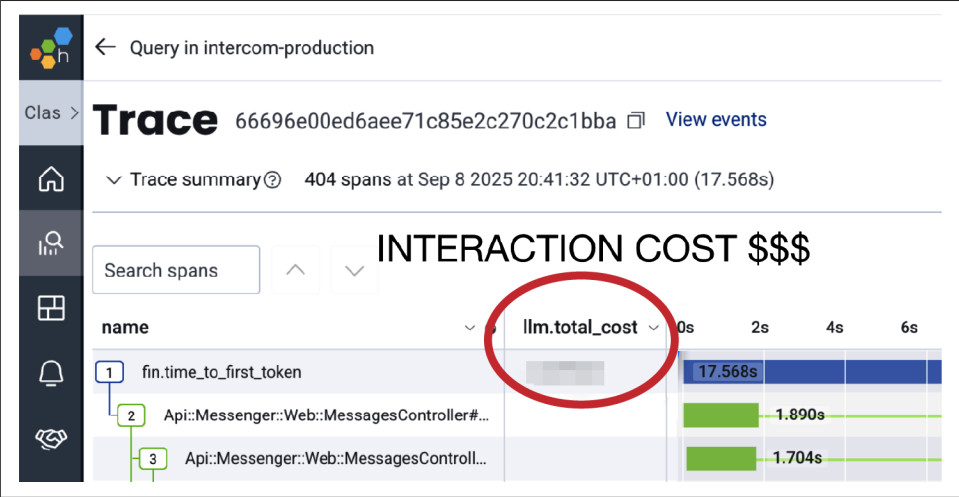


Figure 22-11. Because the previous query was too slow for real-time usage, we instrumented our traces to include an attribute that quantified total LLM costs for every single request.

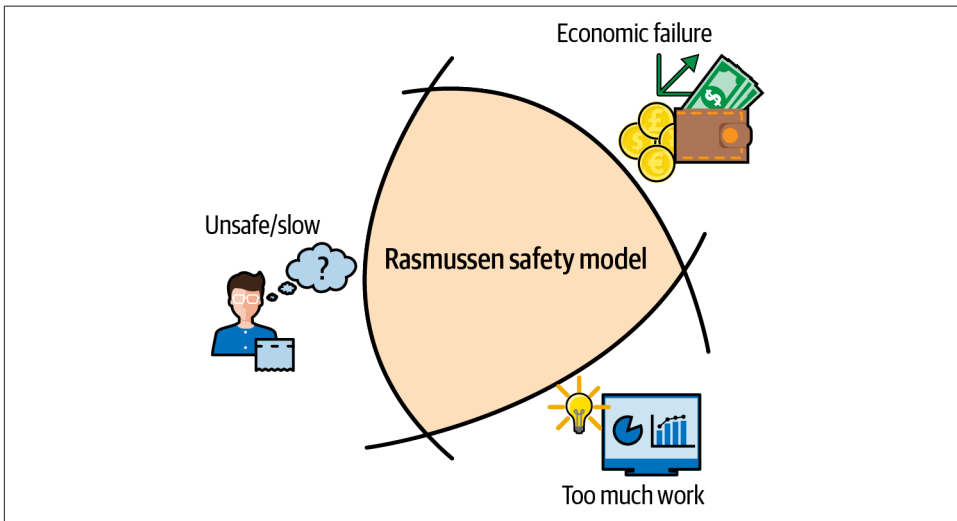
Of course, we put an SLO on it to make sure we were notified of degradations to Fin’s cost.

## Empathy

Our customers were happy, as we were able to empathize with them and experience Fin’s speed in exactly the same way they did. We also built empathy with the business by surfacing a business-centric signal. Fin’s cost per interaction strongly correlates with cost per resolution, which we ultimately charge our customers for. Crucially, both customer- and business-centric signals are attached to highly detailed telemetry that enables engineers to investigate and fix issues, making them happy too.

One might think our experience and learnings are specific to us, but in fact, it’s easy to generalize this approach.

The Rasmussen safety model, shown in [Figure 22-12](#), explains how sociotechnical systems can drift toward failure (for more about sociotechnical systems, see [Chapter 24](#)).



*Figure 22-12. The Rasmussen safety model triangle (or Rasmussen’s “drift to danger” model) is a conceptual diagram that illustrates how systems move toward failure. It represents the, usually invisible, boundary of safe operation and how constant organizational, financial, and efficiency pressures cause operators to “drift” toward, and eventually cross, this boundary.*

The model argues that every system operates within three boundaries:

*Boundary of acceptable performance*

Cross that boundary, and the performance/availability of your system is affected, making your customers sad.

*Boundary of economic failure*

Cross that boundary, and your business profitability is at risk.

*Boundary of unacceptable workload*

Cross that boundary, and you risk putting too much pressure on your engineers.

For a regular operator, those boundaries are elusive and rarely understood. But they do exist! Your customers will complain when things get too slow. Your finance team knows exactly where the economical failure boundary is; they already promised that gross margin to your investors. And finally, unless you equip your engineering team well, they might crumble under pressure by not being able to increase performance and reduce cost.

That's where observability takes a central role. The art of observability is to surface those safety boundaries to the operators and create a real-time feedback loop that empowers them to take risks and know exactly where they are in the Rasmussen triangle. Practically speaking, the art of observability is to create a dataset that combines a customer-centric signal with a business-centric one. Those signals must be updated in real time and connected to highly detailed, low-level tracing telemetry that enables operators and engineers to proactively monitor and fix issues as they arise.

This is the last mile of observability, and arguably, the hardest one. Many teams focus too much on instrumentation, telemetry pipelines, and vendor management, and overlook this. It's easy to get bogged down with implementation details and miss the forest for the trees. That's where observability becomes just another cost center, where the only conversation you can have with your finance team is about the cost of your logging cluster.

Instead, if you walk the last mile, observability could be strongly associated with your customers' happiness and business success. The good news is that, to do this, the only thing required is empathy. You have to deeply empathize with your customers, business, and engineers. Understand their needs. Feel their pain the same way they do.

As ironic as it sounds, in the age of AI, empathy is all you really need.

## Conclusion

Kesha's story of engineering at Fin embodies what the rest of the book has been building toward: observability as a practical discipline that changes outcomes for real teams in real systems. Rather than treating instrumentation, tracing, SLOs, and iterative analysis as separate topics, Kesha's story shows how they become a single operating model that ties every change, experiment, and optimization back to a customer-centric measurement. It also makes that measurement debuggable enough for engineers to act on it quickly.

This story also demonstrates the "last mile" of observability: building empathy with customers, engineers, and the business by surfacing the right signals, in real time, with enough low-level context to explain why those signals moved. Organizations learn how to ship fast without drifting into danger because they can see the boundaries of acceptable performance, economic viability, and operator workload. With that, they can course correct before pressure pushes them across the line. This story is a concrete example of how modern engineering teams turn observability into a learning system.

In the next part of the book, we're going to look at how to create top-down organizational alignment to make these types of outcomes possible.





---

# Observability Governance

So far, this book has been written for engineers: the people instrumenting code, running production services, and trying to understand what their software actually does in the real world. Parts **I–V** are written from the practitioner’s perspective, bottom up.

Part VI switches gears. We now write for technical leaders.

When we asked the software engineering community what was missing from observability guidance, we heard the same stories over and over: millions of dollars spent on tools nobody learned to use, CTOs who’d written observability off as irrelevant in a post-AI world, and organizations where nobody could agree on basic questions—what observability even is, what problems it’s supposed to solve, or who’s responsible for it. That’s a leadership problem, not a tooling problem.

Technical leadership comes in many forms—staff, principal, and distinguished engineers, as well as directors, vice presidents (VPs), senior VPs, CTOs, and similar roles. We don’t think this is solely the domain of executives. Technical contributors shape these decisions too, and we hope that comes through in these chapters.

The biggest problem we see right now is a widening gap: organizations are shipping code faster than ever, and simultaneously losing their grip on what that code actually does. Part VI is about closing that gap—through better decisions, better feedback loops, and leadership that takes both the technical and human sides of that problem seriously.

**Chapter 23** opens as a letter to CTOs. In the AI era, shipping code is no longer your biggest constraint; now you are constrained by understanding, validating, and your

organizational learning speed. This chapter shows how much of that drag traces back to unpaid sociotechnical debt, and how observability connects to paying it down.

**Chapter 24** goes a layer deeper, using systems thinking to examine how change actually moves through sociotechnical systems. Most organizations try to hire or buy their way out of debt. It doesn't work. This chapter shows what does work, and why observability is a leverage point, not just a tool.

**Chapter 25** helps you figure out what kind of feedback loops your organization actually needs, and then connects those needs to the tooling landscape. It also cuts through the AI noise to keep you focused on outcomes.

**Chapter 26** walks you through building a business case for observability investment. Is yours a cost center or a strategic investment? The answer changes what you buy, how you manage it, and how you measure success.

**Chapter 27** is a diagnostic. If your organization is feeling cost pressure on observability tooling, that's usually a symptom—you're paying observability prices for monitoring outcomes. This chapter helps you figure out what went wrong and how to course-correct.

**Chapter 28** is a guest-contributed chapter from Rick Clark, Global Head of Cloud Advisory at UST. Rick gets practical about how organizational shifts actually happen, and how to navigate the hurdles you'll face trying to accelerate how fast your org learns.

**Chapter 29** examines the decision of build versus buy versus open source in detail—the real trade-offs, not the vendor-pitch version.

**Chapter 30** covers vendor partnerships. If you're buying a solution, then building a partnership matters almost as much as the technology. This chapter covers how to design and run a proof of concept (POC) as well as how to build trust and credibility with internal stakeholders, reciprocity, and influence with vendors.

**Chapter 31** gives implementation guidance for the teams that actually drive observability adoption across an organization: instrumentation patterns, org-wide rollout considerations, and common environments like large enterprises and regulated, highly secure industries.

**Chapter 32** looks to the future. Now that the economics of code production have been inverted, what will be the downstream consequences over the next few years, and how can technical organizations prepare to meet them? We argue that it's time for the rigor to move from the code to the system.

# Organizational Learning Speed Is Now Your Biggest Constraint: An Open Letter to CTOs

The most disruptive consequence of the AI era isn't faster code; it's that your organization's learning speed has become its single greatest bottleneck. In this chapter, we'll address two uncomfortable truths: (1) in an AI era, the new constraint is how fast your org can learn, and (2) the biggest drag on learning speed in most orgs is unpaid sociotechnical debt.

Like technical debt, sociotechnical debt comes from choosing quick and easy solutions in the moment that create more work later. Most technology shifts have required a corresponding change in how people work to actually succeed. But changing behavior is often much harder than changing tools. Organizations that haven't made those behavioral changes have accumulated inefficient processes, misaligned leadership, and unresolved cultural problems.

These debts show up as reduced speed, engagement, and flexibility. Left alone, they can limit a company's ability to grow, learn, compete, and ultimately survive in fast-moving markets. For most organizations, this debt hasn't been a dealbreaker—yet. Most know it's there, but they've worked around it. The common attitude is: “we'll fix it later.”

Boards (and competitive threats) have been pushing execs to develop an AI strategy, so they can capitalize on faster delivery. And all that accumulated debt is now about to collect a lot of interest, because:

- In the AI era, software delivery is no longer constrained by how fast you can write code. It's constrained by how fast you can validate, understand, and learn.
- For software delivery, observability is what enables validation and understanding, which establishes the baseline for organizational learning speed.

- Most companies still rely on tools and techniques developed by IT/ops, even for their own core product offerings. Those tools work well for infrastructure and operators, but lack the precision, context, and connective tissue it takes for software engineers to understand their changes in production. As a result, individual engineers are called upon to fill in the blanks with heroics and intuition—at the precise moment when we are waking up to the reality that *if our wisdom is not encoded into the system, as far as AI is concerned, it doesn't exist.*

Many organizations have tried to make these technical shifts, but haven't fully addressed the human and organizational challenges that come with them. This chapter identifies which sociotechnical debts are real blockers to moving at AI speed, and offers concrete actions to address them. We'll introduce concepts and solutions at a high level here, then go deeper in the chapters that follow.

Accelerating organizational learning requires fundamental strategic shifts in leadership, budget, and culture. As bottom-up transformation efforts will stall without executive air cover (and because people typically want to read what their boss has read), this chapter is framed as an open letter to all CTOs.

## An Open Letter to CTOs

Dear CTO,

By now, you've probably survived multiple "everything changes now" moments: cloud, DevOps, microservices, Kubernetes, serverless, edge computing, and more. Each wave delivered real capability. And over time, each one revealed the same uncomfortable truth: most organizations change their infrastructure much faster than they change how they work.

Beneath the surface, many fundamental behavioral patterns have stayed the same. Dev teams still act like production is what happens after development is finished, not like it's the penultimate stage of the development process (and the only validation that ultimately matters). Production is still organized around protecting the health of the service against all threats, but anything that doesn't threaten the health of the system is poorly understood (or it is not known). Incidents still trigger blame, heroics, and alert fatigue. And "transformation" still too often means adopting a new platform, buying new tools, and renaming some teams—without reworking the incentives and feedback loops that actually drive behavior change.

Now we're in the latest "everything changes" moment with AI. And that shift has revealed something exciting and new: AI doesn't eliminate the need for engineering discipline. It eliminates the excuses for not having it.

"Moving fast" used to mean shipping a few times a week without breaking production. Now it means moving at the speed of AI—from idea to code to deploy to

learning, and on to the next idea—in a fraction of the time. *That pace is not a shortcut around rigor; it's a stress test of it.* Like the shifts before it, AI reveals another uncomfortable truth—it doesn't just make creating software faster; it makes everything faster, including your mistakes.

That's why this shift feels so urgent: you're under pressure to roll out AI quickly, and you can see all the failure modes—drift, regressions, compounding risk—waiting for any organization that tries to scale speed without discipline.

The real advantage in the AI era is being able to accelerate without crashing. Your engineering teams need to move *swiftly and confidently*. Their habits, feedback loops, and operational clarity need to be strong enough that AI amplifies excellence rather than chaos. Intuition does not scale. All the duct tape and baling wire that hold your systems together at the rate of change you evolved to tolerate starts flying off at 10x speed, let alone 100x.

The behavioral patterns you've built up through cloud, DevOps, microservices, and beyond are about to matter a great deal. If what you've accumulated is sociotechnical debt, that's about to become your biggest blocker. The best time to have resolved it was years ago. The next best time is now.

That's what we're here to talk about. Which debts actually matter? Where do they fit into your strategic priorities? How do you fix them?

First let's understand which, if any, of the common sociotechnical debts apply to you.

## The Sociotechnical Debts That Will Hold You Back

Sociotechnical debt is the hidden, accumulated cost of misalignments between an organization's social structures (culture, teams, processes) and its technical systems (software, infrastructure).¹ Like technical debt, not all sociotechnical debt needs to be (or ever *can* be) fully paid off. But debts that put your most important goals at risk must be addressed.

Our guiding principle is this: any sociotechnical debt that prevents your teams from moving swiftly and confidently today will also be a blocker to their ability to accelerate and build, ship, and validate at the speed of AI.

These debts are not the only obstacles in your path. AI skills and literacy are vital for engineers, managers, and execs alike. Organizational scaffolding must be built, guardrails established, tools tested and bought and rolled out. But as we said in [Chapter 1](#), this is not a book about AI. This is a book about observability, written for

---

¹ Damian A. Tamburri and Elisabetta Di Nitto, “When Software Architecture Leads to Social Debt”, in *Proceedings of the 12th Working IEEE/IFIP Conference on Software Architecture (WICSA 2015)*, Montreal, QC, Canada, May 2015, 61–64.

software engineering teams, derived from first principles. And the role of observability in an AI-first world is more pronounced and critical than ever before.

Whether or not the venerable SDLC will remain relevant in a post-AI world is not yet knowable. As many have noted, agentic development compresses all stages of the software lifecycle into one fluid, interactive blur. All stages, that is, except one: the observability aspect of the maintenance stage.

### As Our Friend Boris Tane Says

When agents ship code faster than humans can review it, observability is no longer a nice-to-have dashboarding layer. It's the primary safety mechanism for the entire collapsed lifecycle. Every other safeguard, the design review, the code review, the QA phase, the release sign-off, has been absorbed or eliminated. Monitoring is what's left. It's the last line of defense.

The SDLC was a wide loop. Requirements → Design → Code → Test → Review → Deploy → Monitor. Linear. Sequential. Full of handoffs and waiting.

The new lifecycle is a tight loop.

Intent. Build. Observe. Repeat.

No tickets. No sprints. No story points. No PRs sitting in a queue. No separate QA phase. No release trains.

Just a human with intent and an agent that executes.

So what is left?

Context. That's it.

The quality of what you build with agents is directly proportional to the quality of context you give them. Not the process. Not the ceremony. The context.

The SDLC is dead. The new skill is context engineering. The new safety net is observability.²

In the old SDLC, monitoring (or observability) isn't explicitly named, although it is conventionally assumed to be part of the maintenance stage. And therein lies the problem. When structured, handoffs give way to fluid interactivity, and all the old quality gates go too. Observability is the only gate that survives, and it not only survives; it becomes the mechanism that validates the entire lifecycle.

---

² Boris Tane, "[The Software Development Lifecycle Is Dead](#)", February 20, 2026.

But observability will first have to shed its skin—the accumulated layers of static dashboards, siloed pillars, and old policies and habits that combine to keep developers walled off from their code in production, unable to observe or validate with specificity. These are the sociotechnical debts most likely to keep you from achieving your goals with AI. So this is where we start. What are the debts that currently hamper your ability to ship “swiftly and confidently”?

## When Developers Live In a World of Tests, Not Reality

In retrospect, the DevOps movement was trying to do something deceptively simple: build a feedback loop between developers and their code in production. The point wasn't more automation; it was to bring developers closer to real outcomes—users, reliability, cost, performance—so they could learn from each diff, and their software would continuously improve.

We failed to achieve that goal. **Twenty years later**, most orgs are still stuck with dev versus ops and a split-brain problem. Most developers live in a loop of “build, test, merge,” and then move on to the next task while someone else handles deploying it. Developer intent is forgotten and lost before it can be validated, and developers go on viewing production as a place they go to fix bugs sometimes—not the source of truth of user experience.

In those organizations, the only team with production in their feedback loop is operations (also known as DevOps, infrastructure, platform engineering, SRE, etc.). Feedback arrives as alerts, escalations, customer complaints, and support tickets. That loop is necessary, but it's laggy, lossy, and threshold-driven. It tends to surface issues only after they've grown large enough to threaten the whole system, rather than when they first begin to degrade user experience or product quality.

Elite engineering teams close that gap. They wire production signals back into daily development so teams can learn from each release while the context is still fresh. That's part of what makes “move swiftly with confidence” possible: frequent deployments, short lead times, lower change failure rates, and fast recovery when things go wrong. Elite performance doesn't happen by luck. It happens because teams can see the effect of small changes and respond early.

That gap—between teams that have closed the dev/production loop and teams that haven't—has always mattered. AI makes it the ballgame. When code generation accelerates, the constraint shifts completely: generating code is no longer the hard part. Validating it and building confidence in it is. How soon can your teams detect the impact of a change, understand it, and adjust? How small and precise are the anomalies you can detect?

If your production feedback loop still depends on waiting for an alert or a ticket, AI will help you ship lots of changes very fast, while the truth about what happened arrives much later. That's not moving swiftly and confidently. That's accumulating risk at the speed of AI.

A developer ships code on a Friday. On Monday morning, do they know how it performed in production—latency, error rates, user behavior—because that's a normal part of their workflow? Or, do they find out something went wrong two weeks later when a support ticket lands in the wrong inbox? If it's the latter, production isn't in your developers' feedback loop. You've structurally chosen slower organizational learning. That has to change.

*Has your organization built fast feedback loops between development and production? Is instrumentation a habit? Can every developer pinpoint the effects of the last diff they shipped?*

## When Telemetry Gets Treated Like Infrastructure, Not Product

Here's the debt that quietly piled up: as systems got more complex, most organizations kept treating telemetry the way they always had—as something ops maintains, something you bolt on, something you optimize for cost. They didn't change how developers related to their systems. They just built bigger monitoring stacks.

It was a reasonable response at the time. Back in 2009, a \$50k monitoring bill seemed reasonable. Most teams were running LAMP stacks. A senior engineer could more or less reason about the whole system in their head. Monitoring—predefined indicators, thresholds, alerts—worked well enough for that world.

Then came microservices, dynamic infrastructure, ephemeral containers, serverless, edge, multi-cloud. Complexity didn't just grow; it distributed. No single person could reason about “the system” anymore, because the system was constantly changing. The right response would have been to change how teams understood their systems—to treat telemetry as a product capability, owned by developers, instrumented intentionally, designed for exploration. Most organizations instead chose to go broad: more tools, more dashboards, more alerts, more “visibility.”

When observability emerged in the late 2010s, that choice became explicit. You could go deep—changing how your people understood your systems—or you could go broad, piling on more signals without changing any of the underlying workflows or architecture. Most orgs went broad. Most vendors, facing pressure from customers to give them “observability,” simply rebranded the products they already had. The result was a buffet of monitoring tools with observability logos on them.

The bill kept growing. A Gartner reference customer went from spending \$50,000 a year on monitoring in 2009 to \$24 million a year on “observability” in 2024—a



48% year-over-year increase for fifteen consecutive years. That figure isn't an outlier; observability now accounts for 20–30% of infrastructure spend at most companies.

And yet: production is still poorly understood. Incidents still trigger blame and heroics. Alert fatigue is still everywhere. The cognitive dissonance for CTOs is that they're paying observability prices for monitoring outcomes—large spend, sprawling stacks, and workflows still dominated by thresholds and painfully slow debugging.

That mismatch was tolerable when change was slower. At AI speed, it becomes a critical blocker. The limiting factor has shifted from “how fast can we produce code?” to “how fast can we understand and validate that code in production?” AI increases the rate of change, and a higher change rate increases the surface area for small, granular failures—subtle latency spikes, partial outages, edge-case errors, cost regressions, correctness drift. These won't trigger a monitoring alert. They'll compound quietly until they become cascading outages. Teams that can't see them early will find them late, struggle to explain them, and won't be able to trace them back to the specific change that introduced them. That turns speed into risk.

When something goes wrong in production, can anyone on the team identify what happened, regardless of their experience or familiarity with the system? Or, do most incidents start with on-call engineers searching for the senior “rockstar” who holds all the institutional knowledge and knows which dashboard to check? Do incidents take 15 minutes to resolve if a particular engineer is there, and 45 minutes if they aren't? If that sounds familiar, your sociotechnical practices haven't changed. You've chosen slower organizational learning.

*Did you make the shift to truly observable systems, or did you just keep adding more monitoring?*

## **When Developer Tools Were Never Designed as Products**

Internal engineering tooling was never designed: it accumulated. And it accumulated from the infrastructure side of the house, where it was optimized for keeping systems running, not for helping developers understand what their code does. Developers were never treated like customers. Nobody ran discovery on their workflows, nobody optimized for their cognitive load, nobody asked what would make doing the right thing the easy thing.

The result is that “add good telemetry” isn't one task from a developer's perspective—it's a maze of decisions. Should that telemetry be a metric, a log line, a trace, a profiling signal, an error, or an exception? If it's a metric, should it be a counter, a gauge, a timer, or a histogram? What cardinality is expected? After deployment: when did your code actually ship? How many other changes went out at the same time? Where does the telemetry you just wrote even show up? Navigating all of this demands deep operational knowledge and uninterrupted time. Most developers don't

have either. So the desired behaviors don't materialize—even when everyone knows what they are and agrees they're important.

Platform engineering is the organizational response to that indictment. Done right, it brings product and design thinking to a domain that has historically had none of it, treating internal developers like customers, running discovery on their actual workflows, and building paved paths that encode best practices into defaults, standard libraries, and guardrails. Reliability and observability stop being heroic acts and become the path of least resistance.

But most organizations have faced a fork here, and most have taken the wrong turn. The first failure mode is treating platform like a project: launch it, celebrate, then move the team on to other work. The second is pure rebranding: renaming an existing DevOps or infrastructure team “platform engineering” without changing the operating model, discovery process, or the orientation toward developer needs. Same team, same incentives, same infrastructure mindset. Developers are still not being treated like customers. The debt persists under a new name.

At AI speed, that gap becomes existential. AI increases throughput, which means inconsistent practices and missing feedback loops get amplified exponentially. Your wisdom must be encoded into the system—in defaults, in templates, in the path of least resistance—because at 10x change volume, you cannot rely on developers remembering to consult the right docs. If the default path is accumulated infrastructure cruft rather than intentional product design, both your developers and your AI will go off the rails. And they'll do it faster than ever.

Pick a developer who joined in the last six months. Ask them to instrument a new service correctly and show you what it does in production. Did they know where to start? Did they get there without pinging three people on Slack? Or, did they end up with something half-done because the right path was too hard to find? If it's the latter, your best practices are aspirational, not operational. You've chosen slower organizational learning.

*Does your organization use product and design principles to ensure that developers using your internal tools naturally do the right things—because that's the path of least resistance?*

## These Debts Will Sabotage Your Adoption of AI

If these fundamental behavioral dynamics haven't changed in the last 15–20 years, why should they change now? This is a fair question, and the answer is that AI has exposed a pattern that was always true: delivery speed is not the same thing as learning speed.

AI makes it easier to produce software changes. But **DORA's 2025 findings** underline the catch: rather than creating organizational excellence, AI simply amplifies the behaviors your organization already has. If your engineering system has strong feedback loops and disciplined practices, AI compounds the advantage. But if it's held together by lagging signals, inconsistent standards, and operational heroics, AI compounds the chaos.

That's why the "fix it later" era is over. These behavioral debts become painfully visible when change volume accelerates. The best time to have built organizational excellence was incrementally, throughout the earlier waves of technological change. The next best time is before you scale the consequences. What you've done in the past and what you're planning next determines how urgent this should be. Fixing these issues now is a matter of risk management.

There's also a fundamental shift in where developer attention should go. As code generation gets easier, the highest-value work for developers increasingly shifts toward system-level reasoning and user-centric decision making. Developers should focus their cognition on understanding intent, impact, and verification: delivering what users actually need, knowing how systems will respond, anticipating what could break, and using production feedback to confirm or challenge what the team believes. That shift is what unlocks your competitive advantage.

But that higher-order thinking only works if the organization has already made production feedback and system understanding part of daily development—supported by the right practices and observability capabilities, rather than delayed escalations. Reducing uncertainty contains costs. Empowering engineers to deliver value and understand their systems is what drives talent retention.

Platform engineering is the practical bridge. It reduces cognitive burden and encodes best practices into the default workflows that both developers and AI tools use. That's how you keep humans and AI aligned with the right way to build software, instead of hoping everyone remembers to consult a wiki page while shipping 10x more changes than ever.

Ultimately, the "why now?" is competitive. In the short term, everyone will see a productivity boost—all teams are shipping features faster than ever with AI. But without validation, that boost will be short lived. Engineering teams that bake production feedback into development will learn faster while shipping better customer experiences. Teams that don't will ship more changes, but they'll just be compounding chaos—and that chaos will reclaim the gains through regressions, incidents, customer pain, and even more debt to unwind.

*In an AI era, will your team compound advantage or compound chaos? How prepared is your organization to win at that challenge?*

# Turning the Ship Around

If these sociotechnical debts still exist in your organization, here's the good news: you can change this. The path isn't mysterious—it mostly comes down to deciding what you're optimizing for, and then building the feedback loops to support it.

The proven strategy is to decide: how to measure value, what to build, and who builds it. Then be clear on which actions need to change and ensure you have organizational support. Let's break this down.

## Measuring Value

The observability tools you choose will be central to driving change. The specific vendor matters less than whether the tools actually enable the workflows and deliver the value your engineers need. The right choice should be evaluated on two criteria: external outcomes (customer experience and business impact) and internal outcomes (developer capability and delivery performance).

Unfortunately, many CTOs measure observability tools against neither. For that cohort, observability is about operational outcomes: is the service up or down, is something slow, is something erroring? It's treated as a cost center and managed like one. The only times developers are expected to look at production are for bug fixes and incidents. In that scenario, there's often little support for driving the sociotechnical changes that matter.

Some CTOs have shifted toward measuring the value of observability through customer experience and business impact. That's a step in the right direction, but still an incomplete view for our goal here, which is to validate and accelerate organizational learning speed.

Simply put, you must measure the value of observability against its ability to accelerate both developer learning and delivery speed—at the same time. Getting this right matters enormously. This dual value is what reduces uncertainty, changes human and system behaviors, retains talent, and creates a flywheel that accelerates organizational learning. If it's off, it slows down—or simply doesn't help—your ability to turn things around. (For more detail, see [Chapter 26](#).)

For now, understand that observability can fundamentally change how information flows through your systems. You'll use it to drive learning feedback loops.

## Build Good Feedback Loops

Your engineering organization is not a factory. It's a complex sociotechnical system. Engineering organizations are no longer constrained by their ability to produce code, they are constrained by their ability to validate that code and learn from it. That is why building good feedback loops is a strategic choice, not a tactical tooling upgrade.

Software delivery is built on feedback loops:

- *Deployments*: Did the change work in production?
- *Code review*: Is this safe, coherent, and maintainable?
- *Incidents*: What broke and why, and what assumptions were wrong?
- *Product development*: Are users getting value?

Observability is what connects the dots and closes the loops. It's the shared sensemaking layer that turns production signals into understanding. When observability is fast and accessible, the cost of curiosity is low, and engineers learn to ask more questions. Systems become more transparent, teams catch small issues early, and mental models get refined continuously. Learning compounds, and that compounding creates the “swiftly and confidently” motion you're after.

When observability is slow, expensive, or requires specialized expertise, the opposite happens. Engineers stop looking. Production feedback collapses into delayed alerts, helpdesk tickets, and customer complaints. Systems become opaque. Teams slow down because uncertainty drives coordination overhead. Knowledge concentrates on a few overburdened experts. Learning stalls.

This is why precise, excellent, ergonomic observability is one of the highest-leverage investments an organization can make. It is the infrastructure for organizational learning. This makes it cheap to validate each change and connect the dots between developer intent and customer impact, which is what you must do to move at the speed of AI without turning throughput into risk.

## Change Actions That Drive This Strategy

A mindset shift is also necessary before teams can move swiftly and confidently at the speed of AI.

Many organizations are spending more than ever on “observability,” yet still operating with the same mindset they developed around monitoring. The result is predictable: bigger stacks with bigger bills, and the same outcomes—noisy alerts, slow investigations, and unhappy customers.

This mismatch is largely structural. Monitoring and observability are designed for different jobs. As we discussed in [Chapter 1](#), logs and metrics were built for operations teams to monitor infrastructure: they enable operational outcomes. Modern observability was built for software development teams to understand their product and how users are using it; it enables software dependability and delivery outcomes.

If your organization uses monitoring-era tools and behaviors to pursue dependability and delivery outcomes, it will keep getting monitoring results. The following three

shifts are necessary to convert observability from tool spend into organizational capability.

### **Match tools to architecture and engineer workflows**

If your architecture is distributed but your observability tooling is optimized for monolithic-era workflows, there's a mismatch. The cost shows up as friction: engineers have to manually stitch together context, and feedback loops slow down.

The legacy three pillars observability model—separate tools for metrics, logs, and traces—often reinforces this mismatch. It creates cognitive overhead by forcing engineers (or AI in some cases) to decide where to start, jump between systems, reconcile identifiers and timestamps, and make error-prone correlations. That overhead breaks the fast learning cycles required to ship confidently at higher rates of change.

The observability vendor you choose matters less than functional fit: does the tool enable fast, iterative investigation across signals and services, and does it support the workflows that engineers need to connect change to impact? Make sure your tooling matches the outcomes you're building toward.

### **Run observability as a platform, not infrastructure**

Observability shouldn't be "owned" as a tool-operations function. The mission isn't to keep dashboards alive. The mission is to make it effortless for every engineer to understand what their code does in production, as close to real time as possible.

That requires a product mindset: internal developers as customers, paved paths and defaults, strong onboarding, and continuous improvement. It also means measuring success in human terms: time to understanding, ease of investigation, adoption, and confidence after a deploy—not just uptime and data volume.

This is also why rebranding an existing DevOps or infrastructure team as a "platform team" often fails: if the operating model doesn't change, the outcomes won't either.

### **Treat product observability as an investment, not a cost center**

When observability is managed as a cost center, the organization optimizes for reduced spend rather than increased capability. At AI speed, internal capability becomes the constraint. If engineers can't see what they're doing, throughput turns into risk, coordination overhead grows, and learning slows.

Observability inherits the properties of the code you observe. If the code is a cost center, observing it is also a cost center. But if the code is an investment, if it's what makes you money and is why you exist as a business, then building better products and more performant interfaces is also an investment. That's a job for observability.

Agentic workflows break the long-held mental model of “developers own preproduction, operations own postproduction.” Nondeterminism is a feature with agents. The *only* way to understand what they are doing is by instrumenting the code and observing what they do.

Managing observability as an investment shifts the focus toward the outcomes you actually need: faster feedback loops, better understanding, and sustained delivery performance. We’ll cover this further in [Chapter 26](#).

## Make it real

Together, these three shifts make the earlier strategies executable. Without them, you can talk about AI-driven productivity gains all you want—and for a while, those gains will even be real. But eventually, as your team compounds risk with each uncertain deployment, that productivity erodes and your debts collect interest.

Putting these shifts into practice should look something like this:

*Adopt the loop of institutionalizing a deploy, observing, and learning.*

The best time to learn is right after you ship, while developer intent is crisp and context is fresh. If you rely on incidents or escalations as the primary learning mechanism, you miss most of the signal (see [Chapter 25](#)).

*Treat observability like an internal platform product.*

Build like a product, enable developers, and optimize for adoption and time to understanding so that production learning becomes routine and scalable (see [Chapter 28](#)).

*Align goals and spend to business outcomes.*

Choose tools and workflows based on the feedback loops you need to strengthen, and avoid the trap of paying observability prices for monitoring outcomes.

You’ll know these changes are working when the median time shrinks between a new deployment and a validated understanding of customer impact (for example, measured as mean time to detect and interpret impact). You should also see a shift in where issues are first discovered (for example, measured as the percentage of material incidents first identified through internal telemetry versus customer reports or support escalations). And your teams should start adopting the learning loops you’ve provided (for example, measured as the percentage of deploys with a post-deploy outcome check completed within a defined window).

With these shifts, you can lay the groundwork for your teams to accelerate their learning, move swiftly with confidence, and turn those productivity gains into compounding competitive advantages.

# A Letter from a CTO: How Fin Engineering Optimizes for Learning

## A Note from Charity, Liz, George, and Austin

To ground the strategic arguments of this chapter's open letter in executive reality, we are delighted to include this contribution from a leader who has navigated these shifts firsthand. Darragh Curran, CTO of Fin (formerly Intercom), details how his organization successfully operationalized fast organizational learning to transition to an AI-first culture. His perspective shows how these principles translate into competitive advantage in a fast-moving market.

The thing that sparked Fin into existence was an unignorable itch to make business online a little less awful. To put an end to: “Dear valued customer,” “You are ticket number 123,” and “Do not reply to this email.” Instead, to make business online personal and human. Like you’d expect in a store you love: a friendly welcome, personalized service, and real care if something goes wrong.

To do this, we made Fin. It didn’t magically make your business better, but for people who cared, it gave them the right tools. And many did care. They realized good service was good for business.

These tools didn’t exist yet. The shape and surface area weren’t figured out. And figuring out the shape of a product—then evolving and refining it—benefits hugely from fast feedback loops:

Do a thing. Learn something. Do the next thing.

That is the core function of a software team. Do it well, pair it with product vision and judgment, and you can build great products, fast.

Doing a thing means putting software in customers’ hands. Until you ship, your software creates no value. If shipping is sluggish, risky, or error-prone, you’ll do it infrequently—and sluggishness infects your entire company and culture. If shipping is fast, reliable, and high-confidence, you’ll do it often. Frequent shipping means frequent learning, better decisions, and faster progress in the right direction.

We wrote this down 13 years ago as: “*Shipping is your company’s heartbeat.*” It rings true now more than ever.

And then came the AI wave.

Times of change force you to ask: what really matters? What do you keep? What do you change?



For us, the focus on enabling incredible customer service stayed constant. But we saw a clear line of sight to a future where AI could actually *do* the work of customer service—and could do it faster and better than humans. That led to the birth of Fin, the highest-performing AI agent for customer service. We pivoted our entire research and development team to building it, and shifted our culture to be AI-first. Fin is now on a trajectory to be far more successful than our original product ever could have been.

Once again, we're in uncharted product territory: a huge amount of software to build, an uncertain shape, and fast-evolving technology. But that plays to our strength—shipping and iterating at speed. And AI itself has given us new tools and agents to move faster than ever before.

But it's also made me appreciate the most essential skill of software engineers, especially in an era where AI threatens to eat the job of writing code: the ability to *understand* what's really going on in the messy, wonderful systems we create. Without fast understanding, you lose confidence and precision. You're either guessing, or wasting endless time trying to figure things out.

To be a great engineer, you need to be great at:

- *Understanding your customers:* How they use your product, what's working, what's not
- *Understanding your production systems:* Why something is slow, why something's broken, where to fix it
- *Understanding how your software is built and deployed:* Why it's brittle or slow in places, and what it would take to improve it

Each of these gets harder in the AI era, which only increases the importance of this core skill:

- *Products are more complex.* Instead of deterministic behavior, AI is probabilistic. It will surprise you. It won't just work because it worked on your machine.
- *Production systems now depend on costly, slow, unreliable (but magical) LLMs.*
- *Development environments are cohabited by AI agents that are prolific at producing "sort-of-working" code.* Your job is to harness the upside and contain the downside—while moving at speed.

The complexity ratchets up, but the truth becomes clearer: the bottleneck isn't shipping (we've largely optimized that), and it isn't even writing code (which is now easier than ever). The bottleneck is *understanding*. Developing the depth and speed to know exactly what to do next to make your product or system better.

*If shipping is your heartbeat, then understanding is your breathing.* Thankfully, we can use AI tooling for faster and sharper understanding. The deeper and more effective your breathing, the more oxygen you pull in. In turn, that fuels sharper decisions, faster fixes, and better products.

## Conclusion

In the AI era, your organization's competitive advantage is no longer determined by how fast you can ship code, but by how fast you can validate code and learn from it.

Observability, when operationalized as a core platform capability, is the only system that can bridge the gap between developer intent and production reality at the speed of AI. It's the only way to accelerate without turning your increased throughput into systemic risk.

In the next few chapters, we'll unpack a number of concepts and implementation details that will more concretely help your teams drive this strategy. In the next chapter, we'll take the next step by learning how to reexamine software delivery via systems thinking.

---

# Systems Thinking for Software Delivery

In the previous chapter, we explored how AI amplifies the existing behaviors of your organization. The critical question then becomes: how do we optimize the specific behaviors that define organizational learning speed? To answer this, we must look beyond isolated tools and examine software delivery as a complex sociotechnical system.

Every complex system has points of leverage, and software delivery is no different. By taking a step back from individual tools and looking at the system as a whole, we can find the right leverage points to accelerate organizational learning.

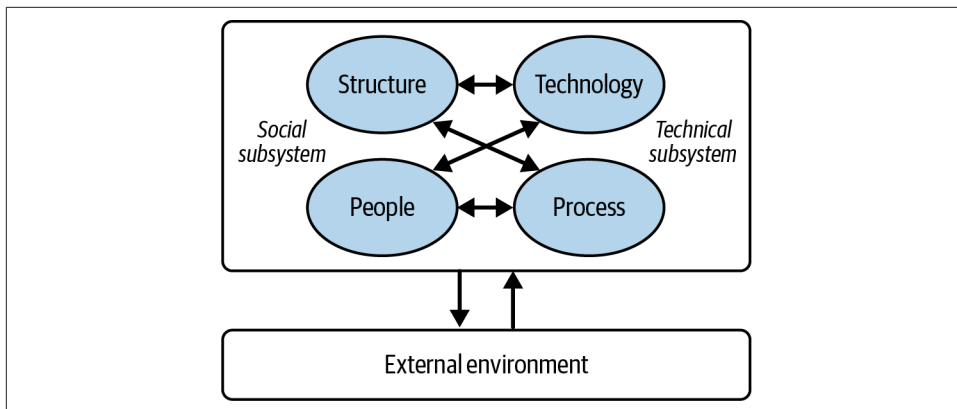
The **2025 DORA report** describes this need bluntly: “successful AI adoption is a systems problem, not a tools problem” (page 4). It also states, “when AI dramatically accelerates software development, our control systems—that’s us—must also speed up....We need fast feedback loops—faster than ever—to match AI-accelerated code generation” (page 9).

In this chapter, we’ll use systems thinking to illustrate the feedback loops that shape incentives, norms, workflows, and technical capabilities. We’ll look at what sociotechnical systems are, what levers shape them, and the essential role observability plays in creating effective feedback loops. You’ll learn which information flows to target—and why those produce better results than changing budgets, metrics, or org charts.

## Sociotechnical Systems

Let’s start by using systems thinking to examine the software delivery process. *Systems thinking* means holistically understanding complex problems by viewing them as interconnected wholes, rather than as isolated parts. Systems thinking focuses on the relationships, patterns, and feedback loops between elements to show how actions in one area affect the entire system.

The first thing to acknowledge about software delivery is that it's a sociotechnical system. As shown in **Figure 24-1**, a sociotechnical system is any system where technology and human behavior are deeply intertwined. Changing the technology changes organizational behavior, and changing organizational constraints changes system behavior.



*Figure 24-1. People and structure influence what work gets done, how it gets done, and what gets prioritized. The technical subsystem and its demands shape cognitive load, handoffs, failure modes, and the structures that form around them. These forces are reciprocal. You can't solve one side and move on; the system is continuously coproducing outcomes.*

Your production environment isn't just code and infrastructure; it's also the teams that operate it, the structures they work within, the incentives they respond to, and the communication patterns they rely on. People, structure, incentives, processes, and constraints all shape each other.

The behaviors you want are produced (or suppressed) by how those elements interact. You must adjust and adapt for both in order to drive meaningful change.

## You Must Optimize Both the Social and Technical at the Same Time

The *Trist principle* states that organizations must jointly optimize their social and technical systems for peak performance. Improving one while ignoring the other reliably produces suboptimal outcomes.¹

Every technical decision (architecture, tools, platforms) has social implications (team structure, hiring, processes), and every social decision has technical ones. At senior

---

¹ Eric Trist, *The Evolution of Socio-Technical Systems: A Conceptual Framework and an Action Research Program* (Ontario Ministry of Labour, 1981).

levels, both managers and individual contributors need a grounded understanding of both the technical reality *and* the organizational dynamics around it.

In practice, the most common violation of this principle is when managers, directors, and vice presidents make all the important tool decisions without consulting the engineers closest to the problem. Engineering executives often believe their technical skills are sufficient to represent both sides of the system, even if they haven't written production code in a decade. Sometimes they are right. But getting buy-in from the people who will have to live with these decisions and own them is never a bad idea.

Remember that your goal is to change how your sociotechnical system learns. There are many obstacles in your way. Management is often frustrated by the slow pace of progress. Engineering cycles often slip away into firefighting. You will want to throw simple solutions at these problems.

Unfortunately, you can't hire your way out of this problem (many have tried). You can't buy your way out of it with more tools (many have tried).

But you *can* fix it—together. And systems thinking is the place to start.

## Changing Information Flows

In her seminal work on systems thinking, Donella Meadows identified a hierarchy of intervention points in complex systems.² The weakest are surface-level adjustments: tweaking budgets, changing metrics, shifting locations. These create temporary effects the system quickly absorbs and neutralizes.

If your organization carries the unresolved sociotechnical debts from [Chapter 23](#), surface-level tweaks won't help—those are what got you here. You need to apply force at points of leverage to create lasting change. The most powerful interventions *change the information flows* that govern how the system behaves.

An information flow is the transmission of data, signals, or knowledge between components of a system—influencing how the system acts, adapts, or maintains stability. Changing information flows changes what people can see. That changes their decisions and behaviors.

Effective change isn't about how much effort you apply—firefighting consumes enormous energy but doesn't change the system. To change the system, you need leverage points and feedback loops.

When you change your observability, you change the information dynamics that govern how hundreds of other decisions get made, every single day. It's a change to your underlying sociotechnical OS.

---

2 Donella H. Meadows, *Thinking in Systems: A Primer* (Chelsea Green Publishing, 2008).

Observability is an information flow. It's your code reporting back from production with vital information about your users, your product, your system, and your code. Most people understand this. Less widely understood is the role observability plays in sociotechnical systems: it creates cyclical information flows—feedback loops. No observability? *No feedback loop*. Just a chaotic mess of causes and effects happening all over, with no logical link between them.

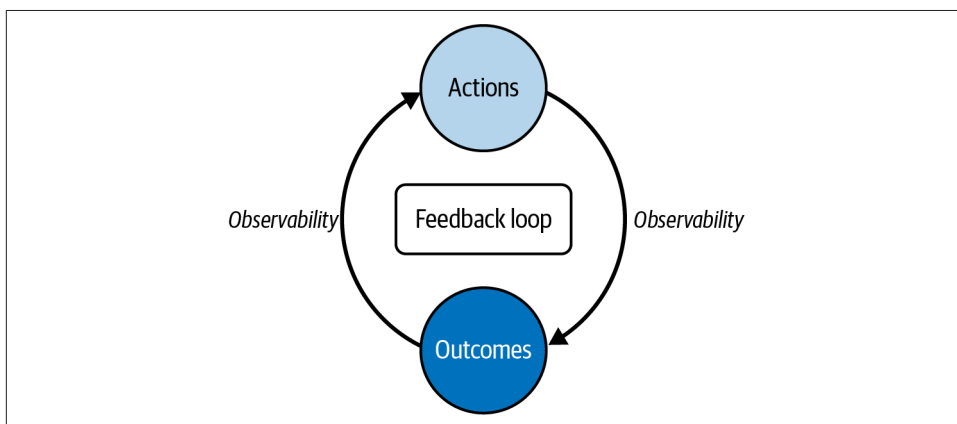
This focus on feedback loops as the core point of leverage is the foundation for everything that follows.

## How Feedback Loops Drive Change

Most organizations are swamped. Resources are overcommitted, goals are slipping, teams depend on other teams that are also behind, and constant firefighting makes progress hard. With so little slack capacity, how does anyone move forward?

The only way out is to *identify key feedback loops and address them upstream*. That work will create shifts so that teams can go from fighting raging bonfires every day, to fighting smaller trash fires every few days, to fires that can be easily stomped out to—ultimately—preventing the matches from being struck in the first place.

Software delivery is composed of feedback loops. A feedback loop exists whenever actions influence outcomes, and outcomes influence future actions, as shown in [Figure 24-2](#). Did a team invest in great tools because they were such a great team? Or did investing in better tools make them a better team? (The answer to both: yes!)



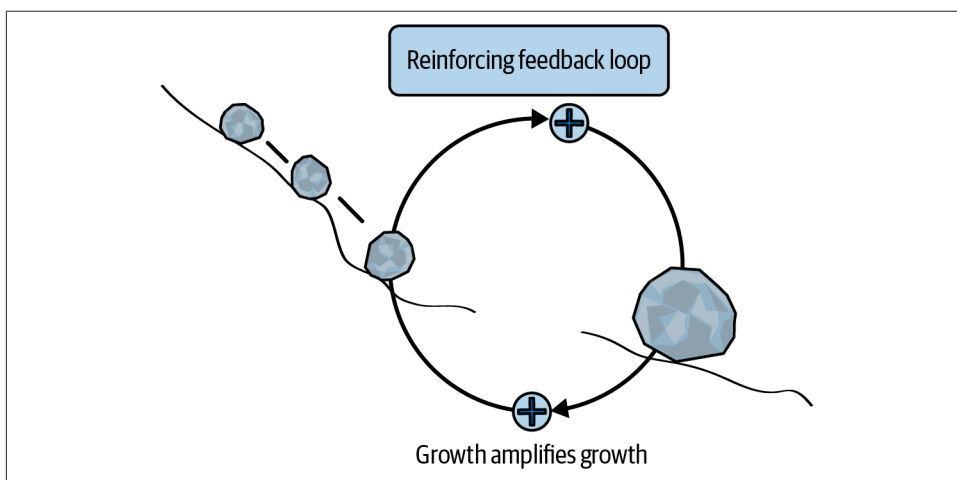
*Figure 24-2. A simple feedback loop: actions influence outcomes, which influence future actions. Both arrows represent observability.*

There are two types of feedback loops: amplifying and balancing. These describe whether a loop accelerates change or resists it. All organizations have both.

## Amplifying Feedback Loops Accelerate Change

An amplifying feedback loop (also known as a “reinforcing” or “positive” loop,³ as shown in [Figure 24-3](#)) is a self-reinforcing cycle, where a change in the system triggers a response that increases the original effects, causing the changes to accelerate.

If you dropped a rock at the top of a snowy mountain, snow would stick to it as gravity rolled it downhill, making it heavier, stickier, and faster as it rolled, until it became (or set off) an avalanche. If you dropped the same rock at the bottom, without the downhill slope to kickstart a loop, you would get...a dent in the snow.



*Figure 24-3. Gravity and momentum take whatever is happening (snow accumulating on a rolling snowball) and make more of it happen (the snowball gets bigger).*

Amplifying loops can spiral upward into virtuous cycles or downward into death spirals. They amplify whatever you’ve put into them. Amplifying loops are the source of exponential, nonlinear effects in organizations, and they are a powerful driver of change.

## Balancing Feedback Loops Create Stability and Equilibrium

A balancing (or “negative”) loop counters change by pushing in the opposite direction, as shown in [Figure 24-4](#). If you heat a room, the thermostat will kick in to regulate the temperature. Without it, the room becomes an oven.

---

³ In business, they sometimes call this a “flywheel.”

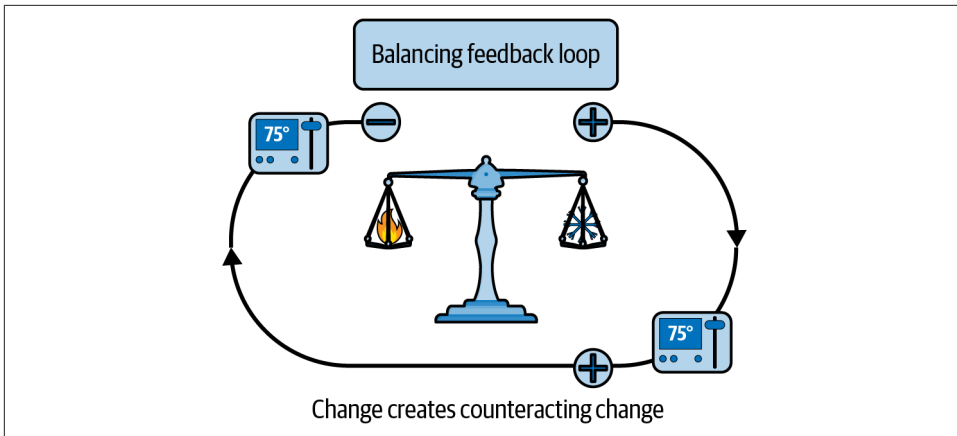


Figure 24-4. A thermostat regulates temperature. Once the room drops below 75 degrees, it warms. Once it exceeds 75 degrees, it cools. The system reinforces equilibrium.

Balancing loops create stability and maintain equilibrium. They resist spiraling into growth or decline, and they are the source of most self-regulating and persistent effects in organizations.

## Small Shifts Can Trigger Massive Changes

When a long-stable situation suddenly experiences dramatic collapse or upheaval, it's rarely attributable entirely to some new force in the system. More likely, an amplifying loop has been straining against a balancing feedback loop for quite some time, until a small shift in circumstances overwhelmed its ability to balance.⁴

This is often the story behind a talent exodus: a company seems to have stable retention for years, but then loses half its senior engineers in three months. Beneath the surface, dissatisfaction had been building for years: arbitrary promotions, canceled initiatives, few advancement opportunities—an amplifying loop. But inertia, golden handcuffs, and a tough job market kept people in place—a balancing loop.

Then one well-respected engineer quits, lands somewhere great, and posts about it online. Other unhappy employees realize that leaving is possible; the balancing loop is removed. More leave. The perception forms that “everyone’s leaving,” and departures snowball until it becomes a reality.

⁴ In proverbial terms, this is often cited as “the straw that broke the camel’s back.”



## The Difference Between a Virtuous Cycle and a Death Spiral Is the Ability to Self-Correct

The difference between a virtuous cycle and a death spiral is rarely about intent. It's about whether the system can see clearly enough to correct itself. Let's look at an example.

Suppose team A decides to break up their monolith. They spin off one new service, it works well, deployment gets faster, and they feel accomplished. They create another, and then another. A virtuous cycle begins.

In the virtuous scenario, they instrument each service carefully, tracking cross-service latency, deployment frequency, and failure rates. When their eighth service starts causing timeouts and cascading failures, they see it immediately in their traces and are able to understand why. They recognize they've hit the limits of their operational maturity and pause. They invest in better tooling, improved debugging, and more on-call training. Then they resume. The feedback loop gave them the visibility to self-correct.

Now suppose that same team had no ability to see small failures—basic logs and metrics, no traces or precision tooling. When the eighth service causes the same problems, they may not even notice. If they do, they can't explain why. They guess that “better service isolation” is the answer, so they create three more services to “fix” the dependencies, which makes debugging even harder. Incidents start taking longer. They add more services to “simplify,” and conditions degrade further. Engineers start leaving because nothing works. Two years later, the system is unmaintainable and the team is working on a costly rewrite.

The intent in both scenarios was identical. The engineers were always trying to do the right thing. The difference was whether the team could see clearly enough to know when to stop, adjust, or reverse course.

## Feedback Loops in Software Delivery Systems

Feedback loops are continuously at work in your software delivery organization. As mentioned in [Chapter 23](#), all core capabilities are loops:

- *Deployment*: I shipped code. → What happened? → Adjust.
- *Code review*: I wrote this. → Does it meet our standards? → Improve.
- *Incidents*: Something broke. → What and why? → Fix and learn.
- *Product development*: We built this feature. → Are users getting value? → Iterate.

Each loop requires sensing, interpretation, and response. *Observability is the sense mechanism that enables all of your feedback loops to function.* It's the information channel that connects actions to consequences across deployments, incidents, and product decisions.

When sensing is degraded—lossy, laggy, unpredictable, wrong, or missing—organizations adapt in ways that reduce productivity. They slow down, batch changes, introduce heavyweight approval processes, and institutionalize workarounds. The system becomes safer in the short term but slower and less adaptable over time.

The quality of your observability plays a key role in how your sociotechnical systems interpret, respond, and take future actions. Let's look at how signal quality translates into outcomes.

## Observability in Amplifying Loops

Amplifying loops accelerate and escalate with each iteration, making them powerful drivers of behavior change. Without good observability, your software development loops are far more likely to spin into doom loops than build toward virtuous cycles. Here are three concrete examples.

### The confidence loop

High-quality observability creates a confidence loop, building the confidence to deploy more frequently, leading to small-batch releases that are easier to debug. Fast feedback enables faster issue detection, which decreases MTTR. Engineers learn to trust their ability to fix problems quickly, so they take more risks and move faster. The loop builds momentum with every cycle.

Poor observability triggers the opposite: fear of the unknown leads to infrequent deployments. Changes pile up, deployments become large and failure-prone, and incidents take longer to resolve. Deployments become something to fear. The team slows to a crawl.

### The learning accumulation loop

The learning accumulation loop is a virtuous cycle that rewards curiosity and creates more of it. Engineers who can easily find answers ask more questions. They build deeper, more accurate mental models, which enables faster debugging. Their intuitions improve, driving continuous exploration—and expertise compounds, as shown in [Figure 24-5](#).

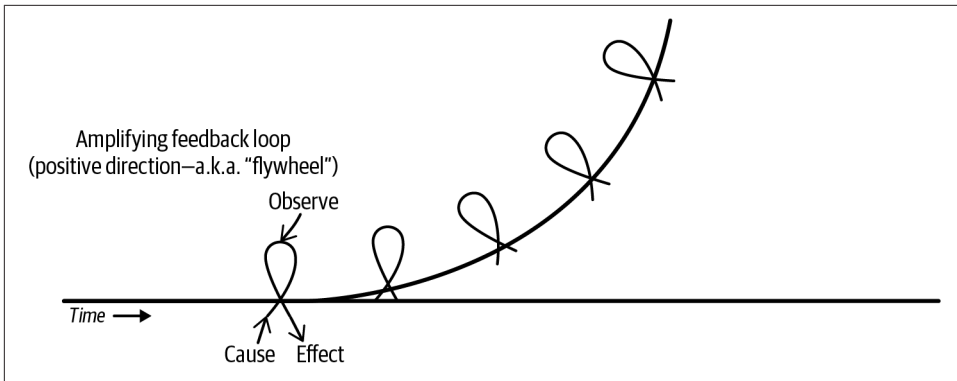


Figure 24-5. An amplifying loop escalates with each repetition until acted upon by another force or new information. Amplifying feedback loops are the greatest drivers of change in an organization.

But when answers are hard to find, engineers stop asking questions, as shown in [Figure 24-6](#). The system becomes opaque and teams rely on shallow pattern-matching, and critical knowledge concentrates in a few individuals, creating serious institutional risk if they leave.

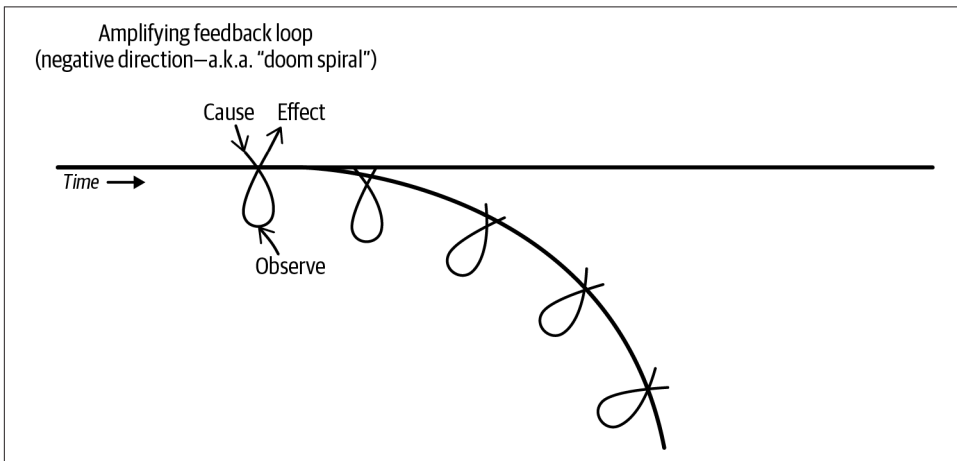


Figure 24-6. An amplifying loop can also be the source of exponential degradation. In most cases, the intent is the same. Observability gives people the clear, timely information they need to self-correct.

The intention behind amplifying feedback loops and death spirals is usually the same. People generally try to do their best. The difference in direction is driven by whether they have enough clear, timely, accurate observability data to identify what's happening and self-correct.

## The system quality loop

The system quality loop is one where robust observability lets engineers detect and resolve small anomalies in real time—like subtle latency spikes and edge-case errors. That creates a clean baseline, making future issues easier to spot and allowing quality to compound.

Without it, small defects accumulate into a “normally broken” state. New problems are masked by background noise. Teams get trapped in a firefighting cycle, addressing only the most catastrophic failures while reliability and resilience steadily degrade.

## Observability in Balancing Loops

Balancing loops play a critical role in ensuring continuity and stability of operations. But in order to balance, they need timely, accurate information.

Without good observability, balancing loops fail to provide balance or equilibrium; instead they create wild oscillations or freeze up into risk aversion and stasis.

### Incident involvement breadth

During major incidents, teams pull in more people for faster resolution and broader learning. But too many people creates coordination chaos, while too few means missed learning opportunities. The system should match incident participation to complexity and learning value.

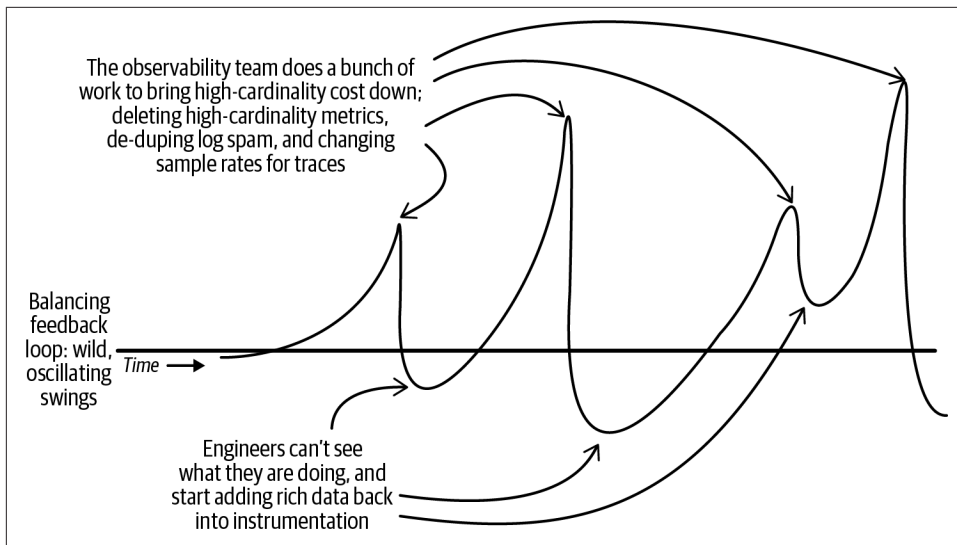
Without observability, teams either create incident mobs (where everyone gets pulled in, and nobody knows who’s doing what) or exclude everyone except the hero engineer (where learning doesn’t spread and that person just burns out). They can’t tell whether more people helped or just created confusion, and they swing between “all hands on deck” and “too many cooks.”

With observability, teams can measure resolution time against participation, and track learning spread against coordination overhead. They can detect patterns, like “engineers who shadow more than three incidents learn to resolve independently, but shadowing more doesn’t help.” They can build structure—defined roles, spectator modes for learners, recordings for broader sharing. The system calibrates: the right people in the room to solve it, the right people observing to learn from it.

### Observability cost and usage regulation

Engineers instrument systems and run queries, consuming storage, compute, tokens, and network resources. If they have no visibility into what that costs, they can’t self-regulate.

Without observability of observability, teams experience either cost explosion (“our bill just hit \$2 million a month—turn everything off!”), followed by overcorrection when finance complains (as shown in [Figure 24-7](#)), or cost anxiety that leads to underinstrumenting critical systems. They can’t tell which metrics nobody looks at or which queries are scanning terabytes uselessly.



*Figure 24-7. Without visibility into consumption, costs grow unchecked until finance complains—then observability teams slash cardinality and drop the most valuable data.*

However, with real-time observability of their usage, teams can see exactly what’s driving costs:

- “The checkout service is generating 60% of our trace volume but only 10% of our queries reference it.”
- “This one dashboard query scans 500 gigabytes (GB) every time it loads and nobody has looked at it in three months.”

They can identify high-cardinality explosions:

- “Adding `user_id` as a dimension multiplied our metric cardinality by 10,000x.”

They can measure value:

- “The fraud detection team runs \$5,000 worth of queries per month but that instrumentation caught a \$2 million incident in 10 minutes.”

They can optimize intelligently—increasing sampling where it matters, removing unused metrics, and tuning retention by datatype. Costs stay bounded while coverage expands where it counts. Teams use instrumentation to make targeted investments based on expected returns instead of rocketing back and forth between runaway spending and reactive scarcity. And if each team has a budget, they can manage to that budget, as shown in [Figure 24-8](#).

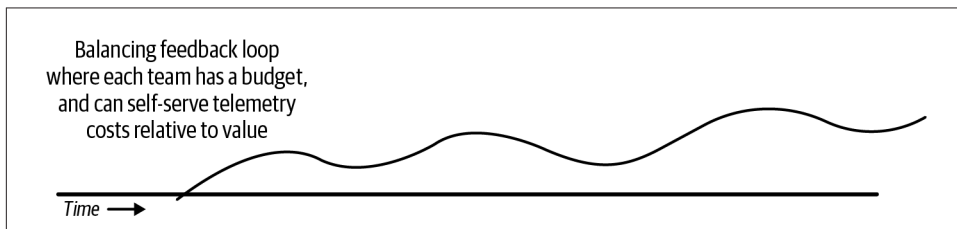


Figure 24-8. With real-time cost visibility, teams can self-correct and stay within budget.

The meta insight is that you can't regulate what you can't measure. Observability about observability is about taking the same sociotechnical practices, context-rich data, and precision tools that you use in production and applying them to your own use of observability. This closes the loop and lets organizations invest strategically rather than lurch between overspending and panic cuts.

## Partial Feedback Creates Systemic Distortions

So far in our examples, we've assumed that observability is either very good or very poor. Reality is usually more nuanced.

Inconsistent observability quality can be more damaging than uniformly poor observability. A false sense of security builds while risk quietly accumulates. High-quality observability needs to be present throughout the critical paths of your software delivery organization. When it isn't, the effects ripple everywhere:

*Feedback is lossy: you only see part of what happened.*

You deploy a change and the error rate doesn't spike, so you assume it worked. But you're not seeing that latency increased for a specific customer segment, or that a code path is swallowing errors, or that the change broke an edge case affecting 0.5% of users.

You make decisions based on incomplete information. You might even double down on the bad behavior—"that deploy went fine, so let's do more like that"—when it actually caused problems that you just can't see.

*Feedback is laggy: you see what happened, but only much later*

You ship on Tuesday. A customer complains on Friday. By then, you've shipped 40 other changes. You can't isolate cause and effect, and the feedback loop cycle time is so long that you can't iterate on what you learn.

You're flying in the dark. You're making the next decision before you know if the last one worked. That's like trying to learn to ride a bike when the bike's response is delayed by 30 seconds; you're going to crash.

*Feedback is unpredictable: sometimes you see things and sometimes you don't.*

A cryptic error appears on your dashboards—twice a day? Three times? It's hard to tell because the ingestion pipeline sometimes hiccups. You can't tell if you're seeing a real pattern or just gaps in the data.

When this happens, engineers stop investigating intermittent issues because they've learned they can't trust what they're seeing. The feedback loops exist in theory but fail in practice.

*Feedback is wrong: the signal is misleading.*

Your metrics say everything is fine, but customers are angry. Or alerts fire constantly for nonissues, so engineers learn to ignore them. Eventually, your teams start missing real problems.

False positives and false negatives both destroy the feedback loop—one by creating noise that drowns out signal, and the other by creating false confidence. Engineers learn that the observability system lies to them, so they route around it with intuition and heuristics.

*Feedback is missing altogether: it's an open-loop system.*

This may be the most insidious of all. You're making changes with no way to know if they worked—or even worse, you *do* have a way, but *no one ever looks*. Every deployment is a coin flip. Engineers develop superstitions and self-imposed constraints—the classic, “don't deploy on Fridays.” Because engineers have no way to confidently connect cause and effect, problems strike with no warning. The system is degrading over time in countless small ways, but nobody can see them until they are big enough to fail catastrophically. So, catastrophic failures are all you get. This is like driving with your eyes closed and only turning the wheel after you hit something. Of course you're going to drive slowly: you're terrified, *and you should be*.

Degraded feedback doesn't just slow you down; it actively makes you worse. With good feedback, you learn from every action and apply it to the next. *With bad feedback, you learn the wrong lessons*, building incorrect mental models and losing trust in your ability to change things safely. So you stop changing things, and the system ossifies.

And because these are *amplifying loops*, degradation accelerates. Slow feedback leads to less frequent deploys, which means even slower feedback. Lossy feedback leads to incorrect fixes, which create more problems and add more noise.

The worst part is that engineers adapt to broken feedback loops by creating work-arounds that become institutionalized. They may create elaborate test environments that can never replicate production, change approval boards that slow everything down, or runbooks that are perpetually out of date. They may establish “safe deployment windows” and “deploy freeze periods,” despite evidence that these make things worse.

All of these are theatrical attempts to add safety when you can’t see what your code is doing. What you end up with is an organization optimized for *not breaking things* rather than *making things better*. Which may even sound prudent, until you realize that in a competitive landscape, standing still is falling behind.

## Leverage Points in Sociotechnical Systems

Observability is a powerful capability, but it operates within a larger hierarchy of system influence. To apply force for maximum impact, you need to find leverage points—places where a small shift produces large changes everywhere else.

In software delivery, information flows and feedback loops are powerful leverage points. But there are others outside the domain of observability.

### Leverage Points and the Limits of Observability

In *Thinking in Systems*, Donella Meadows describes 12 leverage points in any complex system, ordered roughly by their power to produce change.

The following is an abbreviated subset in the context of software delivery, from highest to lowest impact. While points 1, 2, and 3 are not explicitly about information flows, they do require high quality signals in order to function and course correct. And points 4, 5, and 6 are exactly within the domain of observability:

1. *Goals*: The goals refer to the purpose or function of the system. They’re the highest leverage point available to most executives. Individual contributors who don’t “own” the goal can and should play a vital role in vetting and influencing the goals that are set.
2. *Self-organization*: Self-organization is the ability to add, change, or evolve system structure. Resist the urge to read this as being about org charts: this is about agency and autonomy, especially at the team level.



3. *Rules*: Rules, followed or not followed, lead to incentives, punishments, and constraints. The right guardrails add consistency and predictability, and prevent everyone from writing their own database from scratch.
4. *Information flows*: This refers to who does or doesn't have access to information. Missing or flawed information flows are among the most common sources of dysfunction in any system. Information can also be lost in a flood of undifferentiated detail.
5. *Amplifying feedback loops*: These are the source of explosive change. The more a loop works, the more power it picks up to do more work.
6. *Balancing feedback loops*: Though not drivers of change, they are essential nonetheless. As Meadows says, "a system with an unchecked reinforcing loop ultimately will destroy itself" (page 155). Balancing loops are reservoirs of stability and resilience.

Goals, autonomy, and rules are extremely powerful points of leverage, and while good observability can help you design, build, and refine these things, it can't overcome the lack of them.

Feedback loops that are timely, precise, and relevant enable *self-awareness* in humans and *self-governance* in teams. They generally produce the right sociotechnical system behaviors without needing constant correction or oversight.

Excellent signal and fast feedback loops will never compensate for a lack of purpose, disempowerment, or unchecked chaos in the system. But it's equally hard to set the right goals without good information.

## Push in the Right Direction

One final caution: Meadows observes that people embedded in a system tend to intuitively sense where the problems are. They can describe the system, identify the forces at work, and guess with remarkable accuracy where the best leverage points are.

However, people have a pronounced tendency to push the right leverage point in the *wrong direction*.

Deploys are a classic example. Everyone knows that deploys introduce change, and change can be hard. Earlier generations of software professionals saw this too—and responded by introducing change advisory boards, monthly release trains, and a slew of processes that made it slower, harder, and more dangerous to ship. They found the right leverage point, but they pushed the wrong way.

Something to remember as you go looking for leverage points to push in your own organization.

## Conclusion

Let's go back to the overarching goal: AI makes your ability to *move swiftly with confidence* more important than ever. The limiting factor for how fast your organization can move is increasingly how fast you can validate and learn from the code you just shipped.

AI amplifies the systems you already have. Fast, accurate feedback loops compound your gains. Slow, lossy loops compound your failure modes. More buggy code shipped means less understanding, and more complexity to navigate means moving at a slower pace, with fewer chances to learn.

This is why observability is such a powerful leverage point. It is the *only* intervention that can turn amplifying loops into virtuous cycles while also stabilizing balancing loops toward self-regulation and healthy persistence.

“Observability” is many things: a type of software, a flow of information, and the sense mechanism that creates feedback loops from cause and effect. When you improve the quality of this signal, it becomes an accelerant that unlocks agency and makes coordination easier. Improving it doesn't just make one thing better—it *accelerates every feedback loop* and thus your organization's ability to learn, decide, and adapt.

In the next chapter, we'll apply these sociotechnical systems concepts specifically to observability. In successive chapters, we'll show you how to diagnose your current state, make the business case, and drive the necessary change inside your organization.

# The Observability Landscape Through a Systems Lens

In the previous chapters, we established the *why* and *what* of accelerating organizational learning speed through feedback loops. In this chapter, we get into the *how*.

Amplifying feedback loops can spiral into virtuous cycles or death spirals. Balancing feedback loops can create stability or devolve into chaotic swings or rigidity. What makes the difference between learning and flailing is how you're using those loops, and the quality of the observability tools feeding data into them.

We'll start by looking at how feedback loops typically work in most organizations. Then we'll contrast them with the feedback loops that actually accelerate learning, and examine why closing that gap has been hard for so many teams. We'll look at two generations of observability tooling and which loops each one supports, and we'll close by examining how AI is both accelerating the need for change and opening up new ways of working.

This chapter focuses on recognizing the capabilities you need so that you can choose observability tooling that fits. The next few chapters build on that by examining the business case, how to diagnose your current investments, and how to drive organizational change.

## The Landscape Feels Noisy Because the Labels Are Noisy

Before we begin, let's talk about the elephant in the room: the "observability" category has become almost unfathomably large. It started as monitoring, logs, and application performance monitoring (APM) thrown into a blender and quickly metastasized into a budget line item that eats up to a third of infrastructure spend in some companies. With that growth came aggressive marketing. The term is now applied to a wide

range of tools and niches: data observability, AI observability, large language model (LLM) observability, and more.

As a result, many vendors claim similar outcomes using similar language, despite products (and outcomes) that are fantastically different. Unfortunately for you—the reader of this book and the consumer of those tools—that makes the entire landscape a snarled, expensive mess. Let us be the first to say: *we’re sorry*. This is not what we intended when we first coined the term.

But the lessons from earlier in this section can help. Zooming out, a systems lens offers a cleaner organizing principle: instead of starting with vendor categories or feature lists, start with the feedback loops you need to strengthen, and the decisions those loops need to support. Our goal is to learn fast.

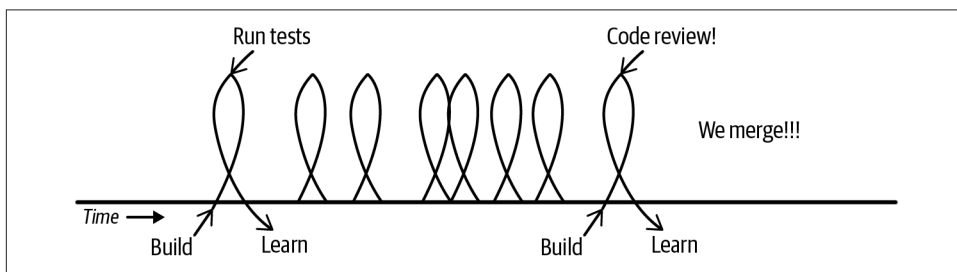
Let’s start with the loops that most organizations run today, and then look at the ones you actually need.

## The Loops Most Organizations Run Today (and What Is Missing)

Despite the last decade and a half of DevOps culture and “you build it, you run it,” the reality for most teams is this: of the two main feedback loops—development loops and operational loops—only one of them includes production.

### Development Feedback Loops

The software development feedback loop is fast, but isolated. It’s optimized for implementing and validating business logic. The phases are: build → run tests → iterate → merge → move on, as shown in [Figure 25-1](#).



*Figure 25-1. The software development feedback loop focuses on local builds, running tests, and iterating until tests pass. The loop continues until code is reviewed and merged.*

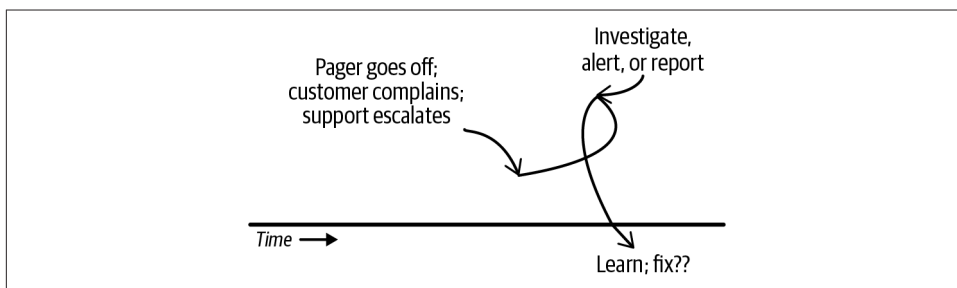
Most developers live in their development environment, their task tracker, their chat app, and a handful of other tools. They build, run tests, and learn. They build, run tests, and learn—again and again—until they deploy to an integration environment, click around manually, and submit a pull request. Once it’s reviewed and merged, the job is done. On to the next feature.

Developers know their code will eventually get deployed to production, but they often don’t know when or what happens afterward. Unless there’s a bug or an incident, they typically don’t look at production at all.

Most organizations don’t have a feedback loop connecting developers to their changes in production. The only loops that include production learning are operational ones.

## Operational Feedback Loops

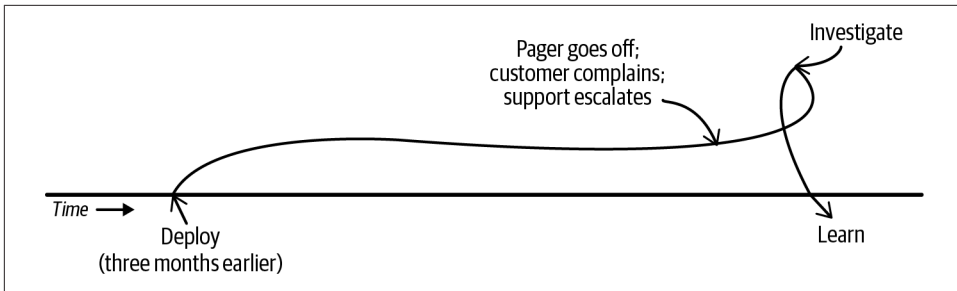
Operational teams run a different loop, oriented around service health. The phases are: alert → investigate → fix, as shown in [Figure 25-2](#).



*Figure 25-2. The operational feedback loop protects service health. It’s triggered by an alert, escalation, or customer complaint, moves to triage and investigation, and ends when a fix is deployed.*

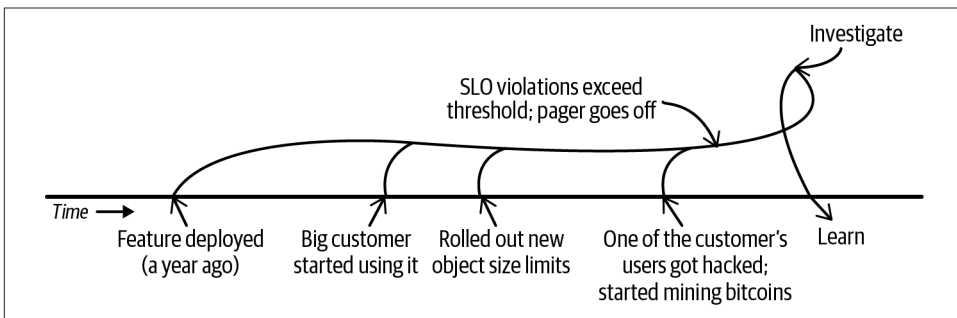
For ops teams (i.e., DevOps, SRE, platform engineering, cloud engineering, or other names), the feedback often comes from dashboards, alerts, and customer complaints. They are typically working reactively, often scrambling to investigate, learn, and fix what went wrong after it has already affected customers.

The ability to learn from these loops is limited, because it’s often not obvious what caused the problem. Alerts are triggered by thresholds—which means they fire only after the impact is significant, once a performance issue is bad enough to threaten the whole system. Sometimes the operational loop is triggered by a recent deployment. Far more often, it’s a customer complaining about something that shipped days, weeks, or months ago, as shown in [Figure 25-3](#).



*Figure 25-3. A customer complaint may be tied to an action taken long in the past, making it nearly impossible to connect the dots back to the source.*

When caused by an issue that shipped long ago, the issue is often the result of combinatorial conditions. For example, when an issue has been lurking for a while, it's usually the result of compounding conditions. Data volume reaches a threshold that causes timeouts, which stacks up threads, which triggers the alert. That's why operational loops more often look like [Figure 25-4](#).



*Figure 25-4. A more common operational feedback loop spans weeks, months, or years. The compounding conditions that trigger an alert often surface long after the code changes that caused them.*

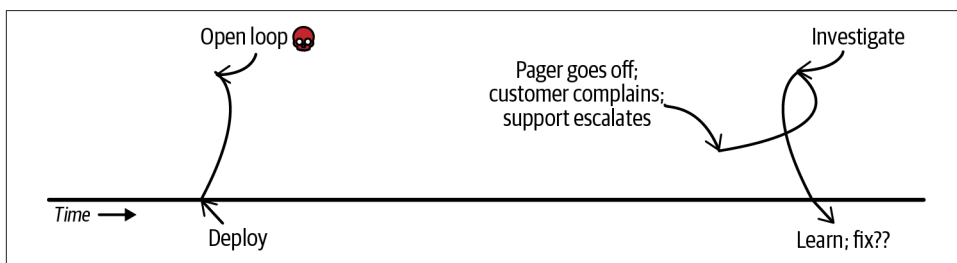
This is where the disconnect between development and operations becomes clear. Operational loops are oriented around bugs, errors, reliability, and outages. Those are important. But reliability is only a tiny part of what developers actually need to know about their code in production.

Most developers should be asking questions like: how are users behaving? Is this feature getting adopted? Where are users dropping off? Why are they quitting? How does what we shipped map to what we intended? Reliability and bugs are part of that picture, but not the main one.

Most developers don't ask these questions, because they don't have the tools to easily ask these questions. As we discussed in the last chapter, if curiosity gets punished, it eventually stops asking, and the opacity of the system becomes its own reinforcing loop.

## The Missing Loop

What's missing is a consistent loop that connects the developer's concern of "I wrote this" to "I can see and understand what it did in production." The developer loop stops at merge; the operational loop starts when something breaks. The gap between them is where learning slows down and costs compound, as shown in [Figure 25-5](#).



*Figure 25-5. In most orgs, there is no connection between the developer loop and production: the developer loop stops after code is merged. The operational loop only begins when something is visibly broken. The gap between the two is where velocity goes to die.*

The terms “inner loop” and “outer loop” are sometimes used for dev and ops, respectively. Fast iteration on the inner loop is fine—but once you're ready to put code into production, you need to follow it there and validate it.

When the developer loop doesn't include production, all “learning” comes from tests. Tests are a proxy for production. But as any experienced engineer knows, what you learn when all your tests pass is...that all your tests pass. Depending on the quality of your test suite, that may be a lot or a little, but it is never a substitute for what you'll learn in production. Production always surprises you, because people are agents of chaos.

In order to accelerate organizational learning, we must build feedback loops that make it easy to connect developer intent with consequences in production, because that is the loop where value gets created.

# The Feedback Loop for Value Creation

At the end of [Chapter 23](#), we included a letter from Darragh Curran, the CTO of Fin, which ended with this:

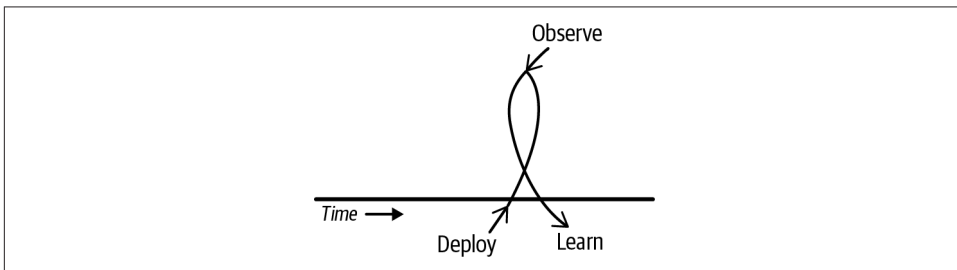
The complexity ratchets up, but the truth becomes clearer: the bottleneck isn't shipping (we've largely optimized that), and it isn't even writing code (which is now easier than ever). The bottleneck is *understanding*...

*If shipping is your heartbeat, then understanding is your breathing...* The deeper and more effective your breathing, the more oxygen you pull in. In turn, that fuels sharper decisions, faster fixes, and better products.

Let's unpack that.

## Shipping Is Your Heartbeat

Every time you ship a diff, you create new value for the business, as shown in [Figure 25-6](#). This is the core loop of value creation, and it probably happens many times a day.



*Figure 25-6. The core loop of value creation: deploy, observe, learn. You don't create value until code gets shipped. You don't start learning until code is in production. That's why most development methodologies emphasize shipping small diffs, frequently.*

As Darragh says, “shipping is your company’s heartbeat”—so it should be as common, easy, and uncomplicated as possible.

This is a theoretical loop, of course.

This is how progress gets made at any tech company: build → deploy → observe → learn → then wrap what you learned into the next thing you build (see [Figure 25-7](#)).

A healthy delivery system executes this loop repeatedly, applying what was learned to each subsequent change. Each deploy is a way to validate assumptions, catch unintended outcomes, and refine both product behavior and system reliability.



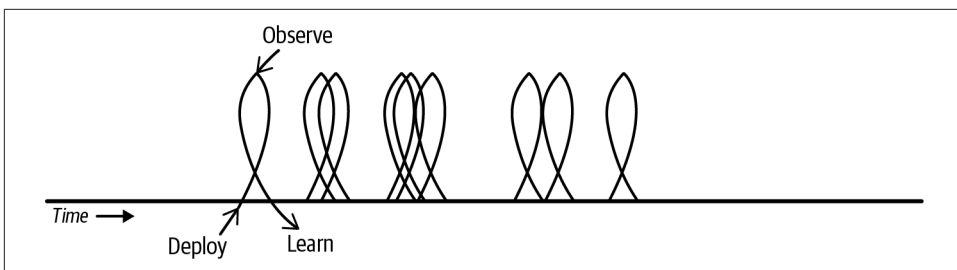


Figure 25-7. Every deploy is a chance to learn. Technology companies often optimize for delivery speed—deploy, deploy, deploy.

Some organizations have made this a high-frequency habit. Fin (formerly Intercom), for example, ships its Ruby monolith around 70 times per day. Each deploy takes 8–15 minutes from merge to full production rollout, supported by powerful, granular observability.

But having an “opportunity to learn” is not the same as “learning.” What happens if you don’t observe? Unfortunately, most companies still have feedback loops similar to Figure 25-8.

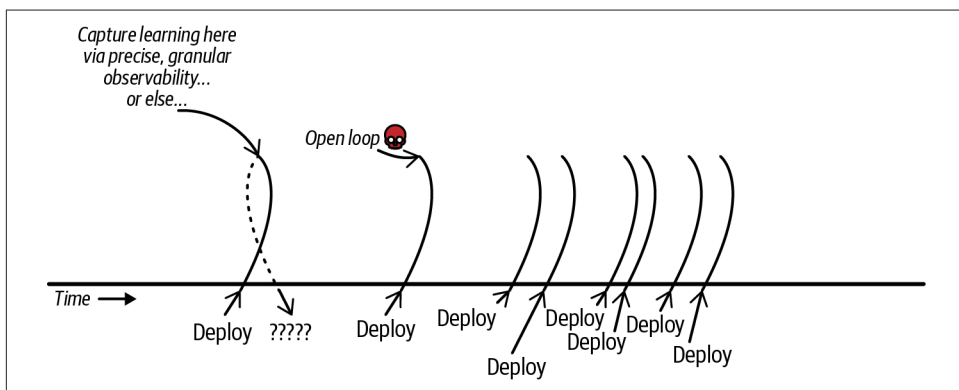


Figure 25-8. If learning isn’t captured by following a code change into production, the opportunity is lost. Deploying frequently is not the same as learning fast. Without closing the loop, feedback is lost and learning stalls.

Every time you deploy and don’t observe to validate, you create an *open loop*. The learnings get lost because you’re making changes without capturing understanding.

Are you creating value, even if you aren’t looking at it? Yes...well, probably...actually, who knows?

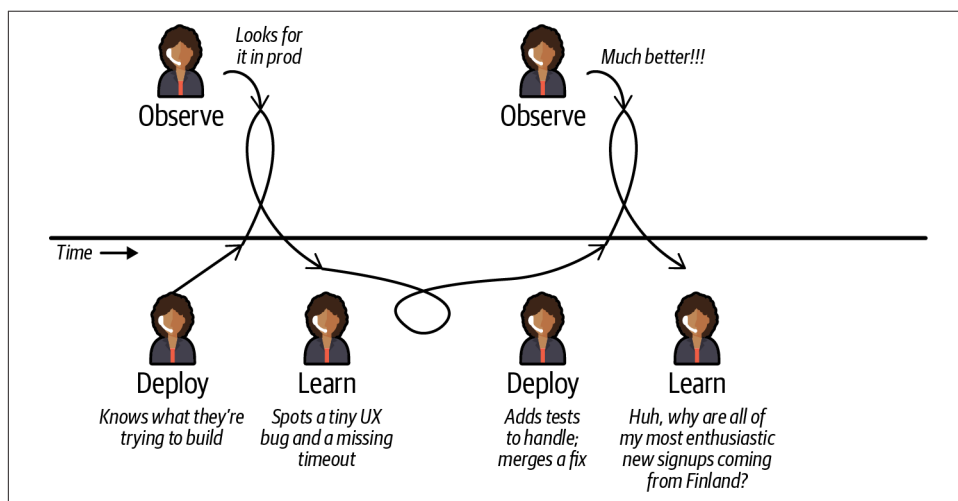
If you wait for an operational alert to tell you something is wrong, you guarantee that you'll only ever learn about problems large enough to threaten the whole system, or severe enough for customers to complain.

## How to Build the Loop and Close the Gap

Production is a zone of chaos. Your system may be processing millions of requests per second across thousands of services for millions of customers. In that environment, even a fleet of AI agents scanning for anomalies is of limited use. Weird things are *always* happening; weird things are the norm. You can't distinguish actionable signal from baseline noise just by looking at the whole haystack.

But if you know exactly *what* you're looking for, *when* it happened, and *where* to look, even very small needles—a single weird request—can be found quickly and easily with the right tools.

To drive real sociotechnical change, the person who wrote the code needs to be able to see what it's doing, understand how users are experiencing it, and continuously improve it from that understanding, as shown in **Figure 25-9**.



*Figure 25-9. A developer encodes their intent in instrumentation, then deploys, and then validates that intent in production. Precision tooling surfaces small issues missed by preproduction testing. The developer adds fixes, redeploys, and looks again—and notices something interesting: new user signups are climbing, driven by users from Finland. They keep exploring to find out why.*

You can't close the loop by having one person write the code and another person watch production. You can't close it by having a developer glance at a dashboard

of high-level aggregate metrics. And you can't close the developer learning loop by connecting it to a downstream operational loop.

The way you close the loop is by tracking developer intent, from writing or generating the code all the way through to production, and then validating that intent using rich context data and precise observability tools.

“No spike in errors” is not precise. Precise looks like this: “Compaction rate for long strings went up 5% and disk footprint went down 30% for requests with a specific `build_id`, combination of feature flags, and test users on our 10% canary deploy versus the 90% baseline—and the distribution of return codes looks unchanged.”

## Why Closing the Gap Is Rare: The Economics of Cognitive Load

As you read this, it can sound so simple and powerful to close the gap. We believe it is both. Yet very few teams—and almost no large organizations—actually operate this way. Why?

Organizations often blame cultural factors: developers don't care about production, or they don't use the tools available to them. A more useful explanation is that the tools haven't been worth the time and energy it has taken to use them.

The industry is trapped in a circular dependency loop: they don't have precision data, so it isn't worth the lift for developers to look at their changes in production, so they don't look, so the tools never get any better.

Our systems evolved to tolerate a certain rate of change, held together with duct tape and intuition. The old ways worked well enough, most of the time, so there has been no competitive pressure to jolt us into disruptive tool changes.

AI looks like it may finally break this logjam by attacking on multiple fronts. AI provides the technical breakthroughs, the competitive pressure, and the disruptive need for developers to follow their changes into production, since that is the *only* way to know what nondeterministic agents will do there.

Let's look at the two generations of observability tooling on the market today, and the feedback loops each was designed to support.

## Two Observability Models, Two Feedback Loops

There are two generations of observability tooling on the market today: the three pillars model and the unified model, as discussed in [Chapter 1](#). We will recap them briefly here, along with which feedback model each excels at solving.

## Three Pillars Model (Built for Operational Outcomes)

The three pillars model evolved out of the monitoring and logging space. It's designed for infrastructure, where engineers must operate hundreds if not thousands of third-party components that they did not write and cannot change. Your networking gear, hard drives, and container IDs spew out a steady stream of metrics and/or logs.

You have limited ability (or desire) to configure this output; you just need to find somewhere cheap to stash it. So you stream metrics into a metrics store, and logs into a logging store, and maybe traces into a tracing engine. (And if you have them, you do the same for profiling data, errors, exceptions, anomaly detection, product analytics, and so on. You can see where the cost ballooning comes from, since you end up storing many different copies of the same data.)

This model is well-suited for infrastructure and the operational loop. It dominates the ecosystem of integrations, so it's easy to get started with. Where it flails—hard—is with developer loops.

To understand the experience of each user or the impact of each change shipped, developers need precision data, exploratory capabilities, and fast feedback loops. The relationships between the datapoints are the most important parts of the data; the three pillars approach destroys these at write time.

Any time you need to crosscut dimensions like user experience, application behavior, or business context, users of this model will face correlation difficulties. The three pillars model requires a lot of manual correlation and intuitive leaps. It works well enough for fixing bugs and responding to incidents, but not enough to understand what happened, the quality of your product, or how users are using it.

## Unified Storage Model (Built for Developer Learning)

The second model, unified storage, emerged from the intersection of business intelligence and APM tools. In this architecture, signal types are all ingested as structured event logs into a single, unified data store, typically backed by columnar storage. All the relationships are preserved, removing the need for manual correlations or pillar hopping (see [Chapter 5](#)).

Columnar storage engines support both high cardinality and high dimensionality (defined in [Chapter 1](#)), so they support very precise questions.

Precision is a function of context richness and cardinality. The beauty of context is that it makes data exponentially, combinatorially more powerful. If you have 29 fields on a single trace or canonical event, and you add a 30th, that 30th is more powerful than all 29 combined, because you can now generate exponentially more combinations and subsets of dimensional detail.

We know these things intuitively. The high-cardinality identifier “Patrick Snuffleupagus” is more identifying than just “boy,” and neither is valuable out of context.

This model is well-suited for understanding the quality and behavior of your application code and the experience of each user. It supports the developer feedback loop (deploy → observe → learn). Its flexibility and speed make it easy to turn questions into fast feedback loops. This model works exceptionally well for turning deployments into developer learning.

The unified model shines at answering exploratory, open-ended questions like, “Show me all requests from mobile users in the European Union with feature flag X enabled that had errors in the checkout flow.” These questions are impossible to formulate on the fly with the three pillars model.

Where the unified model struggles is with operational infrastructure monitoring. It can do some of this work, but it does not shine there.

## AI Changes the Game and Opens New Interaction Models

At the time of writing, the SDLC is undergoing a compressed period of change. New patterns and philosophies for code generation seem to emerge every passing day. While we don’t know yet where we are headed, we can identify some patterns in broad strokes.

AI has upended the economics of software development. Writing the code used to be the most time-consuming and expensive part for many organizations; now it’s all but free. The constraint has moved from “code producing” to “code coherence.” We can produce artifacts much faster than we can reason about them.

The bottleneck has shifted from generating code to validating code. Generation is cheap; confidence is not. In a world where most code is disposable and regenerable, code functions more like a cache. And most of the code in the world is not greenfield code; it’s brownfield or blackfield code,¹ where production is the sole tattered remnant of developer intent. If it’s going to become regenerable, that code will first have to cross over, as discussed in [Chapter 2](#).

Today, wise engineering leaders are laying the foundation for their teams to cross over to generated workflows, and they are dealing with the problems that are already here. The most pressing of which is this: when software changes faster than an organization can understand its consequences, risk accumulates faster than it can be resolved (or code simply never gets reviewed). *The learning feedback loop becomes the limiting resource.*

---

¹ “Blackfield development” is a term for a severely chaotic brownfield project, likened to navigating a dark cave without a light.

AI can reduce or eliminate much of the friction that developers have encountered by trying to fit infrastructure/ops tools to their use case, and MCPs (Model Context Protocols) may have some ability to bridge the pillars and derive insights.² AI can deliver data directly to developers in their dev environments or via Slack.

Ultimately, much of the pain of the developer loop will be best solved by agentic workflows. Developers can declare intent, write or generate code to implement it, test it, deploy it, and validate that intent in production. Agents can do the tedious work of checking back over time, circling back every hour, day, or week, so that even slow-burning issues (a memory leak, a cache drift) can be identified and remediated *before* they are big enough to pose a threat to the system and thus trigger an operational alert.

Some of this can also be contrasted with a complementary approach: adversarial “continuous review” agents that do things like flagging potential memory leaks, inspecting if important constraints of the system are being respected, and so on. Faster feedback loops are a unifying theme here, whether the entity receiving the feedback is a human or not.

This is not just convenience; it’s a structural way to reduce loop friction and increase learning speed. The pieces for this exist today. But they require a very specific technical foundation: unified, structured, high-context events that AI can reason about. The business intelligence approach is what makes these outcomes possible.

## Both Feedback Loops Matter: Lead with the Right One

Operational feedback loops are essential safety mechanisms. They monitor your infrastructure, track aggregate system health, and ensure you’re meeting your SLOs. They’re the primary way to understand infrastructure health, and they’re the safety net that catches the big, system-threatening problems that slip through testing, staging, and progressive deployment.

But they can’t be the *only* mechanism by which developers learn what their changes did—not if you want to develop at the speed that modern competition demands.

Operational feedback loops are downstream, delayed, and oriented around system-threatening conditions. They’re driven by customer complaints and system failures. By the time an operational loop activates, users have already been impacted and your team is already in reactive mode.

Developer-facing observability moves learning upstream—toward prevention, iteration, and product quality. Developer feedback loops let you see problems forming,

---

² Reports to date are mixed, and evidence strongly suggests that agents themselves will go out of their way to seek out richer upstream data, with connective tissue intact, instead of logs and metrics.

understand user experience proactively, and validate that your code does what you think it does before users notice. It's the difference between prevention and cure, and between learning continuously and learning from disasters.

## Align on the Outcomes You're Building Toward

The observability landscape can be legitimately overwhelming. Many of the hundreds of observability vendors are functionally identical, other than pricing and packaging. Other vendors are meaningfully and radically different under the hood. As a customer, it's hard to tell the difference, because vendors are, how shall we say, "not bound by technical accuracy in the sales process."

But you do not need to start off by gathering detailed feature comparisons or performing elaborate bake-offs. A more effective way to start is by getting leaders aligned around one question: which feedback loop is your bottleneck, dev or ops?

If your pain is operational stability—keeping systems up, getting visibility into infrastructure, managing chaotic incident response—then tools built for operational outcomes are the right place to begin. The three pillars model is mature, robust, and purpose-built for infrastructure and other operational use cases.

If your pain is delivery velocity—i.e., you need to ship swiftly, with confidence, and understand the quality of your product as experienced by each customer—then tools built for developer learning are where to start. The unified storage model is built for this.

If your pain is due to an unprecedented influx of AI-generated code that no one understands, or if you are preparing to transition from handcrafted, artisanal code to code written and reviewed by AI, or if you need to trace agentic workflows in production systems, then you too should focus on developer learning.

There is no universal right place to start. However, most organizations are starting from a place where they have invested heavily in their operational feedback loops and very little in their developer loops. This is despite the fact that ops loops are supposed to be a safety net, not the primary driver of organizational learning, and are supposed to deliver rapidly diminishing returns on marginal investments once stability is reached.

In sports terms, most organizations are blowing their wad on the goalie. They aren't even staffing the rest of the field, just giving the goalie more and more dough. And then they wonder why they can't score any points.

Most organizations need both operational feedback loops and developer feedback loops. But you can't build a business case without first building alignment around which problem you are trying to solve, and which feedback loop is at the heart of it. Without that shared understanding, you can spend limitless sums yet still fail to

produce meaningful sociotechnical change. The observability landscape is complex, but the feedback loops that matter are not. Start there.

## Conclusion

In this chapter, we cut through the noise of the observability landscape by focusing on its core purpose: accelerating organizational learning through feedback loops. We established that the reality in most organizations is defined by a critical gap between a fast, isolated development loop and a slow, reactive operational loop. Relying solely on operational feedback for production learnings means losing out on the continuous learning necessary for value creation.

If you want to build at the speed of AI, you need to learn and validate at the speed of AI as well. That means learning from production after each change, through the developer feedback loop. Fortunately, AI is not only driving the need for these capabilities—it's also powering new workflows that enable them and substantially reduce cognitive load.

In the coming chapters, we'll get practical—covering the business case for observability, how to evaluate your current investments, how to drive organizational change, when to build versus buy, and how to build strong vendor partnerships. All of these practical steps depend on first getting clear on what you're trying to accomplish.



---

# The Business Case for Observability

In the AI era, our primary constraint has shifted from developing code to understanding and validating it. To do this swiftly, easily, and consistently, you need to connect developers to their code in production through a single, fast, efficient feedback loop.

Production is where developer intent meets reality. Every deploy is a unique confluence of infrastructure, software, user traffic, and time, which happens once and can never be precisely repeated; observability is how we make sense of that intersection.

This chapter takes the next step: how to get funding support and protect the work that turns feedback loops into sociotechnical change that persists.

Modern observability is rooted in infrastructure, and many leaders are still accustomed to managing it as a cost center. But observability for developers is one of the most powerful levers for accelerating learning and converting engineering labor into product delivery as efficiently as possible. If you're a leader trying to do just that, this chapter will help you make the business case.

In this chapter, we'll talk about how to anchor your business case in the feedback loop you want to accelerate, how to quantify value for your organization, and how to frame observability in a way that survives annual planning cycles.

## Identifying Your Priorities

Developing the business case for observability shouldn't start with comparing vendor feature lists. Instead, leadership alignment begins by identifying which feedback loop is the primary bottleneck in your organization right now: the operational loop or the developer learning loop.

By establishing shared intent, you ensure that your observability spend can translate into measurable improvements that solve real organizational problems. You need a sustainable business case aligned with your strategic priorities—whether that’s the stability of your existing infrastructure or the speed of your development teams.

In this section, we’ll look at the complementary roles of the two loops and how to decide which to prioritize.

## Complementary Roles of the Two Feedback Loops

This isn’t a purely binary choice. Most organizations need both loops to function. The goal is to get clear on which problems you’re trying to solve, because that tells you where to put your energy.

Operational feedback loops are essential safety mechanisms. They monitor your infrastructure, track overall system health, and ensure you meet your SLOs.

Infrastructure (*n.*): The code you have to run, in order to run the code you want to run.

—Charity Majors

Operational feedback loops are downstream, delayed, and threshold-based, driven by customer complaints and system failures. By the time your operational loop fires, users have already been impacted and the threat is sizable.

Operational loops are very important and you definitely need them. But they should not be the only way (or the primary way!) that developers learn about their code in production. They are guardians of stability, the last line of defense, protectors of the last resort.

Developer learning loops let you track developer intent, verify user experience (UX), and validate that your code is working as intended—and identify problems *before* users notice anything is wrong. These loops move learning upstream toward prevention, iteration, and product quality.

This is the difference between prevention and cure, between learning continuously and only learning from disasters.

## Read the Fire Codes

For many organizations, there’s no clean choice between which loops to prioritize. The reality is that many teams are fighting fires constantly while also being under pressure to ship faster. On this point, we’ll borrow wisdom from one of our favorite SREs at Honeycomb:

It’s great that you’re so good at firefighting, but maybe it’s time to read the fire codes?

—Fred Hebert

Many engineering orgs are proud of their firefighters—and they should be. Incident response requires calm under pressure, sharp instincts, and a willingness to take ownership when things go wrong. But when an organization’s identity is built around firefighting, improvement becomes reactive by default. When it takes a crisis to learn anything, all your capacity gets spent on reaction and recovery rather than progress.

“Reading the fire codes” is a metaphor for strengthening the upstream mechanisms. The bigger leverage is in prevention: designing systems and practices that stop fires from starting in the first place. That happens by building feedback loops that reduce the frequency and severity of incidents, and learning loops that make every change safer and more informative.

## The Business Case for Operational Loops

The business case for operational loops is essentially the same as the traditional business case for monitoring and logging. The business cares about system health, and aggregate measures can often satisfy that goal.

The degree of observability in your system matters. Signs of poor or insufficient observability include: customers finding your bugs before you do, small incidents snowballing into major outages, and debugging by intuition that burns out your senior engineers.

Regardless of your current observability level, the business case is the same. ROI is demonstrated clearly and measurably through:

- Improved uptime
- Faster incident response
- Cost savings from prevented outages
- Reduced employee burnout

Operational loops are downstream and time-sensitive. They are triggered by alerts (threshold-based or SLO violations) or by humans noticing problems (customer reports, helpdesk tickets). Operational loops succeed when teams can quickly answer three questions: what is happening, who is affected, and what will restore service fastest?

## Mapping Observability Models to Operational Outcomes

Both the three pillars and unified models can support operational loops:

### *The three pillars model*

It works well for reactive operational workflows and infrastructure, where usage is dominated by aggregate system metrics. With naming discipline and carefully

built bridges between tools, this model can serve dashboards and power simple debugging use cases (with the aid of intuition and deep familiarity).

#### *The unified model*

It also supports operational workflows, and shines at outlier detection and exploratory investigations—by tenant, region, endpoint, feature state, and so on. It reduces reliance on intuition and deep familiarity, and eliminates the need for tool hopping or cross-correlation. The unified model uniquely shines at connecting SLO violations back and forth with the requests they were derived from.

Given the overlap in outcomes, the formula for determining value is the same regardless of which model you use. Individual tool performance will vary, but the mandate is consistent.

## **The Operational Mandate**

To understand the value of operational loops, look at how operational instability affects your business. Signs you may be approaching a breaking point include:

- Customers discover and report critical production bugs before you do.
- Minor incidents take so long to detect and fix that they escalate into extended outages.
- Your backlog of investigations keeps growing because new problems pile up faster than you can triage them.
- Your teams spend more time on break/fix work than on building new features.
- Customer satisfaction is low because of repeated performance issues that the support teams can't verify, reproduce, or fix.
- New features are delayed by weeks or months because engineers are tangled up in figuring out how services are interacting.
- You hesitate to ship AI-generated diffs to production because nobody wrote them or really understands how they work.

This list is entirely about bugs, incidents, and performance issues. If these are your biggest constraints, your operational feedback loops need work.

The business case for improving operational loops hinges on four things:

#### *Higher incremental revenue*

Better uptime and performance directly translate to more revenue from improved code quality.

#### *Cost savings from faster incident response*

This results in faster mean time to detect (MTTD) and mean time to repair (MTTR), quicker query response, faster bottleneck identification, reduced on-call time, and fewer rollbacks all reduce labor costs.

#### *Cost savings from avoided incidents*

Finding problems before they become critical prevents outages from happening in the first place.

#### *Cost savings from lower turnover*

Better tooling reduces burnout, alert fatigue, and on-call misery, which means fewer engineers leave.

Other benefits may apply depending on implementation, but these four should be universal for any business that adopts the practices in this book. Rather than getting caught up in feature comparisons between vendors, focus on whether—and how much—they actually improve business impact in these areas.

Operational feedback loops are a vital safety net for almost every organization. If improving operational outcomes is your goal, then observability is your primary tool.

But because operational loops are essentially table stakes, let's turn our attention to the business case for developer learning loops, which is where the real leverage lies.

## The Business Case for Developer Learning Loops

Developer learning loops are upstream from operational loops, and have compounding effects downstream. If developers only look at production after something breaks, most of the learning is being lost.

Developer learning loops are about far more than just bugs and incidents. They are about understanding the intersection of code, infrastructure, and users, and all that this entails. Production is a very noisy place, and users are agents of chaos, so you need to be able to ask exploratory questions and follow the trail, wherever it goes. These investigations often start with open-ended questions, like “Am I building the right product?”, “How are my users using it?”, and “What happened when I shipped that change?”

Recall from [Chapter 24](#) that we said observability is the sense-making layer for complex sociotechnical systems. It's what closes open loops and turns slow, lossy feedback into tight, accelerating ones. The questions that matter for reducing uncertainty are crosscutting by nature: impact by cohort, outcomes by feature state, regressions by request path, behavior under real load.

Given that, two things should become clear: (1) tooling capabilities matter a lot here, and (2) this is a different kind of investment. With high-quality, granular

observability as your information channel, your business case is about applying sustained pressure to the single greatest point of leverage in any software engineering organization: the value creation loop.

Unified observability unlocks developer learning in ways that the three pillars cannot. We discussed both models in [Chapter 1](#) (and briefly in [Chapter 25](#)):

#### *The unified storage model*

This gives you speed, flexibility, explorability, and no practical limits on cardinality and context. Precision tools like this can track developer intent over time against a noisy baseline and validate precise changes in production. When teams can ask any question, explore any scenario with arbitrary specificity, and get reliable answers in seconds, learning becomes routine. Curiosity is rewarded and compounds across your teams.

#### *Three pillars observability*

This can answer some of these questions, too, albeit at a much higher cost—manual correlation, reliance on experts, queries that take minutes, and investigations that take hours. Learning loops work best when they are easy and fast. If validating a change is hard, teams revert to larger batches, slower releases, and more guesswork. When curiosity is punished, people stop asking questions.

The difference in outcomes is stark. The business case here hinges on how you quantify the impact of developer learning. Before we get into the mandate, let's look at prior research we can build on.

## Estimating the Value of Developer Learning

If “developer learning” feels squishy and hard to justify, recall the research of leading organizations like DORA, SPACE, DX,¹ GitHub, and others (see links in [Chapters 23](#) and [24](#)), that have been quantifying the measurable effects of developer productivity for years.

When thinking about the business case for developer learning, notice how many tools in your current budget already try to buy it indirectly: CI/CD, test infrastructure, feature flags, experimentation platforms, progressive delivery, internal developer platforms. The developer tools category is enormous because the economics are real—engineering time is expensive.

As the number of developer tools has grown, so has the theoretical ceiling for engineering velocity. Yet very few organizations have come close to those theoretical gains; most see only modest improvements. Why?

---

¹ If you are a DX user, each point on the “DX index” correlates to X hours of time saved per developer per year. It's something worth tracking, even if estimates like these will always be imprecise.

In our view, this is less mysterious than one might guess. Developer productivity is a system, and many tools succeed weakly (if at all) because they *don't close the loop*. They improve one part of the system while leaving the feedback step weak. In the absence of learning, all you have is activity. Some omnipresent examples include:

#### *Chaos engineering*

If you're injecting failures into your systems but can't identify precisely where and how things broke, chaos engineering is just chaos. The shift to learning requires clear attribution: what degraded, where, and why?

#### *Feature flags*

Flipping flags to expose features to subsets of users gives you control, but control without validation is a trust-fall activity—the control is illusory.

#### *Progressive delivery*

Rolling out features gradually reduces risk, but only when regressions can be precisely identified by cohort and slice. Otherwise, you have a slower deployment pipeline and the same uncertainty.

Many modern engineering practices only succeed when they form a closed loop: change → observe → act. Observability is what closes that loop and turns these practices into fast, specific, *actionable* learning.

A credible business case doesn't need to prove that developer time has value. It needs to show how observability reduces wasted cycles, accelerates outcomes, and aids in the value creation loop.

## The Developer Mandate Has Two Halves

The business case for investing in observability to accelerate learning from development loops has two halves to measure:

#### *External measures*

Outcomes experienced by customers

#### *Internal measures*

Outcomes experienced by engineers and engineering-adjacent roles

Where you invest internally should reflect your business model, your competitive position, and where your biggest opportunities lie. Let's take them one at a time.

### **External measures: Product differentiators and customer experience**

From an external perspective, observability plays a supporting role in every customer interaction. This includes dependability, but also the quality of the product and the experience of each user, as well as the ability to understand and respond to real users'

questions and behavior in real time (e.g., “Where did my package go?” or “Where did my deposit end up?”).

Start by listing out your core differentiators as a business—your mission, vision, and product roadmap. What are your unique value propositions? What’s in your critical path, what do you have an SLO on? How do you make money? Start with those.



At Honeycomb, for example, we have an internal target that query builder and boards queries should return in under 5 seconds, 85% of the time, and much more complex “deep dive” queries should return in less than 30 seconds, 85% of the time. We have similar targets for availability of data (99.95% of the time, submitted data will be queryable in under 60 seconds) and reliability of data (99.99% of data submitted will be persisted to disk). These things are very important to us; they differentiate us, and we are proud of them. We should invest in rich telemetry around these critical paths so we can understand them extremely well.

We are investing heavily in exploring agentic workflows and other AI-enabled interfaces, so we also invest in making sure those engineers are instrumenting their code as they go.

On the other hand, services that are not user facing, not in the critical path, or not one of our key SLO targets need a lot less precision and investment.

Turning this into monetary estimates is harder to give guidance on, but one go-to example is UX. At scale, small changes in latency and reliability can move business metrics in meaningful ways. A few public examples:

- Amazon reports that every 100 milliseconds of latency costs the company 1% in sales.²
- Walmart found a 2% increase in conversions for every second it shaved off of loading time.³
- Staples saw a 10% increase in conversions by reducing home page load time by one second.⁴

---

2 Steven van Vessum, “[Amazon study: Every 100ms in Added Page Load Time Cost 1% in Revenue](#)”, *Conductor*, June 14, 2023.

3 Cedric Moore, “[The revenue impact of every second while your website loads](#)”, *Trueform*, February 10, 2025.

4 Martyna Kozłowska, “[7 Ways Website Performance Impacts Your Business and Marketing Goals](#)”, *Reffine*.



The more strategic external value isn't just knowing whether systems are “up.” Real value is unlocked when you can see and understand what users are actually doing: what they're trying to accomplish, and where reality diverges from your assumptions.

Other terms pleasing to the ear of finance include gems like: “fully-loaded engineering cost,” “recovered capacity from less firefighting,” “faster time to market,” “reduced incident investigation labor,” and “avoided wasted build cycles.” These are all ways to express the practical consequence of tighter feedback loops; your finance team can help you attach numbers to them. Pair your estimates with clear baselines and conservative assumptions.

The decisions you make about how to instrument your code, or how to collect and analyze your telemetry, are not operational decisions. *These are product decisions.*

### The Cost of Not Paying Attention

What gets missed when developers only look at production for bugs and incidents?

Almost everything.

Instagram grew out of an app called Burbn, a location-based check-in app. The founders observed that users were gravitating toward the photo-sharing feature, so they pivoted and doubled down on it.

YouTube started off as a video dating site, but users seemed more interested in the videos than the dating. Twitter (now X) was originally a podcasting platform called Odeo. Slack emerged from a failed gaming company's internal chat tool.

It's remarkable how often this pattern pops up. Some of the most interesting features and creative uses come out of watching what users do and following their lead. But if you can't or don't look, you will miss it.

The external half of the dual mandate is about being able to learn from customers and evolve the product toward better outcomes—before your competitors do, and before internal narratives harden into the wrong roadmap.

### Internal measures: Velocity as a competitive advantage

Internal value is the part that leaders undervalue because it's harder to put on a slide. It doesn't always show up as a clean “+1% revenue” headline. But it's where the real leverage lives, because it determines whether your organization can actually execute and learn without drowning in its own complexity.

Instrumentation and enablement around internal developer feedback loops—CI/CD, deploys, test suites, agentic monitoring, specs, SLOs, developer sandboxes, visual tests, progressive delivery, feature flags, and so forth—is the difference between shipping hundreds of times a day and shipping once a week. It’s the difference between small and nimble teams that can self-serve their needs, and slow and lumbering process overhead. It’s the difference between engineering labor converting efficiently into outcomes or evaporating into rework, escalations, and task trackers.

And increasingly, it’s about which teams are going to survive the leap to AI-generated code. Teams that struggle to complete a migration or understand a user session in production are not equipped to validate nondeterministic agentic workflows in production, much less what it will take for their code to cross over (as discussed in [Chapter 2](#)).

The reason is simple: uncertainty is expensive. When engineers can’t see what their code is doing in production, every change carries more risk. Organizations respond with more coordination and risk mitigation. Uncertainty creates drag, and so do most of the ways people respond to it. It’s how capable teams slow down without even noticing.

You can’t hire your way out of a velocity problem caused by uncertainty. More bodies don’t help, they just increase the coordination tax.

The way to break a velocity stalemate is by accelerating your ability to validate each change in production. You need to build confidence and you need to build it fast. That is the best way to efficiently convert engineering labor into outcomes: with fewer regressions, less rework, and tighter learning loops.

The business case for developer learning can be assembled using internal velocity metrics, external experience metrics, or both. The investment returns on external-facing developer loops come from leaning into your product differentiators to increase revenue; the investment returns on internal-facing developer loops come from allowing each developer to accelerate and move with confidence.

In an era where AI is being used to accelerate the development process, your ability to capture those gains and convert them into business results will depend on your ability to validate each change and understand the impact it has on your customers.

Next, let’s look at how management needs to change in order to manage developer learning as an investment.

# Strategic Investment or Cost Center Optimization?

There's a reason many organizations habitually manage observability as IT or cost center technology: that's how the industry got here. Monitoring and logging were traditional operations tools, as discussed in [Chapter 3](#), and many of our tools and practices still reveal signs of their origins.

## Cost Centers Versus Investments

A cost center is a department or function within a company that incurs expenses but doesn't directly generate revenue. It's tracked as a cost to the business rather than a profit contributor. Classic examples include IT, human resources, legal, finance, facilities, ops, and infrastructure. They're necessary for the business to operate, but their value is indirect. They enable the revenue-generating parts of the org to function.

The term often carries a slightly loaded connotation: calling something a cost center is sometimes a way of signaling that it is lesser-than or shouldn't exist.

An investment, on the other hand, is something you spend money on now in order to get more value back from later. The key distinction is the expectation of return. An investment is supposed to compound.

The tricky part is that the return on many investments is delayed, indirect, or just hard to measure. If you can't draw a straight line from "we spent X on observability tooling" to "we made Y more dollars," the finance brain defaults to treating it as overhead.

Engineers often find ROI estimates endlessly exasperating, because they seem fuzzy and imprecise—they are, but that's not the point.

The point is: framing matters. Cost centers get cut; investments get funded. The time you spend crafting estimates is time well spent.

"Is observability a cost center or an investment?" is a trick question. It's actually both, or it can be, because *observability inherits the properties of the code* you are observing. Think about it:

- The observability you use for infrastructure and operational outcomes is a cost center, because infrastructure itself is a cost center. You can't make more money by spinning up extra pools of idle capacity, and you can't make money by ladling on more monitoring checks than you actually need. Infrastructure is about operating as efficiently as you can.

- Your revenue-generating application code is an investment; you might add new features, improve the recommendation engine, or make it faster and more interactive in order to boost sales and conversion rates. The instrumentation and telemetry you use are *also* an investment, if you are able to use better observability tools to improve the product or UX in ways that generate more revenue than the marginal costs of those tools.
- The code you use to deploy, test, validate in production, and so forth, is not revenue-generating code. However, engineering salaries are the largest single category of expenses for most technical product companies, and internal tooling and enablement can be a force multiplier on developer productivity as well as overall confidence and execution speed.

Observability and precision data are also dependencies for the coming AI revolution, but no one knows how to price that yet.

In practice, managing all observability like a cost center doesn't just reduce tool spend—it degrades the information channel the organization depends on to detect, diagnose, and improve its systems. This has many unfortunate consequences downstream.

This is not to say that there are no effective techniques for constraining costs, or that you shouldn't try to limit your spending. (There are, and you should.) But it would be short-sighted to manage all telemetry spend the same way. If you prevented developers from gathering high-cardinality data because it was too expensive, and as a result they don't have enough specificity to identify outliers or improve performance, you are only hurting yourselves.

## Observability as Organizational Investment

Observability is the sense-making apparatus for sociotechnical systems with constant flux, dynamic components, nondeterministic agents, and countless emergent behaviors.

It is the mechanism that connects cause and effect to create feedback loops. Observability is what builds the bridge that connects developer intent to user impact. And context-rich data and precision tooling are the primary safety mechanism for the emerging domain of automated coding techniques like harness engineering.

Yeah, it's kind of a big deal.

If observability has not historically been managed as an investment or a strategic priority, what might that look like from an organizational perspective? Here is what needs to change:

### *Reporting structure*

Infrastructure, operations, and IT excel at managing cost center technologies to a strict budget, which is why observability should not report to them. It should roll up to engineering leadership—your vice president of engineering, CTO, or sometimes chief product officer (CPO). Those roles typically have more experience managing technology as an investment, not just as overhead.

### *Team structure*

Observability teams should be staffed and run like platform engineering teams, not like infrastructure/ops. Follow the platform “Prime Directive”: build the components that are unique to the company, but shared across teams. Use design principles and user research, treating software engineers as their customers. Minimize cognitive overhead for developers. Make auto-instrumentation feel like magic. Make it easy to do the right thing and hard to do the wrong thing.⁵

Remember, the worst possible outcome isn’t picking the wrong tool; that’s recoverable. A worse outcome would be if you start pouring money, time, and engineering cycles into a solution when you don’t yet have alignment around questions like “What are we trying to solve?”, “What does success look like?”, and “What feedback loop are we trying to improve?”

## Conclusion

In this chapter, we showed how to build a business case around operational response (firefighting) or developer learning (reading the fire codes). Each loop has different success measures, and the business case should be built around those measures instead of specific tools or features.

We also identified the management shifts required to unlock funding and manage observability as an investment. Operational outcomes are cost-center optimizations and belong under IT/ops, but developer learning loops are a strategic investment and should live within engineering.

When you translate faster learning and validating into conservative estimates of recovered capacity, reduced waste, and reduced risk, the case becomes fundable and sustainable.

In the next chapter, we’ll cover some more practical diagnostics, as well as questions you can use to figure out if your investments are paying off or not.

---

⁵ Do not, under any circumstances, simply *call* yourselves a “platform engineering team” and go on to run and operate a giant mess of open source software. Any team that either (a) writes and maintains a sizable corpus of code or (b) operates a substantial production service is *not a platform team*; they are a software team or an operations team, respectively.



---

# Diagnosing Your Observability Investment

If you’ve felt growing pressure to “cut observability costs,” you’re not alone. Observability spend has climbed steadily for years. According to Gartner,¹ the cost of observability has been rising at least 40% per year for the past 15 years straight. Observability is now the second largest software line item in R&D organizations—second only to cloud—and most orgs spend somewhere between 10–25% of their infrastructure bill on observability tools.

In the last chapter, we talked about how to make the business case for observability. It’s rarely pleasant when leadership or finance starts asking for justifications. But cost pressure can be useful when it forces us to ask hard questions, like is our investment working?

That question often gets misread as “can we justify this line item?” It shouldn’t. The actual question is, “did we buy the right capabilities, and are we getting the right returns?” Many organizations are experiencing a disorienting mismatch—for example, paying for observability as if it were a strategic capability while operating it like a cost center and using it like monitoring. If you aren’t getting the returns that you feel you should based on what you’re paying, you might be right.

In this chapter, we’ll show you how to diagnose your observability expenditures. If you decide to invest in better observability, it may cost you. Or it may not: outrageously high bills are the most common consequence of poor tool fit, so you *might* actually pay far less to get better results. Either way, you should be able to point to real outcomes: tighter feedback loops, richer understanding, faster learning.

---

¹ This information was collected from the Gartner webinar, “I&O Leaders, Get Control of Your Observability Costs”, recorded in early 2025.

Before you start slashing costs, start with a proper diagnosis: is your observability delivering on the capabilities you need it to?

## Don't Pay Observability Prices for Monitoring Outcomes

In the last chapter, we identified two different feedback loops to focus on, depending on your organization's biggest bottleneck. These loops require different investments. That doesn't *necessarily* mean they require different tools or products,² but they do need to be considered and evaluated as different problems:

### *Operational loops*

Protect system stability: detect, respond, recover, prevent recurrence.

### *Developer learning loops*

Accelerate learning and validation of code shipped: deploy, observe, learn, adjust.

These loops are complementary, and most organizations need both. But they represent different priorities and require different capabilities. A common failure mode is conflating them: buying tools and building practices that mainly support operational workflows while expecting the learning gains that only come from developer learning loops. In [Chapter 23](#), we called this “observability prices for monitoring outcomes.”

If your focus is operational outcomes, the right governance model is a cost center. Cost centers optimize for reduced spend, predictable overhead, and coverage. These budgets get squeezed when money is tight. Unfortunately, if operational stability is your biggest priority, those cuts can be genuinely harmful.

## The Firefighting Trap

There's a trap that teams fall into when they overprioritize operational loops. Teams that pride themselves on their firefighting and incident management often can't explain why fires keep starting. Incident response expertise is valuable, but operational feedback loops are the longest, lossiest, and most expensive kind in terms of downtime, customer impact, and lost revenue (see [Chapter 25](#)).

Alerts and escalations *cannot* be the only way you learn about your systems. Operations should be your goalie, the last line of defense when everything else fails. If your goalie is making 50 saves a game, your defense is *not working*.³

---

² Given unlimited time, money, and engineering creativity, all data tools are effectively fungible. You *can* debug production using Excel spreadsheets if you must, and some engineers have in fact done so. The question is not “is it technically possible?” but “is this the best use of your time and resources?”

³ The metaphor keeps going. If your goalie is the only one who is trying to score, you're *really* screwed.



Spending cuts hurt observability because they usually mean ingesting less data. That creates blind spots, which directly lead to higher operational risk, more downtime, and higher long-term costs (a counterintuitive result). Cost pressure also tends to produce tool sprawl, degraded data quality, and a slower ability to recover from and learn from incidents.



A useful diagnostic question here is: what percentage of your observability investment goes toward fighting fires versus preventing them?

## An Investment Posture Mismatch

With developer learning loops, the right governance model is strategic investment. Strategic investments optimize for capability growth, learning speed, and competitive advantage. They're evaluated by what they produce, and they're much easier to defend when money is tight.



A useful diagnostic question for strategic investments: does it produce organizational capabilities that affect business metrics?

It's worth reiterating that neither the cost center nor the strategic investment governance models are wrong. But you can't get faster development cycles or learning speed outcomes with cost center governance. This mismatch is what creates the frustration that leadership and finance are expressing when they pressure you. We'll spend the rest of this chapter examining what to ask in order to diagnose whether your observability investment is working.

## Activities Versus Learning

When engineering teams make strategic investments, it's common to take on a set of new activities meant to change outcomes. But there's a meaningful difference between activities and learning. Activities are the specific tasks, methods, and resources your teams use day to day. Learning is the cognitive change, the skill acquisition, and the actual knowledge gained from those experiences. Activities are the means; learning is the outcome.

When feedback loops are long, laggy, and lossy, you lose the mechanisms that enable learning. Many “productivity” investments turn into activities and reliability theater instead of genuine learning:

- Chaos engineering without attribution becomes disruption.
- Feature flags without validation become complexity.
- Progressive delivery without detection becomes a slower deployment pipeline.

In each case, observability closes the loop by providing attribution, validation, and detection. That closing is what turns activities into learning. You don’t capture the value of your productivity investments until your organization actually *learns something*.



Useful diagnostic questions here: For each practice you’ve invested in, can you point to the feedback mechanism that turns it into learning? What percentage of the time is it being used?

## How to Know If Your Investment Is Working (or Not)

The dual mandate from the last chapter—internal measures for productivity, external measures for core differentiators—is what you used to make the business case for observability. How it gets implemented will vary by organization. If you don’t have much experience translating technical outcomes into business needs, that can be a real challenge. However, understanding if your investments are paying off can be relatively simple.

For operational loops, your investment is working if:

- Time to identify known causes is decreasing.
- Alert quality is improving (fewer false positives, more actionable ones).
- Incident escalation rates are dropping.
- Recovery is getting faster.

For developer learning loops, your investment is working if:

- Engineers can answer novel questions about production.
- Deployment frequency is going up while batch sizes are coming down.
- More engineers can debug independently, not just the same few people.

- Time from hypothesis to validation is shrinking.
- The number of issues found by engineers is going up, while the number of issues that customers find and escalate via support is decreasing.

If your observability spend has gone up but none of these indicators have improved, or the set of indicators you were targeting have not improved, then the investment isn't working.

High spend with few returns over a long period is usually what triggers questions from finance. If spend is being questioned and you can't point to capabilities or concrete improvements, the uncomfortable answer might be: the investment didn't deliver.

When an observability investment doesn't deliver, it usually isn't a communication problem. It's evidence that something needs to change—tooling, instrumentation, organizational approach, or all three.

## Advice for Observability Investments

Beyond the diagnosis of your current observability investment, the following advice provides a set of targeted, practical steps you can take to proactively manage it as an investment instead of a cost center:

*Identify the point of diminishing returns on each investment.*

Yes, track and manage the cost of your telemetry. But also ask, “Where can we make targeted investments in observability that will unlock disproportionate business value?” This means understanding your specific differentiators, competitive position, and strategic priorities. For every targeted investment you make, figure out where the point of diminishing returns sets in and cap it there.

*Quantify your gaps in visibility.*

If you have experienced downtime or outages as a direct consequence of your weak observability or inability to connect cause and effect, consider preparing an impact statement. An impact statement is a comprehensive assessment that goes beyond immediate revenue loss to quantify the full financial, operational, and reputational damage caused by a system interruption. These are designed to expose “hidden” costs, such as employee downtime, SLA penalties, customer churn, and so on.

*Implement telemetry pipelines with tiered service.*

This is the only cost-effective way to scale a heterogeneous environment, where not all services are equally important. Consider defining at least two tiers of service, “low” and “high” (most companies end up adding a third which is somewhere in the middle):

1. Services in your critical path might be defined as ones that are revenue generating, customer facing, or under active development. They get wide, rich, distributed traces or canonical log data at a high tail-based sample rate—that’s the “high” level of observability.
2. Services that are stable, decoupled, and not customer-facing might just get end-to-end health checks and a synthetic or two—that’s the “low” level. Control the verbosity and telemetry level with feature flags, so you don’t have to redeploy your instrumentation just to change which signals are emitted.

### *Invest in attribution.*

We previously referred to this as “*observability for your observability*.” Cost is a vital component of architecture, and so are tokens. One of the most wonderful aspects of the modern, unified telemetry model is that you can pack hundreds and hundreds of attributes into every single trace or event. Tag every request, session, or transaction with the cost of running it, which line of code it came from, which normalized queries it executed, and so on.

There are many, many creative uses for attributes and attributions, but one of our favorites is assigning a budget to each team and letting them track and manage their own observability budgets. As we said in [Chapter 24](#), good observability enables self-awareness in humans and self-governance in teams. This is a good example of how that works.

## Conclusion

If you’re hearing “we need to cut observability spend,” that’s a signal that’s less about cost and more about outcomes. Cost pressure is often the moment that organizations are forced to ask the question they should have been asking all along: is the investment working? Remember, you don’t have to justify a line item: you need to demonstrate that capabilities are materializing for your organization. Are those dollars converting into tighter feedback loops, faster understanding, and faster learning?

Cost anxiety most often arises from a mismatch in expectations—paying for observability as if it were a strategic capability, while operating it like a cost center and using it like monitoring. You must be clear on your organizational priorities. Are you building operational loops that protect stability, or developer learning loops that accelerate learning and validate change in production?

You don't have to guess whether your investment is paying off. If operational investment is working, you should see faster identification of known causes, improving alert quality, fewer escalations, and faster recovery. If developer learning investment is working, you should see things like:

- Engineers answering novel questions about production
- Higher deploy frequency with smaller batch sizes
- Broader debugging capability beyond a small set of experts
- More issues detected internally before customers complain

In the next chapter, we'll look at how to build a coalition, defang the resistance, secure a mandate, and drive organizational change through rocky waters.



# The Organizational Shift

*This chapter is contributed by Rick Clark,  
Global Head of Cloud Advisory at UST*

## A Note from Charity, Liz, George, and Austin

In the previous chapter, we looked at strategies for diagnosing your observability investment.

If finance is pressuring you to justify your spending on observability, that's not a bad thing. Constraints are healthy. If your observability spend is a good investment, you should be able to see results. If it's not a good investment, you want to know this so you can make a change.

The rest of this chapter is written from Rick Clark's perspective.

You've invested in observability. Dashboards are everywhere. All three pillars, checked: metrics, logs, traces. You've paid your vendors and staffed your teams. But when you look at your organization, the reality doesn't match the promise.

War rooms still happen. Correlation is still manual. Engineers ship code and have no idea if it works. You never deploy on Fridays. When something breaks, the same three people get pulled in because nobody else can figure out the tools. Customers discover bugs before you do. Mysteries persist so long that they become folklore.

You spent millions. You hired and trained all the people. Your dashboards proudly display green. Why?

Here is the uncomfortable truth: you have legacy observability masquerading as modern observability. You bought "observability" tools. You renamed teams from "monitoring" to "observability." But the outcomes have not changed. You still cannot

trace a single user's experience, debug problems you have never seen before, or answer a novel question from leadership.

The label does not change the capability.

This chapter is about recognizing that gap and building a path to close it. It demands a strategic dismantling of the status quo. In the sections that follow, I'll define the tests to separate genuine observability from legacy monitoring that's masquerading as modern tooling. I'll then map out the human landscape, exploring how to neutralize the resistance response, recruit a coalition of allies, and secure the executive mandate for change. Finally, I'll provide a tactical roadmap for building a dedicated team to establish a permanent organizational capability that can withstand the complexity of the AI-augmented future.

## Recognizing Legacy Masquerading as Modern Observability

Before organizational change can happen, you need to see clearly where you actually stand. Not where the vendor says you stand. Not where your dashboards say you stand. Where *the evidence* says you stand. You need a clear diagnosis: do you have legacy monitoring masquerading as modern observability, or do you have the real thing? Not feelings or suspicions. Evidence.

In this section, I'll take you through five tests that can help reveal where you actually stand. Not feelings or suspicions. Evidence.

### The Ownership Test

Ask any engineer, "After you merge code, how do you know whether it is working in production?" The responses reveal one of two realities:

#### *Legacy observability*

Code goes into a black hole. You have no idea when it deployed. You have no visibility into whether it is working. If it was terrible, someone might get paged hours or days later. A customer might discover the bug months from now. The news may eventually make its way back to you. Or not.

#### *Modern observability*

Code goes live within minutes. You get a notification with a link showing your build being canaried out. You watch key metrics in real time. If something regresses, you see it immediately. If everything looks good, you watch your change roll out with confidence.



The real question this test asks: how much energy, prior knowledge, and creativity do you demand of every engineer just to understand the code they shipped? In organizations with legacy observability, understanding your own changes requires heroic effort. In organizations with strong observability, it is effortless.

*If the feedback loop does not exist, it does not matter how much software you build or buy. You are pouring water into a leaky bucket.*

## The Two-or-Three People Test

When an engineer needs to answer a question about production, do they pull up their tools, or do they turn and ask one of the two or three people who know how to use the tools? The responses reveal one of two realities:

### *Legacy observability*

Only a handful of senior engineers can effectively investigate production problems. Everyone else is helpless. The tools are so complex, limited, or poorly designed that only specialists with years of accumulated knowledge can use them. When those specialists leave, the knowledge leaves with them.

### *Modern observability*

Sixty to eighty percent of engineers can independently investigate issues. The tools make investigation accessible. Knowledge is encoded in the system rather than in people's heads.

In large enterprises, this test reveals something worse than a skills gap. It reveals a structural vulnerability. Those two or three people are single points of failure across hundreds of engineers and thousands of services. They get pulled into every incident. They become bottlenecks for the entire organization. Then, they burn out.

*If your observability requires a firefighter, you do not have observability. You have a dependency on humans who will eventually leave.*

## The Mystery Test

Ask engineers directly: “What’s something weird about production that we just accept?” The responses reveal one of two realities:

### *Legacy observability*

Engineers accept shallow mysteries that persist for months. “We have a load spike around midnight and do not know where it comes from.” “That job just fails sometimes.” “Latency is higher on Tuesdays for some reason.” After spending two days investigating without resolution, they learned to stop investigating odd patterns. The mysteries become folklore. Nobody bothers asking why anymore.

### *Modern observability*

Mysteries become specific and investigable. Engineers will say, “We have a load spike,” which becomes “The batch analytics job triggers 40,000 database queries because the query plan changed.” Mysteries get solved, not accepted.

You can surface this question in a team meeting, during an incident retrospective, or in one-on-ones. You could also search your chat history for phrases like “just does that sometimes” or “we never figured out.” Look at recent postmortems and count how many of them list the root cause as unknown or indeterminate. Check your runbooks for procedures that amount to “restart and hope.”

The mystery test is particularly revealing in enterprises with long histories. Accumulated folklore is a symptom of accumulated failure. Every mystery your organization has stopped investigating is evidence that your observability could not answer a question that mattered.

*Mysteries that persist are not mysteries. They are the shape of your blindness.*

## **The Arbitrary Question Test**

Give an engineer a question they have never been asked before. Something that requires slicing data by dimensions you did not preconfigure. For example, “Show me all failed checkout attempts from mobile users in California using version 2.3.1 of the app during lunch hour over the past week.” The responses reveal one of two realities:

### *Legacy observability*

They need to check if those dimensions were indexed. Probably not. They wait for new indexes to build. Hours or days. They resort to manual log analysis. Painfully slow. They give up and say, “We don’t track that combination.”

### *Modern observability*

They answer in under 60 seconds. The data exists. The query runs. The answer appears. No one had to anticipate this question for it to be answerable.

This test separates monitoring from observability. Monitoring answers questions you predicted. Observability answers questions you did not. In a world where novel problems are the norm, the ability to ask novel questions is not optional.

*If you can only investigate what you predicted, you cannot debug the unexpected.*

## **The Deployment Confidence Test**

Ask engineering leadership: “How much fear is there around deployments?” The responses reveal one of two realities:

### *Legacy observability*

Deployments happen weekly or less. They require war rooms. Engineers are genuinely afraid of breaking things. “Read-only Fridays” ban deployments before weekends. Junior engineers are not allowed to deploy. Change Advisory Boards exist to slow things down because slowing down is the only safety mechanism anyone trusts.

### *Modern observability*

Deployments happen multiple times daily. They are automated and routine. Engineers deploy confidently because they can see the impact of changes. Problems are detected quickly. Rollbacks are fast. Fear is replaced by evidence.

Fear is a signal. This test tells you that engineers do not trust their ability to see what their changes will do. They are not afraid of deploying. They are afraid of not knowing.

*If your engineers are afraid to ship, they are telling you they are operating in the dark. Listen to them.*

## **After the Tests**

The results of these five tests are not a scorecard. When an engineer cannot tell you whether their code is working, when the same three people get pulled into every incident, when midnight load spikes become folklore, you’re seeing evidence. You’re seeing failures that can be named.

Closing those gaps is not about buying more tools or hiring more people. It is about changing how your organization thinks about production, who owns what, and what “good” actually looks like. That is organizational change.

You *will* face resistance. Vendors will fight to keep their contracts. Existing teams will fight to keep their identities. Leadership will fight to avoid admitting that millions were spent on the wrong things. The test results are your answer to all of it. Evidence that cannot be dismissed as opinion. Language for a fight that is already underway.

## **Understanding the Resistance**

The pushback you will face is not born of obstinacy. People resist because they have real incentives to protect, deep professional identities at stake, and legitimate fears about what change means for them. Any significant shift in how an organization operates triggers an automatic response designed to protect the status quo.

The resistance has two dimensions. First, there is the organization itself (a system that reflexively protects its current state regardless of who is pushing for change), and second, there are the specific groups whose incentives make them natural sources of friction. Understanding both is the first step to navigating either.

## The Organizational Immune Response

Committees will form to “evaluate options” and slow-walk decisions until the initiative loses momentum. Pilot programs will be quietly starved of resources, and then declared failures. Success criteria will be redefined so the current state qualifies. Someone will produce a report showing that everything is fine, actually.

People aren’t being difficult for the sake of being difficult; it’s the organization’s subconscious attempt to protect its current state.

Call attention to this dynamic, this antibody response, when you see it. “I notice we’re forming a committee to evaluate something we already know doesn’t work” is a sentence you may need to say out loud. The immune response defeats initiatives silently. By naming it, you transform a silent, automatic response into a visible, addressable problem. Naming it is how you fight back.

The immune response takes different forms depending on where it originates. Knowing where it comes from makes it possible to address it.

## Why Legacy Vendors Fight

Your legacy observability vendor is a business responding to the incentives of their own market. Their contracts renew automatically. Their sales teams are compensated for selling dashboards, storage tiers, and seat licenses, not for whether you can actually debug production. Their product roadmap was built around the legacy three pillars model because that is what the market bought for 15 years.

To navigate this resistance, you must recognize their “hard sells.” They might offer a new AI feature as a magic bullet to fix the gaps in their underlying model. They might suggest that your lack of insight is a tier problem, not an architectural one (and you need their premium tier). They might suggest that you need professional services to “optimize your implementation.” They say the tools are fine; you are just using them wrong.

They are not evil. (They are not even wrong.) Their incentives are simply misaligned with your goals. While they make money when you buy more, you succeed when you understand more. Those are not the same thing, and sometimes they are opposites.

## Why Existing Teams Resist

The people who built and maintain your current observability infrastructure have rational reasons to be skeptical of change.

Their identities are wrapped up in current tools. “I’m the Splunk person” is not just a job description. It is years of accumulated expertise, certifications, conference talks, and professional reputation. Asking someone to abandon that in favor of a brand-new tool is asking them to start over. It can feel like an existential threat.

This is *vendor-silo alignment*: when someone's professional identity becomes inseparable from a specific tool. It is not a character flaw. It is a rational response to how organizations reward and recognize people. Breaking it requires offering something better, not just demanding they abandon what they know.

Their headcount is tied to maintaining current systems. If those systems go away, what happens to them? “Unified observability” sounds a lot like “fewer jobs” if you are not careful about how you talk about it.

Existing teams have seen this before. The last modernization initiative. The one before that. Grand plans that stalled, got defunded, or declared victory without changing anything. Why should this time be different?

And here is the thing: they are often right to be skeptical. Most tool consolidation efforts fail. Most transformation initiatives die quietly in committee. Their skepticism is earned through experience.

Your job is not to overcome their resistance. It is to show them something different. Acknowledge that the odds are against you: most of these efforts have failed before, and they know it. Be honest that their jobs will change. Then be specific about what that means: their tool expertise does not become worthless, it becomes portable. Their headcount does not disappear, it shifts toward work that does not require heroics to sustain. The current effort is different not because you say so, but because you can show them evidence they helped gather.

## Why Leadership Is Skeptical

Executives remember writing large checks for the existing observability stack. “We already invested millions in this stack” is not an excuse for not replacing it. It is a sunk cost that feels very real when someone proposes spending more money to replace what was supposed to solve this problem. For executives, replacing the current stack may feel like admitting a multimillion-dollar mistake.

A vendor might suggest, “You just need to upgrade to the enterprise tier.” Executives can see the option as being cheaper and easier than admitting the whole approach was wrong. They also remember the last consolidation attempt. The one that was supposed to rationalize the tool sprawl and ended up adding three more tools to the pile.

They are not wrong to be cautious. Your job is to show them that modern observability is about capability—the ability to understand production—not about consolidation or cost savings. Those might come later, but they are not why you are advocating for change. Connect observability to the outcomes they are already accountable for. Find the business results that are bleeding:

- Revenue lost to outages in critical transaction flows
- Customer churn tied to reliability incidents
- Senior engineer salaries consumed by war rooms instead of product development
- Bugs discovered by customers instead of caught internally
- Release cycles slowed by deployment fear instead of accelerated by confidence

When you can show that the current state is costing them something they're already measured on, the conversation stops being about tools and starts being about risks they can't afford to ignore.

## Why You Cannot Fight It Alone

Since the immune response operates on an organization-wide scale, you cannot overcome it alone. You must transform individual frustration into collective power by recruiting others who recognize that the status quo is no longer sustainable.

## Building the Coalition

To bypass the collective resistance to organizational change, you need allies, not supporters in principle, but people who feel the failure personally and have the standing to do something about it. The difference matters. Passive agreement doesn't survive the first difficult meeting. What you need are people with skin in the game, credibility in the organization, and the willingness to spend political capital when it counts. So, where do you find them?

## Find the People Whose Pain Is Immediate

Look for engineers who dread being on-call, not because they dislike responsibility, but because they know that only two people can actually debug anything. When they get paged, they are not responders; they are dispatchers, escalating to the same two seniors every time.

Look for SREs who spend hours in war rooms doing correlation by hand: "Check the database dashboard. Now check the API dashboard. Now check the customer service tickets. Does anyone see a pattern?"

Look for security teams that cannot trace access patterns across services, platform teams that cannot answer "What broke after that deploy?", customer support that cannot explain to users what happened, and engineering managers whose teams refuse to ship on Fridays.

These people already know something is wrong. They live it. You are giving them language and a path forward.

## Honor the Heroes Who Held It Together

Before you successfully transition to modern observability, someone is keeping production running. Probably several someones. They wrote the scripts that still execute at 3 a.m. They know which dashboard lies and which one tells the truth. They remember why that one service restarts every Tuesday and what happens if you let it skip.

These people are not obstacles. They are organizational memory. The knowledge in their heads is not written anywhere, and you cannot buy it from a vendor.

So it's imperative that you include them in your transformation plans early. Not as an afterthought once you have your mandate, but during coalition building. Ask for their perspective. Learn what they have tried before and why it failed. Understand the constraints they navigate that nobody documented.

I learned this the hard way. During a cloud modernization effort, I failed to include the principal network architect in early planning. In my head, he was a dinosaur attached to legacy. The champion of the old way. He represented everything I was trying to change, so I worked around him.

This was a mistake.

He controlled IP address allocation. When the work needed network resources, requests that should have taken hours took weeks. Approvals stalled. Questions multiplied. Everything ground to a halt.

I had to go back, acknowledge my error, and give him the respect I should have offered from the start. Once he was genuinely included, once his expertise was valued rather than bypassed, the obstacles disappeared. He became an asset.

It's easy to cast the keepers of legacy systems as opponents. They slow things down. They ask hard questions. They keep raising failures you've never heard of. But that framing says more about your assumptions than their capabilities.

The heroes who held things together have standing in the organization. They have relationships, institutional credibility, and control over resources you may not even know you need. If you exclude them, they can slow your work to a crawl without ever saying "no" directly. If you include them, their credibility becomes yours.

This is not manipulation. It is recognition. They earned their standing by solving hard problems with inadequate tools. Honor that. The transformation you are planning does not erase what they built. It builds on it.

## What Makes a Good Ally

Pain is necessary but not sufficient. A good ally feels the pain personally, not abstractly. They are in the war rooms, not reading reports about them. They have credibility in the organization, which means people listen when they speak. They are willing to spend political capital, which is rarer than you think. And they will stay engaged through setbacks, because there *will* be setbacks.

One committed ally beats 10 passive supporters. You need people who will show up to the difficult meetings and say the difficult things.

## Building the Case Together

Once assembled, your coalition has one job: build the case that cannot be dismissed. This is evidence-gathering, not consensus-building. You are not trying to get everyone to agree.

Run the five tests together. Document the results. When the ownership test reveals that engineers have no idea if their code works, write that down. When the two-or-three people test shows that the same three seniors get pulled into every incident, write that down.

Collect stories: the incident that took four hours because only two people could query the logs, the customer escalation where support had no way to see what happened, the deployment that went wrong and nobody knew for three days.

Measure the burden: how many on-call escalations per week, how many hours spent in war rooms per month, what percentage of bugs are discovered by customers versus engineering.

The point is to collect real stories that have *meaning* in your organization. When someone says, “I don’t think we have a problem,” the evidence says otherwise, and that is what converts your vision for change into the leverage needed to secure a formal mandate.

## Securing the Mandate

Allies and evidence are necessary, but they are not enough to successfully change an organization. You need authority—the organizational power to make the new way mandatory, not optional. You need a mandate.

A mandate means that new services get instrumented in the new way. This is not optional. New teams onboard to new standards; they aren’t grandfathered into legacy approaches. Legacy systems migrate on a published timeline, not “someday when we have time.”



This makes the change a policy, not a suggestion. Without it, you are just adding another tool to the pile. Without it, teams will naturally default to whatever's easiest, which is usually the legacy approach they already know. You need an organizational forcing function to change that.

## Sponsorship Versus Authority

There is a difference between sponsorship and authority. Sponsorship means an executive says “I support this initiative” in meetings. Authority means that an executive can say “New services must use this approach,” and make it stick.

Sponsorship without authority is theater. You will get polite attention in steering committees while nothing actually changes. Teams will nod along and then do whatever is easiest, which is whatever they were already doing.

You need someone who can make the new way mandatory for new work. That is the minimum viable mandate.

## Finding the Right Executive

The right sponsor is not necessarily the CTO. It is often the person who owns the pain most directly.

It might be a VP of engineering who is tired of explaining incident costs to the board. It might be the head of SRE who owns MTTR and on-call burden metrics that are not improving despite years of investment. It might be a business leader who keeps asking “what happened?” and getting unsatisfying answers.

Follow the pain upstream until you find someone with budget authority and organizational patience. This is a multiyear journey; you need someone who will still care in year two.

## The Pitch That Works

Securing a mandate requires more than a list of technical requirements; it requires a narrative that resonates with your leadership's strategic goals. When you walk into the room, you are not pitching a tool; you are pitching risk reduction, capability building, and operational velocity:

*As risk reduction*

“We cannot reliably debug production problems, and that risk is growing.”

*As capability building*

“Only three people in the organization can effectively investigate incidents. That is a single point of failure across hundreds of engineers.”

*As burden reduction*

“Our best engineers spend 30% of their time in war rooms. That is expensive talent fighting fires instead of improving our products.”

Do not frame it as tool consolidation, even if consolidation is a benefit. Consolidation efforts have a bad reputation because most of them fail. This is not about consolidation, but about capability building.

Do not lead with cost savings, even if savings will come. Cost-driven initiatives get cut when budgets tighten. Capability-focused work survives because it addresses problems that do not go away.

If leadership fears writing another check that leads nowhere, open standards are the answer. Vendor-neutral instrumentation means the work your teams do today remains valuable regardless of which backend you choose tomorrow.

If the executive cares about connecting system behavior to business outcomes, note that modern observability makes that possible. Legacy observability does not. You cannot measure what you cannot see.

To succeed, your pitch must use the evidence you’ve gathered to prove that the current state is no longer an acceptable business risk.

## The AI Forcing Function

Here is something leadership may not see yet: the transition to AI-assisted development will break legacy observability entirely.

AI coding assistants generate code faster than humans can instrument it. AI-driven systems make decisions that are difficult to trace with traditional logging. The volume, velocity, and complexity of AI-augmented software development will overwhelm approaches built for human-speed workflows.

Organizations that lack real observability, the ability to ask novel questions about production, will find AI acceleration makes them more fragile, not more productive. They will ship faster and understand less. That is not a competitive advantage; it is a liability.

This is not a pitch for AI observability tools. It is a warning that the pressure on your current approach is about to increase dramatically. The sooner you build genuine observability capability, the better positioned you are for what is coming.

Securing the mandate is the final piece that transforms your technical vision for change into an institutional priority. You are now in a phase where progress is measured by adoption rather than permission. Next, you need a plan to make the new way a reality.

# The Roadmap

You have the diagnosis, the coalition, and now the mandate. The mandate creates the organizational authority that makes the new way required. But a mandate alone is not enough. Organizations that rely only on enforcement get compliance at best and sabotage at worst. Organizations that pair authority with a product mindset get genuine adoption. Your goal is enthusiastic adoption. Engineers instrument thoroughly because you treated them like customers and listened to their needs, not because they were mandated to.

You need a roadmap that provides the strategy for success and doesn't collapse under its own weight. Approach implementation with a product mindset, focusing on depth over breadth and enthusiasm over compliance. Use the roadmap to build momentum incrementally by proving the value of observability.

## Start Small: One Domain, Full Depth

Pick one business domain with a willing team. Not the most critical system, because you do not want that pressure. Not the most neglected system, because you do not want to fight technical debt while proving a concept. Pick something in the middle: important enough to matter, functional enough to instrument cleanly, with a team that wants to try something new.

Instrument it completely. Wide events with full context. Request identifiers that flow across every service boundary. Enough cardinality to ask arbitrary questions.

Do not do shallow instrumentation everywhere. Instead, do deep instrumentation somewhere. Your proof point needs to be undeniable, not broad.

For success criteria, pay attention to whether questions that previously required a war room can now be answered with a query. If you can demonstrate that, you have something real. Protect this team and this proof point. It's your ammunition for expansion.

## Pave the Path: Make the New Way Easier

Adoption that requires heroics will fail. If instrumenting correctly takes a week of work per service, engineers will not do it. If the libraries are confusing, they will use them wrong or not at all.

Build instrumentation libraries that developers actually want to use: golden paths where a few lines of code give you context that flows automatically across service boundaries, and defaults that are correct for 90% of cases.

The goal is to make the new way the path of least resistance. When the right thing is also the easy thing, adoption happens. When the right thing requires extra effort, it does not.

## Platform Engineering Principles

Platform engineering thinking is critical to success here. Without it, observability teams become order-takers or, worse, enforcers. Both fail.

The core principle: you have customers, and you must serve them. Developers. SREs who respond to incidents at 3 a.m. The security team tracing access patterns. Customer support trying to understand what a user experienced. Leadership connecting system health to business outcomes.

Each customer has different needs. Developers need fast feedback on their changes. SREs need quick paths to root cause. Security needs audit trails. Support needs customer context. Leadership needs to understand how system behavior affects revenue, retention, and reputation. Your instrumentation and tooling must serve all of them, not just the loudest or most technical.

This means your observability team operates like a product team. You research what your customers actually need, not what you assume they need. You build for their workflows, not dictate new ones. You gather feedback continuously and treat complaints as data, not ingratitude. You measure success by adoption and satisfaction, not by features shipped or standards published.

When any of these customers struggle with your tools, that is not resistance. That is a product failure. When teams find workarounds, that is feedback. The market is telling you something.

One last principle: build as little custom software as possible. Your competitive advantage is not a better tracing library. It is making observability work for your organization. Use open standards and open source tools, like OpenTelemetry, where you can. Buy commodity capabilities. Build only what's actually necessary, does not exist, or is a unique requirement of your business. Focus your energy on the integration, the developer experience, and the organizational change. That is where you add value.

## Demonstrate Wins: Rerun the Tests

Remember those tests that reveal whether you have real observability or just monitoring with a new label? Run them again after six months into the transition:

### *The ownership test*

Do engineers now get feedback on their changes, or does code still disappear into a black hole?

### *The two-or-three people test*

Has the fire brigade expanded, or is it still the same three people?

### *The mystery test*

Are old mysteries getting solved, or just accumulating?

### *The arbitrary question test*

Can more engineers answer novel questions without escalating?

### *The deployment confidence test*

Are engineers less afraid to ship?

These are the metrics that matter. Track them visibly. When they improve, make sure leadership sees the improvement.

You can also track business-adjacent metrics if they apply: time to resolve customer-reported issues, hours spent in war rooms per incident, percentage of bugs discovered internally versus by customers. These are things organizations already measure. Connecting observability improvement to existing metrics makes the value concrete.

Celebrate milestones. Do not declare victory. This is a long journey, and premature celebration invites complacency.

## **Expand Deliberately**

With the mandate in place, new initiatives have modern instrumentation built in. Legacy migrates domain by domain, not as a big bang.

Stop the bleeding before you heal the wounds. Every new service instrumented correctly is one fewer service you need to migrate later. Every team onboarded to new standards is one fewer team resisting change.

Set expectations clearly. This is a multiyear journey, so treat it as an operational capability, not a project. Projects end; capabilities evolve. Organizations that try to transform everything at once usually transform nothing. But organizations that commit to sustained, incremental progress actually get somewhere.

If you can influence budget, structure the transition as an incremental replacement.¹ Each year, a planned percentage shifts from legacy to modern. Year one might be 10%. Year three might be 40%. This creates momentum that compounds and makes the transition inevitable rather than aspirational.

Use your early wins to evangelize. Engineers who have benefited from modern observability become your best advocates. Let them tell their stories. Their credibility with peers exceeds yours.

---

¹ This is sometimes referred to as the “strangler pattern,” a violent but visceral (and memorable) turn of phrase.

You will know you are done when the war rooms stop, when the mysteries get solved instead of named, or when engineers ship on Fridays without a second thought. The outcome is not about “better tools.” It is about comprehensively healthier sociotechnical systems that can execute swiftly with confidence.

## Building the Team

The coalition got you the mandate. Now you need a team to make it real. Transformation requires people. Not just executive sponsors and coalition allies, but a team dedicated to making observability work. Their role is not to do observability for the organization. It is to make observability possible for everyone in it.

## What the Observability Team Owns

The observability team does not exist to build dashboards for other people; they exist to build the platform that makes manual dashboarding obsolete. They own the platform. They build and maintain instrumentation libraries and SDKs. They define standards for what good telemetry looks like. They manage vendor relationships and tool selection. They own the developer experience for observability.

What they do not own is instrumenting every service themselves. That does not scale. It is the quickest way to become a bottleneck. Their job is to make instrumentation easy enough that product teams can do it correctly without heroic effort.

## Capabilities of the Observability Team

Effective observability teams require a mix of skills that span the full lifecycle of code, from commit to production incident. This is not a comprehensive list, and every company might implement this differently:

### *Pipeline integration knowledge*

Someone needs to understand CI/CD systems, deployment infrastructure, and where telemetry gets injected automatically. This is the plumbing work that makes instrumentation happen without requiring heroics from every product team.

### *Production expertise*

Someone needs to have felt the pain of debugging at 3 a.m. They understand which information actually helps during incidents, which queries answer real questions, and which failure modes matter. Without this perspective, teams build instrumentation that looks complete but misses what operators actually need.

### *Developer experience*

Someone needs to care deeply about how engineers interact with observability tools and libraries. Confusing SDKs get misused. Bad defaults propagate everywhere. This is product work applied to internal tooling.

### *Customer understanding*

Someone needs to stay close to every audience the platform serves. Developers, SREs, security, support, and leadership all have distinct needs and workflows. The team that builds only for engineers will leave half its stakeholders underserved.

### *Standards and architecture*

Someone needs to define what good telemetry looks like across the organization: naming conventions, context propagation patterns, cardinality guidelines. Without this, every team invents their own approach and nothing correlates.

### *Vendor and tool strategy*

Someone needs to own relationships with vendors, evaluate alternatives, and make build-versus-buy decisions. The bias should be toward buying commodity capabilities and adopting open standards. Custom code should be reserved for genuinely unique requirements.

### *Integration*

Someone needs to make all the pieces work together in your specific environment: connecting tools to workflows, adapting generic capabilities to local constraints, and building the connective tissue that makes everything function as a system rather than a pile of parts. This work is easy to undervalue but often determines whether the platform succeeds or just exists.

These capabilities do not map one-to-one to job titles. A single senior engineer might cover three of them, a larger team might have specialists in each, and a large enterprise may choose to have a small team of observability specialists backed by a multidisciplinary virtual team. The point is that all of these capabilities must exist somewhere. Gaps in any one will limit the success of your observability transformation.

## **Bringing the Existing Team Along**

If you honored the heroes early, some of these engineers may already be in your coalition. They believed in the transformation before it had momentum. Now you need to define their roles in the new world. For the rest of the legacy monitoring and observability team, this is your best chance to convert them into supporters rather than detractors.

Here is where most transformations fail: they treat existing monitoring teams as obstacles instead of assets.

These engineers have skills you need. They understand production systems. They know where the bodies are buried. They have relationships across the organization. The question is not whether they are valuable; the question is what role fits their skills and interests in the new world.

**Simon Wardley's model** of pioneers, settlers, and town planners is useful here:

*Some engineers are pioneers.*

They love greenfield work, new technologies, and building things that did not exist before. These people will thrive in the early phases of transformation: building new instrumentation, experimenting with approaches, and proving concepts. Give them the uncharted territory.

*Some engineers are settlers.*

They take what pioneers create and make it robust, scalable, and production ready. Once your proof point exists, settlers make it real. They build the libraries, harden the infrastructure, and create the documentation. They turn experiments into platforms.

*Some engineers are town planners.*

They optimize mature systems, reduce costs, improve efficiency, and keep things running. As your new observability approach matures, town planners ensure it stays reliable and cost-effective. They may also continue maintaining legacy systems during the long migration, which is valuable work that keeps the organization running.

The failure mode is forcing pioneers to be town planners, or expecting town planners to be pioneers. Match people to phases of work that fit their skills and temperament.

## Breaking the Vendor Identity Trap

Remember the identity problem? “I’m the Splunk person” felt like an identity because it was. Vendor-silo alignment is not just an organizational obstacle; It is a career trap. When someone’s expertise is bound to a proprietary tool, their value depends on that tool’s continued relevance.

You break that attachment by offering something better: skills that are not trapped inside one vendor’s ecosystem.

Open standards make this concrete. Engineers who learn OpenTelemetry are learning industry-wide patterns, not your organization’s idiosyncratic tooling. “I’m the Splunk person” becomes “I’m a telemetry architect,” a skill that travels with them regardless of which vendor comes next.



## What Not to Promise

When building the observability team, do not promise “No one’s job changes.” That is not true, and saying it destroys trust. Jobs *will* change. Some people will thrive in new roles. Some will find better opportunities elsewhere. Pretending otherwise insults everyone’s intelligence.

*Do* promise: “We need your expertise, and here is how.” Be specific about paths forward: pioneers get new territory, settlers get scaling challenges, and town planners get optimization work. Everyone gets a path that uses their strengths.

The goal is an organization where every engineer can see what their code does. The observability team exists to make that possible.

## Conclusion

This is organizational change. It’s slow. It’s political. It’s worth doing anyway.

You will face setbacks. Initiatives will stall when sponsors change roles. Budget cycles will squeeze your timeline. Champions will leave for other opportunities. Some pilot programs will fail, and skeptics will treat those failures as proof that transformation was never possible.

Keep going.

The organizations that succeed treat observability as a foundational capability, not a project. Projects end; capabilities evolve. Projects have deadlines; capabilities have roadmaps. Projects get funded once; capabilities get sustained investment.

The goal is an organization that learns from production, not just survives it. That capability compounds over time. Teams with modern observability debug faster, ship more confidently, and burn out less. They spend their energy building instead of firefighting. That difference accumulates.

Teams without modern observability have to hold war rooms, accept mysteries, and learn about problems from customers. They move cautiously because they cannot see. They exhaust their best engineers on incident response instead of product development.

This chapter helped you recognize the gaps. You now have the tests to diagnose them, the framework to understand resistance, and the roadmap to close these gaps. What remains is the commitment to sustained effort and the patience to see it through.

In the next chapter, you will learn about how to decide which tools to use. Should you build your own, buy from a vendor, use open source, or use a custom mix of all three?



---

# **Build Versus Buy (Versus Open Source)**

The choice between building, buying, or adopting open source software (OSS) is rarely a simple technical comparison; it's a high-stakes economic decision that determines where your organization's most precious resource—engineering cycles—will be spent. While generating code has never been faster or easier in the era of AI, the long-term cost of maintaining bespoke infrastructure remains as high as ever. Every hour your team spends maintaining a nondifferentiating internal tool is an hour they are not spending on the core products that drive your market share.

This chapter provides a framework to help you navigate these trade-offs with clear eyes. We will examine the true costs—both obvious and hidden—of each approach and help you recognize whether your investments will compound into a competitive advantage or simply add to your technical debt. By the end of this discussion, you will be able to distinguish between the infrastructure that should be a commodity and what justifies custom development.

After all, build versus buy is not a binary choice; it's a multidimensional spectrum. Pretty much everyone is going to end up buying some software, using some open source, and writing some code of their own. The important questions are: which, where, and why?

This topic is not new, but the trade-offs and relevance have shifted quite a bit in recent years. As mentioned in previous chapters, many companies now spend more on their observability bills than almost on any other category of tools, second only to cloud spend.

The next chapter will guide you through the process of evaluating vendors, managing vendor relationships, and extending influence across company and industry lines. Paired together, this chapter and the next form a block of knowledge that will help you navigate the thicket of business and engineering complexity in observing modern day systems.

## Which Do You Hate More: Your Money or Your Time?

There's an old DBA joke: Oracle is what you use when you love your time and hate your money; MySQL is what you use when you hate your time and love your money. (Hey, we warned you the joke was old. Kids, pretend we said "DuckDB" and "Snowflake," OK?) While the technologies have evolved, the fundamental trade-off remains constant.

In every build-versus-buy discussion, you are choosing which type of pain to tolerate:

### *The "hate money" path*

Teams on this side of the spectrum are all but impervious to sticker shock. They view engineering focus as their most precious and finite resource. To them, money is "cheap" if it buys back time and focus for core product innovation.

### *The "hate time" path*

For these teams, capital is the scarcest resource. They may find it easier to justify \$1 million in engineering headcount than a \$100,000 tool budget. They're willing to accept the long-term costs to avoid an upfront bill.

The mistake most teams make is failing to realize that they are making this choice. They view building an internal tool as "free" because the salaries are already paid, ignoring the staggering opportunity cost of what those engineers aren't building instead.

## Surfacing Our Biases

Now that you have considered whether you prioritize protecting your capital or your engineering cycles, we must look at why your answer is likely skewed by human nature. Let's explicitly name some of the biggest biases and fallacies that reliably show up in the build-versus-buy debate.

First is our own bias as employees of an observability software vendor. We've seen over and over again that organizations radically underestimate the true cost of building their own solutions, or even the cost of operating open source tools. What looks like a cheap solution adds up swiftly once you factor in salaries, cloud spend, and opportunity costs. As a result, we tend to think buying a solution provides better value.

Second, and conversely, is the engineering bias. As Rick Clark—formerly of Mastercard, VMware, Cisco, and Pivotal Software, whose experience we’ll be drawing on extensively in this chapter—says: “Any time a CTO asks their engineers if they should build or buy, the engineers are going to say ‘build.’ Engineers want to build just like surgeons want to cut—they have a professional bias toward their skill set.”

This bias is human nature. We became engineers because we love to build. But just because you *can* build something doesn’t mean you *should*. The opportunity cost of building nondifferentiating infrastructure can be staggering. Maintenance costs always dwarf the costs of building by a margin that’s large enough to be scary, yet unknowable up front.¹

Those two may be the biggest biases, but there are many more that skew the debate. The sunk cost fallacy shows up regularly: “We spent *so much time* building this; we can’t abandon it now!” and so does the NIH (not invented here) bias: “We are such a special snowflake; no one else can *possibly* understand our requirements.” There are valid fears of vendors going out of business, or using vendor lock-in and data gravity to raise your prices. Migration itself may even loom as an outsized fear in your mind, if your organization has tried and failed to migrate in the past.

By naming these biases up front, we can move past the emotional appeal of “building it ourselves” or the marketing appeal of “buying the solution.” If you are running the buying process, encourage each stakeholder to lay their real fears on the table so you can collectively evaluate their degree of risk.

There are frameworks you can use to evaluate your options and drive the process to a wise conclusion; we will introduce two in this chapter. But before we apply a standard decision framework to this choice, we must acknowledge a fundamental truth: observability isn’t like any other software.

## Observability Is Not Like Other Software

For the most part, we have been treating observability like any other piece of software. It is not. *Observability is the software that watches the watchers*. This means you probably have higher reliability requirements for your observability tools than for other types of tools, and you may want to think seriously about how that risk is bundled.

The biggest risk is this: if you run your observability services on the same cloud provider, in the same region, using the same stack as the rest of your production systems, it’s going to go down or suffer degraded service any time that any of your

---

¹ Yes, this is true even in the era of AI-assisted coding, where the cost of new code is zero. The maintenance costs for AI-generated code are no different; if anything, those maintenance costs are *higher* than those for code we wrote ourselves.

other dependencies has a problem, which is the worst possible time for your visibility to go dark or glitch out.

Part of what you’re paying a vendor for is not just the tool or the interface, but high availability and a stack that is radically decoupled from your own dependencies.

If you don’t need high availability (meaning your customers are fine with 98%, 99%, *maybe* 99.5% availability—and *you’re sure about that*), then you are probably correct in thinking you should be able to pay substantially less by running your own open source stack.

Let’s start with a simple 2 × 2 for evaluating the landscape.

## A Simple Framework for Evaluating “Build Versus Buy”

The build-versus-buy framework we’ll discuss in this chapter was first developed by Rick Clark, mentioned earlier, and is now used by dozens of companies. Shown in [Figure 29-1](#), it’s a simple 2 × 2 matrix that can clarify your thinking remarkably fast.

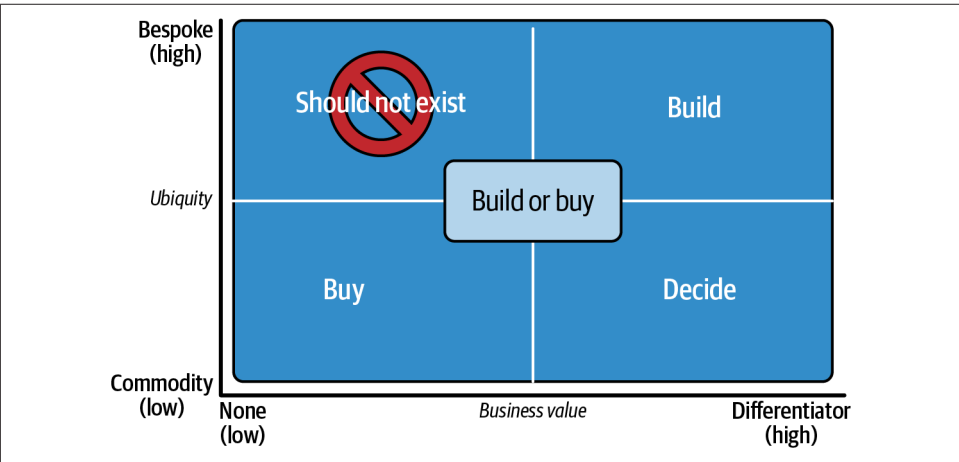


Figure 29-1. Build-versus-buy matrix, with ubiquity on the vertical axis and business value on the horizontal axis.



For the sake of this exercise, “build” should be interpreted as including everything from writing your code from scratch to pulling together a composable pipeline of open source components. “Buy” means paying a vendor. We’ll tackle a more nuanced set of options later on in this chapter.

This framework can be used for any component of your architecture, from network routers to visual design libraries and everything in between. In this chapter, we'll discuss it through the context of building versus buying observability solutions and different elements of your observability stack.

First, consider the axes:

- The horizontal axis represents *business value*, from “none” (will not add revenue or market share) to “differentiator” (which could be a significant competitive advantage in your market).
- The vertical axis represents *ubiquity*, from “commodity” (available everywhere, highly standardized) to “bespoke” (unique to your specific needs).

## Business Value

Before we move along, let's get crisp on what we mean by “business value.” Engineers sometimes confuse business value with “importance,” but those are very different things. Business value means *market share* and *revenue impact*. Will building this thing bring you more customers? Will customers pay you more if you build it?

Every company runs lots of software that is desperately important, but does not intrinsically add market share or revenue.² For example, your secrets vault is very important, but running it on a `u-24tb1.112xlarge` high-memory instance type (448 CPUs, 24 TB RAM) isn't going to increase your market share or revenue impact.

Another way to think about it: every organization has two types of problems.³ There are problems that are common to all businesses (How do we authenticate users? How do we monitor our systems? How do we manage DNS?), and problems that are unique to your business model and competitive position. In general, you want to channel engineering “build” cycles toward the second category.

This framework is both liberating and time-saving in the way it lets you swiftly rule out large swaths of software that should obviously be built or bought. (If you have anything in the “should not exist” quadrant, that is worth bringing to light!) This means you can reserve your time and energy for talking through the most interesting decisions.

---

² This is also referred to as “infrastructure.”

³ Technically, every business has *three* kinds of problems: problems that are common to all businesses, problems that are unique to your business but common across teams, and problems that are unique to your business and constitute your core value proposition. For the sake of simplicity, we're lumping the first two together.

# The Four Quadrants: Build, Buy, Should Not Exist, or Decide

The four quadrants guide your decision making: build, buy, should not exist, or decide. You should be able to map each piece of technology you operate or need to acquire into one of the quadrants.

## Build (upper right)

The “build” quadrant represents high business value and bespoke needs. This is where your competitive advantages and organizational core competencies live. For Honeycomb, one of these differentiators is Retriever, our custom-built columnar storage engine (discussed in [Chapter 13](#)). These are the kinds of differentiators that justify building custom solutions.

## Buy (lower left)

The “buy” quadrant represents tech that has low business value and is a commodity. This is where most infrastructure components live: service catalogs, identity management, logging infrastructure, and observability platforms. If it’s standard and doesn’t differentiate you, buy it.

## Should not exist (upper left)

The “should not exist” quadrant represents low business value combined with bespoke implementation. These are things that are unique to your organization but don’t actually help your business. They shouldn’t exist, but unfortunately they often do. And when they do, they are often the worst kind of technical debt.

Here’s a real-life example of something an organization should not have built:

[financial institution] had written its own threaded request handler module in Apache HTTPD 1.3.x. It was five times faster than the stock scheduler. It was tightly integrated with all the company’s processes and apps. Years later, Apache 2.0 came out and had a new module API and a threaded scheduler that was even faster, and the original developers were long since gone. They were stuck and actually disadvantaged with slower connections/sec than others. When support for HTTPD 1.3.x finally ended, they paid a consulting company over \$10 million to fix their mess (but not before the whole thing turned into a regulatory compliance issue, because unsupported software versions can’t get security updates).

You are better off figuring out how to obliterate this software entirely. Find out how to do the same sort of work with other open source or commercial systems.



Some of these codebases do end up being open sourced but they are, for all intents and purposes, abandonware projects immediately upon release.



## Decide (lower right)

The “decide” quadrant represents “high” business value and is a commodity. These are things that genuinely differentiate your velocity and product quality, but where decent off-the-shelf options exist. This is where all of the hard and interesting conversations happen.

There is no one right outcome to be had here; you’ll have to convene the right stakeholders and hammer this out between yourselves. Sometimes the decision may come down to whether you have the right subject-matter expertise in house, or how much of a competitive advantage you think is at stake. These discussions are often heavily influenced by your organizational habits and ethos. Some shops are commercial off the shelf (COTS) all the way. Others are heavily OSS-driven.

We caution you to avoid making these decisions by managerial fiat. These decisions are heavily weighted toward the technical side of sociotechnical systems, and they should be owned by your technical domain experts, ideally ones with skin in the game. This is what staff+ engineers are there for. You will get better outcomes if they have a say in the decisions that get made.

### Staff+ Engineering Incentives Should Align with Business Outcomes

*To staff+ engineers:* One of the best things you can do for the sake of your own career is develop a reputation with management that you can be trusted to make good decisions based on the actual needs of the business, not your own desire to build cool stuff.

*To the managers of staff+ engineers:* Please promote and level engineers based on the caliber of their decisions and the strength of those business outcomes, not on how many lines of code they write. Engineers typically assume they are more likely to get promoted to principal engineer by spending 18 months and \$20 million worth of engineering cycles building an observability solution from scratch than by picking a good vendor, building some integrations, and rolling it out internally—even if the latter costs a quarter as much, takes half the time (though they often deliver a better product overall).

Engineers aren’t making this up; they see this exact kind of thing coming out of promotion processes *all the time*. It’s one of the biggest drivers of technical debt in the world. If you want your engineers to *make* good decisions based on business outcomes, then as leaders you must *reward* good decisions based on business outcomes.

When you find yourself in the “decide” quadrant with a decision to be made, here are the key questions to ask:

*What precisely is the job to be done by this software?*

Develop an objective rubric that the organization can see and weigh in on. Start high level, with stories and scenarios, not with a laundry list of features or any vendor-specific implementations. We talk about this process more in the next chapter on vendor engineering.

*How unique are our requirements?*

If you think “no one’s like us,” you’re probably wrong. Most companies are far more similar than they believe. Take your rubric and compare it to similar architectures and case studies from peer companies. How did they already solve this problem? What are the deltas for your use case that existing systems don’t solve for?

*What’s the smallest version of this problem we can solve?*

Don’t build a massive custom platform if you can solve 80–90% of your needs with a commercial tool and build the remaining bits as extensions.

*Do we have product management expertise to dedicate to this?*

Building internal tools requires product thinking, user research, iteration cycles, and roadmap management, as well as internal evangelism and marketing. Without that discipline, you’ll end up with an unloved tool that never sees adoption.

*What’s our timeline?*

Can your business afford to wait months or years for a custom solution, or do you need to get started immediately?

*Who will support it?*

Who will support it when the people who built it move on?

If you end up deciding to build your own because there were no solutions available that fit the bill perfectly, but some were close, set a reminder for yourselves to check back in a year.

One unscientific rule of thumb to keep in mind: the closer you get to laying bits down on disk, the more conservative you should be about preferring battle-tested code. User experience (UX) is on the other end of the spectrum; the closer you get to UX, the more likely you are to end up writing custom code to handle your use case, and the less paranoid you should be about it.

You need a really, *really* good reason to write a new database, operating system, or filesystem format. But writing a new library on top of OTel to give your frontend team intuitive, easy instrumentation that delivers end-to-end insights on app responsiveness? Yeah, that sounds pretty normal.

## Mapping Examples to Quadrants

Let's apply our rubric to a few common “jobs to be done” in the observability arena.

### Example 1: Custom SLO/error budget tracking

*Jobs to be done:* Engineers need to understand if their services are meeting reliability targets and have budget left over for experimentation.

*How unique are our requirements?* Your service level objectives (SLOs) are probably: latency percentiles, error rates, availability—same as everyone else. The deltas might be: “We have unusual traffic patterns (batch jobs, retail cycles)” or “We need to tie this to deployment gates.” Every vendor does this.⁴ You think you're special because you have 37 microservices? So does everyone.

*Smallest version:* Use vendor SLO features for 80% of services. Build custom tooling only for the 3–5 services with genuinely weird requirements (maybe your machine learning inference pipeline has different latency characteristics than HTTP services).

*Do we have product management expertise?* This is where internal SLO tools die. Someone builds it, it's hard to use, no one iterates on it, and everyone goes back to their old dashboards.

*Timeline:* Your CFO wants reliability metrics next quarter. But you can't wait a year for your custom solution.

*Who will support it?* The senior engineer who built it just got promoted and doesn't want to deal with it anymore.

*Verdict:* **Buy** for 90% of use cases, and maybe **build** thin wrappers for genuinely unique business logic.

### Example 2: Custom instrumentation library wrapper

*Jobs to be done:* Frontend developers need to instrument their React apps without becoming OTel experts. We want consistent patterns and our specific context automatically included.

*How unique are our requirements?* Your React app is...a React app. But the UX requirement—making it dead simple for your team—is actually legitimate. The generic OTel SDK requires too much ceremony for your developers' workflow.

---

⁴ The one distinguishing feature is that most vendors don't derive their SLOs from data in a bidirectional way. You can't say “show me the requests that violated my SLO, so I can look for patterns” with the three pillars generation of tooling or with vendors that specialize in SLOs. This is a killer capability you only get with modern, unified observability tools.

*Smallest version:* A thin wrapper around OTel that:

- Autocaptures your standard context (user ID, session ID, feature flags, whatever else)
- Provides three to four helper functions for common patterns
- Has maybe 200 lines of code

*Do we have product management expertise?* You need someone who understands your developers' workflow and will iterate based on adoption. This can be lightweight—maybe a senior engineer who cares about DX.

*Timeline:* You could ship this in two weeks.

*Who will support it?* This is lightweight enough that your frontend platform team can own it. Low surface area means low maintenance burden.

*Verdict:* **Build!** This is a UX layer, not infrastructure. You're not building a new tracing backend; you're making OTel ergonomic for your context.

### Example 3: Cost attribution and chargeback

*Jobs to be done:* Engineering teams need to understand how much their observability costs, tied to their service ownership, so they can make informed trade-offs about what to instrument and how aggressively to sample.

*How unique are our requirements?* Your organizational structure is unique to you—which teams own which services, how you do internal billing, whether you chargeback by cost center or business unit. The underlying problem (measuring and allocating observability spend) is universal at scale, so every vendor will have an approach. Most will give you org-level billing and maybe some basic tagging for cost allocation. Few give you “team X spent \$4,500 this month; here's the breakdown by service.” Even fewer integrate with your internal chargeback systems.

*Smallest version:* Use vendor tagging to get 80% there. Build a lightweight script that pulls billing data via API, maps it to your internal team structure, and generates a monthly report. Don't build a full cost management platform.

*Do we have product management expertise?* This is political as much as technical. You need someone who understands both the billing side and the engineering culture. Without ongoing iteration, teams will game your cost allocation or ignore it entirely.

*Timeline:* If you're bleeding money on observability, you need visibility this quarter, not next year.

*Who will support it?* This sits awkwardly between finance, platform, and engineering. Make sure someone actually owns it or it'll die when priorities shift.

*Verdict:* **Buy** the observability platform, and **build** the attribution layer if (and only if) you have material spend and a real chargeback culture. Most companies should just use vendor tags and manual reporting.

## The Generalized Pattern

The pattern that emerges from these examples:

*Infrastructure/storage*

This is almost always **buy**.

*UX/workflow*

This is often **build** (small, focused tools, or thin layers and wrappers).

*Intelligence/decision making (anything that will take you more than three months to develop)*

**Build** only with strong project management commitment.

The “decide” quadrant is full of things where the technical capability is increasingly commoditized, but your workflow, your business context, or your org structure creates deltas. The question is whether those deltas justify a full rebuild instead of an extension of vendor tools.

## What About Open Source?

Open source and source-available code are options that don’t fit neatly into the previous build-versus-buy matrix. It’s not quite “building”—you’re not creating something from scratch. But it’s not quite “buying” either—you’re not paying a vendor to handle the complexity for you. However, you may be purchasing systems integration work to get it installed with the rest of your infrastructure or, if nothing else, service and support to keep it up and running.

Open source is best understood as a third option that shifts where you pay costs. Instead of paying upfront licensing or usage fees, you pay in:

- Engineering time to evaluate, deploy, and integrate
- Infrastructure costs to run the software
- Ongoing maintenance and upgrade work
- Lost opportunity costs of engineers maintaining infrastructure rather than building product features
- Recruiting and retaining engineers with expertise in those specific open source tools

As developer advocate Heidi Waterhouse memorably put it, OSS is “free as in puppies, not free as in beer.” A puppy costs \$0 to acquire, but requires years of feeding, vet visits, training, and care. The same principle applies to open source observability tools.

This doesn’t mean you shouldn’t use open source; many organizations derive tremendous value from it. But you must be fully aware of what it’s really costing you, and make that decision with clear eyes.

Source-available code is much like open source, with the key difference being that you cannot customize, extend, or even *fix* the code directly.

## A Framework That Includes Open Source

If open source adoption is part of the mix (or acquisitions, consultancies, and other hybrid solutions), you may find this adaptation of the decision framework by Peter Corless to be helpful, as shown in [Figure 29-2](#).

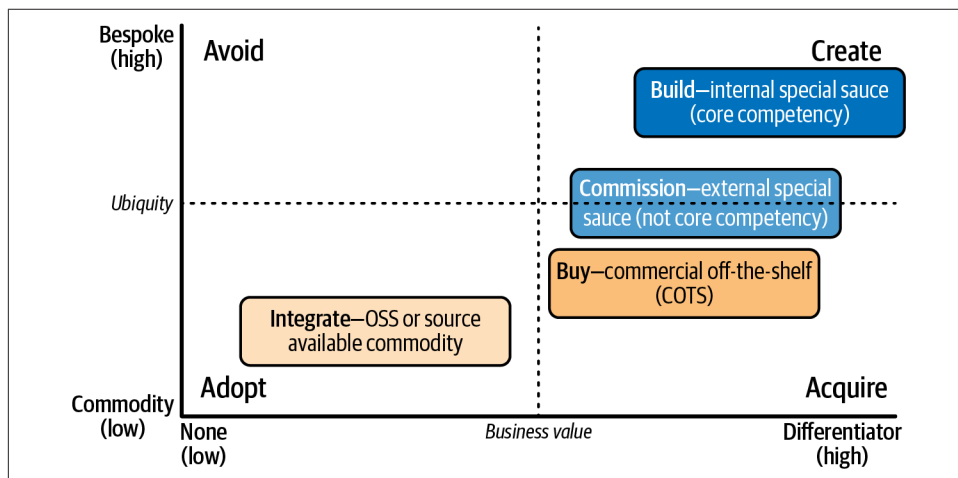


Figure 29-2. Build-versus-buy  $2 \times 2$  matrix expanded to include open source, source available, COTS, commissioning, consultants, and acquisitions as options.

The axes here are the same as before:

- The x-axis represents business value, from “none” to “differentiator.”
- The y-axis represents ubiquity, from “commodity” to “bespoke.”

In the simpler version of the  $2 \times 2$ , our quadrants were called: build, buy, should not exist, and decide.

In this “enriched” version, the corresponding quadrants are: create, adopt, acquire, and avoid. The quadrants mean roughly the same thing, but have been renamed to better suit the context of open source.

The categories we want to plot don’t map neatly to entire quadrants this time, but we will describe where each one falls on the  $2 \times 2$  map. These categories include: “integrate” (OSS or source available), which falls in the “adopt” quadrant; “buy” (COTS, certain SaaS) in the “acquire” quadrant; and “commission” (consultants, usually), spanning the “create” and “acquire” quadrants; in addition to the standard quadrants of “build” and “should not exist.”

## The Four Quadrants: Avoid, Create, Adopt, Acquire

These four quadrants inherit similarities from the  $2 \times 2$  matrix discussed in [“A Simple Framework for Evaluating ‘Build Versus Buy’” on page 510](#). Where they are the same, we will not repeat ourselves, but we will note where they differ.

### Create (upper right)

The build category is the same as the simple framework for evaluation: high business value combined with bespoke solution.

This is software that has potentially significant intellectual property value. You may decide to keep it as a trade secret, trademark it, or file for patents. It could become the basis of your own COTS product or service offerings. More virtuous leaders in the industry might even make it available as source available or OSS for other organizations, thus bringing the cycle full circle.

### Avoid (upper left)

The “avoid” category is also the same as the simple framework for evaluation: low business value problems with bespoke solutions shouldn’t exist.

### Adopt (lower left)

The “adopt” category represents low business value combined with commodity. This is the realm of OSS and source-available licensed code, and where most infrastructure components live: agents, SDKs, service catalogs, identity management, logging infrastructure, observability platforms. If it’s standard and doesn’t differentiate you—just download it and adopt it. From a software developer perspective, the advantage is that you can see the code in order to diagnose low-level problems and, in the case of OSS, you can even contribute the fix.

### Acquire (lower right)

The “acquire” category represents some business value above and beyond what can be found in open source, and critical differentiation. This is the realm of COTS software products and SaaS offerings with advanced features you are willing to pay for. You can customize it for your environment, maybe write a few custom plug-ins or connectors, but you can’t fundamentally change the codebase. In most cases, you can’t even *see* the codebase, which might make it hard to diagnose and troubleshoot the thorniest of problems.

### Commission (middle right)

Straddling the border between “create” and “acquire” is the “commission” category: the realm of custom engineering and work for hire. You do this when you need software that isn’t available off the shelf, but it isn’t something you need to develop as an internal core competency—or you simply don’t have time or internal resources. It’s pure mercenary work for a job that needs to be done well and fast. You still have to create the spec, but the hired gun does the job. Maybe it is just one of those custom plug-ins or connectors for COTS software you acquired. Or maybe it’s something decidedly more sophisticated, like customized dashboards, CI/CD workflows, or data science algorithms and models. The advantage of commissioned software is that you own the code after it’s written. Simultaneously, that’s also the big disadvantage.

## There’s Always Vibe Coding

This is far from a comprehensive accounting of all the ways you can acquire software. Our list doesn’t even count Claude or ChatGPT, because we are only listing ways that have some form of quality, accountability, and governance baked into the model. If you’d like to ask Claude to go generate your enterprise observability solution, be our guest.

Even before generative AI came along, the true cost of software had very little to do with the time it took to write code, and everything to do with the cost of operating, scaling, changing, extending, refactoring, and monitoring that code. Which is as true now as it has ever been.

## The Real Costs of Building Your Own

Most companies today will end up buying services from one or more observability vendors. We buy observability from a vendor for the same reason we buy DNS, storage, networking, and email: because if it’s not your core differentiator, it probably doesn’t make sense for you to expend valuable engineering cycles on building it yourself. It’s probably a distraction from your business goals, and distractions can be very expensive.



Opportunity costs are usually the biggest expense, because resources are finite. By choosing to build an observability tool, what did you choose not to do with those engineering cycles? What potential business value have you sacrificed?

Anything you can build in a year with a small team of engineers is unlikely to compare favorably to a service with hundreds or thousands of engineers building it. It's not just a numbers game, either: observability companies are subject matter experts—it's what they do. Presumably it is *not* what you do.

To calculate opportunity costs, you must first calculate the financial cost of each option, quantify the benefits received, and compare the two. Let's start with an example of calculating the costs of building your own.

## The Hidden Costs of “Free” Software

You could choose to build your own observability stack using open source components that you then assemble yourself. For simplicity, we'll assume this is the case. (Writing the code yourself from scratch would cost at *least* an order of magnitude more.) To illustrate actual operating costs, we'll use a real example.

We spoke with an engineering team manager who was considering investing in a commercial observability solution after having built an in-house ELK (Elasticsearch, Logstash, and Kibana) stack. We'd given him a quote of about \$80,000 per month to provide observability for his organization.

He immediately balked at the sticker shock of a nearly \$1 million annual solution and pointed to the “free” ELK stack his team was running internally.

However, he wasn't counting:

- The cost of dedicated hardware to run the ELK cluster, at \$80,000 per month
- Three additional engineers recruited specifically to help run it at \$250,000–\$300,000 each per year
- One-time recruiting costs at 15–25% of annual earnings, or about \$75,000 per engineer
- Organizational time spent hiring and training them to become effective

The seemingly free solution had been costing his company *over \$2 million each year*—more than twice the cost of the commercial option. But he'd been spending that in ways that were less visible to the organization.

This pattern repeats constantly (see [Chapter 27](#) for more analysis). Engineers are expensive, and recruiting them is hard. It's especially wasteful to devote their talent to building bespoke infrastructure solutions in the name of saving money, when the reality is often much further from the truth.

The first step to understanding ROI is to truly understand the hidden and less-visible costs of running supposedly free software. Calculate:

*Full-loaded engineer costs*

Salary, benefits, recruiting fees, training time, management overhead

*Infrastructure costs*

Compute, storage, networking, and the engineering time to manage that infrastructure

*Opportunity costs*

What could these engineers be building instead that would actually differentiate your business?

*Ongoing maintenance burden*

Version upgrades, security patches, integration work, scaling challenges

If you've never really stopped to think about how much your own time costs or how much your team's time costs, first of all, don't feel bad. Most engineering managers we hear from haven't done this either. But *do this*. Your time, your energy, your focus...they are *not cheap*. *You own your attention and your calendar*. No one else is going to protect it for you.

## The Benefits of Building Your Own

In many organizations, the deciding factor comes down to justifying visible spending, which is one of the worst ways to make a complex sociotechnical decision. But there are real benefits to investing local engineering cycles into the domain:

*Building creates in-house expertise.*

When you decide to build custom tooling, it becomes someone's job to deeply understand your organizational needs and turn those into functional requirements. Over time, you develop institutional knowledge about both observability best practices and your specific systems' characteristics.

*Building enables deep customization.*

You can create integrations with your bespoke internal systems, enforce naming conventions that matter to your organization, and build workflows that match exactly how your teams operate. Rather than using generic prebuilt software designed for many use cases, you can optimize for your specific needs. (But note that this benefit can turn into a liability, if you aren't careful; optimizations and customizations scale poorly compared to standard interfaces.)

*Building teaches you the domain deeply.*

If you choose to build your own observability solution, you'll inherently need to internalize and implement the concepts in this book—from the necessary

data granularity provided by events, to automating the core analysis loop, to optimizing your datastore to performantly return results against large quantities of unpredictable high-cardinality data. You'll learn exactly where metrics serve your business and where they don't by slamming headfirst into their limitations.

*Building might be good for the bottom line.*

If you choose to build, it might be that what you're building can save the company considerably over existing commercial offerings, especially if there are vendor lock-ins that you can't otherwise escape. Yet remember to pay attention to the true cost of building as just outlined. Beyond cost savings, sometimes building software can actually become part of your company's product or services offering, actively bringing in revenue. Or it could be part of its intellectual property portfolio, such as being patented, if there is a chance to license it to others. However, be realistic as to the probabilities of your organization actually doing anything of commercial value with an internally built tool.

*Building your own may be the only viable option.*

This may be the case if, for example, you want a cheap, basic monitoring service and none of the big vendors are building that.

## The Risks of Building Your Own

The benefits of building don't come without serious risks. The era of building infrastructure like infrastructure—and exposing the rough edges of that infrastructure to software engineers—is over. Anything you build in-house for other engineers needs to get built like a product.⁵

*Product management expertise is rare.*

Does your observability team have a product manager (whether by title or function) whose job is to interview users, determine use cases, manage feature development, deliver a minimum viable product, gather feedback for iteration, and balance constraints against business needs? If not, you're likely to build something that doesn't meet user needs effectively. When that happens, it can damage senior management's trust in engineering's ability to make sound technical decisions.

Product-led organizations will have in-house product expertise that can make custom development successful. But whether that product management expertise will be allocated away from core business objectives toward building internal tooling is another question entirely.

---

⁵ For a longer discussion of why this matters so much, follow Abby Bangser on LinkedIn/Bluesky, or read the article “[Infrastructure Gravity & Domain Engineering](#)” by Jack Danger.

*Building takes time.*

Custom solutions delay your observability adoption initiatives. Can your business afford to wait several months to get a functional solution in place instead of getting started immediately with a commercial product? This is like asking whether your business should wait to get operations underway while you first construct an office building. Sometimes that makes sense for long-term considerations, but often reduced time to value is preferable.

*Ongoing support is never-ending.*

Building your own observability product is not a one-and-done solution. It requires ongoing maintenance. Underlying third-party components will get updates and patches. Organizational workflows and systems are constantly evolving. Unless those components and integrations stay current, custom solutions risk becoming obsolete...and Google won't be very helpful when it breaks.

*Adoption might be low.*

If you don't have the organizational ability to deliver a system with excellent UX, workflow flexibility, and speed, it's entirely likely that you'll have invested substantial time and money into a solution that never sees widespread adoption beyond those intimately familiar with its rough edges and workarounds.

An unfortunately common pattern in enterprises is to devote development effort toward building an initial version of internal tools and then move on to the next project. Internal tools get abandoned once development stalls, adoption is low, and the effort required to make them meet current requirements isn't a high enough priority compared to competing interests.

## The Real Costs of Buying Software

On one hand, the costs of buying software tend to be more visible and upfront, which is good. On the other hand, cost may or may not go up in tandem with value. Recall that Gartner study we mentioned in [Chapter 23](#), with a representative customer whose observability costs have been going up 48% per year for the past 15 years straight. Are *you* getting 35,800% (358x) more value out of your tools now than you were in 2010?

## The Hidden Financial Costs of Commercial Software

Let's start with the most obvious cost of buying software: the financial cost. In a vendor relationship, this is the most tangible expense—you see it whenever your vendor sends you a bill.

A reality of commercial software is that vendors must make money to survive, and their pricing schemes factor in profit margins. You will pay a vendor more to use their software than it costs them to build and operate it for you. That's not unfair—it's

how businesses work. (If you are paying them *less* than it costs to run your workload, you should be very concerned—about their survival as a business, and about whether or not they care about retaining you as a happy customer.)

What *is* considerably less fair is obscuring the real costs, avoiding transparency, or stacking so many pricing dimensions that you cannot reasonably predict what your usage patterns will cost in the future.

A common antipattern to watch for is *pricing strategies that obscure total cost of ownership*. These include per-seat, per-host, per-service, per-query, or per-any-other-impossible-to-predict metering mechanism. Pay-as-you-go pricing may seem approachable at first—until your usage explodes at a rate disproportionately more expensive than the revenue it helps you generate.

Consider these questions as your organization grows:

- Will your team size change as your startup becomes more successful? (Almost certainly yes.)
- Will the number of hosts significantly grow in the future? (Likely yes, though instance resizing complicates projections.)
- Will there be more services? (Possibly, especially if you decompose monoliths into microservices.)
- What about the number of queries you run? (Almost certainly yes, if the tool is successfully adopted.)

When it comes to observability specifically, you should predict exponential growth in data queries as a culture of observability spreads through your organization. The recommended practice is to analyze observability data with increasing curiosity. You should see query volume expand dramatically as more teams find value in understanding system behavior.

Therefore, avoid observability tools with pricing schemes that penalize adoption and curiosity. You want to ensure that as many people in your company as possible have the ability to understand how your customers use your software, in as many ways as possible. Observability is not just for engineers! Support, product, design, security, marketing...the more people stay grounded in the truth of production, the better for everyone.

As a consumer, demand that your vendors be transparent and help you calculate hidden costs and forecast usage patterns. Ask them to walk you through detailed cost estimates of both your starting point today and your likely usage patterns one to two years in the future. Be wary of any vendor unwilling to provide exceptional detail about how their pricing will change as your business grows.

## The Hidden Nonfinancial Costs of Commercial Software

A secondary hidden cost of commercial solutions is *time through vendor lock-in*. Yes, buying a ready-made solution lowers your time to value initially. But once you’ve committed to a commercial solution, how much time and effort would be required to migrate to a different solution?

The most labor-intensive part of observability adoption is instrumenting your applications to emit telemetry data. Many vendors shortcut time to adoption by creating proprietary agents or instrumentation libraries prebuilt to generate telemetry in proprietary formats their tools expect. While those proprietary agents may be quicker to start with, any time and effort devoted to making them work must be repeated when evaluating alternative solutions.

The open source OTel project solves this problem by providing an open standard that many observability tools support. OTel allows you to shortcut time to value with proprietary tools by using OTel distributions (distros) that layer boilerplate configurations on top of standard OTel libraries.

Your strategy to avoid vendor lock-in should be to use native OTel functionality by default in your applications and rely on distros or exporters to handle vendor-specific configuration. Should you wish to evaluate or migrate to a different tool, you can replace configurations without rewriting all your instrumentation.

### Use OTel, and Model Costs For Growth

“Don’t use their own agent! Use an agnostic agent like the OpenTelemetry Collector, even if it means having to jump through a few hoops to understand how data is meant to be transformed and shipped around. Be open to both decorating telemetry data with useful fields and removing useless fields to save on cardinality and dimensionality.

Make sure to model your costs for your current usage and potential future usage as your company grows. Build in a buffer to account for misconfigurations that could potentially explode your bill. Usually providers can forgive those accidental costs but don’t put yourself in a tricky position by not covering yourself. Do not trust anything that’s not in writing!”

Matthew Sanabria, Engineering, Oxide Computer Company

Some vendors nominally support OTel, but will leverage their weight and influence during the sales and proof-of-concept process to disparage OTel and try to convince you to use their proprietary integrations instead. They might say how much easier and simpler and better it is to just use their agents, or how much more “mature” and “battle-tested” their agents are.

These vendors are entirely capable of making the OTel experience as good or better than their proprietary agents. They have chosen not to do so because they are trying to lock you in. It is, regrettably, a very effective strategy.

## The Benefits of Buying Commercial Software

The benefits of buying software are mostly what you'd expect. In the next chapter, we will talk in depth about how to build enduring, mutually beneficial partnerships with software vendors, so we'll keep this part brief:

*The most obvious benefit is time to value.*

You don't need to build a custom solution—you're purchasing one that's ready to go. In most cases, you can start within minutes or hours and quickly gain new understanding of your production systems.

*Commercial solutions generally offer more streamlined user experiences than open source alternatives.*

Open source tools are typically designed as building blocks for you to assemble yourself. Commercial software tends to be more opinionated—a finished product with specific workflows. Product designers work to streamline ease of use and maximize time to value.

*You offload maintenance and support to your vendor.*

Rather than burning your own engineering cycles, you pay someone else to handle that burden. You benefit from economies of scale—your vendor operates the same infrastructure for hundreds or thousands of customers, driving down per-customer costs and improving reliability through shared learnings.

*You potentially gain access to domain expertise.*

By purchasing tooling from a company whose core competency is solving observability challenges, you can tap into years of expertise addressing the problem in various ways. This commercial relationship can provide invaluable guidance that would otherwise take years to build internally.

## The Risks of Buying Commercial Software

The benefits of buying commercial software don't come without risk. Importantly, some of the bigger risks are baked into the largest benefits:

*By delegating responsibility to your vendor, you risk not developing your own in-house observability expertise.*

In this way, that last benefit of buying commercial software is also one of the bigger risks. By simply consuming a ready-made solution, your organization may not dive deeply enough into the problem domain to understand how observability applies to your particular business needs.

*Commercial products are built for wide audiences.*

While they have opinionated designs, they're also generally built to adapt to various use cases. By definition, commercial products are not specifically built to accommodate your special needs. Features critical to you may not be priorities for your vendor, leaving you waiting for functionality essential to your business.

*You don't control them.*

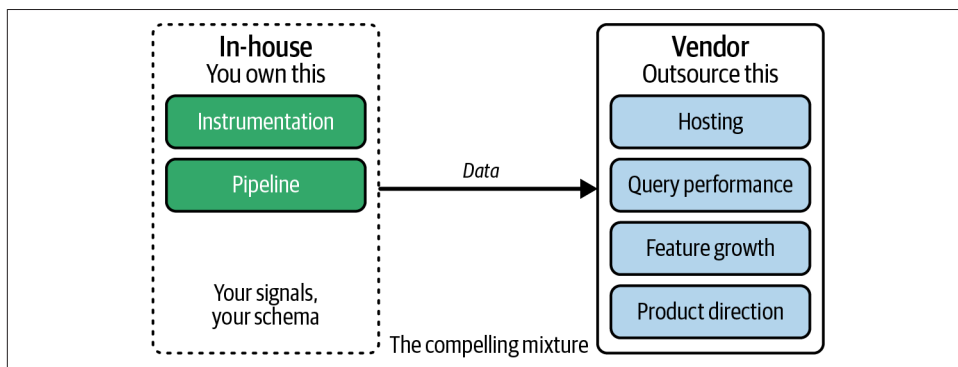
The vendor may get acquired, run out of money, decide to pursue a different target market, or have massive, days-long reliability outages of their own. (Then again, your company might too. Nothing is guaranteed, not even tomorrow.)

Fortunately, there's a way to mitigate most of these risks and develop your own in-house expertise while adopting commercial observability tools. Because this is not the kind of all-or-nothing situation where you have to build everything or buy everything. You can, and probably should, do some of both.

## Build Versus Buy Is Not a Binary Choice

The choice to build or buy presents a false dichotomy. A third option is to buy *and* build. We recommend this approach for most organizations.

Buy an underlying vendor to do the heavy lifting, but craft libraries, tools, and interfaces for your engineers that feel friendly, idiomatic, and do as much automation and wrapping as possible (as shown in [Figure 29-3](#)).



*Figure 29-3. A compelling mixture: outsource the hosting, query performance, feature development, and product direction, but keep instrumentation and pipeline in house.*

This is the platform model, after all: build things that are unique to your company, but shared across teams.



## This Is Where Your Observability Team Comes In

Observability needs an owner, whether you buy from a vendor or not. It's a massive expense and force multiplier, and it will be a load-bearing pillar of any push to adopt generative AI workflows.

Buying a vendor solution doesn't mean your company won't need an observability team. It means that instead of spending their time tuning ELK stacks and Prometheus metrics and operating pipelines, the team acts as an intermediary integration point between the vendor and your engineering organization, making that interface seamless and friendly.

We know from industry research that the key to success when outsourcing work is to carefully curate how that work makes its way back into the organization.⁶ This is what your observability team is here to do.

Your observability team should:

- Write libraries and useful abstractions.
- Standardize naming schemes across the organization.
- Handle updates and upgrades gracefully.
- Consult with other engineering teams on effective instrumentation.
- Manage vendor relationships.
- Make decisions about adopting new technologies.
- Educate teams about observability practices.
- Deliver training and sample code.
- Integrate telemetry data sources if your company still has three pillar tooling.
- Look for ways to simplify and reduce friction and costs.

For organizations considering building their own observability stack for customization to specific needs, this approach is especially effective.

## Get the Best of Both Worlds

Using an observability team to build on top of an extensible vendor solution minimizes sunken costs that distract from delivering core business value. Your team can build integrations that mount the wheel to the fast-moving car you've already built, rather than reinventing the wheel.

---

⁶ Nicole Forsgren, Dustin Smith, Jez Humble, and Jessie Frazelle, *Accelerate State of DevOps 2019*, (DORA, 2019).

The key to enabling that balance is to ensure that your team uses a product that has a well-developed API that lets you configure, manage, and run queries against your observability data programmatically. Last-mile problem-solving requires substantially fewer resources than building everything yourself.

## Important Caveats

Given the breadth of this topic, keep in mind that we are talking straight down the middle of the industry. We are trying to address technology businesses that are extremely price-sensitive as well as companies that are all but immune to cost.

If your company is an outlier in some exceptional way—maybe you’re a hyperscaler, maybe you’d be fine with monitoring circa 2009 (if it came with a 2009 price tag)—our advice may not apply to you. That’s fine. We trust you know your business better than we do.

There are three key caveats to keep in mind, accordingly.

First, there is currently *no large commercial offering* for monitoring capabilities at monitoring price points.⁷ There are some smaller shops that seem to offer this kind of thing, but you’ll need to do your own independent research. If your needs are simple—checking if services are up, down, or slow—and your architecture is straight-forward enough to reason about in your head, you may need to drive off the beaten path a bit. The major observability vendors have largely abandoned this segment of the market in favor of more complex (and expensive) use cases.

Second, the *build-versus-buy quadrant may break down* at hyperscale. If you’re with Google, Meta, or another hyperscaler, the economics do shift. At that scale, it may become cheaper to build your own solutions, even for commodity products. That’s why Meta builds its own custom power grids.

We hear a lot of companies fret over whether any vendors can possibly handle their scale. Let’s put it this way: if you aren’t building your own power grids, you’re probably fine. (Even for hyperscalers, building their own products is usually driven more by wanting control than scale.)

Third, there is *no one-size-fits-all solution* at enterprise scale. While you should always seek to avoid unnecessary duplication of data, every strategy to manage costs at scale is going to involve tiers of service. You might use:

---

⁷ Plenty of large observability vendors offer a free or cheap, entry-level price point (as in “the first hit is free”), but not one that scales up to large traffic volumes.

- High-fidelity observability for your most critical applications, anything customer-facing, money-handling, or under active development
- Traces and tail sampling for customer-facing applications that are stable and not under active development
- Basic metrics and synthetic health checks for noncustomer-facing, less critical applications
- Open source solutions for specific use cases where commercial offerings don't justify their cost

One caveat to our list of caveats: in our experience, *lots* of people believe their company is exceptional for some reason or another...which often turns out to be completely ordinary.

## Conclusion

Observability is not like any other type of tool. It's the software you use to understand your own products: the code that makes them, the users that use them, and the intersection of all three.

Ten to fifteen years ago, every observability SaaS option was basically a hosted version of what you could run yourself. Vendors took open source monitoring/logging tools, ran them as managed services, and charged for the convenience.

That's no longer true. The products available commercially have meaningfully diverged from what's available via open source. For most companies (or at least most of the companies with more money than time), the default position is to invest in a commercial observability solution as a baseline, and focus engineering time on building business-differentiating features. Build custom integrations and abstractions on top of commercial tools where needed, but don't rebuild the entire platform. For most companies, observability falls into the lower-left quadrant: commodity software, low business value.

The exceptions—companies that should seriously consider building—are those with:

- Truly unique requirements that no commercial vendor addresses
- Scale so massive that custom solutions become economically viable
- Observability as their core business capability
- Regulatory or security requirements that prevent a company from using external vendors, or even running other vendors' code on their systems (if these exist)

For everyone else, the path forward is clear: buy your observability platform, build your integration layer, and focus your engineering talent on the problems only you can solve.

This leads us to the next chapter: how to evaluate and work effectively with observability vendors.

---

# The Art and Science of Vendor Partnerships

The buying process for observability is *broken*. The world has changed so fast, and process and alignment and structure have not caught up.

And observability is so inextricably interwoven with your company's pace, feedback loops, ability to see clearly and make good decisions, that if your observability evaluation and procurement process is broken, your company is broken, too. These decisions have ripple effects that linger on for years.

This chapter is not intended to be an intensive guide to the software buying process from start to finish (which would be impossible anyway), but it does offer an introduction to the art and science of vendor engineering. We cover topics like leveraging trust and credibility to extend your influence across organizational boundaries; driving sociotechnical transformations from within; and building enduring, positive-sum partnerships across industry and company lines.

It's good for the company, it's good for the industry, and it's good for you. These are some executive-level skills—one of the rare ways to earn them as an individual contributor (IC)—and they are durable in a post-AI era.

Most software transformations fail. Each one is ridiculously unlikely to succeed. Those that do succeed rely not only on technical excellence and top-down mandates, but on trust and credibility.

In this chapter, we will give you some tools to get started. We'll talk about who your stakeholders are internally, and how you can build trust and credibility with them. And we'll talk about who your stakeholders are on the other side of the transaction, and how you can build reciprocity with them.

We will also give you some guidance on the trickiest parts of the process and some common pitfalls: how to communicate up to C-levels, how to design a meaningful

proof of concept (POC), how to transition and hand off to another team at the end, and the importance of encoding the relationship using service level agreements (SLAs).

This may not sound like glamorous work, but it's engineering: it's hard, and it's meaningful. Few do it well, but those people are some of the most influential and effective in the industry.

## Does Buying Software Even Count as Engineering?

There's an unspoken hierarchy of tasks among software engineers, influencing who we look up to as well as our leveling docs and promotion packets. At the top: new code, new features, new products. Lower down are less glamorous tasks, like refactoring, tests, and infrastructure. "Buying software" isn't on the list—*is it really even engineering?*

You bet it is. Or it can be, and it *should* be.

In fact, we will argue that vendor engineering is one of the most elite engineering disciplines,¹ with an effort-to-impact ratio that is second to none.

The decisions you make about which observability vendors to partner with will affect the productivity of every engineer and the experience of every customer. If you're hungry for impact, it's hard to beat that.

We'll talk more about what we mean by "vendor engineering" in just a moment. But first, here is a story from Charity to illustrate the point.

### A Story About Engineering Influence

Hi, it's Charity again. I'd like to tell you a story about one of the most effective engineers I've ever worked with.

Let me set the scene. It's 2013, and at least 30 people are crammed around a massive table in the largest meeting room in MPK 16 (the Menlo Park Facebook campus): engineers, managers, executives, and sales people representing Parse, Facebook, and MongoDB, including the MongoDB CEO and CTO. And me.

I didn't know why so many people were there and I didn't care. I was *mad*. We were using the free version of MongoDB at Parse and it was crashing constantly, sometimes unrecoverably. The memory-mapped storage engine would grow and grow indefinitely, so we kept having to RAID together more and more EBS volumes.

---

¹ At very large enterprises, this work often happens at the principal, distinguished, or chief architect level. We apologize for minting (another) new term, but technical decisions across company lines is something that almost every company needs to do now, not just the ones large enough to have an architect track.

We'd paid tens of thousands of dollars for MongoDB consultants who turned out to know less than we did. We were constantly being woken up by an array of crashes and data corruption bugs; on-call had been miserable for months. I was in that room for one reason: to make sure MongoDB knew how mad I was and how much I thought their software sucked.

In retrospect, I understand more about that scene. The MongoDB execs were there to see if they could get their toe in the door at Facebook. The Facebook execs were there because this was the first meeting with an important vendor for an acquisition they'd just made, and they understood better than I did how much Parse's future was bound up with MongoDB's. They knew this was an important relationship for them to build.

I didn't know any of that. I just wanted my bugs fixed.

I did most of the talking for most of the meeting, visibly irritated. The MongoDB contingent took notes, made sympathetic noises, and then asked Facebook many probing questions about whether there might be any potential use case for MongoDB at Facebook. (There was not.)

Toward the end, some guy on the other end of the long table spoke up. He was polite and soft-spoken. He asked a few follow-up questions about the problems I described, and then asked about their roadmap and the plans for their storage engine. He clearly knew his stuff, and when he was talking, everyone looked up and leaned in. Nobody interrupted him. At one point he said, "I always listen to operations. You *have* to listen to what your operators are telling you" (which won me over completely).

In the hallway afterward, my boss said, "Do you know who that was? That's Mark Callaghan, one of the most legendary engineers at Facebook. I heard they invented E9 [the highest engineering level] just for him."

Mark took a personal interest in our database woes. He started coming along to MongoDB events and building a friendly relationship with the Mongo folks. He looked into the MMAPv1 btree engine, and began lobbying for a pluggable storage engine interface. MongoDB agreed, likely because Mark secured resources from the RocksDB team at Facebook to develop the first client storage engine in tandem, and Parse agreed to be the first customer and guinea pig. Mark also brokered an introduction to Keith Bostic and the WiredTiger team, which MongoDB later acquired, and is the default storage engine in use today.

I don't know what all went on behind the scenes, obviously, but it's my personal opinion that MongoDB could not have gone public on the strength of their old storage engine alone. When we rolled MongoRocks (still the best product name ever) out to our replica sets, we cut the footprint on disk by up to 90% and made write-heavy workloads possible for the first time.

This means that Mark not only saved Parse from what we referred to as our imminent "Mongogeddon," he also played a part in helping MongoDB complete its IPO and helping RocksDB become the multihyphenate write-optimized storage wonder it is

today. And he did all of this quietly, behind the scenes, within the space of a few conversations, leveraging his decades of expertise and credibility in the space (and reputation for holding vendors accountable) on our behalf. As a side project.

That's my Mark Callaghan story. If you talk to anyone else who works in databases, they probably have their own Mark Callaghan stories.

In retrospect, working with Mark changed me as a technologist. It shifted my entire perspective on what world-class engineering power and influence can look like.

Mark wasn't a manager or an executive, just a software engineer—always has been, as far as I can tell. He doesn't write masses of code, work the conference circuit, or post on Hacker News. (He's not very good at self-promotion.) Mark's capabilities are an advanced form of what I've come to think of as *vendor engineering*: leveraging influence across company lines to move the industry forward in ways that benefit us all.

## The Difference Between Vendor Engineering and Buying Software

As you can see from Charity's story, true vendor engineering is about leveraging influence to move the industry forward, which is a significant step beyond simply "buying software." This evolution from mere purchasing to strategic vendor partnerships helps explain a fundamental truth of our time: modern tech companies are dramatically more productive than they were 20 years ago. There are a lot of reasons for this, but one of the biggest is that *software companies have gotten better at leveraging each other's work*. This lets each of us specialize and make better products, instead of every company building everything from scratch.

The biggest technical innovations are well-known: cloud computing! APIs! SaaS! Sure. But there's a whole messy middle of decisions we don't talk about as much: how we decide which companies to bet on, how to partner with them, and how to drive outcomes that benefit everyone involved.

Easy to say. *Hard to do*.

Vendor engineering is about wielding influence without authority across company and industry lines. Buying software just takes money, while vendor engineering takes technical depth and breadth, strategic thinking, and a solid grasp of business priorities as well as the levers to drive organizational change.

## Training Grounds for Executive Skills

If that sounds suspiciously like an executive job description, you're not wrong. These are skills that most engineers only get to practice or train in on the management track.



Vendor engineering is an opportunity to build credibility and trust in key relationships around the company, and to learn how to navigate complex organizations and operate their levers.

These are durable skills in an era where AI has begun to generate more lines of code than people do. If you believe that engineering is about solving business problems with technology, these are some of the biggest engineering problems of all.

The best vendor engineers end up shaping entire industries, holding vendors to account, pushing them to be better and do better. We can't all be Mark Callaghan, changing the trajectory of multiple billion-dollar companies with a conversation or two (even *Mark* took decades to become Mark Callaghan). But for the skill of translating between tech and business, speaking to C-levels and engineers, and representing stakeholders at all levels—there are few training grounds as intense and educational as the software purchasing and procurement process.

If this kind of work interests you, here are some entry points and things to know up front.

## What to Know Before You Start

It can be challenging to write about the software purchasing and procurement process, because it seems like every company has a different process, for every different tool, every different time. That's not entirely fair (larger companies in particular tend to be better at imposing some rigor), but there's no getting away from it: there's a lot of hyper-local variance.

This can be tough on vendors, but it's great for you. You should be able to find an entry point without much difficulty.

### Find Local Guidance

Start with the docs. Your company probably has some existing policy documents to structure and guide the software purchasing process.

But docs are never enough. Find the local experts, people who have done this before and are willing to share their expertise.

If you're a staff engineer who wants to grow into this kind of work, offer yourself up as an apprentice to someone like this. Offer to run their errands and do legwork in exchange for the chance to watch them up close and learn. This arrangement benefits everyone.

## Understand Your Role

If you've signed up to drive the evaluation and purchasing process, your job is to *keep the machinery moving*. There's a certain amount of project management involved, which is hard to outsource. That means it's yours.

Your purchase will likely flow through security review, legal review, and technical architecture review. Each has different timelines, different concerns, and different needs for context. Your job becomes connecting specialists on your side to the vendor, sharing context about why this purchase matters, helping reviewers prioritize, and providing realistic timeframes about how long each stage will take.

The broad lesson: assume nobody else cares enough to keep the purchase moving. *You're the sheepdog*. Stay on top of every stage, provide regular updates to your stakeholders, escalate when needed, and accept that this may consume significant time.

Let's take a look at each of those stakeholders, and what their department typically needs to know or care about when it comes to a software purchase.

## Mapping Your Stakeholders

Software purchases at scale involve far more stakeholders than seems reasonable, each with legitimate but often conflicting concerns. Stakeholder management is one of the reasons that vendor management has such a high difficulty score. You aren't just pushing packets around; your job is to *understand*.

For each stakeholder, ask yourself what are their needs, what are their red lines, where are they coming from? The more you understand their point of view, the more you can partner with them creatively to navigate obstacles. This is what makes vendor engineering an *engineering* problem, not just project management.

## Engineering

Most developers aren't deeply opinionated about observability vendors. Few feel strongly attached to specific tooling. This cuts both ways—less political resistance, but also less organic enthusiasm unless you can demonstrate concrete improvements to their daily work.

The key to winning engineering support is showing the tool solving their specific problems. This means getting involved in their codebases, understanding their pain points, and demonstrating genuine improvements to the things that hurt them today.

It's hard for anyone who isn't a software engineer to get other software engineers excited about observability, which is one of the most compelling reasons for an

engineer to drive this process. Show, don't tell. Find some champions; you'll need them. They exist. Follow the pain.

## Management

Engineering leadership wants to know how a software purchase fits into the broader technical direction, what opportunity costs you're taking on, and how you'll drive adoption across the organization.

They typically control the organizational levers you will need to access in the migration stage. Feed them written progress updates to keep the drumbeat going.

They need to know what you expect their teams to contribute, how this intersects with their roadmap, and how much of their existing work they'll have to deprioritize. Ideally, you'll have had a chance to rack up some major tech wins for them, so they will be looking for excuses to prioritize this work instead of dragging their feet (we will talk about building champions in a moment).

## Finance

Finance stakeholders care about cost management, budget predictability, tooling consolidation, ROI, time to value, and contract terms.

With finance stakeholders, you need to translate your purchase or budget requests into concrete business outcomes. How does your proposal reduce customer-impacting incidents, accelerate development velocity, or enable new capabilities that drive revenue?²

## IT/Ops and Security

Internal IT and security teams evaluate system alignment, cybersecurity implications, compliance requirements, authentication integration, deployment complexity, and ongoing support burden. In regulated industries, these concerns intensify.

For observability specifically, security teams often get heavily involved because of the nature of data flowing through the system. Customer data, business logic, system internals—all of it becomes visible in your observability platform. Security teams need assurance about data handling, retention, access controls, and compliance with regulations like General Data Protection Regulation (GDPR) or industry-specific requirements. (For more on handling highly secure instrumentation data and dealing with compliance, see [Chapter 31](#).)

---

² As engineers, we often find deep discomfort in imprecise estimates backed by handwavey guesses; it feels almost like *lying*. Well, we have to get over it. Finance is used to dealing with handwavies. It's fine. Think of it more like a directional orientation than a precise commitment.

Most of what falls under the rubric of compliance, regulation, and security can be surprisingly flexible—as long as you have the right story and evidence for your auditors. This is impossible to do without active assistance from security.



If you keep hearing “no” by proxy, as in “they will never allow that” or “they are not going to like this” or “they won’t let you,” then figure out who “they” are and go talk to them. Never accept a proxy no.³

## Procurement

At large organizations, you’ll work with procurement specialists—and experiences vary wildly. Some procurement departments are efficient partners who understand their role is facilitating good purchasing decisions. Others seem designed primarily to prevent purchases from happening, drowning you in process while providing little actual value.

It doesn’t matter. Treat procurement like your friend. Find out what’s holding them back, what their biggest pain points are, and see if you can help them out. See if you can help deliver a win for them that looks great on their performance review. Definitely do *not* exclude them or route around them just because they make your life troublesome (even if that is what they do).

## Communicating Up the Ladder

When decisions require C-level approval, you probably won’t be in the room. Structure your case and put it in writing so your boss or executive sponsor can effectively advocate up the chain. Frame everything in business outcomes rather than technical details,⁴ as discussed in Chapters 27 and 28.

No matter how much your boss insists that artifacts don’t need polish, if there’s any chance they’ll get escalated up the ladder, make them presentable. Executives trust polished slides and documents far more than engineer-quality artifacts, however technically sound.

Surprises kill deals. You don’t want anyone in that room to be surprised by anything they learn there. Go overboard in making sure bad news gets surfaced early on and dealt with, objections handled satisfactorily, stakeholders appeased, obstacles noted, and contingencies planned for—no surprises.

---

³ P.S. It’s *usually* security.

⁴ A list of “major concerns” with green checkboxes saying “we dealt with these and they’re not roadblockers” goes a long way too.

The one thing all internal stakeholders hold in common is this: *you are all on the same team*. You all want your company to succeed, and you are all working together to achieve that goal. Trust and credibility are the key to successful internal transformation efforts, so let's talk a bit about what those are and how to build them.

## Trust and Credibility with Internal Stakeholders

Mapping your stakeholders reveals the “who” and “what” of a purchase, but building trust and credibility provides the “how” for actual execution. While these two terms are often used interchangeably, they represent different currencies in an organization. Understanding the distinction is the difference between being a strategic partner and being just another petitioner.

### Credibility Versus Trust

Credibility is built when you assert something is true, and it turns out to be true. This is where your technical expertise translates into an asset. When you assert that something will or will not work, do people believe what you say? If you say one thing and someone else says another, who gets believed? It will speed things up if you are credible, so be conservative in your claims, and be cautious about speaking for others.

Trust is built when you say that you have someone's best interests at heart, and your actions prove you do. This is rarely a technical consideration. When you need someone to trust you, extend yourself first. Do something to demonstrate that you understand their needs and concerns, and you're willing to earn it.

Ideally, the change you want to drive is one that will genuinely make your stakeholders' lives better. Find those points of overlap and prove them out. If they invest the time to get creative in helping you solve your problems, will they also be indirectly solving their own problems? That's what we mean by “alignment.” If you can't show alignment with a stakeholder, the job gets harder. Then you are just a petitioner—one of many.

Trust is grease in the machinery; it opens doors, attracts allies, and soothes the organizational immune response. It builds up over time, increases with use, and can be lent around like capital. One of the biggest reasons to work on improving your credibility and trust is so you can lend yours to causes that you care about.

### Building Trust with Stakeholders

How to build trust with stakeholders is not mysterious: figure out what makes them win and help them achieve that. What OKRs (objectives and key results) do they have, and which ones are they most worried about? Are there any chronic sources of pain or threats that you can make go away?

Do some of the grunt work. Get your hands dirty. Facilitate key interactions that could be helpful. If you already have influence in the org, put your name on any “this will turn out well and I’m sticking my neck out to make sure it happens” type of lists on their behalf.

With these stakeholders, you’re the supplicant. You represent more work, more risk, and more exposure, and you are just one in a long line of questionable supplicants they have to deal with. It’s on you to earn their trust.

## Breaking Trust with Stakeholders

One of the most reliable ways to build trust is to tell the truth, especially when it doesn’t portray you in the greatest light or if the news is bad. Surprises are a direct breach of trust; they leave execs (and other stakeholders) feeling unmoored and suspicious.

To navigate the setbacks effectively:

- Never sit on bad news or big, scary numbers. When you learn something that is likely to upset someone, tell them right away.
- Being forthcoming is twice as important if you bear any personal responsibility for the situation at hand.
- Eliminate surprises. Again, surprises are a breach of trust. Bad news, big mistakes, and disappointment are par for the course; that’s life. But unexpected shocks at the C-level will likely tank a deal.

With internal stakeholders, remember that you are the supplicant: you need something from them, and you will work hard to get it, but you are all on the same team with the same overarching goal. The dynamic with your vendor relationships is exactly the opposite.

## Trust and Reciprocity with External Stakeholders

While internal stakeholders share your company’s overarching goals, the dynamic with external vendors is reversed: they want to make a sale, and you want what’s best for your company. Most vendor relationships are transactional, but high-functioning partnerships operate like two teams within the same organization—a state only reached through transparency and reciprocity.

## Building Vendor Relationships

With your coworkers, you should assume you all want the same thing—for your company to succeed. When there is conflict, it’s a disagreement in good faith about how to achieve this.

You cannot assume the same of your vendors. They want something from you, and they will work hard to get it. With your vendors, you should assume you want different things. They want to sell you something. You want the best outcome for your company. These things may converge, or they may not.

Remind yourself that vendors will be helpful and useful for exactly as long as you appear to keep making progress toward their sale, and no further. If a better outcome for you lies elsewhere, you are on your own to find it—and if they know you’re looking, they may try to actively mislead you and thwart you. (Yes, this includes lying.)

This is normal. This is capitalism. This is the working basis of most friendly, productive contractual partnerships. It is not how *all* vendors operate, but it is the right baseline assumption.

## The Vendor Trust Coefficient

Over time, as you engage with a vendor and a sales team, you will build up a mental model of how they operate and whether they care about what’s in your best interest—almost like a trust coefficient for each vendor.

Will they answer you honestly, even when it reflects poorly on them? Do they actively lie to you, or only mislead by omission? How much time do they devote to spreading FUD (fear, uncertainty, and doubt) about other vendors, or trying to use you to set “traps” for the competition (feeding you questions designed to elicit an answer that favors them)?

In short: how shady do they act?

You will want and need their help. Vendors are experts in their field, after all, and they’ve run a ton of POCs. There’s no reason not to lean on them. But:

- Are they working to help you assess *your* critical needs, and to design, instrument and run a POC to exercise those needs, including the rough edges and hard cases? It’s not shady to help you figure out how to map your problems to their solution, or to show the value they can add.
- Or, do they helpfully offer to do the work for you in ways that elide their product’s weaknesses or approximate your hardest problems? Do they reassure you that test datasets are just as good, performance is never a problem, or sway you away from use cases that might show their product in a bad light? That means they are *actively working against your best interests*.

Just because a vendor is a little shady, doesn’t mean you can’t work together profitably. It just means you have to keep a closer eye on them. Double-check any information they give you, especially about their competitors. (Alternatively, just assume that any

time they mention a competitor, they are lying and a little scared.) Make sure the work is done to *your* specification, not theirs. For every lie you catch them in, assume they've told a hundred more.

After all, you have a professional responsibility to act in your company's best interest, and that does not change—no matter how lovely your sales contacts are.

## The Reciprocity Principle

We believe that the best vendor–customer partnerships are built on the rule of reciprocity. You should want to be a customer that your vendor will fight to retain and make happy; your vendor should want to provide a service that is valuable enough for you to fight to keep and want to pay for.

If both of you feel like high-fiving after you negotiate a new contract, that is a promising start to a great partnership.

Not everyone agrees, and that's fine. We've seen engineers gloating when they think it costs their vendor more to run their workload than they are being paid to run it—the vendor is serving them at a loss, in other words. It's always fun to get something for free, but in a B2B relationship, it's usually more valuable to you if your vendor actually *wants* your business. (And if the product is good, presumably you want to keep using it, which means they need to stay in business.)

The reverse can happen, too. Some vendors have a pricing strategy that is designed to “leave no money on the floor.” This means finding the exact point at which the price is so painful it will force the customer to leave, and charging right up to that point. This might not actually sour the relationship—as the people who pay the bills are usually different from the people who use the product—but it can lead to internal tensions between payers and users over whether the service is worth it.

A certain slice of enterprise tech is all but impervious to price; what they do care about is trust, reciprocity, and the ability to influence the vendor's roadmap (or strategically plan around it).

When your company is really well-aligned with a company you're buying from—and there's a high degree of respect, transparency, and reciprocity on both sides—it can come to feel like your successes are their successes, and their successes are yours. That's the dream. It doesn't usually happen. But when it does, it's incredible.

That kind of trust can only be built over time. But it's not where you start out—and, fair warning, it's not where you usually end up, either.

## How to Influence Another Company's Roadmap

The ability to influence a vendor's roadmap is one of the more advanced forms of vendor engineering—one that transforms you from a passive consumer into a



strategic partner. This comes in many forms: product roadmap updates, feature previews, alpha or beta access, executive dinners after events where long-term plans get discussed off the record. Access like this gives you confidence that you and your vendor are strategically aligned for the foreseeable future.

Influence on another company's roadmap can be earned or cultivated in a number of ways. Briefly (this topic alone could fill a book!), these include:

- Being an important customer, valued for your logo, your contract size, or both
- Strategic alignment—your companies are going in the same direction, there may even be opportunities for cross-selling or co-marketing
- Sweat equity—are you willing to commit some of your own resources to test or develop together?
- Technical credibility and personal relationships, both formal and informal
- The levers that drive organizational commitments, such as SLAs, OKRs, and KPIs

In [“A Story About Engineering Influence” on page 534](#), about how Mark Callaghan helped to get the pluggable storage engine work on the MongoDB roadmap, you can see most of these levers at work.

After the Facebook acquisition, MongoDB really wanted to win Facebook as a customer, so they started paying a lot more attention to Parse. Mark was able to leverage his technical credibility, as one of the engineers who built the pluggable storage engine for MySQL, to argue that it would be a game changer for MongoDB to do the same. He also introduced them to key figures in the storage engine community, and used his influence within Facebook to get the MongoDB storage engine support on the RocksDB roadmap. He showed up at their conferences, spoke when invited, joined the MongoDB Community Champions (formerly Masters) community, and built relationships in the company and community.

This is not a one-way street, by the way. Vendors also need the ability to influence their customers' roadmaps at times, starting with the migration. They need to map out the organization and figure out who can actually sign the deal, and what the migration process should look like. This is a normal part of de-risking any deal. A migration that doesn't appear on a customer's roadmap or on the OKRs of any of their engineering teams will probably never happen.

And *never* underestimate the power of personal relationships. A reliable back channel can be the only way to cut through layers of formality and confusion and find out what the hell is going on.

While technical credibility and personal relationships are valuable, those informal back channels are fragile. For influence to endure, you need to move from the

handshake of strategic alignment to the contract of operational reality by codifying your expectations in a formal agreement.

## SLAs Encode the Vendor–Customer Relationship

SLAs are not just paperwork; they are the encoding of your relationship and the agreements you have made. SLAs are to the vendor–customer relationship what SLOs are to the engineering–sales relationship: the assurances that mutual trust can be built on.



An SLA is a formal, legally binding contract between a service provider and a customer that defines the expected quality, availability, and responsibilities of services provided. It sets clear performance metrics (e.g., uptime) and remedies or penalties if these standards are not met.

Don't wait until the very end to think about them, don't forget to get stakeholder buy-in, and *definitely* don't leave the vendors and procurement teams to hash it out together without your full understanding and participation.

SLAs are some of your most effective levers for driving behavior. Well-crafted SLAs can help you influence future roadmap and planning decisions in a company where you have no authority or direct influence, even after you and your negotiating counterparts have all moved on.

You might assume that the more you trust a vendor, the less important your SLAs are, but that would be incorrect. Good fences make good neighbors, good boundaries make good friendships, and good SLAs make good vendor partnerships. They prevent misunderstandings, align incentives, encourage long-term thinking, and even protect the vendor from potential lawsuits (always a relationship buzzkill). They protect you both, so take them seriously.

The topic of buying software and forging partnerships is long and complicated, and we don't pretend to take a comprehensive approach. But there are some things you should seek to clarify from the start, during vendor evaluation—confusion on any of these points can easily derail the entire process or lead to a substandard outcome later on.

## Early-Stage Considerations

Every org-scale sociotechnical transformation that succeeds has achieved two things by the time it finishes: a mandate for change from the top, and a bottoms-up groundswell of support. Execs ultimately have to believe it belongs on the roadmap, ahead of

other important work, and engineers need to see the value and want it for themselves, before you can tap into their creative energy for problem-solving.

In the early stages of a vendor evaluation, you might not have either. This is normal. Your first step should be to get clear on scope, success criteria, and executive sponsor. What kind of support and mission parameters do you already have, and which are you supposed to build?

Here are three very different (fictional) scenarios that help illustrate the point. The first is an insurgency, the second is a consolidation directive, and the third is an open-ended exploration:

#### *Fort Pillars Financial (the insurgency)*

This Fortune 100 financial services organization has run the same three pillars observability vendor for 17 years at \$40 million annually. Their 8,000+ engineers have no direct access to production telemetry: all requests flow through centralized operations. Mean time to resolution (MTTR) averages six to eight hours, and is primarily spent correlating signals across siloed systems—by hand.

Your vice president has commissioned you as her agent of insurgency: find a modern alternative, recruit two or three product teams for a POC, and generate compelling ROI evidence. She needs quantifiable improvements in MTTR, developer productivity, and cost to force a strategic reassessment at the C-level. Her words: “Make their eyes pop.”

#### *Tool Sprawl Industries (the consolidation mandate)*

This technology conglomerate pursued “developer empowerment” and a competitive acquisition strategy that resulted in over 40 observability tools across the organization. A recent P0 outage took 14 hours to resolve because the relevant signals were scattered across five different systems. Annual observability spend exceeds \$60 million with massive redundancies—and also, at the same time, massive gaps.

You’re a principal engineer with 16 years at this company. The CEO, CTO, and CFO have all signed off on a consolidation directive: you are being asked to assess all use cases and tools, recommend one modern platform to replace them, and develop a migration template. The budget has been preapproved with an expectation of 40–50% cost reduction. You have 90 days.

#### *Agent Smith Systems (exploring on the cutting edge)*

This pre-IPO software company adopted OTel early and has consistently demonstrated technical craft and rigor. AI-assisted development has increased the rate of new code deployed 5–8x over the last 18 months. But their observability infrastructure can’t keep pace: the vendor’s Model Context Protocol (MCP) integration times out, hallucinates metrics, and can’t handle AI-generated code paths (or anything else with high cardinality).

They are starting to operationalize agentic workflow, but getting anxious about going any faster without observability that can validate as fast as they ship. They haven't really thought about whether the goal is to replace their existing tools or simply augment them. A big migration wouldn't frighten them. But right now they are just trying to prove that something, anything, can work at the speed of AI at production scale.

All three of these scenarios are quite common, but each calls for a radically different approach when it comes to engaging with internal engineering teams, designing a POC, and aligning around success criteria.

Before the first line of code can be instrumented, you must also clear the final two hurdles: proactively managing internal friction that may arise, and making the foundational OTel decision to protect your organization from vendor lock-in. Let's look at each consideration.

## Define the Scope of Work

How you frame the problem and scope the work is often the determining factor in whether or not the work succeeds.

Aim too small and, even if you succeed, you'll have missed a rare opportunity to solve the bigger problem; at best, you or someone else will just have to go back and do it all over again. Aim too big, and you will inevitably founder on the rocks of sunk costs, scope creep, and territorial pushback.

The Goldilocks size will be set by how much air cover you have (from how high up), how much evidence you have already assembled to prove your case, and the populist ratio of anger to inertia among engineering teams.

## Identify Your Executive Sponsor

Find out who can sign the contract. This sounds basic but trips up many purchases, as the nominal decision-maker often lacks signature authority. (Also find out who has budget authority. This may or may not be the same person.)

If there are multiple layers of management between you and the person who can sign the check, make sure that every hop of the chain stays informed and engaged.

Try to learn as much as you can about how the decision-maker is likely to make their decision. If you have a clear mental model of what that person cares about, objections they have thrown in the past, their top priorities and goals, complaints they typically register, and so forth, you can proactively address those things. Again, surprises are never good.

This can be challenging, because people are not always very consistent or self-aware, but you have to try. Any exec who wants to say no is going to say no, and will

find a way to retroactively justify their decision. Don't assume that silence means agreement. Farm for dissent, early and often.

## Define Success Criteria Before You Start

The biggest POC failure mode is never defining what success looks like. Too many POCs get run by ICs or managers who lack budget authority and decision-making power. By the time the results trickle back to an exec who can actually approve the purchase, context has been lost and the evaluation criteria feel arbitrary.

Success criteria should be specific and measurable. They should translate the problem you are trying to solve into functional requirements, architectural characteristics, and budgetary guidelines. Make your case in terms of the business, as discussed in [Chapter 26](#).

Most critically, someone close to the budget needs to understand these criteria and agree they're the right ones. Figure this out now, not after three months of hard work.

## What to Do When Your Coworkers Perceive You as a Threat

Everything we said about how “you and your coworkers are on the same team” is true. However, this does not prevent you from having incompatible opinions about what is in the company's best interest. When people feel threatened, they are rarely at their best. Two common triggers you may accidentally set off are threats to their tools or threats to their turf:

### *Threats to their tools*

Sometimes a tool is more than a tool—it becomes part of someone's identity. If someone (or a team) has been using the same tool for a very long time, they may take pride in their expertise. They may be active in that tool's community, be widely known as an expert and sought out for their opinion, or write highly regarded essays and talks about it. They may genuinely believe it's the best tool for the job, and they might even be right. But they are usually ill-equipped to evaluate alternatives with an open mind.

### *Threats to their turf*

This one can be tough to avoid with observability, because observability is everywhere. You probably already have some tools, and those tools have owners who take pride in their work. In [Chapter 28](#), we provided some advice on how to acknowledge the heroes who got us here, and how to try and bring them along with you in a way that delivers a win for them too.

When ego and expertise are on the line, it's easy to lose sight of the common good. When people feel threatened, they are very likely to reach for any levers they have to sabotage your work, go around your back, escalate above your head, or get you shut down. They will notify their favored vendor of the threat (i.e., you) and work

whatever angles they can to eliminate that threat, up to and including CEO-to-CEO contact, if possible. Don't be naive about this.

The best advice we can give you is to seek counsel from local experts, execs, and people who have done this before, and never stop trying to extend a hand and build positive sum outcomes for everyone. Yes, even the backstabbers.

There's one more decision you should make before you start, but it's an easy one.

## The OpenTelemetry Decision

During vendor evaluation, if you haven't already adopted OTel at the instrumentation layer, now is the time to start. If you have to go through and reinstrument everything anyway, just to evaluate a new vendor, let this be the last time you need to do that.

If a vendor discourages you from using OTel, *they are trying to lock you in*. If a vendor locks you in, they are reducing market pressure to improve their own product.⁵ You have been warned.

The most important technical contribution to any software buying process comes at the POC stage. Converting your business case into a meaningful POC—one that exercises the hardest questions your engineering teams are facing—requires a deep understanding of your team, your technical problems, and the needs of your business.

## Design and Run a Meaningful Proof of Concept

A POC for observability software should be a load test of your hardest problems and representative use cases in production. Your job is to build a POC and set of criteria that are sturdy enough to demonstrate value and support a swift decision.

If more than one vendor is being evaluated, part of your job is to develop a rubric for comparing them. What does each vendor do best, and how much does that matter to you? Maybe vendor A is exceptional at a particular use case, but weak at two or three others. If their strong suit happens to overlap with all of your revenue-generating requests, it might still be the best option, but if their strong suit is only 20% of what you care about, you can easily rule them out.

Another place that is both art and science is in deciding whether to instrument your code to see each vendor at their best, or apply the same apples-to-apples instrumentation across all the vendors you trial. Apples-to-apples may take less work from your engineering teams,⁶ but resist the urge to shortcut. Instrumentation is not a bolt-on;

---

⁵ And if they say “sure, you *can* use OTel, but our bespoke integrations are more mature/performant/battle-tested/whatever,” remember: it is that way because they want it to stay that way.

it's part of the solution (yes, even when it's OTel). When you are evaluating multiple vendors, you need to see each vendor at its best, or it isn't a true evaluation.

If you are running the POC process in partnership with multiple engineering teams, coach each of them on how to do this, and make sure each team is running a real, representative load sample in production. Trust but verify.

## What Does a Good Proof of Concept Look Like?

POCs, like the buying process itself, are a bit like snowflakes; hard to generalize about, hyperlocal, and each one a little different. Your organization probably has some existing prior art in the form of templates or vendor checklists.

However, as people who work for observability companies, we will take the liberty of being a bit more prescriptive. In our experience, successful POCs for observability tools usually include the following elements:

### *Description of your pain*

Include a short description of the challenges that are causing you to look at alternative solutions. Include any precipitating events, if such exist; this might mean a pattern of lengthy incidents that violated customer SLAs, or any other modernization initiatives in flight.

### *Your definition of success*

This usually looks like a list of primary outcomes you are trying to achieve. For example: understand each customer's experience, shorten feedback loops for developers, add observability to unknown-unknowns without incurring additional cost, and so forth.

### *Required capabilities*

Make a list of the kinds of questions you rely on, and the questions you wish you could ask more easily. Try to avoid including solutions in your requirements (e.g., “a query language that supports joins”), but if you're struggling to list which capabilities you wish you had, you can just write down any workflows that are currently lossy, laggy, frustrating, or time-consuming.

### *Scope of the POC process.*

Write down what the evaluation will focus on—which teams, workflows, hard problems or edge cases, datasets—as well as what is out of scope for the POC.

---

⁶ Another great reason to use OTel: AI combined with OTel can bring the burden of reinstrumentation down substantially. The same goes for migration.

The POC should be timebound (you don't have all year to work on this) and well-resourced, with both dedicated in-house support and vendor support lined up for the duration.

Vendors will usually offer to write the requirements for you, to “save you time.” True, it's usually much faster and easier for them to do it. But it's your responsibility to make sure that what is written down is actually reflective of *your* real needs and *your* hardest problems, and not composed expressly to make this vendor the obvious choice.

If you can, try to get your colleagues to agree up front that if the tool passes all the POC tests, you will proceed based on the data—and not relitigate endlessly based on the feelings or vibes of too many stakeholders. This may be unenforceable (execs can always change their mind), but it sets the right expectations and makes people take their part more seriously.

## Structure Your Technical Evaluation

Each of your technical stakeholders is going to have their own concerns, must-haves, and wishlist items. Your job is to validate that the list remains reasonable: comprehensive enough to include the jobs to be done, but not tied up with preserving backward parity at the expense of net new capabilities.

Identify which engineering teams are likely to be users of yours (users of the tool you buy), and which engineering teams are gatekeepers. They have different needs.

If you know you have internal competition or friction over turf with other teams, use the POC to keep everyone honest. Be alert for “capabilities” that are actually poison pills from other vendors, designed expressly to make other tools fail. Define success in terms of the business use cases, not specific features or implementation details.

One of the biggest things to test for in a POC is how well the new solution plays with the old solution. Both solutions will need to run in parallel for a while, so you may need to deploy a lot of instrumentation that's turned off until you can fully migrate off the old vendor. Can you set up dual writes? OTel can help by letting you wire up multiple exporters.

## Validate Pricing at Scale (If Cost Is a Concern)

Most companies (but not all!) are sensitive to cost. If that's you, one of the most important aspects of your POC will be validating the pricing model and getting some ballpark estimates.

This is surprisingly difficult to do without sending all your data through the system. Most teams compromise by sending a representative sample—perhaps a single cluster's worth of data—and extrapolating to full production scale.



The price on the vendor's website is meaningless—nobody pays that. Always see if you can find a customer with an architecture or product similar to yours who is willing to speak freely about how much they pay. Don't forget to ask how many ongoing engineering cycles they devote to managing costs (this can be substantial, as in several engineer-salaries worth).

Vendors can help with introductions and research. If you trust your vendor, this can save you a lot of time and energy. If your vendor is on the shady side, do the sourcing yourself to avoid shenanigans.

## **Build Champions and Advertise Your Wins**

Not every POC is a winner; not every migration is worth doing. Engineers run POCs all the time that they don't particularly care about, and that's fine. But if you have developed a strong conviction that this one is a winner, and adopting it is in the best interests of your company, you sit in a unique position of influence. By developing champions, solving hard problems, and enthusiastically broadcasting your shared triumphs, you can substantially improve the odds of success.

The best POC processes solve real problems for real teams in production. This is by far the best way to build champions—not just out of developers, but also out of their management layers. Don't start with the easy problems; go straight for the toughest, gnarliest knots in your org. Who is understaffed, and can't get out of firefighting mode? Which team has been getting woken up night after night after night, and still can't identify the issue? Which teams are losing good people to burnout and exhaustion? Who has a principal engineer who's been pulled into every SEV0 or SEV1 for the past year?

Follow the pain. Help solve their problems. Turn relieved engineers into enthusiastic champions and advocates for the cause.

Don't let those wins get lost or buried! Start a brag list: write down every time an engineer uses instrumentation and tooling to identify inefficiencies, save the company money, find a bug before a customer does, accelerate time to delivery, decrease MTTR, or pinpoint lurking issues that have been plaguing other people for months.

Help your coworkers do things worth bragging about, then praise their achievements in public. Make sure their manager sees it, and their manager's manager, all the way up the ladder. The bigger the win, the more you celebrate. Make other engineers want to see what all the fuss is about. You want them lining up and begging to get in on it.

## **Maintain Momentum Through Procurement**

You are responsible for keeping the purchase moving through your organizational machinery. If something is held up on your side, say something. Don't just go dark. Sometimes the vendor can be useful in helping unblock you, but only if they know.

Again, you are the sheepdog. Woof.

The deal isn't done until the contract is signed on both sides; be wary of premature complacency. But once the signatures are done—*congrats!* This is huge!

After the deal is signed, it's time to switch gears and start planning the process of migrating and cutting over to your new vendor.

## Migration and Follow-Through

Ha, you thought you were done? What happens *after* the contract is signed will determine whether your investment actually pays off or not.

### Handing Off to Another Team

Shortly after the contract is signed, the people staffing the deal will often (but not always) change over. The vendor side may do a handoff from solutions architects to customer architects, or presales to post sales. On the customer side, this is often where platform engineering or your observability team steps in to start planning a migration path for affected teams.

A bit of turnover is normal and OK, but a lot of turnover on either side can lead to a stalled project and dropped packets.

You may be handing the baton off, but don't drop and dash off. Stick around to make introductions and help draft the migration path, along with timelines and milestones. Share whatever useful bits you've learned from the POC. Set up a regular checkpoint with whoever you are handing off to. Someone needs to continue to ride herd on your side; if it stops being you, make sure the handoff is explicit and known to both sides.

### Most Migrations Have Three Steps

Successful migrations often follow a three-stage structure:

1. De-risk the migration by proving it works with a nontrivial but manageable example.
2. Execute the bulk migration—get everything moved (accept some messiness).
3. Iterate service by service to clean up and optimize.

Migrations at enterprise scale have historically taken months, if not years: no wonder people avoid doing them at all costs. Modern LLM tools have brought these timelines down substantially, usually on the order of weeks—occasionally days. But this will depend to some extent on how practiced you are with migrations. If you've been

using the same tool for the past 15 years, it's going to be a heavy lift; there's no way around it. On the other hand, this is a muscle you need to build.

Plug milestones into your organizational operating system (KPIs, OKRs, whatever), or the work will just get bumped.

Don't try to use an export of the old tool configuration as input to the new tool. It will be full of cruft. Start with clarity around business requirements, then build the new config in the new tool from scratch, using new tooling approaches and workflows.

If you are still connected to the project, your critical contribution is this: present the messy reality as linear progress to stakeholders. Stakeholders want clear narrative arcs. Give them one, even though the actual work involves lots of back-and-forth iteration and problem-solving. Part of your skill in driving migrations is turning messy loops into linear progression that stakeholders can understand and support.

## You Aren't Done Until You Decommission

Adding a tool is easy compared to decommissioning one. The last 10% of a decommissioning project is the hardest and slowest, as only the difficult, scary parts remain.

Ensure you publish a timeline for deprecation and deletion of the old tool, and try to stick to it. Only when the old tool is gone will you reap the full cost benefits of all your hard work.

You have likely moved on to other projects by the time the migration reaches this point (which could be months, or even years later). If so, consider circling back and helping with the dirty work for a few weeks. It could give everyone the boost they need to get past the finish line. (And then: throw a party!⁷)

## Plan for the Next Transition

One last counterintuitive detail: start planning immediately for the eventual replacement of the tool you just bought. Write down a recap of the buying process while it's fresh in your mind. What was harder than expected? What went well? What do you wish you had known at the outset? How effective was your POC? Are there any lingering concerns in your mind about the vendor you chose and their ability to execute? Was there a close runner-up?

Requalify tools and vendors on a regular cadence. Examine cost versus value. When cost approaches or exceeds value, reassess your requirements, evaluate the current market, and consider alternatives. If you find something compelling, set up demos or

---

⁷ When we decommissioned the last unicorn (a Ruby HTTP server) at Parse, we held a ceremony in the Facebook quad where we burned unicorn pinatas with lighter fluid, and then had whiskey and cake. Seriously: celebrate!

run parallel tests to understand switching costs. This will often get you meaningful discounts from your current vendor if they know you are actively testing other tools.⁸ Meaningful discounts are a successful outcome indicating impactful work.

Migration is a muscle—an organizational muscle. With regular practice (and a little help from our LLMs), it gets so, so much less scary.

Regular review is often a requirement in regulated environments anyway, so taking note of what you need during the initial procurement process and ensuring the requalification goes smoothly makes things significantly easier down the road for both you and the vendor.

## Conclusion

When you're a rookie engineer, maybe you dream of writing the next operating system or distributed consensus algorithm—some kind of pure, hard computer science. Few of us dream of growing up to send reminder emails to procurement teams and post-sales training teams.

Yet partnerships and vendor engineering are often the most effective ways of solving your company's hardest sociotechnical problems at scale. In a world where AI generates more code than people do, your value still lies in solving business problems with technology.

Companies that try to build too much themselves will fail to compete with those that are nimble enough to leverage the best of the industry. Decisions about who you partner with compound over time; a good partnership, like good architecture, can move mountains.

By treating the buying process with the same rigor as a production system—designing robust POCs, building trust with stakeholders, encoding protections into SLAs—you ensure that your infrastructure remains an asset rather than a liability. Buying software isn't the end of the journey; it's the beginning of a partnership.

In the following chapter, we'll look at instrumentation from a governance perspective. What should observability teams know about schemas, semantic conventions, and data requirements in environments with a high bar for security and compliance?

---

⁸ For engineers who are working on the requalification and POCs, demos, and so on, cost evals can feel a bit like *Groundhog Day*.

---

# Instrumentation for Observability Teams

*This chapter was written by Hazel Weakly,  
Fellow at the Nivenly Foundation, and  
Austin Parker, Director of AI Strategy at Honeycomb*

Ralph Waldo Emerson once wrote, “language is fossil poetry.”¹ Social scientists and philosophers have long observed that language itself both shapes (and is shaped by) society. Wherever two or more people gather, a language is sure to be either developed or refined. Culture itself is the crucible of language—the way we interact and relate to one another.

An observability engineering team grappling with an instrumentation strategy for their system faces similar burdens. Code ages like the value of a new car driven off the lot: it depreciates the moment it’s written. While software (as a whole) may provide value, code itself contains a burden of maintenance and care that requires careful planning, strategy, and gardening in order to avoid overwhelming operators as their system scales. If that describes the kind of problems that keep you up at night, then you’re in the right place.

Observability teams may take different forms depending on the size, scale, and remit of an organization. In smaller orgs, “observability team” may be a role that’s foisted upon a site reliability engineer. Larger organizations may have multiple teams, each responsible for a certain set of logical systems. Your bosses may be in information technology, or they might be in software engineering. Your job may look more like vendor management than anything else. There’s a wide spectrum of practices.

But in general, we believe that an observability team should own instrumentation at a pretty wide level. That includes instrumentation code and frameworks, telemetry

---

¹ Ralph Waldo Emerson, “The Poet,” in *Essays: Second Series* (1844).

pipeline configuration and management, alerting infrastructure, SLO tooling, and so on. As an observability team, you may not be responsible for actually writing the instrumentation code yourself, but you're likely evaluated on how much everyone else does.

In this chapter, we're going to cover instrumentation for observability teams, including what your "north star" should be, regardless of size and shape. We'll start with instrumentation foundations and what everyone should know. Then, we'll discuss how instrumentation changes in secure, regulated, or otherwise unique environments. We'll cover observability learnings and management at very large enterprises, and finally discuss how observability teams should think about infrastructure metrics and observability.

## What Observability Teams Need To Know

As previously mentioned, language is a construct constantly evolving based on social interactions. Instrumentation is very similar: it's a construct that is created from the evolving operation of software. If we had perfect visibility into what our code was doing without logging, metrics, profiling, or other technology, then we wouldn't need to worry about instrumentation.

For many years, we've seen various *tabulae rasae* for the semantics of software appear. Unfortunately, they often wind up as being overfit for a particular domain. For example, the .NET and Microsoft Stack have long enjoyed a vertical integration between runtime and operating system that allowed Microsoft to produce a more or less consistent dialect of system state metadata, but it generalized poorly (and then Microsoft decided to embrace Linux and subsequently threw away half of it).

Similarly, Java has its own dialect and semantics that are integrated up and down the stack—that is, until you add in a React frontend or a mobile app written in Swift. Then you're back to square one. *nix systems in general have a *somewhat* consistent and self-referential dialect of telemetry, but like most things involving *nix, the learning curve can be more of a learning cliff.

Ultimately, the north star for an observability team is being able to define and manage the constructed language for their organization. That requires an ability to play archeologist with the terms and definitions that have slowly ossified into the informal language of software engineering at a company, while making a bridge to the *lingua franca* of the wider world. Your role as part of an observability team isn't just handing out instrumentation and documentation. You must also provide the framework that individual contributors and leaders across teams, business units, and companies use to communicate intent and action. That's a big responsibility and role.

## Semantic Conventions

The most valuable part of OpenTelemetry (OTel) for an observability team is in its **semantic conventions**. Semconv—as they’re sometimes colloquially referred to—are a collaborative effort to define standard human definitions for common concepts in software, hardware, and telemetry itself.

A subtle goal of semconv is to make telemetry a trainable skill, rather than something that must be learned at the knee of more experienced developers at each new job. Because semconv are universal and portable across languages, clouds, and runtimes, they work more as a dialect than as an encyclopedia. You shouldn’t necessarily need to memorize them, but knowing how the structure works helps you interpret unfamiliar conventions.

Semantic conventions are a shared vocabulary and grammar for observability meta-data. The vocabulary is the set of names for attributes, metrics, spans, events, and so forth—things like `http.request.method`, `db.system`, `exception.type`, `GET <route>`. The grammar is how those names are organized into reusable patterns: dot-separated namespaces, families like `http.request.header.<key>`, requirement and maturity levels, and stability.

That design offers scalability: you can reasonably infer other domain shapes from existing ones, making discovery and mapping easier. This makes observability a more teachable practice. Rather than needing to remember the details and minutiae of each individual service, once an engineer learns the basic namespaces, they’re able to trace through a service call chain easily. Most of the heavy semantic lifting is covered by two heuristics: domains are namespaced in order of specificity, left to right, and actors are defined by client/server or producer/consumer.

The other advantage of semantic conventions is that you don’t have to do all of the work yourself. The OTel project **defines a significant number of domains** and provides instrumentation for them, along with tooling such as **Weaver**, which allows you to create and define your own domains. That domain definition and promulgation is one of the most important aspects of instrumentation design for observability teams, so let’s focus on that next.

## Telemetry Schemas in Practice

The benefits of API contracts and versioning are well-known: they improve the safety of deployments, provide stable integration points for consumers, and allow teams to more easily scale. So, why not **treat telemetry** the same way? **Telemetry schemas** are a concept that brings the best practices behind modern API design to the observability space by providing a textual representation of the telemetry data a service emits and providing tooling for the discovery, linting, creation, and code generation of those schemas.

As of this writing, introducing a telemetry schema is not a turnkey process. But, the building blocks exist for you to adapt and implement. In lieu of a step-by-step user guide, let's discuss the benefits of schemas and how to write new ones.

Schemas provide a single source of truth for the telemetry emitted by an application or service. This is beneficial for developers and SREs alike. Instead of a potpourri of fields and subtly duplicated data, users can refer to a single document that lives near the code. That discoverability has myriad applications as well: tools can leverage these schemas for easier access to end-user discoverability, auditing becomes easier, and AI assistants can ingest these documents into their context for safer editing or agentic SRE practices.

In addition, these schemas allow for both compile time and runtime verification of telemetry. Builds can be failed or flagged when attributes or signals that aren't specified appear in the code, and pipelines can be created that drop mystery attributes from generated data.

Finally, schemas allow for safe evolution of data. Pipeline processors can use schemas to update or modify telemetry data in flight, normalizing data before it's ingested and exposed to alerting rules or dashboards.

Writing new schemas is an ideal way to move toward your north star of normalizing the constructed language of observability in your organization. Here are a few tips:

#### *Start from the “why.”*

Base your schema on the questions that users are likely to ask, while also supporting the requisite variety of fields and attributes to answer emergent questions about system behavior. Make the common questions cheap, reliable, and efficient to answer.

#### *Define your namespace.*

Identify the shared business attributes that are truly consistent across your organization and place them at the right level of a namespace.

For instance, if a transaction ID is truly shared across the organization (in multiple systems—line-of-business applications, billing systems of record, etc.), then this should be expressed at a higher level than a system-specific or service-specific ID (`acme.transaction.id` versus `acme.serviceName.transaction.id`).

The reason is that you don't want to make your grammar confusing to users who are working outside of their normal domain. A business analyst may want to answer questions like “Based on annual revenue, which customers that have filed support tickets in the last quarter have had the worst API performance?” If you intermix identifiers, then it's probable that some data will be incorrectly included or excluded. This is more important than ever with the rise of AI assistants throughout organizations; AI will pattern-match very strongly, and may not have



the sophistication required to realize why these values are different (or that they are at all).

*Abstract at the right level: actors and roles over protocol details.*

While you should expose protocol information to your telemetry, the relationship between services should remain decoupled. This is the rationale behind the client/server and consumer/producer delineation in OTel, and you should mirror that. Transports can change, after all.

*DRY.*

Shared attribute keys should be shared across signal types. If the same attribute appears on different signals, then the values for that attribute should also be consistent. A good example is our aforementioned `transaction.id`. That attribute contains extremely high-cardinality values, and it's not recommended to have high-cardinality fields on time series metrics.

Instead, you can layer attributes between signals (see [Chapter 7](#)) that improve your ability to navigate from one to another while also maintaining limits on cardinality. In this case, you may wish to add a `customer.region` (or `tier`, etc.) that has lower cardinality and enables you to preserve some semantic customer metadata on time series, and reserve higher-cardinality attributes on signals that are better-suited for them.

## Identifiers and Security Teams

In regulated environments, there are a variety of identifiers that raise the compliance level of data by their existence (e.g., internal customer identifiers). That presents a challenge to the observability team: security thrives off of isolation, compliance thrives off of evidence, but observability thrives off of correlation.

In cases where useful identifiers would trigger compliance or security guidelines, you should consider one-way hashes of sensitive data. For example, an internal customer identifier can be anonymized in such a way that it can't be reversed into its original format. As long as the hash method is consistent across your organization, you can still gain the benefits of correlation without incurring the oversight of security and compliance.

One critique of telemetry schemas is that they appear to be a very heavyweight solution, especially for smaller organizations. That's a reasonable critique, but it speaks more to the frustration and conflict present in the misaligned incentives for observability teams and development teams in organizations, rather than an inherent flaw.

Dev teams often don't see the same constraints as observability teams because their scope is more limited. They may only have to care about a single service, or a small group of them, whereas observability teams are exposed to the entire firehose of data (and are often scored on keeping the costs of that firehose controlled). You can't necessarily browbeat other teams to take those concerns seriously, either. Direct accounting of costs is a strategy, but it's one that usually engenders resentment and poor observability outcomes. So then, how should an observability team create the right incentives to encourage desired behaviors?

## Tooling, Migrations, and Paved Paths

Aligning development, operations, business, and observability teams on telemetry data isn't just publishing a wiki page about "best practices" and calling it a day. Nor is it dropping in some sort of monitoring agent and giving people a bunch of prebuilt dashboards.

We've seen many well-meaning DevOps teams try to "bring in observability" by spending months carefully analyzing each infrastructure and service dependency, rolling out agent configurations and dashboards for those services, presenting them to development teams as a *fait accompli*, and then having them collect dust because the data being collected isn't relevant to the day-to-day work of any of the aforementioned groups.

Recall that your north star is to build a shared language for the organization and to adopt the techniques that society uses to promulgate language: education, dictionaries, rites, and repetition (namely, culture). In a more practical sense, you need to build tools and frameworks that make it easy for users to adopt the language that you're building, create repeatable and accessible training opportunities for users as they adopt it, embed touchpoints for your instrumentation practice and strategy into your SDLC, and ingrain these into your processes.

Let's look at each of these.

### Tools and Frameworks

OTel is designed to be extensible and interoperable with existing telemetry systems, cloud environments, and SDLC tooling. When considering how to build your own tools, keep in mind two basic things:

1. Don't wrap the API, but you *can* extend it or build indirection around it. OTel is intended to be a universal API, with universal idioms. Wrapping the API and changing the names of method calls for consistency with existing telemetry suites or preferences is a distinct antipattern. You can extend the API—e.g., writing

decorators that bundle multiple calls into one (think starting a span and updating a bundle of metrics at the same time)—but avoid changing it entirely.

You can also choose to bundle and reexport methods from the upstream library. For example, you can create a single indirect package that bundles together OTel along with dozens of other existing instrumentation APIs. Doing that enables you to immediately migrate many services without breaking callers while rewiring them behind the scenes.

2. Hide the right complexity from users. The OTel SDK can be highly verbose, with what feels like a mountain of boilerplate and customization options. Most of this complexity can be distilled into a one-liner that you add to a service framework. Current work in the space is aiming to make this even more straightforward, with a [configuration file–based approach](#).

The single thing that trips up most development teams is context propagation between services. Solving this problem is a perfect opportunity to start as you plan to keep going, by embedding OTel into your remote procedure call or service framework with a basic telemetry schema, and ensuring that this framework handles service-to-service context propagation. If adopting OTel is as simple as picking up a new version of an internal library, teams will be far more likely to adopt it.

On the backside of telemetry generation, you’ll want to focus on managing and understanding data volume and data quality. The OpenTelemetry Collector and tooling like Weaver are crucial here. The Collector is wildly configurable, but also highly customizable. Rather than trying to handle everything in config, we suggest writing custom extensions (it’s just a little Go) and building your own collector distribution using the OpenTelemetry Collector Builder (OCB). That choice gives you significantly more flexibility in handling unique filtering, sampling, or transformation scenarios, as well as reduced resource usage and reduced security attack surface. Weaver can be integrated into your build or deployment pipelines to [live check](#) emitted telemetry data against registered schemas, or even added as a hook for local build/test scenarios.

AI can also be an ally here. Explore AI-assisted remediation for existing services by capturing telemetry from preproduction environments, feeding it through Weaver, and then handing the code and Weaver outputs to an AI agent to suggest fixes for. If you’re feeling adventurous (and have great tests), you can even have the agent go off and make the PRs for you.

For the enterprise users, Weaver offers additional benefits. While it’s good hygiene to ensure that your telemetry matches a schema, regulated industries will often need a compliance review of code or outputs in order to verify that they’re not leaking sensitive data. Gathering evidence for the compliance team and auditors using Weaver in CI can automate a tedious process.

## Paved Paths and Special Hats

Too often, we see OTel (and observability) initiatives in general meet a staggering shrug of indifference from users. You can lead a developer to water, but you can't make them drink. That shouldn't be terribly surprising. Observability can feel like a chore, even at the best of times, and it's little wonder why the industry seems to gravitate toward "easy buttons" and magic agents that promise to do all the work for you (or, these days, magic AI agents that will understand and solve your problems for you).

A possibly persuasive story comes from the early days of distributed tracing at Google. One of the engineers responsible for Dapper (Google's internal tracing tool) was wondering how to get more users. Dapper had been rolled out and was collecting traces from a variety of services (search, shopping, etc.). A guerrilla marketing approach of "telling people to use it" had stalled.

The engineer's mentor suggested figuring out where people already were: what tools, screens, and services were they using to understand performance? Figure that out, then the path should become clear.

The Dapper team interviewed and analyzed engineer behavior, and ultimately added a single link to the monitoring graphs that teams would regularly consult when they were paged that would lead into Dapper. Usage doubled and then tripled, almost overnight. Engineers who discovered Dapper would tell their colleagues on other teams about it. Before long, distributed tracing had become a core tool in production monitoring and observability at Google.

Your engineering teams probably have similar places they congregate: existing dashboards, alerts, wiki pages, and so on. Identify those, understand their pain points, pave a path to your observability data and OTel solutions, and let the magic happen. You're not going to win over everyone immediately, but you have to start somewhere.

Make adoption as easy as possible by building tools, as previously mentioned. A little upfront work will pay dividends over time, especially with recalcitrant teams. The "right way" and the "fastest way" should be the *same way*. One tactic we've seen in successful organizational adoption is compiling and promulgating OTel wins—keep a brag book of where your observability efforts are helping and make sure management sees them. Make stickers, t-shirts, or little charms to incentivize adoption and usage. Get creative!

Finally, make adoption safe. Leverage Collector pipelines to dual-send data to cheap blob storage as well as telemetry databases—especially when doing migrations or changing sampling rules—to compare old versus new, and have confidence that you're not dropping anything. Add rules (either through the transform processor, schemas, or other mechanisms) to avoid cardinality explosions by dropping high-cardinality fields on specific signal types like metrics. Ship default pipelines for local

development to make it easy for devs to understand what telemetry is being emitted, and centralize the management of pipeline configurations so it's easy to tell why some data is being dropped.

The last tip for adoption is especially important for sampling. One of the biggest frustration moments for developers is when they go looking for data that they *think* should exist, but doesn't. Dual-send can help this by providing a fallback when data is sampled out. But you can also help by doing more education about sampling, its benefits, and its drawbacks. Make sure sampling rates are easy to understand and avoid overly complex sampling strategies at first. It's generally better to get "too much" data when starting out and then walk it back as you build confidence in your tools.

To wrap up, it can be helpful to think of yourself as an observability guide when rolling out instrumentation and OTel as an observability team. You're not necessarily the one who's going to stand at the summit and get your picture taken, but you're going to try really hard to make sure nobody who's coming with you gets themselves killed. OTel gives you the framework to create safe, reliable, and repeatable instrumentation paths—and as it matures, it'll become easier to do so. It gives you the grammar and vocabulary that you and your organization need as a floor to scaffold your own concepts on top of. It provides tools to help manage and constrain the volume of telemetry generated by modern, cloud native software, and it can help you avoid common pitfalls if you use the features it offers. Learn it, leverage it, maybe even love it, but don't dismiss it—it's a wave that you can surf to accelerate observability adoption at your organization.

## Instrumentation in Highly Secure Environments

In highly secure environments, two of the chief concerns about instrumentation are compliance and security. While both are immensely complex topics, from the perspective of observability, we're going to think of compliance as being largely about the evidence one provides, and security as being largely about the isolation one guarantees.

Unfortunately, that means we quickly run into a tricky situation: observability's superpower is data correlation. Done naively, that very superpower can easily compromise security isolation guarantees and stymie compliance evidence collection. However, if we approach observability as a shared language between the entire enterprise, it becomes possible to *facilitate* compliance without compromising security.

Given how these two concerns make implementing observability much more difficult, it might be surprising to know that it's still very similar to implementing observability in any enterprise. One main difference comes from how high the bar is for implementing security capabilities; many "best practices" become requirements.

Another significant difference is how heavy the burden of proof is. Do you have end-to-end encryption? Do you secure customer data properly? Is your OTel Collector using FIPS 140-2 encryption? Lovely! You'll need evidence for all of that. In addition, much of the success here relies on properly communicating with and designing incentives for various teams: leadership, engineering, product, architecture, and every other function need to be on the same page.

If you thought it was hard to convince stakeholders to work with you to implement observability, convincing them when they're in an organization that has 10 times the compliance-related busywork on their plate is another story entirely.

Many organizations in this situation fall into the trap of only being able to make progress on “mandatory” items; unfortunately, *observability is almost impossible to make mandatory*. Success here often requires designing incentives in a way that everyone involved looks good when they help you; they'll often want the observability, but they need to justify it against a massive pile of mandatory work. Help them do that.

This is one reason why it makes sense to talk about facilitating compliance evidence collection with observability.²

## Use Compliance Tiers to Your Advantage

Luckily, there's another component to highly secure environments that can make observability easier to implement: very rarely does every environment need every compliance control.

Nobody wants to add unnecessary work to their plate, so it's common to encounter multiple tiers of compliance, multiple tiers of regulation, and a graduated approach to environments. Use this to your advantage and provide seamless ways to “graduate” services from one level of compliance to another. While startups might recoil at the thought of a 12-month process to get an application “in a production environment,” this is the norm for regulated industries.

That said, just because a service is not “in production” doesn't mean they're not deploying code. Often, services are built out in local environments, and then in development, testing, staging, and finally production. Importantly, every level has more stringent requirements. A common mistake might be to shift implementation burden to the development teams by providing a “production ready” observability framework.

Rather than requiring development teams to do months of busywork in order to “just get some data” locally, enable them to experiment as rapidly as possible given

---

² Another reason: security projects are typically better funded than other projects, which can be a good “in” for making progress on observability rollouts.

the constraints of each environment. You'd be surprised at how quickly they become your champions when you work to make their life easier. Don't just shift observability tooling left; shift the value left too.

Speaking of shifting the value left, this often means providing a minimal set of capabilities as a platform that integrates observability into the needs of the company.

## A Minimal Set of Capabilities for Observability Teams

If we're facilitating compliance and security concerns, the following are a good starting point of capabilities to think about:

### *Vendor management*

You'll have many requirements from your regulators. They'll want to know that your vendors are capable, and you'll need some strong business agreements and financial incentives to ensure your vendor relationships are mutually beneficial. Be prepared to pay for what you're asking for.

### *Encryption (end to end, at rest, and more)*

End to end as a requirement is more thorough than you might expect. You need mTLS between *every* component, even services in the same trusted boundary. If it talks over the network, it needs mTLS, period. This is a strong reason to prefer sockets for intraservice communication when possible.

While mTLS takes care of most end-to-end encryption, encrypting the data at rest is also important. Depending on your requirements, encrypted hard drives may or may not be sufficient.

### *Multilayer data analysis*

This is the hardest component. You have to prove you're satisfying all your regulatory requirements with every byte of data you send over the network to *anywhere*. Merely satisfying it isn't enough; you have to show the work too.

For example, many data regulations require that only minimally relevant data is sent to locations that customers explicitly authorize. You will need to be very careful to not accidentally send identifying data in telemetry. The superpowers of observability will be working against you here: correlation is a strength of observability, but it means that it becomes extremely easy to accidentally send telemetry that is "safe" locally, but globally can be constructed into de-anonymized and identifiable data.

You'll notice a pattern: all of the aforementioned capabilities are invisible. No customer-facing roadmap is going to prioritize developing these capabilities, and even observability teams will struggle to demonstrate value when implementing them. Consequently, it becomes critical that they can be implemented as quickly and efficiently as possible.

## Valuable Capabilities in a Regulated Environment

Because of this, the following capabilities become quite valuable for operating in a regulated environment that it's almost infeasible to attempt observability without them:

### *Chargeback*

This becomes very useful for ensuring that the observability team is suitably staffed and that the value of it can be shown in the company. Without that, observability efforts might die out from lack of buy-in.

This can also be viewed as building cost and operational concerns into the “language” of the business. Business-intelligence shaped concerns are often hand-in-hand with regulatory requirements, especially regulations that have language around undue burden.

### *Schema control*

You can do multilayer data analysis without this, but it's vastly easier to be able to have multiple levels of confidential data when you can check things automatically against a schema. Schemas also make it easier to prevent global correlation from compromising regulated anonymity of client data. In other words, judicious use of the OTel Weaver project can be the difference between needing to roll out an observability deployment per *application* versus one per *environment*. It can make or break a team's ability to be allowed debugging information at all.

### *Data transport through proxies you control*

You don't strictly need this, but runtime scanning of data for compliance and security reasons is required in almost every regulated environment. Doing it without proxies is possible, but needlessly painful.

Sufficiently powerful proxies are key for enabling policy agents to become an effective tool for enforcing company-wide data controls. They're exceptionally useful for providing evidence on otherwise hard-to-prove requirements such as what data is allowed to be sent to a third-party vendor.

### *Tokenization and data anonymization*

Tokenization enables you to do something very powerful: it can help prevent locally safe data from being globally correlated. If the data can't be anonymized because it's encrypted, you don't have to worry about identification in the same way.

Tokenization has another strong benefit: it allows you to do side channel correlation outside of your tooling. What might otherwise be too sensitive or verbose to put in observability data—such as AI chat logs or database queries—can be made useful when combined with a technique such as tokenization.



As mentioned earlier, there are benefits to correlation of anonymized identifiers. Using safe, one-way, hash functions that are consistent across the organization provides a level of functionality while minimizing data exposure risk.

#### *Authorization and authentication*

Your data handling needs to have certain guarantees around it being handled by agents that are allowed to do so. This is significantly easier to achieve if your mTLS utilizes short-term credentials, attested identities, and granular permissions.

In addition, many regulatory requirements deal with “nontechnical” aspects of running a technical organization. You will encounter a need for documenting and executing long-term support plans. You will also need to develop operating models for migrations. Best practice guidelines evolve into blueprints and frameworks. Documentation graduates into requirements around teaching courses and internal support SLAs. Code scanning and schema validation become something that needs to happen at development, in CI, during deployment, and throughout the lifecycle of an application operating at runtime.

## **This Is Hard Work, But Work Worth Doing**

As an observability team, it can feel overwhelming to think about all of these regulatory requirements, and it can also feel like you’re doing all the security and compliance work “in addition to” or “instead of” the value-delivering work of observability. However, partnering proactively with security and compliance teams will pay dividends down the road in establishing trust and unlocking the roadblocks for engineering teams.

After all, if engineering teams aren’t allowed to instrument their code effectively, nothing else about observability will end up mattering.

Just like spoken human languages, the power of observability really shines through when everyone is on the same page and communicating effectively with each other. Giving them that language and the tools with which to express it safely, securely, and compliantly, is the secret to delivering modern engineering capabilities in highly secure environments.

## **Conclusion**

The role of an observability team is ultimately not to claim the success of an observability program, but to enable that success for the entire organization. That makes defining success harder; data volume and dashboard count are a lot easier to measure.

Your wins show up in subtler ways. Issues that would have been a week of conference bridges shrink to days. Teams communicate expectations in SLOs, and they're echoed by directors and executives. Compliance stops being a four-letter word. Remember, though, that this is probably the best possible time to start building an observability program.

OTel provides a level foundation for you to build rich, business-specific context on top of. The technical artifacts surrounding it, like semantic conventions and telemetry schemas, provide a pathway to cultural goals. You have the ability to make the right way and the fast way the same way, and to place observability in the places that your engineers already are. At the end of the day you'll know you're succeeding when the *language* of observability becomes part of the engineering and business record.

In the next chapter, we'll explore where to go from here. We'll check in on predictions from the first edition of this book and make new predictions for where observability is heading that may (or more likely may not) age just as well.

---

# Where Do We Go From Here?

Welcome to the end of the second edition. You made it.

We have covered observability from many different angles in this book: its origins in control theory, how software engineers should instrument and understand the code they write, deep dives and use cases, and how technical decision-makers should think about observability for their orgs.

Our goal in this edition was to update the technical content, but also to underscore how observability is derived from first principles and reasoning that endures, so you can guide your own teams and organizations through turbulent waters.

Our industry is undergoing a convulsive period of compressed change. Roles and identities are being stressed in new ways. Anxiety is endemic because the things we long took for granted are no longer safe bets.

In this chapter, we'll look at where the industry is headed over the next few years, and what engineers and technical decision-makers can do to prepare.

## Developers Must Return to Production

Before we look at what's next, we must address the “wish” we made at the close of the first edition (paraphrased slightly):

The space we will be watching closely is that of developer workflows...Developers need fast feedback loops and the ability to understand how each code change affects users in production. Incredibly few developers have the ability to do that.

We believe the industry needs fundamentally better developer workflows in production, and we predict observability will be the path for many to get there.

Developers *still* don't have the ability to validate a diff they just shipped, or identify what effect it had on any user or subset of users. Developers still live in a world where testing means "correct in theory," not correct in reality.

Consider this our one prediction for the second edition: *the dev/ops dividing line must be destroyed*. Developers are no longer crafters of code; they are builders and architects of systems. That system is production. Test in prod, or live a lie.

By this, we do not mean larding on yet another mandate onto overflowing plates. We mean that core development workflows must include instrumentation and production telemetry, validating user expectations and commitments as they actually exist. We can no longer afford to stub reality out of the picture. Software engineers must become as fluent in the messy world of production as they are in the clean, structured world of staging environments, CI/CD pipelines, and tests. Only one of those worlds holds real value. Only one is production.

This has been forecasted before. This time is different. The reason why is AI.

## The Changing Cost Model of Software

The dev/ops divide has endured long past its sell-by date because it encodes the tension between change and stability in a way that is legible to org charts, budgets, and career seekers. Over time, it has calcified into policy, friction, tools, and expectations that combine to make production inaccessible and inhospitable to developers.

AI attacks this consensus on many simultaneous fronts: as a technological enabler, it brings practices that have required exceptional engineering discipline and skill within reach of the median engineering org. It provides the competitive pressure to accelerate, and introduces nondeterminism as a feature—the *only* way to validate agentic workflows is by instrumenting your code and observing production, then porting what you learn back into evals.

But the most powerful and intriguing assault on the status quo comes from the changing economics of code production.

For as long as we've had software, we have treasured the code artifact. We have obsessed over its elegance and readability, and developed elaborate rituals around changing it safely. As one of your authors declared earlier this year:

There is no test suite on earth that could make me trust a new chunk of code more than I trust the code that's been running in production for two years. That's how trust works; it takes time.

—Charity Majors

This is a truism where durable code is concerned. The trust lives in the code, implicitly.

All of this makes sense—indeed, it is the only sensible approach—in a world where producing code is time-consuming and hard, and good code is scarce. The true cost of software has always been defined by the cost of maintaining it. Our focus on elegant code made sense because readability was a reasonable proxy for maintainability. We couldn't predict the future, but we knew our ability to read and understand the code would gate any change.

But that gate has been necessary for the same reason that the only way to safely change the code has been with a series of tiny incremental diffs: because most behaviors have always been implicit and poorly understood, discovered only when we break them. The most important parts of our systems and software—the expectations of our users, the commitments we have made to uphold stability and continuity—exist nowhere at all. Not in the code, and not in the system.

## The Most Important Parts of Our System Have Never Been Specified

The code is not the system. The system is the unique and irreplicable intersection of code, infrastructure, users, builders, data, and time. Artifacts accrue meaning as users develop expectations around existing behaviors. These layers of acquired, nonarticulated meaning are what make rewrites, migrations, and replacements so infamously perilous and particular.

Code that makes it into production becomes more than just code. It becomes the fossilized record—the only record that exists—of developer intent, business needs, and user expectations and dependencies that have built up over time.

The durable model of software delivery is agonizingly expensive. The cost of software has always been defined by its maintenance costs as a direct consequence of those sedimentary layers of accrued expectations and forgotten intent. If nobody understands them or knows what they are, we certainly can't test for them.

If we are ever going to bring the cost of software development materially down, we are going to have to unbundle the expectations from the artifact and make them explicit—and live elsewhere.

# The Great Unbundling

Few reach their second decade of software development without acquiring a robust fear of large diffs. Indeed, we ourselves have argued that an intuitive sense for how much change is too much change, along with a pessimistic stance to production systems in general, is part of what defines maturity in a senior developer.

Chad Fowler, engineer and CTO at BlueYard, wrote a post called “**The Deletion Test**”, in which he proposed a thought experiment: what do we lose when we delete all the source code? *Why* is it so scary?

When people say, “We can’t just throw the code away,” what they usually mean is something more precise:

- We don’t know exactly what behavior is required.
- We don’t know which failures are unacceptable.
- We don’t know what invariants must always hold.
- We don’t know how to tell if a new version is correct.
- We don’t know which bugs are intentional fixes for forgotten edge cases.

Those are not code problems. They are evaluation problems.

Code becomes precious when it is the only place knowledge lives.

Fowler observes that the bottleneck has shifted from generating code to validating code, as we have argued in this book. He goes on to reframe the question, from “How do we write safer code?” to “What must exist so that code can be replaced safely?”:

That question is harder. It requires deciding what truly matters and making it explicit, testable, and durable. It requires relocating rigor out of the implementation and into the system around it.

If you can delete and regenerate the code while upholding all of the commitments you have made to users, the business, and other parts of the system, then code is no longer the sedimentary record of long-forgotten developer intent or the residue of past promises, only a materialized view of current understanding. *Code becomes cache*. Maintainability becomes a property of the system, not the code.

## The Human Cost of Compressed Change

Generative AI has moved so quickly from novelty to utility that many are still processing the shock. We don’t wish to catastrophize or equate the dislocation of technological advancement with far greater traumas. But the change has come fast, which makes it hard.

Many seem to be exhibiting mild symptoms of disorientation, disconnection, or depression. We have lost track of how many engineers have privately confided their plans to retire instead of processing the abrupt obsolescence of the skills they had cultivated for decades, building careers and communities and identities around mastery of practice. Others show traces of mania, obsession, or magical thinking, issuing overheated exhortations around how “this changes everything.”

Silicon Valley, of course, prefers—some might say actively cultivates—the latter.

## The Claim: “AI Is Unprecedented”

It has grown popular in these parts to speak of AI as *sui generis*, the trigger mechanism that will unleash unbounded exponential growth by achieving artificial general intelligence (AGI). We will build robots that do our jobs better than we can, that will build robots better than we could, and *those* robots will build other robots beyond the capability of our understanding, and thus inaugurate a new era of boundless exponential growth for our species, where we are no longer in control of our destiny.

Climate change? Solvable and solved, thanks to more powerful models. Poverty? Solved by UBI (universal basic income) via some vague, undescribed AGI-related mechanism that is *definitely* not taxing the rich.

This strikes us as a fantasy. At best, it’s a thought experiment that remains unproven. “Extraordinary claims require extraordinary evidence,” as Carl Sagan said. If AI can usher in exponential forever growth, that violates the one thing we know for certain about exponential growth: it *will end*, either by flattening out into an S-curve or by consuming all resources and crashing.

Some of the pressure people are experiencing is due to real structural change and uncertainty, but much of it is speculation or marketing. To pick on **one representative post** that blanketed the airwaves in early 2026:

The AI labs made a deliberate choice. They focused on making AI great at writing code first...because building AI requires a lot of code. If AI can write that code, it can help build the next version of itself. A smarter version, which writes better code, which builds an even smarter version. Making AI great at coding was the strategy that unlocks everything else. That’s why they did it first. My job started changing before yours not because they were targeting software engineers...it was just a side effect of where they chose to aim first.

They’ve now done it. And they’re moving on to everything else.

We must resist being credulous. Until proven otherwise, we still live in history, in a world governed by finite resources and human constraints. To claim that we are on the verge of exponential liftoff, where robots build robots beyond our understanding, is quite extraordinary.

Reality is enough to deal with, we feel, without borrowing hypotheticals from the future to stress about.

## This Moment Has Many Precedents

A more practical reason to resist the framing of AI as *sui generis* is that it robs us of our history. And history, it turns out, has many instructive precedents for our current upheaval:¹

- The craft of weaving, replaced by the powered loom
- The craft of scribing, replaced by the printing press, the typewriter, the word processor, and Google Docs
- The shift from handcrafted server pets to disposable, immutable infrastructure cattle in the early years of our own careers

The history of technology tells us that the most dramatic gains are made in the earliest years of any technological breakthrough, as the easiest problems get solved first. Once the low-hanging fruit has been harvested, it gets harder to come up with new ideas, and progress slows and becomes ever more incremental. As Adam Becker said, in Chapter 2 of *More Everything Forever* (Basic Books): “It is around 18 times harder today to generate the exponential growth behind Moore’s Law than it was in 1971. Ideas, and the exponential growth they imply, are getting harder to find.”

To us, it seems unlikely that the AI labs chose software opportunistically to accelerate their exponential takeover of humanity. Software is built out of language and logic. Software, in other words, is the low-hanging fruit.

## The Sysadmins Went Through This

Ever since the first reprogrammable computer was built—the ENIAC, 80 years ago—the code artifact has been our most holy and sacred of relics. Lines of code built the world’s most valuable companies and many of the nouveau-iest fortunes. The loveliness of code has built reputations, landed jobs, and justified exorbitant salaries compared to other roles in a post-post-industrialized world economy.

Of *course* it’s disorienting to live through the rapid dethroning of code as singular repository of meaning, value, craft, contracts, and ledger of history.

But if you, like us, are old enough to have worked in tech in the 2000s, there is also something familiar about this feeling.

---

¹ This has all happened before, and it will all happen again, as the Cylons said in *Battlestar Galactica*. Also, apparently, Peter Pan.



Do you remember the sysadmins?

We do. We remember their pride and dedication to craft; encyclopedic knowledge about kernel modules and filesystems, how to calculate a netmask or configure a routing table. The long running in-joke of obfuscated Perl code awards. We also remember the panic and paranoia and mounting embitterment that surged in some corners of the community, once the value of those skills began to crater, replaced by automation and regeneration.

The emotional reaction is normal. The weavers felt it, the sysadmins felt it. It happens, it's hard, you move through it. Most sysadmins ultimately adapted. They picked up a programming language, started writing more code, and automated themselves out of a job. They built the systems that replaced the work they used to do, and then they worked on those systems.

## From Software Code to Software Systems

The shift from handcrafted servers to immutable infrastructure taught us that mutability is the sworn enemy of understanding. Any artifact that is edited in place creates drift. Drift is what makes systems impossible to maintain.

Our ability to kill and regenerate infrastructure components is the reason we trust it. At Honeycomb, we kill the oldest Kafka node off via cron every Tuesday. That's why we are confident in our bootstrapping and balancing processes: everything is repeatable, the data can be regenerated, the commitments live elsewhere.

The fact that we cannot regenerate our code in the same way is a sign that we don't understand it. We don't know which commitments we have made. We don't know which dependencies will break. We find them by breaking them, mostly.

In another post, [Fowler](#) said:

The discomfort with “fresh code” is understandable, but misplaced.

What people actually fear is undetected behavior change, performance regressions, security regressions, and silent drift. Those failures are caused by unobserved change, not by newness.

A system with stable contracts, strong evaluations, continuous monitoring, and clear rollback paths can safely tolerate very fresh code. A system without those things is dangerous even if the code is ten years old.

Throughout computing history, working code has been expensive and hard to produce, so we have conserved it and changed it parsimoniously.

Every change was gated on human understanding, and we had little cognitive capacity left over. As a result, everything outside the code got neglected. Architecture. Coherence. User experience. We have treated these like afterthoughts, emergent properties and effluvia of our precious code, but actually they are what matters.

Now that working code is abundant and cheap, we can reclaim that cognitive capacity and use it to build the parts that matter.

## Value Must Be Backed by Durability

Disposable software is great for iterating fast and trying lots of experiments cheaply. But when that software meets reality, it needs to *slow down*. Durability is not what *creates* value, but value usually needs to be backed by durability.

People prefer a consistent experience and a coherent mental model. Behaviors create expectations that we need to honor. Businesses run on structure and continuity. No one wants to open Slack every morning and find all their buttons rearranged.

Code in production accumulates dependencies. This is a good thing. It means people use it. It means it has value.

Dependencies require dependability, whether those dependencies are users, businesses, or other components. The cost of maintaining software is driven by the challenges of stably replacing its components.

In [Chapter 2](#), we talked about the messy, gritty business of code crossing over—that liminal state between states while fulfilling commitments to both.

Rewriting a system in flight is far more difficult than simply writing the original legacy code or the new replacement; the challenge lies in crossing over. Dropping the cost of code to zero doesn't substantially alter the cost structure defined by maintenance. Your migration path still consists of inching along, instrumenting and observing, discovering commitments and entanglements as you go, mostly by breaking them.

Moving to a world of specification, evaluation, and testable commitments—where any component can be replaced behind its interface—changes the cost structure entirely.

## The Tools to Build Durable Systems Will Come

We feel lost and disoriented because our eyes have not yet adjusted to our new reality. It is easier to see the practices we have lost than the tools, practices and ideas we will gain.

The tools to build durable systems—to extract commitments and make them explicit, to preserve intent at the right level of granularity, to make architecture the versionable, reviewable artifact from which code is generated—do not exist yet. They will come. The economics of scarcity dominated our past, but the economics of abundance will govern our future.

The problems of abundance are different problems. Our ability to generate code has skyrocketed, but our ability to understand it has not kept pace; if anything, understanding gets harder as volume and throughput pick up. All of this pushes toward unbundling the code. If the interfaces and expectations are extracted from code, if we can validate each commitment, then we don't *have* to understand the implementation.

The tools may not exist, but many of the ideas do exist. Most come from operations and QA, two domains that software engineering has historically been rather snobbish about. These tests and techniques are not about testing for correctness or what ought to be happening. Rather, they are about observing and encoding what *is* happening: behavioral tests, characterization tests, capture/replay, traffic splitters.

But you don't need to wait for better tools to come. You can start today. Use instrumentation to capture your intent, and then follow it out into production. Can you validate it there? Can your coworker? Can an agent?

Arm yourself with observability tools that give you precision data—context and cardinality—and a scalpel-like ability to identify the consequences of every change you ship. Build the right seams to make your code composable, observable, upgradable, maintainable. Become as fluent in the real world of production as you are in the abstract world of stubbed-out tests.

## Conclusion

The transition from treating code as a sacred relic to code as cache will be unavoidably messy. As we unbundle our expectations from the artifact and move toward explicit specification, evaluation, and commitments, the traditional dev/ops divide simply ceases to make sense.

Nostalgia aside, nobody misses the bad old days of handcrafted servers, Perl scripts, and Snowflake cron jobs. Sedimentary layers of untangleable purpose and long-lost intent create all the most frustrating parts about working on software.

No one is going to miss the bad old days of handcrafted code, capture/replay, and strangler figs either. One day, we will reminisce about the worst software rewrites we have been part of, and the kids will listen in dismay.

The tools for this future don't yet fully exist, but the principles of observability are already in your hands. It starts with understanding. It starts with observability in production.



## A

- activities, 183, 481
- agent + gateway deployment pattern, 302
- agent-only deployment, 302
- agentic AI, 145, 157-159
  - (see also AI agents)
  - autonomy, 147
  - hallucinations, 146
  - personas, 145-146
  - use cases, 146-147
- agentic investigations, 16-17
- agents
  - redirection, telemetry pipelines, 305-306
  - telemetry pipelines, 300
- aggregation
  - averages, 67
  - log-aggregation service, 10
  - metrics, 66-68
    - TSDBs, 218
  - mobile and web apps, 344
  - pre-aggregation, 14
  - raw events, 44
  - telemetry pipelines, 298
- AI (artificial intelligence), 46-47, 144
  - adoption sabotage, 426-427
  - agentic, 157-159 (see agentic AI)
  - application development
    - observability data, 398-400
  - applications
    - telemetry for, 396-398
  - economics of software development, 461
  - future of observability, 574-576
  - generative (see generative AI)
  - instrumentation and,
    - interaction models, 461-462
    - prompts, 189-191
- AI agents, 128
  - (see also agentic AI)
  - alert audits, 167
  - constraints, 161
  - context necessary, 169-171
  - data errors, 166
  - data patterns, 166
  - definition, 128
  - for observability, 162-164
  - incident response, 166
  - instrumentation, 128-129
  - instrumentation and
    - quality, 166-167
  - LLMs, 128, 162
  - OTel and, 127-131
  - prompts, 189-191
  - queries, 164-165
  - rules, 130
  - SLOs, 168
  - strategies, 130-131
  - traces, 166
- AI sandwich architecture, 321
  - deterministic workflow, 321
  - dynamic AI workflow, 322
- alert fatigue, 173
  - threshold-based alerting, 174-175
- alerts
  - auditing with AI agents, 167
  - error budget, 195-197
  - SLOs and, 179-181
    - case study, 181-184
  - threshold-based, 136, 174-175

- known-unknowns, 175-178
  - zero-level, 198-199
    - predictive burn, 201-208
    - relative burn, 200-201
    - thresholds, 199-200
- Amazon Cloudwatch, 380
- amplifying feedback loops, 439
  - confidence loop, 442
  - learning accumulation loop, 442-443
  - system quality loop, 444
- APM (application performance monitoring), 129, 451
  - routing and, 298
  - unified storage model and, 460
- ASOF JOIN, 252
- async requests, 98-99
  - attributes, 98-99
- attribute capture
  - async requests, 98-99
  - build information, 89-90
  - caching, 105
  - customer information, 101-103
  - errors, 99-101
  - feature flags, 91
  - filtering attributes, 110
  - HTTP information, 93-95
    - route parameters, 96-97
  - localization attributes, 105
  - metric attributes, 107-108
  - rate limits, 103-105
  - service metadata, 86-89
  - specific applications, 110
  - timings, 97-98
  - uptime attributes, 106-107
  - user information, 101-103
  - versioning attributes, 92
- attributes
  - async request attributes, 98-99
  - build attributes, 89-90
  - caching attributes, 105
  - custom header attributes, 95
  - error information attributes, 100
  - feature flag attributes, 91
  - filtering attributes, 110
  - HTTP information, 93
  - infrastructure attributes, 87-88
  - localization attributes, 105
  - map, 77-79
  - metric attributes, 107-108
  - metrics, 66
  - orchestration attributes, 88
  - PaaS orchestration attributes, 88
  - rate limit attributes, 103-105
  - route information attributes, 96-97
  - service attributes, 86-87
  - specific applications, 110
  - timing attributes, 97
  - uptime attributes, 106-107
  - user agent attributes
    - custom, 95
  - user and customer attributes, 101-103
  - user-agent attributes, 94-95
  - versioning attributes, 92
- automatic instrumentation, 29, 54, 118
  - attributes, 95
  - context management, 116
  - RPCs (remote procedure calls), 382
  - serverless and, 122
  - versus custom, 55-56
- averages, 67

## B

- balancing loops, 439-440
  - cost, 444-446
  - incident involvement, 444
  - usage, 444-446
- behavioral patterns, 420-421
- Bigtable, 221, 222-223
- Bindplane, 293, 294
  - OTel Collector, 301
  - resilience, 299
  - scaling, 302
  - telemetry adoption phases, 304
- blackfield code, 461
- Bloom filters, 256
- blue-green deployments, 32
- bottlenecks, 254
  - fleet-wide profiling, 376
  - removing, 338
- brownfield code, 461
- build attributes, 89-90
- build observability ROI, 328
- build system attributes, 89-90
- build times case study
  - bottlenecks, 338
  - data transfers, 338

- Docker Bake, 338
  - network fetches, caching, 339
  - parallel builds, 338
  - steps, 338
  - building
    - benefits, 522
    - biases, 508-509
    - business value, 511
    - buy and build option, 528-530
    - commercial software
      - hidden costs, 524-527
      - risks, 527-528
    - costs, free software, 521-522
    - framework for evaluation, 510
      - build quadrant, 512
      - business value (horizontal axis), 511
      - buy quadrant, 512
      - decide quadrant, 513-514
      - examples, 515-517
      - patterns, 517
      - should not exist quadrant, 512
      - ubiquity (vertical axis), 511
    - hate money path, 508
    - hate time path, 508
    - open source code, 517
      - framework, 518-520
    - risks, 523-524
    - staff+ engineering incentives, 513
    - versus buy, 507
  - burn alerts
    - acting on, 210-212
    - linear extrapolation, 203
  - buy and build option, 528-530
- ## C
- caching
    - attribute caching, 105
    - ClickHouse cache, 247
  - Callaghan, Mark, 535
  - canary deployments, 33
  - Cantrill, Bryan, 40
  - capabilities, observability teams, 567-569
  - capital expenditure purchasing model, 373
  - capture/replay, 35
  - cardinality, 8
    - arbitrary, 236-237
    - caps, 40
    - cost, 355
    - cost growth caps, 46
    - events, 216
    - explosions, 218, 445
    - high cardinality, 14, 26, 29, 41, 234
      - ClickHouse, 240-241
      - data, 44
      - event data, 215
      - Honeycomb, 397
    - metric, 60
    - shared attribute keys, 561
    - tagging, 152
    - tracing data, 385
    - TSDBs, 218
    - unified storage model, 460, 470
  - CAS (Cluster Autoscaler), 378
  - case studies
    - build times at Honeycomb, 336-340
    - Fin, 404-415
    - Retriever implementation, 224-237
    - SLO-based alerts culture change, 181-184
  - Cassandra, 220, 237
  - CD (continuous deployment), signal parity, 319-321
  - change actions, 429
    - observability as a platform, 430
    - observability as investment, 430
    - tools, 430
    - workflow engineering, 430
  - CI (continuous integration), 333
    - deterministic layer, 318
    - incrementality, 335-336
    - ontologies, team workflow, 319
    - semantic layer, 319
    - simulation layer, 319
  - CI/CD (continuous integration/continuous deployment), 327
    - pipelines
      - constraints, 333
      - developer feedback, 328
      - instrumentation and, 330-331
      - need for, 328
      - ontologies, 329-330
      - reproducibility, 334-335
      - SLIs, 331
  - CircleCI, DLC (Docker layer caching), 338
  - ClickHouse, 215, 221
    - as backend, 240
    - caching, 247

- Cloud, 271
- clusters
  - multiregion, 269-270
  - single-region, 268-269
- data collection
  - exporter, 248
  - Kafka, 249
  - OpenTelemetry collector, 248
  - S3Queue, 249
- data modeling
  - vertical scaling, 255-261
- HyperDX, 250
- Keeper, 263
- MergeTree, 240-243
- overview, 239
- primary keys
  - design, 246-247
- queries
  - complex joins, 252-254
  - log event filtering, 243-244
  - Map datatype, 245-246
  - simple queries, 250-251
  - structured logs, 245
- replication, 263-267
- scaling
  - horizontal, 263-271
  - vertical, 254-262
- sharding, 267-268
- SharedMergeTree, 270-271
- skip indexes, 255-257
- visualization, 272
- workloads, 272

cloud computing, 41

- cold starts, 380
- Google Cloud, 181, 232

Cluster Autoscaler (CAS), 378

clusters

- multiregion, 269-270
- single-region, 268

code

- deletion test, 574
- disposable, 22
- durable, 21-22
- instrumentation, 29-30
- maintenance horizon, 22-23
- production quality, 23-24
- testing, 28

column-based storage, 221

ColumnIO, 221, 223

columns

- materializing, 257-258
- scanning, 244

compaction, 222, 241, 459

complex joins, ClickHouse, 252-254

compliance

- instrumentation and, 566-569
- SLO, 196
- telemetry pipelines, 297-309

compute platforms, 373-375

confidence loop, 442

constant-probability sampling, 277

Container Storage Interface (CSI), 379

context propagation, 119

- interprocess propagation, 115
- intraprocess propagation, 115

context, AI agents and, 169-171

context-aware burn alerts, 206-208

contract tests, 28

control plane, 299

- real-time, 356
- telemetry pipelines, 300

controllability, 4

Conway's law, 118

core analysis loop, debugging from, 139-140

- brute-force, 141-144

cost centers, 475, 480

cost optimization

- applications and, 382-388
- cloud providers, 380-381
- fleet-wide, 375-376
- infrastructure purchasing models, 373-375
- Kubernetes, 376-380
- signal types, 385-388

costs, 46-47

- free software, 521-522
- software cost model, 572-573

CPU profiling tools

- cost optimization and, 383-385

critical path, 23-24

critical user journeys, 57-58

CSI (Container Storage Interface), 379

CTOs (chief technology officer) open letter, 420-431

custom header attributes, 95



## D

DAG (directed acyclic graph), 79, 331-332, 384

dashboards

static, 14

visibility and, 299

data

high-cardinality, 44

high-dimensionality, 44

data collection

ClickHouse, 248

Kafka, 249

S3Queue, 249

data lifecycle management, ClickHouse, 261-262

data modeling

column materialization, 257-258

merge tables, 259-261

projections, 258

skip indexes, 255-257

views, materialized, 258-259

data normalization, telemetry pipelines, 296-297

data patterns, AI agents and, 166

data refinement, sampling and, 275-276

data resilience, telemetry pipelines, 295, 298

data sampling, 60-62

data storage, 12

(see also unified storage model)

Bigtable, 221

column-based, 221

ColumnIO, 221, 223

Dremel, 221, 223

mobile and web apps, 354-357

partitioning by timestamp, 224-226

raw events, 44

row-based, 221

Bigtable, 222-223

segmentation, 226

sharding, 223

tiering, 232

databases (see TSDB (time-series database))

datastores, 237

(see also data storage)

building, 237

datatypes

for telemetry, 53

three pillars model, 64

DAU (daily active users), 345

debugging

core analysis loop, 139-140

brute-force, 141-144

first principles, 138-144

hypothesis-driven, 139-140

known conditions, 136-138

microservices and, 152-153

mobile and web apps, 344

runbooks, 136

samples and, 276

tracing, 73-74

visualizations and, 140

deduplication, 297

defined capacity purchasing models, 373

dependencies, 22

deploys

agent + gateway, 302

agent-only, 302

blue-green, 32

canary, 33

feature flags and, 30-31

Kubernetes and, 377

rolling, 33

telemetry pipelines, 301-302

developer feedback loops, working investment, 482

developer intent, 16

validating, 19-36

developer learning loops, 480

agentic workflow, 462

business case, 469-474

external measures, 471-473

internal measures, 473-474

developers

future of, 571

testing loop, 423-424

development cycle

AI application data, 398-400

observability in, 150

dimensionality, 15

directed acyclic graph (DAG), 79, 331-332, 384

disposable code, 22

distributed tracing (see tracing)

DLC (Docker layer caching), 338

Docker Bake, 338

Docker layer caching (DLC), 338

Dremel, 221, 223

durability, 234-237

- durable code, 21-22
- dynamic rate sampling, 288-290
- dynamic sampling, 279
- E**
- eBPF (extended Berkeley Packet Filters), 123-124, 384
- elastic capacity purchasing models, 373
- Elasticsearch, 220, 237
- ELK (Elasticsearch, Logstash, and Kibana) stack, 521, 529
- end-to-end tests, 28
- enrichment, telemetry pipelines, 297
- error budget
  - alerts, 195-197
  - over expenditures, 198
  - predictive burn alerts, 201
    - baseline window, 209-210
    - context-aware, 206-208
    - lookahead window, 202-208
    - short-term, 204-206
  - relative burn alerts, 200-201
  - threshold-crossing alerts, 199-200
- error information attributes, 100
- errors
  - AI agents and, 166
  - attribute capture, 99-101
- events
  - as database record, 64
  - JSON objects, 64
  - sampling, 276
  - sampling and, 278-279
  - structured, 64
- extended Berkeley Packet Filters (eBPF) (see eBPF (extended Berkeley Packet Filters))

- F**
- failure modes, 97, 164
  - AI agents, 401
  - distributed systems, 176
  - known-unknowns, 176
- fault detection, 7
- feature flag attributes, 91
- feature flags
  - attribute capture, 91
  - deploys, 30-31
  - releases, 31

- testing and, 32
- feedback loops, 14, 35
  - amplifying, 439
    - confidence loop, 442
    - learning accumulation loop, 442-443
    - system quality loop, 444
- balancing, 439-440
  - cost, 444-446
  - incident involvement, 444
  - usage, 444-446
- building, 428-429
- developer learning loops, 480
  - business case, 469-474
  - external measures, 471-473
  - internal measures, 473-474
  - working investment, 482
- developers and production, 26-27
- negative, 439-440
- operational, 453-455
- operational loops, 480
  - business case, 467-469
  - firefighting trap, 480-481
  - operational instability and, 468-469
  - three pillars model, 467
  - unified model, 468
  - working investment, 482
- partial, 446-448
- prioritizing, 465-467
- software delivery systems, 441-448
- software development, 452-453
- systems thinking and, 438-441
- value creation and, 456-459
- filesystem cache, 248
- filters
  - attributes, 110
  - Bloom filters, 256
  - eBPF (extended Berkeley Packet Filters), 123-124
  - telemetry pipelines, 297
- Fin (AI agent), 404
  - empathy, 413-415
  - resolution rate, 404-405
  - speed, 405-406
  - speed increase, 409-411
  - time to first token, 406-408
- firefighting trap, operational loops, 480-481
- first principles, debugging from, 138-144
- fixed-rate sampling, 281

fleet-wide cost optimization, 375-376  
fleet-wide profiling, 376

## G

GDPR (General Data Protection Regulation), 539  
General Data Protection Regulation (GDPR), 539  
generative AI  
    analysis automation, 144-147  
    SLOs and, 184-191  
golden data, 393  
Google Cloud, 232  
governance  
    self-governance, 449  
    telemetry pipelines, 297  
Grafana Tempo, 220  
greenfield app, telemetry pipeline adoption, 304  
guardrails, 17

## H

hallucinations in AI, 146  
head sampling, 126  
heroics, 33, 199, 502-504  
    honoring heroes, 495  
    legacy observability, 489  
    required, 499  
Hidalgo, Alex, 179, 188, 199, 210  
high-cardinality data, 44  
Honeycomb, 39-48  
    BubbleUp, 97, 141-142  
    build times case study, 336-340  
    debugging, 137  
    MCP (Model Context Protocol), 397  
    Query Assistant, 399  
horizontal scaling  
    multiregion cluster, 269-270  
    replication, 263-267  
    sharding, 267-268  
    SharedMergeTree, 270-271  
    single-region cluster, 268-269  
HTTP information attributes, 93  
    route parameters, 96-97  
HyperDX, 250, 272  
hypothesis-driven debugging, 139-140

## I

incident response  
    AI agents, 166  
    operational information, 105-110  
    questions for, 58  
incrementing  
    CI (continuous integration), 335-336  
    materialized views, 258  
index scanning, 244  
InfluxDB, 220  
infrastructure attributes, 87-88  
instrumentation  
    agentic AI and, 157  
    AI agents, 128-131  
    automatic, 55-56, 118  
    CI/CD process, 330-331  
    cost, 60  
    custom, 55-56  
        critical user journeys, 57-58  
    manual, 347  
    mobile and web apps, 346-348, 350  
        architecture, 351-352  
    observability teams, 557  
        frameworks, 562-563  
        identifiers, 561  
        secure environments, 565-569  
        security teams, 561  
        semconv (semantic conventions), 559  
        telemetry schemas, 559-562  
        tools, 562-563  
OTel, 54  
overview, 52  
ownership, 56-57  
pipelines, 59  
POC (proof of concept) and, 551  
quality, 153-155, 166-167  
scaling and, 59  
serverless architecture, 122  
volume, 60  
integration test, 28  
interprocess propagation, 115  
intraprocess propagation, 115  
investment, diagnosing, 479-484  
investments versus cost centers, 475

## J

Jaeger, 220

JFR (Java Flight Recorder), 384

joins

- complex, ClickHouse, 252-254
- Retriever, 231

JSON

- ClickHouse, 246
- object events, 64
- structured events, 81
- structured logs, 70

## K

Kafka, 249

keys

- sampling and, 278-279
- sampling by, 287-290

known-unknowns, 45

- threshold-based alerting, 175-178

Kubernetes

- CAS (Cluster Autoscaler), 378
- cost optimization, 376-380
- deploying with, 377
- logging data, 378
- resources consumed, 379

## L

large language models (LLMs) (see LLMs (large language models))

layered telemetry, 125-127

learning, 481

learning accumulation loop, 442-443

legacy observability, 12

testing

- arbitrary question test, 490
- deployment confidence test, 490-491
- mystery test, 489-490
- ownership, 488-489
- two or three people, 489

linear extrapolation, burn alerts, 203

LLMs (large language models), 128, 391

- AI agents and, 162
- auto-instrumentation and, 29
- evaluations, 393-395
- golden data, 393
- observability importance, 392-393
- prompts, 392
- question evals, 393
- SLI crafting, 186-188

task evals, 393

telemetry design, 395-396

localization attributes, 105

log files, 68

log-structured merge-tree (LSM tree), 241

logs, 10, 53, 68

as structured data, 69

converting to distributed traces, 76-81

parsers, 70

structured, 70-71

timestamps, 76

tracking operations, 71-72

LSM tree (log-structured merge-tree), 241

## M

maintenance horizon, 22-23

Majors, Charity, 39-48

manual instrumentation, 347

MCP (Model Context Protocol), Honeycomb, 397

mean time to detect (MTTD), 469

mean time to repair (MTTR), 253

measurement, 66

mental model of production, 168-170

MergeTree, 240

inserts, 241-243

ReplicatedMergeTree, 263

SharedMergeTree, 270-271

table engines, 241

tables, 240

user-defined keys, 241

metric attributes, 107-108

metrics, 10, 53

aggregation, 66-68

TSDBs, 218

attributes, 66

definition, 65

exemplars, 120

measurements, 66

names, 65

overview, 65-66

span metrics, 126

timestamps, 65

type, 66

microservices, debugging and, 152-153

mobile and web apps

aggregation, 344

debugging and, 344

- domains and, 348-353
- food delivery app, 363-366
- instrumentation, 346-348, 350
  - architecture, 351-352
- performance tests, 345
- production analytics, 345
- real-time control, 354-357
- sandboxes, 352-353
- storage, 354-357
- user-focused observability, 359-363
  - iterative analysis, 363-366
- UX (user experience) quantification, 361-362

Model Context Protocol (MCP), Honeycomb, 397

modern observability

- testing
  - arbitrary question test, 490
  - deployment confidence test, 490-491
  - mystery test, 489-490
  - ownership, 488-489
  - two or three people, 489

Monitorama, 41

monolithic systems, 7

Morrell, Jeremy, 64

MTTD (mean time to detect), 469

MTTR (mean time to repair), 253

multi-tier topology, 302

## N

- negative loops, 439-440
- nondeterminism, 42
- NoSQL databases, 220

## O

OBI (OpenTelemetry eBPF Instrumentation), 124

observability

- applications, 4-7
- as a platform, 430
- as investment, 430
- as property of software system, 5
- control theory definition, 44-45
- defining, 41-42
- future of, 571-579
- investment (see investment)
- legacy, 12

- overview, 3
- shifting left, 155
- software dependability and, 6-7
- terminology, 4
- traditional, 12

Observe Inc., 220

OCB (OpenTelemetry Collector Builder), 563

ontologies

- AI proposal, 323
- AI sandwich architecture, 321
- CD (continuous deployment)
  - signal parity, 319-321
- CI (continuous integration)
  - deterministic layer, 318
  - semantic layer, 319
  - simulation layer, 319
  - team workflow, 319
- CI/CD process, 329-330
- designing, 313-318
- deterministic comparison, 323
- deterministic core, 316-317
- entities, 314
  - defining, 314
- invariants, 314
  - defining, 314-315
- metadata, 316-317
- observability role, 312-313
- schema, 316-317
- semantic conventions, 314
- semantic grammar, 315

open source code, 517

- framework, 518
  - acquire quadrant, 520
  - adopt quadrant, 519
  - avoid quadrant, 519
  - commission quadrant, 520
  - create quadrant, 519

OpenTelemetry (OTel) (see OTel (OpenTelemetry))

OpenTelemetry Collector Builder (OCB), 563

OpenTelemetry eBPF Instrumentation (OBI), 124

operational feedback loops, 453-455

operational information, 105-110

operational loops, 480

- business case, 467-469
- choosing best one, 462
- firefighting trap, 480-481

- three pillars model, 460, 467
- unified model, 468
- working investment, 482
- optimization
  - tables, 255-261
- orchestration attributes, 88
- organizational shift
  - coalition formation, 494-496
  - expansion, 501
  - mandate, 496-498
  - observability team, 502-505
  - platform engineering, 500
  - resistance, 491-494
  - strategies, 499-502
  - testing legacy versus modern observability, 488-491
  - wins, 500
- OTel (OpenTelemetry), 51, 54-55
  - AI agents and, 127-131
  - collection step, 296
  - Collector, 248, 301
    - as DaemonSet, 377
  - design, 125-127
  - instrumentation, 54
  - layered telemetry, 116
  - metric exemplars, 120
  - modeling for LLMs, 395
  - semantics, 125-127
  - trace-first telemetry, 115-124
  - Weaver, 559

## P

- P99 latency metric time series, 81-83
- parallel builds, 338
- parallelism, 233-234
- parent-child spans, 74
- parsers, 70
- parsing
  - normalization, 296
  - payload parsing, 97
- partial feedback, 446-448
- partitioning
  - by timestamp, 224-226
  - segment partitioning, 225
- partitions
  - Kafka partitions, 234
  - pruning, 244
- performance engineering, 369-372

- building a practice, 372-373
  - infrastructure purchasing models, 373-375
- persistent data, 24
- pipelines (see telemetry pipelines)
  - instrumentation and, 59
- platform as a service (PaaS), orchestration
  - attributes, 88
- POC (proof of concept), 550-556
- positive loops, 439
- pre-aggregation, 14
- predictive burn alerts, 201
  - baseline window, 209-210
  - lookahead window, 202-208
    - context aware, 206-208
  - short-term alerts, 204-206
- product analytics, 345
- production complexity, 42-43
- production quality code, 23-24
- profiling
  - DAG (directed acyclic graph), 384
  - eBPF (extended Berkeley Packet Filters), 384
  - Go binaries, 383
  - JFR (Java Flight Recorder), 384
  - scaling, 384
- progressive delivery, 32-34
  - blue-green deployments, 32
  - canary deployments, 33
  - rolling deployments, 33
- prompts (LLMs), 392
- proportional extrapolation, 205

## Q

- queries
  - AI agents, 164-165
  - ClickHouse
    - complex joins, 252-254
    - log event filtering, 243-244
    - Map data type, 245-246
    - profiling, 254
    - simple queries, 250-251
    - structured logs, 245
  - Retriever, 228-230
    - ad hoc values, 230
    - real-time queries, 232
    - traces, 230
- Query Assistant (Honeycomb), 399
- query cache, 247

query condition cache, 247  
question evals, LLMs, 393

## R

RAG (retrieval augmented generation), 185  
Rasmussen safety model, 413-414  
rate limits, 103-105  
rate, error, duration (RED) metrics, 118  
real user monitoring (RUM), 344  
reciprocity principle, vendors, 544  
RED (rate, error, duration) metrics, 118  
reinforcing loops, 439  
relative burn alerts, 200-201  
replication, 263-267  
    Keeper, 264  
request and execution flow, 92-101  
reserved capacity purchasing models, 373  
retrieval augmented generation (RAG), 185  
Retriever, 215  
    cardinality, 234  
    durability, 234-237  
    implementation case study, 224-237  
    joins, 231  
    partitioning by timestamp, 224-226  
    queries  
        ad hoc values, 230  
        real-time, 232  
        traces, 230  
    query workloads, 228-230  
    scaling, 234-237  
    segmentation, 226  
rolling deployments, 33  
root spans, 74  
route information attributes, 96-97  
row-based storage, 221  
RUM (real user monitoring), 344  
runbooks, 136-137

## S

S3 (Simple Storage Service), 41, 232-234, 236, 249, 338  
S3Queue, 249  
sampling, 60-62  
    by key, 287-290  
    coding for, 280-291  
    consistency, 283-284  
    constant-probability, 277

data refinement, 275-276  
debugging and, 276  
dynamic, 279  
dynamic rates, 288-290  
event content (keys), 278-279  
events, 276  
fixed-rate, 281  
head sampling, 126  
rate  
    recording, 281-282  
    static, multiple, 286-287  
target-rate, 284-286, 287-288  
telemetry pipelines, 298  
trace spans, 279-280  
traffic volume, 278  
scaling, 234-237, 376-378  
    horizontal  
        multiregion cluster, 269-270  
        replication, 263-267  
        sharding, 267-268  
        SharedMergeTree, 270-271  
        single-region cluster, 268-269  
    instrumentation and, 59  
    telemetry pipelines, 302  
    vertical, 254  
        data lifecycle management, 261-262  
        data modeling, 255-261  
        profiling queries, 254  
schemas, 126  
    telemetry schemas, 559-562  
Scuba, 43, 220  
ScyllaDB, 220  
SDLC (software development lifecycle), 16  
    SLOs and, 184  
security  
    instrumentation and, 565-569  
    telemetry pipelines, 296-297  
segment partitioning, 225  
semantic conventions (semconv), 313, 314 (see semconv)  
    metadata, 316-317  
semantic failures, 311  
semconv (semantic conventions), instrumenta-  
    tion and, 559  
serverless architecture  
    cost optimization and, 380-381  
    traces, 122-123  
service attributes, 86-87

- service level indicators (SLIs), 179-180
- service level objectives (SLOs), 9
- sharding, 223, 267-268
- SharedMergeTree, 270-271
- shifting left, 155
- short-term burn alerts, 204-206
- signals
  - CD (continuous deployment), 319-321
  - cost optimization and, 385-388
- simultaneous multithreading (SMT), 375
- skip indexes, 255-257
- SLAs (service level agreements), 546
- SLIs (service level indicators), 179-180
  - CI/CD process, 331
  - LLMs and, 186-188
- SLOs (service level objectives), 9
  - AI agents and, 168
  - alerting and, 179-181
    - case study, 181-184
  - compliance, 196
  - error budget alerts, 195-197
  - fixed windows, 197-198
  - generative AI and, 184-191
  - SDLC (software development lifecycle) and, 184
    - sliding windows, 197-198
  - SMT (simultaneous multithreading), 375
- SNMP (Simple Network Management Protocol), 65
- Snowflake, 220
- sociotechnical debt, 421-426
- sociotechnical systems, 435-438
  - leverage points, 448-449
  - Trist system, 436
- software
  - buying versus vendor engineering, 536
  - commercial
    - benefits, 527
    - financial costs, 524-525
    - nonfinancial costs, 526-527
    - risks, 527-528
  - delivery, 20
    - ecosystem, 25
  - delivery speed, 156-157
  - purpose, 20
- software delivery systems
  - feedback loops, 441-448
- software dependability, 5-6
  - observability as property, 6-7
- software development feedback loops, 452-453
- software development lifecycle (SDLC), 16
- Software Observability, 40
- span metrics, 126
- spans, 74
  - creation, 117
  - traces, 79
- stakeholders
  - communications to higher ups, 540-541
  - engineering, 538
  - external
    - reciprocity principle, 544
    - relationship building, 542-543
    - SLAs and, 546
    - vendor trust coefficient, 543-544
  - finance, 539
  - internal
    - credibility, 541
    - trust, 541-542
  - IT/Ops, 539
  - management, 539
  - procurement, 540
  - security, 539
- stateless Lambda tracing, 123
- storage (see data storage)
- strace for microservices, 153
- strangler fig pattern, 35
- streaming architecture
  - traces and, 119-121
- structured event data versus time-series data, 212-214
- structured events, 64
  - logs, 68-72
  - metrics, 65-68
  - structured log comparison, 71
  - traces, 72-81
  - wide, 85
- structured logs, 70-71
  - ClickHouse, 245
    - Map datatype, 245-246
  - structured events comparison, 71
- system quality loop, 444
- system views, 81-83
- systems thinking
  - feedback loops, 438-441
  - information flows, 437-438
  - sociotechnical systems, 435-438



virtuous cycles, 441

## T

tables

- merge tables, 259-261

- MergeTree, 240

- optimization, 255-261

- views, materialized, 258-259

target-rate sampling, 284-286, 287-288

task evals, LLMs, 393

task functions, evaluations, 393

TDD (test-driven development), 149-150

telemetry

- aggregation

  - pre-aggregation, 14

- AI applications, 396-398

- as infrastructure versus product, 424-425

- as trainable skill, 559

- business context, 101-105

- datatypes for, 53

- layered, 116, 125-127

- LLMs

  - designing for, 395-396

- logs, 10, 53

- metrics, 10, 53

- sampling and, 60-62

- schemas, 559-562

- signals, 9

- three pillars model, 9-12

- traces, 10, 53

- unified storage model, 12-15

- user context, 101-105

telemetry pipelines

- adoption phases, 304

  - agent redirection, 305-306

  - greenfield app, 304

- agents, 300

- aggregation, 298

- build versus buy, 303

- business case, 308-309

- collection step, 296

- control, 295

- control plane, 300

- core functions, 295, 300

- cost, 295

- data resilience and, 295, 298

- deduplication, 297

- deployment patterns, 301-302

- enrichment and, 297

- filtering, 297

- gateways, 300

- governance, 297

- lookups, 297

- normalization and security step, 296-297

- pipeline control, 299-300

- reduce step, 297-298

- route step, 298

- sampling, 298

- scaling, 302

- stages, 295

- standardization, 295

- use cases, 307-308

test-driven development (TDD) (see TDD (test-driven development))

testing

- code, 28

- contract tests, 28

- end-to-end tests, 28

- feature flags and, 32

- integration tests, 28

- legacy observability

  - arbitrary question test, 490

  - deployment confidence test, 490-491

  - mystery test, 489-490

  - ownership, 488-489

  - two or three people, 489

- modern observability

  - arbitrary question test, 490

  - deployment confidence test, 490-491

  - mystery test, 489-490

  - ownership, 488-489

  - two or three people, 489

- performance tests

  - mobile and web apps, 345

- unit testing, 28

three pillars model, 460-460

- operational loops, 467

three pillars telemetry model, 9-12

threshold-based alerts, 136

- alert fatigue, 174-175

- known-unknowns, 175-178

- threshold crossing, 199-200

tiered storage, 232

time series generation, 81-83

time to first token, 406-408

- time-series data versus structured event data, 212-214
- time-series database (TSDB) (see TSDB (time-series database))
- timestamps, 65
  - logs, 76
  - partitioning by, 224-226
- timing attributes, 97-98
- tools, 25
  - change actions, 430
- topology
  - multi-tier, 302
  - service topology, 169
- trace contexts, 79-81
- trace-first correlation, 125-127
- trace-first telemetry, 113
  - interprocess propagation, 115
  - intraprocess propagation, 115
- traces, 10, 53
  - AI agents, 166
  - asynchronous architecture, 121-122
  - converting to from logs, 76-81
  - eBPF (extended Berkeley Packet Filters), 123-124
  - HTTP information, 93-95
    - route parameters, 96-97
  - queries, Retriever, 230
  - sampling and, 279-280
  - serverless architecture, 122-123
  - spans, 79
  - stateless Lambda tracing, 123
  - trace spans, 74
  - waterfall visualization, 74
- tracing
  - ad hoc relationships, 252
  - async requests, 98-99
  - debugging and, 73-74
  - interdependencies and, 73
  - libraries, 72
  - streaming architecture, 119-121
- traditional observability, 12
- traffic splitters, 35
- traffic volume sampling, 278
- Trist principle, 436
- TSDB (time-series database), 10, 216, 217-218
  - aggregated metrics, 218

## U

- unified storage model, 460-461
  - operational loops, 468
  - telemetry, 12-15
- unit testing, 28
- units, burn alerts, 204
- unknown-unknowns, 45
- uptime attributes, 106-107
- user and customer attributes, 101-103
- user experience, 178-179
- user-agent attributes, 94-95
  - custom, 95
- userspace page cache, 248
- UX (user experience), 346
  - mobile and web apps, 361-362

## V

- value, measuring, 428
- variability, 150
- vendor engineering, 534-536
  - company roadmap influence, 544-546
  - POC (proof of concept), 550-556
  - preparing for, 537-538
  - reciprocity principle, 544
  - relationship building, 542-543
  - skill building, 536-537
  - SLAs, 546
- stakeholders
  - communications to higher ups, 540-541
  - credibility, 541
  - engineering, 538
  - external, 542-546
  - finance, 539
  - IT/Ops, 539
  - management, 539
  - procurement, 540
  - security, 539
  - trust, 541-542
- transformation
  - executive sponsor, 548
  - OTel and, 550
  - scope, defining, 548
  - success criteria, 549
  - threat perceptions, 549-550
- transformation scenarios, 547-550
- vendor trust coefficient, 543-544
- versus buying software, 536

- versioning attributes, 92
- vertical scaling, 254
  - data modeling
    - column materialization, 257-258
    - lifecycle management, 261-262
    - merge tables, 259-261
    - projections, 258
    - skip indexes, 255-257
    - views, materialized, 258-259
  - profiling queries, 254
- virtuous cycles, 441
- visualization
  - ClickHouse, 272
  - debugging and, 140
  - HyperDX, 240, 250, 272
  - Kubernetes cluster data, 377

## W

- waterfall visualizations

- traces, 74
- Weaver, 559
- wide structured events, 85
- windows of time
  - SLOs, 197-198
- workloads, 272

## Y

- Yen, Christine, 39

## Z

- zero-level alerts, 198-199
  - predictive burn, 201
    - baseline window, 209-210
    - lookahead window, 202-208
  - relative burn, 200-201
  - thresholds and, 199-200

## About the Authors

---

**Charity Majors** is the cofounder and CTO of honeycomb.io. She pioneered the concept of modern observability, drawing on her years of experience building and managing massive distributed systems at Parse (acquired by Facebook), Facebook, and Linden Lab building Second Life. She is the coauthor of *Observability Engineering* (O'Reilly, 2022) and *Database Reliability Engineering* (O'Reilly, 2017). She loves free speech, free software, and single malt scotch.

**Liz Fong-Jones** is a developer advocate, labor and ethics organizer, and site reliability engineer (SRE) with over two decades of experience. She is currently a technical fellow at honeycomb.io, and previously was an SRE working on products ranging from the Google Cloud Load Balancer to Google Flights. She lives in Vancouver, BC, with her wife Elly and partners, and in Sydney, NSW.

**George Miranda** is the VP of marketing at InsightFinder AI. His devtools career includes Honeycomb, PagerDuty, Chef Software, and others. Previously, he spent 15+ years building large-scale distributed systems in the finance and video game industries.

**Austin Parker** is director of AI strategy at Honeycomb, where he works on AI-driven observability products and explores how AI is reshaping the way engineers understand production systems. A cofounder of the OpenTelemetry project and member of its Governance Committee, Austin has spent over two decades building cloud native platforms and developer tooling. He is the author of *Learning OpenTelemetry* (O'Reilly, 2024), coauthor of *Distributed Tracing in Practice* (O'Reilly, 2020), and cochair of Observability Day at KubeCon. When he's not writing or speaking, he's probably cooking.

## Colophon

---

The animal on the cover of *Observability Engineering* is a maned wolf (*Chrysocyon brachyurus*). Maned wolves are the largest canids in South America and can be found in Argentina, Brazil, Paraguay, Bolivia, and parts of Peru. Their habitat includes the Cerrado biome, which is made up of wet and dry forests, grasslands, savannas, marshes, and wetlands. Despite the name, maned wolves are not actually wolves but a separate species entirely.

Maned wolves have narrow bodies, large ears, and long black legs that allow them to see above tall grasses as they run. They stand nearly 3 feet tall at the shoulder and weigh around 50 pounds. Much of their bodies are covered in a mix of black and long, golden-red hairs that stand straight up when danger is near. Unlike gray wolves and most other canid species that form packs, the maned wolf is solitary and often hunts alone. They are omnivorous and crepuscular, venturing out during dusk and

dawn to prey on small mammals, rabbits, birds, and insects, and to scavenge for fruit and vegetables like lobeira, a small berry whose name means “fruit of the wolf.”

Maned wolves live in monogamous pairs sharing a territory of 10 miles during the breeding season, which lasts from April to June in the wild. In captivity, maned wolves give birth to litters of one to five pups. Both parents have been known to groom, feed, and defend their young in captivity, but they are rarely seen with their pups in the wild. Pups develop quickly; they are usually considered fully grown and ready to leave their parents’ territory after one year. The maned wolf is classified as “near threatened” by the IUCN, mostly due to loss of habitat. Many of the animals on O’Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on an antique line engraving from *Brockhaus Lexicon*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag’s Ubuntu Mono.